

Tina4 for Ruby Developers

One Framework, Native Conventions, Shared Contracts

v3.13.138 • tina4.com

The Intelligent Native Application 4ramework

Table of Contents

Getting Started with Tina4 Ruby 70

1. What Is Tina4 Ruby . . . 70

2. Prerequisites and Installation . . . 70

What You Need . . . 70

Installing the Tina4 CLI . . . 71

Creating a New Project . . . 71

Starting the Dev Server . . . 72

3. Project Structure Walkthrough . . . 72

4. Your First Route . . . 73

Test It . . . 74

Understanding What Happened . . . 74

Adding More HTTP Methods . . . 74

5. Your First Template . . . 75

Create a Base Layout . . . 75

Create a Product Listing Page . . . 76

Create the Route That Renders the Template . . . 77

See It in the Browser . . . 77

How Template Rendering Works . . . 77

About tina4css . . . 78

6. Understanding .env . . . 78

7. The Dev Dashboard . . . 79

8. Manual Setup (No CLI) . . . 80

Step 1: Create Gemfile . . . 80

Step 2: Create app.rb . . . 80

Step 3: Create the Folder Structure . . . 81
Step 4: Create .env . . . 81
Step 5: Run It . . . 81
9. Request & Response Fundamentals . . . 81
Reading Query Parameters . . . 81
Reading URL Path Parameters . . . 81
Reading the Request Body . . . 82
Reading Headers . . . 82
Sending JSON Responses . . . 82
Sending HTML / Template Responses . . . 82
Status Codes . . . 82
Worked Example: A Complete Route File . . . 83
10. Exercise: Greeting API + Product List Template . . . 84
Exercise Part A: Greeting API . . . 84
Exercise Part B: Product List Page . . . 85
11. Solutions . . . 86
Solution A: Greeting API . . . 86
Solution B: Product List Page . . . 86
12. Gotchas . . . 88
1. File not auto-discovered . . . 88
2. "Uninitialized constant" errors . . . 88
3. JSON response shows HTML . . . 88
4. Template not found . . . 88
5. Port already in use . . . 88
6. Changes not reflected . . . 89
7. .env not loaded . . . 89
8. Debug mode in production . . . 89

Routing 90

1. How Routing Works in Tina4 . . . 90

2. HTTP Methods . . . 90

HEAD and OPTIONS: automatic, no boilerplate . . . 91

Explicit Tina4::Router.head and Tina4::Router.options registration . . . 92

3. Path Parameters . . . 92

Typed Parameters . . . 92

Typed Parameters in Action . . . 93

4. Query Parameters . . . 94

5. Route Groups . . . 94

6. Middleware . . . 95

Middleware on a Single Route . . . 96

Blocking Middleware . . . 96

Middleware on a Group . . . 96

Multiple Middleware . . . 97

7. Route Decorators: @noauth and @secured . . . 97

@noauth -- Public Routes . . . 97

@secured -- Protected GET Routes . . . 97

8. Route Chaining: .secure and .cache . . . 98

.secure . . . 98

.cache . . . 98

Chaining Both . . . 98

9. Wildcard and Catch-All Routes . . . 98

Wildcard Routes . . . 98

Catch-All Route (Custom 404) . . . 99

10. Route Listing via CLI . . . 99

11. Organizing Route Files . . . 100

Pattern 1: One File Per Resource . . . 100
Pattern 2: Subdirectories by Feature . . . 100
12. Exercise: Build a Full CRUD API for Products . . . 100
Requirements . . . 101
13. Solution . . . 102
14. Gotchas . . . 104
1. Trailing Slashes Matter . . . 104
2. Parameter Names Must Be Unique in a Path . . . 104
3. Method Conflicts . . . 104
4. Route Handler Must Return a Response . . . 104
5. Block Syntax Matters . . . 105
6. Middleware Must Be a Named Function or String . . . 105
7. Group Prefix Must Start with a Slash . . . 105
Request & Response **106**
1. The Two Objects You Use Everywhere . . . 106
2. The Request Object . . . 106
method . . . 106
path . . . 106
params . . . 106
query . . . 107
body . . . 107
headers . . . 107
ip . . . 107
cookies . . . 107
files . . . 107
Inspecting the Full Request . . . 107
3. The Response Object . . . 108

json -- JSON Response . . . 108
html -- Raw HTML Response . . . 108
text -- Plain Text Response . . . 109
render -- Template Response . . . 109
redirect -- Redirect Response . . . 109
file -- File Download Response . . . 109
xml -- XML Response . . . 109
error -- Error Envelope . . . 109
4. Status Codes . . . 110
5. Custom Headers . . . 111
CORS Headers . . . 111
6. Cookies . . . 111
7. File Uploads . . . 112
Handling a Single File Upload . . . 113
Saving the Uploaded File . . . 113
Handling Multiple Files . . . 114
8. File Downloads . . . 114
9. Content Negotiation . . . 115
10. Input Validation . . . 116
The Validator Class . . . 116
The Error Response Envelope . . . 116
Upload Size Limits . . . 116
11. Exercise: Build an Image Upload API . . . 117
Requirements . . . 117
Test with: . . . 117
12. Solution . . . 117
13. Gotchas . . . 119

1. Forgetting the response method . . . 119
2. request.body is nil for GET requests . . . 120
3. Body is nil for JSON requests . . . 120
4. Content-Type Mismatch . . . 120
5. File Uploads Return Empty . . . 120
6. Redirect Loops . . . 120
7. Cookie Not Set . . . 120
8. Large Request Body Rejected . . . 121
9. Chaining Response Methods in Wrong Order . . . 121

Templates **122**

1. Beyond JSON -- Rendering HTML . . . 122
2. Variables and Expressions . . . 122
Accessing Nested Data . . . 122
Method Calls on Values . . . 123
Expressions . . . 123
3. Template Inheritance . . . 123
Base Layout . . . 123
Child Template . . . 124
Using {{ parent() }} . . . 124
4. Includes . . . 125
Passing Variables to Includes . . . 126
5. For Loops . . . 126
The loop Variable . . . 126
Empty Lists . . . 127
Looping Over Key-Value Pairs . . . 127
6. Conditionals . . . 127
if / elseif / else . . . 127

Ternary Operator . . . 127
Testing for Existence . . . 127
Truthiness . . . 127
{% set %} -- Local Variables . . . 128
7. Filters . . . 128
Complete Filter Reference . . . 129
String Filters . . . 129
Array Filters . . . 130
Encoding Filters . . . 131
Numeric Filters . . . 131
JSON Filters . . . 131
Dict Filters . . . 132
Other Filters . . . 132
Chaining Filters . . . 132
Dumping Values for Debugging . . . 132
8. Macros . . . 133
Defining a Macro . . . 133
Using Macros . . . 133
9. Special Tags . . . 134
{% raw %} -- Literal Output . . . 134
{% spaceless %} -- Remove Whitespace . . . 134
{% autoescape %} -- Control Escaping . . . 135
Comments . . . 135
10. tina4css Integration . . . 135
Layout Classes . . . 135
Common Components . . . 136
Utility Classes . . . 136

11. Exercise: Build a Product Catalog Page . . . 136
Requirements . . . 136
Data . . . 137
Test with: . . . 137
12. Solution . . . 137
13. Gotchas . . . 140
1. {% extends %} Must Be the First Tag . . . 140
2. Undefined Variables Show Nothing . . . 140
3. Auto-Escaping Prevents HTML Output . . . 140
4. Variable Scope in Includes . . . 140
5. Macro Arguments Are Positional . . . 140
6. Template File Extension Does Not Matter . . . 141
7. Filters Are Not Ruby Methods . . . 141
Database **142**
1. From Arrays to Real Data . . . 142
2. Connecting to a Database . . . 142
The Default: SQLite . . . 142
Connection Strings for Other Databases . . . 142
Firebird URL Forms . . . 142
Separate Credentials . . . 143
Programmatic Connection . . . 143
Connection Pooling . . . 143
Verifying the Connection . . . 143
3. Getting the Database Object . . . 144
4. Raw Queries . . . 144
fetch -- Get Multiple Rows . . . 144
DatabaseResult . . . 144

Properties . . . 145
Iteration . . . 145
Index Access . . . 145
Countable . . . 145
Conversion Methods . . . 145
Schema Metadata with column_info . . . 145
fetch_one -- Get a Single Row . . . 146
execute -- Run a Statement . . . 146
Full Example: A Simple Query Route . . . 146
5. Parameterised Queries . . . 146
A Safe Search Endpoint . . . 147
6. Transactions . . . 147
7. Schema Inspection . . . 148
tables . . . 148
columns . . . 148
table_exists? . . . 149
driver_name . . . 149
A Schema Info Endpoint . . . 149
8. Batch Operations with execute_many . . . 149
9. Helper Methods: insert, update, delete . . . 150
insert . . . 150
update . . . 150
delete . . . 150
10. Migrations . . . 150
File Naming . . . 150
Generating a Migration . . . 151
SQL Migrations . . . 151

Ruby Class Migrations . . . 151
Running Migrations . . . 152
Checking Migration Status . . . 152
Rolling Back . . . 152
A Real Migration Sequence . . . 152
Migration Tracking . . . 153
11. Query Caching . . . 153
12. Exercise: Build a Notes App . . . 153
Requirements . . . 153
Test with: . . . 154
13. Solution . . . 155
Migration . . . 155
Routes . . . 155
14. Seeder -- Generating Test Data . . . 158
FakeData . . . 158
Deterministic Output . . . 158
Seeding with ORM . . . 159
Overrides . . . 159
Seeding a Raw Table . . . 159
Batch Seeding . . . 159
Seed Files . . . 160
When to Use It . . . 160
15. Gotchas . . . 160
1. Forgetting the transaction block . . . 160
2. Connection String Formats . . . 160
3. SQLite File Paths . . . 161
4. Parameterised Queries with LIKE . . . 161

5. Boolean Values in SQLite . . . 161
6. Migration Already Applied . . . 161
7. fetch Returns Empty Array, Not Nil . . . 162
8. SQL Injection Through String Interpolation . . . 162

ORM **163**

1. From SQL to Objects . . . 163
ORM at a Glance: Four Languages, One Shape . . . 163
Defining a Model . . . 163
Common Query Operations . . . 164
2. Defining a Model . . . 165
Field Types . . . 166
Field Options . . . 167
Field Mapping . . . 167
find() vs where() -- naming convention . . . 168
auto_map and Case Conversion Utilities . . . 168
3. create_table -- Schema from Models . . . 168
4. CRUD Operations . . . 169
save -- Create or Update . . . 169
create -- Build and Save in One Step . . . 169
findbyid -- Fetch One Record by Primary Key . . . 169
find -- Query by Filter Hash . . . 170
where -- Query with SQL Conditions . . . 170
delete -- Remove a Record . . . 170
Listing Records . . . 170
ModelCollection -- The Page and the Total . . . 171
select_one -- Fetch a Single Record by SQL . . . 171
load -- Populate an Existing Instance . . . 172

count -- Count Records . . . 172

5. toh, todict, to_json, and Other Serialisation . . . 172

toh and todict . . . 172

to_json . . . 172

Other Serialisation Methods . . . 173

6. Relationships . . . 173

foreignkeyfield: Auto-Wired Relationships . . . 173

Declarative Relationships . . . 174

has_many . . . 174

has_one . . . 175

belongs_to . . . 175

Imperative Relationships . . . 175

7. Eager Loading . . . 175

Nested Eager Loading . . . 176

to_h with Nested Includes . . . 176

8. Soft Delete . . . 176

Deleting and Restoring . . . 177

Including Soft-Deleted Records . . . 177

Counting with Soft Delete . . . 177

Custom Soft Delete Field . . . 178

When to Use Soft Delete . . . 178

9. Auto-CRUD . . . 178

The auto_crud Flag . . . 178

Manual Registration . . . 178

What the Generated Routes Do . . . 179

Custom Routes Alongside Auto-CRUD . . . 180

10. Scopes . . . 180

Class Methods as Scopes . . . 180
Dynamic Scopes with scope() . . . 181
11. Input Validation . . . 181
12. Exercise: Build a Blog with Relationships . . . 182
Requirements . . . 182
13. Solution . . . 182
14. Gotchas . . . 185
1. Forgetting to call save . . . 185
2. findbyid returns nil . . . 185
3. find takes a hash, not keyword arguments . . . 185
4. to_h includes everything . . . 185
5. Validation runs on save, but check errors . . . 186
6. Foreign key not enforced . . . 186
7. N+1 query problem . . . 186
8. Auto-CRUD endpoint conflicts . . . 186
9. Soft-deleted records appearing in queries . . . 186
10. Table name pluralisation . . . 187
15. Raw SQL . . . 187
16. QueryBuilder Integration . . . 188
QueryBuilder **189**
1. Why a Query Builder? . . . 189
2. Creating a QueryBuilder . . . 189
3. Selecting Columns . . . 189
4. Filtering with where . . . 189
Multiple where Calls . . . 190
5. OR Conditions with or_where . . . 190
6. Joins . . . 191

Inner Join . . . 191
Left Join . . . 191
Multiple Joins . . . 191
7. Ordering . . . 191
8. Limiting and Offsetting . . . 192
9. Grouping and Having . . . 192
Group By . . . 192
Having . . . 192
10. Executing Queries . . . 193
get -- Multiple Rows . . . 193
first -- Single Row . . . 193
count -- Row Count . . . 193
exists? -- Boolean Check . . . 193
11. Inspecting the SQL with to_sql . . . 193
12. Using QueryBuilder in Routes . . . 194
A Filtered API Endpoint . . . 194
A Dashboard Summary Endpoint . . . 195
13. Exercise: Build a Product Search API . . . 195
Requirements . . . 195
Test with: . . . 196
14. Solution . . . 196
Search Endpoint . . . 196
Stats Endpoint . . . 197
15. NoSQL: MongoDB Queries . . . 198
Operator Mapping . . . 198
Example . . . 198
16. Gotchas . . . 199

1. Default Limit of 100 . . . 199
2. Parameter Order Matters . . . 199
3. No Database Connection . . . 199
4. count Replaces Your Columns . . . 199
5. to_sql Does Not Show Parameter Values . . . 199
6. or_where as First Condition . . . 200

Authentication 201

1. Locking the Door . . . 201
2. JWT Tokens . . . 201
Generating a Token . . . 201
Token Expiry . . . 201
Validating a Token . . . 202
Reading the Payload . . . 202
The Secret Key and Algorithm . . . 202
3. Password Hashing . . . 203
Hashing a Password . . . 203
Checking a Password . . . 203
Registration Example . . . 203
4. The Login Flow . . . 204
5. Using Tokens in Requests . . . 205
6. Protecting Routes . . . 205
Auth Middleware . . . 205
Applying Middleware to Routes . . . 206
Applying Middleware to a Group . . . 206
7. @noauth and @secured Decorators . . . 206
@noauth -- Skip Authentication . . . 206
@secured -- Require Authentication for GET Routes . . . 207

Role-Based Authorization . . . 207
8. CSRF Protection . . . 207
Generating a Token . . . 207
Validating the Token . . . 208
9. Sessions . . . 208
Using Sessions . . . 208
10. Exercise: Build Login, Register, and Profile . . . 209
Requirements . . . 209
Test with: . . . 210
11. Solution . . . 210
Migration . . . 210
Middleware . . . 210
Routes . . . 211
12. Gotchas . . . 214
1. Token Expiry Confusion . . . 214
2. Secret Key Management . . . 214
3. CORS with Authentication . . . 214
4. Storing Tokens in localStorage . . . 215
5. Forgetting @noauth on Login . . . 215
6. Password Hash Column Too Short . . . 215
7. Token in URL Query Parameters . . . 215
Sessions & Cookies **216**
1. State in a Stateless World . . . 216
2. How Sessions Work . . . 216
3. The Session API . . . 216
Full API Reference . . . 217

4. File Sessions (Default) . . . 217
5. Redis Sessions . . . 218
6. MongoDB Sessions . . . 218
7. Valkey Sessions . . . 218
8. Database Sessions . . . 218
9. Reading and Writing Session Data . . . 218
Storing Complex Data . . . 219
10. Flash Messages . . . 220
Setting a Flash Message . . . 220
Reading and Clearing Flash Messages . . . 220
Using Flash Messages in Templates . . . 220
11. Setting and Reading Cookies . . . 221
Setting a Cookie . . . 221
Reading a Cookie . . . 221
Deleting a Cookie . . . 221
When to Use Cookies vs Sessions . . . 221
12. Remember Me Functionality . . . 222
13. Session Security . . . 222
Configuration Options . . . 222
Session Regeneration . . . 223
Destroy a Session . . . 223
14. Exercise: Build a Shopping Cart with Session Storage . . . 223
Requirements . . . 224
Business Rules . . . 224
Test with: . . . 224
15. Solution . . . 225
16. Gotchas . . . 227

1. Sessions Do Not Work with curl Without Cookie Flags . . . 227
2. Session Data Disappears After Server Restart . . . 227
3. Session Cookie Not Sent Over HTTP in Production . . . 227
4. Flash Messages Show Twice . . . 227
5. Large Session Data Causes Slow Requests . . . 227
6. Remember Me Token Not Invalidated on Password Change . . . 227
7. Session Fixation . . . 228

Middleware & Security **229**

1. The Gatekeepers . . . 229
2. What Middleware Is . . . 229
3. Built-in CorsMiddleware . . . 230
4. Built-in RateLimiter . . . 230
Custom Limits Per Group . . . 231
Built-in RequestLoggerMiddleware . . . 231
Built-in SecurityHeadersMiddleware . . . 231
Combining All Four Built-In Middleware . . . 232
5. Writing Custom Middleware . . . 232
Request Logging Middleware . . . 232
Request Timing Middleware . . . 233
IP Whitelist Middleware . . . 233
Request Validation Middleware . . . 233
Writing Class-Based Middleware . . . 233
JWT Authentication Middleware (Class-Based) . . . 234
6. Applying Middleware to Individual Routes . . . 235
7. Route Groups with Shared Middleware . . . 235
8. Middleware Execution Order . . . 236

9. Short-Circuiting . . . 236
Maintenance Mode . . . 237
10. Modifying Requests in Middleware . . . 237
11. Real-World Middleware Stack . . . 237
12. Exercise: Build an API Key Middleware . . . 238
Setup . . . 238
Test with: . . . 239
13. Solution . . . 239
14. Gotchas . . . 240
1. Middleware Must Be a Named Method . . . 240
2. Forgetting to Return next_handler Result . . . 240
3. Middleware Order Matters . . . 240
4. CORS Preflight Returns 404 . . . 240
5. Rate Limiter Counts Preflight Requests . . . 241
6. Middleware File Not Auto-Loaded . . . 241
7. Short-Circuiting Skips Cleanup Middleware . . . 241
15. Security . . . 241
1. Secure-by-Default Routing . . . 242
Making a Write Route Public . . . 242
Protecting a GET Route . . . 242
The Rule . . . 242
2. CSRF Protection . . . 243
How It Works . . . 243
The Template . . . 243
The Middleware . . . 243
AJAX Requests . . . 243
Tokens in Query Strings: Blocked . . . 244

Skipping CSRF Validation . . . 244
Disabling CSRF Globally . . . 244
3. Session-Bound Tokens . . . 244
How Stolen Tokens Fail . . . 244
4. Security Headers . . . 245
HSTS: Enforcing HTTPS . . . 245
Customising Headers . . . 245
5. SameSite Cookies . . . 246
6. Login Flow: Complete Example . . . 246
The Login Route . . . 247
The Login Form . . . 248
Protected Pages: Checking the Session . . . 248
Logout: Destroying the Session . . . 248
7. Handling Expired Sessions . . . 249
The Pattern: Redirect to Login, Then Back . . . 249
Token Refresh . . . 249
8. Rate Limiting . . . 250
9. CORS and Credentials . . . 250
10. Security Checklist . . . 251
Gotchas . . . 251
1. "My POST route returns 401 but I didn't add auth" . . . 251
2. "CSRF validation fails on AJAX requests" . . . 251
3. "I disabled CSRF but forms still fail" . . . 252
4. "My Content-Security-Policy blocks inline scripts" . . . 252
5. "User stays logged in after session expires" . . . 252
Exercise: Secure Contact Form . . . 252
Solution . . . 253

Caching 255

1. From 800ms to 3ms . . . 255

2. Layer 1: Request-Scoped Query Cache . . . 255

3. Layer 2: Persistent Database Query Cache . . . 256

Sharing the cache across instances . . . 256

Bypassing the cache per query . . . 256

When to use the persistent DB cache . . . 257

When not to use it . . . 257

4. Layer 3: Response Cache Middleware . . . 257

Default TTL and limits . . . 258

Cache headers . . . 258

Caching with query parameters . . . 258

What not to cache . . . 258

5. Cache Backends . . . 258

Redis . . . 259

File . . . 259

Graceful fallback . . . 260

6. The Direct Cache API . . . 260

cache_set . . . 260

cache_get . . . 260

cache_delete . . . 260

clear_cache . . . 260

Real-world pattern: cache-aside . . . 260

7. Cache Invalidation Strategies . . . 261

Strategy 1: Time-based expiry (TTL) . . . 262

Strategy 2: Event-based invalidation . . . 262

Strategy 3: Write-through cache . . . 262

8. TTL Management . . . 263
Dynamic TTL . . . 263
9. Cache Statistics . . . 264
Response / KV cache stats . . . 264
Database cache stats . . . 264
10. Combining Cache Layers . . . 264
11. Exercise: Cache an Expensive Product Listing Endpoint . . . 266
Requirements . . . 266
Test with: . . . 266
12. Solution . . . 266
13. Gotchas . . . 268
1. Caching authenticated responses . . . 268
2. Memory cache lost on restart . . . 268
3. Stale data after a database update . . . 269
4. Cache key collisions . . . 269
5. Serialization overhead . . . 269
6. Forgetting that writes flush the request cache . . . 269
7. Reaching for a backend you do not have . . . 269
Queue System **271**
1. Do Not Make the User Wait . . . 271
2. Why Queues Matter . . . 271
3. File Queue (Default) . . . 272
Creating a Queue and Pushing a Job . . . 272
Convenience Method: produce . . . 272
Push with Priority . . . 272
Delaying a Job . . . 273
Queue Size . . . 273

4. Pushing from Route Handlers . . . 273
5. Consuming Jobs . . . 273
Automatic Retry . . . 274
Retry Backoff . . . 274
Manual Retry with Delay . . . 274
Manual Pop . . . 275
Pop a Specific Job by ID . . . 275
6. Job Lifecycle . . . 275
Job Methods . . . 275
Job Properties . . . 276
7. Retry and Dead Letters . . . 276
Max Retries . . . 276
Inspecting Retrying Jobs . . . 276
Dead Letters . . . 276
Reviving Dead Letters . . . 277
Purging Jobs . . . 277
8. Producing Multiple Jobs . . . 277
9. Switching Backends . . . 278
Default: File . . . 278
RabbitMQ . . . 278
Kafka . . . 278
MongoDB . . . 278
10. Separate Producer and Consumer Patterns . . . 278
11. Exercise: Build an Email Queue . . . 279
Requirements . . . 279
Test with: . . . 279
12. Solution . . . 279

13. Gotchas . . . 281
1. Always call complete or fail . . . 281
2. Worker not picking up messages . . . 281
3. Payload must be serializable . . . 281
4. Dead letters pile up . . . 281
5. File backend for production . . . 281
6. dead_letters and failed return Hashes . . . 282
Events **283**
1. Decouple With Events . . . 283
2. Basic Usage . . . 283
Registering a Listener . . . 283
Emitting an Event . . . 283
3. Multiple Listeners . . . 283
4. Priority . . . 284
5. Once: Fire a Listener One Time . . . 284
6. off: Remove a Listener . . . 285
7. clear: Remove All Listeners . . . 285
8. Events in Route Handlers . . . 285
9. Listing Registered Events . . . 286
10. Gotchas . . . 287
1. Listeners run synchronously . . . 287
2. Exceptions in listeners . . . 287
3. clear in tests . . . 287
4. once is not thread-safe by design . . . 287
Localization **288**
1. Your App Speaks One Language . . . 288

2. Locale Files . . . 288
3. Basic Translation . . . 289
4. Interpolation . . . 289
5. Switching Locales . . . 289
6. Fallback Locale . . . 290
7. Per-Request Locale in Routes . . . 290
8. Available Locales . . . 290
9. Locale in Templates . . . 291
10. Date and Number Formatting . . . 291
11. Gotchas . . . 292
1. Missing locale file returns fallback silently . . . 292
2. Nested key typo . . . 292
3. Reload locale files in development . . . 292
4. Interpolation keys are case-sensitive . . . 292
Structured Logging 293
1. puts Is Not a Logging Strategy . . . 293
2. Log Levels . . . 293
3. Basic Usage . . . 293
4. Controlling the Log Level . . . 294
5. Reconfiguring the Logger . . . 294
6. Logging in Route Handlers . . . 294
7. Logging Errors with Exceptions . . . 295
8. Request Logging Middleware . . . 295
9. JSON Output Format . . . 295
10. Writing to a File . . . 296
11. Checking the Current Level . . . 296

12. Gotchas . . . 296
1. Never log passwords or tokens . . . 296
2. Avoid string interpolation in the message . . . 296
3. DEBUG in production floods logs . . . 296
4. Logging an error does not exit . . . 296
Email with Messenger **297**
1. Every App Sends Email . . . 297
2. Messenger Configuration via .env . . . 297
Staging delivery safety . . . 298
Common Provider Configurations . . . 298
3. Constructor Override Pattern . . . 299
4. Sending Plain Text Email . . . 299
The send Method Signature . . . 300
5. Sending HTML Email with Text Fallback . . . 300
6. Adding Attachments . . . 301
Multiple Attachments . . . 301
7. CC, BCC, and Reply-To . . . 302
8. Custom Headers . . . 302
9. Reading Inbox via IMAP . . . 302
Listing Inbox Messages . . . 303
Reading a Specific Message . . . 304
Searching Messages . . . 304
Other IMAP Operations . . . 304
10. Dev Mailbox Capture . . . 305
Sending from a development environment . . . 305
11. Using Templates for Email Content . . . 305
Rendering and Sending . . . 306

12. Sending Email via Queues . . . 307
13. Exercise: Build an Email Notification System . . . 308
Requirements . . . 308
Test with: . . . 308
14. Solution . . . 308
15. Gotchas . . . 309
1. Gmail Requires App Passwords . . . 309
2. Emails Go to Spam . . . 309
3. HTML Email Rendering Differences . . . 309
4. Attachment File Not Found . . . 309
5. SMTP Connection Timeout . . . 309
6. Dev Mode Emails Disappear on Restart . . . 310
7. Unicode Characters Display as Question Marks . . . 310
8. IMAP Connection Fails . . . 310
Frontend with tina4css **311**
1. The Problem with Frontend Toolchains . . . 311
2. What Ships with Tina4 . . . 311
3. The Grid System . . . 311
Responsive Breakpoints . . . 312
4. Cards . . . 312
Card Grid . . . 312
5. Tables . . . 313
6. Forms . . . 313
7. Buttons and Alerts . . . 314
Buttons . . . 314
Alerts . . . 314

8. frond.js -- AJAX Without jQuery . . . 314
API Calls . . . 314
Token Management . . . 315
Form Submission via AJAX . . . 315
9. Dark Mode . . . 315
10. Building a Dashboard . . . 315
11. SCSS Customization . . . 317
Live SCSS Compilation . . . 318
12. frond.js API Reference . . . 318
Including the Scripts . . . 318
tina4.min.js -- Core Utilities . . . 318
loadPage(url, targetId) . . . 318
saveForm(formId, url, method) . . . 318
sendRequest(url, method, data, callback) . . . 319
frond.min.js -- Template Engine Client-Side Helpers . . . 319
AJAX Requests . . . 319
Token Management . . . 319
Loading Indicators . . . 320
WebSocket Auto-Reconnect . . . 320
tina4js.min.js -- Reactive Frontend Framework . . . 320
Reactive State . . . 320
Web Components . . . 321
13. Responsive Design . . . 321
Responsive Columns . . . 321
Responsive Visibility . . . 321
14. Building a Users Page with AJAX . . . 322
15. Exercise: Build a Product Management Page . . . 322

Requirements . . . 323
16. Solution . . . 323
17. Gotchas . . . 323
1. tina4css Classes Do Not Work . . . 323
2. frond.js Not Loading . . . 323
3. AJAX Calls Return HTML Instead of JSON . . . 323
4. Forms Submit Twice . . . 324
5. Dark Mode Resets on Page Load . . . 324
6. Bootstrap Classes Not Working . . . 324
7. SCSS Changes Not Reflected . . . 324
18. HtmlElement: Programmatic HTML Builder . . . 324
Testing **325**
1. Why Tests Matter More Than You Think . . . 325
2. Your First Test . . . 325
How It Works . . . 326
3. Assertion Reference . . . 326
Equality . . . 326
Nil . . . 326
Truthiness . . . 326
Collections and Strings . . . 326
Exceptions . . . 326
JSON / HTTP . . . 327
4. Testing Routes . . . 327
TestClient Methods . . . 328
TestResponse . . . 328
5. Testing ORM Models . . . 328
Test database isolation . . . 329

6. Testing Authentication . . . 330
7. Setup and Teardown . . . 331
8. Running Tests . . . 331
Embedding the runner in your own scripts . . . 332
9. Testing Best Practices . . . 332
Test one thing per it . . . 332
Use descriptive test names . . . 332
Isolate tests . . . 333
Generate unique values . . . 333
Compare floats with tolerance . . . 333
10. Exercise: Write Tests for a Notes API . . . 333
Requirements . . . 333
11. Solution . . . 334
12. Gotchas . . . 335
1. Files must be named correctly . . . 335
2. Database state carries between tests . . . 336
3. TestClient doesn't start a real server . . . 336
4. Argument order on equality assertions . . . 336
5. Floating point comparisons . . . 336
6. Tests can't find your models . . . 336
7. Tests pass locally but fail in CI . . . 336
Scaffolding **337**
The CRUD Generator . . . 337
What Each File Contains . . . 337
Run It . . . 339
Individual Generators . . . 339
Model . . . 340

Route . . . 340
Migration . . . 340
Middleware . . . 340
Test . . . 340
Form . . . 340
View . . . 340
CRUD . . . 340
Auth . . . 341
AutoCRUD . . . 341
Usage . . . 341
The Auth Generator . . . 341
Field Types . . . 342
Table Naming Convention . . . 342
Combining Generators . . . 343
Exercise: Scaffold a Blog . . . 343
Gotchas . . . 344
Generators Do Not Overwrite . . . 344
Run Migrate After Generate . . . 344
File Naming Matters . . . 344
Field Changes Need New Migrations . . . 344
Singular Table Names . . . 344
API Documentation with Swagger **345**
1. The 47-Endpoint Problem . . . 345
2. What Swagger/OpenAPI Is . . . 345
3. Enabling Swagger . . . 345
The Swagger JSON Endpoint . . . 346

4. Adding Descriptions to Routes . . . 346
Documenting Path Parameters . . . 346
Documenting Query Parameters . . . 347
5. Documenting Request and Response Schemas . . . 347
Request Body . . . 347
6. Tags for Grouping Endpoints . . . 348
7. Example Values . . . 348
8. Try-It-Out from the Swagger UI . . . 348
Authentication in Try-It-Out . . . 349
9. Customizing the Swagger Info Block . . . 349
10. Generating Client SDKs from the Spec . . . 349
11. A Complete Documented API . . . 349
12. Exercise: Document a Complete User API . . . 350
Requirements . . . 350
Test by visiting: . . . 351
13. Solution . . . 351
14. Gotchas . . . 353
1. Annotations Must Be Directly Above the Route . . . 353
2. Missing @tags Makes Endpoints Hard to Find . . . 353
3. @body Must Be Valid JSON . . . 353
4. Swagger Shows Routes You Did Not Annotate . . . 353
5. Response Examples Do Not Match Actual Responses . . . 353
6. Swagger UI Not Available in Production . . . 353
7. SDK Generation Produces Incorrect Types . . . 353
API Client **354**
1. Talking to Other Services . . . 354
2. Creating a Client . . . 354

With Default Headers . . . 354
3. GET Requests . . . 354
With Query Parameters . . . 354
Response Object . . . 354
4. POST Requests . . . 355
5. PUT Requests . . . 355
6. DELETE Requests . . . 355
7. Authentication Headers . . . 355
Bearer Token . . . 355
API Key Header . . . 356
Basic Auth . . . 356
Per-Request Header Override . . . 356
8. Timeouts . . . 356
9. Using the Client in Route Handlers . . . 357
10. Shared Client Instances . . . 357
11. Error Handling . . . 358
12. Gotchas . . . 358
1. Never hard-code credentials . . . 358
2. Check success? before reading body . . . 358
3. Timeout is per-request . . . 358
4. Base URL trailing slash . . . 358
GraphQL and SOAP **359**
1. The Problem GraphQL Solves . . . 359
2. GraphQL vs REST -- A Quick Comparison . . . 359
3. Enabling GraphQL . . . 360
4. Defining Queries . . . 360

Testing the Queries . . . 361
5. Mutations . . . 361
Testing Mutations . . . 363
6. Nested Types and Relationships . . . 364
7. Auto-Generating Schema from ORM Models . . . 365
Example: ORM + Custom Queries . . . 366
8. Authentication in GraphQL . . . 366
9. The GraphiQL Playground . . . 367
10. Query Variables . . . 367
11. Exercise: Build a GraphQL API for a Blog . . . 368
Requirements . . . 368
Test with these queries: . . . 369
12. Solution . . . 369
13. Input Validation . . . 371
Example: Missing Required Argument . . . 372
14. Field-Level Auth Directives . . . 372
Passing Auth Context . . . 373
Schema Example . . . 373
Behavior on Failed Auth . . . 373
15. Gotchas . . . 374
1. Resolver Not Called . . . 374
2. Nested Data Returns Wrong Shape . . . 374
3. Mutation Returns Null . . . 374
4. N+1 Query Problem . . . 374
5. GraphQL Playground Returns 404 . . . 375
6. Type Mismatch Between Schema and Data . . . 375
7. Authentication Not Applied . . . 375

8. Error Messages Expose Internal Details . . . 375

14. SOAP / WSDL Services . . . 376

What is SOAP? . . . 376

Defining a SOAP Service . . . 376

Type Annotations Map to XSD . . . 377

A More Complete Example . . . 378

Lifecycle Hooks . . . 378

Testing with curl . . . 379

When to Use SOAP vs REST vs GraphQL . . . 379

Real-time with WebSocket **380**

1. The Refresh Button Problem . . . 380

2. What WebSocket Is . . . 380

3. Router.websocket -- WebSocket as a Route . . . 380

Shorthand Syntax . . . 381

Starting the Server . . . 381

4. Connection Events . . . 381

A Complete Handler . . . 382

5. Connection Methods . . . 382

Sending to a Single Client . . . 382

Closing a Connection . . . 383

6. Broadcasting to All Clients . . . 383

Broadcast Excluding Sender . . . 384

7. Path Parameters and Scoped Isolation . . . 384

8. Building a Live Chat . . . 385

WebSocket Handler . . . 385

9. Live Notifications . . . 386

10. Connecting from JavaScript . . . 387

Using Plain WebSocket . . . 387
Using Frond.js . . . 387
Auto-Reconnect . . . 388
11. A Complete Chat Page . . . 388
12. Server Compatibility . . . 389
13. Exercise: Build a Real-Time Chat Room . . . 390
Requirements . . . 390
Test by: . . . 390
14. Solution . . . 390
15. Scaling with a Backplane . . . 391
Configuration . . . 391
Requirements . . . 392
When to Choose a Backplane . . . 392
16. Gotchas . . . 392
1. WebSocket Needs a Persistent Server . . . 392
2. Events Are Symbols, Not Strings . . . 393
3. Messages Are Strings, Not Objects . . . 393
4. Connection Count Is Per-Path . . . 393
5. Broadcasting Does Not Scale Across Servers . . . 393
6. Large Messages Cause Disconnects . . . 393
7. Memory Leak from Tracking Connected Users . . . 393
8. No Authentication on WebSocket Connect . . . 393
Server-Sent Events (SSE) 394
1. The Polling Problem . . . 394
2. What SSE Is . . . 394
3. Your First Stream . . . 395

4. The SSE Protocol . . . 396
The data: Field . . . 396
The event: Field . . . 396
The id: Field . . . 396
The retry: Field . . . 396
The Delimiter . . . 397
5. Frontend: EventSource . . . 397
Basic Usage . . . 397
Named Events . . . 397
Closing the Connection . . . 398
With Authentication . . . 398
6. Real Examples . . . 398
Live Dashboard . . . 398
Build Progress . . . 399
LLM Chat Streaming . . . 399
File Processing Progress . . . 400
7. Custom Content Types . . . 401
NDJSON Streaming . . . 401
Consuming NDJSON on the Client . . . 401
8. SSE vs WebSocket . . . 402
9. Production Considerations . . . 403
Nginx Proxy Buffering . . . 403
Connection Limits . . . 403
Heartbeat Messages . . . 403
10. Exercise: Build a Live Metrics Dashboard . . . 404
Requirements . . . 404
Test by: . . . 404

11. Solution 404

12. What You Learned 406

WSDL / SOAP **407**

1. When the World Still Uses SOAP 407

2. What SOAP and WSDL Are 407

3. Defining a WSDL Service 407

4. Mounting the Service 408

5. Auto-Generated WSDL 408

6. Calling the SOAP Service 409

7. Calling a Remote WSDL Service 409

8. Complex Types 410

9. Error Handling in Operations 411

10. Gotchas 411

1. WSDL caching in production 411

2. Character encoding 411

3. Namespace conflicts 411

4. SOAP is not REST 411

Dependency Injection Container **412**

1. The Problem With Hard Dependencies 412

2. Registering a Service 412

Registering Multiple Services 412

3. Resolving a Service 413

4. Checking Registration 413

5. Using Services in Route Handlers 413

6. reset: Reset the Container 414

7. Testing With the Container 414

8. Service Dependencies . . . 415
9. Gotchas . . . 416
1. reset drops cached instances . . . 416
2. Singletons by default . . . 416
3. Circular dependencies cause infinite loops . . . 416
4. Registration order matters . . . 416
Service Runner **417**
1. Work That Runs Forever . . . 417
2. Defining a Service . . . 417
3. Starting All Services . . . 417
4. Starting a Single Service . . . 417
5. Stopping Services . . . 418
6. Checking Service Status . . . 418
7. Automatic Restart on Failure . . . 418
8. Queue Consumer Service . . . 419
9. Scheduled Task Service . . . 419
10. Metrics Collector Service . . . 420
11. Listing Registered Services . . . 420
12. Shutdown Hooks . . . 420
13. Gotchas . . . 421
1. Services share the main process . . . 421
2. sleep is required in loops . . . 421
3. Thread safety . . . 421
4. stop on shutdown . . . 421
MCP Dev Tools **422**
1. What is MCP? . . . 422

2. How It Works 422
Localhost-Only Security 422
3. Connecting AI Tools 422
Claude Code 422
Cursor 423
Manual Connection 423
4. Built-in Tools 423
Database 423
Routes 424
Templates 424
Files 424
Migrations 425
Data and ORM 425
Infrastructure 425
Debugging 425
Project introspection 426
Persistent code index 426
Framework documentation (prose) 426
Live API RAG (reflection) 426
Plan management 427
5. Protocol Details 427
Streamable HTTP (current) 428
Legacy HTTP+SSE (still supported) 428
Messages 428
Lifecycle 428
6. Troubleshooting 429

1. Beyond Dev Tools . . . 430
2. Creating an MCP Server . . . 430
3. Registering Tools with mcp_tool . . . 430
4. Registering Resources with mcp_resource . . . 431
5. Class-Based MCP Services . . . 431
6. Securing MCP Endpoints . . . 432
7. Testing MCP Tools . . . 433
8. Complete Example: CRM MCP Server . . . 433
9. Best Practices . . . 434

Dev Tools **436**

1. Debugging at 2am . . . 436
2. Enabling the Dev Dashboard . . . 436
3. Dashboard Overview . . . 436
    System Overview . . . 436
4. The Dev Toolbar . . . 437
    Disabling the Toolbar . . . 437
5. Error Overlay . . . 437
    Example Error . . . 438
    Error Overlay in Production . . . 438
6. The Gallery . . . 438
7. Live Reload . . . 439
    How It Works . . . 439
8. Request Inspector . . . 440
    Request Details Panel . . . 440
    Filtering Requests . . . 440
    Request Replay . . . 440

9. SQL Query Runner . . . 441
Safety . . . 441
10. Queue Monitor . . . 441
11. Log Viewer . . . 442
Logging from Code . . . 442
12. System Info . . . 442
13. Health Check Endpoint . . . 443
14. Exercise: Debug a Broken Endpoint . . . 443
Setup . . . 443
Requirements . . . 444
15. Solution . . . 444
16. Gotchas . . . 445
1. Dev Dashboard Accessible on Network . . . 445
2. Live Reload Causes Connection Drops . . . 445
3. Error Overlay Shows in Production . . . 445
4. Request Inspector Slows Down the Server . . . 445
5. SQL Runner Executes Destructive Queries . . . 445
6. Template Changes Not Reflected . . . 445
7. Log Files Growing Without Bound . . . 446
8. Queue Monitor Shows No Jobs . . . 446
9. Memory Usage Grows During Development . . . 446
Tina4 CLI **447**
1. Getting a New Developer Up to Speed . . . 447
2. tina4 init -- Project Scaffolding . . . 447
Language Detection . . . 447
Explicit Language Selection . . . 448
Init into an Existing Directory . . . 448

3. tina4 serve -- Dev Server . . . 448
Options . . . 449
Serving a Project by Name . . . 449
Bypassing the CLI . . . 449
4. tina4 generate model -- ORM Scaffolding . . . 449
Adding Fields . . . 450
Field Types . . . 450
Options . . . 451
5. tina4 generate route -- CRUD Route Scaffolding . . . 451
Options . . . 452
6. tina4 generate migration -- Migration Scaffolding . . . 453
7. tina4 generate middleware -- Middleware Scaffolding . . . 453
8. Generate All at Once . . . 454
9. tina4 doctor -- Health Check . . . 454
10. tina4 test -- Running Tests . . . 455
Test Options . . . 455
11. tina4 routes -- Route Listing . . . 455
Filtering . . . 455
12. tina4 migrate -- Database Migrations . . . 456
Rollback . . . 456
Migration Table Auto-Upgrade . . . 456
Status . . . 456
Other Migration Commands . . . 456
13. tina4 queue -- Queue Management . . . 457
14. tina4 build -- Build Commands . . . 457
15. tina4 deploy -- Deployment . . . 457

16. Environment-Specific Commands . . . 457
17. Exercise: Scaffold a Feature in 5 Commands . . . 457
Requirements . . . 457
Expected Commands . . . 458
18. Solution . . . 458
19. Gotchas . . . 459
1. tina4 Command Not Found . . . 459
2. Wrong Language Detected . . . 459
3. Generated Files Overwrite Existing Code . . . 459
4. Migration Order Issues . . . 460
5. tina4 serve Uses Wrong Port . . . 460
6. Doctor Shows False Warnings . . . 460
7. Model Name Must Be PascalCase . . . 460
8. build:css Fails . . . 460
9. deploy:docker Generates Wrong Dockerfile . . . 461
20. Documentation . . . 461
21. Test Port (Dual-Port Development) . . . 461
Vibe Coding with AI **462**
Why Tina4 is Built for AI-Assisted Development . . . 462
The Zero-Dependency Advantage . . . 462
CLAUDE.md -- The AI's Instruction Manual . . . 462
Skills -- Deep Framework Knowledge . . . 463
tina4-maintainer . . . 463
tina4-developer . . . 463
tina4-js . . . 463
Supported AI Tools . . . 463
The AI Chat in Dev Dashboard . . . 464

The Convention Advantage . . . 464
Prompt Engineering for Tina4 . . . 464
Creating a New Feature . . . 464
Adding Authentication . . . 465
Building a Dashboard . . . 465
Adding WebSocket . . . 465
The Ruby Advantage for AI . . . 465
Block Syntax . . . 465
Hash Syntax . . . 465
Enumerable Methods . . . 466
Vibe Coding Workflow . . . 466
1. Describe What You Want . . . 466
2. Review the Generated Code . . . 466
3. Run and Test . . . 466
4. Iterate . . . 466
5. Deploy . . . 466
Real-World Example: Building an API in 5 Minutes . . . 467
Why This Matters . . . 468
Exercise: Vibe-Code a Feature . . . 468
Gotchas . . . 468
1. AI Generates Rails Syntax Instead of Tina4 . . . 468
2. AI Invents Non-Existent Methods . . . 469
3. AI Generates Tests That Do Not Match the Implementation . . . 469
4. AI Forgets to Run Migrations . . . 469
5. AI Uses require Instead of Auto-Loading . . . 469
6. AI Generates Over-Engineered Solutions . . . 469
7. AI Does Not Know About to_h . . . 469

Environment Variables 470

Core Server . . . 471

Routing . . . 473

Secrets and Authentication . . . 473

Database . . . 474

CORS . . . 475

Security Headers . . . 476

Rate Limiting . . . 476

Sessions . . . 477

Valkey/Redis session backend . . . 477

Cache . . . 478

Templates . . . 479

GraphQL . . . 479

Queues . . . 479

Kafka queue backend . . . 480

RabbitMQ queue backend . . . 480

MongoDB queue backend . . . 480

WebSocket . . . 481

Email . . . 482

Logging . . . 483

Uploads . . . 484

Services (background tasks) . . . 484

Localisation . . . 485

Application AI Client and Developer AI Tools . . . 485

MCP . . . 486

Swagger / OpenAPI . . . 487

Configuration Recipes . . . 487
PostgreSQL in production . . . 487
A shared cache on Redis . . . 488
Sessions that survive more than one box . . . 488
A queue on RabbitMQ or Kafka . . . 488
WebSocket broadcasts across instances . . . 488
Locked-down production headers . . . 488
The dev dashboard AI, kept local . . . 489
Minimal .env for Development . . . 489
Minimal .env for Production . . . 489
Feature Module Settings . . . 489
Deployment **491**
1. From Development to Production . . . 491
2. Production .env Configuration . . . 491
Key Differences from Development . . . 492
Sensitive Values . . . 492
3. Puma Configuration . . . 492
How Many Workers? . . . 493
4. Docker Deployment . . . 493
Dockerfile . . . 494
.dockerignore . . . 494
Docker Compose . . . 494
Building and Running . . . 495
5. Health Checks . . . 496
6. Graceful Shutdown . . . 496
Configuring Shutdown Timeout . . . 497
Docker Stop Grace Period . . . 497

Graceful Restart Pattern . . . 497
7. Log Rotation . . . 497
Using logrotate (Linux) . . . 497
Docker Logging . . . 498
8. Reverse Proxy with Nginx . . . 498
9. SSL/TLS with Let's Encrypt . . . 499
With Nginx and Certbot . . . 499
With Docker (Using Traefik as Reverse Proxy) . . . 500
Certificate Monitoring . . . 500
10. Process Management with systemd . . . 500
11. Scaling . . . 501
Multiple Workers . . . 501
Load Balancing with Nginx . . . 501
Docker Scaling . . . 502
Scaling Considerations . . . 502
12. Monitoring . . . 502
Log Aggregation . . . 503
Prometheus Metrics . . . 503
Uptime Monitoring . . . 503
Application Performance Monitoring (APM) . . . 503
13. Exercise: Docker Deploy . . . 504
Requirements . . . 504
14. Solution . . . 505
HTTP Caching, Compression, and ETags . . . 505
15. Gotchas . . . 505
1. .env Not Loaded in Docker . . . 505
2. SQLite Database Lost on Container Restart . . . 506

3. WebSocket Connections Drop Behind Nginx . . . 506
4. Puma Workers Not Reconnecting to Database . . . 506
5. Built-in Server Used in Production . . . 506
6. Static Files Slow . . . 506
7. Memory Usage Grows Over Time . . . 507
8. SSL Certificate Not Renewing . . . 507
9. Scaled Instances Have Different State . . . 507
10. Queue Worker Stops Processing . . . 507

Building a Complete App **508**

1. Putting It All Together . . . 508
2. Planning the App . . . 508
Models . . . 508
Relationships . . . 508
Routes . . . 509
Templates . . . 509
3. Step 1: Init Project and Set Up Database . . . 509
Create Migrations . . . 509
4. Step 2: ORM Models . . . 510
5. Step 3: Authentication . . . 511
Test Registration and Login . . . 512
6. Step 4: Task CRUD . . . 513
Test the Task API . . . 516
7. Step 5: Dashboard Stats with Caching . . . 517
8. Step 6: WebSocket Real-Time Updates . . . 518
9. Step 7: Email Notifications . . . 518
10. Step 8: Dashboard Template . . . 519

11. Step 9: Tests . . . 521
12. Step 10: Docker Deployment . . . 523
13. The Complete Project Structure . . . 524
14. What You Built . . . 524
15. What to Build Next . . . 525
16. Gotchas . . . 526
1. Circular Dependencies Between Models . . . 526
2. Cache Not Invalidated on Related Model Changes . . . 526
3. Queue Worker Not Processing Emails . . . 526
4. Token Expired During Long Sessions . . . 526
5. WebSocket Notifications Not Received . . . 526
6. Database Locked Under Load . . . 526
7. Missing Migration Before Deployment . . . 526
17. Closing Thoughts -- The Tina4 Philosophy . . . 527
Release Notes **528**
v3.13.138 (2026-09-24) - Safer requests, templates, and database operations . . . 528
v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests . . . 529
Queue and ORM audit bugs — five fixed . . . 529
Route inspection scans, never boots . . . 529
Startup log tells the truth about @noauth / @secured . . . 530
Hot reload reaches every module that imports the changed one . . . 530
Firebird migration ledger is case-agnostic . . . 530
Test-harness fixes . . . 530
Skills grow a spine . . . 530
Upgrade notes . . . 531
v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land . . . 531
OIDC / OpenID Connect — provider-neutral, session handoff . . . 531

PostGIS spatial support . . . 531
Swagger is SSO-aware . . . 531
v3.13.102 (2026-08-15) - Unified client and CI hardening . . . 531
v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie . . . 532
What changed . . . 532
What ships . . . 532
Failure is data . . . 533
Upgrade notes . . . 533
v3.13.100 (2026-08-14) - Frond keeps the whole page . . . 533
Template inheritance . . . 533
Bounded caches . . . 533
Frond extension scope . . . 533
AI skill installer . . . 533
Release consistency . . . 533
Upgrade notes . . . 533
v3.13.99 (2026-08-13) - Stability, parity, and secure by default . . . 534
Security . . . 534
Data integrity . . . 535
ORM and validation . . . 535
Request model - the big one . . . 536
Migrations . . . 536
HTTP . . . 536
Dev tooling . . . 536
Proven in all four . . . 537
Bug fixes . . . 537
Possible breaking . . . 537
Coming in 3.13.100 . . . 538

v3.13.98 (2026-08-11) - Skills discipline . . . 538
v3.13.97 (2026-08-07) - Behaviour corrections . . . 539
Sessions . . . 539
Queues . . . 539
Proven . . . 539
Possible breaking . . . 539
v3.13.96 (2026-08-07) - One paginate envelope, one message shape . . . 540
Pagination . . . 540
Messenger . . . 540
Swagger . . . 541
Migrations . . . 541
Server . . . 541
Build and skills . . . 541
Proven in all four . . . 541
Possible breaking . . . 541
v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release . . . 542
Queues . . . 542
Logging . . . 542
Databases . . . 542
Sessions . . . 542
Document store . . . 543
Local by default, production by environment variable . . . 543
Bulk insert . . . 543
Tooling . . . 543
Possible breaking . . . 543
v3.13.94 (2026-07-29) - A write with no filter is an error . . . 544
The write path (BREAKING) . . . 544

The Frond sandbox revokes capability, it does not skip a step . . . 545
Messenger: one name, one signature . . . 545
Base images that boot . . . 545
Frond, measured against the engines it replaces . . . 545
Also in this release . . . 546
Ruby specifics . . . 546
v3.13.92 (2026-07-27) - The algorithm you configure is the algorithm that signs . . . 546
The header names the algorithm that signed . . . 546
TINA4JWTALGORITHM is read for real . . . 547
get_token no longer stamps nbf (Breaking, Ruby only) . . . 547
nbf validation gains 60 seconds of leeway . . . 547
The header cannot choose the verifier . . . 547
Overrides on authenticate_request are honoured . . . 547
A misconfigured algorithm raises (Breaking) . . . 548
The live Mongo queue specs stop dropping each other's database . . . 548
The tests that hold it . . . 548
v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing . . . 548
A function is no longer charged for the functions inside it . . . 549
Ruby complexity was double what it should have been . . . 549
LOC means one thing again . . . 550
The dev dashboard reads the coupling it is given . . . 550
v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree . . . 550
One default, defined once (php#186, ruby#33, nodejs#38) . . . 551
A failed write no longer reports success . . . 551
v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding . . . 551
An unknown tag raises instead of leaking its body (Breaking, Security) . . . 551
set works as a block (Fixed) . . . 552

The expression corpus grew to 82 . . . 552
v3.13.88 (2026-07-27) - json_encode gives you JSON again . . . 552
json_encode emits JSON, not entities (Breaking, reverts 3.13.87) . . . 553
A value JSON cannot represent becomes null (Fixed) . . . 553
to_json and tojson are the same filter . . . 553
The expression corpus grew to 79 . . . 554
What to change in your templates . . . 554
v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks . . . 554
Booleans print as true or false (Breaking) . . . 554
{% import "file" as alias %} now works (New) . . . 555
The not operator works in output (Fixed) . . . 555
json_encode escaping . . . 555
The gate that keeps this true . . . 555
v3.13.86 (2026-07-25) - Writes return a DatabaseResult . . . 556
v3.13.85 (2026-07-24) - The dev-admin bundle ships once . . . 556
Why the surviving file is still called .min.js . . . 556
A gate, so the duplicate cannot come back . . . 556
v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts . . . 557
serve no longer kills PID 1 (all four frameworks) . . . 557
Ruby: the image builds . . . 558
A gate, so this cannot rot again . . . 558
Also in the CLI (tina4 v3.8.58) . . . 558
v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production . . . 558
The banner advertises only what answers (python#99) . . . 559
logger and base64 are declared dependencies . . . 559
The CLI runs no watcher in production (tina4 v3.8.57) . . . 559
A stale CA no longer turns a missing certificate into six failures (python#98) . . . 559

v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency . . . 559

v3.13.81 (2026-07-21) - tina4ruby test exit code, locked in . . . 561

v3.13.79 (2026-07-19) - Session cookies get Secure behind a proxy, honour SameSite, and a renamed cookie is read back . . . 561

v3.13.77 (2026-07-16) - Background task scheduling confirmed, no code change . . . 561

v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55 . . . 562

v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh . . . 562

v3.13.74 (2026-07-13) - A lock-in test for the connection tester . . . 562

v3.13.73 (2026-07-13) - A failed migration re-applies cleanly . . . 563

v3.13.72 (2026-07-12) - Frond number_format, filter precedence, and a sandbox fix . . . 563

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance . . . 564

v3.13.70 (2026-07-11) - Column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata . . . 564

- ORM honours column defaults on INSERT (#165) . . . 564
- Firebird connection charset override (#160) . . . 564
- Stacked Swagger metadata all survives (#59) . . . 564

v3.13.69 (2026-07-10) - The Api client grows up: uploads, downloads, redirects, and a breaking upload signature . . . 565

- Breaking . . . 565
- Also shipping . . . 565

v3.13.68 (2026-07-10) - Steadier test suite . . . 565

v3.13.67 (2026-07-10) - The MCP table browser, locked down . . . 566

v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests . . . 566

- The self-describing CLI . . . 566
- Generators now ship a test with the code . . . 566
- Databases . . . 567
- Fixes . . . 567
- Breaking . . . 567

v3.13.56 (2026-07-08) - Skills that own up when they drift . . . 567
v3.13.55 (2026-07-07) - One migration tracking schema on every engine . . . 568
v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive . . . 568
v3.13.53 (2026-07-06) - JSONField: JSON document columns . . . 569
v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions . . . 569
v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes . . . 570
v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix) 571
v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation . . . 571
v3.13.46 (2026-06-24) - MSSQL fetch_one fix + real MySQL/MSSQL batch tests . . . 572
v3.13.45 (2026-06-24) - Real-service test hardening . . . 572
v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks) . . . 573
v3.13.43 (2026-06-22) - Queue: MongoDB fail() now dead-letters correctly . . . 574
v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs . . . 574
v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery) . . . 575
v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul . . . 576
v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth . . . 577
v3.13.38 (2026-06-19) - Coordinated security & robustness release . . . 578
v3.13.37 (2026-06-18) - Dev-admin editor: syntax highlighting now works . . . 578
v3.13.36 (2026-06-18) - Instant WebSocket dev-reload + dev-admin file browser fix . . . 578
v3.13.35 (2026-06-17) - Live MCP endpoint for AI agents . . . 579
v3.13.34 (2026-06-17) - Cross-framework parity release (no functional change in Ruby) . . . 579
v3.13.33 (2026-06-17) - Queues: priority pop + automatic dead-lettering (Warning: behavioural change) . . . 579
v3.13.32 (2026-06-17) - Caching: per-query bypass + X-Cache headers (chapter rewritten to match code) . . . 579
v3.13.31 (2026-06-17) - Request/response parity with Python/PHP/Node (Warning: breaking: request.body) . . . 579

v3.13.30 (2026-06-16) - Cross-framework parity release (no functional change in Ruby) 580
v3.13.29 (2026-06-16) - Live API search ranks qualified queries + resolves natural names . . 580
v3.13.27 (2026-06-16) - Frond template-engine parity fixes 580
v3.13.26 (2026-06-16) - pooling parity confirmed; standalone writes auto-commit 581
v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache 581
v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default 582
v3.13.22 (2026-06-15) - session default TTL standardised to 1 hour 582
v3.13.21 (2026-06-15) - docs: render() corrections + version re-sync 582
v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder 582
response serializes domain objects 583
Construct a model from a JSON object string 583
Warning: Breaking - one database binder: bind_database 583
v3.13.18 (2026-06-15) - ORM Model.query + foreign-key wiring fixes 583
v3.13.17 (2026-06-15) - PostgreSQL reads return native Ruby types 584
v3.13.16 (2026-06-15) - create_table works on PostgreSQL + DatabaseResult index access . 584
Root cause: get_database_type didn't exist 584
DatabaseResult index + slice access 585
v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48) 585
Per-request logging - on by default in dev 585
What changed (stdout) 585
Why it spanned all four 585
Schema-qualified tables (#48) + a PostgreSQL fetch() regression 586
Tests 587
v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + fetch_all correctness 587
fetch_all actually fetches ALL rows now (no silent 100-row truncation) 587
Trailing ; is now stripped from user SQL in fetch / fetch_one 587

Ruby ORM now auto-discovers TINA4_DATABASE_URL (the binding fix) . . . 588
Cross-framework parity . . . 588
Tests . . . 588
v3.13.11 (2026-06-11) - ORM correctness pass . . . 589
#50.1 - Callable Proc defaults are now resolved per-instance . . . 589
BooleanField - engine-aware DDL on PG / MySQL / MSSQL . . . 589
#50.2 - natural-key INSERT (already correct, now pinned) . . . 589
PG error-visibility fixes (Python only) . . . 589
Tests . . . 589
v3.13.9 (2026-06-10) . . . 589
The bug . . . 590
The fix . . . 590
Same algorithm in Python / PHP / Node . . . 590
Tests . . . 590
What you'll see when you re-install . . . 590
v3.13.7 (2026-06-10) . . . 591
NEW: tina4.request.error event . . . 591
FIX: Stack trace removed from production 500 body (CWE-209) . . . 591
Tests . . . 591
Background . . . 592
v3.13.6 (2026-06-09) . . . 592
Spec contamination from Frond static-facade - fixed . . . 592
Better driver install hints (#47) . . . 592
#46 - PostgreSQL transaction cascade (no fix needed) . . . 592
Tests . . . 592
v3.13.5 (2026-06-05) . . . 593
What changes . . . 593

Instance form still works . . . 593
clearRegistry() for test fixtures . . . 593
Implementation notes per framework . . . 594
Test count . . . 594
Upgrade . . . 594
v3.13.4 (2026-06-04) . . . 594
PY-10-02 - @middleware() no longer silently disables auth (SECURITY) . . . 594
PY-10-03 - request.headers is now case-insensitive . . . 595
PY-10-01 - Function-based middleware now runs . . . 595
Python chapter rewrites - book + docs . . . 596
Test count . . . 596
Upgrade . . . 596
v3.13.3 (2026-06-03) . . . 596
Env typed env-var helpers (tina4-python#43) . . . 596
Function-name in log lines (tina4-python#41) . . . 597
Test count . . . 597
Upgrade . . . 598
v3.13.2 (2026-06-03) . . . 598
SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1) . . . 598
Router.group docs taught a crashing pattern (tina4-python#44) . . . 598
DATABASEURL -> TINA4DATABASE_URL drift (tina4-python#45) . . . 598
Test count . . . 599
Upgrade . . . 599
v3.13.1 (2026-06-02) . . . 599
Convenience parity additions (Groups A / B) . . . 599
Decorator-style GraphQL resolvers across the family . . . 599
Class-based service pattern across the family . . . 600

PHP chapter rewrites (docs/php/ and book-2-php/) . . . 600
Test count . . . 601
Upgrade . . . 601
v3.13.0 (2026-06-01) . . . 601
The headline fire: Test class with HTTP helpers . . . 601
Auth.valid_token now returns the payload, not a bare bool - BREAKING . . . 602
Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches) . . . 602
PHP-specific: Tina4\Debug shim . . . 603
Documentation sweep . . . 603
Aspirational chapters rewritten . . . 604
tina4-js doc drift caught . . . 604
Test counts . . . 604
Upgrade . . . 605
v3.12.14 (2026-06-01) . . . 605
Python - tina4_python.test xUnit testing surface . . . 605
Cross-framework parity check (testing) . . . 606
PHP - :named placeholder translation across non-PDO adapters . . . 606
Cross-framework parity check (:named placeholders) . . . 607
Upgrade . . . 607
v3.12.13 (2026-05-29) . . . 607
Cross-framework dev-admin parity sweep (Tier 1-5) . . . 607
Folded-in from PHP 3.12.11 - file upload regression (tina4-book#139) . . . 609
Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 tina4_migration auto-upgrade (#115) . . . 609
Folded-in from PHP 3.12.11 - request URL parity . . . 610
Upgrade . . . 610
v3.12.10 (2026-05-14) . . . 611
PHP - ORM->save() no longer swallows write failures (#114) . . . 611

Also in the PHP 3.12.7-3.12.9 patch line . . . 611
Python / Ruby / Node . . . 611
Upgrade . . . 611
v3.12.6 (2026-05-06) . . . 612
Python - psycopg2 % substitution no longer trips PL/pgSQL function bodies (#40) . . . 612
Long-standing tina4-js #37 confirmed fixed . . . 612
Upgrade . . . 612
v3.12.5 (2026-05-06) . . . 613
PHP - multipart bodies with file uploads now parse correctly (#135) . . . 613
Upgrade . . . 613
v3.12.4 (2026-05-06) . . . 613
25 new env vars across all 4 frameworks . . . 613
Logging - env-driven file output + rotation . . . 614
Documentation-truth CI gate now strict on both axes . . . 614
Tests added . . . 614
Upgrade path . . . 615
v3.12.3 (2026-05-05) . . . 615
Breaking changes (Ruby + PHP only) . . . 615
Doc parity - CLAUDE.md and book chapter 33 . . . 615
Other fixes . . . 616
Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3) . . . 616
Upgrade path . . . 616
v3.12.2 (2026-05-05) . . . 616
Firebird URL auto-detect . . . 616
Bug fix specific to PHP - mysqli localhost+port quirk . . . 617
Other fixes . . . 617
v3.12.1 (2026-05-04) . . . 617

v3.12.0 (2026-05-04) . . . 618
Why this release . . . 618
What changed . . . 618
Bug fixes shipped alongside the rename . . . 619
Migration - every renamed var . . . 620
Names that stay un-prefixed (not framework config) . . . 620
How to upgrade . . . 620
Parity . . . 621
v3.11.32 (2026-04-25) . . . 621
v3.11.13 (2026-04-16) . . . 622
v3.11.12 (2026-04-16) . . . 623
v3.11.11 (2026-04-16) . . . 624
v3.11.10 (2026-04-15) . . . 624
v3.11.9 (2026-04-15) . . . 625
v3.10.99 (2026-04-12) . . . 625
v3.10.97 (2026-04-11) . . . 626
v3.10.93 (2026-04-11) . . . 626
v3.10.92 (2026-04-10) . . . 626
v3.10.91 (2026-04-10) . . . 626
v3.10.90 (2026-04-09) . . . 627
v3.10.89 (2026-04-09) . . . 627
v3.10.86 (2026-04-09) . . . 627
v3.10.85 (2026-04-09) . . . 627
v3.10.84 (2026-04-09) . . . 627
v3.10.83 (2026-04-08) . . . 628
v3.10.70 (2026-04-06) . . . 628

v3.10.68 (2026-04-03) - Full Parity Release . . . 628
v3.10.67 (2026-04-03) . . . 629
v3.10.66 (2026-04-03) . . . 629
v3.10.65 (2026-04-03) . . . 629
v3.10.60 (2026-04-03) . . . 629
v3.10.57 (2026-04-02) . . . 630
v3.10.54 (2026-04-02) . . . 630
v3.10.48 - April 2, 2026 . . . 630
Bug Fixes . . . 630
v3.10.46 - April 1, 2026 . . . 631
Test Coverage . . . 631
v3.10.45 - April 1, 2026 . . . 631
Notes . . . 631
v3.10.44 - April 1, 2026 . . . 631
New Features . . . 631
Bug Fixes . . . 631
Test Coverage . . . 632
v3.10.40 - April 1, 2026 . . . 632
Bug Fixes . . . 632
v3.10.39 - April 1, 2026 . . . 632
New Features . . . 632
v3.10.38 -- April 1, 2026 . . . 632
Code Metrics & Bubble Chart . . . 632
AI Context Installer . . . 633
Dashboard Improvements . . . 633
Cleanup . . . 633
v3.10.x -- Previous Releases . . . 633

v3.10.29 -- Version Parity (March 30) . . . 633
v3.10.27 -- Frond Macro HTML Escaping Fix (March 30) . . . 633
v3.10.25 -- ORM Transaction Fix (March 30) . . . 634
v3.10.22 -- Unique Form Tokens (March 30) . . . 635
v3.10.18 -- Frond Ternary Parser Fix (March 29) . . . 635
v3.10.16 -- Template Filters: tojson, jsescape (March 28) . . . 636
v3.10.15 -- Replace Filter Backslash Fix (March 28) . . . 636
v3.10.14 -- getnextid() (March 28) . . . 636
v3.10.13 -- ORM Auto-Commit on Write (March 28) . . . 636
v3.10.12 -- Session GC and NATS Backplane (March 28) . . . 636
v3.10.11 -- Frond Variable Key Access (March 28) . . . 636
v3.10.10 -- Firebird Migration Runner Fixes (March 28) . . . 637
v3.10.6 -- WSDL/SOAP Rewrite (March 28) . . . 638
v3.10.5 -- Frond Quote-Aware Operator Matching (March 28) . . . 638
v3.10.4 -- Auto-CRUD REST Endpoint Generator (March 28) . . . 638
v3.10.2 -- Frond Hash Method Calls (March 28) . . . 638
v3.10.1 -- autoMap and Case Conversion (March 28) . . . 638
v3.10.0 -- Optimized For-Loops (March 28) . . . 638
v3.9.x . . . 638
v3.9.0 -- QueryBuilder, Sessions, Path Injection (March 26) . . . 638
v3.9.1 -- Security Defaults (March 27) . . . 639
v3.9.2 -- NoSQL QueryBuilder, WebSocket Backplane (March 27) . . . 640
v3.8.x . . . 640
v3.8.0 -- Base64 Filters, Template Cache (March 25) . . . 640
v3.8.1 -- Security Headers Middleware (March 25) . . . 640
v3.8.2 -- Connection Pooling (March 26) . . . 641
v3.8.3 -- Claude Code Commands (March 26) . . . 641

v3.8.7 -- Benchmark and Stability (March 26) . . . 641
v3.7.x . . . 641
v3.7.0 -- Template Auto-Serve, Firebird Migrations (March 25) . . . 641
v3.7.1 -- Full Template Auto-Serve (March 25) . . . 641
v3.6.x . . . 641
v3.6.0 -- Architectural Parity (March 25) . . . 641
v3.5.x . . . 642
v3.5.0 -- Bundled Frontend, Swagger CRUD, Middleware (March 24) . . . 642
v3.4.x . . . 642
v3.4.0 -- Auth, WebSocket, DatabaseResult (March 24) . . . 642
v3.3.x . . . 644
v3.3.0 -- Queue API, Route Chaining (March 24) . . . 644
v3.2.x . . . 645
v3.2.0 -- Flexible Route Handlers (March 23) . . . 645
v3.1.x . . . 646
v3.1.0 -- ORM Relationships, Caching, Queues (March 22) . . . 646
v3.1.1 -- DevMailbox Fix (March 22) . . . 647
v3.1.2 -- Documentation Fixes (March 22) . . . 647
v3.0.x . . . 647
v3.0.0 -- Initial Release (March 21) . . . 647
Pre-Release (v0.x) . . . 648
Feature Catalog **649**
Read the status correctly . . . 649
Where the features live . . . 650
Current parity boundary . . . 650
Dependency boundary . . . 651
Numbered catalog . . . 651

Foundation . . . 651
Database and providers . . . 651
ORM and data layer . . . 652
HTTP core . . . 652
HTTP policies . . . 653
HTTP runtime . . . 653
Frond template engine . . . 654
Frontend assets . . . 654
Authentication . . . 654
Sessions and providers . . . 655
Cache and providers . . . 655
Integrations and storage . . . 656
Enterprise authentication . . . 657
Application runtime . . . 657
CLI . . . 658
Developer runtime . . . 658
Testing and verification tools . . . 659
What parity means . . . 659

Real-time Collaboration (WebRTC) 660

1. The Media Server You Do Not Have to Run . . . 660
2. What You Get, and How It Differs from Raw WebSocket . . . 660
3. Mounting the Module: Tina4::Realtime.mount . . . 661
    What each feature wires . . . 662
4. The Config Bootstrap: GET {p}/api/rtc/config . . . 662
5. ICE and TURN Servers: Tina4::Realtime.ice_servers . . . 663
    Environment variables . . . 664

6. Signalling: The Mesh Relay . . . 664
7. Chat: Secured Channels, Presence, History . . . 665
Chat history: GET {p}/api/channels/{id}/messages . . . 666
8. Files: Upload and Download . . . 667
POST {p}/api/files - upload . . . 667
GET {p}/api/files/{key} - download (secured) . . . 667
Storage backends . . . 668
9. Auth, Identity, and the Data Model . . . 668
Data model . . . 669
10. A Complete Example . . . 670
Backend - app.rb . . . 670
Browser - bootstrap from the config endpoint . . . 670
Browser - the chat side-panel (secured, needs a JWT) . . . 671
11. Footguns and Hard Rules . . . 672
Where to Go Next . . . 673
AI Client **674**
Configure a provider . . . 674
Complete one prompt . . . 674
Hold a chat . . . 675
Stream text . . . 675
Create embeddings . . . 676
Handle failures . . . 676
Timeouts and retries . . . 676
OpenID Connect SSO **678**
Configure SSO . . . 678
Use the identity . . . 678

Claims and provider recipes 679

GIS and PostGIS **680**

Define a point field 680

Query and return GeoJSON 680

IoT and MQTT **681**

Configure and publish 681

--- outline: deep ---

<div v-pre>

Getting Started with Tina4 Ruby

1. What Is Tina4 Ruby

Tina4 Ruby is a Rack-based web framework for Ruby 3.1+. The gem declares the packages it needs for serving, SQLite, mail, XML, logging, and encoding; selected providers can add more. Tina4 Ruby contributes to the framework family's 140-entry feature catalog; the catalog is an inventory, not a claim that every entry has reached parity.

It belongs to the Tina4 family: four backend implementations in Python, PHP, Ruby, and Node.js. They share contracts, project structure, template syntax, CLI commands, and `.env` variables. The parity audit records where an implementation still differs.

Tina4 Ruby follows Ruby conventions. Method names use `snake_case` -- `fetch_one`, `soft_delete`, `has_many`. Class names use `PascalCase`. Constants use `UPPER_SNAKE_CASE`.

By the end of this chapter, you will have a running Tina4 Ruby project with an API endpoint and a rendered HTML page.

---

2. Prerequisites and Installation

What You Need

Four things. Nothing more.

- **Ruby 3.1 or later** -- check with:

```
ruby -v
```

You should see output like:

```
ruby 3.3.0 (2023-12-25 revision 5124f9ac75) [arm64-darwin23]
```

Below 3.1? Upgrade first.

- **Bundler** -- Ruby's dependency manager:

```
bundle --version
```

```
Bundler version 2.5.6
```

Not installed? `gem install bundler`.

- **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

macOS (Homebrew):

```
brew install tina4stack/tap/tina4
```

Linux / macOS (install script):

```
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash
```

Windows (PowerShell):

```
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify:

```
tina4 --version
```

```
tina4 0.1.0
```

- **SQLite3 development libraries** -- most systems ship these:

```
# macOS (already included)
# Ubuntu/Debian
sudo apt-get install libsqlite3-dev

# Fedora
sudo dnf install sqlite-devel
```

Installing the Tina4 CLI

```
cargo install tina4
```

Or download from GitHub Releases.

The `tina4` CLI manages project scaffolding, development servers, migrations, and more across all Tina4 frameworks.

Creating a New Project

One command:

```
tina4 init ruby my-store
```

`tina4 init` installs the Tina4 CLI globally (via cargo, homebrew, or direct download), then scaffolds a complete project with routes, templates, database, and configuration.

```
Creating Tina4 project in ./my-store ...
  Detected language: Ruby (Gemfile)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created .keys/
```

```
  Created tests/

Project created! Next steps:
  cd my-store
  bundle install
  tina4 serve
```

Install the Ruby dependencies:

```
cd my-store
bundle install
```

```
Fetching gem metadata from https://rubygems.org/...
Resolving dependencies...
Installing tina4 (3.2.1)
Bundle complete! 1 Gemfile dependency, 1 gem installed.
```

One gem. No dependency tree. No version conflicts. Just `tina4`.

Starting the Dev Server

```
tina4 serve
```

```
 _____ _              _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 Ruby v3.2.1
  Server running at http://0.0.0.0:7147
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Open `http://localhost:7147`. The Tina4 welcome page appears.

Hit the health endpoint:

```
curl http://localhost:7147/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 12,
  "version": "3.2.1",
  "framework": "tina4-ruby"
}
```

Your project is alive.

---

3. Project Structure Walkthrough

Here is what `tina4 init` built:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .env.example            # Template for other developers
■■■ .gitignore              # Pre-configured
```

```
■■■ Gemfile                 # Gem dependencies
■■■ Gemfile.lock            # Locked dependency versions
■■■ app.rb                  # Application entry point
■■■ migrations/             # SQL migration files (project root)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond/Twig templates
■   ■   ■■■ errors/         # Custom 404.html, 500.html
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■   ■■■ frond.js    # Auto-provided JS helper library
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css   # Built-in CSS utility framework
■   ■   ■■■ scss/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ locales/            # Translation files
■       ■■■ en.json
■■■ data/                   # SQLite databases (gitignored)
■■■ logs/                   # Log files (gitignored)
■■■ .keys/                  # JWT keys (gitignored)
■■■ tests/                  # Your test files
```

Five directories matter:

- `src/routes/` -- Every `.rb` file here is auto-loaded at startup. Drop your route definitions here. Subdirectories work too.
- `src/orm/` -- Every `.rb` file here is auto-loaded. ORM model classes live here.
- `src/templates/` -- Frond (Tina4's built-in template engine -- see Chapter 4: Templates) looks here when you call `response.render("my-page.html", data)`.
- `src/public/` -- Files served directly. `src/public/images/logo.png` maps to `/images/logo.png`.
- `data/` -- The default SQLite database (`app.db`) lives here. Gitignored. Databases do not belong in version control.

---

4. Your First Route

Create `src/routes/greeting.rb`:

```
Tina4::Router.get("/api/greeting/{name}") do |request, response|
  name = request.params["name"]
  response.json({
    message: "Hello, #{name}!",
    timestamp: Time.now.iso8601
  })
end
```

Save the file. The dev server picks up the change. If not, restart with `tina4 serve`.

> ***Cross-language note.*** *Ruby, PHP, and Node read path parameters from* `request.params` / ~~`$request->params`~~ / ~~`req.params`~~*. Python passes them as*

function arguments instead. Same concept, two shapes; pick the chapter that matches your runtime.

Test It

```
http://localhost:7147/api/greeting/Alice
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Or curl:

```
curl http://localhost:7147/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00+00:00"}
```

Force pretty output:

```
curl "http://localhost:7147/api/greeting/Alice?pretty=true"
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Understanding What Happened

- You created a file in `src/routes/`. Tina4 discovered it at startup.
- `Tina4::Router.get("/api/greeting/{name}")` registered a GET route with a path parameter `{name}`.
- A request to `/api/greeting/Alice` matched the pattern. The router called your handler block.
- `request.params["name"]` returned `"Alice"` from the URL.
- `response.json(...)` serialized the hash to JSON, set `Content-Type: application/json`, and sent a `200 OK`.

Adding More HTTP Methods

Add a POST endpoint. Update `src/routes/greeting.rb`:

```
Tina4::Router.get("/api/greeting/{name}") do |request, response|
  name = request.params["name"]
  response.json({
    message: "Hello, #{name}!",
    timestamp: Time.now.iso8601
  })
end

Tina4::Router.post("/api/greeting") do |request, response|
  name = request.body["name"] || "World"
  language = request.body["language"] || "en"

  greetings = {
    "en" => "Hello",
    "es" => "Hola",
```

```
    "fr" => "Bonjour",
    "de" => "Hallo",
    "ja" => "Konnichiwa"
  }

  greeting = greetings[language] || greetings["en"]

  next response.json({
    message: "#{greeting}, #{name}!",
    language: language
  }, 201)
end.no_auth
```

::: tip Two Ruby-isms to notice

- `.no_auth` is chained on the block's return value. Tina4 secures `POST`, `PUT`, `PATCH`, and `DELETE` routes by default, so without `no_auth` the example returns `401 Unauthorized`. In production, remove `no_auth` and send an `Authorization: Bearer <token>` header instead.
- `next response.json(...)` exits the block with the response value. Inside a Tina4 route block, never use the `return` keyword, which is a `LocalJumpError` in Ruby (blocks are not methods). Use `next` when you need an early exit, otherwise rely on the block's final expression.

:::

Test:

```
curl -X POST http://localhost:7147/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

Status code: `201 Created`.

---

5. Your First Template

Tina4 uses **Frond** -- a zero-dependency, Twig-compatible template engine built from scratch. If you know Twig, Jinja2, or Nunjucks, you know Frond.

Create a Base Layout

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
```

```
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em;
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

Two blocks: `title` and `content`. Child templates override only what they need. `tina4.css` provides built-in styling. `frond.js` provides JS helpers. Both ship with every project.

Create a Product Listing Page

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.in_stock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
                {% if not loop.last %}
                    {# Don't add separator after the last item #}
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
        <p>No products available at the moment.</p>
    {% endif %}
{% endblock %}
```

Create the Route That Renders the Template

Create `src/routes/pages.rb`:

```
Tina4::Router.get("/products") do |request, response|
  products = [
    {
      name: "Wireless Keyboard",
      description: "Ergonomic wireless keyboard with backlit keys.",
      price: 79.99,
      in_stock: true
    },
    {
      name: "USB-C Hub",
      description: "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
      price: 49.99,
      in_stock: true
    },
    {
      name: "Monitor Stand",
      description: "Adjustable aluminum monitor stand with cable management.",
      price: 129.99,
      in_stock: false
    },
    {
      name: "Mechanical Mouse",
      description: "High-precision wireless mouse with 16,000 DPI sensor.",
      price: 59.99,
      in_stock: true
    }
  ]

  response.render("products.html", { products: products })
end
```

See It in the Browser

Open `http://localhost:7147/products`. You see:

- A dark nav bar with "Home" and "Products"
- The heading "Our Products"
- "Showing 4 products"
- Four product cards with name, description, price, and stock badge
- "Monitor Stand" wears a red "Out of Stock" badge
- The other three wear green "In Stock" badges

How Template Rendering Works

- `response.render("products.html", { products: products })` tells Frond to render `src/templates/products.html`.
- Frond sees `{% extends "base.html" %}` and loads the base template.
- `{% block content %}` in `products.html` replaces the same block in `base.html`.
- `{{ product.name }}` outputs the value, auto-escaped for HTML safety.

- `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.
- `{% for product in products %}` loops through the array.
- `{% if product.in_stock %}` renders the right badge.
- `{{ products | length }}` returns the item count.

About tina4css

`tina4.css` is Tina4's built-in CSS utility framework. Layout utilities, typography, common UI patterns. No Bootstrap. No Tailwind. No npm. It ships with every scaffolded project.

---

6. Understanding .env

Open `.env` at the project root:

```
TINA4_DEBUG=true
```

That is it. The scaffold creates a minimal `.env`. Everything else uses defaults.

The defaults that matter:

Variable	Default Value	What It Means
`TINA4_PORT`	`7147`	Server runs on port 7147
`TINA4_DATABASE_URL`	`sqlite:///data/app.db`	SQLite database in `data/`
`TINA4_LOG_LEVEL`	`ALL`	All log messages output
`TINA4_CORS_ORIGINS`	*(empty)*	No cross-origin browser requests; list your front end's origin to allow it
`TINA4_RATE_LIMIT`	`100`	100 requests per minute per IP

Log levels control how much output Tina4 produces:

Level	Behaviour
`ALL` / `DEBUG`	Full verbose output. DevReload active (live-reload, error overlay).
`INFO`	Standard logging. Startup messages, request summaries.
`WARNING`	Warnings and errors only.
`ERROR`	Errors only. Minimal output.

Set `TINA4_LOG_LEVEL=DEBUG` during development for maximum visibility. Use `WARNING` or `ERROR` in production.

To change the port, use the CLI flag or `.env`:

```
tina4 serve --port 8080
```

Or add it to your `.env` file:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server. It now runs on 8080.

How port resolution works: The Rust CLI (`tina4 serve`) determines the port using this priority order:

- **CLI flag** (highest priority): `tina4 serve --port 8080`
- `.env` **file**: `TINA4_PORT=8080`
- **Environment variable**: `PORT=8080`
- **Framework default** (PHP: 7145, Python: 7146, Ruby: 7147, Node.js: 7148)

The CLI reads your `.env` file and checks for `TINA4_PORT` (and falls back to `PORT`). The resolved port is passed to the Ruby server. All three methods work -- use whichever fits your workflow.

For the complete `.env` reference with all 68 variables, see Environment Variables.

---

7. The Dev Dashboard

With `TINA4_DEBUG=true` in your `.env`, Tina4 provides a built-in development dashboard. No additional environment variables are needed.

Restart and navigate to:

```
http://localhost:7147/__dev
```

The dashboard shows:

- **System Overview** -- framework version, Ruby version, uptime, memory, database status
- **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request for full headers, body, queries, and template renders.
- **Error Log** -- unhandled exceptions with stack traces and occurrence counts
- **Queue Manager** -- pending, reserved, failed, dead-letter messages
- **WebSocket Monitor** -- active connections with metadata
- **Routes** -- all registered routes with methods, paths, and middleware

The dev dashboard is a debugging powerhouse. It shows you what your application does without print statements.

HTML pages also get a **debug overlay** -- a toolbar at the bottom showing:

- Request details (method, URL, duration)
- Database queries executed (with timing)
- Template renders (with timing)
- Session data
- Recent log entries

The overlay vanishes when `TINA4_DEBUG=false`. Production users never see it.

---

8. Manual Setup (No CLI)

The `tina4` CLI scaffolds everything for you. But if you start from an empty folder (just Ruby and Bundler) here is the minimum you need.

Step 1: Create `Gemfile`

```
source "https://rubygems.org"

gem "tina4ruby", "~> 3.0"

# Required: Ruby 3.0+ removed webrick from the standard library
gem "webrick", "~> 1.8"

# Required: the default SQLite database driver
gem "sqlite3", "~> 2.0"
```

Then install:

```
bundle install
```

::: warning Two dependencies you must declare

- `webrick` - Tina4's dev server uses WEBrick. Ruby 3.0 dropped WEBrick from the standard library, so you must list it in your `Gemfile` or `tina4 serve` fails with `LoadError: cannot load such file -- webrick`.
- `sqlite3` - the default `TINA4_DATABASE_URL` points to SQLite. Without this gem the server restarts in a loop with `LoadError: cannot load such file -- sqlite3`.

`tina4 init ruby` adds both gems automatically. You only need to add them by hand when bootstrapping an empty project. :::

Step 2: Create `app.rb`

This is the entry point. Create a file called `app.rb` in your project root:

```
require "tina4"
Tina4.initialize!(__dir__)
app = Tina4::RackApp.new
Tina4::WebServer.new(app, port: 7147).start
```

Four lines. `initialize!` sets up the project directory. `RackApp` builds the Rack application. `WebServer` starts it on the given port.

Step 3: Create the Folder Structure

Tina4 expects this layout:

```
my-project/
■■■ app.rb
■■■ Gemfile
■■■ .env
■■■ src/
    ■■■ routes/       # Route files go here
    ■■■ templates/    # Twig templates go here
    ■■■ public/       # Static files (CSS, JS, images)
```

Create the directories:

```
mkdir -p src/routes src/templates src/public
```

Step 4: Create `.env`

```
TINA4_DEBUG=true
```

Step 5: Run It

```
tina4 serve
```

The server starts on `http://localhost:7147`. You should see the Tina4 welcome page. From here, add route files in `src/routes/` and templates in `src/templates/`, the same way as a CLI-scaffolded project.

> ***Note:*** *Tina4 Ruby refuses to start without the Rust CLI. To bypass it (for example inside a Docker image that already wraps the framework) set* `TINA4_OVERRIDE_CLIENT=true` *in* `.env` *and run* `bundle exec ruby app.rb` *directly.*

---

9. Request & Response Fundamentals

Before jumping into the exercises, let's consolidate how route handlers work in Tina4 Ruby. Every handler receives two arguments: `request` (what the client sent) and `response` (what you send back). Here is the complete picture.

Reading Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them through `request.params`:

```
# URL: /api/search?q=laptop&page=2
request.params["q"]                  # "laptop"
request.params["page"]               # "2" (always a string)
request.params["sort"] || "name"     # "name" (default -- param was not sent)
```

Reading URL Path Parameters

Route patterns like `/users/{id}` capture segments of the URL. Access them through `request.params`:

```
Tina4::Router.get("/users/{id:int}/posts/{slug}") do |request, response|
  id = request.params["id"]      # 5 (int, because of :int)
```

```
  slug = request.params["slug"]  # "hello-world" (string)
  response.json({ user_id: id, slug: slug })
end
```

The `{id:int}` syntax tells Tina4 to convert the value to an integer. Without `:int`, it stays a string.

Reading the Request Body

POST, PUT, and PATCH requests carry a body. Tina4 parses JSON bodies into a hash automatically (as long as the client sends `Content-Type: application/json`):

```
Tina4::Router.post("/api/items") do |request, response|
  name = request.body["name"] || ""
  price = request.body["price"] || 0
  response.json({ received_name: name, received_price: price })
end
```

Reading Headers

Headers are available as a hash with their original casing:

```
content_type = request.headers["Content-Type"] || "not set"
auth_token = request.headers["Authorization"] || ""
custom = request.headers["X-Custom-Header"] || ""
```

Sending JSON Responses

`response.json` converts a hash to JSON and sets the correct `Content-Type`. Pass a status code as the second argument:

```
response.json({ id: 1, name: "Widget" })        # 200 OK (default)
response.json({ id: 1, name: "Widget" }, 201)    # 201 Created
response.json({ error: "Not found" }, 404)        # 404 Not Found
```

Sending HTML / Template Responses

`response.render` renders a Frond template from `src/templates/` and passes data to it:

```
response.render("products.html", { products: product_list, title: "Our Products" })
```

For raw HTML without a template file:

```
response.html("<h1>Hello</h1><p>This works too.</p>")
```

Status Codes

The most common status codes you will use:

Code	Meaning	When to Use
`200`	OK	Successful GET (default)
`201`	Created	Successful POST that created something
`400`	Bad Request	Client sent invalid input
`404`	Not Found	Resource does not exist

`500`	Internal Server Error	Something broke on the server

Worked Example: A Complete Route File

Here is a full route file that ties everything together. It builds a small book lookup API with query parameters, path parameters, JSON responses, and proper status codes. Read through it before attempting the exercises -- it is your reference.

Create `src/routes/books.rb`:

```
# In-memory data store
books = [
  { id: 1, title: "Dune", author: "Frank Herbert", year: 1965 },
  { id: 2, title: "Neuromancer", author: "William Gibson", year: 1984 },
  { id: 3, title: "Snow Crash", author: "Neal Stephenson", year: 1992 }
]

Tina4::Router.get("/api/books") do |request, response|
  # List all books. Supports ?author= filter and ?sort=year.
  author = request.params["author"] || ""
  sort_by = request.params["sort"] || ""

  result = books

  # Filter by author if the query param is present
  unless author.empty?
    result = result.select { |b| b[:author].downcase.include?(author.downcase) }
  end

  # Sort by year if requested
  if sort_by == "year"
    result = result.sort_by { |b| b[:year] }
  end

  response.json({ books: result, count: result.length })
end

Tina4::Router.get("/api/books/{id:int}") do |request, response|
  # Get a single book by ID. Returns 404 if not found.
  id = request.params["id"]
  book = books.find { |b| b[:id] == id }

  if book.nil?
    next response.json({ error: "Book with id #{id} not found" }, 404)
  end

  response.json(book)
end

Tina4::Router.post("/api/books") do |request, response|
  # Create a new book from the JSON body. Returns 201 on success.
  title = request.body["title"] || ""
  author = request.body["author"] || ""
  year = request.body["year"] || 0

  if title.empty? || author.empty?
```

```
    next response.json({ error: "title and author are required" }, 400)
  end

  new_book = {
    id: books.map { |b| b[:id] }.max + 1,
    title: title,
    author: author,
    year: year
  }
  books << new_book

  response.json(new_book, 201)
end.no_auth
```

::: tip `next` vs `return` inside route blocks Ruby raises `LocalJumpError: unexpected return` if you use `return` inside a block. Route handlers in Tina4 are blocks (`do ... end`), so use `next value` for early exits; it yields `value` as the block's result, just like `return` would inside a method. The final line of the block is its implicit return, so the terminal `response.json(...)` at the bottom needs no `next`. :::

Test it:

```
# List all books
curl http://localhost:7147/api/books

# Filter by author
curl "http://localhost:7147/api/books?author=gibson"

# Sort by year
curl "http://localhost:7147/api/books?sort=year"

# Get a single book
curl http://localhost:7147/api/books/2

# Get a book that does not exist (returns 404)
curl http://localhost:7147/api/books/99

# Create a new book
curl -X POST http://localhost:7147/api/books \
  -H "Content-Type: application/json" \
  -d '{"title": "Foundation", "author": "Isaac Asimov", "year": 1951}'
```

This example covers every building block the exercises use: reading query parameters, reading path parameters, reading the request body, returning JSON with different status codes, and handling missing data. Refer back to it as you work through the exercises below.

---

10. Exercise: Greeting API + Product List Template

Build two features from scratch. No peeking at the examples above.

Exercise Part A: Greeting API

Create an API endpoint at `GET /api/greet` that:

- Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)

- Defaults to `"Stranger"` if `name` is missing
- Returns JSON:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

- Calculates `time_of_day` from the server's current hour:
- 5:00 - 11:59 = "morning"
- 12:00 - 16:59 = "afternoon"
- 17:00 - 20:59 = "evening"
- 21:00 - 4:59 = "night"

Test:

```
curl "http://localhost:7147/api/greet?name=Sarah"
curl "http://localhost:7147/api/greet"
```

Exercise Part B: Product List Page

Create a page at `GET /store` that:

- Displays at least 5 products (hardcoded)
- Each product has: name, category, price, and a boolean `featured` flag
- Featured products get a visual highlight (different background, border, or badge)
- The page shows total product count and featured count
- Uses template inheritance -- a layout template and a page that extends it
- Includes `tina4.css` and `frond.js`

Product data for your route handler:

```
products = [
  { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
  { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
  { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
  { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured: true },
  { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
]
```

Expected browser output:

- A page titled "Our Store"
- "5 products, 3 featured"
- Product cards with name, category, price, and "Featured" badge on highlighted items
- Featured products wear a distinct visual style

---

11. Solutions

Solution A: Greeting API

Create `src/routes/greet.rb`:

```
Tina4::Router.get("/api/greet") do |request, response|
  name = request.params["name"] || "Stranger"
  hour = Time.now.hour

  time_of_day = if hour >= 5 && hour < 12
                  "morning"
                elsif hour >= 12 && hour < 17
                  "afternoon"
                elsif hour >= 17 && hour < 21
                  "evening"
                else
                  "night"
                end

  response.json({
    greeting: "Welcome, #{name}!",
    time_of_day: time_of_day
  })
end
```

Test:

```
curl "http://localhost:7147/api/greet?name=Sarah"
```

```
{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}
```

```
curl "http://localhost:7147/api/greet"
```

```
{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

Solution B: Product List Page

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; paddin
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px
    </style>
```

```
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
    </header>
    <div class="container">
        {% block content %}{% endblock %}
    </div>
    <script src="/js/frond.js"></script>
</body>
</html>
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.rb`:

```
Tina4::Router.get("/store") do |request, response|
  products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, featured: true },
    { name: "Yoga Mat", category: "Fitness", price: 29.99, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, featured: true },
    { name: "Noise-Canceling Headphones", category: "Electronics", price: 199.99, featured: true
    { name: "Water Bottle", category: "Fitness", price: 24.99, featured: false }
  ]

  featured_count = products.count { |p| p[:featured] }

  response.render("store.html", {
    products: products,
    featured_count: featured_count
  })
end
```

Open `http://localhost:7147/store`**.** You see:

- A dark header reading "Our Store"

- "5 products, 3 featured"
- Five product cards in a grid
- Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) wear a yellow border, light yellow background, and "Featured" badge
- Two cards (Yoga Mat, Water Bottle) wear a standard white background with gray border
- Each card shows name, category, and price formatted to 2 decimal places

---

12. Gotchas

1. File not auto-discovered

Problem: You created a route file but the URL returns nothing.

Cause: The file is not in `src/routes/`. It must live inside `src/routes/` (or a subdirectory), and must end with `.rb`.

Fix: Move the file to `src/routes/your-file.rb` and restart the server.

2. "Uninitialized constant" errors

Problem: `NameError: uninitialized constant Tina4::Router`.

Cause: The Tina4 gem is not loaded.

Fix: Confirm your `Gemfile` includes `gem "tina4"` and you ran `bundle install`. Route files in `src/routes/` auto-load -- no `require` statements needed.

3. JSON response shows HTML

Problem: Your JSON endpoint returns HTML.

Cause: You returned a string instead of calling `response.json`. Plain strings become HTML in Tina4.

Fix: Use `response.json(data)` for JSON endpoints. Not `puts data.to_json`.

4. Template not found

Problem: `Template "my-page.html" not found`.

Cause: The template file is not in `src/templates/`, or the filename has a typo.

Fix: Check that the file exists at `src/templates/my-page.html`. The name in `response.render` is relative to `src/templates/`.

5. Port already in use

Problem: `Error: Address already in use (port 7147)`.

Cause: Another process owns port 7147.

Fix: Stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or: `tina4 serve --port 8080`.

6. Changes not reflected

Problem: You edited a file but the browser shows the old version.

Cause: Live reload may not be active. Browser caching can serve stale versions.

Fix: Hard-refresh (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that fails, restart with `Ctrl+C` and `tina4 serve`.

7. .env not loaded

Problem: Environment variables have no effect.

Cause: The `.env` file must sit at the project root (same directory as `Gemfile`).

Fix: Move `.env` to the project root.

8. Debug mode in production

Problem: Production shows stack traces and query details.

Cause: `TINA4_DEBUG=true` in production.

Fix: Set `TINA4_DEBUG=false`. This hides debug information, enables HTML minification, and activates `.broken` file health checks.

Routing

1. How Routing Works in Tina4

Every web application maps URLs to code. You type `/products` in your browser. The framework finds the handler for `/products`, runs it, sends back the result. That mapping is routing.

In Tina4, routes live in Ruby files inside `src/routes/`. Every `.rb` file in that directory (and its subdirectories) is auto-loaded at startup. No registration. No central config. Drop a file in. It works.

The simplest possible route:

```
Tina4::Router.get("/hello") do |request, response|
  response.json({ message: "Hello, World!" })
end
```

Save that as `src/routes/hello.rb`, start the server, visit `http://localhost:7147/hello`:

```
{"message":"Hello, World!"}
```

One line registers the route. One block handles the request. Done.

---

2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each one lives on `Tina4::Router`:

```
Tina4::Router.get("/products") do |request, response|
  response.json({ action: "list all products" })
end

Tina4::Router.post("/products") do |request, response|
  response.json({ action: "create a product" }, 201)
end

Tina4::Router.put("/products/{id}") do |request, response|
  id = request.params[:id]
  response.json({ action: "replace product #{id}" })
end

Tina4::Router.patch("/products/{id}") do |request, response|
  id = request.params[:id]
  response.json({ action: "update product #{id}" })
end

Tina4::Router.delete("/products/{id}") do |request, response|
  id = request.params[:id]
  response.json({ action: "delete product #{id}" })
end
```

Test each one:

```
curl http://localhost:7147/products

{"action":"list all products"}
```

```
curl -X POST http://localhost:7147/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'

{"action":"create a product"}

curl -X PUT http://localhost:7147/products/42

{"action":"replace product 42"}

curl -X PATCH http://localhost:7147/products/42

{"action":"update product 42"}

curl -X DELETE http://localhost:7147/products/42

{"action":"delete product 42"}
```

`GET` reads. `POST` creates. `PUT` replaces. `PATCH` patches. `DELETE` removes. REST convention. Predictable API.

HEAD and OPTIONS: automatic, no boilerplate

Tina4 handles `HEAD` and `OPTIONS` for you. **You don't register them.** They follow from your `Tina4::Router.get(...)` / `Tina4::Router.post(...)` / etc. routes:

- `HEAD` is identical to `GET` except the response carries no body (RFC 9110 §9.3.2). Every `GET` route automatically responds to `HEAD`: the framework strips the response body on the way out and preserves `Content-Length` pointing at the byte count the equivalent `GET` would have sent. Cache validators, link checkers, and uptime probes work without you writing anything.
- `OPTIONS` on any registered path returns `204 No Content` with an `Allow:` header listing every method the path supports (RFC 9110 §9.3.7).
- **Wrong method on an existing path** returns `405 Method Not Allowed` with the same `Allow:` header (RFC 9110 §15.5.6 + §10.2.1). Sending `PUT` to a `GET`-only route used to return `404`, which was semantically wrong and confusing for link checkers. Now you get a real `405`.
- `TRACE` **and** `CONNECT` are rejected with `405` (security default for origin servers).

Note: real CORS preflight (requests with `Origin` + `Access-Control-Request-Method`) is still handled by the CORS middleware as before. Plain RFC 9110 `OPTIONS` introspection hits the router's `Allow`-header response.

```
# HEAD on a GET route - same headers, empty body
curl -sI http://localhost:7147/products
# HTTP/1.1 200 OK
# Content-Type: application/json
# Content-Length: 33

# OPTIONS - discover what the path supports
curl -sI -X OPTIONS http://localhost:7147/products
# HTTP/1.1 204 No Content
# Allow: GET, POST, HEAD, OPTIONS

# Wrong method - clear 405 with Allow header
curl -sI -X PUT http://localhost:7147/products
# HTTP/1.1 405 Method Not Allowed
# Allow: GET, POST, HEAD, OPTIONS
```

Explicit `Tina4::Router.head` and `Tina4::Router.options` registration

The automatic behaviour is enough for 95% of apps. When you need custom HEAD or OPTIONS handlers, register them explicitly:

```
# HEAD handler that doesn't run the full GET body - useful for
# expensive endpoints where the client only needs to check existence
# or read validators (ETag, Last-Modified).
Tina4::Router.head("/expensive/{id}") do |request, response|
  response.header("ETag", compute_etag(request.params[:id]))
  response.call(nil, 200)
end

# OPTIONS handler that returns more than just Allow - for example
# a discovery endpoint.
Tina4::Router.options("/api/discovery") do |_req, response|
  response.json({ version: "v1", endpoints: [] })
end
```

The framework still strips the response body for `HEAD` handlers (RFC 9110 §9.3.2 is a MUST), so you can't accidentally leak content even if your handler returns a body.

---

3. Path Parameters

Path parameters capture values from the URL. Wrap the name in curly braces:

```
Tina4::Router.get("/users/{id}/posts/{post_id}") do |request, response|
  user_id = request.params[:id]
  post_id = request.params[:post_id]

  response.json({
    user_id: user_id,
    post_id: post_id
  })
end
```

```
curl http://localhost:7147/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice: `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are strings by default.

> ***Auto-casting:*** *Tina4 automatically casts path parameter values that are purely numeric to integers. For example, requesting `/users/42/posts/99` will give you `request.params[:id]` as the integer `42` and `request.params[:post_id]` as the integer `99` -- no explicit `:int` type hint required. The `:int` type hint adds validation (rejecting non-numeric values with a 404), but the auto-casting happens regardless.*

Typed Parameters

Enforce a type with a colon after the parameter name:

```
Tina4::Router.get("/orders/{id:int}") do |request, response|
  id = request.params[:id]
  response.json({
```

```
    order_id: id,
    type: id.class.name
  })
end
```

```
curl http://localhost:7147/orders/42
```

```
{"order_id":42,"type":"Integer"}
```

Pass a non-integer? The route refuses to match. You get a 404:

```
curl http://localhost:7147/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

Type	Matches	Auto-cast	Example
`int`	Digits only	Integer	`{id:int}` matches `42` but not `abc`
`float`	Decimal numbers	Float	`{price:float}` matches `19.99`
`path`	All remaining path segments (catch-all)	String	`{slug:path}` matches `docs/api/auth`

The `{name}` form (no type) matches any single path segment and returns it as a string.

Typed Parameters in Action

Here is a complete example showing the most commonly used typed parameters together:

```
# Integer parameter -- only digits match, auto-cast to Integer
Tina4::Router.get("/products/{id:int}") do |request, response|
  id = request.params[:id] # Integer, e.g. 42
  response.json({
    product_id: id,
    type: id.class.name
  })
end

# Float parameter -- decimal numbers, auto-cast to Float
Tina4::Router.get("/products/{id:int}/price/{price:float}") do |request, response|
  id = request.params[:id]
  price = request.params[:price]
  response.json({
    product_id: id,
    price: price,
    type: price.class.name
  })
end

# Path parameter -- catch-all, captures remaining segments as a string
Tina4::Router.get("/files/{filepath:path}") do |request, response|
  filepath = request.params[:filepath]
  # filepath could be "images/photos/cat.jpg"
```

```
  response.json({
    filepath: filepath,
    type: filepath.class.name
  })
end

# Integer route -- matches digits, returns an Integer
curl http://localhost:7147/products/42

{"product_id":42,"type":"Integer"}

# Integer route -- non-integer gives a 404
curl http://localhost:7147/products/abc

{"error":"Not found","path":"/products/abc","status":404}

# Path catch-all -- captures everything after /files/
curl http://localhost:7147/files/images/photos/cat.jpg

{"filepath":"images/photos/cat.jpg","type":"String"}
```

The `:int` and `:float` types act as both a constraint and a converter. If the URL segment does not match the expected pattern, the route is skipped entirely and Tina4 moves on to the next registered route (or returns 404 if nothing matches). The `:path` type is greedy -- it consumes all remaining segments, making it ideal for file paths and documentation URLs.

---

4. Query Parameters

Query parameters are key-value pairs after the `?` in a URL. Access them through `request.params`:

```
Tina4::Router.get("/search") do |request, response|
  q = request.params["q"] || ""
  page = (request.params["page"] || 1).to_i
  limit = (request.params["limit"] || 10).to_i

  response.json({
    query: q,
    page: page,
    limit: limit,
    offset: (page - 1) * limit
  })
end

curl "http://localhost:7147/search?q=keyboard&page=2&limit=20"

{"query":"keyboard","page":2,"limit":20,"offset":20}
```

Missing query parameter? `request.params["key"]` returns `nil`. Use `||` to provide defaults.

---

5. Route Groups

A set of routes shares a common prefix. `Tina4::Router.group` eliminates repetition:

```
Tina4::Router.group("/api/v1") do
```

```
  Tina4::Router.get("/users") do |request, response|
    response.json({ users: [] })
  end

  Tina4::Router.get("/users/{id:int}") do |request, response|
    id = request.params[:id]
    response.json({ user: { id: id, name: "Alice" } })
  end

  Tina4::Router.post("/users") do |request, response|
    response.json({ created: true }, 201)
  end

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [] })
  end

end
```

These routes register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. Short paths inside the group. Tina4 prepends the prefix.

```
curl http://localhost:7147/api/v1/users
```

```
{"users":[]}
```

```
curl http://localhost:7147/api/v1/products
```

```
{"products":[]}
```

Groups nest:

```
Tina4::Router.group("/api") do
  Tina4::Router.group("/v1") do
    Tina4::Router.get("/status") do |request, response|
      response.json({ version: "1.0" })
    end
  end

  Tina4::Router.group("/v2") do
    Tina4::Router.get("/status") do |request, response|
      response.json({ version: "2.0" })
    end
  end
end
```

```
curl http://localhost:7147/api/v1/status
```

```
{"version":"1.0"}
```

```
curl http://localhost:7147/api/v2/status
```

```
{"version":"2.0"}
```

---

6. Middleware

Middleware is code that runs before or after your route handler. Authentication. Logging. Rate limiting. Input validation. Anything that belongs on multiple routes but not in every handler.

Middleware on a Single Route

Pass middleware as the third argument:

```
log_request = lambda do |request, response, next_handler|
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)
  $stderr.puts "[#{Time.now.strftime('%Y-%m-%d %H:%M:%S')}] #{request.method} #{request.path}"

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
  $stderr.puts "  Completed in #{duration}ms"

  result
end

Tina4::Router.get("/api/data", middleware: "log_request") do |request, response|
  response.json({ data: [1, 2, 3] })
end
```

The middleware receives `request`, `response`, and `next_handler`. Call `next_handler.call(request, response)` to proceed to the route handler. Skip the call and the handler never runs -- a gatekeeper that blocks unauthorized requests.

Blocking Middleware

Middleware that checks for an API key:

```
require_api_key = lambda do |request, response, next_handler|
  api_key = request.headers["X-API-Key"] || ""

  if api_key != "my-secret-key"
    return response.json({ error: "Invalid API key" }, 401)
  end

  next_handler.call(request, response)
end

Tina4::Router.get("/api/secret", middleware: "require_api_key") do |request, response|
  response.json({ secret: "The answer is 42" })
end
```

```
curl http://localhost:7147/api/secret
```

```
{"error":"Invalid API key"}
```

Status: `401 Unauthorized`.

```
curl http://localhost:7147/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

Middleware on a Group

Apply middleware to every route inside a group:

```
Tina4::Router.group("/api/admin", middleware: "require_auth") do

  Tina4::Router.get("/dashboard") do |request, response|
    response.json({ page: "admin dashboard" })
  end
```

```
  Tina4::Router.get("/users") do |request, response|
    response.json({ page: "user management" })
  end

end
```

Every route in the group now demands the `Authorization` header. No per-route repetition.

Multiple Middleware

Chain them with an array:

```
Tina4::Router.get("/api/important", middleware: ["log_request", "require_api_key", "require_auth
  response.json({ data: "important stuff" })
end
```

Middleware runs in order: `log_request` first, then `require_api_key`, then `require_auth`, then the route handler. If any middleware skips `next_handler`, the chain stops there.

---

7. Route Decorators: @noauth and @secured

Tina4 provides two decorators for controlling authentication at the route level.

@noauth -- Public Routes

When your application has global authentication middleware, `@noauth` marks specific routes as public:

```
# @noauth
Tina4::Router.get("/api/public/info") do |request, response|
  response.json({
    app: "My Store",
    version: "1.0.0"
  })
end
```

The `@noauth` comment tells Tina4 to skip authentication for this route, even if global auth middleware guards the parent group.

@secured -- Protected GET Routes

`@secured` marks a GET route as requiring authentication:

```
# @secured
Tina4::Router.get("/api/profile") do |request, response|
  # request.user is populated by the auth middleware
  response.json({ user: request.user })
end
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are secured. `GET` routes are public unless you add `@secured`. Reading is open. Writing demands credentials.

---

8. Route Chaining: .secure and .cache

Routes return a chainable object. Two methods you can call on any route: `.secure` and `.cache`.

.secure

`.secure` requires a valid bearer token in the `Authorization` header. If the token is missing or invalid, the route returns `401 Unauthorized` without ever reaching your handler:

```
Tina4::Router.get("/api/account") do |request, response|
  response.json({ account: request.user })
end.secure
```

```
curl http://localhost:7147/api/account
# 401 Unauthorized

curl http://localhost:7147/api/account -H "Authorization: Bearer eyJhbGci..."
# 200 OK
```

.cache

`.cache` enables response caching for the route. Once the handler runs and produces a response, subsequent requests to the same URL return the cached result without re-executing the handler:

```
Tina4::Router.get("/api/catalog") do |request, response|
  # Expensive database query
  response.json({ products: products })
end.cache
```

Chaining Both

Chain `.secure` and `.cache` together:

```
Tina4::Router.get("/api/data") do |request, response|
  response.json({ data: data })
end.secure.cache
```

This route requires a bearer token and caches the response. Order does not matter -- `.cache.secure` produces the same result.

---

9. Wildcard and Catch-All Routes

Wildcard Routes

Use `*` at the end of a path to match everything after it:

```
Tina4::Router.get("/docs/*") do |request, response|
  path = request.params["*"] || ""
  response.json({
    section: "docs",
    path: path
  })
end
```

```
curl http://localhost:7147/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}

curl http://localhost:7147/docs/api/authentication/jwt

{"section":"docs","path":"api/authentication/jwt"}
```

Catch-All Route (Custom 404)

Handle any unmatched URL:

```
Tina4::Router.get("/*") do |request, response|
  response.json({
    error: "Page not found",
    path: request.path
  }, 404)
end
```

Define this route last (or in a file that sorts alphabetically after your other route files). Tina4 matches routes in registration order. First match wins.

Or create a custom 404 page at `src/templates/errors/404.html`:

```
{% extends "base.html" %}

{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 uses this template for unmatched routes when the file exists.

---

10. Route Listing via CLI

As your application grows, you need to see all registered routes at a glance:

```
tina4 routes

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/dashboard          require_auth        public
GET      /api/admin/users              require_auth        public
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
GET      /docs/*                       -                   public
```

Filter by method:

```
tina4 routes --method POST
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
POST     /products                     -                   secured
POST     /api/v1/users                 -                   secured
```

Search for a path pattern:

```
tina4 routes --filter users
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/users              require_auth        public
```

---

11. Organizing Route Files

Organize route files however you want. Tina4 loads every `.rb` file in `src/routes/` recursively. Two common patterns:

Pattern 1: One File Per Resource

```
src/routes/
■■■ products.rb     # All product routes
■■■ users.rb        # All user routes
■■■ orders.rb       # All order routes
■■■ pages.rb        # HTML page routes
```

Pattern 2: Subdirectories by Feature

```
src/routes/
■■■ api/
■   ■■■ products.rb
■   ■■■ users.rb
■   ■■■ orders.rb
■■■ admin/
■   ■■■ dashboard.rb
■   ■■■ settings.rb
■■■ pages/
    ■■■ home.rb
    ■■■ about.rb
```

Both work the same. The directory structure has no effect on URL paths -- only the route definitions inside the files matter. Pick whichever pattern keeps your project navigable.

---

12. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. Data lives in a Ruby array (no database yet -- Chapter 5 handles that).

Requirements

Create `src/routes/product_api.rb` with these routes:

Method	Path	Description
`GET`	`/api/products`	List all products. Support `?category=` filter.
`GET`	`/api/products/{id:int}`	Get a single product by ID. Return 404 if not found.
`POST`	`/api/products`	Create a new product. Return 201.
`PUT`	`/api/products/{id:int}`	Replace a product. Return 404 if not found.
`DELETE`	`/api/products/{id:int}`	Delete a product. Return 204 with no body.

Each product has: `id` (int), `name` (string), `category` (string), `price` (float), `in_stock` (bool).

Seed data:

```
products = [
  { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, in_stock: true },
  { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true },
  { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, in_stock: false },
  { id: 4, name: "Standing Desk", category: "Office", price: 549.99, in_stock: true },
  { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true }
]
```

Test with:

```
# List all
curl http://localhost:7147/api/products

# Filter by category
curl "http://localhost:7147/api/products?category=Fitness"

# Get one
curl http://localhost:7147/api/products/3

# Create
curl -X POST http://localhost:7147/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "in_stock": true}'

# Update
curl -X PUT http://localhost:7147/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "in_stock": true}'

# Delete
curl -X DELETE http://localhost:7147/api/products/3

# Not found
```

```
curl http://localhost:7147/api/products/999
```

---

13. Solution

Create `src/routes/product_api.rb`:

```
# In-memory product store (resets on server restart)
$products = [
  { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, in_stock: true },
  { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true },
  { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, in_stock: false },
  { id: 4, name: "Standing Desk", category: "Office", price: 549.99, in_stock: true },
  { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true }
]

$next_id = 6

# List all products, optionally filter by category
Tina4::Router.get("/api/products") do |request, response|
  category = request.params["category"]

  if category
    filtered = $products.select { |p| p[:category].downcase == category.downcase }
    response.json({ products: filtered, count: filtered.length })
  else
    response.json({ products: $products, count: $products.length })
  end
end

# Get a single product by ID
Tina4::Router.get("/api/products/{id:int}") do |request, response|
  id = request.params[:id]

  product = $products.find { |p| p[:id] == id }

  if product
    response.json(product)
  else
    response.json({ error: "Product not found", id: id }, 404)
  end
end

# Create a new product
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  if body["name"].nil? || body["name"].empty?
    return response.json({ error: "Name is required" }, 400)
  end

  product = {
    id: $next_id,
    name: body["name"],
    category: body["category"] || "Uncategorized",
    price: (body["price"] || 0).to_f,
    in_stock: body["in_stock"] != false
  }
```

```
  $next_id += 1
  $products << product

  response.json(product, 201)
end

# Replace a product
Tina4::Router.put("/api/products/{id:int}") do |request, response|
  id = request.params[:id]
  body = request.body

  index = $products.index { |p| p[:id] == id }

  if index.nil?
    response.json({ error: "Product not found", id: id }, 404)
  else
    $products[index] = {
      id: id,
      name: body["name"] || $products[index][:name],
      category: body["category"] || $products[index][:category],
      price: (body["price"] || $products[index][:price]).to_f,
      in_stock: body.key?("in_stock") ? body["in_stock"] : $products[index][:in_stock]
    }
    response.json($products[index])
  end
end

# Delete a product
Tina4::Router.delete("/api/products/{id:int}") do |request, response|
  id = request.params[:id]

  index = $products.index { |p| p[:id] == id }

  if index.nil?
    response.json({ error: "Product not found", id: id }, 404)
  else
    $products.delete_at(index)
    response.json(nil, 204)
  end
end
```

Expected output for the test commands:

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"in_stock
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"in_stock":true},{"id"
```

Get one:

```
{"id":3,"name":"Coffee Grinder","category":"Kitchen","price":49.99,"in_stock":false}
```

Create:

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"in_stock":true}
```

(Status: `201 Created`)

Update:

```
{"id":3,"name":"Burr Coffee Grinder","category":"Kitchen","price":59.99,"in_stock":true}
```

Delete: empty response with status `204 No Content`.

Not found:

```
{"error":"Product not found","id":999}
```

(Status: `404 Not Found`)

---

14. Gotchas

1. Trailing Slashes Matter

Problem: `/products` works but `/products/` returns 404.

Cause: Tina4 treats `/products` and `/products/` as different routes by default.

Fix: Pick one convention. Stick with it. Set `TINA4_TRAILING_SLASH_REDIRECT=true` in `.env` and Tina4 redirects `/products/` to `/products`.

2. Parameter Names Must Be Unique in a Path

Problem: `/users/{id}/posts/{id}` behaves wrong -- both parameters share the same name.

Cause: The second `{id}` overwrites the first in `request.params`.

Fix: Use distinct names: `/users/{user_id}/posts/{post_id}`.

3. Method Conflicts

Problem: You defined `Tina4::Router.get("/items/{id}", ...)` and `Tina4::Router.get("/items/{action}", ...)` and the wrong handler runs.

Cause: Both patterns match `/items/42`. First registered wins.

Fix: Use typed parameters: `Tina4::Router.get("/items/{id:int}", ...)` matches integers only, leaving `/items/export` free. Or restructure: `/items/{id:int}` and `/items/actions/{action}`.

4. Route Handler Must Return a Response

Problem: Handler runs but the browser shows an empty page or 500 error.

Cause: No call to `response.json`, `response.render`, or another response method. Without a return value from the response object, Tina4 has nothing to send.

Fix: Every handler must end with `response.json(...)`, `response.html(...)`, or `response.render(...)`.

5. Block Syntax Matters

Problem: Route handler raises a syntax error about unexpected blocks.

Cause: Ruby blocks with `do...end` and `{...}` have different precedence.

Fix: Use `do |request, response| ... end` for route blocks. The curly brace form works for single-line handlers but causes parsing issues with method arguments.

6. Middleware Must Be a Named Function or String

Problem: Passing an inline lambda as middleware causes unexpected behavior.

Cause: Tina4 expects middleware referenced by name (a string), resolved at runtime.

Fix: Define middleware as a named method or lambda. Pass the name as a string: `"my_middleware"`.

7. Group Prefix Must Start with a Slash

Problem: `Tina4::Router.group("api/v1")` produces routes that do not match.

Cause: The group prefix needs a leading `/`.

Fix: Start group prefixes with `/`: `Tina4::Router.group("/api/v1")`.

Request & Response

1. The Two Objects You Use Everywhere

Every route handler receives two arguments: `request` and `response`. The request carries what the client sent. The response builds what you send back. These two objects handle every HTTP scenario you will face.

A file-sharing service shows the pattern. A user uploads a document. You validate the file type. You check the session. You return success JSON or an error page. Every step flows through `request` and `response`.

```
Tina4::Router.get("/echo") do |request, response|
  response.json({
    method: request.method,
    path: request.path,
    your_ip: request.ip
  })
end
```

```
curl http://localhost:7147/echo
```

```
{"method":"GET","path":"/echo","your_ip":"127.0.0.1"}
```

The pattern holds for every route: inspect the request, build the response, return it.

---

2. The Request Object

The `request` object opens every piece of the incoming HTTP request. Headers, body, cookies, files, IP address -- all accessible through named properties.

method

The HTTP method arrives as an uppercase string:

```
Tina4::Router.get("/api/check") do |request, response|
  response.json({ method: request.method })
end
```

```
curl http://localhost:7147/api/check
```

```
{"method":"GET"}
```

path

The URL path strips query parameters:

```
# Request to /api/users?page=2
request.path # "/api/users"
```

params

Path parameters from the URL pattern (see Chapter 2). The parameter names match the `{name}` placeholders in the route:

```
# Route: /users/{id}/posts/{post_id}
# Request: /users/5/posts/42
request.params["id"]      # "5" (or 5 if typed as {id:int})
request.params["post_id"] # "42"
```

query

Query string parameters live in the same `params` hash:

```
# Request: /search?q=keyboard&page=2&sort=price
request.params["q"]    # "keyboard"
request.params["page"] # "2"
request.params["sort"] # "price"
```

body

The parsed request body. JSON requests become a hash. Form submissions contain the form fields. GET requests produce `nil`:

```
# POST with {"name": "Widget", "price": 9.99}
request.body["name"]  # "Widget"
request.body["price"] # 9.99
```

headers

Request headers as a hash. Header names keep their original casing:

```
request.headers["Content-Type"]  # "application/json"
request.headers["Authorization"] # "Bearer eyJhbGci..."
request.headers["X-Custom"]      # "my-value"
```

ip

The client's IP address. Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers behind a reverse proxy:

```
request.ip # "127.0.0.1"
```

cookies

A hash of cookies the client sent:

```
request.cookies["session_id"]  # "abc123"
request.cookies["preferences"] # "dark-mode"
```

files

Uploaded files (section 7 covers the details):

```
request.files["avatar"] # hash: file["filename"], file["type"], file["size"], file["content"]
```

Inspecting the Full Request

A route that dumps everything:

```
Tina4::Router.post("/debug/request") do |request, response|
  response.json({
    method: request.method,
    path: request.path,
    params: request.params,
    query: request.params,
```

```
    body: request.body,
    headers: request.headers,
    ip: request.ip,
    cookies: request.cookies
  })
end
```

```
curl -X POST "http://localhost:7147/debug/request?page=1" \
  -H "Content-Type: application/json" \
  -H "X-Custom: hello" \
  -d '{"name": "test"}'
```

```
{
  "method": "POST",
  "path": "/debug/request",
  "params": {},
  "query": {"page": "1"},
  "body": {"name": "test"},
  "headers": {
    "Content-Type": "application/json",
    "X-Custom": "hello",
    "Host": "localhost:7147",
    "User-Agent": "curl/8.4.0",
    "Accept": "*/*",
    "Content-Length": "16"
  },
  "ip": "127.0.0.1",
  "cookies": {}
}
```

---

3. The Response Object

The `response` object builds HTTP responses. Every route handler must return one. Methods chain, so you can set headers, cookies, and content in a single expression.

json -- JSON Response

The workhorse for APIs. Pass any hash or value and it becomes JSON:

```
response.json({ name: "Alice", age: 30 })
```

```
{"name":"Alice","age":30}
```

Pass a status code as the second argument:

```
response.json({ id: 7, name: "Widget" }, 201)
```

Returns `201 Created` with the JSON body.

html -- Raw HTML Response

Return an HTML string:

```
response.html("<h1>Hello</h1><p>This is HTML.</p>")
```

Sets `Content-Type: text/html; charset=utf-8`.

text -- Plain Text Response

Return plain text:

```
response.text("Just a plain string.")
```

Sets `Content-Type: text/plain; charset=utf-8`.

render -- Template Response

Render a Frond template with data (Chapter 4: Templates goes deep):

```
response.render("products.html", {
  products: products,
  title: "Our Products"
})
```

Tina4 finds the template in `src/templates/`, renders it, returns the HTML.

redirect -- Redirect Response

Send the client elsewhere:

```
response.redirect("/login")
```

Sends a `302 Found` by default. Pass a different status for permanent redirects:

```
response.redirect("/new-location", 301)
```

file -- File Download Response

Send a file for download:

```
response.file("/path/to/report.pdf")
```

Tina4 sets the right `Content-Type` from the file extension and adds `Content-Disposition` so the browser downloads instead of displaying.

Custom filename:

```
response.file("/path/to/report.pdf", "monthly-report-march-2026.pdf")
```

This sets the `Content-Disposition: attachment` header with your chosen filename.

xml -- XML Response

Return XML content:

```
Tina4::Router.get("/api/feed") do |request, response|
  response.xml("<feed><entry><title>Hello</title></entry></feed>")
end
```

Sets `Content-Type: application/xml; charset=utf-8`.

error -- Error Envelope

Return a structured error response:

```
Tina4::Router.post("/api/things") do |request, response|
  response.error("VALIDATION_FAILED", "Name is required", 400)
end
```

This produces:

```
{"error":true,"code":"VALIDATION_FAILED","message":"Name is required","status":400}
```

Three arguments: an error code string, a human-readable message, and the HTTP status code. Clients check `error: true` and switch on the `code` field. Use this pattern across your API for consistent error handling.

---

4. Status Codes

Every response method accepts a status code. The ones you will use most:

Code	Meaning	When to Use
`200`	OK	Default. Successful GET, PUT, PATCH.
`201`	Created	Successful POST that created a resource.
`204`	No Content	Successful DELETE. No body needed.
`301`	Moved Permanently	URL has permanently changed.
`302`	Found	Temporary redirect.
`400`	Bad Request	Invalid input from the client.
`401`	Unauthorized	Missing or invalid authentication.
`403`	Forbidden	Authenticated but not allowed.
`404`	Not Found	Resource does not exist.
`409`	Conflict	Duplicate or conflicting data. Two users claim the same username.
`413`	Payload Too Large	Body exceeds `TINA4_MAX_UPLOAD_SIZE`.
`422`	Unprocessable Entity	Valid JSON but fails business rules. The data parses but the logic rejects it.
`500`	Internal Server Error	Something broke on the server.

Set the status with chaining:

```
response.status(201).json({ id: 7, created: true })
```

Equivalent to `response.json({ id: 7, created: true }, 201)`. Some developers prefer the chained form.

---

5. Custom Headers

Add custom headers with the `header` method. Chain calls before the final response:

```
Tina4::Router.get("/api/data") do |request, response|
  response
    .header("X-Request-Id", SecureRandom.hex(8))
    .header("X-Rate-Limit-Remaining", "57")
    .header("Cache-Control", "no-cache")
    .json({ data: [1, 2, 3] })
end
```

```
curl -v http://localhost:7147/api/data 2>&1 | grep "< X-"
```

```
< X-Request-Id: 65f3a7b8c1234567
< X-Rate-Limit-Remaining: 57
```

Convention: `Title-Case` for custom headers, prefixed with `X-`.

CORS Headers

Tina4 handles CORS based on the `TINA4_CORS_ORIGINS` setting in `.env`. By default it is empty, which denies every cross-origin request. List the origins your front end is served from:

```
TINA4_CORS_ORIGINS=https://myapp.com,https://admin.myapp.com
```

Manual override when needed:

```
response
  .header("Access-Control-Allow-Origin", "https://myapp.com")
  .header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE")
  .header("Access-Control-Allow-Headers", "Content-Type, Authorization")
  .json({ data: "value" })
```

---

6. Cookies

Set cookies on the response:

```
Tina4::Router.post("/login") do |request, response|
  # After validating credentials...
  response
    .cookie("session_id", "abc123xyz", {
      http_only: true,
      secure: true,
      same_site: "Strict",
      max_age: 3600,       # 1 hour in seconds
      path: "/"
    })
```

```
    .json({ message: "Logged in" })
end
```

Cookie options:

Option	Type	Default	Description
`http_only`	bool	`false`	JavaScript cannot access it
`secure`	bool	`false`	Only sent over HTTPS
`same_site`	string	`"Lax"`	`"Strict"`, `"Lax"`, or `"None"`
`max_age`	int	session	Lifetime in seconds
`path`	string	`"/"`	URL path scope
`domain`	string	current	Domain scope

Read cookies from the request:

```
Tina4::Router.get("/profile") do |request, response|
  session_id = request.cookies["session_id"]

  if session_id.nil?
    return response.json({ error: "Not logged in" }, 401)
  end

  response.json({ session: session_id })
end
```

Delete a cookie by setting `max_age` to `0`:

```
response
  .cookie("session_id", "", { max_age: 0, path: "/" })
  .json({ message: "Logged out" })
```

---

7. File Uploads

Uploaded files live in `request.files`, keyed by field name. Each file is a hash: `file["filename"]`, `file["type"]`, `file["size"]` (bytes) and `file["content"]` (the raw bytes). String and symbol keys both work.

Handling a Single File Upload

```
Tina4::Router.post("/api/upload") do |request, response|
  if request.files["image"].nil?
    next response.json({ error: "No file uploaded" }, 400)
  end

  file = request.files["image"]

  response.json({
    filename: file["filename"], # "photo.jpg"
    type: file["type"],         # "image/jpeg"
    size: file["size"]          # 245760 (bytes)
  })
end
```

Test with curl:

```
curl -X POST http://localhost:7147/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "filename": "photo.jpg",
  "type": "image/jpeg",
  "size": 245760
}
```

Saving the Uploaded File

Write the bytes to a permanent path:

```
Tina4::Router.post("/api/upload") do |request, response|
  if request.files["image"].nil?
    next response.json({ error: "No file uploaded" }, 400)
  end

  file = request.files["image"]

  # Validate file type
  allowed_types = ["image/jpeg", "image/png", "image/gif", "image/webp"]
  unless allowed_types.include?(file["type"])
    next response.json({ error: "Invalid file type. Allowed: JPEG, PNG, GIF, WebP" }, 400)
  end

  # Validate file size (max 5MB)
  max_size = 5 * 1024 * 1024
  if file["size"] > max_size
    next response.json({ error: "File too large. Maximum size: 5MB" }, 400)
  end

  # Generate a unique filename
  extension = File.extname(file["filename"])
  filename = "img_#{SecureRandom.hex(8)}#{extension}"
  upload_dir = File.join(__dir__, "../public/uploads")
  destination = File.join(upload_dir, filename)

  # Ensure the uploads directory exists
  FileUtils.mkdir_p(upload_dir) unless Dir.exist?(upload_dir)

  # Write the file
```

```
  File.binwrite(destination, file["content"])

  response.json({
    message: "File uploaded successfully",
    filename: filename,
    url: "/uploads/#{filename}",
    size: file["size"]
  }, 201)
end
```

```
curl -X POST http://localhost:7147/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "message": "File uploaded successfully",
  "filename": "img_a1b2c3d4e5f6a7b8.jpg",
  "url": "/uploads/img_a1b2c3d4e5f6a7b8.jpg",
  "size": 245760
}
```

The file now lives at `http://localhost:7147/uploads/img_a1b2c3d4e5f6a7b8.jpg`.

Handling Multiple Files

When the HTML form uses `multiple` or you have multiple file inputs:

```
Tina4::Router.post("/api/upload-many") do |request, response|
  results = []

  request.files.each do |key, file|
    extension = File.extname(file["filename"])
    filename = "file_#{SecureRandom.hex(8)}#{extension}"
    upload_dir = File.join(__dir__, "../public/uploads")
    destination = File.join(upload_dir, filename)

    FileUtils.mkdir_p(upload_dir) unless Dir.exist?(upload_dir)
    File.binwrite(destination, file["content"])

    results << {
      original_name: file["filename"],
      saved_as: filename,
      url: "/uploads/#{filename}"
    }
  end

  response.json({ uploaded: results, count: results.length }, 201)
end
```

---

8. File Downloads

Send files with `response.file`:

```
Tina4::Router.get("/api/reports/{filename}") do |request, response|
  filename = request.params["filename"]
  filepath = File.join(__dir__, "../../data/reports", filename)

  unless File.exist?(filepath)
    return response.json({ error: "Report not found" }, 404)
```

```
  end

  response.file(filepath)
end
```

The browser downloads the file. Tina4 detects the MIME type from the extension and sets the right headers.

Force a specific download filename:

```
response.file(filepath, "Q1-2026-Sales-Report.pdf")
```

---

9. Content Negotiation

The same endpoint can return different formats based on what the client asks for. Check the `Accept` header:

```
Tina4::Router.get("/api/products/{id:int}") do |request, response|
  id = request.params["id"]
  product = {
    id: id,
    name: "Wireless Keyboard",
    price: 79.99
  }

  accept = request.headers["Accept"] || "application/json"

  if accept.include?("text/html")
    response.render("product-detail.html", { product: product })
  elsif accept.include?("text/plain")
    text = "Product ##{id}: #{product[:name]} - $#{product[:price]}"
    response.text(text)
  else
    # Default to JSON
    response.json(product)
  end
end
```

```
# JSON (default)
curl http://localhost:7147/api/products/1
```

```
{"id":1,"name":"Wireless Keyboard","price":79.99}
```

```
# Plain text
curl http://localhost:7147/api/products/1 -H "Accept: text/plain"
```

```
Product #1: Wireless Keyboard - $79.99
```

```
# HTML (renders the template)
curl http://localhost:7147/api/products/1 -H "Accept: text/html"
```

```
<!DOCTYPE html>
<html>...rendered template...</html>
```

---

10. Input Validation

Tina4 ships a `Validator` class for declarative input validation. Chain rules together, then check the result. If validation fails, `response.error` returns a structured error envelope (see section 3).

The Validator Class

```
Tina4::Router.post "/api/users" do |request, response|
  v = Tina4::Validator.new(request.body)
  v.required("name").required("email").email("email").min_length("name", 2)

  unless v.valid?
    return response.error("VALIDATION_FAILED", v.errors.first[:message], 400)
  end

  # proceed with valid data
end
```

The `Validator` accepts the request body (a hash) and provides chainable methods:

Method	Description
`required(field)`	Field must be present and non-empty
`email(field)`	Field must be a valid email address
`min_length(field, n)`	Field must have at least `n` characters
`max_length(field, n)`	Field must have at most `n` characters
`numeric(field)`	Field must be a number
`in_list(field, values)`	Field must be one of the allowed values

Call `v.valid?` to check all rules. Call `v.errors` to get the list of failures, each with a `:field` and `:message` key.

The Error Response Envelope

`response.error` returns a consistent JSON error envelope:

```
response.error("VALIDATION_FAILED", "Name is required", 400)
```

This produces:

```
{"error": true, "code": "VALIDATION_FAILED", "message": "Name is required", "status": 400}
```

The three arguments are: an error code string, a human-readable message, and the HTTP status code. Use this pattern across your API for consistent error handling.

Upload Size Limits

Tina4 enforces a maximum upload size through the `TINA4_MAX_UPLOAD_SIZE` environment variable. The value is in bytes. The default is `10485760` (10 MB).

```
TINA4_MAX_UPLOAD_SIZE=10485760
```

When a client sends a body larger than this limit, Tina4 returns a `413 Payload Too Large` response before your handler runs. The check fires before body parsing begins. Your route code never executes.

To allow 50 MB uploads, set this in your `.env` file:

```
TINA4_MAX_UPLOAD_SIZE=52428800
```

Calculate the value by multiplying: megabytes times 1,048,576. For 25 MB, that is `26214400`. For 100 MB, `104857600`. Choose the smallest limit that covers your use case. Large limits invite abuse.

---

11. Exercise: Build an Image Upload API

Build an API that handles image uploads and serves them back.

Requirements

Method	Path	Description
`POST`	`/api/images`	Upload an image. Validate type and size. Return the image URL.
`GET`	`/api/images/{filename}`	Return the uploaded image file. Return 404 if not found.

Rules:

- Accept JPEG, PNG, and WebP only
- Maximum file size: 2MB
- Save files to `src/public/uploads/` with a unique filename
- Return original filename, saved filename, file size in KB, and the URL
- The GET endpoint serves the file (not JSON)

Test with:

```
# Upload
curl -X POST http://localhost:7147/api/images \
  -F "image=@/path/to/photo.jpg"

# Download
curl http://localhost:7147/api/images/img_a1b2c3d4e5f6a7b8.jpg --output downloaded.jpg
```

---

12. Solution

Create `src/routes/images.rb`:

```
require "securerandom"
require "fileutils"

Tina4::Router.post("/api/images") do |request, response|
  # Check if a file was uploaded
  if request.files["image"].nil?
    next response.json({ error: "No image file provided. Use field name 'image'." }, 400)
  end

  file = request.files["image"]

  # Validate file type
  allowed_types = ["image/jpeg", "image/png", "image/webp"]
  unless allowed_types.include?(file["type"])
    next response.json({
      error: "Invalid file type",
      received: file["type"],
      allowed: allowed_types
    }, 400)
  end

  # Validate file size (max 2MB)
  max_size = 2 * 1024 * 1024
  if file["size"] > max_size
    next response.json({
      error: "File too large",
      size_bytes: file["size"],
      max_bytes: max_size
    }, 400)
  end

  # Generate unique filename preserving extension
  extension = File.extname(file["filename"]).downcase
  saved_name = "img_#{SecureRandom.hex(8)}#{extension}"
  upload_dir = File.join(__dir__, "../public/uploads")
  destination = File.join(upload_dir, saved_name)

  # Create uploads directory if it does not exist
  FileUtils.mkdir_p(upload_dir) unless Dir.exist?(upload_dir)

  # Move the uploaded file
  File.binwrite(destination, file["content"])

  response.json({
    message: "Image uploaded successfully",
    original_name: file["filename"],
    saved_name: saved_name,
    size_kb: (file["size"] / 1024.0).round(1),
    type: file["type"],
    url: "/uploads/#{saved_name}"
  }, 201)
end

Tina4::Router.get("/api/images/{filename}") do |request, response|
  filename = request.params["filename"]

  # Prevent directory traversal
  if filename.include?("..") || filename.include?("/")
    next response.json({ error: "Invalid filename" }, 400)
```

```
  end

  filepath = File.join(__dir__, "../public/uploads", filename)

  unless File.exist?(filepath)
    next response.json({ error: "Image not found", filename: filename }, 404)
  end

  response.file(filepath)
end
```

Expected output for upload:

```
{
  "message": "Image uploaded successfully",
  "original_name": "photo.jpg",
  "saved_name": "img_a1b2c3d4e5f6a7b8.jpg",
  "size_kb": 240.0,
  "type": "image/jpeg",
  "url": "/uploads/img_a1b2c3d4e5f6a7b8.jpg"
}
```

(Status: `201 Created`)

Expected output for invalid type:

```
{
  "error": "Invalid file type",
  "received": "application/pdf",
  "allowed": ["image/jpeg", "image/png", "image/webp"]
}
```

(Status: `400 Bad Request`)

Expected output for file too large:

```
{
  "error": "File too large",
  "size_bytes": 5242880,
  "max_bytes": 2097152
}
```

(Status: `400 Bad Request`)

The GET endpoint returns the raw image file with the correct `Content-Type` header. The browser displays it. Curl with `--output` saves it to disk.

---

13. Gotchas

1. Forgetting the response method

Problem: Handler runs (log output confirms it) but the browser gets an empty response or 500 error.

Cause: No call to `response.json`, `response.html`, or another response method. Ruby returns the last expression in a block, but if it is not a response, Tina4 has nothing to send.

Fix: End every handler with a response method call.

2. request.body is nil for GET requests

Problem: Accessing `request.body` in a GET handler returns `nil`.

Cause: GET requests carry no body. Browsers and curl send none by convention.

Fix: Use `request.params` for GET parameters. The body populates only for POST, PUT, and PATCH requests.

3. Body is nil for JSON requests

Problem: `request.body` is `nil` or empty even though you sent JSON.

Cause: Missing `Content-Type: application/json` header. Without it, Tina4 does not parse the body as JSON.

Fix: Include `-H "Content-Type: application/json"` with curl. In frontend JavaScript, `fetch()` with `JSON.stringify()` needs `headers: {"Content-Type": "application/json"}`.

4. Content-Type Mismatch

Problem: `response.json` sends HTML, or `response.html` sends plain text.

Cause: A middleware or error handler is overwriting the response. Or you used `puts` instead of a response method.

Fix: Use `response.json(...)`, `response.html(...)`, or another response method. `puts` bypasses the response object entirely.

5. File Uploads Return Empty

Problem: `request.files` is empty despite uploading a file.

Cause: The form lacks `enctype="multipart/form-data"`, or curl uses `-d` instead of `-F`.

Fix: HTML forms need `<form enctype="multipart/form-data">`. Curl needs `-F "field=@file.jpg"` (with `@`), not `-d`.

6. Redirect Loops

Problem: The browser shows "too many redirects" or hangs.

Cause: A route redirects to another route, which redirects back. Example: `/login` redirects to `/dashboard`, and `/dashboard` redirects to `/login` because the user is not authenticated.

Fix: Trace the redirect chain with the browser's network inspector. Make sure auth checks do not redirect authenticated users away from pages they can access.

7. Cookie Not Set

Problem: `response.cookie(...)` was called but the browser shows no cookie.

Cause: `secure: true` means the cookie travels over HTTPS only. Local development uses `http://localhost`, so the cookie is dropped.

Fix: Set `secure: false` during development, or use `secure: ENV["TINA4_DEBUG"] != "true"` to auto-switch.

8. Large Request Body Rejected

Problem: POST requests with large bodies return 413.

Cause: The body exceeds the configured maximum size.

Fix: Increase `TINA4_MAX_UPLOAD_SIZE` in `.env`. Default is `10mb`. For file uploads, you may need `50mb` or more:

```
TINA4_MAX_UPLOAD_SIZE=50mb
```

9. Chaining Response Methods in Wrong Order

Problem: Headers or cookies set after calling `response.json` have no effect.

Cause: `response.json` finalizes and returns the response object. Calling methods on a separate line after the return is dead code.

Fix: Chain headers and cookies before the final response method:

```
# Correct
response
  .header("X-Custom", "value")
  .cookie("token", "abc", { http_only: true })
  .json({ ok: true })

# Wrong -- header is set after json already returned
result = response.json({ ok: true })
response.header("X-Custom", "value")  # Too late
result
```

Templates

1. Beyond JSON -- Rendering HTML

Every route so far returns JSON. That works for APIs. Web applications need HTML -- product listings, dashboards, login forms, email templates. Tina4 uses the **Frond** template engine for this work.

Frond is a zero-dependency template engine built from scratch. Its syntax matches Twig, Jinja2, and Nunjucks. Three constructs drive the entire engine: `{{ }}` for output, `{% %}` for logic, `{# #}` for comments. That is the whole grammar.

Templates live in `src/templates/`. When you call `response.render("page.html", data)`, Frond loads `src/templates/page.html`, processes the tags and expressions, and returns the final HTML.

This chapter builds toward a product catalog page. Items in a grid. Featured products highlighted. Prices formatted. Layout inherited from a shared template. One engine handles it all.

---

2. Variables and Expressions

Output a variable with double curly braces:

```
<h1>Hello, {{ name }}!</h1>
```

Route handler:

```
Tina4::Router.get("/welcome") do |request, response|
  response.render("welcome.html", { name: "Alice" })
end
```

Create `src/templates/welcome.html`:

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

Browser output:

```
Hello, Alice!
```

Accessing Nested Data

Dot notation reaches into nested hashes:

```
data = {
  user: {
    name: "Alice",
    email: "alice@example.com",
    address: {
```

```
      city: "Cape Town",
      country: "South Africa"
    }
  }
}

response.render("profile.html", data)
```

```
<p>{{ user.name }} lives in {{ user.address.city }}, {{ user.address.country }}.</p>
```

Output:

```
Alice lives in Cape Town, South Africa.
```

Method Calls on Values

If a Hash value is a `Proc` or `lambda`, you can call it directly in a template expression:

```
data = {
  user: {
    name: "Alice",
    t: ->(key) { I18n.t(key) },
    greet: ->(greeting) { "#{greeting}, Alice!" }
  }
}

response.render("page.twig", data)
```

```
<p>{{ user.t("welcome_message") }}</p>
<p>{{ user.greet("Hello") }}</p>
```

This works for any callable value -- `Proc`, `lambda`, or `method` objects. Arguments are parsed from the parenthesized expression.

Expressions

Basic expressions work inside `{{ }}`:

```
<p>Total: ${{ price * quantity }}</p>
<p>Discounted: ${{ price * 0.9 }}</p>
<p>Full name: {{ first_name ~ " " ~ last_name }}</p>
```

The `~` operator concatenates strings.

---

3. Template Inheritance

Template inheritance is the headline feature. Define a base layout once. Extend it in every page.

Base Layout

Create `src/templates/base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My App{% endblock %}</title>
```

```
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My App. All rights reserved.</p>
    </footer>

    <script src="/js/frond.js"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Four blocks: `title`, `head`, `content`, `scripts`. Child templates override only the blocks they need.

Child Template

Create `src/templates/about.twig`:

```
{% extends "base.twig" %}

{% block title %}About Us{% endblock %}

{% block content %}
    <h1>About Us</h1>
    <p>We have been building things since {{ founded_year }}.</p>
    <p>Our team has {{ team_size }} members across {{ office_count }} offices.</p>
{% endblock %}
```

Route handler:

```
Tina4::Router.get("/about") do |request, response|
  response.render("about.twig", {
    founded_year: 2020,
    team_size: 12,
    office_count: 3
  })
end
```

Browser output: A full HTML page with the nav, the "About Us" content, and the footer. The `<title>` tag reads "About Us". The `head` and `scripts` blocks stay empty -- the child did not override them.

Using `{{ parent() }}`

Add to a block instead of replacing it:

```
{% extends "base.twig" %}
```

```
{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/contact-form.css">
{% endblock %}

{% block content %}
    <h1>Contact Us</h1>
    <form>...</form>
{% endblock %}
```

The `head` block now contains everything from the base plus the extra stylesheet.

---

4. Includes

Break templates into reusable pieces with `{% include %}`:

Create `src/templates/partials/header.twig`:

```
<header>
    <div class="logo">{{ site_name | default("My App") }}</div>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/contact">Contact</a>
    </nav>
</header>
```

Create `src/templates/partials/product-card.twig`:

```
<div class="product-card{{ product.featured ? ' featured' : '' }}">
    <h3>{{ product.name }}</h3>
    <p class="price">${{ product.price | number_format(2) }}</p>
    {% if product.in_stock %}
        <span class="badge-success">In Stock</span>
    {% else %}
        <span class="badge-danger">Out of Stock</span>
    {% endif %}
</div>
```

Use them in a page:

```
{% extends "base.twig" %}

{% block content %}
    {% include "partials/header.twig" %}

    <h1>Products</h1>
    {% for product in products %}
        {% include "partials/product-card.twig" %}
    {% endfor %}
{% endblock %}
```

The `product` variable is available inside the included template because it exists in the current scope (the for loop).

Passing Variables to Includes

Pass specific variables with `with`:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} %}
```

Use `only` to isolate the included template from the parent scope:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} only %}
```

With `only`, the included template sees `site_name` and nothing else.

---

5. For Loops

Loop through arrays with `{% for %}`:

```
<ul>
{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
</ul>
```

The `loop` Variable

Inside a for loop, Frond provides a special `loop` variable:

Property	Type	Description
`loop.index`	int	Current iteration (1-based)
`loop.index0`	int	Current iteration (0-based)
`loop.first`	bool	True on the first iteration
`loop.last`	bool	True on the last iteration
`loop.length`	int	Total number of items
`loop.revindex`	int	Iterations remaining (1-based)

```
<table>
    <thead>
        <tr><th>#</th><th>Name</th><th>Price</th></tr>
    </thead>
    <tbody>
    {% for product in products %}
        <tr class="{{ loop.index is odd ? 'row-light' : 'row-dark' }}">
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ product.price | number_format(2) }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

Empty Lists

Handle empty lists with `{% else %}`:

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
    </div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

If `products` is empty or undefined, the `else` block renders instead.

Looping Over Key-Value Pairs

```
{% for key, value in metadata %}
    <dt>{{ key }}</dt>
    <dd>{{ value }}</dd>
{% endfor %}
```

---

6. Conditionals

if / elseif / else

```
{% if user.role == "admin" %}
    <a href="/admin">Admin Panel</a>
{% elseif user.role == "editor" %}
    <a href="/editor">Editor Dashboard</a>
{% else %}
    <a href="/profile">My Profile</a>
{% endif %}
```

Ternary Operator

Inline conditionals:

```
<span class="{{ is_active ? 'text-green' : 'text-gray' }}">
    {{ is_active ? 'Active' : 'Inactive' }}
</span>
```

Testing for Existence

Use `is defined` to check if a variable exists:

```
{% if error_message is defined %}
    <div class="alert alert-danger">{{ error_message }}</div>
{% endif %}
```

Truthiness

These values are false: `false`, `nil`, `0`, `""` (empty string), `[]` (empty array). Everything else is true.

```
{% if items %}
    <p>{{ items | length }} items found.</p>
{% else %}
    <p>No items.</p>
```

```
{% endif %}
```

{% set %} -- Local Variables

Create or update a variable inside a template:

```
{% set greeting = "Hello" %}
{% set full_name = user.first_name ~ " " ~ user.last_name %}
{% set total = price * quantity %}
{% set discount = total - rebate %}

<p>{{ greeting }}, {{ full_name }}!</p>
<p>Total: {{ total }}, After discount: {{ discount }}</p>
```

The `~` operator concatenates strings. Arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`) work in `set` and expressions.

When combining filters with arithmetic, assign the filtered values first:

```
{% set dr = account.dr|default(0) %}
{% set cr = account.cr|default(0) %}
{% set balance = dr - cr %}
<p>Balance: {{ balance }}</p>
```

---

7. Filters

Filters transform values. Apply them with the `|` (pipe) character:

```
{{ name | upper }}
```

Complete Filter Reference

String Filters

Filter	Example	Description	
`upper`	`{{ name | upper }}`		Convert to uppercase
`lower`	`{{ name | lower }}`		Convert to lowercase
`capitalize`	`{{ name | capitalize }}`		Capitalize first letter
`title`	`{{ name | title }}`		Capitalize each word
`trim`	`{{ name | trim }}`		Strip leading/trailing whitespace
`ltrim`	`{{ name | ltrim }}`		Strip leading whitespace
`rtrim`	`{{ name | rtrim }}`		Strip trailing whitespace
`slug`	`{{ title | slug }}`		Convert to URL-friendly slug
`wordwrap(80)`	`{{ text | wordwrap(80) }}`		Wrap text at N characters
`truncate(100)`	`{{ text | truncate(100) }}`		Truncate to N characters with ellipsis
`nl2br`	`{{ text | nl2br }}`		Convert newlines to ` ` tags
`striptags`	`{{ html | striptags }}`		Remove all HTML tags
`replace("a", "b")`	`{{ text | replace("old", "new") }}`		Replace occurrences of a substring

Array Filters

Filter	Example	Description	
`length`	`{{ items \	length }}`	Count items in array or string length
`reverse`	`{{ items \	reverse }}`	Reverse order of items
`sort`	`{{ items \	sort }}`	Sort items ascending
`shuffle`	`{{ items \	shuffle }}`	Randomly shuffle items
`first`	`{{ items \	first }}`	Get the first item
`last`	`{{ items \	last }}`	Get the last item
`join(", ")`	`{{ items \	join(", ") }}`	Join array items with separator
`split(",")`	`{{ csv \	split(",") }}`	Split string into array
`unique`	`{{ items \	unique }}`	Remove duplicate values
`filter`	`{{ items \	filter }}`	Remove falsy values from array
`map("name")`	`{{ items \	map("name") }}`	Extract a property from each item
`column("name")`	`{{ items \	column("name") }}`	Extract a column from array of objects
`batch(3)`	`{{ items \	batch(3) }}`	Group items into batches of N
`slice(0, 3)`	`{{ items \	slice(0, 3) }}`	Extract a slice from offset with length

Encoding Filters

Filter	Example	Description	
`escape` (`e`)	`{{ text | escape }}`	HTML-escape special characters
`raw` (`safe`)	`{{ html | raw }}`	Output without auto-escaping
`url_encode`	`{{ text | url_encode }}`	URL-encode a string
`base64_encode` (`base64encode`)	`{{ text | base64_encode }}`	Base64-encode a string
`base64_decode` (`base64decode`)	`{{ data | base64_decode }}`	Base64-decode a string
`md5`	`{{ text | md5 }}`	Compute MD5 hash
`sha256`	`{{ text | sha256 }}`	Compute SHA-256 hash

Numeric Filters

Filter	Example	Description	
`abs`	`{{ num | abs }}`	Absolute value
`round(2)`	`{{ price | round(2) }}`	Round to N decimal places
`number_format(2)`	`{{ price | number_format(2) }}`	Format with decimals and thousands separator
`int`	`{{ val | int }}`	Cast to integer
`float`	`{{ val | float }}`	Cast to float
`string`	`{{ val | string }}`	Cast to string

JSON Filters

Filter	Example	Description	
`json_encode`	`{{ data | json_encode }}`	Encode value as JSON string
`to_json` (`tojson`)	`{{ data | to_json }}`	Encode value as JSON string (alias)
`json_decode`	`{{ str | json_decode }}`	Decode JSON string to object
`js_escape`	`{{ text | js_escape }}`	Escape string for safe use in JavaScript

Dict Filters

Filter	Example	Description	
`keys`	`{{ obj | keys }}`	Get dictionary keys as array	
`values`	`{{ obj | values }}`	Get dictionary values as array	
`merge(other)`	`{{ defaults | merge(overrides) }}`	Merge two dictionaries	

Other Filters

Filter	Example	Description	
`default("fallback")`	`{{ name | default("Guest") }}`	Fallback when value is empty or undefined	
`date("Y-m-d")`	`{{ created | date("Y-m-d") }}`	Format a date value	
`format(val)`	`{{ "%.2f" | format(price) }}`	Format string with value (sprintf-style)	
`data_uri`	`{{ content | data_uri }}`	Convert to a data URI string	
`dump`	`{{ var | dump }}` or `{{ dump(var) }}`	Debug output, gated on `TINA4_DEBUG=true` (see *Dumping Values*)	
`form_token`	`{{ form_token() }}`	Generate a CSRF hidden input with token	
`formTokenValue`	`{{ formTokenValue("context") }}`	Return just the raw JWT token string	
`to_json`	`{{ data | to_json }}`	JSON-encode a value (safe, no double-escaping)	
`js_escape`	`{{ text | js_escape }}`	Escape for safe use in JavaScript strings	

Chaining Filters

Filters chain left to right:

```
{{ name | trim | lower | capitalize }}
{# "  ALICE SMITH  " -> "Alice smith" #}
```

Dumping Values for Debugging

The `dump` helper lets you inspect any variable mid-template. Two interchangeable forms are supported:

```
{{ user | dump }}
{{ dump(user) }}
```

Both produce the same `<pre>`-wrapped, HTML-escaped `inspect` of the value. Handles hashes, arrays, class instances, and cyclic references: Ruby's `inspect` prints `{...}` for back-edges.

```
{{ dump(order) }}

{# Output: #}
{# <pre>#&lt;Order:0x00007f8 @id=42, @items=[...], @total=99.99&gt;</pre> #}
```

dump is gated on `TINA4_DEBUG=true`. In production (env var unset or `false`) **both** the filter and function form silently return an empty `SafeString`. This prevents accidental leaks of internal state, object shapes, or sensitive values into rendered HTML if a developer leaves a `{{ dump(x) }}` call in a template.

```
# .env - dev
TINA4_DEBUG=true    # dump() outputs the value

# .env - production
TINA4_DEBUG=false   # dump() is a no-op
```

You can rely on this gate for safety, but treat `dump` as a development-only convenience. For structured output in production code paths, use `to_json`.

---

8. Macros

Macros are reusable template functions. Components you define once and call many times.

Defining a Macro

Create `src/templates/macros.twig`:

```
{% macro button(text, url, style) %}
    <a href="{{ url | default('#') }}" class="btn btn-{{ style | default('primary') }}">
        {{ text }}
    </a>
{% endmacro %}

{% macro alert(message, type) %}
    <div class="alert alert-{{ type | default('info') }}">
        {{ message }}
    </div>
{% endmacro %}

{% macro input(name, label, type, value) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label | default(name | capitalize) }}</label>
        <input type="{{ type | default('text') }}" id="{{ name }}" name="{{ name }}" value="{{ v
    </div>
{% endmacro %}
```

Using Macros

Import macros in your template:

```
{% from "macros.twig" import button, alert, input %}

{% extends "base.twig" %}
```

```
{% block content %}
    {{ alert("Your profile has been updated.", "success") }}

    <form method="POST" action="/profile">
        {{ input("name", "Full Name", "text", user.name) }}
        {{ input("email", "Email Address", "email", user.email) }}
        {{ input("phone", "Phone Number", "tel", user.phone) }}

        {{ button("Save Changes", "", "primary") }}
        {{ button("Cancel", "/dashboard", "secondary") }}
    </form>
{% endblock %}
```

Expected output (simplified):

```
<div class="alert alert-success">
    Your profile has been updated.
</div>

<form method="POST" action="/profile">
    <div class="form-group">
        <label for="name">Full Name</label>
        <input type="text" id="name" name="name" value="Alice">
    </div>
    <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" name="email" value="alice@example.com">
    </div>
    <div class="form-group">
        <label for="phone">Phone Number</label>
        <input type="tel" id="phone" name="phone" value="">
    </div>

    <a href="#" class="btn btn-primary">Save Changes</a>
    <a href="/dashboard" class="btn btn-secondary">Cancel</a>
</form>
```

---

9. Special Tags

{% raw %} -- Literal Output

When you need to output literal `{{ }}` or `{% %}` (for Vue.js or Angular templates):

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

Outputs the literal text `{{ message }}` without processing it as a Frond expression.

{% spaceless %} -- Remove Whitespace

Strip whitespace between HTML tags:

```
{% spaceless %}
    <div>
```

```
        <span>Hello</span>
    </div>
{% endspaceless %}
```

Output:

```
<div><span>Hello</span></div>
```

Useful for inline elements where whitespace creates unwanted gaps.

{% autoescape %} -- Control Escaping

Override auto-escaping for a block:

```
{% autoescape false %}
    {{ trusted_html }}
{% endautoescape %}
```

Everything inside outputs without HTML escaping. Equivalent to `| raw` on every variable, but more convenient for large blocks of trusted content.

Comments

Template comments stay invisible in the output:

```
{# This comment will not appear in the HTML output #}

{#
    Multi-line comments work too.
    Use them to document template logic.
#}
```

---

10. tina4css Integration

Every Tina4 project includes `tina4.css` -- a built-in CSS utility framework available at `/css/tina4.css`. Layout, typography, common UI patterns. No external dependencies.

Include it in your base template:

```
<link rel="stylesheet" href="/css/tina4.css">
```

Layout Classes

```
<div class="container">
    <div class="row">
        <div class="col-6">Left half</div>
        <div class="col-6">Right half</div>
    </div>
</div>
```

Common Components

```
<!-- Buttons -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-danger">Danger</button>

<!-- Cards -->
<div class="card">
    <div class="card-header">Title</div>
    <div class="card-body">Content here</div>
    <div class="card-footer">Footer</div>
</div>

<!-- Alerts -->
<div class="alert alert-success">Operation completed.</div>
<div class="alert alert-danger">Something went wrong.</div>
<div class="alert alert-warning">Please review your input.</div>

<!-- Forms -->
<div class="form-group">
    <label for="name">Name</label>
    <input type="text" id="name" class="form-control">
</div>
```

Utility Classes

```
<p class="text-center">Centered text</p>
<p class="text-right">Right-aligned text</p>
<div class="mt-4">Margin top</div>
<div class="p-3">Padding all around</div>
<span class="text-muted">Gray text</span>
<span class="text-primary">Primary color text</span>
```

No Bootstrap needed. No Tailwind needed. If you prefer those, swap the `<link>` tag. tina4css stays out of the way.

---

11. Exercise: Build a Product Catalog Page

Build a product catalog page with a base layout, product cards, category filters, and a reusable card macro.

Requirements

- Create a base layout at `src/templates/catalog-base.twig` with blocks for `title`, `content`, and `scripts`
- Create a macro file at `src/templates/catalog-macros.twig` with:
- A `productCard(product)` macro that renders a styled card with name, category, price, stock status, and optional featured badge
- A `categoryFilter(categories, active)` macro that renders filter buttons
- Create a page template at `src/templates/catalog.twig` that:
- Extends the base layout

- Uses the macros
- Shows a heading with total product count
- Shows category filter buttons (All, and one per unique category)
- Shows product cards in a grid
- Shows featured products with a distinct style
- Handles empty filter results
- Create a route at `GET /catalog` that accepts an optional `?category=` filter

Data

Use this product list in your route handler:

```
products = [
  { name: "Espresso Machine", category: "Kitchen", price: 299.99, in_stock: true, featured: true
  { name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true, featured: false },
  { name: "Standing Desk", category: "Office", price: 549.99, in_stock: true, featured: true },
  { name: "Blender", category: "Kitchen", price: 89.99, in_stock: false, featured: false },
  { name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true, featured: false }
  { name: "Desk Lamp", category: "Office", price: 39.99, in_stock: true, featured: true },
  { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, in_stock: true, featured: fals
]
```

Test with:

```
http://localhost:7147/catalog
http://localhost:7147/catalog?category=Kitchen
http://localhost:7147/catalog?category=Fitness
```

---

12. Solution

Create `src/templates/catalog-base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Product Catalog{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; m
        .container { max-width: 1000px; margin: 0 auto; padding: 20px; }
        .header { background: #2c3e50; color: white; padding: 20px; margin-bottom: 24px; }
        .header h1 { margin: 0; }
        .header p { margin: 4px 0 0; opacity: 0.8; }
        .filters { margin-bottom: 20px; }
        .filter-btn { display: inline-block; padding: 6px 14px; margin: 0 6px 6px 0; border-radi
        .filter-btn.active { background: #2c3e50; color: white; border-color: #2c3e50; }
        .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1f
        .product-card { background: white; border: 2px solid #e9ecef; border-radius: 8px; paddin
        .product-card:hover { border-color: #adb5bd; }
        .product-card.featured { border-color: #f39c12; background: #fef9e7; }
        .product-name { font-size: 1.1em; font-weight: 600; margin: 0 0 4px; }
```

```
        .product-category { font-size: 0.85em; color: #6c757d; margin: 0 0 8px; }
        .product-price { font-size: 1.2em; font-weight: bold; color: #27ae60; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.75em;
        .badge-featured { background: #f39c12; color: white; }
        .badge-stock { background: #d4edda; color: #155724; }
        .badge-nostock { background: #f8d7da; color: #721c24; }
        .empty-state { text-align: center; padding: 40px; color: #6c757d; }
    </style>
</head>
<body>
    {% block content %}{% endblock %}

    <script src="/js/frond.js"></script>
    {% block scripts %}{% endblock %}
</body>
</html>
```

Create `src/templates/catalog-macros.twig`:

```
{% macro productCard(product) %}
    <div class="product-card{{ product.featured ? ' featured' : '' }}">
        <p class="product-name">
            {{ product.name }}
            {% if product.featured %}
                <span class="badge badge-featured">Featured</span>
            {% endif %}
        </p>
        <p class="product-category">{{ product.category }}</p>
        <p class="product-price">
            ${{ product.price | number_format(2) }}
            {% if product.in_stock %}
                <span class="badge badge-stock">In Stock</span>
            {% else %}
                <span class="badge badge-nostock">Out of Stock</span>
            {% endif %}
        </p>
    </div>
{% endmacro %}

{% macro categoryFilter(categories, active) %}
    <div class="filters">
        <a href="/catalog" class="filter-btn{{ active is not defined or active == '' ? ' active'
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}" class="filter-btn{{ active == cat ? ' active'
        {% endfor %}
    </div>
{% endmacro %}
```

Create `src/templates/catalog.twig`:

```
{% extends "catalog-base.twig" %}

{% from "catalog-macros.twig" import productCard, categoryFilter %}

{% block title %}{{ active_category | default("All") }} Products - Catalog{% endblock %}

{% block content %}
    <div class="header">
        <h1>Product Catalog</h1>
        <p>{{ products | length }} product{{ products | length != 1 ? 's' : '' }}{% if active_ca
```

```
    </div>

    <div class="container">
        {{ categoryFilter(categories, active_category) }}

        {% if products | length > 0 %}
            <div class="product-grid">
                {% for product in products %}
                    {{ productCard(product) }}
                {% endfor %}
            </div>
        {% else %}
            <div class="empty-state">
                <h2>No products found</h2>
                <p>Try a different category or <a href="/catalog">view all products</a>.</p>
            </div>
        {% endif %}
    </div>
{% endblock %}
```

Create `src/routes/catalog.rb`:

```
Tina4::Router.get("/catalog") do |request, response|
  all_products = [
    { name: "Espresso Machine", category: "Kitchen", price: 299.99, in_stock: true, featured: tr
    { name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true, featured: false },
    { name: "Standing Desk", category: "Office", price: 549.99, in_stock: true, featured: true }
    { name: "Blender", category: "Kitchen", price: 89.99, in_stock: false, featured: false },
    { name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true, featured: false
    { name: "Desk Lamp", category: "Office", price: 39.99, in_stock: true, featured: true },
    { name: "Cast Iron Skillet", category: "Kitchen", price: 44.99, in_stock: true, featured: fa
  ]

  # Get unique categories
  categories = all_products.map { |p| p[:category] }.uniq.sort

  # Filter by category if specified
  active_category = request.params["category"] || ""
  products = if active_category.empty?
               all_products
             else
               all_products.select { |p| p[:category].downcase == active_category.downcase }
             end

  response.render("catalog.twig", {
    products: products,
    categories: categories,
    active_category: active_category
  })
end
```

Browser output for `/catalog`:

- A dark header with "Product Catalog" and "7 products"
- Filter buttons: All (active), Fitness, Kitchen, Office
- A grid of 7 product cards

- Three cards (Espresso Machine, Standing Desk, Desk Lamp) wear a gold border and "Featured" badge
- The Blender card wears a red "Out of Stock" badge

Browser output for `/catalog?category=Kitchen`:

- Header shows "3 products in Kitchen"
- Kitchen filter button is active
- Three cards: Espresso Machine, Blender, Cast Iron Skillet

---

13. Gotchas

1. `{% extends %}` Must Be the First Tag

Problem: Template inheritance does not work. The page renders without the base layout.

Cause: `{% extends "base.twig" %}` must be the first tag in the template. Any text, whitespace, or comment before it breaks inheritance.

Fix: Make `{% extends %}` the absolute first thing in the file. Move `{% from %}` imports after the extends tag.

2. Undefined Variables Show Nothing

Problem: `{{ username }}` renders as empty instead of raising an error.

Cause: Frond outputs nothing for undefined variables. By design. But it can hide bugs.

Fix: Use the `default` filter: `{{ username | default("Guest") }}`. Or check with `{% if username is defined %}`.

3. Auto-Escaping Prevents HTML Output

Problem: You pass HTML content but it appears as literal text in the page.

Cause: Auto-escaping converts `<` to `<` and `>` to `>` for security.

Fix: Trusted content gets `{{ content | raw }}`. Never use `raw` on user-supplied input.

4. Variable Scope in Includes

Problem: A variable defined inside a `{% for %}` loop vanishes after the loop ends.

Cause: Loop variables are scoped to the loop. They do not leak into the outer scope.

Fix: Use `{% set %}` before the loop and update inside it. Or restructure to keep logic within the loop.

5. Macro Arguments Are Positional

Problem: `{{ button("Click", style="danger") }}` does not work as expected.

Cause: Frond macros use positional arguments. Order matters.

Fix: Pass arguments in the order they are defined: `{{ button("Click", "/url", "danger") }}`. Many optional arguments? Consider passing a single object.

6. Template File Extension Does Not Matter

Problem: Should you use `.html`, `.twig`, or `.tpl`?

Cause: Frond does not care about file extensions. It processes any file in `src/templates/`.

Fix: Pick one and stay consistent. This book uses `.twig` for templates with Twig syntax and `.html` for simple HTML. Both work the same.

7. Filters Are Not Ruby Methods

Problem: `{{ items | count }}` or `{{ name | upcase }}` raises an error.

Cause: Frond filters follow Twig conventions, not Ruby conventions.

Fix: Use `{{ items | length }}` not `count`. Use `{{ name | upper }}` not `upcase`. Use `{{ text | lower }}` not `downcase`. See the filter table in section 7.

Database

1. From Arrays to Real Data

In Chapters 2 and 3, all your data lived in Ruby arrays. Server restart. Data gone. Fine for learning routing. Useless for production.

This chapter covers Tina4's database layer: raw queries, parameterised queries, transactions, schema inspection, helper methods, and migrations.

Tina4 speaks to five database engines: SQLite, PostgreSQL, MySQL, Microsoft SQL Server, and Firebird. The API is identical across all five. Switch databases by changing one line in `.env`.

---

2. Connecting to a Database

The Default: SQLite

When you scaffold with `tina4 init`, Tina4 creates a SQLite database at `data/app.db`. The default `.env` contains:

```
TINA4_DEBUG=true
```

No explicit `TINA4_DATABASE_URL`? Tina4 defaults to `sqlite:///data/app.db`. The health check at `/health` shows `"database": "connected"` with zero configuration.

Connection Strings for Other Databases

Set `TINA4_DATABASE_URL` in `.env` to use a different engine:

```
# SQLite (explicit)
TINA4_DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
TINA4_DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
TINA4_DATABASE_URL=mysql://localhost:3306/myapp

# Microsoft SQL Server
TINA4_DATABASE_URL=mssql://localhost:1433/myapp

# Firebird
TINA4_DATABASE_URL=firebird://localhost:3050/path/to/database.fdb
```

Firebird URL Forms

Firebird is the awkward one: every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through `URI.parse`, which is unintuitive.

Tina4 normalises five equivalent forms. Pick whichever reads best:

```
# Classic double-slash absolute path -- the URL spec way
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050//firebird/data/app.fdb

# Single-slash absolute path -- what most people instinctively type
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/firebird/data/app.fdb

# Windows drive-letter path (also accepts /C%3A/Data/app.fdb)
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@host:3050/C:/Data/app.fdb

# Firebird alias (single token, no slashes)
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/employee
```

For ops setups that keep the server URL and database location in separate config layers -- or for Windows backslash paths -- set `TINA4_DATABASE_FIREBIRD_PATH`:

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

The env override wins over whatever path is in the URL.

Separate Credentials

If you prefer to keep credentials out of the connection string (recommended for production), use separate environment variables:

```
TINA4_DATABASE_URL=postgres://localhost:5432/myapp
TINA4_DATABASE_USERNAME=myuser
TINA4_DATABASE_PASSWORD=secretpassword
```

Tina4 merges these with the connection string at startup. The credentials in the separate variables take precedence over any embedded in the URL.

Programmatic Connection

You can also create a database connection directly in Ruby code:

```
db = Tina4::Database.new("sqlite://app.db", username: nil, password: nil)
```

Connection Pooling

For applications that handle many concurrent requests, enable connection pooling with the `pool` parameter:

```
db = Tina4::Database.new("postgres://localhost/mydb", pool: 5)
```

The `pool` parameter controls how many database connections are maintained:

- `pool: 0` (the default) -- a single connection is used for all queries
- `pool: N` (where N > 0) -- N connections are created and rotated round-robin across queries

Pooled connections are thread-safe. Each query is dispatched to the next available connection in the pool. This eliminates contention when multiple route handlers query the database simultaneously.

Verifying the Connection

After updating `.env`, restart the server and check:

```
curl http://localhost:7147/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3,
  "version": "3.0.0",
  "framework": "tina4-ruby"
}
```

If the database is not reachable, you will see `"database": "disconnected"` with an error message.

---

3. Getting the Database Object

In your route handlers, access the database via the `Tina4::Database` class:

```
Tina4::Router.get("/api/test-db") do |request, response|
  db = Tina4.database

  result = db.fetch("SELECT 1 + 1 AS answer")

  response.json(result)
end
```

```
curl http://localhost:7147/api/test-db
```

```
{"answer": 2}
```

`Tina4.database` returns the active database connection. You call methods like `fetch`, `execute`, and `fetch_one` on this object.

---

4. Raw Queries

fetch -- Get Multiple Rows

```
db = Tina4.database

# Returns an array of hashes
products = db.fetch("SELECT * FROM products WHERE price > 50")
```

Each row is a hash with column names as keys:

```
# products looks like:
[
  { "id" => 1, "name" => "Keyboard", "price" => 79.99 },
  { "id" => 4, "name" => "Standing Desk", "price" => 549.99 }
]
```

DatabaseResult

`fetch` returns a `DatabaseResult` object. It behaves like an array but carries extra metadata about the query.

Properties

```
result = db.fetch("SELECT * FROM users WHERE active = ?", [1])

result.records      # [{ "id" => 1, "name" => "Alice" }, { "id" => 2, "name" => "Bob" }]
result.columns      # ["id", "name", "email", "active"]
result.count        # total number of matching rows
result.limit        # query limit (if set)
result.offset       # query offset (if set)
```

Iteration

A `DatabaseResult` is enumerable. Use it directly in loops:

```
result.each do |user|
  puts user["name"]
end
```

Index Access

Access rows by index like a regular array:

```
first_user = result[0]
```

Countable

`length` works on the result:

```
puts result.length  # number of records in this result set
```

Conversion Methods

```
result.to_json       # JSON string of all records
result.to_csv        # CSV string with column headers
result.to_array      # plain array of hashes
result.to_paginate   # { "records" => [...], "count" => 42, "limit" => 10, "offset" => 0 }
```

`to_paginate` is designed for building paginated API responses. It bundles the records with the total count, limit, and offset in a single hash.

Schema Metadata with column_info

`column_info` returns detailed metadata about the columns in the result set. The data is lazy-loaded -- it only queries the database schema when you call the method for the first time:

```
info = result.column_info
# [
#     { "name" => "id", "type" => "INTEGER", "size" => nil, "decimals" => nil, "nullable" => fal
#     { "name" => "name", "type" => "TEXT", "size" => nil, "decimals" => nil, "nullable" => fals
#     { "name" => "email", "type" => "TEXT", "size" => 255, "decimals" => nil, "nullable" => tru
#     ...
# ]
```

Each column entry contains:

Field	Description
`name`	Column name
`type`	Database type (e.g. `INTEGER`, `TEXT`, `REAL`)

`size`	Maximum size (or `nil` if not applicable)
`decimals`	Decimal places (or `nil`)
`nullable`	Whether the column allows `NULL`
`primary_key`	Whether the column is part of the primary key

This is useful for building dynamic forms, generating documentation, or validating data before insert.

fetch_one -- Get a Single Row

```
product = db.fetch_one("SELECT * FROM products WHERE id = 1")
# Returns: { "id" => 1, "name" => "Keyboard", "price" => 79.99 }
```

If no row matches, `fetch_one` returns `nil`.

execute -- Run a Statement

For INSERT, UPDATE, DELETE, and DDL statements that do not return rows:

```
db.execute("INSERT INTO products (name, price) VALUES ('Widget', 9.99)")
db.execute("UPDATE products SET price = 89.99 WHERE id = 1")
db.execute("DELETE FROM products WHERE id = 5")
db.execute("CREATE TABLE IF NOT EXISTS logs (id INTEGER PRIMARY KEY, message TEXT, created_at TE
```

Full Example: A Simple Query Route

```
Tina4::Router.get("/api/products") do |request, response|
  db = Tina4.database

  products = db.fetch("SELECT * FROM products ORDER BY name")

  response.json({
    products: products,
    count: products.length
  })
end
```

```
curl http://localhost:7147/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Keyboard", "price": 79.99, "in_stock": 1},
    {"id": 2, "name": "Mouse", "price": 29.99, "in_stock": 1},
    {"id": 3, "name": "Monitor", "price": 399.99, "in_stock": 0}
  ],
  "count": 3
}
```

---

5. Parameterised Queries

Never concatenate user input into SQL strings. That road leads to SQL injection:

```
# NEVER do this:
db.fetch("SELECT * FROM products WHERE name = '#{user_input}'")
```

Instead, use parameterised queries. Pass parameters as the second argument:

```
db = Tina4.database

# Positional parameters with ?
product = db.fetch_one(
  "SELECT * FROM products WHERE id = ?",
  [42]
)

# Multiple parameters
products = db.fetch(
  "SELECT * FROM products WHERE price BETWEEN ? AND ? ORDER BY price",
  [10.00, 100.00]
)
```

The database driver handles escaping. Your input is never part of the SQL string.

A Safe Search Endpoint

```
Tina4::Router.get("/api/products/search") do |request, response|
  db = Tina4.database

  q = request.params["q"] || ""
  max_price = (request.params["max_price"] || 99999).to_f

  if q.empty?
    return response.json({ error: "Query parameter 'q' is required" }, 400)
  end

  products = db.fetch(
    "SELECT * FROM products WHERE name LIKE ? AND price <= ? ORDER BY name",
    ["%#{q}%", max_price]
  )

  response.json({
    query: q,
    max_price: max_price,
    results: products,
    count: products.length
  })
end
```

```
curl "http://localhost:7147/api/products/search?q=key&max_price=100"
```

```
{
  "query": "key",
  "max_price": 100,
  "results": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "in_stock": 1}
  ],
  "count": 1
}
```

---

6. Transactions

Multiple operations must succeed or fail together. Transactions enforce that contract:

```
Tina4::Router.post("/api/orders") do |request, response|
  db = Tina4.database
  body = request.body

  begin
    db.transaction do |tx|
      # Create the order
      tx.execute(
        "INSERT INTO orders (customer_id, total, status) VALUES (?, ?, 'pending')",
        [body["customer_id"], body["total"]]
      )

      # Get the new order ID
      order = tx.fetch_one("SELECT last_insert_rowid() AS id")
      order_id = order["id"]

      # Create order items
      body["items"].each do |item|
        tx.execute(
          "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (?, ?, ?, ?)",
          [order_id, item["product_id"], item["quantity"], item["price"]]
        )

        # Decrease stock
        tx.execute(
          "UPDATE products SET stock = stock - ? WHERE id = ?",
          [item["quantity"], item["product_id"]]
        )
      end
    end

    response.json({ order_id: order_id, status: "created" }, 201)
  rescue => e
    response.json({ error: "Order failed: #{e.message}" }, 500)
  end
end
```

If any step inside the block fails, the transaction is automatically rolled back. If the block completes without error, the transaction is automatically committed. The database never sits in a half-finished state.

---

7. Schema Inspection

Tina4 provides methods to inspect your database structure at runtime:

tables

```
db = Tina4.database
tables = db.tables
```

Returns an array of table names:

```
["orders", "order_items", "products", "users"]
```

columns

```
columns = db.columns("products")
```

Returns an array of column definitions:

```
[
  { "name" => "id", "type" => "INTEGER", "nullable" => false, "primary" => true },
  { "name" => "name", "type" => "TEXT", "nullable" => false, "primary" => false },
  { "name" => "price", "type" => "REAL", "nullable" => true, "primary" => false },
  { "name" => "in_stock", "type" => "INTEGER", "nullable" => true, "primary" => false }
]
```

table_exists?

```
if db.table_exists?("products")
  # Table exists, safe to query
end
```

Returns `true` if the table exists, `false` otherwise.

driver_name

```
db.driver_name
# "sqlite", "postgres", "mysql", "mssql", or "firebird"
```

Returns a lowercase string identifying the active database engine. This matters when you write engine-specific SQL in a multi-database setup.

A Schema Info Endpoint

```
Tina4::Router.get("/api/schema") do |request, response|
  db = Tina4.database
  tables = db.tables

  schema = {}
  tables.each do |table|
    schema[table] = db.columns(table)
  end

  response.json({ tables: schema })
end
```

---

8. Batch Operations with execute_many

Insert or update many rows efficiently:

```
db = Tina4.database

params_list = [
  ["Widget A", 9.99],
  ["Widget B", 14.99],
  ["Widget C", 19.99],
  ["Widget D", 24.99]
]

db.execute_many(
  "INSERT INTO products (name, price) VALUES (?, ?)",
  params_list
)
```

`execute_many` prepares the statement once and executes it for each item in the array. This is significantly faster than calling `execute` in a loop because the SQL only needs to be parsed once.

---

9. Helper Methods: insert, update, delete

Tina4 provides shorthand methods so you do not have to write SQL for simple operations.

insert

```
db = Tina4.database

# Insert a single row
db.insert("products", {
  name: "Wireless Mouse",
  price: 34.99,
  in_stock: 1
})

# Insert multiple rows
db.insert("products", [
  { name: "USB Cable", price: 9.99, in_stock: 1 },
  { name: "HDMI Cable", price: 14.99, in_stock: 1 },
  { name: "DisplayPort Cable", price: 19.99, in_stock: 0 }
])
```

update

```
# Update rows matching a filter
db.update("products", { price: 39.99, in_stock: 1 }, { id: 7 })
```

The second argument is the data to set, and the third is a filter hash for the WHERE clause. Each key-value pair becomes an `AND` condition.

delete

```
# Delete rows matching a filter
db.delete("products", { id: 7 })
```

These helpers generate SQL for you. Convenient for simple CRUD. Raw queries still own complex joins, subqueries, and aggregations.

---

10. Migrations

Migrations are versioned scripts that evolve your schema over time. No manual `CREATE TABLE` statements. Write migration files. Tina4 applies them in order.

Ruby's migration system is unique among Tina4 implementations: it supports both `.sql` and `.rb` migration files.

File Naming

Two naming patterns are accepted:

Pattern	Example
Sequential	`000001_create_products_table.sql`
Timestamp	`20260324120000_create_products_table.sql`

Pick one pattern and stick with it. Do not mix sequential and timestamp naming in the same project.

Generating a Migration

Use the CLI to scaffold migration files:

```
tina4 generate migration create_products_table
```

```
Created migration: migrations/20260324120000_create_products_table.sql
Created migration: migrations/20260324120000_create_products_table.down.sql
```

The generator creates both the up and down files. The timestamp prefix ensures migrations always run in chronological order.

SQL Migrations

Edit the generated file `migrations/20260324120000_create_products_table.sql`:

```
-- migrations/20260324120000_create_products_table.sql
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    category TEXT NOT NULL DEFAULT 'Uncategorized',
    price REAL NOT NULL DEFAULT 0.00,
    in_stock INTEGER NOT NULL DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

The down migration goes in the separate `.down.sql` file:

```
-- migrations/20260324120000_create_products_table.down.sql
DROP TABLE IF EXISTS products;
```

The `.down.sql` file is optional. If it does not exist, rollback for that migration is skipped.

The SQL parser supports `$$` delimited stored procedures and block comments, so you can include complex database objects in a single migration file.

Ruby Class Migrations

As an alternative to SQL files, you can write migrations as Ruby classes. This is useful when you need conditional logic, loops, or data transformations during a migration:

```
# migrations/20260324120000_create_products_table.rb
class CreateProductsTable < Tina4::MigrationBase
  def up
    execute "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT NOT NULL, price REAL DEFAU
  end

  def down
```

```
    execute "DROP TABLE IF EXISTS products"
  end
end
```

For `.rb` migrations, the `down` method handles rollback directly in the same file. No separate `.down.sql` file is needed.

Running Migrations

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260324120000_create_products_table.sql
Migrations complete. 1 applied.
```

Checking Migration Status

```
tina4 migrate:status
```

```
Migration                                    Status     Applied At
---------                                    ------     ----------
20260324120000_create_products_table.sql      applied    2026-03-24 12:00:00
20260324130000_create_orders_table.sql        pending    -
```

Rolling Back

```
tina4 migrate:rollback
```

```
Rolling back last batch...
  [ROLLED BACK] 20260324130000_create_orders_table.sql
Rollback complete. 1 rolled back.
```

Rollback undoes the entire last batch of migrations. If you applied three migrations in one `tina4 migrate` run, all three are rolled back together. You can configure the number of steps if you need finer control.

For `.sql` migrations, rollback runs the corresponding `.down.sql` file. For `.rb` migrations, rollback calls the `down` method.

A Real Migration Sequence

Here is what a typical project's migrations look like:

```
migrations/
■■■ 20260324120000_create_products_table.sql
■■■ 20260324120000_create_products_table.down.sql
■■■ 20260324121000_create_users_table.sql
■■■ 20260324121000_create_users_table.down.sql
■■■ 20260324122000_create_orders_table.rb
■■■ 20260325091500_add_email_index_to_users.sql
```

Notice the mix of `.sql` and `.rb` files. Both types can coexist in the same project. The `create_orders_table.rb` file contains both `up` and `down` methods, so it does not need a separate down file.

The last SQL migration might look like:

```
-- migrations/20260325091500_add_email_index_to_users.sql
CREATE INDEX idx_users_email ON users (email);
```

With its down file:

```
-- migrations/20260325091500_add_email_index_to_users.down.sql
DROP INDEX IF EXISTS idx_users_email;
```

Migration Tracking

Migrations run in filename order. Each migration runs once. Tina4 tracks applied migrations in a `tina4_migration` table with the following columns:

Column	Description
`id`	Auto-increment primary key
`migration_name`	The filename of the migration
`batch`	The batch number (all migrations applied in one run share a batch)
`executed_at`	Timestamp of when the migration was applied

The batch system is what makes rollback work: `migrate:rollback` undoes all migrations in the highest batch number.

---

11. Query Caching

For read-heavy applications, enable query caching:

```
TINA4_DB_CACHE=true
```

When enabled, Tina4 caches the results of `fetch` and `fetch_one` calls. Identical queries with identical parameters return cached results instead of hitting the database again.

The cache is automatically invalidated when you call `execute`, `insert`, `update`, or `delete` on the same table.

You can also control caching per-query:

```
# Force a fresh query (bypass cache)
products = db.fetch("SELECT * FROM products", [], false) # third arg = use cache

# Clear the entire cache
db.cache_clear
```

---

12. Exercise: Build a Notes App

Build a notes application backed by SQLite. Create the database table via a migration and build a full CRUD API.

Requirements

- Create a migration that creates a ~~`notes` table with columns~~:

- `id` -- integer, primary key, auto-increment
- `title` -- text, not null
- `content` -- text, not null
- `tag` -- text, default "general"
- `created_at` -- text, default current timestamp
- `updated_at` -- text, default current timestamp
- Build these API endpoints:

Method	Path	Description
`GET`	`/api/notes`	List all notes. Support `?tag=` and `?search=` filters.
`GET`	`/api/notes/{id:int}`	Get a single note. 404 if not found.
`POST`	`/api/notes`	Create a note. Validate title and content are not empty.
`PUT`	`/api/notes/{id:int}`	Update a note. 404 if not found.
`DELETE`	`/api/notes/{id:int}`	Delete a note. 204 on success, 404 if not found.

Test with:

```
# Create
curl -X POST http://localhost:7147/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Shopping List", "content": "Milk, eggs, bread", "tag": "personal"}'

# List all
curl http://localhost:7147/api/notes

# Search
curl "http://localhost:7147/api/notes?search=shopping"

# Filter by tag
curl "http://localhost:7147/api/notes?tag=personal"

# Get one
curl http://localhost:7147/api/notes/1

# Update
curl -X PUT http://localhost:7147/api/notes/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Shopping List", "content": "Milk, eggs, bread, butter"}'

# Delete
curl -X DELETE http://localhost:7147/api/notes/1
```

---

13. Solution

Migration

Generate the migration:

```
tina4 generate migration create_notes_table
```

Edit `migrations/20260324120000_create_notes_table.sql`:

```
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    tag TEXT NOT NULL DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

Edit `migrations/20260324120000_create_notes_table.down.sql`:

```
DROP TABLE IF EXISTS notes;
```

Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_notes_table.sql
Migrations complete. 1 applied.
```

Routes

Create `src/routes/notes.rb`:

```
# List all notes with optional filters
Tina4::Router.get("/api/notes") do |request, response|
  db = Tina4.database

  tag = request.params["tag"] || ""
  search = request.params["search"] || ""

  sql = "SELECT * FROM notes"
  params = []
  conditions = []

  unless tag.empty?
    conditions << "tag = ?"
    params << tag
  end

  unless search.empty?
    conditions << "(title LIKE ? OR content LIKE ?)"
    params << "%#{search}%"
    params << "%#{search}%"
  end

  sql += " WHERE #{conditions.join(' AND ')}" unless conditions.empty?
```

```
  sql += " ORDER BY updated_at DESC"

  notes = db.fetch(sql, params)

  response.json({
    notes: notes,
    count: notes.length
  })
end

# Get a single note
Tina4::Router.get("/api/notes/{id:int}") do |request, response|
  db = Tina4.database
  id = request.params["id"]

  note = db.fetch_one("SELECT * FROM notes WHERE id = ?", [id])

  if note.nil?
    response.json({ error: "Note not found", id: id }, 404)
  else
    response.json(note)
  end
end

# Create a note
Tina4::Router.post("/api/notes") do |request, response|
  db = Tina4.database
  body = request.body

  # Validate
  errors = []
  errors << "Title is required" if body["title"].nil? || body["title"].empty?
  errors << "Content is required" if body["content"].nil? || body["content"].empty?

  unless errors.empty?
    return response.json({ errors: errors }, 400)
  end

  db.execute(
    "INSERT INTO notes (title, content, tag) VALUES (?, ?, ?)",
    [body["title"], body["content"], body["tag"] || "general"]
  )

  note = db.fetch_one("SELECT * FROM notes WHERE id = last_insert_rowid()")

  response.json(note, 201)
end

# Update a note
Tina4::Router.put("/api/notes/{id:int}") do |request, response|
  db = Tina4.database
  id = request.params["id"]
  body = request.body

  existing = db.fetch_one("SELECT * FROM notes WHERE id = ?", [id])

  if existing.nil?
    return response.json({ error: "Note not found", id: id }, 404)
  end
```

```
  db.execute(
    "UPDATE notes SET title = ?, content = ?, tag = ?, updated_at = CURRENT_TIMESTAMP WHERE id =
    [
      body["title"] || existing["title"],
      body["content"] || existing["content"],
      body["tag"] || existing["tag"],
      id
    ]
  )

  note = db.fetch_one("SELECT * FROM notes WHERE id = ?", [id])

  response.json(note)
end

# Delete a note
Tina4::Router.delete("/api/notes/{id:int}") do |request, response|
  db = Tina4.database
  id = request.params["id"]

  existing = db.fetch_one("SELECT * FROM notes WHERE id = ?", [id])

  if existing.nil?
    return response.json({ error: "Note not found", id: id }, 404)
  end

  db.execute("DELETE FROM notes WHERE id = ?", [id])

  response.json(nil, 204)
end
```

Expected output for create:

```
{
  "id": 1,
  "title": "Shopping List",
  "content": "Milk, eggs, bread",
  "tag": "personal",
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

(Status: `201 Created`)

Expected output for list:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

Expected output for search:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

Expected output for validation error:

```
{"errors": ["Title is required", "Content is required"]}
```

(Status: `400 Bad Request`)

---

14. Seeder -- Generating Test Data

Testing with an empty database tells you nothing. Testing with hand-typed rows is slow and brittle. The `FakeData` class generates realistic test data. `seed_orm` and `seed_table` insert it in bulk.

FakeData

```
fake = Tina4::FakeData.new

fake.name        # "Grace Lopez"
fake.email       # "bob.anderson123@demo.net"
fake.phone       # "+1 (547) 382-9104"
fake.sentence    # "Magna exercitation lorem ipsum dolor sit."
fake.paragraph   # Three sentences of filler text
fake.integer     # 7342
fake.numeric     # 481.29
fake.date        # "2023-07-14"
fake.uuid        # "a3f1b2c4-d5e6-f7a8-b9c0-d1e2f3a4b5c6"
fake.address     # "742 Oak Avenue"
fake.boolean     # 0 or 1
fake.city        # "Tokyo"
fake.country     # "Germany"
fake.company     # "TechGlobal Inc"
fake.url         # "https://example.com/some-random-slug"
fake.password    # "xK9mPq2nRt5vWz8a"
```

Every method draws from built-in word banks -- no network calls, no external packages.

Deterministic Output

Pass a seed to get reproducible results. The same seed produces the same sequence every time:

```
fake = Tina4::FakeData.new(seed: 42)
fake.name   # Always the same name with seed 42
fake.email  # Always the same email with seed 42
```

Deterministic data means deterministic assertions. This matters for tests.

Seeding with ORM

`seed_orm` combines `FakeData` with your ORM models. It reads the model's field definitions and generates appropriate data for each column:

```
Tina4.seed_orm(User, count: 100)
```

This inserts 100 rows into the `users` table. `FakeData` inspects each column name and type to generate matching data -- email columns get email addresses, phone columns get phone numbers, name columns get names.

Overrides

Static values or custom generators apply through the `overrides` hash:

```
Tina4.seed_orm(User, count: 50, overrides: {
  role: "member",
  active: 1,
  email: ->(fake) { fake.email }
})
```

Every row gets `role = "member"` and `active = 1`. Lambda values receive the `FakeData` instance for custom generation.

Seeding a Raw Table

When you have no ORM class, use `seed_table` with a column map:

```
Tina4.seed_table("users", {
  name: :string,
  email: :string,
  phone: :string,
  bio: :text,
  age: :integer
}, count: 100)
```

The column types tell `FakeData` what kind of data to generate for each field.

Batch Seeding

Use `seed_models` when the models have relationships. It topologically orders parents before children and clears in reverse order, regardless of the order supplied:

```
Tina4.seed_models([Order, User], count: 20, clear: true, seed: 42)
```

For an explicitly ordered batch, use `seed_batch`. Pass `clear: true` to delete existing records first:

```
Tina4.seed_batch([
  { orm_class: User, count: 20 },
  { orm_class: Order, count: 100, overrides: { status: "pending" } }
], clear: true)
```

`seed_orm` and `seed_models` create and seed their own `FakeData`. `seed_table` uses caller-provided generators, so create `Tina4::FakeData.new(seed: 42)` and close over that instance when deterministic raw-table data is required.

FakeData is suitable for tests, development, demos, and load data. It is not cryptographically secure: never use it to create production passwords, tokens, API keys, or other secrets.

Seed Files

Place seed scripts in a `seeds/` folder. Each file is a Ruby script that runs `seed_orm` or `seed_table`:

```
# seeds/001_users.rb
Tina4.seed_orm(User, count: 50, seed: 1)

# seeds/002_orders.rb
Tina4.seed_orm(Order, count: 200, overrides: {
  status: ->(f) { f.choice(%w[pending shipped delivered]) }
})
```

Run all seed files:

```
Tina4.seed(seed_folder: "seeds")
```

Files run in alphabetical order. Prefix with numbers to control sequence. Files starting with `_` are skipped.

When to Use It

- Populating a development database with realistic data
- Writing integration tests that need rows in the database
- Load testing with thousands of records
- Demos and screenshots that look real without using real data

---

15. Gotchas

1. Forgetting the transaction block

Problem: You run multiple related queries but they are not atomic -- some succeed while others fail, leaving the database in an inconsistent state.

Cause: Without wrapping them in a `transaction` block, each query is committed independently.

Fix: Use `db.transaction { |tx| ... }` to wrap related operations. The block automatically commits on success and rolls back on any exception.

2. Connection String Formats

Problem: The database will not connect and you see a cryptic error about the connection string.

Cause: Each database engine expects a specific URL format. A common mistake is using `mysql://user:pass@host/db` when the engine expects the port.

Fix: Always include the port: `mysql://localhost:3306/mydb`. Here are the default ports:

Engine	Default Port
PostgreSQL	5432
MySQL	3306
MSSQL	1433
Firebird	3050
SQLite	(file path, no port)

3. SQLite File Paths

Problem: SQLite creates a new empty database instead of using the existing one.

Cause: The path in `TINA4_DATABASE_URL` is relative and resolves to the wrong directory, or you used `sqlite://` (two slashes) instead of `sqlite:///` (three slashes).

Fix: Use three slashes for a relative path: `sqlite:///data/app.db`. For an absolute path, use four slashes: `sqlite:////var/data/app.db`. The third slash separates the scheme from the path; the fourth starts the absolute path.

4. Parameterised Queries with LIKE

Problem: `WHERE name LIKE ?` with `["%search%"]` works, but `WHERE name LIKE '%?%'` does not.

Cause: Parameters inside quotes are treated as literal text, not as placeholders.

Fix: Include the `%` wildcards in the parameter value, not in the SQL: `["%#{search}%"]`. The SQL should be `WHERE name LIKE ?`.

5. Boolean Values in SQLite

Problem: You insert `true` or `false` but the database stores `1` or `0`. When you read it back, you get integers, not booleans.

Cause: SQLite does not have a native boolean type. It stores booleans as integers.

Fix: Cast in your Ruby code: `row["in_stock"] == 1` or use `!!row["in_stock"]` to convert to a boolean. Ruby treats `0` as truthy (unlike some other languages), so be explicit with comparisons.

6. Migration Already Applied

Problem: You edited a migration file and ran `tina4 migrate` again, but nothing changed.

Cause: Tina4 tracks applied migrations by filename in the `tina4_migration` table. Once applied, a migration will not run again even if you change its contents.

Fix: Create a new migration for schema changes. Do not edit applied migrations. If you are in early development and want to start fresh, use `tina4 migrate:rollback` to undo the last batch and then `tina4 migrate` to reapply. Use `tina4 migrate:status` to see which migrations are applied and which are pending.

7. fetch Returns Empty Array, Not Nil

Problem: You check `if result.nil?` but it never matches, even when the table is empty.

Cause: `fetch` always returns an array. An empty result is `[]`, not `nil`. Only `fetch_one` returns `nil` when no row matches.

Fix: Check with `if result.empty?` or `if result.length == 0`.

8. SQL Injection Through String Interpolation

Problem: Your application is vulnerable to SQL injection attacks.

Cause: You used string interpolation to build SQL queries with user input: `"WHERE name = '#{name}'"`.

Fix: Use parameterised queries: `"WHERE name = ?", [name]`. This is the single most important security practice for database code. Tina4 handles escaping and quoting for you.

ORM

1. From SQL to Objects

The last chapter was raw SQL. It works. It also gets repetitive. Every insert demands an INSERT statement. Every update demands an UPDATE. Every fetch maps column names to hash keys. Over and over.

Tina4's ORM turns database rows into Ruby objects. Define a model class with fields. The ORM writes the SQL. It stays SQL-first -- you can drop to raw SQL at any moment -- but for the 90% case of CRUD operations, the ORM handles the grunt work.

Picture a blog. Authors, posts, comments. Authors own many posts. Posts own many comments. Comments belong to posts. Modeling these relationships with raw SQL means JOINs and manual foreign key management. The ORM makes this declarative.

ORM at a Glance: Four Languages, One Shape

The ORM does the same job in every Tina4 book. Define a model. Save it. Query it. Each language wears its own clothes (PHP uses typed properties, Python uses field class instances, Ruby uses a DSL, Node uses config objects) but the operations line up. If you know the API in one book, you can read the others.

Defining a Model

The same `Post` model with `id`, `title`, `body`, and `created_at`:

Python: field class instances on the class body:

```
from tina4_python.orm import ORM, IntegerField, StringField, DateTimeField

class Post(ORM):
    table_name = "posts"

    id = IntegerField(primary_key=True, auto_increment=True)
    title = StringField(required=True, max_length=200)
    body = StringField(default="")
    created_at = DateTimeField()
```

PHP: native typed properties:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
    public string $tableName = "posts";

    public int $id;
    public string $title;
    public string $body = "";
    public string $createdAt;
```

```
}
```

Ruby: class-level DSL declarations:

```
class Post < Tina4::ORM
  table_name "posts"

  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false, length: 200
  string_field :body, default: ""
  datetime_field :created_at
end
```

Node.js (TypeScript): config objects in a `static fields` block:

```
import { BaseModel } from "tina4-nodejs/orm";

export default class Post extends BaseModel {
  static tableName = "posts";
  static fields = {
    id:        { type: "integer" as const, primaryKey: true, autoIncrement: true },
    title:     { type: "string"  as const, required: true, maxLength: 200 },
    body:      { type: "string"  as const, default: "" },
    createdAt: { type: "datetime" as const },
  };
}
```

Common Query Operations

Same operation, four shapes:

Operation	Python	PHP	Ruby	Node.js
Find by primary key	`Post.find_by_id(1)`	`Post::findById(1)`	`Post.find_by_id(1)`	`Post.findById(1)`
Filter by attributes	`Post.find({"title": "x"})`	`Post::find(["title" => "x"])`	`Post.find(title: "x")`	`Post.find({ title: "x" })`
Raw SQL where clause	`Post.where("title = ?", ["x"])`	`(new Post())->where("title = ?", ["x"])`	`Post.where("title = ?", ["x"])`	`Post.where("title = ?", ["x"])`
Build and save	`Post.create(title="x")`	`Post::create(["title" => "x"])`	`Post.create(title: "x")`	`Post.create({ title: "x" })`
Save an instance	`post.save()`	`$post->save()`	`post.save`	`post.save()`
Fetch every row	`Post.all()`	`(new Post())->all()`	`Post.all`	`Post.all()`

Delete a record	`post.delete()`	`$post->delete()`	`post.delete`	`post.delete()`
Count rows	`Post.count()`	`(new Post())->count()`	`Post.count`	`Post.count()`

A few details worth noting. `find()` takes attribute names and applies the field map; `where()` takes raw SQL and skips translation. PHP needs `(new Post())` for instance methods like `where()` and `all()`; the rest are static. Ruby methods drop the parentheses by convention.

For full detail on field options, relationships, eager loading, soft delete, validation, and Auto-CRUD, read the rest of this chapter, which shows the API for the language of this book.

---

2. Defining a Model

Create a model file in `src/orm/`. Every `.rb` file in that directory is auto-loaded.

Create `src/orm/note.rb`:

```
class Note < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false
  string_field :content, default: ""
  string_field :category, default: "general"
  boolean_field :pinned, default: false
  datetime_field :created_at
  datetime_field :updated_at
end
```

A complete model. Here is what each piece does:

- The table name defaults to the lowercase, pluralised class name (`Note` -> `notes`). Override it with `table_name "my_table"` inside the class body.
- `primary_key: true` on a field marks it as the primary key (defaults to `:id` if none is specified)
- Each field is a DSL declaration that creates a getter and setter on the model

Field Types

Field Type	Ruby Type	SQL Type	Description
`integer_field`	`Integer`	`INTEGER`	Whole numbers
`string_field`	`String`	`VARCHAR(255)`	Text strings
`text_field`	`String`	`TEXT`	Long text
`float_field`	`Float`	`REAL`	Floating-point numbers
`decimal_field`	`Float`	`REAL`	Decimal numbers (precision/scale options)
`numeric_field`	`Float`	`REAL`	Alias for float_field
`boolean_field`	`Integer`	`INTEGER` (0/1)	True/False stored as 0/1
`date_field`	`String`	`DATE`	Date values
`datetime_field`	`String`	`DATETIME`	Date and time
`timestamp_field`	`String`	`TIMESTAMP`	Timestamps
`blob_field`	`String`	`BLOB`	Binary data
`json_field`	`String`	`TEXT`	JSON stored as text

For foreign keys, use `integer_field`. There is no separate foreign key field type -- the relationship is defined through `has_many`, `has_one`, and `belongs_to` declarations instead.

Field Options

Option	Type	Description
`primary_key`	`bool`	Marks this field as the primary key
`auto_increment`	`bool`	Auto-incrementing integer
`nullable`	`bool`	Whether the field accepts nil (default: `true`)
`default`	any	Default value when not provided
`length`	`int`	String length for `string_field` (default: 255)
`precision`	`int`	Decimal precision for `decimal_field`
`scale`	`int`	Decimal scale for `decimal_field`

Field Mapping

When your Ruby attribute names do not match the database column names, use `field_mapping` to define the translation. `field_mapping` is a hash that maps Ruby attribute names to DB column names.

```
class User < Tina4::ORM
  self.field_mapping = {
    "first_name"    => "fname",      # Ruby attr -> DB column
    "last_name"     => "lname",
    "email_address" => "email"
  }

  integer_field :id, primary_key: true, auto_increment: true
  string_field :first_name, nullable: false
  string_field :last_name, nullable: false
  string_field :email_address, nullable: false

  table_name "user_accounts"
end
```

With this mapping, `user.first_name` reads from and writes to the `fname` column. The ORM handles the conversion in both directions -- on reads via `from_hash` and on writes via `to_db_hash`. This is useful with legacy databases or third-party schemas where you cannot rename the columns.

A common use case is Firebird or Oracle, which store column names in uppercase:

```
class Account < Tina4::ORM
  self.table_name   = "ACCOUNTS"
```

```
  self.field_mapping = {
    "id"           => "CUST_ID",
    "account_no"   => "ACCOUNTNO",
    "store_name"   => "STORENAME",
    "credit_limit" => "CREDITLIMIT",
  }

  integer_field :id, primary_key: true, auto_increment: true
  string_field :account_no
  string_field :store_name
  float_field :credit_limit, default: 0.0
end
```

Ruby code uses clean snake_case names (`account.account_no`, `account.credit_limit`). The ORM maps them to the uppercase DB columns automatically.

find() vs where() -- naming convention

The two query methods have a deliberate difference in how they handle column names:

- `find(filter)` uses **Ruby attribute names**. The ORM translates them via `field_mapping`.
- `where(conditions, params)` uses **raw DB column names** in the SQL string. No translation is done.

```
# find() -- use Ruby attribute names
accounts = Account.find(account_no: "A001")   # translates to ACCOUNTNO = ?

# where() -- use DB column names directly in the SQL
accounts = Account.where("ACCOUNTNO = ?", ["A001"])  # raw SQL, no translation
```

This means `find()` is portable across database engines, while `where()` gives you full control of the SQL.

auto_map and Case Conversion Utilities

The `auto_map` flag exists on the ORM base class for cross-language parity with the PHP and Node.js versions. In Ruby it is a no-op because Ruby convention already uses `snake_case`, which matches database column names.

For cases where you need to convert between naming conventions (for example, when serialising to a camelCase JSON API), two utility methods are available:

```
Tina4.snake_to_camel("first_name")   # "firstName"
Tina4.camel_to_snake("firstName")    # "first_name"
```

---

3. create_table -- Schema from Models

You can create the database table directly from your model definition:

```
Note.create_table
```

This generates and runs the CREATE TABLE SQL based on your field definitions. It is good for development and testing. For production, use migrations (Chapter 5) for version-controlled schema changes.

```
irb
irb> require_relative "app"
irb> Note.create_table
```

---

4. CRUD Operations

save -- Create or Update

```
Tina4::Router.post "/api/notes" do |request, response|
  note = Note.new
  note.title = request.body_parsed["title"]
  note.content = request.body_parsed["content"] || ""
  note.category = request.body_parsed["category"] || "general"
  note.pinned = request.body_parsed["pinned"] || false
  result = note.save

  if result
    response.json({ message: "Note created", note: note.to_h }, status: 201)
  else
    response.json({ errors: note.errors }, status: 422)
  end
end
```

`save` detects whether the record is new (INSERT) or existing (UPDATE) based on whether the primary key has a value and the record is persisted. It returns `self` on success, so you can chain calls. It returns `false` on failure -- check `note.errors` for details.

create -- Build and Save in One Step

When you have a hash of data ready, `create` builds the model and saves it in one call:

```
note = Note.create({
  title: "Quick Note",
  content: "Created in one step",
  category: "general"
})
```

find*by*id -- Fetch One Record by Primary Key

```
Tina4::Router.get "/api/notes/{id:int}" do |request, response|
  note = Note.find_by_id(request.params["id"].to_i)

  if note.nil?
    response.json({ error: "Note not found" }, status: 404)
  else
    response.json(note.to_h)
  end
end
```

`find_by_id` takes a primary key value and returns a model instance, or `nil` if no row matches. If soft delete is enabled, it excludes soft-deleted records.

Use `find_or_fail` when you want an exception raised instead of `nil`:

```
note = Note.find_or_fail(id)  # Raises RuntimeError if not found
```

find -- Query by Filter Hash

The `find` method accepts a hash of column-value pairs and returns an array of matching records:

```
# Find all notes in the "work" category
work_notes = Note.find({ category: "work" })

# Find with pagination and ordering
recent = Note.find({ pinned: true }, limit: 10, order_by: "created_at DESC")

# Find all records (no filter)
all_notes = Note.find
```

Both hash syntax (`find({category: "work"})`) and keyword syntax (`find(category: "work")`) are accepted. Ruby attribute names are used in the filter -- the ORM applies `field_mapping` automatically when translating to SQL column names.

where -- Query with SQL Conditions

For more complex queries, `where` takes a SQL WHERE clause with `?` placeholders:

```
notes = Note.where("category = ?", ["work"])
```

delete -- Remove a Record

```
Tina4::Router.delete "/api/notes/{id:int}" do |request, response|
  note = Note.find_by_id(request.params["id"].to_i)

  if note.nil?
    response.json({ error: "Note not found" }, status: 404)
  else
    note.delete
    response.json(nil, status: 204)
  end
end
```

Listing Records

```
Tina4::Router.get "/api/notes" do |request, response|
  category = request.query["category"]

  if category
    notes = Note.where("category = ?", [category])
  else
    notes = Note.all
  end

  response.json({
    notes: notes.map(&:to_h),
    count: notes.length
  })
end
```

`where` takes a WHERE clause with `?` placeholders and an array of parameters. It returns a `ModelCollection` (covered just below). `all` fetches all records. Both support pagination:

```
# With pagination
notes = Note.where("category = ?", ["work"])
```

```
# Fetch all with pagination and ordering
notes = Note.all(limit: 20, offset: 0, order_by: "created_at DESC")

# SQL-first query -- full control over the SQL
notes = Note.select(
  "SELECT * FROM notes WHERE pinned = ? ORDER BY created_at DESC",
  [1], limit: 20, offset: 0
)
```

ModelCollection -- The Page and the Total

Pagination hides an awkward gap. You ask for 20 rows and you get 20 rows, but a pager needs to say "page 3 of 13", and that means knowing the total number of matching rows, not the 20 sitting on this page. The old answer was a second `COUNT(*)` query with the same filter written out again by hand.

Tina4 closes the gap. `where`, `select`, `find` (the filter form), `all`, and `with_trashed` all return a `ModelCollection` -- a subclass of `Array`, so nothing you already wrote changes. You iterate it, index it, map it, call `.length` on it, and serialise it to JSON exactly as before. It just carries one extra thing: the total for the filter, independent of `limit` and `offset`.

```
rows = User.where("active = ?", [1], limit: 20, offset: 40)  # a page of up to 20 models
rows.get_total_records   # 250 -- the whole matching set, ignoring limit/offset
```

That total is free. Every one of those methods already runs a `COUNT(*)` probe when it fetches the page, and the ORM used to hydrate the models and throw the count away. `ModelCollection` keeps it instead, so `get_total_records` fires no second query. It is a method, not a `count` property, on purpose: `Array#count` already exists, so a `count` accessor would shadow a built-in.

The single-record finders are untouched. `find(pk)`, `find_by_id`, and `select_one` still return one model or `nil`.

Call `to_paginate` for a ready-made pagination envelope. It hands back the same seven keys as `db.fetch(...).to_paginate`, so a route paginates the same way whether the data came through the ORM or through raw SQL:

```
Tina4::Router.get "/api/notes" do |request, response|
  page = Note.where("category = ?", ["work"], limit: 20, offset: 40)
  response.json(page.to_paginate)
  # { "records" => [...20...], "total" => 250, "page" => 3, "per_page" => 20,
  #   "total_pages" => 13, "limit" => 20, "offset" => 40 }
end
```

The keys (`records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset`) are snake_case and identical in all four frameworks, so a client reads the same JSON everywhere.

select_one -- Fetch a Single Record by SQL

When you need exactly one record from a custom SQL query:

```
note = Note.select_one("SELECT * FROM notes WHERE slug = ?", ["my-note"])
```

Returns a model instance or `nil`.

load -- Populate an Existing Instance

The `load` method fills an existing model instance from the database:

```
note = Note.new
note.id = 42
note.load  # Loads data for id=42

# Or with a filter string
note = Note.new
note.load("slug = ?", ["my-note"])
```

Returns `true` if a record was found, `false` otherwise.

count -- Count Records

```
total = Note.count
work_count = Note.count("category = ?", ["work"])
```

Respects soft delete -- only counts non-deleted records.

---

5. to*h, to*dict, to_json, and Other Serialisation

to*h and to*dict

Convert a model instance to a hash. `to_dict` is a direct alias for `to_h` -- use whichever reads more naturally in your code:

```
note = Note.find_by_id(1)

data = note.to_h
# {id: 1, title: "Shopping List", content: "Milk, eggs", category: "personal", pinned: false, cr

# to_dict is identical
data = note.to_dict
```

The `include` parameter adds relationship data to the output (see Eager Loading below). Pass an array of relationship names:

```
# Include relationships in the hash
data = note.to_h(include: [:comments])
data = note.to_dict(include: [:comments])  # same result
```

to_json

Convert directly to a JSON string:

```
json_string = note.to_json
# '{"id":1,"title":"Shopping List",...}'
```

Other Serialisation Methods

Method	Returns	Description
`to_h(include: nil)`	`Hash`	Primary hash method with optional relationship includes
`to_hash(include: nil)`	`Hash`	Alias for `to_h`
`to_dict(include: nil)`	`Hash`	Alias for `to_h`
`to_assoc(include: nil)`	`Hash`	Alias for `to_h`
`to_object`	`Hash`	Alias for `to_h`
`to_json(include: nil)`	`String`	JSON string
`to_array`	`Array`	Flat list of values (no keys)
`to_list`	`Array`	Alias for `to_array`

---

6. Relationships

Tina4 Ruby supports two styles of relationships: declarative (class-level DSL) and imperative (instance-level method calls). Both produce the same queries. Declarative relationships enable eager loading; imperative relationships are ad-hoc.

foreign*key*field: Auto-Wired Relationships

Declaring a column with `foreign_key_field :user_id, references: User` automatically wires both sides of the relationship. The declaring class gets a `belongs_to` accessor (the column name with `_id` stripped), and the referenced class gets a `has_many` accessor (the declaring class name lowercased with `s` appended, or whatever you pass via `related_name:`).

```
class User < Tina4::ORM
  table_name "users"
  integer_field :id, primary_key: true
  string_field :name
end

class Post < Tina4::ORM
  table_name "posts"
  integer_field :id, primary_key: true
  string_field :title

  # Auto-wires post.user (belongs_to) and user.posts (has_many)
  foreign_key_field :user_id, references: User
end
```

With just the `foreign_key_field` declaration, both sides are accessible:

```
post = Post.find_by_id(1)
puts post.user.name        # "Alice"
```

```
user = User.find_by_id(1)
user.posts.each do |p|
  puts p.title
end
```

For a custom `has_many` name, pass `related_name:`:

```
foreign_key_field :user_id, references: User, related_name: :blog_posts
# user.blog_posts instead of user.posts
```

If the referenced class is defined later, the framework handles deferred wiring: as soon as the referenced class applies its own field definitions, the `has_many` is injected.

Declarative Relationships

Define relationships at the class level. The ORM creates accessor methods on each instance.

has_many

An author has many posts:

Create `src/orm/author.rb`:

```
class Author < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :name, nullable: false
  string_field :email, nullable: false
  string_field :bio, default: ""
  datetime_field :created_at

  has_many :posts, class_name: "BlogPost", foreign_key: "author_id"
end
```

Create `src/orm/blog_post.rb`:

```
class BlogPost < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  integer_field :author_id, nullable: false
  string_field :title, nullable: false
  string_field :slug, nullable: false
  string_field :content, default: ""
  string_field :status, default: "draft"
  datetime_field :created_at
  datetime_field :updated_at

  table_name "posts"

  belongs_to :author, class_name: "Author", foreign_key: "author_id"
end
```

Now access an author's posts:

```
Tina4::Router.get "/api/authors/{id:int}" do |request, response|
  author = Author.find_by_id(request.params["id"].to_i)

  if author.nil?
    response.json({ error: "Author not found" }, status: 404)
  else
    posts = author.posts  # Calls the has_many accessor

    data = author.to_h
```

```
    data[:posts] = posts.map(&:to_h)
    response.json(data)
  end
end
```

has_one

A user has one profile:

```
class User < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :name, nullable: false

  has_one :profile, class_name: "Profile", foreign_key: "user_id"
end

profile = user.profile  # Returns a single instance or nil
```

belongs_to

A post belongs to an author:

```
Tina4::Router.get "/api/posts/{id:int}" do |request, response|
  post = BlogPost.find_by_id(request.params["id"].to_i)

  if post.nil?
    response.json({ error: "Post not found" }, status: 404)
  else
    author = post.author  # Calls the belongs_to accessor

    data = post.to_h
    data[:author] = author&.to_h
    response.json(data)
  end
end
```

Imperative Relationships

For ad-hoc queries without class-level declarations, use `query_has_many`, `query_has_one`, and `query_belongs_to` on any instance:

```
# Same result as declarative has_many, but without a class-level declaration
posts = author.query_has_many(BlogPost, foreign_key: "author_id")

# Has one
profile = user.query_has_one(Profile, foreign_key: "user_id")

# Belongs to
author = post.query_belongs_to(Author, foreign_key: "author_id")
```

These work identically to the declarative accessors but do not support eager loading.

---

7. Eager Loading

Calling relationship methods inside a loop creates the N+1 problem. Load 10 authors. Call `author.posts` for each one. That fires 11 queries -- 1 for authors, 10 for posts. The page drags.

The `include` parameter on `all`, `where`, `find`, and `select` solves this. It eager-loads relationships in bulk:

```
Tina4::Router.get "/api/authors" do |request, response|
  authors = Author.all(include: ["posts"])

  data = authors.map do |author|
    author_dict = author.to_h
    author_dict[:posts] = author.posts.map(&:to_h)
    author_dict
  end

  response.json({ authors: data })
end
```

Without eager loading, 10 authors and their posts cost 11 queries. With eager loading: 2 queries. That is the difference between a fast page and a slow one.

Nested Eager Loading

Dot notation loads multiple levels deep:

```
# Load authors, their posts, and each post's comments
authors = Author.all(include: ["posts", "posts.comments"])
```

Authors, their posts, and each post's comments. Three queries total instead of hundreds.

to_h with Nested Includes

When eager loading is active, `to_h(include: ...)` embeds the related data:

```
post = BlogPost.find_by_id(1)
# Manually trigger eager load first, or use select with include
posts = BlogPost.select(
  "SELECT * FROM posts WHERE id = ?", [1],
  include: ["author", "comments"]
)
post = posts.first
data = post.to_h(include: ["author", "comments"])
```

```
{
  "id": 1,
  "title": "Getting Started with Tina4",
  "author": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  },
  "comments": [
    {"id": 1, "body": "Great post!", "author_name": "Bob"}
  ]
}
```

---

8. Soft Delete

Sometimes a record needs to disappear from queries without leaving the database. Soft delete handles this. The row stays. A flag marks it as deleted. Queries skip it.

```
class Task < Tina4::ORM
  self.soft_delete = true  # Enable soft delete

  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false
  boolean_field :completed, default: false
  integer_field :is_deleted, default: 0  # Required for soft delete (0 = active, 1 = deleted)
  string_field :created_at
end
```

When `soft_delete` is set to `true`, the ORM changes its behaviour:

- `task.delete` sets `is_deleted` to `1` instead of running a DELETE query
- `Task.all`, `Task.where`, and `Task.find_by_id` filter out records where `is_deleted = 1`
- `task.restore` sets `is_deleted` back to `0` and makes the record visible again
- `task.force_delete` permanently removes the row from the database
- `Task.with_trashed` includes soft-deleted records in query results

Deleting and Restoring

```
# Soft delete -- sets is_deleted = 1, row stays in the database
task = Task.find_by_id(1)
task.delete

# Restore -- sets is_deleted = 0, record is visible again
task.restore

# Permanently delete -- removes the row, no recovery possible
task.force_delete
```

`restore` is the inverse of `delete`. It sets `is_deleted` back to `0` and commits the change. The record reappears in all standard queries.

Including Soft-Deleted Records

Standard queries (`all`, `where`, `find_by_id`) exclude soft-deleted records. When you need to see everything -- for admin dashboards, audit logs, or data recovery -- use `with_trashed`:

```
# All tasks, including soft-deleted ones
all_tasks = Task.with_trashed

# Soft-deleted tasks matching a condition
deleted_tasks = Task.with_trashed("completed = ?", [1])
```

`with_trashed` accepts the same filter parameters as `where`. The only difference: it ignores the `is_deleted` filter that standard queries apply.

Counting with Soft Delete

The `count` class method respects soft delete. It only counts non-deleted records:

```
active_count = Task.count
active_work = Task.count("category = ?", ["work"])
```

Custom Soft Delete Field

By default, the ORM uses `:is_deleted` as the soft delete column. You can change this:

```
class Task < Tina4::ORM
  self.soft_delete = true
  self.soft_delete_field = :deleted_flag

  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false
  integer_field :deleted_flag, default: 0
end
```

When to Use Soft Delete

Soft delete suits data that users might want to recover -- emails, documents, user accounts. It also serves audit requirements where regulations demand retention. For temporary data (sessions, cache entries, logs), hard delete keeps the table lean.

---

9. Auto-CRUD

Writing the same five REST endpoints for every model gets tedious. Auto-CRUD generates them from your model class. Define the model. Register it. Five routes appear.

The auto_crud Flag

The simplest approach -- set `self.auto_crud = true` on your model class:

```
class Note < Tina4::ORM
  self.auto_crud = true  # Generates REST endpoints automatically

  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false
  string_field :content, default: ""
end
```

The moment Ruby loads this class, the ORM registers it with `AutoCrud`. Five routes appear after `Tina4::AutoCrud.generate_routes` is called.

Manual Registration

You can also register models explicitly using `AutoCrud.register`:

```
Tina4::AutoCrud.register(Note)
```

Then generate all routes at once:

```
Tina4::AutoCrud.generate_routes(prefix: "/api")
```

Both approaches produce the same result:

Method	Path	Description

`GET`	`/api/notes`	List all with pagination (`limit`, `offset`, `page`, `per_page` params)
`GET`	`/api/notes/{id}`	Get one by primary key
`POST`	`/api/notes`	Create a new record
`PUT`	`/api/notes/{id}`	Update a record
`DELETE`	`/api/notes/{id}`	Delete a record

The endpoint prefix derives from the table name. The `notes` table becomes `/api/notes`. Pass a custom prefix to change it:

```
Tina4::AutoCrud.generate_routes(prefix: "/api/v2")
# Routes: /api/v2/notes, /api/v2/notes/{id}, etc.
```

What the Generated Routes Do

GET /api/notes returns paginated results with optional filtering and sorting:

```
curl "http://localhost:7147/api/notes?limit=10&offset=0"
curl "http://localhost:7147/api/notes?filter[category]=work&sort=-created_at"
```

```
{
  "data": [
    {"id": 1, "title": "Shopping List", "content": "Milk, eggs", "category": "personal", "pinned
    {"id": 2, "title": "Sprint Plan", "content": "Review backlog", "category": "work", "pinned":
  ],
  "total": 2,
  "limit": 10,
  "offset": 0
}
```

Sorting uses the `sort` parameter. Prefix a field with `-` for descending order: `?sort=-created_at,name`.

Filtering uses `filter[field]=value` syntax: `?filter[category]=work&filter[pinned]=true`.

POST /api/notes validates input before saving:

```
curl -X POST http://localhost:7147/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "New Note", "content": "Created via auto-CRUD"}'
```

If validation fails (for example, a required field is missing), the endpoint returns a 422 with error details:

```
{"errors": ["title cannot be null"]}
```

DELETE /api/notes/1 respects soft delete. If the model has `self.soft_delete = true`, the record is marked deleted instead of removed.

Custom Routes Alongside Auto-CRUD

If you need special logic for one endpoint (custom validation, side effects, complex queries), define that route yourself and let Auto-CRUD handle the rest. Which one answers depends on the order they register in, because the router follows two rules:

- Registering the **same method and the same path string** again replaces the earlier route, so whichever registers last wins.
- A **different pattern** for the same URL (`/api/notes/{id:int}` against Auto-CRUD's `/api/notes/{id}`) replaces nothing. Both stay registered, and the one registered first answers.

Auto-CRUD routes are added when `Tina4::AutoCrud.generate_routes` runs. Call it after your route files have loaded and a generated route replaces your override on the identical path. The dependable choice is a path of its own for the custom route, or calling `generate_routes` before you define the override.

---

10. Scopes

Scopes are reusable query filters baked into the model. Ruby supports two approaches: instance methods and the `scope` class method.

Class Methods as Scopes

```
class BlogPost < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :title, nullable: false
  string_field :status, default: "draft"
  datetime_field :created_at

  table_name "posts"

  def self.published
    where("status = ?", ["published"])
  end

  def self.drafts
    where("status = ?", ["draft"])
  end

  def self.recent(days = 7)
    where(
      "created_at > datetime('now', ?)",
      ["-#{days} days"]
    )
  end
end
```

Use them in your routes:

```
Tina4::Router.get "/api/posts/published" do |request, response|
  posts = BlogPost.published
  response.json({ posts: posts.map(&:to_h) })
end
```

```
Tina4::Router.get "/api/posts/recent" do |request, response|
  days = (request.query["days"] || 7).to_i
  posts = BlogPost.recent(days)
  response.json({ posts: posts.map(&:to_h) })
end
```

Dynamic Scopes with scope()

Register scopes dynamically with the `scope` class method:

```
BlogPost.scope("active", "status != ?", ["archived"])

# Now call it:
active_posts = BlogPost.active
```

Scopes keep query logic in the model where it belongs. Route handlers stay thin.

---

11. Input Validation

Field definitions carry validation rules through the `nullable` option. Call `validate` before `save` and the ORM checks every constraint:

```
class Product < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :name, nullable: false
  string_field :sku, nullable: false
  float_field :price, nullable: false
  string_field :category
end

Tina4::Router.post "/api/products" do |request, response|
  product = Product.new(request.body_parsed)

  errors = product.validate
  if errors.any?
    response.json({ errors: errors }, status: 400)
  else
    product.save
    response.json({ product: product.to_h }, status: 201)
  end
end
```

If validation fails, `validate` returns an array of error messages:

```
{
  "errors": [
    "name cannot be null",
    "sku cannot be null",
    "price cannot be null"
  ]
}
```

Note that `save` also runs field validation internally and returns `false` if any required fields are missing. Check `model.errors` after a failed save.

---

12. Exercise: Build a Blog with Relationships

Build a blog API with authors, posts, and comments.

Requirements

- Create these models:

Author: `id`, `name` (required), `email` (required), `bio`, `created_at`

Post: `id`, `author_id` (integer foreign key), `title` (required), `slug` (required), `content`, `status` (default: draft), `created_at`, `updated_at`

Comment: `id`, `post_id` (integer foreign key), `author_name` (required), `author_email` (required), `body` (required), `created_at`

- Build these endpoints:

Method	Path	Description
`POST`	`/api/authors`	Create an author
`GET`	`/api/authors/{id:int}`	Get author with their posts
`POST`	`/api/posts`	Create a post (requires author_id)
`GET`	`/api/posts`	List published posts with author info
`GET`	`/api/posts/{id:int}`	Get post with author and comments
`POST`	`/api/posts/{id:int}/comments`	Add comment to a post

---

13. Solution

Create `src/orm/author.rb`:

```
class Author < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  string_field :name, nullable: false
  string_field :email, nullable: false
  string_field :bio, default: ""
  datetime_field :created_at

  has_many :posts, class_name: "BlogPost", foreign_key: "author_id"
end
```

Create `src/orm/blog_post.rb`:

```
class BlogPost < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  integer_field :author_id, nullable: false
```

```
  string_field :title, nullable: false
  string_field :slug, nullable: false
  string_field :content, default: ""
  string_field :status, default: "draft"
  datetime_field :created_at
  datetime_field :updated_at

  table_name "posts"

  belongs_to :author, class_name: "Author", foreign_key: "author_id"
  has_many :comments, class_name: "Comment", foreign_key: "post_id"

  def self.published
    where("status = ?", ["published"])
  end
end
```

Create `src/orm/comment.rb`:

```
class Comment < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  integer_field :post_id, nullable: false
  string_field :author_name, nullable: false
  string_field :author_email, nullable: false
  string_field :body, nullable: false
  datetime_field :created_at

  belongs_to :post, class_name: "BlogPost", foreign_key: "post_id"
end
```

Create `src/routes/blog.rb`:

```
Tina4::Router.post "/api/authors" do |request, response|
  author = Author.new
  author.name = request.body_parsed["name"]
  author.email = request.body_parsed["email"]
  author.bio = request.body_parsed["bio"] || ""

  errors = author.validate
  if errors.any?
    response.json({ errors: errors }, status: 400)
  else
    author.save
    response.json({ author: author.to_h }, status: 201)
  end
end

Tina4::Router.get "/api/authors/{id:int}" do |request, response|
  author = Author.find_by_id(request.params["id"].to_i)

  if author.nil?
    response.json({ error: "Author not found" }, status: 404)
  else
    posts = author.posts

    data = author.to_h
    data[:posts] = posts.map(&:to_h)
    response.json(data)
  end
end
```

```
Tina4::Router.post "/api/posts" do |request, response|
  body = request.body_parsed

  # Verify author exists
  author = Author.find_by_id(body["author_id"])
  if author.nil?
    next response.json({ error: "Author not found" }, status: 404)
  end

  blog_post = BlogPost.new
  blog_post.author_id = body["author_id"]
  blog_post.title = body["title"]
  blog_post.slug = body["slug"]
  blog_post.content = body["content"] || ""
  blog_post.status = body["status"] || "draft"

  errors = blog_post.validate
  if errors.any?
    response.json({ errors: errors }, status: 400)
  else
    blog_post.save
    response.json({ post: blog_post.to_h }, status: 201)
  end
end

Tina4::Router.get "/api/posts" do |request, response|
  posts = BlogPost.published
  data = []

  posts.each do |p|
    post_dict = p.to_h
    post_dict[:author] = p.author&.to_h
    data << post_dict
  end

  response.json({ posts: data, count: data.length })
end

Tina4::Router.get "/api/posts/{id:int}" do |request, response|
  blog_post = BlogPost.find_by_id(request.params["id"].to_i)

  if blog_post.nil?
    response.json({ error: "Post not found" }, status: 404)
  else
    data = blog_post.to_h
    data[:author] = blog_post.author&.to_h
    comments = blog_post.comments
    data[:comments] = comments.map(&:to_h)
    data[:comment_count] = comments.length
    response.json(data)
  end
end

Tina4::Router.post "/api/posts/{id:int}/comments" do |request, response|
  blog_post = BlogPost.find_by_id(request.params["id"].to_i)

  if blog_post.nil?
    next response.json({ error: "Post not found" }, status: 404)
  end
```

```
    comment = Comment.new
    comment.post_id = request.params["id"].to_i
    comment.author_name = request.body_parsed["author_name"]
    comment.author_email = request.body_parsed["author_email"]
    comment.body = request.body_parsed["body"]

    errors = comment.validate
    if errors.any?
      response.json({ errors: errors }, status: 400)
    else
      comment.save
      response.json({ comment: comment.to_h }, status: 201)
    end
  end
```

---

14. Gotchas

1. Forgetting to call save

Problem: You set properties on a model but the database does not change.

Cause: Setting `note.title = "New Title"` only changes the Ruby object. The database remains unchanged until you call `note.save`.

Fix: Call `save` after modifying properties. Check the return value -- `save` returns `self` on success and `false` on failure.

2. find*by*id returns nil

Problem: You call `Note.find_by_id(id)` but get `nil` instead of a note object.

Cause: `find_by_id` returns `nil` when no row matches the given primary key. If soft delete is enabled, `find_by_id` also excludes soft-deleted records.

Fix: Check for `nil` after `find_by_id`: `if note.nil?` and return 404. Use `find_or_fail` if you want an exception raised instead.

3. find takes a hash, not keyword arguments

Problem: You call `Note.find(category: "work")` expecting a filter, but get unexpected results.

Cause: `find` takes a hash argument: `find({category: "work"})`. Passing `category: "work"` as a keyword argument does not filter -- it gets interpreted differently.

Fix: Use `find({column: value})` with an explicit hash. Use `find_by_id(id)` for primary key lookups.

4. to_h includes everything

Problem: `user.to_h` includes `password_hash` in the API response.

Cause: `to_h` includes all fields by default.

Fix: Build the response hash manually, omitting sensitive fields: `{id: user.id, name: user.name, email: user.email}`. Or create a helper method on your model class that returns only safe fields.

5. Validation runs on save, but check errors

Problem: You call `save` and it returns `false`, but you do not know why.

Cause: `save` validates required fields internally. When validation fails, it populates `model.errors` and returns `false`.

Fix: Call `errors = model.validate` before `save` for explicit error messages. Or check `model.errors` after a failed save.

6. Foreign key not enforced

Problem: You save a post with `author_id = 999` and it succeeds, even though no author with ID 999 exists.

Cause: SQLite does not enforce foreign key constraints by default. The ORM defines the relationship through `has_many`/`belongs_to` declarations, but the database itself may not enforce it.

Fix: Enable SQLite foreign keys with `PRAGMA foreign_keys = ON;` in a migration, or validate the foreign key in your route handler before saving.

7. N+1 query problem

Problem: Listing 100 authors with their posts runs 101 queries (1 for authors + 100 for posts), and the page loads slowly.

Cause: You call `author.posts` inside a loop for each author.

Fix: Use eager loading with the `include` parameter on `all`, `where`, or `select`:

```
authors = Author.all(include: ["posts"])
```

Two queries instead of 101.

8. Auto-CRUD endpoint conflicts

Problem: Custom route at `/api/notes/{id}` stops working after registering Auto-CRUD for the Note model.

Cause: Both routes answer the same URL. With the identical path the route registered last replaces the other, so a `generate_routes` call after your route files wins. With different patterns the route registered first answers.

Fix: Give the custom route a path of its own, or call `Tina4::AutoCrud.generate_routes` before you define the override.

9. Soft-deleted records appearing in queries

Problem: You soft-deleted a record, but queries still return it.

Cause: Soft delete requires `self.soft_delete = true` on the model class and an `integer_field :is_deleted, default: 0` field. Without both, soft delete is inactive.

Fix: Verify both the `self.soft_delete = true` flag and the `integer_field :is_deleted, default: 0` field exist on the model. The column stores `0` for active records and `1` for deleted ones.

10. Table name pluralisation

Problem: Your model `Category` maps to `categorys` instead of `categories`.

Cause: The ORM appends `s` by default unless the name already ends in `s`. It does not handle irregular plurals.

Fix: Set the table name explicitly with `table_name "categories"` inside the class body. Disable auto-pluralisation with the `TINA4_ORM_PLURAL_TABLE_NAMES=false` environment variable.

---

15. Raw SQL

The ORM handles 90% of queries. The other 10% need custom SQL -- reports, aggregations, complex joins. Drop down to the database directly:

```
result = Tina4.database.fetch("SELECT * FROM users WHERE active = ?", [1])
result.each { |row| puts row["name"] }
```

`fetch` returns an array of hashes. Column names are the keys. Combine raw SQL with ORM serialisation:

```
rows = Tina4.database.fetch(
  "SELECT u.*, COUNT(p.id) AS post_count FROM users u LEFT JOIN posts p ON p.user_id = u.id GROU
  []
)

rows.each do |row|
  puts "#{row["name"]}: #{row["post_count"]} posts"
end
```

When your raw query returns rows that match a model's shape, you can hydrate them via `from_hash`:

```
rows = Tina4.database.fetch("SELECT * FROM users WHERE active = ?", [1])
users = rows.map { |row| User.from_hash(row) }
users.each { |user| puts user.to_h }
```

`from_hash` applies `field_mapping` during hydration, so DB column names are translated to Ruby attribute names automatically.

---

16. QueryBuilder Integration

ORM models provide a `query` class method that returns a `QueryBuilder` pre-configured with the model's table name and database connection. This gives you a fluent API for building complex queries without writing raw SQL:

```
# Fluent query builder from ORM
results = User.query
  .where("active = ?", [1])
  .order_by("name")
  .limit(50)
  .get

# First matching record
user = User.query
  .where("email = ?", ["alice@example.com"])
  .first

# Count
total = User.query
  .where("role = ?", ["admin"])
  .count

# Check existence
exists = User.query
  .where("email = ?", ["test@example.com"])
  .exists?
```

See the QueryBuilder chapter for the full fluent API including joins, grouping, having, and MongoDB support.

QueryBuilder

1. Why a Query Builder?

Chapter 5 covered raw SQL. Chapter 6 covered the ORM. Both work. But raw SQL is error-prone for dynamic queries -- the more conditions you add, the more string concatenation you juggle. The ORM handles simple CRUD but steps aside for complex filtering, joins, and aggregations.

The QueryBuilder sits between the two. It gives you a fluent, chainable Ruby API that builds SQL for you. You describe *what* you want. It handles the syntax. No string concatenation. No misplaced commas. No forgotten WHERE keywords.

---

2. Creating a QueryBuilder

Every query starts with `Tina4::QueryBuilder.from`:

```
query = Tina4::QueryBuilder.from_table("users", db: db)
```

The first argument is the table name. The `db:` keyword argument is the database connection. If you omit `db:`, the QueryBuilder will fall back to `Tina4.database` when you execute the query -- but it will raise an error if no connection is available.

You can also access the QueryBuilder from an ORM model:

```
query = User.query
```

This creates a QueryBuilder pre-configured with the model's table name and the active database connection.

---

3. Selecting Columns

By default, the QueryBuilder selects all columns (`*`). Use `.select` to pick specific columns:

```
query = Tina4::QueryBuilder.from_table("users", db: db)
  .select("id", "name", "email")
```

Pass as many column names as you need. Each is a separate string argument:

```
query = Tina4::QueryBuilder.from_table("products", db: db)
  .select("name", "price", "category")
```

If you call `.select` with no arguments, the columns remain unchanged.

---

4. Filtering with where

Add conditions with `.where`. Use `?` placeholders for parameters:

```
results = Tina4::QueryBuilder.from_table("users", db: db)
  .where("active = ?", [1])
```

```
  .get
```

The first argument is the SQL condition. The second is an array of parameter values that replace the `?` placeholders. The database driver handles escaping. Your input never touches the SQL string directly.

Multiple where Calls

Chain multiple `.where` calls. They combine with AND:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .where("price > ?", [10.00])
  .where("category = ?", ["Electronics"])
  .get
```

This generates:

```
SELECT * FROM products WHERE price > ? AND category = ?
```

The first `.where` produces the condition directly. Every subsequent `.where` prepends `AND`.

---

5. OR Conditions with or_where

Use `.or_where` when you need OR logic:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .where("category = ?", ["Electronics"])
  .or_where("category = ?", ["Books"])
  .get
```

This generates:

```
SELECT * FROM products WHERE category = ? OR category = ?
```

You can mix `.where` and `.or_where` freely:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .where("price > ?", [10.00])
  .where("in_stock = ?", [1])
  .or_where("featured = ?", [1])
  .get
```

This generates:

```
SELECT * FROM products WHERE price > ? AND in_stock = ? OR featured = ?
```

If you need grouped conditions (parentheses), write the group as a single condition string:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .where("price > ?", [10.00])
  .where("(category = ? OR category = ?)", ["Electronics", "Books"])
  .get
```

---

6. Joins

Inner Join

```
results = Tina4::QueryBuilder.from_table("orders", db: db)
  .select("orders.id", "users.name", "orders.total")
  .join("users", "users.id = orders.user_id")
  .get
```

This generates:

```
SELECT orders.id, users.name, orders.total FROM orders INNER JOIN users ON users.id = orders.use
```

Left Join

```
results = Tina4::QueryBuilder.from_table("users", db: db)
  .select("users.name", "orders.total")
  .left_join("orders", "orders.user_id = users.id")
  .get
```

This generates:

```
SELECT users.name, orders.total FROM users LEFT JOIN orders ON orders.user_id = users.id
```

Multiple Joins

Chain as many joins as you need:

```
results = Tina4::QueryBuilder.from_table("order_items", db: db)
  .select("products.name", "order_items.quantity", "orders.status")
  .join("products", "products.id = order_items.product_id")
  .join("orders", "orders.id = order_items.order_id")
  .where("orders.status = ?", ["completed"])
  .get
```

---

7. Ordering

Use `.order_by` to sort results:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .order_by("price ASC")
  .get
```

Pass the column name and direction as a single string. Chain multiple `.order_by` calls for multi-column sorting:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .order_by("category ASC")
  .order_by("price DESC")
  .get
```

This generates:

```
SELECT * FROM products ORDER BY category ASC, price DESC
```

---

8. Limiting and Offsetting

Use `.limit` to cap the number of rows returned:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .order_by("created_at DESC")
  .limit(10)
  .get
```

Pass a second argument for the offset:

```
# Skip 20 rows, return the next 10
results = Tina4::QueryBuilder.from_table("products", db: db)
  .order_by("created_at DESC")
  .limit(10, 20)
  .get
```

If you do not call `.limit`, the QueryBuilder defaults to 100 rows with an offset of 0 when you call `.get`. This prevents accidental full-table dumps.

---

9. Grouping and Having

Group By

```
results = Tina4::QueryBuilder.from_table("orders", db: db)
  .select("status", "COUNT(*) AS order_count")
  .group_by("status")
  .get
```

This generates:

```
SELECT status, COUNT(*) AS order_count FROM orders GROUP BY status
```

Having

Filter grouped results with `.having`:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .select("category", "AVG(price) AS avg_price")
  .group_by("category")
  .having("AVG(price) > ?", [50.00])
  .get
```

This generates:

```
SELECT category, AVG(price) AS avg_price FROM products GROUP BY category HAVING AVG(price) > ?
```

Multiple `.having` calls combine with AND:

```
results = Tina4::QueryBuilder.from_table("products", db: db)
  .select("category", "COUNT(*) AS cnt", "AVG(price) AS avg_price")
  .group_by("category")
  .having("COUNT(*) > ?", [5])
  .having("AVG(price) < ?", [100.00])
  .get
```

---

10. Executing Queries

The QueryBuilder provides four execution methods.

get -- Multiple Rows

```
results = Tina4::QueryBuilder.from_table("users", db: db)
  .where("active = ?", [1])
  .order_by("name ASC")
  .limit(25)
  .get
```

Returns a `DatabaseResult` object. You can iterate over it, access rows by index, call `.to_json`, `.to_csv`, or `.to_paginate` -- everything covered in Chapter 5.

first -- Single Row

```
user = Tina4::QueryBuilder.from_table("users", db: db)
  .where("email = ?", ["alice@example.com"])
  .first
```

Returns a single hash, or `nil` if no row matches. Useful when you expect exactly one result.

count -- Row Count

```
total = Tina4::QueryBuilder.from_table("users", db: db)
  .where("active = ?", [1])
  .count
```

Returns an integer. The QueryBuilder rewrites your columns to `COUNT(*) AS cnt` internally, so it works regardless of what you passed to `.select`.

exists? -- Boolean Check

```
if Tina4::QueryBuilder.from_table("users", db: db)
     .where("email = ?", ["alice@example.com"])
     .exists?
  puts "User found"
end
```

Returns `true` if at least one matching row exists, `false` otherwise. Calls `.count` under the hood.

---

11. Inspecting the SQL with to_sql

Call `.to_sql` to see the generated SQL without executing it:

```
query = Tina4::QueryBuilder.from_table("products", db: db)
  .select("name", "price")
  .where("category = ?", ["Electronics"])
  .where("price > ?", [50.00])
  .order_by("price DESC")
  .limit(10)

puts query.to_sql
```

Output:

```
SELECT name, price FROM products WHERE category = ? AND price > ? ORDER BY price DESC
```

This is invaluable for debugging. If a query returns unexpected results, print the SQL and check the logic. Note that `.to_sql` shows the SQL with `?` placeholders -- the actual parameter values are bound at execution time by the database driver.

---

12. Using QueryBuilder in Routes

A Filtered API Endpoint

```
Tina4::Router.get("/api/products") do |request, response|
  db = Tina4.database

  query = Tina4::QueryBuilder.from_table("products", db: db)

  # Apply filters from query string
  category = request.params["category"]
  min_price = request.params["min_price"]
  max_price = request.params["max_price"]
  search = request.params["search"]

  query = query.where("category = ?", [category]) if category
  query = query.where("price >= ?", [min_price.to_f]) if min_price
  query = query.where("price <= ?", [max_price.to_f]) if max_price
  query = query.where("name LIKE ?", ["%#{search}%"]) if search

  # Pagination
  page = (request.params["page"] || 1).to_i
  per_page = (request.params["per_page"] || 20).to_i
  offset = (page - 1) * per_page

  total = query.count

  results = query
    .order_by("name ASC")
    .limit(per_page, offset)
    .get

  response.call({
    products: results,
    total: total,
    page: page,
    per_page: per_page
  }, Tina4::HTTP_OK)
end
```

```
curl "http://localhost:7147/api/products?category=Electronics&min_price=50&page=2&per_page=10"
```

A Dashboard Summary Endpoint

```
Tina4::Router.get("/api/dashboard/summary") do |request, response|
  db = Tina4.database

  # Total users
  total_users = Tina4::QueryBuilder.from_table("users", db: db).count

  # Active users
  active_users = Tina4::QueryBuilder.from_table("users", db: db)
    .where("active = ?", [1])
    .count

  # Revenue by category
  revenue = Tina4::QueryBuilder.from_table("order_items", db: db)
    .select("products.category", "SUM(order_items.quantity * order_items.price) AS revenue")
    .join("products", "products.id = order_items.product_id")
    .group_by("products.category")
    .order_by("revenue DESC")
    .get

  response.call({
    total_users: total_users,
    active_users: active_users,
    revenue_by_category: revenue
  }, Tina4::HTTP_OK)
end
```

---

13. Exercise: Build a Product Search API

Build a product search endpoint that uses the QueryBuilder for all database access.

Requirements

- Assume the `products` table from Chapter 5 exists with columns: `id`, `name`, `category`, `price`, `in_stock`, `created_at`.
- Build these API endpoints:

Method	Path	Description
`GET`	`/api/products/search`	Search products. Support `?q=`, `?category=`, `?min_price=`, `?max_price=`, `?in_stock=`, `?sort=`, `?page=`, `?per_page=`.
`GET`	`/api/products/stats`	Return category statistics: product count and average price per category. Only include categories with more than 2 products.

Test with:

```
# Full-text search with price range
curl "http://localhost:7147/api/products/search?q=wireless&min_price=20&max_price=100"

# Category filter with pagination
curl "http://localhost:7147/api/products/search?category=Electronics&page=1&per_page=5&sort=pric

# In-stock only
curl "http://localhost:7147/api/products/search?in_stock=1&sort=name_asc"

# Category stats
curl "http://localhost:7147/api/products/stats"
```

---

14. Solution

Search Endpoint

Create `src/routes/product_search.rb`:

```
Tina4::Router.get("/api/products/search") do |request, response|
  db = Tina4.database

  query = Tina4::QueryBuilder.from_table("products", db: db)

  # Text search
  q = request.params["q"]
  query = query.where("name LIKE ?", ["%#{q}%"]) if q && !q.empty?

  # Category filter
  category = request.params["category"]
  query = query.where("category = ?", [category]) if category && !category.empty?

  # Price range
  min_price = request.params["min_price"]
  query = query.where("price >= ?", [min_price.to_f]) if min_price

  max_price = request.params["max_price"]
  query = query.where("price <= ?", [max_price.to_f]) if max_price

  # In-stock filter
  in_stock = request.params["in_stock"]
  query = query.where("in_stock = ?", [in_stock.to_i]) if in_stock

  # Total count before pagination
  total = query.count

  # Sorting
  sort = request.params["sort"] || "name_asc"
  order = case sort
          when "price_asc"  then "price ASC"
          when "price_desc" then "price DESC"
          when "name_desc"  then "name DESC"
          when "newest"     then "created_at DESC"
          else "name ASC"
          end
```

```
  # Pagination
  page = (request.params["page"] || 1).to_i
  per_page = (request.params["per_page"] || 20).to_i
  offset = (page - 1) * per_page

  results = query
    .order_by(order)
    .limit(per_page, offset)
    .get

  response.call({
    products: results,
    total: total,
    page: page,
    per_page: per_page,
    sort: sort
  }, Tina4::HTTP_OK)
end
```

Stats Endpoint

```
Tina4::Router.get("/api/products/stats") do |request, response|
  db = Tina4.database

  stats = Tina4::QueryBuilder.from_table("products", db: db)
    .select("category", "COUNT(*) AS product_count", "ROUND(AVG(price), 2) AS avg_price")
    .group_by("category")
    .having("COUNT(*) > ?", [2])
    .order_by("product_count DESC")
    .get

  response.call({
    categories: stats
  }, Tina4::HTTP_OK)
end
```

Expected output for search:

```
{
  "products": [
    {"id": 5, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock"
    {"id": 8, "name": "Wireless Mouse", "category": "Electronics", "price": 34.99, "in_stock": 1
  ],
  "total": 2,
  "page": 1,
  "per_page": 20,
  "sort": "name_asc"
}
```

Expected output for stats:

```
{
  "categories": [
    {"category": "Electronics", "product_count": 12, "avg_price": 149.99},
    {"category": "Books", "product_count": 8, "avg_price": 24.50},
    {"category": "Home", "product_count": 5, "avg_price": 67.25}
  ]
}
```

---

15. NoSQL: MongoDB Queries

The QueryBuilder can generate MongoDB-compatible query documents with `to_mongo`. This returns a hash containing the filter, projection, sort, limit, and skip -- ready to pass to the Mongo Ruby driver.

Operator Mapping

SQL Operator	MongoDB Operator
`=`	Exact match
`!=`	`$ne`
`>`	`$gt`
`<`	`$lt`
`>=`	`$gte`
`<=`	`$lte`
`LIKE`	`$regex`
`IN`	`$in`
`IS NULL`	`$exists: false`
`IS NOT NULL`	`$exists: true`

Example

```
query = Tina4::QueryBuilder.from_table("users")
  .select("name", "email")
  .where("age > ?", [25])
  .where("status = ?", ["active"])
  .order_by("name ASC")
  .limit(10)
  .offset(5)

mongo = query.to_mongo
```

The returned hash:

```
{
  filter: { "age" => { "$gt" => 25 }, "status" => "active" },
  projection: { "name" => 1, "email" => 1 },
  sort: { "name" => 1 },
  limit: 10,
  skip: 5,
}
```

Pass it directly to the Mongo Ruby driver:

```
collection = client[:users]
cursor = collection.find(mongo[:filter])
  .projection(mongo[:projection])
  .sort(mongo[:sort])
```

```
    .limit(mongo[:limit])
    .skip(mongo[:skip])
```

---

16. Gotchas

1. Default Limit of 100

Problem: You expect all rows but only get 100.

Cause: When you call `.get` without calling `.limit`, the QueryBuilder defaults to `limit: 100, offset: 0`. This is a safety net to prevent accidental full-table dumps.

Fix: Call `.limit` explicitly if you need more rows: `.limit(500)`. For truly unbounded queries, use raw SQL via `db.fetch`.

2. Parameter Order Matters

Problem: Your query returns wrong results or the database throws a type error.

Cause: The `?` placeholders are positional. Parameters from `.where`, `.or_where`, and `.having` are collected in the order you chain them. If you reorder your calls, the parameter positions change.

Fix: Keep your chained calls in a consistent order. Use `.to_sql` to verify the generated SQL matches your parameter array.

3. No Database Connection

Problem: You get `QueryBuilder: No database connection provided.`

Cause: You did not pass `db:` to `.from`, and `Tina4.database` is not configured.

Fix: Either pass the connection explicitly with `Tina4::QueryBuilder.from_table("users", db: db)` or ensure your `.env` has a valid `TINA4_DATABASE_URL` so the framework configures `Tina4.database` at startup.

4. count Replaces Your Columns

Problem: You call `.count` after setting `.select("name", "price")` and get confused about what SQL runs.

Cause: `.count` temporarily replaces your columns with `COUNT(*) AS cnt`, executes the query, then restores your original columns. This is transparent -- your QueryBuilder object is unchanged after the call.

Fix: No action needed. This is by design. If you need both the count and the rows, call `.count` first and then `.get`. The QueryBuilder is reusable.

5. to_sql Does Not Show Parameter Values

Problem: You call `.to_sql` and see `?` placeholders instead of actual values.

Cause: `.to_sql` returns the SQL template. The parameter values are bound separately by the database driver at execution time. This is correct behaviour -- it is how parameterised queries work.

Fix: To debug parameter values, inspect the arrays you pass to `.where` and `.having`. The parameters are applied in order, left to right, matching the `?` placeholders in the SQL output.

6. or_where as First Condition

Problem: Your SQL starts with `WHERE OR condition` and the database rejects it.

Cause: You used `.or_where` as the first filter. The QueryBuilder prepends OR to every `.or_where` condition except when it is the first condition in the list -- in that case, the connector is dropped. However, the intent is unclear. Starting a WHERE clause with OR is semantically wrong.

Fix: Always start with `.where` for the first condition. Use `.or_where` only for subsequent conditions that need OR logic.

Authentication

1. Locking the Door

Every endpoint you have built is public. Anyone with the URL can read, create, update, and delete data. Fine for a tutorial. Reckless for production. A real application needs to know who is making a request and whether they have permission.

This chapter covers Tina4's authentication system. JWT tokens. Password hashing. Middleware-based route protection. CSRF tokens for forms. Session management.

---

2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string containing a payload -- user ID, role, whatever you need. The server mints the token at login. The client sends it with every request. The server verifies it without touching the database.

Generating a Token

```
payload = {
  user_id: 42,
  email: "alice@example.com",
  role: "admin"
}

token = Tina4::Auth.get_token(payload)
```

`get_token` signs the payload and returns a JWT string like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJ
```

The token has three parts separated by dots: header, payload, and signature. The signature ensures the token has not been tampered with.

> ***Legacy aliases:*** `Tina4::Auth.create_token` *still works as an alias for* `Tina4::Auth.get_token`*. Use the primary name in new code.*

Token Expiry

By default, tokens expire after 60 minutes. Configure the expiry when generating the token:

```
# Default: 60 minutes
token = Tina4::Auth.get_token(payload)

# Custom: 24 hours (1440 minutes)
token = Tina4::Auth.get_token(payload, expires_in: 1440)

# Custom: 7 days (10080 minutes)
token = Tina4::Auth.get_token(payload, expires_in: 10080)
```

The `expires_in` value is in **minutes**. Common settings:

Value	Duration

`15`	15 minutes
`60`	1 hour (default)
`1440`	24 hours
`10080`	7 days

Validating a Token

```
payload = Tina4::Auth.valid_token(token)
# Returns the payload hash if valid, nil if invalid or expired
```

`valid_token` returns the decoded payload on success, not a boolean. This lets you validate and read the token in one step. Returns `nil` if the token is invalid or expired.

> ***Legacy alias:*** `Tina4::Auth.validate_token` *works the same way.*

Reading the Payload

```
payload = Tina4::Auth.get_payload(token)
```

Returns the decoded payload hash **without validation** -- it just decodes the token:

```
{
  "user_id" => 42,
  "email" => "alice@example.com",
  "role" => "admin",
  "iat" => 1711112600,  # issued at (Unix timestamp)
  "exp" => 1711116200   # expires at (Unix timestamp)
}
```

If the token cannot be decoded, `get_payload` returns `nil`.

> ***Important:*** `get_payload` *does not verify the signature or check expiry. Use* `valid_token` *when you need to confirm the token is trustworthy.*

The Secret Key and Algorithm

Tina4 Ruby supports two JWT algorithms, auto-detected based on your configuration:

- **HS256** (HMAC-SHA256) -- set `TINA4_SECRET` in `.env`. Uses the standard library. Zero dependencies.
- **RS256** (RSA) -- RSA keys are auto-generated in the `.keys/` folder. Requires the `jwt` gem (included by default).

```
# .env -- HS256 mode (recommended, simplest setup)
TINA4_SECRET=my-super-secret-key-at-least-32-chars
```

If `TINA4_SECRET` is set and no RSA keys exist in `.keys/`, Tina4 uses HS256. If RSA keys exist in `.keys/` instead, Tina4 uses RS256. If neither is configured, Tina4 auto-generates RSA keys in `.keys/` on first run.

Keep this key secret. If someone gets it, they can forge tokens.

---

3. Password Hashing

Plain text passwords are a security breach waiting to happen. Tina4 provides two methods for secure password handling:

Hashing a Password

```
hash = Tina4::Auth.hash_password("my-secure-password")
# Returns: "$2a$12$abc123...long-hash-string..."
```

This uses BCrypt (via the `bcrypt` gem, included by default). Each hash includes a random salt, so hashing the same password twice produces different results.

Checking a Password

```
is_correct = Tina4::Auth.check_password("my-secure-password", stored_hash)
# Returns true if the password matches the hash
```

Registration Example

```
# @noauth
Tina4::Router.post("/api/register") do |request, response|
  body = request.body

  # Validate input
  if body["name"].nil? || body["email"].nil? || body["password"].nil?
    return response.json({ error: "Name, email, and password are required" }, 400)
  end

  if body["password"].length < 8
    return response.json({ error: "Password must be at least 8 characters" }, 400)
  end

  db = Tina4.database

  # Check if email already exists
  existing = db.fetch_one("SELECT id FROM users WHERE email = ?", [body["email"]])
  unless existing.nil?
    return response.json({ error: "Email already registered" }, 409)
  end

  # Hash the password
  password_hash = Tina4::Auth.hash_password(body["password"])

  # Create the user
  db.execute(
    "INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)",
    [body["name"], body["email"], password_hash]
  )

  user = db.fetch_one("SELECT id, name, email FROM users WHERE id = last_insert_rowid()")

  response.json({
    message: "Registration successful",
    user: user
  }, 201)
end
```

```
curl -X POST http://localhost:7147/api/register \
```

```
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

---

4. The Login Flow

Here is the complete login flow: the client sends credentials, the server validates them, and returns a JWT token.

```
# @noauth
Tina4::Router.post("/api/login") do |request, response|
  body = request.body

  if body["email"].nil? || body["password"].nil?
    return response.json({ error: "Email and password are required" }, 400)
  end

  db = Tina4.database

  # Find the user
  user = db.fetch_one(
    "SELECT id, name, email, password_hash FROM users WHERE email = ?",
    [body["email"]]
  )

  if user.nil?
    return response.json({ error: "Invalid email or password" }, 401)
  end

  # Check the password
  unless Tina4::Auth.check_password(body["password"], user["password_hash"])
    return response.json({ error: "Invalid email or password" }, 401)
  end

  # Generate a token
  token = Tina4::Auth.get_token({
    user_id: user["id"],
    email: user["email"],
    name: user["name"]
  })

  response.json({
    message: "Login successful",
    token: token,
    user: {
      id: user["id"],
      name: user["name"],
      email: user["email"]
    }
  })
end
```

Notice the `@noauth` comment. The login endpoint must be public -- you cannot require a token to get a token.

```
curl -X POST http://localhost:7147/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

The client stores this token (in localStorage, a cookie, or memory) and sends it with subsequent requests.

---

5. Using Tokens in Requests

The client sends the token in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7147/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

---

6. Protecting Routes

Auth Middleware

Create a reusable auth middleware:

```
def auth_middleware(request, response, next_handler)
  auth_header = request.headers["Authorization"] || ""

  if auth_header.empty? || !auth_header.start_with?("Bearer ")
    return response.json({ error: "Authorization header required" }, 401)
  end

  token = auth_header.sub("Bearer ", "")

  payload = Tina4::Auth.valid_token(token)
  if payload.nil?
    return response.json({ error: "Invalid or expired token" }, 401)
  end

  request.user = payload  # Attach user data to the request

  next_handler.call(request, response)
end
```

Applying Middleware to Routes

```
Tina4::Router.get("/api/profile", middleware: "auth_middleware") do |request, response|
  response.json({
    user_id: request.user["user_id"],
    email: request.user["email"],
    name: request.user["name"]
  })
end
```

```
# Without token -- 401
curl http://localhost:7147/api/profile
```

```
{"error":"Authorization header required"}
```

```
# With valid token -- 200
curl http://localhost:7147/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
{
  "user_id": 1,
  "email": "alice@example.com",
  "name": "Alice"
}
```

Applying Middleware to a Group

```
Tina4::Router.group("/api/admin", middleware: "auth_middleware") do

  Tina4::Router.get("/stats") do |request, response|
    response.json({ active_users: 42 })
  end

  Tina4::Router.get("/logs") do |request, response|
    response.json({ logs: [] })
  end

end
```

Every route in the group requires a valid token.

---

7. @noauth and @secured Decorators

As introduced in Chapter 2, these decorators control authentication at the route level.

@noauth -- Skip Authentication

Use `@noauth` for public endpoints that should bypass any global or group-level auth:

```
# @noauth
Tina4::Router.get("/api/public/health") do |request, response|
  response.json({ status: "ok" })
end

# @noauth
Tina4::Router.post("/api/login") do |request, response|
  # Login logic
end
```

```
# @noauth
Tina4::Router.post("/api/register") do |request, response|
  # Registration logic
end
```

@secured -- Require Authentication for GET Routes

By default, POST, PUT, PATCH, and DELETE routes are considered secured. GET routes are public. Use `@secured` to explicitly protect a GET route:

```
# @secured
Tina4::Router.get("/api/me") do |request, response|
  # This GET route requires authentication
  response.json(request.user)
end
```

Role-Based Authorization

Combine auth middleware with role checks:

```
def require_role(role)
  lambda do |request, response, next_handler|
    # First check authentication
    auth_header = request.headers["Authorization"] || ""
    if auth_header.empty? || !auth_header.start_with?("Bearer ")
      return response.json({ error: "Authorization required" }, 401)
    end

    token = auth_header.sub("Bearer ", "")
    payload = Tina4::Auth.valid_token(token)
    if payload.nil?
      return response.json({ error: "Invalid or expired token" }, 401)
    end

    # Check role
    if (payload["role"] || "") != role
      return response.json({ error: "Forbidden. Required role: #{role}" }, 403)
    end

    request.user = payload
    next_handler.call(request, response)
  end
end
```

---

8. CSRF Protection

For traditional form-based applications (not SPAs), Tina4 provides CSRF protection with form tokens.

Generating a Token

In your template, include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
    {{ form_token() }}

    <div class="form-group">
```

```
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input field:

```
<input type="hidden" name="_token" value="abc123randomtoken456">
```

Validating the Token

In your route handler, check the token:

```
Tina4::Router.post("/profile/update") do |request, response|
  # Validate CSRF token (form_token() renders a signed JWT, validated like any token)
  unless Tina4::Auth.valid_token(request.body["_token"] || "")
    return response.json({ error: "Invalid form token. Please refresh and try again." }, 403)
  end

  # Process the form...
  response.redirect("/profile")
end
```

---

9. Sessions

Tina4 supports server-side sessions for storing per-user state between requests. Sessions work alongside JWT tokens -- use JWTs for API authentication and sessions for stateful web pages.

Using Sessions

Access session data via `request.session`:

```
Tina4::Router.post("/login-form") do |request, response|
  # After validating credentials...
  request.session["user_id"] = 42
  request.session["user_name"] = "Alice"
  request.session["logged_in"] = true

  response.redirect("/dashboard")
end

Tina4::Router.get("/dashboard") do |request, response|
  if request.session["logged_in"].nil?
    return response.redirect("/login")
  end

  response.render("dashboard.html", {
    user_name: request.session["user_name"]
  })
end

Tina4::Router.post("/logout") do |request, response|
  # Clear all session data
  request.session.clear
```

```
  response.redirect("/login")
end
```

---

10. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

Requirements

- Create a `users` table migration with: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`
- Build these endpoints:

Method	Path	Auth	Description
`POST`	`/api/register`	@noauth	Create an account. Validate name, email, password (min 8 chars).
`POST`	`/api/login`	@noauth	Login. Return JWT token.
`GET`	`/api/profile`	secured	Get current user's profile from token.
`PUT`	`/api/profile`	secured	Update name and email.
`PUT`	`/api/profile/password`	secured	Change password. Require current password.

- Create auth middleware that extracts the user from the JWT and attaches it to `request.user`.

Test with:

```
# Register
curl -X POST http://localhost:7147/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7147/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token from login response, then:

# Get profile
curl http://localhost:7147/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7147/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7147/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"current_password": "securePass123", "new_password": "evenMoreSecure456"}'

# Try with no token (should fail)
curl http://localhost:7147/api/profile
```

---

11. Solution

Migration

Create `migrations/20260322160000_create_auth_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

Middleware

Create `src/routes/middleware.rb`:

```
def auth_middleware(request, response, next_handler)
```

```
  auth_header = request.headers["Authorization"] || ""

  if auth_header.empty? || !auth_header.start_with?("Bearer ")
    return response.json({ error: "Authorization required. Send: Authorization: Bearer <token>"
  end

  token = auth_header.sub("Bearer ", "")

  payload = Tina4::Auth.valid_token(token)
  if payload.nil?
    return response.json({ error: "Invalid or expired token. Please login again." }, 401)
  end

  request.user = payload

  next_handler.call(request, response)
end
```

Routes

Create `src/routes/auth.rb`:

```
# @noauth
Tina4::Router.post("/api/register") do |request, response|
  body = request.body

  errors = []
  errors << "Name is required" if body["name"].nil? || body["name"].empty?
  errors << "Email is required" if body["email"].nil? || body["email"].empty?
  if body["password"].nil? || body["password"].empty?
    errors << "Password is required"
  elsif body["password"].length < 8
    errors << "Password must be at least 8 characters"
  end

  unless errors.empty?
    return response.json({ errors: errors }, 400)
  end

  db = Tina4.database

  existing = db.fetch_one("SELECT id FROM users WHERE email = ?", [body["email"]])
  unless existing.nil?
    return response.json({ error: "Email already registered" }, 409)
  end

  hash = Tina4::Auth.hash_password(body["password"])

  db.execute(
    "INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)",
    [body["name"], body["email"], hash]
  )

  user = db.fetch_one("SELECT id, name, email, role, created_at FROM users WHERE id = last_inser

  response.json({ message: "Registration successful", user: user }, 201)
end

# @noauth
```

```
Tina4::Router.post("/api/login") do |request, response|
  body = request.body

  if body["email"].nil? || body["password"].nil?
    return response.json({ error: "Email and password are required" }, 400)
  end

  db = Tina4.database

  user = db.fetch_one(
    "SELECT id, name, email, password_hash, role FROM users WHERE email = ?",
    [body["email"]]
  )

  if user.nil? || !Tina4::Auth.check_password(body["password"], user["password_hash"])
    return response.json({ error: "Invalid email or password" }, 401)
  end

  token = Tina4::Auth.get_token({
    user_id: user["id"],
    email: user["email"],
    name: user["name"],
    role: user["role"]
  })

  response.json({
    message: "Login successful",
    token: token,
    user: { id: user["id"], name: user["name"], email: user["email"], role: user["role"] }
  })
end

# Get current user profile
Tina4::Router.get("/api/profile", middleware: "auth_middleware") do |request, response|
  db = Tina4.database

  user = db.fetch_one(
    "SELECT id, name, email, role, created_at FROM users WHERE id = ?",
    [request.user["user_id"]]
  )

  if user.nil?
    return response.json({ error: "User not found" }, 404)
  end

  response.json(user)
end

# Update profile
Tina4::Router.put("/api/profile", middleware: "auth_middleware") do |request, response|
  db = Tina4.database
  body = request.body
  user_id = request.user["user_id"]

  if body["email"]
    existing = db.fetch_one(
      "SELECT id FROM users WHERE email = ? AND id != ?",
      [body["email"], user_id]
    )
```

```
    unless existing.nil?
      return response.json({ error: "Email already in use by another account" }, 409)
    end
  end

  current = db.fetch_one("SELECT * FROM users WHERE id = ?", [user_id])

  db.execute(
    "UPDATE users SET name = ?, email = ? WHERE id = ?",
    [body["name"] || current["name"], body["email"] || current["email"], user_id]
  )

  updated = db.fetch_one(
    "SELECT id, name, email, role, created_at FROM users WHERE id = ?",
    [user_id]
  )

  response.json({ message: "Profile updated", user: updated })
end

# Change password
Tina4::Router.put("/api/profile/password", middleware: "auth_middleware") do |request, response|
  db = Tina4.database
  body = request.body
  user_id = request.user["user_id"]

  if body["current_password"].nil? || body["new_password"].nil?
    return response.json({ error: "Current password and new password are required" }, 400)
  end

  if body["new_password"].length < 8
    return response.json({ error: "New password must be at least 8 characters" }, 400)
  end

  user = db.fetch_one("SELECT password_hash FROM users WHERE id = ?", [user_id])

  unless Tina4::Auth.check_password(body["current_password"], user["password_hash"])
    return response.json({ error: "Current password is incorrect" }, 401)
  end

  new_hash = Tina4::Auth.hash_password(body["new_password"])

  db.execute(
    "UPDATE users SET password_hash = ? WHERE id = ?",
    [new_hash, user_id]
  )

  response.json({ message: "Password changed successfully" })
end
```

Expected output for register:

```
{
  "message": "Registration successful",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user",
```

```
    "created_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for profile without token:

```
{"error":"Authorization required. Send: Authorization: Bearer <token>"}
```

(Status: `401 Unauthorized`)

Expected output for successful password change:

```
{"message":"Password changed successfully"}
```

---

12. Gotchas

1. Token Expiry Confusion

Problem: Tokens that worked yesterday now return 401.

Cause: The default token lifetime is 1 hour (3600 seconds). After that, the token is invalid even if the signature is correct.

Fix: Issue a new token at login. If your application needs long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days) that can be used to get a new access token without re-entering credentials.

2. Secret Key Management

Problem: Tokens generated on one server are invalid on another, or tokens stop working after a deployment.

Cause: Each server generated its own RSA keys in `.keys/`. Or the key files were deleted/regenerated during deployment.

Fix: Set `TINA4_SECRET` in `.env` explicitly and use the same value across all servers. Store it in your deployment secrets manager (not in version control). If the key changes, all existing tokens become invalid and users must log in again.

3. CORS with Authentication

Problem: Frontend requests with the `Authorization` header fail with a CORS error, even though `TINA4_CORS_ORIGINS=*` is set.

Cause: When the browser sends an `Authorization` header, it first sends a preflight `OPTIONS` request. The server must respond to the OPTIONS request with the correct CORS headers, including `Access-Control-Allow-Headers: Authorization`.

Fix: Tina4 handles preflight requests automatically. Make sure `TINA4_CORS_ORIGINS` is set correctly. If it is still failing, check that you are not overriding CORS headers in middleware.

4. Storing Tokens in localStorage

Problem: Your token is stolen via an XSS attack because it was stored in `localStorage`.

Cause: Any JavaScript on the page can read `localStorage`, including injected scripts from an XSS vulnerability.

Fix: Store tokens in `httpOnly` cookies when possible -- they cannot be accessed by JavaScript. For SPAs that must use `localStorage`, implement strict Content Security Policy headers and sanitize all user input.

5. Forgetting @noauth on Login

Problem: Your login endpoint returns 401 -- you cannot log in because the endpoint requires authentication.

Cause: If you have global auth middleware, the login endpoint needs the `@noauth` annotation to bypass it.

Fix: Add `# @noauth` before your login and register routes. Without it, users cannot authenticate because authentication is required to authenticate -- a catch-22.

6. Password Hash Column Too Short

Problem: Registration fails with a database error about the password hash being too long.

Cause: BCrypt/PBKDF2 hashes can be long. If your `password_hash` column is defined as `VARCHAR(50)`, it gets truncated.

Fix: Use `TEXT` for the password hash column, or at minimum `VARCHAR(255)`. Never constrain the hash length.

7. Token in URL Query Parameters

Problem: Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and the Referer header.

Cause: Query parameters are visible in many places where headers are not.

Fix: Always send tokens in the `Authorization` header, never in the URL. The only exception is WebSocket connections, where the initial HTTP upgrade request cannot carry custom headers -- use a short-lived token for that case.

Sessions & Cookies

1. State in a Stateless World

Your e-commerce site needs a shopping cart that persists across page loads. A language preference that sticks. Flash messages after form submissions. But HTTP is stateless. Every request arrives with no memory of what came before. Sessions and cookies give the server a way to remember.

Chapter 8 introduced sessions for authentication. This chapter goes deeper. Session backends. Flash messages. Cookies. Remember-me tokens. Security configuration.

---

2. How Sessions Work

Sessions are auto-started. Every route handler receives `request.session` ready to use. No manual setup required.

When a user visits your site for the first time, Tina4 generates a unique session ID (a long random string), stores it in a cookie named `tina4_session` (`HttpOnly`, `SameSite=Lax`) on the user's browser, and creates a server-side storage entry keyed by that ID. On every subsequent request, the browser sends the cookie, Tina4 looks up the session data, and makes it available via `request.session`.

The flow looks like this:

- Browser sends first request (no session cookie)
- Tina4 generates session ID: `abc123def456`
- Tina4 sets cookie: `tina4_session=abc123def456`
- Tina4 creates empty session storage for `abc123def456`
- Browser sends second request with cookie `tina4_session=abc123def456`
- Tina4 loads session data for `abc123def456`
- Your route handler reads and writes `request.session`
- At the end of the request, Tina4 saves the updated session data

The session data is stored server-side. The browser only has the session ID -- it never sees the actual data.

---

3. The Session API

Access session data through `request.session`. It is available in every route handler with zero configuration.

Full API Reference

Method	Description
`request.session.set(key, value)`	Store a value
`request.session.get(key, default)`	Retrieve a value (with optional default)
`request.session.delete(key)`	Remove a key
`request.session.has?(key)`	Check if a key exists
`request.session.clear`	Remove all session data
`request.session.destroy`	Destroy the session entirely
`request.session.save`	Persist session data (auto-called after response)
`request.session.regenerate`	Generate a new session ID, preserve data
`request.session.flash(key, value)`	Set flash data (one-time read)
`request.session.flash(key)`	Read and remove flash data
`request.session.all`	Get all session data as a hash

---

4. File Sessions (Default)

Out of the box, Tina4 stores sessions in files. No configuration needed.

```
Tina4::Router.get("/visit-counter") do |request, response|
  count = (request.session.get("visit_count", 0)) + 1
  request.session.set("visit_count", count)

  response.json({
    visit_count: count,
    message: "You have visited this page #{count} time#{count == 1 ? '' : 's'}"
  })
end
```

```
curl http://localhost:7147/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":1,"message":"You have visited this page 1 time"}
```

```
curl http://localhost:7147/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":2,"message":"You have visited this page 2 times"}
```

The `-c cookies.txt` flag tells curl to save cookies to a file, and `-b cookies.txt` tells it to send them back. This simulates how a browser works.

---

5. Redis Sessions

For production deployments with multiple servers (behind a load balancer), you need a shared session store. Redis is the most common choice.

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_REDIS_PASSWORD=your-redis-password
```

That is the only change. Your code stays exactly the same. `request.session` works identically whether sessions are stored in files, Redis, MongoDB, or Valkey.

---

6. MongoDB Sessions

```
TINA4_SESSION_BACKEND=mongodb
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=27017
TINA4_SESSION_MONGO_DB=myapp
TINA4_SESSION_MONGO_COLLECTION=sessions
```

---

7. Valkey Sessions

```
TINA4_SESSION_BACKEND=valkey
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
```

---

8. Database Sessions

```
TINA4_SESSION_BACKEND=database
```

Stores sessions in the `tina4_session` table using your existing database connection (`TINA4_DATABASE_URL`). The table is auto-created on first use. Works with all 5 database engines (SQLite, PostgreSQL, MySQL, MSSQL, Firebird).

---

9. Reading and Writing Session Data

Session data is a simple key-value store. You read and write it through `request.session`:

```
# Write to session
Tina4::Router.post("/api/preferences") do |request, response|
  body = request.body

  request.session.set("language", body["language"] || "en")
  request.session.set("theme", body["theme"] || "light")
  request.session.set("items_per_page", (body["items_per_page"] || 20).to_i)

  response.json({
```

```
    message: "Preferences saved",
    preferences: {
      language: request.session.get("language"),
      theme: request.session.get("theme"),
      items_per_page: request.session.get("items_per_page")
    }
  })
end

# Read from session
Tina4::Router.get("/api/preferences") do |request, response|
  response.json({
    language: request.session.get("language", "en"),
    theme: request.session.get("theme", "light"),
    items_per_page: request.session.get("items_per_page", 20)
  })
end

# Delete a specific key
Tina4::Router.delete("/api/preferences/{key}") do |request, response|
  key = request.params["key"]
  request.session.delete(key)

  response.json({ message: "Preference '#{key}' removed" })
end

# Clear all session data
Tina4::Router.post("/api/session/clear") do |request, response|
  request.session.clear

  response.json({ message: "Session cleared" })
end
```

Storing Complex Data

Sessions can hold arrays and nested structures:

```
Tina4::Router.post("/api/cart/add") do |request, response|
  body = request.body

  cart = request.session.get("cart", [])

  cart << {
    product_id: body["product_id"].to_i,
    name: body["name"],
    price: body["price"].to_f,
    quantity: (body["quantity"] || 1).to_i,
    added_at: Time.now.iso8601
  }

  request.session.set("cart", cart)

  total = cart.sum { |item| item[:price] * item[:quantity] }

  response.json({
    message: "Added to cart",
    cart_items: cart.length,
    cart_total: total
  })
```

```
end
```

---

10. Flash Messages

Flash messages are session data that lives for one request. Set a flash before redirecting. The next request reads it. Then it vanishes.

Setting a Flash Message

```
Tina4::Router.post("/profile/update") do |request, response|
  body = request.body

  # Update the profile (database logic here)

  request.session.flash("message", "Profile updated successfully")
  request.session.flash("message_type", "success")

  response.redirect("/profile")
end
```

Reading and Clearing Flash Messages

```
Tina4::Router.get("/profile") do |request, response|
  flash_message = request.session.flash("message")
  flash_type = request.session.flash("message_type") || "info"

  response.render("profile.html", {
    user: { name: "Alice", email: "alice@example.com" },
    flash_message: flash_message,
    flash_type: flash_type
  })
end
```

Calling `request.session.flash(key)` with only a key reads the value and removes it in one step.

Using Flash Messages in Templates

```
{% extends "base.html" %}

{% block content %}
    {% if flash_message %}
        <div class="alert alert-{{ flash_type }}">
            {{ flash_message }}
        </div>
    {% endif %}

    <h1>Profile</h1>
    <p>Name: {{ user.name }}</p>
    <p>Email: {{ user.email }}</p>
{% endblock %}
```

---

11. Setting and Reading Cookies

Cookies are small pieces of data stored in the browser. Unlike sessions, the data is stored client-side.

Setting a Cookie

```
Tina4::Router.post("/api/set-language") do |request, response|
  language = request.body["language"] || "en"

  response.cookie("language", language, {
    expires: Time.now + (365 * 24 * 60 * 60),  # 1 year
    path: "/",
    http_only: false,
    secure: false,
    same_site: "Lax"
  })

  response.json({ message: "Language set to #{language}" })
end
```

Reading a Cookie

```
Tina4::Router.get("/api/get-language") do |request, response|
  language = request.cookies["language"] || "en"

  response.json({ language: language })
end
```

Deleting a Cookie

```
Tina4::Router.post("/api/clear-language") do |request, response|
  response.cookie("language", "", {
    expires: Time.now - 3600,
    path: "/"
  })

  response.json({ message: "Language cookie cleared" })
end
```

When to Use Cookies vs Sessions

Use Cookies For	Use Sessions For
Language preference	Shopping cart contents
Theme preference (light/dark)	User authentication state
"Remember this device" flag	Flash messages
Analytics tracking consent	Form wizard progress
Non-sensitive, long-lived data	Sensitive, short-lived data

---

12. Remember Me Functionality

The "remember me" pattern uses a long-lived cookie to re-authenticate users after their session expires.

```
# @noauth
Tina4::Router.post("/login") do |request, response|
  body = request.body
  db = Tina4.database

  user = db.fetch_one(
    "SELECT id, name, email, password_hash FROM users WHERE email = ?",
    [body["email"]]
  )

  if user.nil? || !Tina4::Auth.check_password(body["password"], user["password_hash"])
    return response.json({ error: "Invalid email or password" }, 401)
  end

  request.session.set("user_id", user["id"])
  request.session.set("user_name", user["name"])

  if body["remember_me"]
    remember_token = SecureRandom.hex(32)

    db.execute(
      "UPDATE users SET remember_token = ? WHERE id = ?",
      [Digest::SHA256.hexdigest(remember_token), user["id"]]
    )

    response.cookie("remember_me", remember_token, {
      expires: Time.now + (30 * 24 * 60 * 60),  # 30 days
      path: "/",
      http_only: true,
      secure: true,
      same_site: "Lax"
    })
  end

  response.json({
    message: "Login successful",
    user: { id: user["id"], name: user["name"] }
  })
end
```

---

13. Session Security

Configuration Options

```
TINA4_SESSION_TTL=3600
TINA4_SESSION_SAMESITE=Lax
```

`TINA4_SESSION_SAMESITE` controls cross-site cookie behavior:

Value	Behavior

`Strict`	Never sent with cross-site requests. Safest. Breaks some flows (clicking links from email).
`Lax`	Sent with top-level navigations (clicking links) but not cross-site API calls. Good default.
`None`	Always sent. Requires `TINA4_SESSION_SECURE=true`. Only for cross-site cookie access.

Session Regeneration

After a user logs in, regenerate the session ID to prevent session fixation attacks:

```
Tina4::Router.post("/login") do |request, response|
  # Validate credentials...

  request.session.regenerate

  request.session.set("user_id", user["id"])

  response.redirect("/dashboard")
end
```

Destroy a Session

To completely destroy a session (not just clear its data):

```
Tina4::Router.post("/logout") do |request, response|
  request.session.destroy
  response.redirect("/login")
end
```

---

14. Exercise: Build a Shopping Cart with Session Storage

Build a shopping cart that stores items in the session. No database needed -- the cart lives entirely in session data.

Requirements

Method	Path	Description
`POST`	`/api/cart/add`	Add an item to the cart. Body: `{"product_id": 1, "name": "Widget", "price": 9.99, "quantity": 2}`
`GET`	`/api/cart`	View the cart. Show items, quantities, item subtotals, and cart total.
`PUT`	`/api/cart/{product_id:int}`	Update quantity. Body: `{"quantity": 3}`. Remove item if quantity is 0.
`DELETE`	`/api/cart/{product_id:int}`	Remove an item from the cart.
`DELETE`	`/api/cart`	Clear the entire cart.

Business Rules

- If adding a product that already exists in the cart, increment the quantity instead of adding a duplicate
- Cart total should be calculated dynamically
- Return the full cart state after every operation

Test with:

```
# Add first item
curl -X POST http://localhost:7147/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# Add second item
curl -X POST http://localhost:7147/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 2, "name": "USB-C Hub", "price": 49.99, "quantity": 2}' \
  -c cookies.txt -b cookies.txt

# View cart
curl http://localhost:7147/api/cart -b cookies.txt

# Clear cart
curl -X DELETE http://localhost:7147/api/cart -b cookies.txt -c cookies.txt
```

---

15. Solution

Create `src/routes/cart.rb`:

```
def get_cart(session)
  session.get("cart", [])
end

def cart_response(cart)
  total = 0.0
  item_count = 0
  items = []

  cart.each do |item|
    subtotal = item[:price] * item[:quantity]
    total += subtotal
    item_count += item[:quantity]
    items << item.merge(subtotal: subtotal)
  end

  {
    items: items,
    item_count: item_count,
    unique_items: cart.length,
    total: total.round(2)
  }
end

# Add item to cart
Tina4::Router.post("/api/cart/add") do |request, response|
  body = request.body

  if body["product_id"].nil? || body["name"].nil? || body["price"].nil?
    return response.json({ error: "product_id, name, and price are required" }, 400)
  end

  cart = get_cart(request.session)
  product_id = body["product_id"].to_i
  quantity = (body["quantity"] || 1).to_i

  existing = cart.find { |item| item[:product_id] == product_id }

  if existing
    existing[:quantity] += quantity
  else
    cart << {
      product_id: product_id,
      name: body["name"],
      price: body["price"].to_f,
      quantity: quantity
    }
  end

  request.session.set("cart", cart)

  response.json(cart_response(cart))
end

# View cart
```

```
Tina4::Router.get("/api/cart") do |request, response|
  cart = get_cart(request.session)
  response.json(cart_response(cart))
end

# Update quantity
Tina4::Router.put("/api/cart/{product_id:int}") do |request, response|
  product_id = request.params["product_id"]
  quantity = (request.body["quantity"] || 0).to_i
  cart = get_cart(request.session)

  index = cart.index { |item| item[:product_id] == product_id }

  if index.nil?
    return response.json({ error: "Product not in cart" }, 404)
  end

  if quantity <= 0
    cart.delete_at(index)
  else
    cart[index][:quantity] = quantity
  end

  request.session.set("cart", cart)

  response.json(cart_response(cart))
end

# Remove item
Tina4::Router.delete("/api/cart/{product_id:int}") do |request, response|
  product_id = request.params["product_id"]
  cart = get_cart(request.session)

  index = cart.index { |item| item[:product_id] == product_id }

  if index.nil?
    return response.json({ error: "Product not in cart" }, 404)
  end

  cart.delete_at(index)
  request.session.set("cart", cart)

  response.json(cart_response(cart))
end

# Clear cart
Tina4::Router.delete("/api/cart") do |request, response|
  request.session.set("cart", [])

  response.json(cart_response([]))
end
```

Expected output after adding two items:

```
{
  "items": [
    {"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1, "subtotal": 79
    {"product_id": 2, "name": "USB-C Hub", "price": 49.99, "quantity": 2, "subtotal": 99.98}
  ],
```

```
   "item_count": 3,
   "unique_items": 2,
   "total": 179.97
 }
```

---

16. Gotchas

1. Sessions Do Not Work with curl Without Cookie Flags

Problem: Each curl request sees an empty session, as if it is a new user.

Cause: curl does not automatically save or send cookies.

Fix: Use `-c cookies.txt -b cookies.txt` with curl.

2. Session Data Disappears After Server Restart

Problem: All session data is gone after restarting the dev server.

Cause: File sessions stored in the system temp directory may be cleared on restart.

Fix: Set `TINA4_SESSION_PATH` to a persistent directory. For production, use Redis or Valkey.

3. Session Cookie Not Sent Over HTTP in Production

Problem: Sessions work locally but not in production.

Cause: `TINA4_SESSION_SECURE=true` means the cookie is only sent over HTTPS.

Fix: Ensure your reverse proxy sets `X-Forwarded-Proto: https`, or set `TINA4_SESSION_SECURE=false` for HTTP.

4. Flash Messages Show Twice

Problem: The flash message appears, then appears again on the next page load.

Cause: You read the flash message but did not clear it from the session.

Fix: Use `request.session.flash("message")` to read flash data. It reads and deletes in one step. Do not use `request.session.get()` for flash messages.

5. Large Session Data Causes Slow Requests

Problem: Pages load slowly and performance degrades over time.

Cause: You are storing large amounts of data in the session.

Fix: Keep session data small. Store IDs and references, not entire objects.

6. Remember Me Token Not Invalidated on Password Change

Problem: After a user changes their password, their "remember me" cookies on other devices still work.

Cause: The remember-me token in the database was not cleared when the password changed.

Fix: Clear the `remember_token` column whenever the password is updated.

7. Session Fixation

Problem: An attacker can hijack a user's session by setting a known session ID before the user logs in.

Cause: The session ID is not regenerated after login.

Fix: Call `request.session.regenerate` after successful login.

Middleware & Security

1. The Gatekeepers

Your API needs CORS headers for the React frontend. Rate limiting for public endpoints. Auth checking for admin routes. All without cluttering your handlers.

You could copy-paste 10 lines of CORS code into every handler. That breaks the moment you forget one. You could pile all the checks into a giant `if` tree. That buries business logic under boilerplate.

Middleware solves this. Wrap routes with reusable logic that runs before or after the handler. Each middleware does one job -- check a token, set CORS headers, log the request, enforce rate limits -- then passes control to the next layer. Route handlers stay focused on their actual purpose.

Chapter 2 introduced middleware. This chapter goes deep. Built-in middleware. Custom middleware. Execution order. Short-circuiting. Real-world patterns.

---

2. What Middleware Is

Middleware is code that runs before or after your route handler. It sits in the HTTP pipeline between the incoming request and the response. Every request can pass through multiple middleware layers before reaching the handler.

Tina4 Ruby supports two styles of middleware:

Method-based middleware receives the request, the response, and a `next_handler` callable. Call `next_handler` to continue. Skip it to short-circuit.

```
def passthrough(request, response, next_handler)
  next_handler.call(request, response)
end

def block_everything(request, response, next_handler)
  response.json({ error: "Service unavailable" }, 503)
end
```

Class-based middleware uses naming conventions. Class methods prefixed with `before_` run before the handler. Methods prefixed with `after_` run after it. Each method receives `(request, response)` and returns `[request, response]`.

```
class MyMiddleware
  class << self
    def before_check(request, response)
      # Runs before the route handler
      [request, response]
    end

    def after_cleanup(request, response)
      # Runs after the route handler
      [request, response]
    end
```

```
  end
end
```

Register class-based middleware globally with `Middleware.use`:

```
Tina4::Middleware.use(Tina4::CorsClassMiddleware)
Tina4::Middleware.use(Tina4::RateLimiterMiddleware)
Tina4::Middleware.use(Tina4::RequestLoggerMiddleware)
```

If a `before_*` method returns a response with status >= 400, the handler is skipped (short-circuit).

---

3. Built-in CorsMiddleware

Cross-Origin Resource Sharing (CORS) is the browser mechanism that controls which domains can call your API. Tina4 provides a built-in `CorsMiddleware` that handles this. Configure it in `.env`:

```
TINA4_CORS_ORIGINS=http://localhost:3000,https://myapp.com
TINA4_CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
TINA4_CORS_HEADERS=Content-Type,Authorization,Accept
TINA4_CORS_MAX_AGE=86400
TINA4_CORS_CREDENTIALS=true
```

Apply it to a group:

```
Tina4::Router.group("/api", middleware: "CorsMiddleware") do

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [] })
  end

  Tina4::Router.post("/products") do |request, response|
    response.json({ created: true }, 201)
  end

end
```

---

4. Built-in RateLimiter

Rate limiting prevents a single client from overwhelming your API. Configure it in `.env`:

```
TINA4_RATE_LIMIT=60
TINA4_RATE_WINDOW=60
```

This means 60 requests per 60 seconds per IP. Apply it:

```
Tina4::Router.group("/api/public", middleware: "RateLimiter") do

  Tina4::Router.get("/search") do |request, response|
    q = request.params["q"] || ""
    response.json({ query: q, results: [] })
  end

end
```

When a client exceeds the limit:

```
{"error":"Rate limit exceeded. Try again in 42 seconds.","retry_after":42}
```

Custom Limits Per Group

```
# Public endpoints: 30 requests per minute
Tina4::Router.group("/api/public", middleware: "RateLimiter:30") do
  Tina4::Router.get("/search") do |request, response|
    response.json({ results: [] })
  end
end

# Authenticated endpoints: 120 requests per minute
Tina4::Router.group("/api/v1", middleware: ["auth_middleware", "RateLimiter:120"]) do
  Tina4::Router.get("/data") do |request, response|
    response.json({ data: [] })
  end
end
```

Built-in RequestLoggerMiddleware

The `RequestLoggerMiddleware` logs every request with its timing. It uses two hooks:

- `before_log` stamps the start time before the handler runs
- `after_log` calculates elapsed time and writes a log entry

Register it globally:

```
Tina4::Middleware.use(Tina4::RequestLoggerMiddleware)
```

The log output looks like:

```
[RequestLogger] GET /api/users -> 200 (12.345ms)
[RequestLogger] POST /api/products -> 201 (45.678ms)
```

Built-in SecurityHeadersMiddleware

The `SecurityHeadersMiddleware` adds standard security headers to every response. Register it globally:

```
Tina4::Middleware.use(Tina4::SecurityHeadersMiddleware)
```

It sets the following headers by default:

Header	Default Value
`X-Frame-Options`	`SAMEORIGIN`
`Content-Security-Policy`	`default-src 'self'`, plus `style-src` and `script-src` limited to `'self'` and a per-request nonce
`Strict-Transport-Security`	Not sent. With `TINA4_HSTS` set to a number of seconds: `max-age=<TINA4_HSTS>; includeSubDomains`, on HTTPS requests only

`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`
`X-Content-Type-Options`	`nosniff`

Override any header via environment variables in `.env`:

```
TINA4_FRAME_OPTIONS=SAMEORIGIN
TINA4_CSP=default-src 'self'; script-src 'self' https://cdn.example.com
TINA4_HSTS=63072000
TINA4_REFERRER_POLICY=no-referrer
TINA4_PERMISSIONS_POLICY=camera=(), microphone=(), geolocation=(self)
```

Combining All Four Built-In Middleware

A common production setup registers all four globally:

```
Tina4::Middleware.use(Tina4::CorsClassMiddleware)
Tina4::Middleware.use(Tina4::RateLimiterMiddleware)
Tina4::Middleware.use(Tina4::RequestLoggerMiddleware)
Tina4::Middleware.use(Tina4::SecurityHeadersMiddleware)
```

Order matters. CORS handles preflight first. The rate limiter only counts real requests. The logger measures total time including the other middleware. Security headers are added to every response.

---

5. Writing Custom Middleware

Custom middleware follows the same pattern. You can write method-based or class-based middleware.

Request Logging Middleware

```
def log_request(request, response, next_handler)
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)
  method = request.method
  path = request.path
  ip = request.ip

  $stderr.puts "[#{Time.now.strftime('%Y-%m-%d %H:%M:%S')}] #{method} #{path} from #{ip}"

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
  $stderr.puts "  Completed in #{duration}ms"

  result
end
```

Request Timing Middleware

```
def add_timing(request, response, next_handler)
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
  response.header("X-Response-Time", "#{duration}ms")

  result
end
```

IP Whitelist Middleware

```
def ip_whitelist(request, response, next_handler)
  allowed_ips = (ENV["ALLOWED_IPS"] || "127.0.0.1").split(",")

  unless allowed_ips.include?(request.ip)
    return response.json({
      error: "Access denied",
      your_ip: request.ip
    }, 403)
  end

  next_handler.call(request, response)
end
```

Request Validation Middleware

```
def require_json(request, response, next_handler)
  if %w[POST PUT PATCH].include?(request.method)
    content_type = request.headers["Content-Type"] || ""

    unless content_type.include?("application/json")
      return response.json({
        error: "Content-Type must be application/json",
        received: content_type
      }, 415)
    end
  end

  next_handler.call(request, response)
end
```

Writing Class-Based Middleware

For middleware that needs both before and after hooks, use the class-based pattern:

```
class InputSanitizer
  class << self
    def before_sanitize(request, response)
      if request.body.is_a?(Hash)
        request.body = sanitize_hash(request.body)
      end
      [request, response]
    end

    private

    def sanitize_hash(data)
```

```
      data.transform_values do |value|
        case value
        when String
          CGI.escapeHTML(value)
        when Hash
          sanitize_hash(value)
        else
          value
        end
      end
    end
  end
end
```

Register globally or apply to groups:

```
# Global
Tina4::Middleware.use(InputSanitizer)

# On a specific group
Tina4::Router.group("/api", middleware: "InputSanitizer") do
  # routes here
end
```

JWT Authentication Middleware (Class-Based)

```
class JwtAuthMiddleware
  class << self
    def before_verify_token(request, response)
      auth_header = request.headers["Authorization"] || ""

      unless auth_header.start_with?("Bearer ")
        response.json({ error: "Authorization header required" }, 401)
        return [request, response]
      end

      token = auth_header[7..]
      payload = Tina4::Auth.valid_token(token)

      if payload.nil?
        response.json({ error: "Invalid or expired token" }, 401)
        return [request, response]
      end

      request.user = payload
      [request, response]
    end
  end
end
```

Apply it to protected routes:

```
Tina4::Router.group("/api/protected", middleware: "JwtAuthMiddleware") do

  Tina4::Router.get("/profile") do |request, response|
    response.json({ user: request.user })
  end

  Tina4::Router.post("/settings") do |request, response|
    user_id = request.user["sub"]
```

```
    response.json({ updated: true, user_id: user_id })
  end

end
```

---

6. Applying Middleware to Individual Routes

Pass middleware as a keyword argument to any route method:

```
Tina4::Router.get("/api/data", middleware: "log_request") do |request, response|
  response.json({ data: [1, 2, 3] })
end

Tina4::Router.post("/api/data", middleware: ["log_request", "require_json"]) do |request, respon
  response.json({ created: true }, 201)
end
```

For a single middleware, pass a string. For multiple, pass an array. Each middleware runs in the order listed.

---

7. Route Groups with Shared Middleware

Groups apply middleware to every route inside them:

```
# Public API -- rate limited, CORS enabled
Tina4::Router.group("/api/public", middleware: ["CorsMiddleware", "RateLimiter:30"]) do

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [] })
  end

  Tina4::Router.get("/categories") do |request, response|
    response.json({ categories: [] })
  end

end

# Admin API -- auth required, IP restricted, logged
Tina4::Router.group("/api/admin", middleware: ["log_request", "ip_whitelist", "auth_middleware"]

  Tina4::Router.get("/users") do |request, response|
    response.json({ users: [] })
  end

  Tina4::Router.delete("/users/{id:int}") do |request, response|
    id = request.params["id"]
    response.json({ deleted: id })
  end

end
```

---

8. Middleware Execution Order

When you stack middleware, they execute from outer to inner -- like layers of an onion. The first middleware listed runs first on the way in and last on the way out.

Consider this setup:

```
def middleware_a(request, response, next_handler)
  $stderr.puts "A: before"
  result = next_handler.call(request, response)
  $stderr.puts "A: after"
  result
end

def middleware_b(request, response, next_handler)
  $stderr.puts "B: before"
  result = next_handler.call(request, response)
  $stderr.puts "B: after"
  result
end

def middleware_c(request, response, next_handler)
  $stderr.puts "C: before"
  result = next_handler.call(request, response)
  $stderr.puts "C: after"
  result
end

Tina4::Router.get("/api/test", middleware: ["middleware_a", "middleware_b", "middleware_c"]) do
  $stderr.puts "Handler"
  response.json({ ok: true })
end
```

Server log output:

```
A: before
B: before
C: before
Handler
C: after
B: after
A: after
```

The request flows inward: A, B, C, Handler. The response flows outward: C, B, A.

---

9. Short-Circuiting

When middleware does not call `next_handler`, the chain stops. No subsequent middleware runs and the route handler is never called.

Maintenance Mode

```
def maintenance_mode(request, response, next_handler)
  is_maintenance = (ENV["MAINTENANCE_MODE"] || "false") == "true"

  if is_maintenance
    if request.path == "/health"
      return next_handler.call(request, response)
    end

    return response.json({
      error: "Service is undergoing maintenance",
      retry_after: 300
    }, 503)
  end

  next_handler.call(request, response)
end
```

---

10. Modifying Requests in Middleware

Middleware can add data to the request before passing it to the handler:

```
def add_request_id(request, response, next_handler)
  request_id = SecureRandom.hex(8)
  request.request_id = request_id

  result = next_handler.call(request, response)

  response.header("X-Request-Id", request_id)

  result
end
```

---

11. Real-World Middleware Stack

Here is a realistic middleware setup for a production API:

```
# src/routes/middleware.rb

def add_request_id(request, response, next_handler)
  request.request_id = SecureRandom.hex(8)
  result = next_handler.call(request, response)
  response.header("X-Request-Id", request.request_id)
  result
end

def log_request(request, response, next_handler)
  start = Process.clock_gettime(Process::CLOCK_MONOTONIC)
  $stderr.puts "[#{request.request_id}] #{request.method} #{request.path}"

  result = next_handler.call(request, response)

  duration = ((Process.clock_gettime(Process::CLOCK_MONOTONIC) - start) * 1000).round(2)
```

```
  $stderr.puts "[#{request.request_id}] Completed in #{duration}ms"

  result
end

def require_api_key(request, response, next_handler)
  api_key = request.headers["X-API-Key"] || ""
  valid_keys = (ENV["API_KEYS"] || "").split(",")

  unless valid_keys.include?(api_key)
    return response.json({
      error: "Invalid or missing API key",
      request_id: request.request_id
    }, 401)
  end

  next_handler.call(request, response)
end
```

```
# src/routes/api.rb

Tina4::Router.group("/api/v1", middleware: ["add_request_id", "log_request", "CorsMiddleware", "

  Tina4::Router.get("/products") do |request, response|
    response.json({ products: [
      { id: 1, name: "Widget", price: 9.99 },
      { id: 2, name: "Gadget", price: 19.99 }
    ]})
  end

end
```

---

12. Exercise: Build an API Key Middleware

Build a middleware called `validate_api_key` that:

- Checks for an `X-API-Key` header on every request
- Validates the key against a comma-separated list stored in the `API_KEYS` environment variable
- If the key is missing, returns `401` with `{"error": "API key required"}`
- If the key is invalid, returns `403` with `{"error": "Invalid API key"}`
- If the key is valid, attaches the key to `request.api_key` and continues
- Apply this middleware to a route group with at least two endpoints

Setup

Add this to your `.env`:

```
API_KEYS=key-alpha-001,key-beta-002,key-gamma-003
```

Test with:

```
# No API key -- should get 401
curl http://localhost:7147/api/partner/data

# Invalid API key -- should get 403
curl http://localhost:7147/api/partner/data \
  -H "X-API-Key: wrong-key"

# Valid API key -- should get 200
curl http://localhost:7147/api/partner/data \
  -H "X-API-Key: key-alpha-001"
```

---

13. Solution

Create `src/routes/api_key_middleware.rb`:

```
def validate_api_key(request, response, next_handler)
  api_key = request.headers["X-API-Key"] || ""

  if api_key.empty?
    return response.json({ error: "API key required" }, 401)
  end

  valid_keys = (ENV["API_KEYS"] || "").split(",").map(&:strip)

  unless valid_keys.include?(api_key)
    return response.json({ error: "Invalid API key" }, 403)
  end

  request.api_key = api_key

  next_handler.call(request, response)
end

Tina4::Router.group("/api/partner", middleware: "validate_api_key") do

  Tina4::Router.get("/data") do |request, response|
    response.json({
      authenticated_with: request.api_key,
      data: [
        { id: 1, value: "alpha" },
        { id: 2, value: "beta" }
      ]
    })
  end

  Tina4::Router.get("/stats") do |request, response|
    response.json({
      authenticated_with: request.api_key,
      stats: {
        total_requests: 1423,
        avg_response_ms: 42
      }
    })
  end
```

```
end
```

Expected output -- no key:

```
{"error":"API key required"}
```

(Status: `401 Unauthorized`)

Expected output -- valid key:

```
{
  "authenticated_with": "key-alpha-001",
  "data": [
    {"id": 1, "value": "alpha"},
    {"id": 2, "value": "beta"}
  ]
}
```

---

14. Gotchas

1. Middleware Must Be a Named Method

Problem: You pass an anonymous lambda as middleware and get an error.

Cause: Tina4 expects middleware to be referenced by a string name, not as an inline block. The string is resolved to a method at runtime.

Fix: Define your middleware as a named method: `def my_middleware(request, response, next_handler) ... end` and pass `"my_middleware"` as a string.

2. Forgetting to Return next_handler Result

Problem: Your middleware runs but the route handler never executes. The response is empty or a 500 error.

Cause: You called `next_handler.call(request, response)` but did not return the result.

Fix: Always return the result of `next_handler.call(request, response)`. Without the return, the middleware discards the response from the handler and returns `nil`.

3. Middleware Order Matters

Problem: Your logging middleware does not see the request ID, even though `add_request_id` is in the middleware list.

Cause: `log_request` runs before `add_request_id`. Middleware executes in the order listed.

Fix: Put `add_request_id` before `log_request` in the array: `["add_request_id", "log_request"]`.

4. CORS Preflight Returns 404

Problem: The browser's preflight `OPTIONS` request gets a 404, but `GET` and `POST` work fine when tested with curl.

Cause: You did not apply `CorsMiddleware` to the route, so the `OPTIONS` method is not handled.

Fix: Apply `CorsMiddleware` to the group. It automatically handles `OPTIONS` requests.

5. Rate Limiter Counts Preflight Requests

Problem: Your frontend hits the rate limit faster than expected because every `POST` request counts as two requests.

Cause: The rate limiter counts all requests, including `OPTIONS`.

Fix: Put `CorsMiddleware` before `RateLimiter` in the middleware chain.

6. Middleware File Not Auto-Loaded

Problem: You defined middleware in a file but get "method not found" when referencing it.

Cause: The file is not in `src/routes/`. Tina4 auto-loads all `.rb` files in `src/routes/`, but middleware defined outside that directory is not discovered.

Fix: Put your middleware methods in a file inside `src/routes/`, such as `src/routes/middleware.rb`.

7. Short-Circuiting Skips Cleanup Middleware

Problem: Your timing middleware logs the start time but never logs the completion time for blocked requests.

Cause: When an inner middleware short-circuits, the outer middleware's code after `next_handler.call` still runs. But if the short-circuiting middleware is listed before the timing middleware, the timing middleware never executes at all.

Fix: Put cleanup-dependent middleware (timing, logging) at the outermost layer.

---

15. Security

Every route you write is a door. Chapter 8 gave you locks. Chapter 9 gave you session keys. The first half of this chapter gave you guards. This section ties them together into a defence that works without thinking about it.

Tina4 ships secure by default. POST routes require authentication. CSRF tokens protect forms. Security headers harden every response. The framework does the boring security work so you focus on building features. But you need to understand what it does, and why, so you don't accidentally undo it.

---

1. Secure-by-Default Routing

Every POST, PUT, PATCH, and DELETE route requires a valid `Authorization: Bearer` token. No configuration needed. No method call to remember. The framework enforces this before your handler runs.

```
Tina4::Router.post "/api/orders" do |request, response|
  # This handler ONLY runs if the request carries a valid Bearer token.
  # Without one, the framework returns 401 before your code executes.
  response.call({ created: true }, 201)
end
```

Test it without a token:

```
curl -X POST http://localhost:7147/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product": "widget"}'
# 401 Unauthorized
```

Test it with a valid token:

```
curl -X POST http://localhost:7147/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9..." \
  -d '{"product": "widget"}'
# 201 Created
```

GET routes are public by default. Anyone can read. Writing requires proof of identity.

Making a Write Route Public

Some endpoints need to accept unauthenticated writes, such as webhooks, registration forms, and public contact forms. Chain `.no_auth`:

```
Tina4::Router.post("/api/webhooks/stripe").no_auth do |request, response|
  # No token required. Stripe can POST here freely.
  response.call({ received: true })
end
```

Protecting a GET Route

Admin dashboards, user profiles, account settings: some pages need protection even though they only read data. Chain `.secure`:

```
Tina4::Router.get("/api/admin/users").secure do |request, response|
  # Requires a valid Bearer token, even though it's a GET.
  response.call({ users: [] })
end
```

The Rule

Method	Default	Override
GET, HEAD, OPTIONS	Public	`.secure` to protect
POST, PUT, PATCH, DELETE	Auth required	`.no_auth` to open

Two chainable methods. One rule. No surprises.

---

2. CSRF Protection

Cross-Site Request Forgery tricks a user's browser into submitting a form to your server. The browser sends cookies automatically, including session cookies. Without CSRF protection, an attacker's page can submit forms as your logged-in user.

Tina4 blocks this with form tokens.

How It Works

- Your template renders a hidden token using `{{ form_token() }}`.
- The browser submits the token with the form data.
- The `CsrfMiddleware` validates the token before the route handler runs.
- Invalid or missing tokens receive a `403 Forbidden` response.

The Template

```
<form method="POST" action="/api/profile">
    {{ form_token() }}
    <input type="text" name="name" placeholder="Your name">
    <button type="submit">Save</button>
</form>
```

The `{{ form_token() }}` call generates a hidden input field containing a signed JWT. The token is bound to the current session, so a token from one session cannot be used in another.

The Middleware

CSRF protection is on by default. Every POST, PUT, PATCH, and DELETE request must include a valid form token. The middleware checks two places:

- **Request body** - `request.body["formToken"]`
- **Request header** - `X-Form-Token`

If the token is missing or invalid, the middleware returns 403 before your handler runs.

AJAX Requests

For JavaScript-driven forms, send the token as a header:

```
// frond.min.js handles this automatically via saveForm()
// For manual AJAX, extract the token from the hidden field:
const token = document.querySelector('input[name="formToken"]').value;

fetch("/api/profile", {
    method: "POST",
    headers: {
        "Content-Type": "application/json",
        "X-Form-Token": token
    },
    body: JSON.stringify({ name: "Alice" })
});
```

Tokens in Query Strings: Blocked

Tokens must never appear in URLs. Query strings are logged in server access logs, browser history, and referrer headers. A token in the URL is a token anyone can steal.

Tina4 rejects any request that carries `formToken` in the query string and logs a warning:

```
CSRF token found in query string - rejected for security.
Use POST body or X-Form-Token header instead.
```

Skipping CSRF Validation

Three scenarios skip CSRF checks automatically:

- **GET, HEAD, OPTIONS** - Safe methods don't modify state.
- **Routes with `.no_auth`** - Public write endpoints don't need CSRF (they have no session to protect).
- **Requests with a valid Bearer token** - API clients authenticate with tokens, not cookies. CSRF only matters for cookie-based sessions.

Disabling CSRF Globally

For internal microservices behind a firewall, where no browser ever touches the API, you can disable CSRF entirely:

```
TINA4_CSRF=false
```

Leave it enabled for anything a browser can reach. The cost is one hidden field per form. The protection is worth it.

---

3. Session-Bound Tokens

A form token alone prevents cross-site forgery. But what if someone steals a token from a form? Session binding stops them.

When `{{ form_token() }}` generates a token, it embeds the current session ID in the JWT payload. The CSRF middleware checks that the session ID in the token matches the session ID of the request. A token stolen from one session cannot be replayed in another.

This happens automatically. No configuration. No extra code.

How Stolen Tokens Fail

- Attacker visits your site, gets a form token for session `abc-123`.
- Attacker sends that token from their own session `xyz-789`.
- The middleware compares: `abc-123 != xyz-789`, rejected with 403.

The token is cryptographically valid. But it belongs to the wrong session. The binding catches it.

---

4. Security Headers

Every response from Tina4 carries security headers. The `SecurityHeadersMiddleware` injects them before the response reaches the browser. No opt-in required.

Header	Default Value	Purpose
`X-Frame-Options`	`SAMEORIGIN`	Prevents clickjacking, so your pages cannot be embedded in iframes on other domains.
`X-Content-Type-Options`	`nosniff`	Stops browsers from guessing content types. A script is a script, not HTML.
`Content-Security-Policy`	`default-src 'self'`	Controls which resources the browser loads. Blocks inline scripts from injected HTML.
`Referrer-Policy`	`strict-origin-when-cross-origin`	Limits referrer data sent to external sites. Protects internal URLs from leaking.
`X-XSS-Protection`	`0`	Disabled. Modern CSP replaces this legacy header. Keeping it on can introduce vulnerabilities.
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	Disables browser APIs your app does not need.

HSTS: Enforcing HTTPS

Strict Transport Security tells the browser to always use HTTPS. Disabled by default (it breaks local development on HTTP). Enable it in production:

```
TINA4_HSTS=31536000
```

This sets a one-year HSTS policy with `includeSubDomains`. Once a browser sees this header, it refuses to connect over HTTP, even if the user types `http://`.

Customising Headers

Override any header via environment variables:

```
TINA4_FRAME_OPTIONS=DENY
TINA4_CSP=default-src 'self'; script-src 'self' https://cdn.example.com
TINA4_REFERRER_POLICY=no-referrer
TINA4_PERMISSIONS_POLICY=camera=(), microphone=(), geolocation=(), payment=()
```

---

5. SameSite Cookies

Session cookies control who can send them. The `SameSite` attribute tells the browser when to include the cookie in requests.

Value	Behaviour
`Strict`	Cookie sent only on same-site requests. Even clicking a link from email to your site won't include the cookie. The user must navigate directly.
`Lax`	Cookie sent on same-site requests and top-level navigations (clicking a link). Not sent on cross-site AJAX or form POSTs from other domains.
`None`	Cookie sent on all requests, including cross-site. Requires `Secure` flag (HTTPS only).

Tina4 defaults to `Lax`. This blocks cross-site form submissions (CSRF) while allowing normal link navigation. Users click a link to your site from an email and they stay logged in. An attacker's page submits a hidden form, and the cookie is withheld.

For most applications, `Lax` is the right choice. Change it only if you understand the trade-offs.

---

6. Login Flow: Complete Example

Authentication, sessions, tokens, and security converge in the login flow. Here is a complete implementation.

The Login Route

```
Tina4::Router.post("/api/login").no_auth do |request, response|
  email = request.body["email"].to_s.strip
  password = request.body["password"].to_s.strip

  if email.empty? || password.empty?
    return response.call({ error: "Email and password required" }, 400)
  end

  # Look up user (replace with your database query)
  user = db.fetch_one(
    "SELECT id, email, password_hash, role FROM users WHERE email = ?",
    [email]
  )

  if user.nil?
    return response.call({ error: "Invalid credentials" }, 401)
  end

  # Verify password
  unless Tina4::Auth.check_password(password, user["password_hash"])
    return response.call({ error: "Invalid credentials" }, 401)
  end

  # Generate token with user claims
  token = Tina4::Auth.get_token(
    { sub: user["id"], email: user["email"], role: user["role"] }
  )

  # Store user in session
  request.session["user_id"] = user["id"]
  request.session["email"] = user["email"]
  request.session["role"] = user["role"]
  request.session.save

  response.call({ token: token, user: { id: user["id"], email: user["email"] } })
end
```

The `.no_auth` chain opens this route to unauthenticated requests. The handler validates credentials and issues a token. The session stores the user identity for server-side lookups.

The Login Form

```
{% extends "base.twig" %}
{% block content %}
<div class="container mt-5" style="max-width: 400px;">
    <h2>Login</h2>
    <form id="loginForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="you@example.com" required>
        </div>
        <div class="mb-3">
            <label for="password">Password</label>
            <input type="password" name="password" id="password" class="form-control"
                   placeholder="Your password" required>
        </div>
        <button type="button" class="btn btn-primary w-100"
                onclick="saveForm('loginForm', '/api/login', 'loginMsg', handleLogin)">
            Sign In
        </button>
        <div id="loginMsg" class="mt-3"></div>
    </form>
</div>

<script>
function handleLogin(result) {
    if (result.token) {
        localStorage.setItem("token", result.token);
        window.location.href = "/dashboard";
    }
}
</script>
{% endblock %}
```

Protected Pages: Checking the Session

```
Tina4::Router.get "/dashboard" do |request, response|
  user_id = request.session["user_id"]

  if user_id.nil?
    return response.redirect("/login")
  end

  response.render("dashboard.twig", {
    email: request.session["email"],
    role: request.session["role"]
  })
end
```

Logout: Destroying the Session

```
Tina4::Router.post("/api/logout").no_auth do |request, response|
  request.session.destroy
  response.call({ logged_out: true })
end
```

---

7. Handling Expired Sessions

Sessions expire. Tokens expire. The user clicks a link and finds themselves staring at a broken page or a cryptic error. A good security implementation handles expiry gracefully.

The Pattern: Redirect to Login, Then Back

When a session expires mid-use, the user should:

- See a login page, not an error.
- Log in again.
- Land on the page they were trying to reach, not the home page.

```
require "cgi"

Tina4::Router.get "/account/settings" do |request, response|
  user_id = request.session["user_id"]

  if user_id.nil?
    # Remember where they wanted to go
    return_url = CGI.escape(request.url)
    return response.redirect("/login?redirect=#{return_url}")
  end

  response.render("settings.twig", { user_id: user_id })
end
```

The login handler reads the `redirect` parameter after successful authentication:

```
Tina4::Router.post("/api/login").no_auth do |request, response|
  # ... validate credentials ...

  redirect_url = request.params["redirect"] || "/dashboard"

  response.call({
    token: token,
    redirect: redirect_url
  })
end
```

The login form JavaScript redirects to the saved URL:

```
function handleLogin(result) {
    if (result.token) {
        localStorage.setItem("token", result.token);
        window.location.href = result.redirect || "/dashboard";
    }
}
```

Token Refresh

Tokens expire based on `TINA4_TOKEN_LIMIT` (default: 60 minutes). The `frond.min.js` frontend library handles token refresh automatically: every response includes a `FreshToken` header with a new token. The client stores it and uses it for the next request.

For custom AJAX code, read the header yourself:

```
const res = await fetch("/api/data", {
```

```
    headers: { "Authorization": "Bearer " + localStorage.getItem("token") }
});

const freshToken = res.headers.get("FreshToken");
if (freshToken) {
    localStorage.setItem("token", freshToken);
}
```

---

8. Rate Limiting

Brute-force login attempts. Credential stuffing. API abuse. Rate limiting stops all of them.

Tina4 includes a sliding-window rate limiter that tracks requests per IP address. It activates automatically.

```
TINA4_RATE_LIMIT=100
TINA4_RATE_WINDOW=60
```

One hundred requests per sixty seconds per IP. Exceed the limit and the server returns `429 Too Many Requests` with headers telling the client when to retry:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
```

For login routes, consider a stricter limit:

```
class LoginRateLimit
  def self.before_rate_check(request, response)
    # Custom rate limiter: 5 attempts per 60 seconds for login
    ip = request.ip
    # ... implement per-route rate limiting ...
    [request, response]
  end
end

Tina4::Router.post("/api/login").no_auth.middleware(LoginRateLimit) do |request, response|
  # ... login logic ...
end
```

---

9. CORS and Credentials

When your frontend runs on a different origin than your API (common in development), CORS controls whether the browser sends cookies and auth headers.

Tina4 handles CORS automatically. The relevant security settings:

```
TINA4_CORS_ORIGINS=*
TINA4_CORS_CREDENTIALS=true
```

Two rules to remember:

- `TINA4_CORS_ORIGINS=*` **with** `TINA4_CORS_CREDENTIALS=true` is invalid per the CORS spec. Tina4 handles this: when origin is `*`, the credentials header is not sent. But in production, list your actual origins.

- **Cookies need** `SameSite=None; Secure` for true cross-origin requests. If your API is on `api.example.com` and your frontend is on `app.example.com`, the default `Lax` cookie works because they share the same registrable domain. Different domains need `SameSite=None`.

Production CORS:

```
TINA4_CORS_ORIGINS=https://app.example.com,https://admin.example.com
TINA4_CORS_CREDENTIALS=true
```

---

10. Security Checklist

Before you deploy, verify:

- [ ] `TINA4_SECRET` is set to a long, random string, not the default.
- [ ] `TINA4_DEBUG=false` in production.
- [ ] `TINA4_HSTS=31536000` if serving over HTTPS.
- [ ] `TINA4_CORS_ORIGINS` lists your actual domains, not `*`.
- [ ] `TINA4_CSRF=true` (the default) for any browser-facing application.
- [ ] Login route uses `.no_auth` and validates credentials before issuing tokens.
- [ ] Session is regenerated after login (prevents session fixation).
- [ ] Passwords are hashed with `Tina4::Auth.hash_password()`, never stored in plain text.
- [ ] File uploads are validated and size-limited (`TINA4_MAX_UPLOAD_SIZE`).
- [ ] Rate limiting is active on login and registration routes.
- [ ] Expired sessions redirect to login with a return URL.

Gotchas

1. "My POST route returns 401 but I didn't add auth"

Cause: Tina4 requires authentication on all write routes by default.

Fix: Chain `.no_auth` onto the route definition if the endpoint should be public. Otherwise, send a valid Bearer token with the request.

2. "CSRF validation fails on AJAX requests"

Cause: The form token is not included in the request.

Fix: Send the token as an `X-Form-Token` header. If using `frond.min.js`, call `saveForm()`, which handles tokens automatically.

3. "I disabled CSRF but forms still fail"

Cause: The route still requires Bearer auth (separate from CSRF). CSRF and auth are independent checks.

Fix: Either send a Bearer token or chain `.no_auth` onto the route.

4. "My Content-Security-Policy blocks inline scripts"

Cause: The default CSP is `default-src 'self'`, which blocks inline `<script>` tags and `onclick` handlers.

Fix: Move scripts to external `.js` files (the right approach) or relax the CSP:

```
TINA4_CSP=default-src 'self'; script-src 'self' 'unsafe-inline'
```

Prefer external scripts. Inline scripts are an XSS vector.

5. "User stays logged in after session expires"

Cause: The frontend stores a JWT in localStorage. The token is still valid even after the session is destroyed server-side.

Fix: Check the session on every page load. If the session is gone, redirect to login regardless of the token. Tokens authenticate API calls; sessions track server-side state. Both must be valid.

Exercise: Secure Contact Form

Build a public contact form that:

- Does not require login (`.no_auth`).
- Validates CSRF tokens (form includes `{{ form_token() }}`).
- Rate-limits submissions to 3 per minute per IP.
- Stores messages in the database.
- Returns a success message.

Solution

```
# src/routes/contact.rb

Tina4::Router.get "/contact" do |request, response|
  response.render("contact.twig", { title: "Contact Us" })
end

Tina4::Router.post("/api/contact").no_auth do |request, response|
  name = request.body["name"].to_s.strip
  email = request.body["email"].to_s.strip
  message = request.body["message"].to_s.strip

  if name.empty? || email.empty? || message.empty?
    return response.call({ error: "All fields are required" }, 400)
  end

  db.insert("contact_messages", {
    name: name,
    email: email,
    message: message
  })

  response.call({ success: true, message: "Thank you for your message" })
end
```

```
{# src/templates/contact.twig #}
{% extends "base.twig" %}
{% block title %}Contact Us{% endblock %}
{% block content %}
<div class="container mt-5" style="max-width: 500px;">
    <h2>{{ title }}</h2>
    <form id="contactForm">
        {{ form_token() }}
        <div class="mb-3">
            <label for="name">Name</label>
            <input type="text" name="name" id="name" class="form-control"
                   placeholder="Jane Smith" required>
        </div>
        <div class="mb-3">
            <label for="email">Email</label>
            <input type="email" name="email" id="email" class="form-control"
                   placeholder="jane@example.com" required>
        </div>
        <div class="mb-3">
            <label for="message">Message</label>
            <textarea name="message" id="message" class="form-control" rows="4"
                      placeholder="How can we help?" required></textarea>
        </div>
        <button type="button" class="btn btn-primary"
                onclick="saveForm('contactForm', '/api/contact', 'contactMsg')">
            Send Message
        </button>
        <div id="contactMsg" class="mt-3"></div>
    </form>
</div>
{% endblock %}
```

The form is public. The CSRF token is present. The `.no_auth` chain opens the route. The middleware validates the token. The database stores the message. The user sees confirmation.

Five moving parts. Zero security holes. The framework handles the rest.

Caching

1. From 800ms to 3ms

Your product catalog page runs 12 database queries. 800 milliseconds to render. Every visitor triggers the same queries, the same template rendering, the same JSON serialization -- for data that changes once a day.

Add caching. The first request takes 800ms. The next 10,000 take 3ms each. A 266x improvement from one line of configuration.

Caching stores the result of expensive operations for reuse. Tina4 gives you three layers, and they stack:

- **Request-scoped query cache** -- on by default. Dedupes identical reads inside a single request. Flushes on every write.
- **Persistent database query cache** -- opt-in. Holds query results across requests until they expire.
- **Response cache middleware** -- caches whole HTTP responses so the route handler never runs.

This chapter covers all three, plus the direct key-value API you use for everything else.

---

2. Layer 1: Request-Scoped Query Cache

The first layer needs no setup. Tina4 caches database reads for the lifetime of a single request. Run the same query twice in one handler and the second call returns the first result -- no second trip to the database.

```
Tina4::Router.get("/api/dashboard") do |request, response|
  db = Tina4.database

  # First call hits the database
  user_count = db.fetch_one("SELECT COUNT(*) AS c FROM users")

  # ... other work ...

  # Same query, same request: served from the request-scoped cache
  user_count_again = db.fetch_one("SELECT COUNT(*) AS c FROM users")

  response.json({ users: user_count["c"] })
end
```

This layer is controlled by two environment variables:

```
TINA4_AUTO_CACHING=true       # default -- set to false to turn it off
TINA4_AUTO_CACHING_TTL=5      # default TTL in seconds
```

The cache clears at the start of each request, so one request never sees another request's data. Any write -- `insert`, `update`, `delete`, `execute` -- flushes the cache immediately, so a read after a write always reflects the change.

It lives in process and never touches the network. You get it for free.

---

3. Layer 2: Persistent Database Query Cache

The second layer survives across requests. Turn it on when the same queries run on every request and the underlying data changes rarely.

```
TINA4_DB_CACHE=true           # opt-in -- off by default
TINA4_DB_CACHE_TTL=30         # default TTL in seconds
```

With persistent caching on, identical queries return cached results until the TTL expires:

```
Tina4::Router.get("/api/categories") do |request, response|
  db = Tina4.database

  # First call: executes the query
  # Subsequent calls within 30 seconds (across requests): cached result
  categories = db.fetch("SELECT * FROM categories ORDER BY name")

  response.json({ categories: categories })
end
```

The cache key derives from the SQL and its parameters. Different queries or different parameters get different keys:

```
# Cached separately:
db.fetch("SELECT * FROM products WHERE category = ?", ["Electronics"])
db.fetch("SELECT * FROM products WHERE category = ?", ["Fitness"])
```

Any write operation (`insert`, `update`, `delete`, `execute`) invalidates the cache.

Sharing the cache across instances

By default the persistent cache lives in process memory. To share one cache across multiple server instances, route it through a backend:

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_BACKEND=redis
TINA4_DB_CACHE_URL=redis://localhost:6379
```

`TINA4_DB_CACHE_BACKEND` accepts any of the backends listed in section 5. A shared backend means a write on one instance invalidates the cache for all of them.

Bypassing the cache per query

Some reads must always hit the database -- a live inventory count, a balance check. Pass `no_cache: true` to skip the cache for a single call. No lookup, no store, just a direct query:

```
# Always reads fresh from the database
balance = db.fetch_one(
  "SELECT balance FROM accounts WHERE id = ?",
  [account_id],
  no_cache: true
)
```

The flag works on all three read methods:

```
db.fetch(sql, params, limit: 100, offset: 0, no_cache: false)
db.fetch_one(sql, params, no_cache: false)
db.fetch_all(sql, params, limit: nil, offset: nil, no_cache: false)
```

`no_cache: true` bypasses both Layer 1 and Layer 2 for that call.

When to use the persistent DB cache

- Read-heavy applications where the same queries run repeatedly
- Reference data that changes infrequently (categories, countries, settings)
- Dashboard queries that aggregate large datasets

When not to use it

- Write-heavy applications where data changes constantly
- Queries with real-time requirements (use `no_cache: true` for those reads)
- Queries that must always return the latest data

---

4. Layer 3: Response Cache Middleware

The fastest cache lives at the HTTP response level. The `Tina4::ResponseCache` middleware stores the complete response (body, content type, status) and serves it on later requests without calling your route handler at all.

Attach it as middleware with a TTL:

```
Tina4::Router.get("/api/products", middleware: ["ResponseCache:300"]) do |request, response|
  # This handler runs 12 database queries and takes 800ms
  # With ResponseCache, it only runs once every 5 minutes

  puts "Handler called -- this should only appear once every 5 minutes"

  products = [
    { id: 1, name: "Wireless Keyboard", price: 79.99 },
    { id: 2, name: "USB-C Hub", price: 49.99 },
    { id: 3, name: "Monitor Stand", price: 129.99 }
  ]

  response.json({ products: products, generated_at: Time.now.utc.iso8601 })
end
```

The string form `"ResponseCache:300"` caches the response for 300 seconds (5 minutes). During those 5 minutes:

- The first request runs the handler (800ms)
- The next 10,000 requests serve the cached response (3ms each)
- After 300 seconds, the cache expires and the next request runs the handler again

```
curl http://localhost:7147/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
```

```
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00Z"
}
```

Call it again within 5 minutes. The `generated_at` timestamp stays the same. The handler did not run -- the response came from cache.

The middleware only caches `GET` requests that return a `200` status.

Default TTL and limits

Without a TTL in the middleware string, the response cache reads its settings from the environment:

```
TINA4_CACHE_TTL=60            # default response-cache TTL in seconds
TINA4_CACHE_MAX_ENTRIES=1000  # maximum cached entries (LRU eviction)
```

Cache headers

The `ResponseCache` middleware stamps two headers on every response it handles:

```
X-Cache: HIT
X-Cache-TTL: 247
```

- `X-Cache: HIT` or `X-Cache: MISS` tells you whether the response came from cache
- `X-Cache-TTL` shows the remaining TTL in seconds on a HIT, or the configured TTL on a MISS

Caching with query parameters

The cache key includes the full URL with query parameters. `/api/products?page=1` and `/api/products?page=2` are cached separately:

```
curl "http://localhost:7147/api/products?page=1"  # MISS, stores for page=1
curl "http://localhost:7147/api/products?page=2"  # MISS, stores for page=2
curl "http://localhost:7147/api/products?page=1"  # HIT
```

What not to cache

Do not use `ResponseCache` on:

- **POST, PUT, PATCH, DELETE routes**: only GET responses are cached
- **User-specific endpoints**: `/api/profile` returns different data per user, but the cache keys on URL alone
- **Real-time data**: stock prices, live scores, chat messages
- **Authenticated endpoints**: unless every user shares the same response

---

5. Cache Backends

The response cache and the direct key-value API share one backend, selected with `TINA4_CACHE_BACKEND`. Seven backends ship in the box:

Backend		Notes
Memory	`memory` (default)	In-process LRU. Fastest. Lost on restart.
File	`file`	JSON files on disk. Survives restarts, zero dependencies.
Redis	`redis`	Shared across instances, sub-millisecond.
Valkey	`valkey`	Redis wire protocol; reports as `valkey`.
Memcached	`memcached`	Zero-dependency text protocol.
MongoDB	`mongodb`	TTL collection (requires the `mongo` gem).
Database	`database`	A `tina4_cache` table in any Tina4-supported database.

```
# Memory is the default -- you do not need to set it
TINA4_CACHE_BACKEND=memory
```

Redis

For cache that survives restarts and is shared across instances behind a load balancer:

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379
```

Your code does not change. `cache_get`, `cache_set`, and the `ResponseCache` middleware all work the same way -- only the storage changes. If your credentials are not in the URL, set them separately:

```
TINA4_CACHE_USERNAME=cacheuser
TINA4_CACHE_PASSWORD=secret
```

Valkey, Memcached, and MongoDB use the same single `TINA4_CACHE_URL` (Memcached is unauthenticated).

File

When you want persistence but cannot run Redis:

```
TINA4_CACHE_BACKEND=file
TINA4_CACHE_DIR=/path/to/cache/directory
```

File cache stores each entry as a JSON file on disk. Slower than memory or Redis, but it survives restarts with no extra infrastructure.

Graceful fallback

If a configured backend's driver is missing or its service is unreachable, Tina4 logs a warning and falls back to the **file** backend -- a real, working, persistent cache. It never degrades to a silent no-op. A degraded backend reports its real name (`file`) in `cache_stats`, so you can see the fallback happened.

---

6. The Direct Cache API

For custom caching logic, use the key-value helpers on the `Tina4` module: `cache_get`, `cache_set`, `cache_delete`, `clear_cache`, and `cache_stats`.

cache_set

```
# Cache a value for 300 seconds
Tina4.cache_set("product:42", {
  id: 42,
  name: "Wireless Keyboard",
  price: 79.99,
  in_stock: true
}, ttl: 300)

# Cache a string
Tina4.cache_set("exchange_rate:USD_EUR", "0.92", ttl: 3600)
```

`ttl:` defaults to `0`, which falls back to the configured default TTL.

cache_get

```
product = Tina4.cache_get("product:42")
# Returns the cached value, or nil if not found or expired

if product.nil?
  # Cache miss -- fetch from database
  product = fetch_product_from_database(42)
  Tina4.cache_set("product:42", product, ttl: 300)
end
```

cache_delete

```
# Delete a specific key
Tina4.cache_delete("product:42")

# Delete several keys
Tina4.cache_delete("product:42")
Tina4.cache_delete("product:43")
Tina4.cache_delete("product:44")
```

clear_cache

```
# Flush every entry in the cache
Tina4.clear_cache
```

Real-world pattern: cache-aside

The most common caching pattern is cache-aside (also called lazy loading):

```
Tina4::Router.get("/api/products/{id:int}") do |request, response|
  product_id = request.params["id"]
  cache_key = "product:#{product_id}"

  # 1. Try the cache first
  product = Tina4.cache_get(cache_key)

  unless product.nil?
    # Cache hit -- return immediately
    return response.json(product.merge(source: "cache"))
  end

  # 2. Cache miss -- fetch from database
  db = Tina4.database
  product = db.fetch_one(
    "SELECT id, name, category, price, in_stock FROM products WHERE id = ?",
    [product_id]
  )

  if product.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  # 3. Store in cache for next time
  Tina4.cache_set(cache_key, product, ttl: 600)  # Cache for 10 minutes

  response.json(product.merge(source: "database"))
end
```

```
curl http://localhost:7147/api/products/42
```

First call (cache miss):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "database"
}
```

Second call (cache hit):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "cache"
}
```

---

7. Cache Invalidation Strategies

Cache invalidation is the hard problem. Stale cache serves outdated data. Premature invalidation throws away performance gains. Three strategies handle this.

Strategy 1: Time-based expiry (TTL)

The simplest strategy. Set a TTL and let the cache expire naturally:

```
Tina4.cache_set("products:featured", featured_products, ttl: 600)  # Expires in 10 minutes
```

Good for data where near-real-time accuracy is acceptable. A 10-minute delay in updating the featured products list is usually fine.

Strategy 2: Event-based invalidation

Clear the cache when the underlying data changes:

```
Tina4::Router.put("/api/products/{id:int}") do |request, response|
  product_id = request.params["id"]
  body = request.body
  db = Tina4.database

  db.execute(
    "UPDATE products SET name = ?, price = ? WHERE id = ?",
    [body["name"], body["price"], product_id]
  )

  # Invalidate the cache for this product
  Tina4.cache_delete("product:#{product_id}")

  # Also invalidate list caches that might include this product
  Tina4.cache_delete("products:all")
  Tina4.cache_delete("products:featured")

  updated = db.fetch_one("SELECT * FROM products WHERE id = ?", [product_id])

  response.json(updated)
end
```

This is the most accurate strategy -- the cache is always fresh after a write. The downside: you must remember to invalidate everywhere the data could be cached.

Strategy 3: Write-through cache

Update the cache at the same time as the database:

```
Tina4::Router.put("/api/products/{id:int}") do |request, response|
  product_id = request.params["id"]
  body = request.body
  db = Tina4.database

  db.execute(
    "UPDATE products SET name = ?, price = ? WHERE id = ?",
    [body["name"], body["price"], product_id]
  )

  updated = db.fetch_one("SELECT * FROM products WHERE id = ?", [product_id])

  # Write the new data directly to cache (instead of deleting)
  Tina4.cache_set("product:#{product_id}", updated, ttl: 600)

  response.json(updated)
end
```

The cache always has the latest data. No cache miss after an update -- the next read comes from the already-warm cache.

---

8. TTL Management

Choosing the right TTL depends on how often the data changes and how acceptable stale data is:

Data Type	Suggested TTL	Reasoning
Static config (categories, countries)	3600 (1 hour)	Changes rarely, stale data is harmless
Product catalog	300 (5 min)	Updates several times per day
User profile	60 (1 min)	Users expect changes to appear quickly
Search results	120 (2 min)	Balance between freshness and performance
Dashboard stats	30 (30 sec)	Near-real-time but expensive to compute
Exchange rates	60 (1 min)	Updates frequently, slight delay is acceptable
Shopping cart	0 (no cache)	Must always reflect current state

Dynamic TTL

Adjust TTL based on data characteristics:

```
def get_cached_product(product_id)
  cache_key = "product:#{product_id}"
  product = Tina4.cache_get(cache_key)

  return product unless product.nil?

  product = fetch_product_from_database(product_id)

  # Popular products: shorter TTL (more likely to change)
  # Inactive products: longer TTL (rarely change)
  ttl = product["view_count"].to_i > 1000 ? 60 : 3600

  Tina4.cache_set(cache_key, product, ttl: ttl)

  product
end
```

---

9. Cache Statistics

Two stat surfaces help you verify that caching helps.

Response / KV cache stats

`Tina4.cache_stats` reports the shared response and key-value cache:

```
Tina4::Router.get("/api/cache/stats") do |request, response|
  response.json(Tina4.cache_stats)
end
```

```
curl http://localhost:7147/api/cache/stats
```

```
{
  "hits": 15234,
  "misses": 891,
  "size": 42,
  "backend": "memory",
  "keys": ["GET:/api/products", "direct:product:42"]
}
```

The fields are `hits`, `misses`, `size`, `backend`, and `keys`. The `keys` array is populated for the memory backend and empty for the others.

Hit rate above 90%: your caching strategy works. Below 80%: TTLs are too short, the cache is too small, or you are caching data that does not benefit from caching.

Database cache stats

`db.cache_stats` reports the query cache on a database connection:

```
Tina4::Router.get("/api/db/cache-stats") do |request, response|
  response.json(Tina4.database.cache_stats)
end
```

```
{
  "enabled": true,
  "mode": "request",
  "hits": 318,
  "misses": 47,
  "size": 12,
  "backend": "memory",
  "ttl": 5
}
```

The `mode` field tells you which layer is active:

- `"request"` -- request-scoped caching (the default)
- `"persistent"` -- `TINA4_DB_CACHE=true` is set
- `"off"` -- both layers are disabled

---

10. Combining Cache Layers

For maximum performance, stack all three layers:

```
Tina4::Router.get("/api/catalog", middleware: ["ResponseCache:60"]) do |request, response|
  page = (request.params["page"] || 1).to_i
  cache_key = "catalog:page:#{page}"

  # Layer A: application cache (direct KV API)
  catalog = Tina4.cache_get(cache_key)

  unless catalog.nil?
    return response.json(catalog.merge(cache: "application"))
  end

  # Layer B: database queries (request-scoped + persistent DB cache apply here)
  db = Tina4.database
  limit = 20
  offset = (page - 1) * limit

  products = db.fetch(
    "SELECT p.*, c.name AS category_name
     FROM products p
     JOIN categories c ON p.category_id = c.id
     WHERE p.active = 1
     ORDER BY p.created_at DESC",
    [], limit: limit, offset: offset
  )

  total = db.fetch_one("SELECT COUNT(*) AS count FROM products WHERE active = 1")

  catalog = {
    products: products,
    page: page,
    total: total["count"],
    pages: (total["count"].to_f / limit).ceil,
    generated_at: Time.now.utc.iso8601
  }

  # Store in application cache
  Tina4.cache_set(cache_key, catalog, ttl: 300)

  response.json(catalog.merge(cache: "none"))
end
```

This creates three working layers:

- **ResponseCache** (60 seconds): the entire HTTP response is cached. No Ruby code runs at all.
- **Application cache** (300 seconds): if the response cache expired but the app cache is fresh, skip the database queries.
- **DB query cache**: individual query results are cached -- request-scoped by default, persistent if `TINA4_DB_CACHE=true`.

The first visitor after a full cache expiry waits 800ms. Everyone else gets the response in under 5ms.

---

11. Exercise: Cache an Expensive Product Listing Endpoint

Build a product listing endpoint that caches at multiple levels.

Requirements

- Create a `GET /api/store/products` endpoint that:
- Accepts query parameters: `category`, `page`, `limit`
- Returns a list of products with pagination metadata
- Uses the direct cache API (`Tina4.cache_get` / `Tina4.cache_set`) with a 5-minute TTL
- Includes a `source` field in the response (`"cache"` or `"database"`)
- Create a `POST /api/store/products` endpoint that:
- Creates a new product
- Invalidates the cached product lists
- Create a `GET /api/store/cache-stats` endpoint that shows cache statistics

Test with:

```
# First call -- cache miss, slow
curl "http://localhost:7147/api/store/products?category=Electronics&page=1"

# Second call -- cache hit, fast
curl "http://localhost:7147/api/store/products?category=Electronics&page=1"

# Different category -- cache miss
curl "http://localhost:7147/api/store/products?category=Fitness&page=1"

# Create a product -- should invalidate cache
curl -X POST http://localhost:7147/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'

# Same query again -- cache miss (invalidated by the POST)
curl "http://localhost:7147/api/store/products?category=Electronics&page=1"

# Check cache stats
curl http://localhost:7147/api/store/cache-stats
```

---

12. Solution

Create `src/routes/store_cached.rb`:

```
require "digest"
require "json"

PRODUCT_STORE = [
  { id: 1, name: "Wireless Keyboard", category: "Electronics", price: 79.99, in_stock: true },
  { id: 2, name: "Yoga Mat", category: "Fitness", price: 29.99, in_stock: true },
  { id: 3, name: "Coffee Grinder", category: "Kitchen", price: 49.99, in_stock: false },
  { id: 4, name: "Standing Desk", category: "Electronics", price: 549.99, in_stock: true },
```

```
  { id: 5, name: "Running Shoes", category: "Fitness", price: 119.99, in_stock: true },
  { id: 6, name: "Bluetooth Speaker", category: "Electronics", price: 39.99, in_stock: true },
  { id: 7, name: "Resistance Bands", category: "Fitness", price: 14.99, in_stock: true },
  { id: 8, name: "French Press", category: "Kitchen", price: 34.99, in_stock: true }
]

Tina4::Router.get("/api/store/products") do |request, response|
  category = request.params["category"]
  page = (request.params["page"] || 1).to_i
  limit = (request.params["limit"] || 20).to_i

  # Build cache key from query parameters
  key_data = JSON.generate({ category: category, page: page, limit: limit })
  cache_key = "store:products:#{Digest::MD5.hexdigest(key_data)}"

  # Try cache first
  cached = Tina4.cache_get(cache_key)

  unless cached.nil?
    return response.json(cached.merge("source" => "cache"))
  end

  # Simulate expensive database query
  sleep(0.1)

  products = PRODUCT_STORE.dup

  # Filter by category
  if category
    products = products.select { |p| p[:category].downcase == category.downcase }
  end

  total = products.length
  offset = (page - 1) * limit
  products = products[offset, limit] || []

  result = {
    products: products,
    page: page,
    limit: limit,
    total: total,
    pages: (total.to_f / limit).ceil,
    generated_at: Time.now.utc.iso8601
  }

  # Cache for 5 minutes
  Tina4.cache_set(cache_key, result, ttl: 300)

  response.json(result.merge(source: "database"))
end

Tina4::Router.post("/api/store/products") do |request, response|
  body = request.body

  if body["name"].nil? || body["name"].empty?
    return response.json({ error: "Name is required" }, 400)
  end

  product = {
```

```
    id: rand(100..9999),
    name: body["name"],
    category: body["category"] || "General",
    price: (body["price"] || 0).to_f,
    in_stock: true
  }

  # Invalidate all product list caches
  Tina4.clear_cache

  response.json({
    message: "Product created",
    product: product,
    cache_invalidated: true
  }, 201)
end

Tina4::Router.get("/api/store/cache-stats") do |request, response|
  response.json(Tina4.cache_stats)
end
```

Expected output -- first call (cache miss):

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock"
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": t
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock"
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generated_at": "2026-03-22T14:30:00Z",
  "source": "database"
}
```

Expected output -- second call (cache hit):

Same data, but `source` changes to `"cache"`. The handler did not run.

---

13. Gotchas

1. Caching authenticated responses

Problem: User A's profile is served to User B because the response was cached.

Cause: `ResponseCache` keys on URL only. If `/api/profile` returns different data per user but every request hits the same URL, the first user's response is served to everyone.

Fix: Do not use `ResponseCache` on user-specific endpoints. Use the direct cache API with user-specific keys: `Tina4.cache_set("profile:#{user_id}", data, ttl: 300)`.

2. Memory cache lost on restart

Problem: After restarting the server, performance drops until the cache warms up.

Cause: The memory backend lives in process and is lost when the process restarts.

Fix: For production, use the Redis backend (`TINA4_CACHE_BACKEND=redis`). It survives restarts and is shared across instances. Or write a warmup script that pre-populates frequently accessed keys.

3. Stale data after a database update

Problem: You updated a product's price in the database, but the API still returns the old price.

Cause: The cache still holds the old value and has not expired.

Fix: Invalidate or update the cache when you change the underlying data. Use `Tina4.cache_delete("product:#{product_id}")` after an update, or write through with the new value. For a single live read, pass `no_cache: true` to the query.

4. Cache key collisions

Problem: Two different queries return the same cached data.

Cause: Your cache keys are not specific enough. Using `"products"` for both the full list and a filtered list causes collisions.

Fix: Include every relevant parameter in the key: `"products:category:Electronics:page:1:limit:20"`. Or hash the parameters with MD5.

5. Serialization overhead

Problem: Caching makes certain requests slower, not faster.

Cause: The cached object is very large. Serializing and deserializing it costs more than recomputing it.

Fix: Only cache data that is expensive to compute. If the original operation takes 5ms and cache serialization takes 10ms, caching hurts. Profile before and after to confirm the win.

6. Forgetting that writes flush the request cache

Problem: You expected a read to come from cache, but it hit the database.

Cause: A write earlier in the same request flushed the request-scoped query cache. That is by design -- a read after a write must reflect the change.

Fix: This is correct behavior. If you need a value to survive a write within one request, capture it in a local variable before the write, or store it with the direct cache API.

7. Reaching for a backend you do not have

Problem: You set `TINA4_CACHE_BACKEND=redis` but `cache_stats` reports `"backend": "file"`.

Cause: Redis was unreachable or its driver was missing, so Tina4 fell back to the file backend and logged a warning.

Fix: This is the graceful-fallback safety net -- your cache still works. Check the logs, fix the connection (`TINA4_CACHE_URL`, credentials, the service itself), and the reported backend returns to `redis`.

Queue System

1. Do Not Make the User Wait

Your app sends welcome emails on signup. Generates PDF invoices. Resizes uploaded images. Each task takes 2 to 30 seconds. Run them inside the HTTP request and the user stares at a spinner while the server grinds through email delivery, invoice rendering, and image resizing.

Queues move slow work to a background process. The HTTP handler drops a job onto a queue and responds to the user in under 100 milliseconds. A separate consumer picks up the job and does the work at its own pace. The user sees "Welcome! Check your email." The email arrives 5 seconds later.

Tina4's queue system works out of the box with a file-based backend. No Redis. No RabbitMQ. No external services. Add jobs and process them.

The providers share delivery semantics, but they do not pretend every broker supports every management operation:

Provider	Supported contract	Explicitly unavailable
File / lite	Every queue operation	None
MongoDB	Every queue operation	None
RabbitMQ	Push, consume, complete, fail/reject, stable non-destructive dead letters, broker size where available, close	Priority, delay, pop by ID, purge by status, failed-job listing, retry failed
Kafka	Delivery, acknowledge, fail, stable non-destructive dead letters; `size` honestly returns `0`	Priority, delay, pop by ID, failed-job listing, retry failed

This capability contract is defined by ADR-0022, ADR-0023, and ADR-0024. An unsupported operation raises an error naming the backend and operation. It never succeeds as a no-op or returns a misleading empty result.

---

2. Why Queues Matter

Without queues:

```
User clicks "Sign Up"
  -> Server validates input (10ms)
  -> Server creates user in database (20ms)
  -> Server sends welcome email (3000ms)
  -> Server generates PDF welcome kit (2000ms)
```

```
  -> Server resizes avatar (1500ms)
  -> User sees response (6530ms later)
```

With queues:

```
User clicks "Sign Up"
  -> Server validates input (10ms)
  -> Server creates user in database (20ms)
  -> Server queues: send welcome email (1ms)
  -> Server queues: generate PDF (1ms)
  -> Server queues: resize avatar (1ms)
  -> User sees response (33ms later)

Meanwhile, in the background:
  -> Consumer sends welcome email
  -> Consumer generates PDF
  -> Consumer resizes avatar
```

6.5 seconds becomes 33 milliseconds. The user feels the difference.

---

3. File Queue (Default)

The file-based backend is the default. No configuration needed.

Creating a Queue and Pushing a Job

```
queue = Tina4::Queue.new(topic: "emails")

# Push a job -- returns the Job that was queued
job = queue.push({
  to: "alice@example.com",
  subject: "Order Confirmation",
  body: "Your order #1234 has been confirmed."
})

puts job.id  # the generated job id
```

`push` returns a `Tina4::Job`, not an integer. Read `job.id` when you need the identifier.

Convenience Method: produce

The `produce` method pushes to a specific topic without creating a separate Queue instance. It also returns the `Job`:

```
queue = Tina4::Queue.new(topic: "emails")
queue.produce("invoices", { order_id: 101, format: "pdf" })
```

Push with Priority

Jobs default to priority 0 (normal). The queue pops the highest priority first, and breaks ties oldest-first:

```
# Normal priority (default)
queue.push({ to: "alice@example.com", subject: "Newsletter" })

# High priority -- popped before normal jobs
queue.push({ to: "alice@example.com", subject: "Password Reset" }, priority: 10)
```

Delaying a Job

Pass `delay_seconds` to hold a job back. It stays invisible to consumers until the delay elapses:

```
# Becomes available 60 seconds from now
queue.push({ to: "alice@example.com", subject: "Reminder" }, delay_seconds: 60)
```

Queue Size

Check how many pending messages are in the queue:

```
count = queue.size
```

Pass a status keyword to count jobs in a specific state. `"failed"` and `"dead"` both count jobs in the dead-letter store:

```
dead = queue.size(status: "dead")
```

---

4. Pushing from Route Handlers

```
Tina4::Router.post("/api/register") do |request, response|
  body = request.body

  user_id = 42 # Simulated

  queue = Tina4::Queue.new(topic: "emails")

  queue.push({
    user_id: user_id,
    to: body["email"],
    name: body["name"],
    subject: "Welcome!"
  })

  response.json({
    message: "Registration successful. Welcome email will arrive shortly.",
    user_id: user_id
  }, 201)
end
```

```
curl -X POST http://localhost:7147/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful. Welcome email will arrive shortly.",
  "user_id": 42
}
```

---

5. Consuming Jobs

The `consume` method yields jobs one at a time via a block. Each job must be explicitly completed or failed:

```
queue = Tina4::Queue.new(topic: "emails")
```

```
queue.consume("emails") do |job|
  begin
    send_email(job.payload[:to], job.payload[:subject], job.payload[:body])
    job.complete
  rescue => e
    job.fail(e.message)
  end
end
```

`consume` polls forever by default. When the queue is empty it sleeps for `poll_interval` seconds (default 1.0) and polls again. To drain the queue once and stop, pass `poll_interval: 0`. To stop after a set number of jobs, pass `iterations:`.

Automatic Retry

When you call `job.fail(reason)`, the queue handles the retry for you. It increments the job's attempt count and puts the job straight back onto the pending queue. The next `pop` or `consume` iteration picks it up again. Once a job has been attempted `max_retries` times, the queue stops retrying and moves it to the dead-letter store.

So the consumer loop below retries a failing job up to `max_retries` times on its own, then dead-letters it. No manual `retry_failed` call is needed:

```
# Retry each failed job up to 3 times, then dead-letter it
queue = Tina4::Queue.new(topic: "emails", max_retries: 3)

queue.consume("emails") do |job|
  begin
    send_email(job.payload[:to], job.payload[:subject], job.payload[:body])
    job.complete
  rescue => e
    job.fail(e.message)  # auto re-enqueues until attempts == max_retries
  end
end
```

Retry Backoff

By default a failed job is retried on the very next poll. To wait before the next attempt, set `retry_backoff` (in seconds) when you create the queue:

```
# Wait 30 seconds before each retry
queue = Tina4::Queue.new(topic: "emails", max_retries: 3, retry_backoff: 30)
```

Manual Retry with Delay

`job.retry` is a manual override, distinct from the automatic `fail` path. It always re-queues the job regardless of the retry limit and increments the attempt count. The job already carries its queue reference from `pop`/`consume`, so you only pass the delay:

```
queue.consume("emails") do |job|
  begin
    send_email(job.payload[:to], job.payload[:subject], job.payload[:body])
    job.complete
  rescue => e
    # Re-queue this job to run again 30 seconds from now
    job.retry(delay_seconds: 30)
  end
end
```

Manual Pop

For more control, pop a single message. `pop` returns the highest-priority available job, or `nil` when the queue is empty:

```
job = queue.pop

unless job.nil?
  begin
    send_email(job.payload[:to], job.payload[:subject])
    job.complete
  rescue => e
    job.fail(e.message)
  end
end
```

Pop a Specific Job by ID

Pull one known job out of the pending queue with `pop_by_id`. It returns the matching `Job` (claimed from the queue) or `nil` if no pending job has that id:

```
job = queue.pop_by_id("abc-123")
job.complete if job
```

`consume` accepts the same lookup via `id:` -- it processes that single job and returns:

```
queue.consume("emails", id: "abc-123") do |job|
  send_email(job.payload[:to], job.payload[:subject])
  job.complete
end
```

---

6. Job Lifecycle

Every job moves through these states:

```
push -> PENDING -> pop/consume -> RESERVED -> job.complete -> COMPLETED (terminal)
                                           -> job.fail     -> attempts += 1
                                                                  |
                                              attempts < max_retries
                                                                  |
                                                        PENDING (auto re-queue)
                                                                  |
                                              attempts >= max_retries
                                                                  |
                                                             DEAD LETTER
```

`job.fail` does the bookkeeping automatically: it re-queues the job while it still has retries left, and moves it to the dead-letter store once it has been attempted `max_retries` times.

Job Methods

When you receive a job from `consume` or `pop`, you have these methods:

- `job.complete` -- mark the job as done. Terminal; the job is removed.
- `job.fail(reason)` -- record a failed attempt with a reason string. Auto re-queues while retries remain, then dead-letters.

- `job.reject(reason)` -- alias for `fail`.
- `job.retry(delay_seconds: 0)` -- manual override. Re-queue the job after an optional delay (in seconds), regardless of the retry limit.

Job Properties

- `job.id` -- the generated job id.
- `job.payload` -- the data you pushed.
- `job.topic` -- the topic this job belongs to. Useful when consuming from multiple topics.
- `job.attempts` -- how many times the job has been attempted.
- `job.error` -- the reason from the last `fail`/`reject`.

Always call `complete`, `fail`, or `retry` on every job. If you do not, the job stays reserved.

---

7. Retry and Dead Letters

Max Retries

The default `max_retries` is 3. When a job's attempt count reaches `max_retries`, the queue stops retrying it and moves it to the dead-letter store. Set the limit when you create the queue.

Inspecting Retrying Jobs

`failed` returns the jobs that have failed at least once but are still being retried (attempts above zero, under `max_retries`). These jobs live in the pending queue and will be picked up again automatically. `failed` returns an array of **Hashes**, so use string keys:

```
retrying = queue.failed

retrying.each do |job|
  puts "Retrying job: #{job["id"]} (attempt #{job["attempts"]})"
  puts "  Last error: #{job["error"]}"
end
```

Dead Letters

Jobs that exhausted `max_retries` are dead letters. `dead_letters` returns an array of **Hashes** -- access fields with string keys:

```
dead_jobs = queue.dead_letters

dead_jobs.each do |job|
  puts "Dead job: #{job["id"]}"
  puts "  Payload: #{job["payload"]}"
  puts "  Error: #{job["error"]}"
end
```

Reviving Dead Letters

`retry_failed` revives dead-letter jobs (those still under `max_retries`) back to the pending queue and returns the count re-queued. To revive one specific dead-letter job by id, use `queue.retry(job_id)`:

```
# Revive every eligible dead letter
queue.retry_failed

# Revive one specific dead-letter job
queue.retry("abc-123")
```

Purging Jobs

Remove jobs by status. `"failed"` and `"dead"` remove from the dead-letter store; `"pending"` removes matching jobs from the pending queue:

```
queue.purge("dead")
queue.purge("pending")
```

---

8. Producing Multiple Jobs

Sometimes a single action triggers multiple background tasks:

```
Tina4::Router.post("/api/orders") do |request, response|
  body = request.body
  order_id = 101

  queue = Tina4::Queue.new(topic: "emails")

  queue.push({
    order_id: order_id,
    to: body["email"],
    subject: "Order Confirmation"
  })

  queue.produce("invoices", {
    order_id: order_id,
    format: "pdf"
  })

  queue.produce("inventory", {
    items: body["items"]
  })

  queue.produce("warehouse", {
    order_id: order_id,
    shipping_address: body["shipping_address"]
  })

  response.json({
    message: "Order placed successfully",
    order_id: order_id
  }, 201)
end
```

Four jobs are queued in under 5 milliseconds. The user gets an instant response.

---

9. Switching Backends

Switching backends is a config change, not a code change.

Default: File

```
# No config needed -- file is the default.
# Jobs are stored as JSON files under ./.queue in the working directory.
```

To store jobs somewhere else, pass a `dir:` option to the backend rather than an environment variable:

```
backend = Tina4::QueueBackends::LiteBackend.new(dir: "./data/queue")
queue = Tina4::Queue.new(topic: "emails", backend: backend)
```

RabbitMQ

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://user:pass@localhost:5672
```

Kafka

```
TINA4_QUEUE_BACKEND=kafka
TINA4_QUEUE_URL=localhost:9092
```

MongoDB

```
TINA4_QUEUE_BACKEND=mongodb
TINA4_QUEUE_URL=mongodb://user:pass@localhost:27017/tina4
```

Install the MongoDB driver:

```
gem install mongo
```

Your code does not change. The same `queue.push` and `queue.consume` calls work with every backend.

---

10. Separate Producer and Consumer Patterns

Use separate `Tina4::Queue` instances in different files or services for clarity:

```
# Producer side (e.g., in a route handler)
queue = Tina4::Queue.new(topic: "emails")
queue.push({ to: "alice@example.com", subject: "Hello" })

# Consumer side (e.g., in a worker script)
queue = Tina4::Queue.new(topic: "emails")
queue.consume("emails") do |job|
  process(job)
  job.complete
end
```

The same `Tina4::Queue` class handles both producing and consuming. Separate instances in different files make intent clearer.

---

11. Exercise: Build an Email Queue

Build a queue-based email system with failure handling.

Requirements

- Create these endpoints:

Method	Path	Description
`POST`	`/api/emails/send`	Queue an email for sending
`GET`	`/api/emails/queue`	Show pending email count
`GET`	`/api/emails/dead`	List dead letter jobs
`POST`	`/api/emails/retry`	Revive dead-letter jobs

- The email payload should include: `to` (required), `subject` (required), `body` (required)
- Create a consumer that processes the queue, simulating occasional failures

Test with:

```
curl -X POST http://localhost:7147/api/emails/send \
  -H "Content-Type: application/json" \
  -d '{"to": "alice@example.com", "subject": "Welcome!", "body": "Thanks for signing up."}'

curl http://localhost:7147/api/emails/queue

curl http://localhost:7147/api/emails/dead

curl -X POST http://localhost:7147/api/emails/retry
```

---

12. Solution

Create `src/routes/email_queue.rb`:

```
queue = Tina4::Queue.new(topic: "emails")

# @noauth
Tina4::Router.post("/api/emails/send") do |request, response|
  body = request.body

  errors = []
  errors << "'to' is required" if body["to"].nil? || body["to"].empty?
  errors << "'subject' is required" if body["subject"].nil? || body["subject"].empty?
  errors << "'body' is required" if body["body"].nil? || body["body"].empty?

  unless errors.empty?
```

```
    return response.json({ errors: errors }, 400)
  end

  job = queue.push({
    to: body["to"],
    subject: body["subject"],
    body: body["body"]
  })

  response.json({
    message: "Email queued for sending",
    job_id: job.id
  }, 201)
end

Tina4::Router.get("/api/emails/queue") do |request, response|
  count = queue.size
  response.json({ pending: count })
end

Tina4::Router.get("/api/emails/dead") do |request, response|
  dead_jobs = queue.dead_letters
  items = dead_jobs.map do |job|
    { id: job["id"], payload: job["payload"], error: job["error"] }
  end
  response.json({ dead_letters: items, count: items.length })
end

Tina4::Router.post("/api/emails/retry") do |request, response|
  queue.retry_failed
  response.json({ message: "Dead-letter emails re-queued" })
end
```

Create a separate consumer file `src/workers/email_worker.rb`:

```
queue = Tina4::Queue.new(topic: "emails", max_retries: 3)

queue.consume("emails") do |job|
  payload = job.payload

  puts "Sending email to #{payload[:to]}..."
  puts "  Subject: #{payload[:subject]}"

  begin
    # Simulate sending (replace with real email logic)
    sleep(1)

    # Simulate failure for a specific address
    if payload[:to] == "bad@example.com"
      raise "SMTP connection refused"
    end

    puts "  Email sent to #{payload[:to]} successfully!"
    job.complete

  rescue => e
    puts "  Failed: #{e.message}"
    job.fail(e.message)
  end
```

```
end
```

The consumer retries a failing job to `bad@example.com` automatically. After `max_retries` attempts, `queue.dead_letters` returns that job, and the `/api/emails/dead` endpoint shows it. You investigate, fix the address, and call `/api/emails/retry` to revive it.

---

13. Gotchas

1. Always call complete or fail

Problem: Jobs stay in reserved status forever.

Cause: Your consumer block does not call `job.complete` or `job.fail`. The job stays reserved and is never released.

Fix: Always call one of `job.complete`, `job.fail(reason)`, or `job.reject(reason)` in your consumer block.

2. Worker not picking up messages

Problem: Messages are pushed but nothing happens.

Cause: No consumer process is running, or the consumer is listening on a different topic.

Fix: Make sure the consumer is running. Check that the topic name in `queue.push` matches the topic in `queue.consume`.

3. Payload must be serializable

Problem: `queue.push` throws an error.

Cause: You passed a non-serializable object (database connection, file handle, etc.).

Fix: Only pass simple data types in the payload. If you need to reference a database record, pass the ID. The consumer looks up records by ID.

4. Dead letters pile up

Problem: Dead letters accumulate and nobody notices.

Cause: Jobs that exhaust `max_retries` become dead letters but are never cleaned up.

Fix: Monitor dead letters with `queue.dead_letters`. Set up an alert when the count exceeds a threshold. Investigate, fix, then call `queue.retry_failed` or `queue.purge("dead")`.

5. File backend for production

Problem: Multiple workers cause contention on the file backend.

Cause: The file backend is designed for single-worker setups.

Fix: For production with multiple workers, switch to RabbitMQ, Kafka, or MongoDB via the `TINA4_QUEUE_BACKEND` environment variable.

---

6. dead_letters and failed return Hashes

Problem: `job.id` or `job.payload` raises `NoMethodError` when iterating `dead_letters` or `failed`.

Cause: `dead_letters` and `failed` return arrays of Hashes, not `Job` objects.

Fix: Use string keys: `job["id"]`, `job["payload"]`, `job["error"]`, `job["attempts"]`.

Events

1. Decouple With Events

Your user registration handler validates input, creates the account, sends a welcome email, creates an activity log entry, and notifies an analytics service. Seven responsibilities crammed into one method. Adding an eighth means editing that method again.

Events decouple producers from consumers. The registration handler fires `user.registered`. Any number of listeners react to it (email, analytics, logging) without the handler knowing they exist. Adding a new reaction is adding a new listener.

Tina4's event system is in-process, synchronous by default, and zero dependency. No broker, no serialization, no network.

---

2. Basic Usage

Registering a Listener

```
Tina4::Events.on("user.registered") do |payload|
  puts "Welcome #{payload[:name]}!"
end
```

Emitting an Event

```
Tina4::Events.emit("user.registered", {
  id: 42,
  name: "Alice",
  email: "alice@example.com"
})
```

Output:

```
Welcome Alice!
```

Listeners receive whatever hash you pass to `emit`. Use symbol keys consistently.

---

3. Multiple Listeners

Register as many listeners as you like for the same event. All fire in registration order by default.

```
Tina4::Events.on("order.placed") do |payload|
  puts "Sending confirmation email to #{payload[:email]}"
end

Tina4::Events.on("order.placed") do |payload|
  puts "Reserving inventory for order ##{payload[:order_id]}"
end

Tina4::Events.on("order.placed") do |payload|
  puts "Notifying warehouse for shipping"
```

```
end

Tina4::Events.emit("order.placed", {
  order_id: 101,
  email: "alice@example.com",
  items: [{ sku: "KB-100", qty: 1 }]
})
```

Output:

```
Sending confirmation email to alice@example.com
Reserving inventory for order #101
Notifying warehouse for shipping
```

---

4. Priority

Control listener order with `priority`. Higher numbers run first.

```
Tina4::Events.on("payment.received", priority: 10) do |payload|
  puts "FRAUD CHECK first (priority 10)"
end

Tina4::Events.on("payment.received", priority: 5) do |payload|
  puts "Fulfill order second (priority 5)"
end

Tina4::Events.on("payment.received", priority: 1) do |payload|
  puts "Send receipt last (priority 1)"
end

Tina4::Events.emit("payment.received", { order_id: 201, amount: 99.99 })
```

Output:

```
FRAUD CHECK first (priority 10)
Fulfill order second (priority 5)
Send receipt last (priority 1)
```

Default priority is `0`. Listeners with the same priority run in registration order.

---

5. Once: Fire a Listener One Time

`once` registers a listener that fires exactly once and then removes itself.

```
Tina4::Events.once("app.started") do |payload|
  puts "App started at #{payload[:time]} -- this fires only once"
end

Tina4::Events.emit("app.started", { time: Time.now.utc.iso8601 })
Tina4::Events.emit("app.started", { time: Time.now.utc.iso8601 })
```

Output:

```
App started at 2026-04-02T09:00:00Z -- this fires only once
```

The second emit produces no output. The listener is gone after the first call.

Use `once` for initialization tasks, welcome messages, or any logic that must run exactly one time.

---

6. off: Remove a Listener

To remove a specific listener, keep a reference to the block.

```
logger = Tina4::Events.on("request.received") do |payload|
  puts "Request: #{payload[:method]} #{payload[:path]}"
end

Tina4::Events.emit("request.received", { method: "GET", path: "/api/users" })

# Remove the listener
Tina4::Events.off("request.received", logger)

Tina4::Events.emit("request.received", { method: "POST", path: "/api/users" })
```

Output:

```
Request: GET /api/users
```

The second emit fires no listener -- it was removed.

---

7. clear: Remove All Listeners

Remove all listeners for an event, or all listeners for all events.

```
# Clear listeners for a specific event
Tina4::Events.clear("order.placed")

# Clear all listeners for all events
Tina4::Events.clear
```

Use `clear` in tests to avoid listener bleed between test cases.

```
RSpec.describe "Order processing" do
  after(:each) { Tina4::Events.clear }

  it "sends confirmation email" do
    received = []
    Tina4::Events.on("order.placed") { |p| received << p[:email] }
    Tina4::Events.emit("order.placed", { email: "alice@example.com" })
    expect(received).to eq(["alice@example.com"])
  end
end
```

---

8. Events in Route Handlers

Wire events into your HTTP layer to keep handlers thin.

```
# src/routes/orders.rb

# @noauth
Tina4::Router.post("/api/orders") do |request, response|
  body = request.body

  order = {
    id: rand(1000..9999),
    email: body["email"],
    items: body["items"],
    total: body["total"]
  }

  Tina4::Events.emit("order.placed", order)

  response.json({ message: "Order received", order_id: order[:id] }, 201)
end
```

```
# src/listeners/order_listeners.rb

Tina4::Events.on("order.placed") do |order|
  puts "Email: Confirmation sent to #{order[:email]}"
end

Tina4::Events.on("order.placed") do |order|
  puts "Inventory: #{order[:items].length} item type(s) reserved"
end

Tina4::Events.on("order.placed", priority: 10) do |order|
  puts "Audit: Order #{order[:id]} logged"
end
```

```
curl -X POST http://localhost:7147/api/orders \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@example.com","items":[{"sku":"KB-100","qty":1}],"total":79.99}'
```

```
{ "message": "Order received", "order_id": 5823 }
```

Server output:

```
Audit: Order 5823 logged
Email: Confirmation sent to alice@example.com
Inventory: 1 item type(s) reserved
```

---

9. Listing Registered Events

Inspect what listeners are registered.

```
Tina4::Events.listeners("order.placed").each do |listener|
  puts "Priority #{listener[:priority]}"
end
```

---

10. Gotchas

1. Listeners run synchronously

Events are in-process and synchronous. If a listener takes 3 seconds, `emit` blocks for 3 seconds. For slow work, push to a queue inside the listener instead of doing the work directly.

```
Tina4::Events.on("order.placed") do |order|
  queue = Tina4::Queue.new(topic: "emails")
  queue.push({ to: order[:email], subject: "Order Confirmation" })
end
```

2. Exceptions in listeners

If a listener raises an exception, subsequent listeners for the same event do not run. Wrap listener logic in `rescue` when reliability matters.

```
Tina4::Events.on("payment.received") do |payload|
  begin
    process_payment(payload)
  rescue => e
    puts "Payment listener error: #{e.message}"
  end
end
```

3. clear in tests

Always call `Tina4::Events.clear` in `after(:each)` blocks. Listeners registered in one test persist into the next unless explicitly removed.

4. once is not thread-safe by design

`once` is intended for single-threaded initialization sequences. Do not use it in concurrent request handlers -- the listener may fire multiple times before the first call completes removal.

Localization

1. Your App Speaks One Language

Every string in your app is hard-coded English. A French user sees "Order Confirmation". A German user sees "Error: Invalid email". A Japanese user sees dates as "April 2, 2026" instead of "2026■4■2■".

Localization separates translatable strings from code. Add a locale file per language. Switch the active locale per request. Every `t("key")` call returns the right string for the current user.

---

2. Locale Files

Store locale files as JSON in `src/locales/`. Name them by locale code.

`src/locales/en.json`:

```
{
  "greeting": "Hello, %{name}!",
  "order": {
    "confirmation": "Order Confirmation",
    "placed": "Your order #%{number} has been placed.",
    "total": "Total: %{currency}%{amount}"
  },
  "errors": {
    "invalid_email": "Invalid email address.",
    "required": "%{field} is required."
  },
  "nav": {
    "home": "Home",
    "account": "My Account",
    "logout": "Log Out"
  }
}
```

`src/locales/fr.json`:

```
{
  "greeting": "Bonjour, %{name}!",
  "order": {
    "confirmation": "Confirmation de commande",
    "placed": "Votre commande #%{number} a été passée.",
    "total": "Total: %{currency}%{amount}"
  },
  "errors": {
    "invalid_email": "Adresse e-mail invalide.",
    "required": "%{field} est requis."
  },
  "nav": {
    "home": "Accueil",
    "account": "Mon compte",
    "logout": "Se déconnecter"
  }
}
```

---

3. Basic Translation

`Tina4::Localization` is a module-level singleton. Load the locale files once at startup, then translate with `t`:

```
Tina4::Localization.load          # scans locales/, translations/, i18n/, src/locales/
Tina4::Localization.set_locale("en")

puts Tina4::Localization.t("greeting", name: "Alice")
# => Hello, Alice!

puts Tina4::Localization.t("order.confirmation")
# => Order Confirmation

puts Tina4::Localization.t("nav.logout")
# => Log Out
```

Dot notation navigates nested keys. `"order.confirmation"` maps to `locales["order"]["confirmation"]`.

---

4. Interpolation

Use `%{placeholder}` in locale values. Pass named keyword arguments to `t`.

```
puts Tina4::Localization.t("order.placed", number: "1042")
# => Your order #1042 has been placed.

puts Tina4::Localization.t("order.total", currency: "$", amount: "79.99")
# => Total: $79.99

puts Tina4::Localization.t("errors.required", field: "Email")
# => Email is required.
```

Missing interpolation variables are left as-is in the output string.

---

5. Switching Locales

Call `set_locale` to change the active locale, or pass `locale:` to a single `t` call.

```
Tina4::Localization.set_locale("en")

puts Tina4::Localization.t("greeting", name: "Alice")
# => Hello, Alice!

Tina4::Localization.set_locale("fr")

puts Tina4::Localization.t("greeting", name: "Alice")
# => Bonjour, Alice!

# Or override for a single call without changing the active locale:
puts Tina4::Localization.t("greeting", locale: "fr", name: "Alice")
```

```
# => Bonjour, Alice!
```

---

6. Fallback Locale

When a key is missing from the active locale, `t` automatically falls back to English (`"en"`) before returning the key itself.

```
Tina4::Localization.set_locale("de")

# "de" locale has no "nav.logout" key
puts Tina4::Localization.t("nav.logout")
# => Log Out  (falls back to English)

# You can also supply an explicit default for a single call:
puts Tina4::Localization.t("nav.missing", default: "Menu")
# => Menu
```

Fallback prevents missing key errors in partially translated locales.

---

7. Per-Request Locale in Routes

Read the user's preferred locale from a header, cookie, or query param and pass it to `t` with `locale:`. Passing `locale:` per call avoids mutating the shared active locale across concurrent requests.

```
# @noauth
Tina4::Router.get("/api/greeting") do |request, response|
  locale = request.headers["Accept-Language"]&.split(",")&.first&.strip || "en"
  locale = locale[0..1]  # normalize "fr-FR" to "fr"

  response.json({
    message: Tina4::Localization.t("greeting", locale: locale, name: "World"),
    locale: locale
  })
end
```

```
curl -H "Accept-Language: fr" http://localhost:7147/api/greeting
```

```
{ "message": "Bonjour, World!", "locale": "fr" }
```

```
curl -H "Accept-Language: de" http://localhost:7147/api/greeting
```

```
{ "message": "Hello, World!", "locale": "en" }
```

German falls back to English because `de.json` does not exist.

---

8. Available Locales

List all locale files present in the locales directory.

```
Tina4::Localization.load
puts Tina4::Localization.available_locales.inspect
```

```
# => ["en", "fr"]
```

Use this to build a language switcher UI.

```
# @noauth
Tina4::Router.get("/api/locales") do |request, response|
  response.json({ locales: Tina4::Localization.available_locales })
end
```

---

9. Locale in Templates

Tina4 auto-wires `t()` as a template global, so templates call it directly. Set the active locale for the request before rendering.

```
Tina4::Router.get("/dashboard") do |request, response|
  locale = request.cookies["locale"] || "en"
  Tina4::Localization.set_locale(locale)

  response.render("dashboard.html", { user: { name: "Alice" } })
end
```

In `src/templates/dashboard.html` (Frond template):

```
<nav>
  <a href="/">{{ t("nav.home") }}</a>
  <a href="/account">{{ t("nav.account") }}</a>
  <a href="/logout">{{ t("nav.logout") }}</a>
</nav>

<h1>{{ t("greeting", name=user.name) }}</h1>
```

---

10. Date and Number Formatting

Add locale-specific formatting helpers to your locale files or handle them with Ruby's standard library.

```
# Format a date string for the current locale
date = Time.now

formatted = case Tina4::Localization.current_locale
            when "en" then date.strftime("%B %-d, %Y")
            when "fr" then date.strftime("%-d %B %Y")
            when "de" then date.strftime("%-d. %B %Y")
            else date.strftime("%Y-%m-%d")
            end

puts formatted
# en: April 2, 2026
# fr: 2 avril 2026
# de: 2. April 2026
```

---

11. Gotchas

1. Missing locale file returns fallback silently

If the requested locale file does not exist and a fallback is set, the fallback runs without warning. Log the missing locale in development.

2. Nested key typo

`Tina4::Localization.t("order.confimation")` (typo) returns the key string `"order.confimation"` unchanged. If you see keys appearing raw in the UI, check for spelling mistakes in your `t()` calls.

3. Reload locale files in development

Locale file changes do not reload automatically in a running server. Restart the server or implement a dev-mode watch pattern.

4. Interpolation keys are case-sensitive

`{Name}` in the locale file is not the same as `name:` in the `t()` call. Keep all interpolation placeholders lowercase.

Structured Logging

1. puts Is Not a Logging Strategy

`puts` dumps raw text to stdout. No timestamps. No severity levels. No structured fields. Searching through 50,000 lines of `puts` output for a production error at 2:47 AM is the pain that structured logging prevents.

Structured logging records every entry as a consistent, parseable event: timestamp, level, message, and any additional fields you provide. Log aggregators (Datadog, Elastic, CloudWatch) can filter, group, and alert on structured logs automatically. Plain text output cannot.

Tina4's logger writes structured entries, supports configurable levels, and is available everywhere in your app.

---

2. Log Levels

Four levels, in order of increasing severity:

Level	Method	When to use
DEBUG	`Tina4::Log.debug`	Detailed trace: request internals, query params, timings
INFO	`Tina4::Log.info`	Normal operations: request received, user logged in
WARNING	`Tina4::Log.warning`	Something unexpected but recoverable: deprecated API called
ERROR	`Tina4::Log.error`	Something broke: use for unrecoverable failures too

---

3. Basic Usage

```
Tina4::Log.debug("Cache miss", key: "products:list")
Tina4::Log.info("User logged in", user_id: 42, email: "alice@example.com")
Tina4::Log.warning("Deprecated endpoint called", path: "/api/v1/products")
Tina4::Log.error("Database query failed", table: "orders", error: "connection timeout")
Tina4::Log.error("Out of memory -- shutting down")
```

Output (text format):

```
[2026-04-02 09:00:01 UTC] DEBUG  Cache miss | key=products:list
[2026-04-02 09:00:02 UTC] INFO   User logged in | user_id=42 email=alice@example.com
[2026-04-02 09:00:03 UTC] WARN   Deprecated endpoint called | path=/api/v1/products
```

```
[2026-04-02 09:00:04 UTC] ERROR  Database query failed | table=orders error=connection timeout
[2026-04-02 09:00:05 UTC] ERROR  Out of memory -- shutting down
```

Each entry has a timestamp, level, message, and any keyword arguments you passed as structured fields.

---

4. Controlling the Log Level

Set the minimum log level via the `TINA4_LOG_LEVEL` environment variable. Entries below this level are silently dropped.

```
TINA4_LOG_LEVEL=info
```

Valid values (case-insensitive): `all`, `debug`, `info`, `warning`, `error`, `none`.

With `TINA4_LOG_LEVEL=info`, `Tina4::Log.debug(...)` calls produce no output. With `TINA4_LOG_LEVEL=error`, only `error` entries appear. `none` silences everything.

Default is `info` (set this before the process starts).

---

5. Reconfiguring the Logger

The log level is read from `TINA4_LOG_LEVEL` when the logger first initializes. To pick up a changed value (for example after setting the env var in code), call `configure` to re-read the environment:

```
ENV["TINA4_LOG_LEVEL"] = "debug"
Tina4::Log.configure
```

`configure` re-reads all `TINA4_LOG_*` environment variables (level, directory, file, format, output, rotation). Setting the env var before the process starts is the usual approach.

---

6. Logging in Route Handlers

```
Tina4::Router.get("/api/products/:id") do |request, response|
  id = request.params["id"].to_i

  Tina4::Log.debug("Product lookup", product_id: id)

  product = Product.find(id)

  if product.nil?
    Tina4::Log.warning("Product not found", product_id: id)
    next response.json({ error: "Product not found" }, 404)
  end

  Tina4::Log.info("Product retrieved", product_id: id, name: product.name)
  response.json({ product: product.to_h })
end
```

---

7. Logging Errors with Exceptions

Pass the exception message as a structured field.

```
Tina4::Router.post("/api/orders") do |request, response|
  begin
    body = request.body
    order = create_order(body)

    Tina4::Log.info("Order created", order_id: order[:id], total: order[:total])
    response.json({ order_id: order[:id] }, 201)

  rescue ArgumentError => e
    Tina4::Log.warning("Invalid order payload", error: e.message)
    response.json({ error: e.message }, 400)

  rescue => e
    Tina4::Log.error("Order creation failed",
      error: e.message,
      backtrace: e.backtrace.first(3).join(" | ")
    )
    response.json({ error: "Internal server error" }, 500)
  end
end
```

---

8. Request Logging Middleware

Add the `RequestLogger` middleware to log every inbound request automatically.

```
Tina4::Router.get("/api/users", middleware: ["RequestLogger"]) do |request, response|
  response.json({ users: [] })
end
```

The middleware logs:

```
[2026-04-02 09:01:00 UTC] INFO  Request | method=GET path=/api/users ip=127.0.0.1
[2026-04-02 09:01:00 UTC] INFO  Response | method=GET path=/api/users status=200 duration_ms=4
```

Apply it globally to log every route by registering the middleware class with `Router.use`:

```
# config/app.rb
Tina4::Router.use(Tina4::RequestLoggerMiddleware)
```

---

9. JSON Output Format

For log aggregators, switch to JSON output:

```
TINA4_LOG_FORMAT=json
```

The same calls now produce:

```
{"timestamp":"2026-04-02T09:00:02Z","level":"INFO","message":"User logged in","user_id":42,"emai
{"timestamp":"2026-04-02T09:00:04Z","level":"ERROR","message":"Database query failed","table":"o
```

One JSON object per line. Compatible with Datadog, Elastic, CloudWatch, and any log drain that understands NDJSON.

---

10. Writing to a File

```
TINA4_LOG_FILE=logs/app.log
```

The logger writes to the file and to stdout simultaneously. Log rotation is handled by the OS log rotation tool (logrotate on Linux, newsyslog on macOS).

---

11. Checking the Current Level

```
if Tina4::Log.debug?
  # Build expensive debug payload only if debug logging is active
  Tina4::Log.debug("Query plan", plan: db.explain(query).inspect)
end
```

Available predicates: `debug?`, `info?`, `warning?`, `error?`.

---

12. Gotchas

1. Never log passwords or tokens

```
# Wrong
Tina4::Log.info("Login attempt", password: params["password"])

# Right
Tina4::Log.info("Login attempt", email: params["email"])
```

2. Avoid string interpolation in the message

```
# Slower -- builds the string even if the level is filtered
Tina4::Log.debug("User #{user.id} loaded #{records.count} records")

# Faster -- fields are only serialized if the level passes
Tina4::Log.debug("Records loaded", user_id: user.id, count: records.count)
```

3. DEBUG in production floods logs

Set `TINA4_LOG_LEVEL=info` in production. Debug output at scale produces millions of lines per hour and obscures real errors.

4. Logging an error does not exit

`Tina4::Log.error(...)` logs at the error level but does not call `exit`. Call `abort` or `Process.exit(1)` explicitly after logging an unrecoverable condition.

Email with Messenger

1. Every App Sends Email

Signup confirmations. Password resets. Weekly digests. Invoices with PDF attachments. Every application needs email. Nobody enjoys building it.

SMTP configuration. Plain text fallbacks. Attachment encoding. Connection timeouts. Bounce handling. The details compound. Tina4's `Messenger` class owns all of it. Configure through `.env`. Create an instance. Send. Without an SMTP host, Messenger captures mail in the dev mailbox. An SMTP host enables real delivery, so staging needs an explicit safety setting.

---

2. Messenger Configuration via .env

All email configuration lives in `.env`:

```
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@example.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
TINA4_MAIL_FROM=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

Variable	Description	Common Values
`TINA4_MAIL_HOST`	SMTP server hostname	`smtp.gmail.com`, `smtp.mailgun.org`, `smtp.sendgrid.net`
`TINA4_MAIL_PORT`	SMTP port	`587` (TLS), `465` (SSL), `25` (unencrypted)
`TINA4_MAIL_USERNAME`	Login username	Usually your email address
`TINA4_MAIL_PASSWORD`	Login password or app-specific password	App passwords for Gmail
`TINA4_MAIL_ENCRYPTION`	Encryption method	`tls` (recommended), `ssl`, `none`
`TINA4_MAIL_FROM`	Default "From" address	`noreply@yourdomain.com`
`TINA4_MAIL_FROM_NAME`	Default "From" display name	`My Store`, `Acme Corp`
`TINA4_MAIL_CAPTURE`	Disable SMTP and capture each message locally	`true` on staging when no external delivery is allowed
`TINA4_MAIL_REDIRECT_TO`	Replace every SMTP recipient with a safety list	`qa@example.com,product@example.com`

Messenger also accepts legacy `SMTP_*` prefixed variables as fallback. The `TINA4_MAIL_*` prefix takes priority.

Staging delivery safety

`TINA4_DEBUG` controls developer tools. It does not disable SMTP delivery. Pick one explicit mail policy for staging:

```
# No message leaves the application. Inspect it in the dev mailbox.
TINA4_MAIL_CAPTURE=true
```

Capture wins even when an SMTP host exists. A missing SMTP host also captures mail. `TINA4_MAILBOX_DIR` only selects the dev mailbox directory; it does not turn capture on or off.

Use real SMTP without risking customer delivery by replacing all recipients:

```
TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com
```

Tina4 trims each comma-separated address and drops blank entries. On the SMTP path, the safety list replaces `To`; Tina4 clears `Cc` and `Bcc` and stores all original recipients in `X-Tina4-Original-To`. An unset or empty list leaves delivery unchanged. If capture and redirect are both set, capture wins and nothing reaches SMTP.

Common Provider Configurations

Gmail:

```
TINA4_MAIL_HOST=smtp.gmail.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=your-email@gmail.com
TINA4_MAIL_PASSWORD=your-app-password
TINA4_MAIL_ENCRYPTION=tls
```

Gmail requires an "App Password" (not your regular password) when two-factor authentication is enabled.

Mailgun:

```
TINA4_MAIL_HOST=smtp.mailgun.org
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=postmaster@mg.yourdomain.com
TINA4_MAIL_PASSWORD=your-mailgun-smtp-password
TINA4_MAIL_ENCRYPTION=tls
```

SendGrid:

```
TINA4_MAIL_HOST=smtp.sendgrid.net
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=apikey
TINA4_MAIL_PASSWORD=your-sendgrid-api-key
TINA4_MAIL_ENCRYPTION=tls
```

---

3. Constructor Override Pattern

Different emails need different SMTP accounts. Transactional emails from one server. Marketing from another. Override the configuration in the constructor:

```
# Uses .env defaults
mailer = Tina4::Messenger.new

# Override specific settings
marketing_mailer = Tina4::Messenger.new(
  host: "smtp.mailgun.org",
  port: 587,
  username: "marketing@mg.yourdomain.com",
  password: "marketing-smtp-password",
  encryption: "tls",
  from_address: "newsletter@yourdomain.com",
  from_name: "My Store Newsletter"
)
```

Constructor arguments take priority over `.env` values. Any argument you omit falls back to the environment variable.

---

4. Sending Plain Text Email

The simplest email:

```
Tina4::Router.post("/api/contact") do |request, response|
  body = request.body

  mail = Tina4::Messenger.new
  result = mail.send(
    to: body["email"],
    subject: "Contact Form Submission",
    body: "Name: #{body['name']}\nEmail: #{body['email']}\nMessage:\n#{body['message']}"
  )

  if result[:success]
    response.json({ message: "Email sent successfully" })
  else
    response.json({ error: "Failed to send email", details: result[:message] }, 500)
  end
end
```

```
curl -X POST http://localhost:7147/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "message": "Hello!"}'
```

```
{"message":"Email sent successfully"}
```

The `send` method returns a hash with three keys: `success` (boolean), `message` (string), and `id` (message ID string or nil).

The send Method Signature

```
mail.send(
  to:,              # Recipient(s) -- string or array of strings
  subject:,         # Email subject line
  body:,            # Email body (plain text or HTML)
  html: false,      # If true, body is treated as HTML
  text: nil,        # Plain text alternative (when body is HTML)
  cc: [],           # CC recipient(s) -- string or array
  bcc: [],          # BCC recipient(s) -- string or array
  reply_to: nil,    # Reply-To address
  attachments: [],  # List of file paths or hashes
  headers: {}       # Additional email headers (hash)
)
```

---

5. Sending HTML Email with Text Fallback

Most emails should carry HTML with a plain text fallback. Email clients that cannot render HTML display the text version instead:

```
mail = Tina4::Messenger.new

html_body = <<~HTML
  <html>
  <body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
      <div style="background: #1a1a2e; color: white; padding: 20px; text-align: center;">
          <h1 style="margin: 0;">Welcome to My Store!</h1>
      </div>
      <div style="padding: 20px;">
          <p>Hi Alice,</p>
          <p>Thank you for creating your account. We are excited to have you!</p>
          <p>Here is what you can do next:</p>
          <ul>
              <li>Browse our <a href="https://mystore.com/products">product catalog</a></li>
              <li>Set up your <a href="https://mystore.com/profile">profile</a></li>
              <li>Check out our <a href="https://mystore.com/deals">current deals</a></li>
          </ul>
          <p>Cheers,<br>The My Store Team</p>
      </div>
      <div style="background: #f5f5f5; padding: 12px; text-align: center; font-size: 12px; color
          <p>You received this because you signed up at mystore.com</p>
      </div>
  </body>
  </html>
HTML

text_body = <<~TEXT
  Hi Alice,

  Thank you for creating your account. We are excited to have you!

  Here is what you can do next:
  - Browse our product catalog: https://mystore.com/products
  - Set up your profile: https://mystore.com/profile
  - Check out our current deals: https://mystore.com/deals

  Cheers,
```

```
  The My Store Team
TEXT

result = mail.send(
  to: "alice@example.com",
  subject: "Welcome to My Store!",
  body: html_body,
  html: true,
  text: text_body
)
```

Pass `html: true` to tell Messenger the body contains HTML. The `text` parameter provides the plain text alternative. Messenger builds a `multipart/alternative` message that carries both versions.

---

6. Adding Attachments

Attach files by providing their paths:

```
mail = Tina4::Messenger.new

result = mail.send(
  to: "accounting@example.com",
  subject: "Monthly Invoice #1042",
  body: "<h2>Invoice #1042</h2><p>Please find the invoice attached.</p>",
  html: true,
  attachments: [
    "/path/to/invoices/invoice-1042.pdf",
    "/path/to/reports/monthly-summary.csv"
  ]
)
```

Messenger reads each file, determines its MIME type, and encodes it for email transmission.

Multiple Attachments

```
mail = Tina4::Messenger.new
result = mail.send(
  to: "alice@example.com",
  subject: "Monthly Report",
  body: "Here are this month's reports.",
  attachments: [
    "/reports/sales.pdf",
    "/reports/analytics.xlsx",
    "/reports/summary.csv"
  ]
)
```

---

7. CC, BCC, and Reply-To

```
mail = Tina4::Messenger.new

result = mail.send(
  to: "alice@example.com",
  subject: "Team Meeting Notes",
  body: "<p>Here are the notes from today's meeting.</p>",
  html: true,
  cc: ["bob@example.com", "charlie@example.com"],
  bcc: ["manager@example.com"],
  reply_to: "alice@example.com"
)
```

- **cc**: List of email addresses to carbon copy. All recipients see CC addresses.
- **bcc**: List of email addresses to blind carbon copy. Recipients cannot see BCC addresses.
- **reply_to**: When the recipient clicks "Reply", this address fills the "To" field instead of the "From" address.

Both `cc` and `bcc` accept a single string or an array of strings.

---

8. Custom Headers

Messenger supports custom headers through the `headers` parameter on `send`:

```
result = mail.send(
  to: "customer@example.com",
  subject: "Your Support Ticket #123",
  body: "We are looking into your issue.",
  reply_to: "support@mystore.com",
  headers: {
    "X-Ticket-Id" => "123",
    "X-Priority" => "1",
    "X-Mailer" => "Tina4 Messenger"
  }
)
```

Custom headers serve several purposes. Tracking headers like `X-Ticket-Id` let you correlate emails with support tickets. Priority headers influence some email clients' display. Bulk-sending headers like `Precedence: bulk` help mail servers classify newsletters.

---

9. Reading Inbox via IMAP

Messenger reads email through IMAP. Configure the IMAP server in `.env`:

```
TINA4_MAIL_IMAP_HOST=imap.example.com
TINA4_MAIL_IMAP_PORT=993
```

Messenger reuses `TINA4_MAIL_USERNAME` and `TINA4_MAIL_PASSWORD` for IMAP authentication. You can also override the IMAP host and port in the constructor:

```
mailer = Tina4::Messenger.new(imap_host: "imap.gmail.com", imap_port: 993)
```

Listing Inbox Messages

The `inbox` method fetches message headers from the mailbox:

```
Tina4::Router.get("/api/inbox") do |request, response|
  mailer = Tina4::Messenger.new

  emails = mailer.inbox(limit: 20, offset: 0)

  messages = emails.map do |email|
    {
      uid: email[:uid],
      from: email[:from],
      subject: email[:subject],
      date: email[:date],
      snippet: email[:snippet],
      seen: email[:seen]
    }
  end

  response.json({ messages: messages, count: messages.length })
end
```

```
curl http://localhost:7147/api/inbox
```

```
{
  "messages": [
    {
      "uid": "12345",
      "from": "customer@example.com",
      "subject": "Order question",
      "date": "2026-03-22T10:30:00+00:00",
      "snippet": "Hi, I have a question about my recent order...",
      "seen": false
    }
  ],
  "count": 1
}
```

The `inbox` method returns messages newest-first. Each message contains `uid`, `subject`, `from`, `to`, `date`, `snippet` (first 150 characters of the body), and `seen` (boolean).

Reading a Specific Message

```
Tina4::Router.get("/api/inbox/{uid}") do |request, response|
  mailer = Tina4::Messenger.new
  uid = request.params["uid"]

  email = mailer.read(uid, mark_read: true)

  if email.nil?
    return response.json({ error: "Email not found" }, 404)
  end

  response.json({
    uid: email[:uid],
    from: email[:from],
    to: email[:to],
    cc: email[:cc],
    subject: email[:subject],
    date: email[:date],
    body_html: email[:body_html],
    body_text: email[:body_text],
    attachments: (email[:attachments] || []).map { |a|
      { filename: a[:filename], size: a[:size], content_type: a[:content_type] }
    }
  })
end
```

The `read` method fetches the full message including body and attachments. Pass `mark_read: false` to leave the message unread.

Searching Messages

```
Tina4::Router.get("/api/inbox/search") do |request, response|
  mailer = Tina4::Messenger.new

  results = mailer.search(
    subject: request.params["q"],
    sender: request.params["from"],
    unseen_only: request.params["unread"] == "true",
    limit: 20
  )

  response.json({ messages: results, count: results.length })
end
```

The `search` method accepts `subject`, `sender`, `since` (date string), `before`, and `unseen_only` as filters. All filters combine with AND logic.

Other IMAP Operations

```
mailer = Tina4::Messenger.new

# Count unread messages
count = mailer.unread

# List all mailbox folders
folders = mailer.folders
# ["INBOX", "Sent", "Drafts", "Trash", "Spam"]
```

---

10. Dev Mailbox Capture

Messenger captures outgoing mail when no SMTP host exists or when `TINA4_MAIL_CAPTURE=true`. `TINA4_DEBUG` only makes the dev dashboard available; it does not change delivery.

Navigate to `/__dev` and click "Emails" to see:

- Recipient, subject, and timestamp
- Full HTML preview
- Plain text fallback
- Attachments list
- Headers

This lets you test email without configuring SMTP. Enable `TINA4_DEBUG=true` to browse captured messages in the dev dashboard.

Sending from a development environment

Configure an SMTP host and leave `TINA4_MAIL_CAPTURE` unset or false. The `TINA4_DEBUG` value does not matter to delivery. To test SMTP without reaching customers, set `TINA4_MAIL_REDIRECT_TO` to controlled inboxes.

---

11. Using Templates for Email Content

Hardcoded HTML in Ruby strings is ugly and hard to maintain. Templates fix this.

Create `src/templates/emails/welcome.html`:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
</head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; background: #f5f5
    <div style="background: #1a1a2e; color: white; padding: 24px; text-align: center; border-rad
        <h1 style="margin: 0; font-size: 24px;">Welcome, {{ name }}!</h1>
    </div>
    <div style="background: white; padding: 24px; border-radius: 0 0 8px 8px;">
        <p>Hi {{ name }},</p>
        <p>Your account has been created. Here are your details:</p>

        <table style="width: 100%; border-collapse: collapse; margin: 16px 0;">
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Emai
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ email }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Acco
                <td style="padding: 8px; border-bottom: 1px solid #eee;">#{{ user_id }}</td>
```

```
            </tr>
        </table>

        <p>Get started:</p>
        <ul>
            <li><a href="{{ base_url }}/products" style="color: #1a1a2e;">Our product catalog</a
            <li><a href="{{ base_url }}/profile" style="color: #1a1a2e;">Your profile settings</
        </ul>

        {% if promo_code %}
            <div style="background: #d4edda; padding: 16px; border-radius: 4px; margin: 16px 0;"
                <strong>Special offer!</strong> Use code <code>{{ promo_code }}</code> for 10% o
            </div>
        {% endif %}

        <p>Cheers,<br>The {{ app_name }} Team</p>
    </div>

    <div style="text-align: center; padding: 12px; color: #888; font-size: 12px;">
        <p>You received this because you signed up at {{ app_name }}.</p>
    </div>
</body>
</html>
```

Rendering and Sending

Use `html_template` and `template_data` on the Messenger instance:

```
Tina4::Router.post("/api/register") do |request, response|
  body = request.body

  # Create user (database logic)
  user_id = 42

  mail = Tina4::Messenger.new
  mail.to = body["email"]
  mail.subject = "Welcome to My Store, #{body['name']}!"
  mail.html_template = "emails/welcome.html"
  mail.template_data = {
    name: body["name"],
    email: body["email"],
    user_id: user_id,
    base_url: ENV["APP_URL"] || "http://localhost:7147",
    app_name: "My Store",
    promo_code: "WELCOME10"
  }
  mail.send

  response.json({ message: "Registration successful", user_id: user_id }, 201)
end
```

```
curl -X POST http://localhost:7147/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user_id": 42
}
```

With `TINA4_MAIL_CAPTURE=true`, the email stays in the dev mailbox instead of reaching a real inbox. Enable `TINA4_DEBUG=true` to inspect the rendered HTML and verify the template values in the dashboard.

---

12. Sending Email via Queues

In production, never send email inside a route handler. The SMTP call blocks the response. Use the queue system:

```
# In the route handler, queue the email
Tina4::Router.post("/api/register") do |request, response|
  body = request.body
  user_id = 42  # Simulated

  Tina4::Queue.produce("send-email", {
    to: body["email"],
    subject: "Welcome to My Store, #{body['name']}!",
    template: "emails/welcome.html",
    data: {
      name: body["name"],
      email: body["email"],
      user_id: user_id,
      base_url: "http://localhost:7147",
      app_name: "My Store",
      promo_code: "WELCOME10"
    }
  })

  response.json({ message: "Registration successful", user_id: user_id }, 201)
end

# The consumer sends the actual email
Tina4::Queue.consume("send-email") do |job|
  mail = Tina4::Messenger.new
  mail.to = job.payload["to"]
  mail.subject = job.payload["subject"]

  if job.payload["template"]
    mail.html_template = job.payload["template"]
    mail.template_data = job.payload["data"] || {}
  else
    mail.body = job.payload["body"] || ""
  end

  mail.send
  true
end
```

The route handler returns in under 50 milliseconds. The queue worker sends the email on its own timeline. If the SMTP server is down, retries happen automatically.

---

13. Exercise: Build an Email Notification System

Build an email system that sends different types of notifications.

Requirements

- `POST /api/notify/welcome` -- Send a welcome email with an HTML template
- `POST /api/notify/order` -- Send an order confirmation with order details
- `POST /api/notify/reset` -- Send a password reset email with a token link

Each endpoint should queue the email rather than sending it directly.

Test with:

```
curl -X POST http://localhost:7147/api/notify/welcome \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com"}'

curl -X POST http://localhost:7147/api/notify/order \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "order_id": 101, "total": 159.98}'

curl -X POST http://localhost:7147/api/notify/reset \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "reset_token": "abc123def456"}'
```

---

14. Solution

Create `src/routes/notifications.rb`:

```
# @noauth
Tina4::Router.post("/api/notify/welcome") do |request, response|
  body = request.body

  Tina4::Queue.produce("send-email", {
    to: body["email"],
    subject: "Welcome to My Store, #{body['name']}!",
    template: "emails/welcome.html",
    data: { name: body["name"], login_url: "https://mystore.com/login", year: Time.now.year }
  })

  response.json({ message: "Welcome email queued" })
end

# @noauth
Tina4::Router.post("/api/notify/order") do |request, response|
  body = request.body

  Tina4::Queue.produce("send-email", {
    to: body["email"],
    subject: "Order Confirmation ##{body['order_id']}",
    template: "emails/order-confirmation.html",
    data: { order_id: body["order_id"], total: body["total"] }
  })
```

```
  response.json({ message: "Order confirmation email queued" })
end

# @noauth
Tina4::Router.post("/api/notify/reset") do |request, response|
  body = request.body

  Tina4::Queue.produce("send-email", {
    to: body["email"],
    subject: "Password Reset Request",
    template: "emails/password-reset.html",
    data: { reset_url: "https://mystore.com/reset?token=#{body['reset_token']}" }
  })

  response.json({ message: "Password reset email queued" })
end
```

---

15. Gotchas

1. Gmail Requires App Passwords

Problem: Gmail login fails with "authentication error".

Fix: Enable 2FA on your Google account, then generate an App Password at https://myaccount.google.com/apppasswords. Use the app password, not your regular password.

2. Emails Go to Spam

Problem: Emails arrive in the spam folder.

Fix: Set up SPF, DKIM, and DMARC DNS records for your domain. Use a reputable email service like Mailgun, SendGrid, or Postmark.

3. HTML Email Rendering Differences

Problem: Your email looks different in Gmail, Outlook, and Apple Mail.

Fix: Use inline CSS. Avoid flexbox and grid. Use tables for layout. Test with a tool like Litmus or Email on Acid.

4. Attachment File Not Found

Problem: `attachments: ["/path/to/file.pdf"]` raises a file not found error.

Fix: Use absolute paths. Relative paths resolve from the working directory, which may differ in production.

5. SMTP Connection Timeout

Problem: Sending email hangs for 30 seconds and then times out.

Fix: Check your SMTP host, port, and encryption settings. Common mistake: using port 587 with SSL instead of TLS.

6. Dev Mode Emails Disappear on Restart

Problem: Intercepted emails in the dev dashboard vanish when you restart the server.

Fix: This is expected. Dev mode stores emails in memory. For persistent storage, configure a real SMTP server.

7. Unicode Characters Display as Question Marks

Problem: Non-ASCII characters (accents, CJK) show as `?` in the email.

Fix: Tina4 sets UTF-8 encoding by default. If you are constructing raw headers, make sure you include `Content-Type: text/html; charset=utf-8`.

8. IMAP Connection Fails

Problem: `inbox` or `read` raises a connection error.

Fix: Verify `TINA4_MAIL_IMAP_HOST` and `TINA4_MAIL_IMAP_PORT` in `.env`. Gmail uses `imap.gmail.com` on port `993`. Make sure your email provider allows IMAP access -- some providers disable it by default.

Frontend with tina4css

1. The Problem with Frontend Toolchains

Your client wants a dashboard. Install Node.js. Run `npm install`. Wait for 200MB of `node_modules`. Configure webpack or Vite. Set up PostCSS. Add a CSS framework. Maybe Tailwind with its purge config. Pray nothing breaks when you upgrade a dependency in 6 months.

Tina4 skips all of that. The framework ships with **tina4css** -- a Bootstrap-compatible CSS framework -- and **frond.js** -- a lightweight JavaScript helper library. Both land in your project when you scaffold. No npm. No webpack. No build step. Link the files. Start building.

By the end of this chapter, you will have a complete admin dashboard with a sidebar, navigation, cards, tables, modals, and dark mode support. Zero npm involvement.

---

2. What Ships with Tina4

When you run `tina4 init`, two files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ frond.js         # AJAX helpers, form submission, token management
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/frond.js"></script>
```

---

3. The Grid System

tina4css uses a 12-column responsive grid, compatible with Bootstrap's class names:

```
<div class="container">
    <div class="row">
        <div class="col-md-4"><p>One third</p></div>
        <div class="col-md-4"><p>One third</p></div>
        <div class="col-md-4"><p>One third</p></div>
    </div>
</div>
```

Responsive Breakpoints

Prefix	Min Width	Typical Device
`col-`	0px	All
`col-sm-`	576px	Phones (landscape)
`col-md-`	768px	Tablets
`col-lg-`	992px	Laptops
`col-xl-`	1200px	Desktops

---

4. Cards

```
<div class="card">
    <div class="card-header">Product Details</div>
    <div class="card-body">
        <h3 class="card-title">Wireless Keyboard</h3>
        <p class="card-text">Ergonomic keyboard with backlit keys.</p>
        <p class="price">$79.99</p>
    </div>
    <div class="card-footer">
        <button class="btn btn-primary">Add to Cart</button>
    </div>
</div>
```

Card Grid

```
<div class="row">
    {% for product in products %}
        <div class="col-md-4">
            <div class="card">
                <div class="card-body">
                    <h3 class="card-title">{{ product.name }}</h3>
                    <p class="card-text">${{ product.price | number_format(2) }}</p>
                    <button class="btn btn-primary btn-sm">View</button>
                </div>
            </div>
        </div>
    {% endfor %}
</div>
```

---

5. Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Price</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        {% for product in products %}
            <tr>
                <td>{{ product.id }}</td>
                <td>{{ product.name }}</td>
                <td>${{ product.price | number_format(2) }}</td>
                <td>
                    <button class="btn btn-sm btn-primary">Edit</button>
                    <button class="btn btn-sm btn-danger">Delete</button>
                </td>
            </tr>
        {% endfor %}
    </tbody>
</table>
```

---

6. Forms

```
<form method="POST" action="/api/products">
    <div class="form-group">
        <label for="name">Product Name</label>
        <input type="text" class="form-control" id="name" name="name" required>
    </div>
    <div class="form-group">
        <label for="price">Price</label>
        <input type="number" class="form-control" id="price" name="price" step="0.01" required>
    </div>
    <div class="form-group">
        <label for="category">Category</label>
        <select class="form-control" id="category" name="category">
            <option value="Electronics">Electronics</option>
            <option value="Fitness">Fitness</option>
            <option value="Kitchen">Kitchen</option>
            <option value="Office">Office</option>
        </select>
    </div>
    <button type="submit" class="btn btn-primary">Create Product</button>
</form>
```

---

7. Buttons and Alerts

Buttons

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-info">Info</button>
<button class="btn btn-outline-primary">Outline</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

Alerts

```
<div class="alert alert-success">Product created successfully!</div>
<div class="alert alert-danger">Error: Name is required.</div>
<div class="alert alert-warning">Stock is running low.</div>
<div class="alert alert-info">New features are available.</div>
```

---

8. frond.js -- AJAX Without jQuery

`frond.js` provides lightweight AJAX helpers for form submission, API calls, and token management.

API Calls

```
// GET request
frond.get("/api/products", function (data) {
    console.log(data.products);
});

// POST request
frond.post("/api/products", {
    name: "Widget",
    price: 9.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Widget",
    price: 12.99
}, function (data) {
    console.log("Updated:", data);
});

// DELETE request
frond.del("/api/products/1", function (data) {
    console.log("Deleted");
});
```

Token Management

```
// Store token after login
frond.post("/api/login", { email: "alice@example.com", password: "pass123" }, function (data) {
    frond.setToken(data.token);
});

// All subsequent requests automatically include the Authorization header
frond.get("/api/profile", function (data) {
    console.log(data);  // Token is sent automatically
});
```

Form Submission via AJAX

```
<form id="product-form" data-frond-submit="/api/products" data-frond-method="POST">
    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" class="form-control" name="name" id="name">
    </div>
    <button type="submit" class="btn btn-primary">Create</button>
</form>

<div id="result"></div>

<script>
    frond.onSubmit("#product-form", function (data) {
        document.getElementById("result").innerHTML = '<div class="alert alert-success">Product
    });
</script>
```

---

9. Dark Mode

tina4css supports dark mode via a CSS class on the `<html>` or `<body>` element:

```
<html data-theme="dark">
```

Toggle with JavaScript:

```
function toggleDarkMode() {
    const html = document.documentElement;
    const current = html.getAttribute("data-theme");
    html.setAttribute("data-theme", current === "dark" ? "light" : "dark");
    localStorage.setItem("theme", html.getAttribute("data-theme"));
}

// Load saved theme
const savedTheme = localStorage.getItem("theme") || "light";
document.documentElement.setAttribute("data-theme", savedTheme);
```

---

10. Building a Dashboard

Here is a complete admin dashboard template:

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}Dashboard{% endblock %}

{% block content %}
    <h1>Dashboard</h1>

    <div class="row">
        <div class="col-md-3">
            <div class="card">
                <div class="card-body text-center">
                    <h2>{{ stats.total_products }}</h2>
                    <p class="text-muted">Products</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body text-center">
                    <h2>{{ stats.total_orders }}</h2>
                    <p class="text-muted">Orders</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body text-center">
                    <h2>${{ stats.revenue | number_format(2) }}</h2>
                    <p class="text-muted">Revenue</p>
                </div>
            </div>
        </div>
        <div class="col-md-3">
            <div class="card">
                <div class="card-body text-center">
                    <h2>{{ stats.total_users }}</h2>
                    <p class="text-muted">Users</p>
                </div>
            </div>
        </div>
    </div>

    <div class="row mt-4">
        <div class="col-md-8">
            <div class="card">
                <div class="card-header">Recent Orders</div>
                <div class="card-body">
                    <table class="table table-striped">
                        <thead>
                            <tr><th>Order</th><th>Customer</th><th>Total</th><th>Status</th></tr
                        </thead>
                        <tbody>
                            {% for order in recent_orders %}
                                <tr>
                                    <td>#{{ order.id }}</td>
                                    <td>{{ order.customer }}</td>
                                    <td>${{ order.total | number_format(2) }}</td>
                                    <td><span class="badge badge-{{ order.status_class }}">{{ or
                                </tr>
```

```
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
        <div class="col-md-4">
            <div class="card">
                <div class="card-header">Quick Actions</div>
                <div class="card-body">
                    <a href="/admin/products/new" class="btn btn-primary btn-block mb-2">Add Pro
                    <a href="/admin/orders" class="btn btn-secondary btn-block mb-2">View Orders
                    <a href="/admin/users" class="btn btn-secondary btn-block">Manage Users</a>
                </div>
            </div>
        </div>
    </div>
{% endblock %}
```

Route handler:

```
Tina4::Router.get("/admin") do |request, response|
  response.render("dashboard.html", {
    stats: {
      total_products: 156,
      total_orders: 1423,
      revenue: 45678.90,
      total_users: 342
    },
    recent_orders: [
      { id: 1001, customer: "Alice Smith", total: 159.98, status: "Shipped", status_class: "succ
      { id: 1000, customer: "Bob Jones", total: 49.99, status: "Processing", status_class: "warn
      { id: 999, customer: "Charlie Brown", total: 299.99, status: "Delivered", status_class: "i
    ]
  })
end
```

---

11. SCSS Customization

The default tina4css works out of the box. To customize colors, fonts, or spacing, edit the SCSS source.

Edit `src/public/scss/tina4.scss`:

```
// Override variables before importing the framework
$primary: #2d6a4f;
$secondary: #52b788;
$dark: #1b4332;
$font-family-base: 'Inter', sans-serif;
$border-radius: 8px;

// Import the framework
@import 'tina4-base';
```

Compile SCSS to CSS:

```
tina4 scss
```

```
Compiling SCSS...
  src/public/scss/tina4.scss -> src/public/css/tina4.css
Done (0.12s)
```

The compiled CSS replaces the default `tina4.css`. Your custom colors and fonts take effect across the entire application.

Live SCSS Compilation

During development, run SCSS compilation in watch mode:

```
tina4 scss --watch
```

Every save to a `.scss` file triggers a recompile. Combined with Tina4's live reload, changes appear in the browser within a second.

---

12. frond.js API Reference

Tina4 ships three JavaScript files. Each serves a different purpose. Use them independently or together.

Including the Scripts

```
<script src="/js/tina4.min.js"></script>
<script src="/js/frond.min.js"></script>
<script src="/js/tina4js.min.js"></script>
```

All three live in `src/public/js/` and are served from `/js/`. Include only what you need.

tina4.min.js -- Core Utilities

Low-level helpers for AJAX page loading and form submission.

`loadPage(url, targetId)`

Fetches HTML from `url` and injects it into the element with the given `id`.

```
<nav>
    <a href="#" onclick="loadPage('/dashboard', 'content')">Dashboard</a>
    <a href="#" onclick="loadPage('/settings', 'content')">Settings</a>
</nav>
<div id="content"><!-- pages load here --></div>

<script src="/js/tina4.min.js"></script>
```

`saveForm(formId, url, method)`

Serializes a form and submits it via AJAX. Prevents the default page reload.

```
<form id="product-form">
    <input type="text" name="name" placeholder="Product name">
    <input type="number" name="price" placeholder="Price">
    <button type="button" onclick="saveForm('product-form', '/api/products', 'POST')">
        Save
    </button>
</form>
```

`sendRequest(url, method, data, callback)`

Generic AJAX helper for any HTTP method.

```
sendRequest("/api/products", "GET", null, function (response) {
    console.log("Products:", JSON.parse(response));
});

sendRequest("/api/products", "POST", { name: "Widget", price: 9.99 }, function (response) {
    console.log("Created:", JSON.parse(response));
});
```

frond.min.js -- Template Engine Client-Side Helpers

A companion to the Frond template engine. Handles AJAX form interception, WebSocket connections with auto-reconnect, JWT token management, and dynamic template loading.

AJAX Requests

```
// GET request
frond.get("/api/products", function (data) {
    console.log("Products:", data);
});

// POST request
frond.post("/api/products", {
    name: "New Product",
    price: 29.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", { name: "Updated Product" }, function (data) {
    console.log("Updated:", data);
});

// DELETE request
frond.delete("/api/products/1", function (data) {
    console.log("Deleted:", data);
});
```

Token Management

frond.js manages JWT tokens. Store a token after login, and frond.js attaches it to every request.

```
// Store the token (usually after login)
frond.setToken("eyJhbGciOiJIUzI1NiIs...");

// All subsequent requests include:
// Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
frond.get("/api/profile", function (data) {
    console.log("Profile:", data);
});

// Clear the token (logout)
frond.clearToken();
```

frond.js stores the token in `localStorage` and includes it as a `Bearer` token in the `Authorization` header on every request.

Loading Indicators

frond.js can show and hide a loading element during AJAX requests.

```
frond.get("/api/products", function (data) {
    renderProducts(data.products);
}, null, {
    loading: "#loadingSpinner"
});
```

```
<div id="loadingSpinner" class="text-center p-4" style="display: none;">
    Loading...
</div>
```

frond.js toggles `display: block` and `display: none` on the element. No extra CSS needed.

WebSocket Auto-Reconnect

```
const ws = frond.ws("/ws/notifications");
ws.on("message", function (data) {
    const notification = JSON.parse(data);
    alert(notification.text);
});
// If the server restarts or the network blips, frond.js reconnects.
```

tina4js.min.js -- Reactive Frontend Framework

A standalone reactive framework for building interactive client-side applications. Provides signals, computed values, effects, Web Components, client-side routing, and built-in fetch and WebSocket wrappers.

Reactive State

```
import { signal, computed, effect } from "/js/tina4js.min.js";

const count = signal(0);
const doubled = computed(() => count.value * 2);

effect(() => {
    console.log(`Count: ${count.value}, Doubled: ${doubled.value}`);
});

count.value = 5; // logs "Count: 5, Doubled: 10"
```

Web Components

```
import { Tina4Element, signal, html } from "/js/tina4js.min.js";

class CounterButton extends Tina4Element {
    setup() {
        this.count = signal(0);
    }
    render() {
        return html`
            <button onclick="${() => this.count.value++}">
                Clicked ${this.count} times
            </button>
        `;
    }
}
customElements.define("counter-button", CounterButton);
```

---

13. Responsive Design

tina4css includes responsive breakpoints for adapting layouts to different screen sizes:

Breakpoint	Min Width	Class Prefix
Extra small	0px	(none -- default)
Small	576px	`sm-`
Medium	768px	`md-`
Large	992px	`lg-`
Extra large	1200px	`xl-`

Responsive Columns

```
<div class="row">
    <!-- Full width on mobile, half on medium, third on large -->
    <div class="col-12 col-md-6 col-lg-4">Product 1</div>
    <div class="col-12 col-md-6 col-lg-4">Product 2</div>
    <div class="col-12 col-md-6 col-lg-4">Product 3</div>
</div>
```

Responsive Visibility

```
<!-- Hidden on mobile, visible on medium and up -->
<div class="d-none d-md-block">Desktop Sidebar</div>

<!-- Visible on mobile only -->
<div class="d-block d-md-none">Mobile Menu</div>
```

---

14. Building a Users Page with AJAX

A single-page admin view that loads data without page reloads:

```
{% extends "base.html" %}

{% block title %}User Management{% endblock %}

{% block content %}
<h2>User Management</h2>

<div id="loadingSpinner" class="text-center p-4" style="display: none;">
    Loading users...
</div>

<div id="user-list"></div>

<script src="/js/frond.min.js"></script>
<script>
    function loadUsers() {
        frond.get("/api/users", function (data) {
            var html = '<table class="table"><thead><tr>' +
                '<th>ID</th><th>Name</th><th>Email</th><th>Actions</th>' +
                '</tr></thead><tbody>';

            data.users.forEach(function (user) {
                html += '<tr>' +
                    '<td>' + user.id + '</td>' +
                    '<td>' + user.name + '</td>' +
                    '<td>' + user.email + '</td>' +
                    '<td><button class="btn btn-sm btn-danger" ' +
                    'onclick="deleteUser(' + user.id + ')">Delete</button></td>' +
                    '</tr>';
            });

            html += '</tbody></table>';
            document.getElementById("user-list").innerHTML = html;
        }, null, { loading: "#loadingSpinner" });
    }

    function deleteUser(id) {
        frond.delete("/api/users/" + id, function () {
            loadUsers();
        });
    }

    loadUsers();
</script>
{% endblock %}
```

The page loads instantly. frond.js fetches user data in the background. Delete a user and the list refreshes without a page reload.

---

15. Exercise: Build a Product Management Page

Build a product management page with a table, add/edit form, and AJAX interactions.

Requirements

- `GET /admin/products` -- HTML page with a product table and "Add Product" form
- Use frond.js for AJAX form submission
- Display flash messages for success/error
- Include search and category filter

---

16. Solution

Create `src/templates/admin-products.html` with a table displaying products, a form for adding new ones using `data-frond-submit`, and JavaScript handlers using `frond.post` and `frond.get` for CRUD operations. The template extends `base.html` and uses tina4css for styling.

Create `src/routes/admin_products.rb`:

```
Tina4::Router.get("/admin/products") do |request, response|
  db = Tina4.database
  products = db.fetch("SELECT * FROM products ORDER BY name")

  response.render("admin-products.html", {
    products: products,
    count: products.length
  })
end
```

---

17. Gotchas

1. tina4css Classes Do Not Work

Problem: CSS classes have no effect.

Fix: Make sure `<link rel="stylesheet" href="/css/tina4.css">` is in your HTML head.

2. frond.js Not Loading

Problem: `frond is not defined` error in the browser console.

Fix: Make sure `<script src="/js/frond.js"></script>` is included before your custom scripts.

3. AJAX Calls Return HTML Instead of JSON

Problem: `frond.get` returns HTML instead of JSON.

Fix: Your route handler is returning `response.render` instead of `response.json`. API endpoints should use `response.json`.

4. Forms Submit Twice

Problem: The form submits via AJAX and then also submits normally.

Fix: Call `event.preventDefault()` in your submit handler, or use `data-frond-submit` which handles this automatically.

5. Dark Mode Resets on Page Load

Problem: Dark mode resets to light mode when navigating.

Fix: Save the theme preference in `localStorage` and apply it on page load before the page renders.

6. Bootstrap Classes Not Working

Problem: Some Bootstrap classes do not work with tina4css.

Fix: tina4css is Bootstrap-compatible but not a complete clone. Stick to the documented classes. Complex Bootstrap components (modals, tooltips) require additional JavaScript.

7. SCSS Changes Not Reflected

Problem: You edited `tina4.scss` but the browser shows the old CSS.

Fix: Compile SCSS to CSS with `tina4 scss` or `sass src/public/scss/tina4.scss src/public/css/tina4.css`. The browser loads the compiled CSS file, not the SCSS source.

---

18. HtmlElement: Programmatic HTML Builder

Build HTML in Ruby without string concatenation:

```
el = Tina4::HtmlElement.new("div", { class: "card" }, ["Hello"])
el.to_s  # => '<div class="card">Hello</div>'

# Nesting
card = Tina4::HtmlElement.new("div", { class: "card" }, [
  Tina4::HtmlElement.new("h2", {}, ["Title"]),
  Tina4::HtmlElement.new("p", {}, ["Content"]),
])

# HtmlHelpers mixin
include Tina4::HtmlHelpers
html = _div({ class: "card" }, _p("Hello"), _a({ href: "/" }, "Home"))
```

Void tags (`<br>`, `<img>`, `<input>`) render without closing tags. Boolean attributes render as bare names.

Testing

1. Why Tests Matter More Than You Think

Friday afternoon. Your client reports a critical bug in production. You fix it -- one line of code. But did that fix break something else? 47 routes. 12 ORM models. 3 middleware functions. Clicking through every page: an hour. Running the test suite: 2 seconds.

```
tina4 test

  Product API
    ✓ creates a product
    ✓ loads a product by id
    ✓ updates a product
    ✓ deletes a product
    ✓ filters products by category

  Authentication
    ✓ logs in with valid credentials
    ✓ rejects invalid password
    ✓ protected route requires token
    ✓ allows access with valid token

  9 tests: 9 passed, 0 failed, 0 errors
```

Everything still passes. Deploy with confidence. Weekend intact.

Tina4 ships an inline testing framework -- no RSpec, no Minitest, no setup ceremony. The `Tina4::Testing` module gives you `describe`/`it` blocks, a full set of `assert_*` helpers, and an in-process HTTP test client. `tina4 test` discovers `*_test.rb` and `test_*.rb` files under `tests/`, `test/`, `spec/`, or `src/tests/` and runs them.

---

2. Your First Test

Tests live under `tests/`. Files must be named `*_test.rb` or `test_*.rb` -- the runner ignores anything else.

Create `tests/basic_test.rb`:

```
require "tina4"

Tina4::Testing.describe "Basic tests" do
  it "adds two numbers" do
    assert_equal(4, 2 + 2)
  end

  it "concatenates strings" do
    result = "Hello" + " " + "World"
    assert_equal("Hello World", result)
  end

  it "handles nil correctly" do
    value = nil
    assert_nil(value)
```

```
  end
end
```

Run it:

```
tina4 test
```

```
  Basic tests
    ✓ adds two numbers
    ✓ concatenates strings
    ✓ handles nil correctly

  3 tests: 3 passed, 0 failed, 0 errors
```

How It Works

- `Tina4::Testing.describe` opens a suite. The block defines the suite's tests.
- Each `it` registers a single test case. The description string becomes the label printed to the console.
- Inside an `it` block you call `assert_*` helpers to check behaviour. Any assertion that fails raises `Tina4::Testing::TestFailure` and marks the test as failed.
- `Tina4::Testing.run_all` runs every registered suite. `tina4 test` calls it for you.

---

3. Assertion Reference

Every assertion is defined on `Tina4::Testing::TestContext` and available inside `it` blocks. The convention is `assert_<thing>(expected, actual, message = nil)` -- expected first.

Equality

```
assert_equal(42, result)              # value equality
assert_not_equal(0, result)           # not equal
```

Nil

```
assert_nil(value)                      # value is nil
assert_not_nil(value)                  # value is not nil
```

Truthiness

```
assert(condition, "optional message")   # condition is truthy
assert_true(value)                      # value is truthy
assert_false(value)                     # value is falsy
```

Collections and Strings

```
assert_includes([1, 2, 3], 2)           # collection includes item
assert_match(/@/, "user@example.com")   # string matches pattern
```

Exceptions

```
assert_raises(ArgumentError) do
  raise ArgumentError, "bad input"
end
```

JSON / HTTP

```
data = assert_json(response.body)       # parses body, raises on invalid JSON
assert_status(response, 200)            # asserts HTTP status (Rack tuple OR TestResponse)
```

---

4. Testing Routes

Tina4's `Tina4::TestClient` issues requests against your registered routes in-process -- no socket, no server, no port. It returns a `Tina4::TestResponse` with `status`, `body`, `headers`, and a `json` helper.

Create `tests/product_test.rb`:

```
require "tina4"

Tina4::Testing.describe "Product API" do
  client = Tina4::TestClient.new

  before_each do
    db = Tina4.database
    db.execute("DELETE FROM products") if db
  end

  it "returns the products list" do
    resp = client.get("/api/products")

    assert_status(resp, 200)

    data = resp.json
    assert_not_nil(data, "Response should be valid JSON")
    assert_includes(data.keys, "products")
    assert(data["products"].is_a?(Array), "products should be an array")
  end

  it "creates a product" do
    resp = client.post("/api/products", json: {
      name: "Test Widget",
      category: "Testing",
      price: 9.99
    })

    assert_status(resp, 201)

    data = resp.json
    assert_equal("Test Widget", data["name"])
    assert_equal(9.99, data["price"])
    assert_not_nil(data["id"], "New product should have an id")
  end

  it "returns 404 for a missing product" do
    resp = client.get("/api/products/99999")

    assert_status(resp, 404)
  end

  it "validates required fields" do
```

```
    resp = client.post("/api/products", json: {})

    assert_status(resp, 400)

    data = resp.json
    assert_match(/required/, data["error"])
  end
end
```

TestClient Methods

`Tina4::TestClient` exposes one method per verb. Every method except `get`/`delete` accepts a `json:` keyword for the request body. Use `headers:` to attach a single hash of HTTP headers.

```
client = Tina4::TestClient.new

# GET - with optional query string and headers
resp = client.get("/api/products")
resp = client.get("/api/products?category=Electronics")
resp = client.get("/api/profile", headers: { "Authorization" => "Bearer #{token}" })

# POST / PUT / PATCH - pass a Ruby hash via `json:`
resp = client.post("/api/products", json: { name: "Widget", price: 9.99 })
resp = client.put("/api/products/1", json: { name: "Updated Widget" })
resp = client.patch("/api/products/1", json: { price: 12.99 })

# DELETE - no body
resp = client.delete("/api/products/1")
```

TestResponse

```
resp.status         # HTTP status code (200, 201, 404, ...)
resp.body           # raw response body as a String
resp.headers        # response headers as a Hash (lowercased keys)
resp.content_type   # value of the content-type header
resp.json           # JSON.parse(body); returns nil if the body is not valid JSON
resp.text           # alias for body.to_s
```

If you prefer to assert on the raw Rack tuple instead of a `TestResponse`, use the `get`/`post`/`put`/`delete` helpers exposed on the test context -- they return `[status, headers, body_enumerable]` and work with `assert_status` exactly the same way.

---

5. Testing ORM Models

Drop into the database directly when you need to verify model behaviour. `before_each` is the right place to truncate tables or seed fixtures.

```
require "tina4"

Tina4::Testing.describe "Product model" do
  before_each do
    db = Tina4.database
    db.execute("DELETE FROM products")
  end

  it "saves and reloads a product" do
```

```
    product = Product.new
    product.name = "Test Widget"
    product.category = "Testing"
    product.price = 19.99
    product.save

    assert_not_nil(product.id, "Product should have an id after save")

    loaded = Product.find(product.id)
    assert_equal("Test Widget", loaded.name)
    assert_equal(19.99, loaded.price)
  end

  it "updates a product" do
    product = Product.create(name: "Before", price: 10.0, category: "Testing")
    product.name = "After"
    product.price = 20.0
    product.save

    reloaded = Product.find(product.id)
    assert_equal("After", reloaded.name)
    assert_equal(20.0, reloaded.price)
  end

  it "deletes a product" do
    product = Product.create(name: "Goodbye", price: 5.0, category: "Testing")
    id = product.id

    product.delete

    assert_nil(Product.find(id), "Deleted product should not be findable")
  end

  it "filters by category" do
    Product.create(name: "Phone", category: "Electronics", price: 499.0)
    Product.create(name: "Yoga Mat", category: "Fitness", price: 29.0)

    results = Product.where("category = ?", ["Electronics"])

    assert_equal(1, results.length)
    assert_equal("Phone", results[0].name)
  end
end
```

Test database isolation

By default, the framework uses whatever `TINA4_DATABASE_URL` points at. To keep tests off your development data, point them at a dedicated SQLite file:

```
# .env.test
TINA4_DATABASE_URL=sqlite:///data/test.db
TINA4_DEBUG=false
```

Load it explicitly in your test bootstrap, or set `TINA4_ENV=test` and have your app pick the matching `.env.test` at startup.

---

6. Testing Authentication

Auth flows are easy to test because `TestClient` works in-process -- there is no JWT round-trip over the wire.

```
require "tina4"

Tina4::Testing.describe "Authentication" do
  client = Tina4::TestClient.new

  before_each do
    User.where("email = ?", ["test@example.com"]).each(&:delete)

    client.post("/api/auth/register", json: {
      name: "Test User",
      email: "test@example.com",
      password: "SecurePass123!"
    })
  end

  it "logs in with valid credentials" do
    resp = client.post("/api/auth/login", json: {
      email: "test@example.com",
      password: "SecurePass123!"
    })

    assert_status(resp, 200)

    body = resp.json
    assert_not_nil(body["token"], "Should return a JWT token")
    assert(body["token"].length > 50, "Token should be substantial")
  end

  it "rejects an invalid password" do
    resp = client.post("/api/auth/login", json: {
      email: "test@example.com",
      password: "wrong-password"
    })

    assert_status(resp, 401)
    assert_match(/invalid/i, resp.json["error"])
  end

  it "protects routes without a token" do
    resp = client.get("/api/profile")

    assert_status(resp, 401)
  end

  it "allows access with a valid token" do
    login = client.post("/api/auth/login", json: {
      email: "test@example.com",
      password: "SecurePass123!"
    })
    token = login.json["token"]

    resp = client.get("/api/profile", headers: {
      "Authorization" => "Bearer #{token}"
```

```
    })

    assert_status(resp, 200)
    assert_equal("test@example.com", resp.json["email"])
  end
end
```

---

7. Setup and Teardown

`before_each` runs before every test in the suite. `after_each` runs after every test, even on failure. Use them to create fixtures and clean up state:

```
Tina4::Testing.describe "User Management" do
  client = Tina4::TestClient.new
  user_id = nil

  before_each do
    user = User.new(name: "Test User", email: "fixture@example.com")
    user.save
    user_id = user.id
  end

  after_each do
    User.find(user_id)&.delete if user_id
  end

  it "loads the user" do
    loaded = User.find(user_id)
    assert_equal("Test User", loaded.name)
  end

  it "updates the user" do
    user = User.find(user_id)
    user.name = "Updated Name"
    user.save

    reloaded = User.find(user_id)
    assert_equal("Updated Name", reloaded.name)
  end
end
```

There is no per-suite hook. If you need one-off setup that applies to every test, do it at the top of the file before `describe`, or accept the cost of running it in `before_each`.

---

8. Running Tests

```
# Run every discovered test file
tina4 test
```

The runner walks `tests/`, `test/`, `spec/`, and `src/tests/` (in that order) and loads every `*_test.rb` and `test_*.rb` file it finds. Inline tests declared inside route files are picked up automatically because `tina4 test` also loads `routes/`.

Output uses ANSI colours: green checks for passes, red crosses for failures, and yellow bangs for unexpected exceptions. Exit code is non-zero if any test fails or errors -- perfect for CI.

```
  Product API
    ✓ returns the products list
    ✓ creates a product
    ✗ returns 404 for a missing product: Expected status 404, got 200
    ✓ validates required fields

  4 tests: 3 passed, 1 failed, 0 errors
```

The failure line shows the suite name, the test description, and the assertion message. Find the line. Fix the logic. Run again.

Embedding the runner in your own scripts

```
require "tina4"

# Load whichever test files you want, then:
results = Tina4::Testing.run_all(quiet: false, failfast: false)
exit(results[:failed] > 0 || results[:errors] > 0 ? 1 : 0)
```

`run_all` returns a hash with `:passed`, `:failed`, `:errors`, and a `:tests` array of `{ name:, status:, suite:, message: }` entries. Pass `quiet: true` to suppress console output and inspect the hash directly. Pass `failfast: true` to stop at the first failure.

---

9. Testing Best Practices

Test one thing per `it`

Each test should verify one behaviour. When it fails, you know exactly what broke.

```
# Good - each test verifies one thing
it "returns 201 on create" do
  resp = client.post("/api/products", json: { name: "Widget", price: 9.99 })
  assert_status(resp, 201)
end

it "returns the created product" do
  resp = client.post("/api/products", json: { name: "Widget", price: 9.99 })
  assert_equal("Widget", resp.json["name"])
end

# Bad - testing five things in one block
it "does everything" do
  # Creates, reads, updates, deletes, checks auth, validates input ...
  # When this fails you have no idea which step broke.
end
```

Use descriptive test names

```
# Good
it "returns 404 when the product does not exist"

# Bad
it "works"
```

Isolate tests

Each test should create its own data and clean up after itself. Never depend on rows left behind by another test.

```
# Good
it "deletes a product" do
  product = Product.create(name: "Temporary", price: 1.0)
  product.delete
  assert_nil(Product.find(product.id))
end
```

Generate unique values

Avoid `UNIQUE` constraint failures by mixing in a random or time-based suffix:

```
require "securerandom"

it "registers a user" do
  email = "test-#{SecureRandom.hex(4)}@example.com"
  resp = client.post("/api/auth/register", json: {
    name: "Test", email: email, password: "Pass1234!"
  })
  assert_status(resp, 201)
end
```

Compare floats with tolerance

`assert_equal(9.99, product.price)` can fail when SQLite hands back `9.990000000000001`. When the engine is fussy, use a manual tolerance:

```
diff = (product.price - 9.99).abs
assert(diff < 0.01, "Expected ~9.99, got #{product.price}")
```

---

10. Exercise: Write Tests for a Notes API

Write a test suite for the Notes API from Chapter 5.

Requirements

Cover these scenarios:

- Create a note with valid data (201)
- Reject a note with a missing title (400)
- List all notes (200)
- Get a note by id (200)
- Return 404 for a non-existent note
- Update a note (200)
- Delete a note (204)
- Search notes by content
- Filter notes by tag

---

11. Solution

Create `tests/notes_test.rb`:

```
require "tina4"

Tina4::Testing.describe "Notes API" do
  client = Tina4::TestClient.new

  before_each do
    db = Tina4.database
    db.execute("DELETE FROM notes")
  end

  it "creates a note with valid data" do
    resp = client.post("/api/notes", json: {
      title: "Test Note",
      content: "This is a test",
      tag: "testing"
    })

    assert_status(resp, 201)

    body = resp.json
    assert_equal("Test Note", body["title"])
    assert_equal("testing", body["tag"])
    assert_not_nil(body["id"])
  end

  it "rejects a note with a missing title" do
    resp = client.post("/api/notes", json: { content: "No title here" })

    assert_status(resp, 400)
    assert_match(/title/i, resp.json["error"])
  end

  it "lists all notes" do
    client.post("/api/notes", json: { title: "Note 1", content: "Content 1" })
    client.post("/api/notes", json: { title: "Note 2", content: "Content 2" })

    resp = client.get("/api/notes")

    assert_status(resp, 200)
    assert_equal(2, resp.json["notes"].length)
  end

  it "gets a note by id" do
    created = client.post("/api/notes", json: { title: "Find Me", content: "Here I am" })
    id = created.json["id"]

    resp = client.get("/api/notes/#{id}")

    assert_status(resp, 200)
    assert_equal("Find Me", resp.json["title"])
  end

  it "returns 404 for a non-existent note" do
```

```
    resp = client.get("/api/notes/99999")
    assert_status(resp, 404)
  end

  it "updates a note" do
    created = client.post("/api/notes", json: { title: "Original", content: "Original content" }
    id = created.json["id"]

    resp = client.put("/api/notes/#{id}", json: { title: "Updated", content: "Updated content" }

    assert_status(resp, 200)
    assert_equal("Updated", resp.json["title"])
  end

  it "deletes a note" do
    created = client.post("/api/notes", json: { title: "Delete Me", content: "Goodbye" })
    id = created.json["id"]

    resp = client.delete("/api/notes/#{id}")
    assert_status(resp, 204)

    assert_status(client.get("/api/notes/#{id}"), 404)
  end

  it "searches notes by content" do
    client.post("/api/notes", json: { title: "Shopping", content: "Buy milk and eggs" })
    client.post("/api/notes", json: { title: "Work", content: "Finish the report" })

    resp = client.get("/api/notes?search=milk")

    assert_status(resp, 200)
    assert_equal(1, resp.json["notes"].length)
    assert_equal("Shopping", resp.json["notes"][0]["title"])
  end

  it "filters notes by tag" do
    client.post("/api/notes", json: { title: "Personal", content: "Content", tag: "personal" })
    client.post("/api/notes", json: { title: "Work", content: "Content", tag: "work" })

    resp = client.get("/api/notes?tag=personal")

    assert_status(resp, 200)
    assert_equal(1, resp.json["notes"].length)
    assert_equal("Personal", resp.json["notes"][0]["title"])
  end
end
```

---

12. Gotchas

1. Files must be named correctly

Problem: Your tests don't run.

Cause: The runner only loads `*_test.rb` and `test_*.rb`. A file called `notes_spec.rb` will be skipped.

Fix: Rename to `notes_test.rb` or `test_notes.rb`.

2. Database state carries between tests

Problem: A test passes alone but fails when the whole suite runs.

Fix: Truncate tables in `before_each` or use unique values per test (UUID/timestamp suffix on emails, etc.).

3. `TestClient` doesn't start a real server

Problem: External services (SMTP, Redis, third-party APIs) are not available in tests.

Fix: `TestClient` matches routes and runs handlers in-process. Mock out anything that crosses the network boundary, or use the dev mailbox / fake queue backends shipped with the framework.

4. Argument order on equality assertions

Problem: The failure message reads "Expected: 200, got: 404" but you wanted to compare the other way around.

Fix: The Tina4 convention is `assert_equal(expected, actual)` -- expected first, like Minitest. Keep the order consistent across your suite.

5. Floating point comparisons

Problem: `assert_equal(9.99, product.price)` fails with `9.990000000000001`.

Fix: Compare against a tolerance with `assert(diff < 0.01, ...)` as shown above, or store currency as integer cents.

6. Tests can't find your models

Problem: `NameError: uninitialized constant Product` when the test loads.

Cause: Routes and ORM models are normally loaded by the server boot path. `tina4 test` calls `Tina4.initialize!(Dir.pwd)` followed by route discovery, but a model file that isn't `require`d anywhere won't load on its own.

Fix: Either `require_relative "../src/orm/product"` at the top of the test file, or move the model into the autoloaded `src/orm/` directory.

7. Tests pass locally but fail in CI

Problem: All green on your machine, red in CI.

Cause: Different Ruby version, missing env vars, different SQLite version, time-dependent assertions.

Fix: Pin the Ruby version in CI (`.ruby-version`), commit `.env.test` if it contains no secrets, and avoid asserting on wall-clock time or random values.

Scaffolding

One command. Six files. A working CRUD feature with routes, templates, tests, and Swagger docs -- ready to run.

That is Tina4's scaffolding system. It generates the boilerplate you write by hand in every project: models, migrations, routes, forms, views, and tests. You describe what you want. The generators produce it.

The CRUD Generator

This is the generator most developers reach for first. It creates everything a feature needs in one shot.

```
tina4 generate crud Product --fields "name:string,price:float"
```

That single command creates six files:

#	File	Purpose
1	`src/orm/product.rb`	ORM model with typed fields
2	`migrations/20260401_create_product.sql`	UP migration (CREATE TABLE)
3	`migrations/20260401_create_product.down.sql`	DOWN migration (DROP TABLE)
4	`src/routes/products.rb`	CRUD routes with Swagger annotations
5	`src/templates/products/form.html`	Form template with typed inputs and `form_token`
6	`src/templates/products/view.html`	List and detail templates
7	`spec/products_spec.rb`	RSpec stubs for all CRUD operations

What Each File Contains

The **model** maps the `product` table to a Ruby class:

```
require "tina4"

class Product < Tina4::ORM
  table_name "product"

  field :id, :integer
```

```
  field :name, :string
  field :price, :float
  field :created_at, :datetime
  field :updated_at, :datetime
end
```

The **migration** creates the table:

```
-- migrations/20260401_create_product.sql
CREATE TABLE product (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name VARCHAR(255),
    price FLOAT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

The **down migration** reverses it:

```
-- migrations/20260401_create_product.down.sql
DROP TABLE IF EXISTS product;
```

The **routes** file wires up five endpoints with Swagger docs:

```
require "tina4"
require_relative "../orm/product"

Tina4.get "/api/products", description: "List all products" do |request, response|
  products = Product.new.select
  response.call(products)
end

Tina4.get "/api/products/:id", description: "Get a product by ID" do |request, response|
  product = Product.new
  product.id = request.params[:id]
  product.load
  response.call(product)
end

Tina4.post "/api/products", description: "Create a product" do |request, response|
  product = Product.new(request.body)
  product.save
  response.call(product, 201)
end

Tina4.put "/api/products/:id", description: "Update a product" do |request, response|
  product = Product.new(request.body)
  product.id = request.params[:id]
  product.save
  response.call(product)
end

Tina4.delete "/api/products/:id", description: "Delete a product" do |request, response|
  product = Product.new
  product.id = request.params[:id]
  product.delete
  response.call(nil, 204)
end
```

The **form template** renders typed inputs with CSRF protection:

```
<form method="POST" action="/api/products">
    <input type="hidden" name="form_token" value="{{ form_token }}">
    <label>Name</label>
    <input type="text" name="name" required>
    <label>Price</label>
    <input type="number" step="0.01" name="price" required>
    <button type="submit">Save</button>
</form>
```

The **test file** stubs out CRUD assertions:

```
require "tina4"

RSpec.describe "Products API" do
  let(:client) { Tina4::TestClient.new }

  it "creates a product" do
    response = client.post("/api/products", json: { name: "Widget", price: 9.99 })
    expect(response.status).to eq(201)
  end

  it "lists products" do
    response = client.get("/api/products")
    expect(response.status).to eq(200)
  end

  it "gets a product" do
    response = client.get("/api/products/1")
    expect(response.status).to eq(200)
  end

  it "updates a product" do
    response = client.put("/api/products/1", json: { name: "Updated Widget" })
    expect(response.status).to eq(200)
  end

  it "deletes a product" do
    response = client.delete("/api/products/1")
    expect(response.status).to eq(204)
  end
end
```

Run It

After generating, run the migration and start the server:

```
tina4 migrate
tina4 serve
```

Open Swagger UI at `http://localhost:7147/swagger` and test every endpoint. The scaffolded code works out of the box.

Individual Generators

The CRUD generator calls several smaller generators under the hood. You can call each one directly when you need a single piece.

Model

```
tina4 generate model Product --fields "name:string,price:float"
```

Creates three files: the ORM model (`src/orm/product.rb`), the UP migration, and the DOWN migration. No routes, no templates, no tests.

Route

```
tina4 generate route products --model Product
```

Creates one file: `src/routes/products.rb` with CRUD endpoints and Swagger annotations. The model must exist first.

Migration

```
tina4 generate migration add_category_to_product
```

Creates two files: `migrations/20260401_add_category_to_product.sql` and `migrations/20260401_add_category_to_product.down.sql`. Both are empty stubs. You write the SQL.

Middleware

```
tina4 generate middleware AuthLog
```

Creates one file with before and after stubs:

```
require "tina4"

Tina4.middleware :before do |request|
  puts "Request: #{request.method} #{request.url}"
  request
end

Tina4.middleware :after do |request, response|
  puts "Response: #{response.status}"
  response
end
```

Test

```
tina4 generate test products --model Product
```

Creates one file: `spec/products_spec.rb` with RSpec CRUD stubs.

Form

```
tina4 generate form Product --fields "name:string,price:float"
```

Creates one file: `src/templates/products/form.html` with typed inputs and `form_token`.

View

```
tina4 generate view Product --fields "name:string,price:float"
```

Creates two templates: a list view and a detail view in `src/templates/products/`.

CRUD

```
tina4 generate crud Product --fields "name:string,price:float"
```

Shorthand for running all generators at once: model, migration, route, form, view, and test.

Auth

```
tina4 generate auth
```

Generates the full authentication scaffold: User model, migrations, login/register/logout routes, templates, and tests.

AutoCRUD

AutoCRUD automatically generates REST API endpoints from your ORM models:

- `GET /api/{table}` - List with pagination (`?limit=10&offset=0`)
- `GET /api/{table}/{id}` - Get single record
- `POST /api/{table}` - Create record
- `PUT /api/{table}/{id}` - Update record
- `DELETE /api/{table}/{id}` - Delete record

Usage

```
Tina4::AutoCrud.register(User)
Tina4::AutoCrud.generate_routes(prefix: "/api")
```

`AutoCrud.register` wires up a single model. `generate_routes` mounts the five standard endpoints automatically, no route files needed.

The Auth Generator

Authentication needs more than one file. The auth generator creates seven:

```
tina4 generate auth
```

#	File	Purpose
1	`src/orm/user.rb`	User model with hashed password field
2	`migrations/20260401_create_user.sql`	UP migration
3	`migrations/20260401_create_user.down.sql`	DOWN migration
4	`src/routes/auth.rb`	Login, register, logout routes
5	`src/templates/auth/login.html`	Login form

6	`src/templates/auth/register.html`	Registration form
7	`spec/auth_spec.rb`	Auth flow tests

The generated routes handle password hashing, JWT token creation, and session management. The templates include CSRF tokens. The tests cover registration, login, invalid credentials, and logout.

Run the migration, start the server, and you have working auth:

```
tina4 migrate
tina4 serve
```

Field Types

Generators accept these field types. Each type maps to a specific column type in migrations, input type in forms, and display format in views.

Field Type	Migration Column	Form Input	View Display
`string`	`VARCHAR(255)`	`<input type="text">`	Plain text
`int`	`INTEGER`	`<input type="number">`	Number
`float`	`FLOAT`	`<input type="number" step="0.01">`	Decimal
`bool`	`BOOLEAN`	`<input type="checkbox">`	Yes / No
`text`	`TEXT`	`<textarea>`	Paragraph
`datetime`	`DATETIME`	`<input type="datetime-local">`	Formatted date
`blob`	`BLOB`	`<input type="file">`	Download link

Table Naming Convention

Tina4 uses singular table names. The model name `Product` maps to the table `product`. The model name `OrderItem` maps to `order_item`. The generator handles the conversion.

Combining Generators

Sometimes you want a model with routes but no form. Or a model with a migration but no test. The `--with` flags let you compose:

```
tina4 generate model Product --fields "name:string,price:float" --with-route --with-migration
```

Available flags:

Flag	Adds
`--with-route`	CRUD route file
`--with-migration`	Migration files (included by default with model)
`--with-test`	Test file
`--with-form`	Form template
`--with-view`	View templates

The `generate crud` command is equivalent to using all `--with` flags at once.

Exercise: Scaffold a Blog

Build a blog with three resources using generators.

Step 1: Generate the auth system.

```
tina4 generate auth
```

Step 2: Scaffold the Post resource.

```
tina4 generate crud Post --fields "title:string,body:text,published:bool"
```

Step 3: Scaffold the Category resource.

```
tina4 generate crud Category --fields "name:string,description:text"
```

Step 4: Add a migration to link posts to categories.

```
tina4 generate migration add_category_id_to_post
```

Edit the migration to add the foreign key:

```
ALTER TABLE post ADD COLUMN category_id INTEGER REFERENCES category(id);
```

Step 5: Run all migrations and start the server.

```
tina4 migrate
tina4 serve
```

You now have a working blog with authentication, posts, categories, and Swagger documentation. Total commands: five. Total hand-written SQL: one line.

Gotchas

Generators Do Not Overwrite

If a file exists, the generator skips it and prints a warning. This protects your edits. To regenerate, delete the file first.

Run Migrate After Generate

The model generator creates migration files. Those files do nothing until you run `tina4 migrate`. Generate and migrate are separate steps by design.

File Naming Matters

The generator derives file names from the model name. `Product` becomes `product.rb` for the model and `products.rb` for routes. Do not rename generated files unless you update all requires.

Field Changes Need New Migrations

Changing `--fields` and re-running the generator does not update existing migrations. Create a new migration with `generate migration` and write the ALTER TABLE by hand.

Singular Table Names

Tina4 uses singular table names: `product`, not `products`. The route paths use plural (`/api/products`), but the table stays singular. The generator handles this split.

API Documentation with Swagger

1. The 47-Endpoint Problem

Your team has 47 API endpoints. The frontend developer keeps asking "what does this endpoint accept?" You email a spreadsheet. It goes stale. You write a wiki page. Nobody touches it. You add comments to the code. Nobody reads them.

Swagger (OpenAPI) solves this for good. It generates interactive API documentation from annotations in your route files. The docs stay current because they live in the code. Your frontend developer browses every endpoint, sees expected request and response formats, and tests endpoints from the browser.

Tina4 auto-generates a Swagger UI at `/swagger` from comment annotations on your routes. No build step. No extra tooling. Write the annotations. The documentation appears.

---

2. What Swagger/OpenAPI Is

OpenAPI is a specification format for describing REST APIs. Swagger is the toolset that reads OpenAPI specs and generates documentation, client SDKs, and server stubs.

An OpenAPI spec describes:

- Every endpoint (path + HTTP method)
- What parameters each endpoint accepts (path, query, header, body)
- What each endpoint returns (response codes, response bodies)
- Data schemas (what a "User" or "Product" object looks like)
- Authentication requirements
- Grouping and tagging

Tina4 builds this spec automatically from comment annotations in your Ruby code. You never write JSON or YAML by hand.

---

3. Enabling Swagger

Swagger is available out of the box when `TINA4_DEBUG=true`. Navigate to:

```
http://localhost:7147/swagger
```

You should see the Swagger UI with any routes you have already defined. If you have not added any Swagger annotations yet, you will see the routes listed with default descriptions.

For production, you can explicitly enable or disable Swagger:

```
TINA4_SWAGGER_ENABLED=true
```

The Swagger JSON Endpoint

The raw OpenAPI spec is available at:

```
http://localhost:7147/swagger/json
```

```
curl http://localhost:7147/swagger/json
```

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "My Store API",
    "version": "1.0.0"
  },
  "paths": {
    "/api/products": {
      "get": {
        "summary": "List all products",
        "responses": {
          "200": {
            "description": "Successful response"
          }
        }
      }
    }
  }
}
```

---

4. Adding Descriptions to Routes

Add Swagger annotations as comments above your route definitions:

```
# List all products
# @description Returns a paginated list of all products in the catalog
# @tags Products
Tina4::Router.get("/api/products") do |request, response|
  response.json({ products: [] })
end
```

The first comment line becomes the `summary`. The `@description` tag provides a longer explanation.

Documenting Path Parameters

```
# Get a product by ID
# @description Returns a single product with full details including inventory status
# @tags Products
# @param int $id The unique product identifier
Tina4::Router.get("/api/products/{id:int}") do |request, response|
  id = request.params["id"]
  response.json({ id: id, name: "Wireless Keyboard", price: 79.99 })
end
```

Documenting Query Parameters

```
# Search products
# @description Search the product catalog by name, category, or price range
# @tags Products
# @query string $q Search query (searches product name and description)
# @query string $category Filter by category name
# @query float $min_price Minimum price filter
# @query float $max_price Maximum price filter
# @query int $page Page number (default: 1)
# @query int $limit Items per page (default: 20, max: 100)
Tina4::Router.get("/api/products/search") do |request, response|
  q = request.params["q"] || ""
  page = (request.params["page"] || 1).to_i
  limit = [(request.params["limit"] || 20).to_i, 100].min

  response.json({
    query: q,
    page: page,
    limit: limit,
    results: [],
    total: 0
  })
end
```

---

5. Documenting Request and Response Schemas

Request Body

```
# Create a new product
# @description Creates a product in the catalog. Requires admin authentication.
# @tags Products
# @body {"name": "string", "category": "string", "price": "float", "in_stock": "bool", "descript
# @response 201 {"id": "int", "name": "string", "category": "string", "price": "float", "in_stoc
# @response 400 {"error": "string"}
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  if body["name"].nil? || body["name"].empty?
    return response.json({ error: "Name is required" }, 400)
  end

  response.json({
    id: 1,
    name: body["name"],
    category: body["category"] || "Uncategorized",
    price: (body["price"] || 0).to_f,
    in_stock: body["in_stock"] != false,
    created_at: Time.now.iso8601
  }, 201)
end
```

---

6. Tags for Grouping Endpoints

Tags group related endpoints in the Swagger UI:

```
# List all users
# @tags Users
Tina4::Router.get("/api/users") do |request, response|
  response.json({ users: [] })
end

# List all orders
# @tags Orders
Tina4::Router.get("/api/orders") do |request, response|
  response.json({ orders: [] })
end
```

In the Swagger UI, you will see sections for "Users" and "Orders". An endpoint can belong to multiple groups:

```
# Get user's orders
# @tags Users, Orders
Tina4::Router.get("/api/users/{id:int}/orders") do |request, response|
  response.json({ orders: [] })
end
```

---

7. Example Values

Add example values to make the docs more useful:

```
# Create a new product
# @description Creates a product in the catalog
# @tags Products
# @example request {"name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99, "in
# @example response {"id": 42, "name": "Ergonomic Keyboard", "category": "Electronics", "price":
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  response.json({
    id: 42,
    name: body["name"],
    category: body["category"] || "Uncategorized",
    price: (body["price"] || 0).to_f,
    in_stock: body["in_stock"] != false,
    created_at: Time.now.iso8601
  }, 201)
end
```

---

8. Try-It-Out from the Swagger UI

The Swagger UI includes a "Try it out" button on every endpoint. Clicking it:

- Expands the endpoint with editable input fields
- Pre-fills example values (if provided)

- Lets you edit the parameters, headers, and request body
- Sends the actual HTTP request to your running server
- Shows the response status, headers, and body

This is a live testing tool built into your documentation.

Authentication in Try-It-Out

If your endpoints require authentication, click the "Authorize" button at the top of the Swagger UI. Enter your JWT token or API key, and all subsequent "Try it out" requests will include the authentication header.

---

9. Customizing the Swagger Info Block

Configure the top-level API information in `.env`:

```
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_DESCRIPTION=API for managing products, orders, and users
TINA4_SWAGGER_VERSION=1.0.0
TINA4_SWAGGER_CONTACT_EMAIL=api@mystore.com
TINA4_SWAGGER_LICENSE=MIT
```

---

10. Generating Client SDKs from the Spec

The OpenAPI spec at `/swagger/json` can be used with code generation tools:

```
# Install the OpenAPI Generator CLI
npm install -g @openapitools/openapi-generator-cli

# Generate a TypeScript client
openapi-generator-cli generate \
  -i http://localhost:7147/swagger/json \
  -g typescript-fetch \
  -o ./frontend/api-client
```

---

11. A Complete Documented API

Here is a full example showing all the annotation features together:

```
# List all users
# @description Returns a paginated list of users. Supports filtering by role and searching by na
# @tags Users
# @query int $page Page number (default: 1)
# @query int $limit Items per page (default: 20)
# @query string $role Filter by role (admin, user, moderator)
# @query string $search Search by name or email
# @response 200 {"users": [{"id": "int", "name": "string", "email": "string", "role": "string"}]
# @example response {"users": [{"id": 1, "name": "Alice", "email": "alice@example.com", "role":
Tina4::Router.get("/api/users") do |request, response|
  page = (request.params["page"] || 1).to_i
```

```
  limit = (request.params["limit"] || 20).to_i

  response.json({
    users: [
      { id: 1, name: "Alice", email: "alice@example.com", role: "admin" },
      { id: 2, name: "Bob", email: "bob@example.com", role: "user" }
    ],
    total: 42,
    page: page,
    pages: (42.0 / limit).ceil
  })
end

# Create a new user
# @description Creates a user account. Email must be unique.
# @tags Users
# @body {"name": "string", "email": "string", "password": "string", "role": "string"}
# @response 201 {"id": "int", "name": "string", "email": "string", "role": "string", "created_at
# @response 400 {"errors": ["string"]}
# @response 409 {"error": "string"}
# @example request {"name": "Charlie", "email": "charlie@example.com", "password": "securePass12
# @example response {"id": 3, "name": "Charlie", "email": "charlie@example.com", "role": "user",
Tina4::Router.post("/api/users") do |request, response|
  body = request.body

  errors = []
  errors << "Name is required" if body["name"].nil? || body["name"].empty?
  errors << "Email is required" if body["email"].nil? || body["email"].empty?
  errors << "Password is required" if body["password"].nil? || body["password"].empty?

  unless errors.empty?
    return response.json({ errors: errors }, 400)
  end

  response.json({
    id: 3,
    name: body["name"],
    email: body["email"],
    role: body["role"] || "user",
    created_at: Time.now.iso8601
  }, 201)
end
```

---

12. Exercise: Document a Complete User API

Take the User API from the example above and extend it with the following endpoints. Write full Swagger annotations for each one.

Requirements

Document these additional endpoints (you can use hardcoded data in the handlers):

Method	Path	Description

PUT	/api/users/{id}	Update a user. Body: name, email, role. Response: updated user.
GET	/api/users/{id}/orders	List a user's orders. Query: status filter, pagination.
POST	/api/users/{id}/avatar	Upload user avatar. Body: avatar_url string.

Each endpoint should have:

- A summary (first comment line)
- A `@description`
- A `@tags` annotation
- `@param` for path parameters
- `@query` for query parameters (where applicable)
- `@body` for request body (where applicable)
- `@response` for each possible response code
- `@example` for request and response (where applicable)

Test by visiting:

```
http://localhost:7147/swagger
```

---

13. Solution

Create `src/routes/user_api_documented.rb`:

```
# Update a user
# @description Updates an existing user's profile information. Only provided fields are updated.
# @tags Users
# @param int $id User ID
# @body {"name": "string", "email": "string", "role": "string"}
# @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "updated_at
# @response 404 {"error": "string"}
# @example request {"name": "Alice Smith", "email": "alice.smith@example.com", "role": "admin"}
# @example response {"id": 1, "name": "Alice Smith", "email": "alice.smith@example.com", "role":
Tina4::Router.put("/api/users/{id:int}") do |request, response|
  id = request.params["id"]
  body = request.body

  if id > 100
    return response.json({ error: "User not found" }, 404)
  end

  response.json({
    id: id,
    name: body["name"] || "Alice",
```

```
    email: body["email"] || "alice@example.com",
    role: body["role"] || "user",
    updated_at: Time.now.iso8601
  })
end

# List user orders
# @description Returns a paginated list of orders for a specific user.
# @tags Users, Orders
# @param int $id User ID
# @query string $status Filter by order status (pending, processing, shipped, delivered, cancell
# @query int $page Page number (default: 1)
# @query int $limit Items per page (default: 20)
# @response 200 {"orders": [{"id": "int", "product": "string", "quantity": "int", "total": "floa
# @response 404 {"error": "string"}
# @example response {"orders": [{"id": 101, "product": "Wireless Keyboard", "quantity": 2, "tota
Tina4::Router.get("/api/users/{id:int}/orders") do |request, response|
  id = request.params["id"]
  status = request.params["status"]
  page = (request.params["page"] || 1).to_i

  if id > 100
    return response.json({ error: "User not found" }, 404)
  end

  orders = [
    { id: 101, product: "Wireless Keyboard", quantity: 2, total: 159.98, status: "shipped" },
    { id: 102, product: "USB-C Hub", quantity: 1, total: 49.99, status: "delivered" }
  ]

  orders = orders.select { |o| o[:status] == status } if status

  response.json({ orders: orders, total: orders.length, page: page })
end

# Upload user avatar
# @description Sets or updates the avatar URL for a user.
# @tags Users
# @param int $id User ID
# @body {"avatar_url": "string"}
# @response 200 {"id": "int", "avatar_url": "string", "updated_at": "string"}
# @response 400 {"error": "string"}
# @response 404 {"error": "string"}
# @example request {"avatar_url": "https://cdn.example.com/avatars/alice-2026.jpg"}
# @example response {"id": 1, "avatar_url": "https://cdn.example.com/avatars/alice-2026.jpg", "u
Tina4::Router.post("/api/users/{id:int}/avatar") do |request, response|
  id = request.params["id"]
  body = request.body

  if id > 100
    return response.json({ error: "User not found" }, 404)
  end

  if body["avatar_url"].nil? || body["avatar_url"].empty?
    return response.json({ error: "avatar_url is required" }, 400)
  end

  response.json({
    id: id,
```

```
    avatar_url: body["avatar_url"],
    updated_at: Time.now.iso8601
  })
end
```

---

14. Gotchas

1. Annotations Must Be Directly Above the Route

Problem: Your Swagger annotations do not appear in the docs.

Cause: There is a blank line or other code between the comment block and the `Tina4::Router` call.

Fix: Make sure the comments are on the lines directly before the route definition with no blank lines in between.

2. Missing @tags Makes Endpoints Hard to Find

Problem: All endpoints appear in one giant flat list in the Swagger UI.

Fix: Add `# @tags ResourceName` to every route comment block.

3. @body Must Be Valid JSON

Problem: The Swagger UI shows the body schema as empty or broken.

Fix: Validate your `@body` JSON. Every key and string value must be in double quotes.

4. Swagger Shows Routes You Did Not Annotate

Problem: Unannotated routes appear in the Swagger UI with minimal documentation.

Cause: Tina4 includes all registered routes in the Swagger spec. Add `# @hidden` to hide a route.

5. Response Examples Do Not Match Actual Responses

Problem: The example response in Swagger shows different fields than the actual API response.

Fix: Treat annotations as part of the code. When you change a handler's response format, update the annotations.

6. Swagger UI Not Available in Production

Problem: `/swagger` returns a 404 in production.

Fix: If you want Swagger in production, explicitly set `TINA4_SWAGGER_ENABLED=true` in your `.env`.

7. SDK Generation Produces Incorrect Types

Problem: The generated TypeScript client has `any` types instead of proper interfaces.

Fix: Use correct OpenAPI type format in your annotations: `"name": "string"` (lowercase), `"price": "number"`, `"in_stock": "boolean"`.

API Client

1. Talking to Other Services

Your app does not live in isolation. It calls payment gateways, weather APIs, shipping providers, CRM platforms, and internal microservices. Each call needs a base URL, auth headers, timeout handling, and response parsing.

Tina4's API client handles that boilerplate. One configured instance gives you clean `get`, `post`, `put`, and `delete` calls with a consistent response format and no external gem required.

---

2. Creating a Client

```
client = Tina4::Api.new("https://api.example.com")
```

All requests use this base URL. Paths are appended to it.

With Default Headers

```
client = Tina4::Api.new("https://api.example.com", {
  "Content-Type" => "application/json",
  "Accept"       => "application/json"
})
```

Headers set here are sent with every request.

---

3. GET Requests

```
result = client.get("/products")

if result.success?
  puts result.body.inspect
else
  puts "Error #{result.status}: #{result.body}"
end
```

With Query Parameters

```
result = client.get("/products", params: { category: "keyboards", in_stock: true })
# Requests: GET /products?category=keyboards&in_stock=true
```

Response Object

Every call returns a response object with:

- `result.success?` -- true if HTTP status is 2xx
- `result.status` -- integer HTTP status code
- `result.body` -- parsed JSON (Hash or Array) or raw string
- `result.headers` -- response headers Hash

---

4. POST Requests

```
result = client.post("/orders", body: {
  email: "alice@example.com",
  items: [{ sku: "KB-100", qty: 1 }],
  total: 79.99
})

if result.success?
  order_id = result.body["id"]
  puts "Order created: #{order_id}"
else
  puts "Failed: #{result.body["message"]}"
end
```

The body is serialized to JSON automatically. The `Content-Type: application/json` header is set if not already present.

---

5. PUT Requests

```
result = client.put("/orders/101", body: {
  status: "shipped",
  tracking_number: "1Z999AA10123456784"
})

puts result.success? ? "Updated" : "Failed: #{result.status}"
```

---

6. DELETE Requests

```
result = client.delete("/orders/101")

if result.success?
  puts "Order deleted"
else
  puts "Delete failed: #{result.status}"
end
```

---

7. Authentication Headers

Bearer Token

```
client = Tina4::Api.new("https://api.example.com", {
  "Authorization" => "Bearer #{ENV['API_TOKEN']}",
  "Content-Type"  => "application/json"
})
```

API Key Header

```
client = Tina4::Api.new("https://api.stripe.com", {
  "Authorization" => "Bearer #{ENV['STRIPE_SECRET_KEY']}",
  "Stripe-Version" => "2024-11-20"
})
```

Basic Auth

```
require "base64"

credentials = Base64.strict_encode64("#{ENV['API_USER']}:#{ENV['API_PASS']}")

client = Tina4::Api.new("https://api.example.com", {
  "Authorization" => "Basic #{credentials}"
})
```

Per-Request Header Override

Pass headers directly on a single call to override or extend the defaults:

```
result = client.get("/admin/users", headers: {
  "X-Admin-Token" => "secret-admin-key"
})
```

---

8. Timeouts

```
client = Tina4::Api.new("https://api.slow.com", {}, timeout: 10)
# Raises Tina4::ApiTimeout after 10 seconds with no response
```

Default timeout is 30 seconds.

---

9. Using the Client in Route Handlers

```
# @noauth
Tina4::Router.get("/api/weather") do |request, response|
  city = request.params["city"] || "London"

  weather_client = Tina4::Api.new("https://api.openweathermap.org", {
    "Accept" => "application/json"
  })

  result = weather_client.get("/data/2.5/weather", params: {
    q:     city,
    appid: ENV["OPENWEATHER_API_KEY"],
    units: "metric"
  })

  if result.success?
    data = result.body
    response.json({
      city:        data["name"],
      temperature: data.dig("main", "temp"),
      description: data.dig("weather", 0, "description")
    })
  else
    response.json({ error: "Weather data unavailable" }, 502)
  end
end
```

```
curl "http://localhost:7147/api/weather?city=Paris"
```

```
{
  "city": "Paris",
  "temperature": 14.3,
  "description": "partly cloudy"
}
```

---

10. Shared Client Instances

Define shared clients once and reuse them across routes.

```
# src/clients/stripe.rb
STRIPE = Tina4::Api.new("https://api.stripe.com", {
  "Authorization"  => "Bearer #{ENV['STRIPE_SECRET_KEY']}",
  "Content-Type"   => "application/json",
  "Stripe-Version" => "2024-11-20"
})

# src/routes/payments.rb
Tina4::Router.post("/api/checkout") do |request, response|
  body = request.body

  result = STRIPE.post("/v1/payment_intents", body: {
    amount:   (body["total"].to_f * 100).to_i,
    currency: "usd",
    metadata: { order_id: body["order_id"] }
  })
```

```
  if result.success?
    response.json({ client_secret: result.body["client_secret"] }, 201)
  else
    Tina4::Log.error("Stripe error", status: result.status, error: result.body["error"]["message
    response.json({ error: "Payment failed" }, 502)
  end
end
```

---

11. Error Handling

```
begin
  result = client.get("/products")

  case result.status
  when 200..299
    process(result.body)
  when 401
    raise "Unauthorized -- check API credentials"
  when 429
    Tina4::Log.warning("Rate limited", retry_after: result.headers["Retry-After"])
  when 500..599
    raise "Remote server error: #{result.status}"
  end

rescue Tina4::ApiTimeout => e
  Tina4::Log.error("API request timed out", error: e.message)
rescue => e
  Tina4::Log.error("API request failed", error: e.message)
end
```

---

12. Gotchas

1. Never hard-code credentials

Always read tokens from environment variables. Hard-coded credentials end up in version control.

2. Check success? before reading body

`result.body` on a 4xx or 5xx response contains the error payload, not the data you expect.

3. Timeout is per-request

A shared client instance with `timeout: 5` applies that timeout to every request. Override per call if some endpoints are slower than others.

4. Base URL trailing slash

`Tina4::Api.new("https://api.example.com/")` with a trailing slash and `client.get("/products")` with a leading slash double up to `https://api.example.com//products`. Use a base URL without a trailing slash.

GraphQL and SOAP

1. The Problem GraphQL Solves

Your mobile app needs a product list. Each product carries a name, price, 20 image URLs, a full description, 15 review objects, and 8 fields you do not need. REST sends all of it -- 50KB of JSON when you needed 2KB. On a mobile connection, that waste stings.

Now your web dashboard needs the same products plus category, stock status, and supplier info. REST forces you to make three requests and stitch them together, or build a custom endpoint with query parameter parsing.

GraphQL ends both problems. The client asks for the fields it needs. The server returns those fields. One endpoint. One request. One response shaped to fit.

Tina4 includes a built-in GraphQL engine. No external gems. No Apollo Server. No graphql-ruby gem. Part of the framework.

---

2. GraphQL vs REST -- A Quick Comparison

Aspect	REST	GraphQL
Endpoints	One per resource (`/products`, `/users`)	One endpoint (`/graphql`)
Data shape	Server decides	Client decides
Over-fetching	Common (get all fields)	Never (get only requested fields)
Under-fetching	Common (multiple requests needed)	Never (get related data in one query)
Versioning	`/api/v1/`, `/api/v2/`	Schema evolves, no versioning needed
Caching	HTTP caching works naturally	Requires client-side cache management
Learning curve	Low	Moderate

REST is still great for simple APIs. GraphQL shines when clients have diverse data needs -- mobile apps, dashboards, third-party integrations.

---

3. Enabling GraphQL

GraphQL is available by default in Tina4. Set up a schema and register the route in `src/routes/graphql.rb`:

```
schema = Tina4::GraphQLSchema.new
gql = Tina4::GraphQL.new(schema)
gql.register_route  # POST /graphql, GET /graphql (playground)
```

To change the endpoint path:

```
gql.register_route("/api/graphql")
```

To verify it is working, start the server and send a test query:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

```
{
  "data": {
    "__schema": {
      "queryType": {
        "name": "Query"
      }
    }
  }
}
```

If you see that response, GraphQL is running.

---

4. Defining Queries

Register queries on a `GraphQLSchema` instance. Each query has a name, return type, optional arguments, and a resolver block. The block receives three arguments: `root` (parent value), `args` (query arguments), and `ctx` (context hash).

Create `src/routes/graphql.rb`:

```
schema = Tina4::GraphQLSchema.new

# List all products
schema.add_query("products", type: "[Product]") do |root, args, ctx|
  db = Tina4.database
  products = db.fetch("SELECT * FROM products ORDER BY name")
  products
end

# Get a single product by ID
schema.add_query("product", type: "Product",
                 args: { "id" => { type: "Int!" } }) do |root, args, ctx|
  db = Tina4.database
  db.fetch_one("SELECT * FROM products WHERE id = ?", [args["id"].to_i])
end

gql = Tina4::GraphQL.new(schema)
gql.register_route
```

Testing the Queries

List all products:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price inStock } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "inStock": true},
      {"id": 2, "name": "USB-C Hub", "price": 49.99, "inStock": true},
      {"id": 3, "name": "Standing Desk", "price": 549.99, "inStock": false}
    ]
  }
}
```

The response contains only the four fields you asked for (`id`, `name`, `price`, `inStock`), not `category` or `createdAt`. That is GraphQL in action.

Get a single product with all fields:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 1) { id name category price inStock createdAt } }"}'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "category": "Electronics",
      "price": 79.99,
      "inStock": true,
      "createdAt": "2026-03-22 14:30:00"
    }
  }
}
```

Request a product that does not exist:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 999) { id name } }"}'
```

```
{
  "data": {
    "product": null
  }
}
```

---

5. Mutations

Mutations are how clients create, update, or delete data. Register them with `add_mutation` on the schema.

```
schema.add_mutation("createProduct", type: "Product",
```

```
                    args: {
                      "name"     => { type: "String!" },
                      "category" => { type: "String" },
                      "price"    => { type: "Float!" },
                      "inStock"  => { type: "Boolean" }
                    }) do |root, args, ctx|
  db = Tina4.database

  db.execute(
    "INSERT INTO products (name, category, price, in_stock) VALUES (:name, :category, :price, :i
    {
      name: args["name"],
      category: args["category"] || "Uncategorized",
      price: args["price"].to_f,
      in_stock: args["inStock"] ? 1 : 0
    }
  )

  db.fetch_one("SELECT * FROM products WHERE id = last_insert_rowid()")
end

schema.add_mutation("updateProduct", type: "Product",
                    args: {
                      "id"       => { type: "Int!" },
                      "name"     => { type: "String" },
                      "category" => { type: "String" },
                      "price"    => { type: "Float" },
                      "inStock"  => { type: "Boolean" }
                    }) do |root, args, ctx|
  db = Tina4.database
  id = args["id"].to_i

  sets = []
  params = []

  if args.key?("name")
    sets << "name = ?"
    params << args["name"]
  end
  if args.key?("category")
    sets << "category = ?"
    params << args["category"]
  end
  if args.key?("price")
    sets << "price = ?"
    params << args["price"].to_f
  end
  if args.key?("inStock")
    sets << "in_stock = ?"
    params << (args["inStock"] ? 1 : 0)
  end

  unless sets.empty?
    params << id
    db.execute("UPDATE products SET #{sets.join(', ')} WHERE id = ?", params)
  end

  db.fetch_one("SELECT * FROM products WHERE id = ?", [id])
end
```

```
schema.add_mutation("deleteProduct", type: "Boolean",
                    args: { "id" => { type: "Int!" } }) do |root, args, ctx|
  db = Tina4.database
  id = args["id"].to_i
  db.execute("DELETE FROM products WHERE id = ?", [id])
  true
end
```

Testing Mutations

Create a product:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { createProduct(name: \"Desk Lamp\", category: \"Office\", price: 39.99)
  }'
```

```
{
  "data": {
    "createProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 39.99
    }
  }
}
```

Update a product:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { updateProduct(id: 4, price: 44.99, inStock: false) { id name price inSt
  }'
```

```
{
  "data": {
    "updateProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 44.99,
      "inStock": false
    }
  }
}
```

Delete a product:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { deleteProduct(id: 4) }"
  }'
```

```
{
  "data": {
    "deleteProduct": true
  }
}
```

---

6. Nested Types and Relationships

The real power of GraphQL: traversing relationships in a single query. Authors, posts, comments -- all in one request.

```
schema.add_query("posts", type: "[Post]") do |root, args, ctx|
  db = Tina4.database
  posts = db.fetch("SELECT * FROM posts WHERE published = 1 ORDER BY created_at DESC")

  posts.each do |post|
    post["author"] = db.fetch_one(
      "SELECT id, name, email FROM users WHERE id = ?",
      [post["user_id"]]
    )
    post["comments"] = db.fetch(
      "SELECT * FROM comments WHERE post_id = ? ORDER BY created_at",
      [post["id"]]
    )
    post["commentCount"] = post["comments"].length
  end

  posts
end

schema.add_query("post", type: "Post",
                 args: { "id" => { type: "Int!" } }) do |root, args, ctx|
  db = Tina4.database
  post = db.fetch_one("SELECT * FROM posts WHERE id = ?", [args["id"].to_i])

  return nil if post.nil?

  post["author"] = db.fetch_one(
    "SELECT id, name, email FROM users WHERE id = ?",
    [post["user_id"]]
  )
  post["comments"] = db.fetch(
    "SELECT * FROM comments WHERE post_id = ? ORDER BY created_at",
    [post["id"]]
  )
  post["commentCount"] = post["comments"].length

  post
end

schema.add_query("user", type: "User",
                 args: { "id" => { type: "Int!" } }) do |root, args, ctx|
  db = Tina4.database
  user = db.fetch_one("SELECT id, name, email FROM users WHERE id = ?", [args["id"].to_i])

  return nil if user.nil?

  user["posts"] = db.fetch(
    "SELECT id, title FROM posts WHERE user_id = ?",
    [user["id"]]
  )
```

```
  user
end
```

Now clients can query across relationships in a single request:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "{ posts { id title author { name email } comments { authorName body } commentCount
  }'
```

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {
          "name": "Alice",
          "email": "alice@example.com"
        },
        "comments": [
          {"authorName": "Bob", "body": "Great post!"}
        ],
        "commentCount": 1
      }
    ]
  }
}
```

One request. The fields the client needs. No over-fetching. No under-fetching.

---

7. Auto-Generating Schema from ORM Models

If your ORM models exist, Tina4 can auto-generate GraphQL types and CRUD resolvers from them. Call `from_orm` on the schema:

```
schema = Tina4::GraphQLSchema.new
schema.from_orm(Product)
schema.from_orm(User)

gql = Tina4::GraphQL.new(schema)
gql.register_route
```

With `from_orm`, each model gets:

- A GraphQL type with fields matching the model properties
- A query to list all records (e.g., `products(limit: Int, offset: Int)`)
- A query to get one by ID (e.g., `product(id: ID!)`)
- A mutation to create (e.g., `createProduct(input: ProductInput!)`)
- A mutation to update (e.g., `updateProduct(id: ID!, input: ProductInput!)`)
- A mutation to delete (e.g., `deleteProduct(id: ID!)`)

The `from_orm` method reads field definitions from your model class. It maps Ruby types to GraphQL types:

Ruby field type	GraphQL type
`:integer`, `:int`	`Int`
`:float`, `:double`, `:decimal`	`Float`
`:boolean`, `:bool`	`Boolean`
`:string`, `:text`, `:varchar`	`String`
`:datetime`, `:date`, `:timestamp`	`String`

You can still add custom queries and mutations alongside auto-generated ones. Custom resolvers always take precedence over auto-generated ones.

Example: ORM + Custom Queries

```
schema = Tina4::GraphQLSchema.new

# Auto-generate CRUD for Product
schema.from_orm(Product)

# Add a custom query that filters by category
schema.add_query("productsByCategory", type: "[Product]",
                 args: { "category" => { type: "String!" } }) do |root, args, ctx|
  products, count = Product.where("category = ?", [args["category"]])
  products.map(&:to_hash)
end

gql = Tina4::GraphQL.new(schema)
gql.register_route
```

---

8. Authentication in GraphQL

The resolver block receives a `ctx` hash that includes the current request. Use it to check authentication:

```
schema.add_query("me", type: "User") do |root, args, ctx|
  request = ctx[:request]

  if request.nil? || request.headers["Authorization"].nil?
    raise "Authentication required"
  end

  token = request.headers["Authorization"].sub("Bearer ", "")
  payload = Tina4::Auth.valid_token(token)

  raise "Invalid token" if payload.nil?

  db = Tina4.database
  db.fetch_one("SELECT id, name, email, role FROM users WHERE id = ?", [payload["user_id"]])
end
```

The `ctx` hash is populated by `register_route`, which passes `{ request: request }` to every resolver.

---

9. The GraphiQL Playground

When you call `register_route`, Tina4 serves a GraphiQL interactive playground at the same endpoint via GET:

```
http://localhost:7147/graphql
```

GraphiQL gives you:

- A query editor with syntax highlighting and auto-completion
- A documentation explorer showing all types, queries, and mutations
- A results panel showing the response
- Query history

Type a query on the left, click the play button, and see the results on the right. The documentation explorer on the right sidebar shows every type, field, and argument available in your schema.

In production, disable the playground by using a custom route instead of `register_route`:

```
Tina4.post "/graphql", auth: false do |request, response|
  result = gql.handle_request(request.body, context: { request: request })
  response.json(result)
end
```

---

10. Query Variables

For production use, clients should send query variables separately from the query string. This prevents injection and allows query caching.

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query GetProduct($id: Int!) { product(id: $id) { id name price } }",
    "variables": {"id": 1}
  }'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "price": 79.99
    }
  }
}
```

The query uses `$id` as a placeholder. The actual value comes from the `variables` object. This is safer and more efficient than string interpolation.

Variable definitions sit in the operation signature:

```
query GetProduct($id: Int!) {
  product(id: $id) {
    id
    name
    price
  }
}
```

The `$id: Int!` declaration tells GraphQL the variable name, type, and that it is required (the `!`). The executor substitutes `$id` with the value from the `variables` hash before calling the resolver.

You can define multiple variables:

```
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query Search($category: String!, $limit: Int) { productsByCategory(category: $cate
    "variables": {"category": "Electronics", "limit": 10}
  }'
```

---

11. Exercise: Build a GraphQL API for a Blog

Build a complete GraphQL API for a blog with posts, authors, and comments.

Requirements

- **Schema** -- Define queries and mutations for `User`, `Post`, and `Comment` with these fields:
- `User`: id, name, email, posts (nested)
- `Post`: id, title, body, published, author (nested), comments (nested), commentCount (computed)
- `Comment`: id, authorName, body, createdAt
- **Queries**:
- `posts` -- List all published posts
- `post(id: Int!)` -- Get a single post by ID
- `user(id: Int!)` -- Get a user with their posts
- **Mutations**:
- `createPost(userId: Int!, title: String!, body: String!, published: Boolean)` -- Create a new post
- `addComment(postId: Int!, authorName: String!, body: String!)` -- Add a comment to a post
- Use the ORM models from Chapter 6 (User, Post, Comment).

Test with these queries:

```
# List published posts with author names
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ posts { id title author { name } commentCount } }"}'

# Get a specific post with all comments
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ post(id: 1) { title body author { name email } comments { authorName body } }

# Create a post
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createPost(userId: 1, title: \"GraphQL is great\", body: \"Here is w

# Add a comment
curl -X POST http://localhost:7147/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { addComment(postId: 1, authorName: \"Carol\", body: \"Nice article!\"
```

---

12. Solution

Create `src/routes/graphql_blog.rb`:

```
schema = Tina4::GraphQLSchema.new

# ■■ Queries ■■

schema.add_query("posts", type: "[Post]") do |root, args, ctx|
  db = Tina4.database
  posts = db.fetch("SELECT * FROM posts WHERE published = 1 ORDER BY created_at DESC")

  posts.each do |post|
    post["author"] = db.fetch_one(
      "SELECT id, name, email FROM users WHERE id = ?",
      [post["user_id"]]
    )
    post["comments"] = db.fetch(
      "SELECT * FROM comments WHERE post_id = ? ORDER BY created_at",
      [post["id"]]
    )
    post["commentCount"] = post["comments"].length
  end

  posts
end

schema.add_query("post", type: "Post",
                 args: { "id" => { type: "Int!" } }) do |root, args, ctx|
  db = Tina4.database
  post = db.fetch_one("SELECT * FROM posts WHERE id = ?", [args["id"].to_i])

  return nil if post.nil?

  post["author"] = db.fetch_one(
```

```
    "SELECT id, name, email FROM users WHERE id = ?",
    [post["user_id"]]
  )
  post["comments"] = db.fetch(
    "SELECT * FROM comments WHERE post_id = ? ORDER BY created_at",
    [post["id"]]
  )
  post["commentCount"] = post["comments"].length

  post
end

schema.add_query("user", type: "User",
                 args: { "id" => { type: "Int!" } }) do |root, args, ctx|
  db = Tina4.database
  user = db.fetch_one(
    "SELECT id, name, email FROM users WHERE id = ?",
    [args["id"].to_i]
  )

  return nil if user.nil?

  user["posts"] = db.fetch(
    "SELECT id, title FROM posts WHERE user_id = ?",
    [user["id"]]
  )

  user
end

# ■■ Mutations ■■

schema.add_mutation("createPost", type: "Post",
                    args: {
                      "userId"    => { type: "Int!" },
                      "title"     => { type: "String!" },
                      "body"      => { type: "String!" },
                      "published" => { type: "Boolean" }
                    }) do |root, args, ctx|
  db = Tina4.database

  user = db.fetch_one("SELECT id FROM users WHERE id = ?", [args["userId"].to_i])
  raise "User not found" if user.nil?

  db.execute(
    "INSERT INTO posts (user_id, title, body, published) VALUES (:user_id, :title, :body, :publi
    {
      user_id: args["userId"].to_i,
      title: args["title"],
      body: args["body"],
      published: args["published"] ? 1 : 0
    }
  )

  db.fetch_one("SELECT * FROM posts WHERE id = last_insert_rowid()")
end

schema.add_mutation("addComment", type: "Comment",
                    args: {
```

```
                    "postId"     => { type: "Int!" },
                    "authorName" => { type: "String!" },
                    "body"       => { type: "String!" }
                  }) do |root, args, ctx|
  db = Tina4.database

  post = db.fetch_one("SELECT id FROM posts WHERE id = ?", [args["postId"].to_i])
  raise "Post not found" if post.nil?

  db.execute(
    "INSERT INTO comments (post_id, author_name, body) VALUES (:post_id, :author_name, :body)",
    {
      post_id: args["postId"].to_i,
      author_name: args["authorName"],
      body: args["body"]
    }
  )

  db.fetch_one("SELECT * FROM comments WHERE id = last_insert_rowid()")
end

gql = Tina4::GraphQL.new(schema)
gql.register_route
```

Expected output for `{ posts { id title author { name } commentCount } }`:

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {"name": "Alice"},
        "commentCount": 1
      },
      {
        "id": 2,
        "title": "GraphQL is great",
        "author": {"name": "Alice"},
        "commentCount": 0
      }
    ]
  }
}
```

---

13. Input Validation

Tina4's GraphQL engine validates every argument against its declared type before your resolver runs. You never need to check whether a required argument is present or whether a string arrived where an integer was expected -- the engine rejects the query before your code executes.

The built-in validator covers:

- **Non-null enforcement** -- Arguments marked with `!` must be present and non-null.

• **Scalar type checking** -- `Int`, `Float`, `String`, `Boolean`, and `ID` values are verified against their declared type.

• **Type coercion** -- When safe, the engine coerces compatible values automatically (e.g., an integer `42` passed to a `Float` field becomes `42.0`, or a string `"7"` passed to an `Int` field becomes `7`).

• **List item validation** -- For `[Int!]!` arguments, the engine checks that the value is a list, that no item is null, and that every item is an integer.

Example: Missing Required Argument

Suppose you define a query that requires an `id`:

```
schema.add_query("user", type: "User",
                 args: { "id" => { type: "ID!" } }) do |root, args, ctx|
  db = Tina4.database
  db.fetch_one("SELECT * FROM users WHERE id = ?", [args["id"].to_i])
end
```

Sending a query without the required argument:

```
{
  user {
    name
    email
  }
}
```

Returns an error before the resolver runs:

```
{
  "errors": [
    {
      "message": "Argument 'id' of type 'ID!' is required but not provided.",
      "locations": [{"line": 2, "column": 3}]
    }
  ]
}
```

Your resolver never executes. No nil-check needed inside the block.

---

14. Field-Level Auth Directives

Tina4 supports three directives that control field-level access in your GraphQL schema:

Directive	Effect
`@auth`	Field resolves only when `ctx["user"]` is present (any authenticated user)
`@role(role: "admin")`	Field resolves only when `ctx["user"]["role"]` matches the specified role

`@guest`	Field resolves only when `ctx["user"]` is absent (unauthenticated visitors)

Passing Auth Context

Pass user information through the context parameter of `execute()`:

```
result = gql.execute(query, variables: {}, context: {
  "user" => { "id" => 1, "role" => "admin" }
})
```

When no user is logged in, pass an empty context or omit the `user` key:

```
result = gql.execute(query, variables: {}, context: {})
```

Schema Example

```
type User {
    id: ID!
    name: String!
    email: String! @auth
    role: String! @role(role: "admin")
}

type Query {
    me: User @auth
    publicStats: Stats @guest
    users: [User!]! @role(role: "admin")
}
```

Behavior on Failed Auth

When a directive check fails, the field resolves to `null` silently -- no error is added to the response. The rest of the query executes normally. This prevents leaking information about which fields exist behind authentication.

For example, if a non-admin user queries:

```
{
  users {
    name
    email
    role
  }
}
```

The response is:

```
{
  "data": {
    "users": null
  }
}
```

No error message. No hint that the field requires admin access. The field is simply excluded.

---

15. Gotchas

1. Resolver Not Called

Problem: A query returns `null` even though data exists.

Cause: The query name in `add_query` does not match the name the client sends. GraphQL is case-sensitive.

Fix: Verify that `schema.add_query("products", ...)` matches `{ products { ... } }` in the client query. Check capitalization.

2. Nested Data Returns Wrong Shape

Problem: `post.author` returns the entire post hash instead of the user.

Cause: You attached the nested data to a key that does not match the field name the client requests.

Fix: Use the exact field name the client will query. If the client sends `{ post { author { name } } }`, the hash must have an `"author"` key:

```
post["author"] = db.fetch_one("SELECT id, name FROM users WHERE id = ?", [post["user_id"]])
```

3. Mutation Returns Null

Problem: A mutation executes but returns null.

Cause: `db.execute` returns affected row count, not the data. Your mutation block ends with `db.execute`, so the block returns an integer.

Fix: Follow every `db.execute` with a `db.fetch_one` to return the actual record:

```
db.execute("INSERT INTO products ...")
db.fetch_one("SELECT * FROM products WHERE id = last_insert_rowid()")
```

4. N+1 Query Problem

Problem: Loading 50 posts with their authors generates 51 database queries (1 for posts + 50 for authors).

Cause: Each nested lookup runs inside a loop. When you resolve `author` for each post, that is one query per post.

Fix: Pre-fetch all related records in the parent resolver. Check `ctx["__selections"]` to see which nested fields the client requested -- if `author` is not in the selection set, skip the join. For ORM queries, use the `include` parameter to eager-load relationships in a single query. For manual queries, load all relevant users in one query, build a hash by ID, and attach them:

```
schema.add_query("posts", type: "[Post]") do |root, args, ctx|
  db = Tina4.database
  posts = db.fetch("SELECT * FROM posts ORDER BY created_at DESC")

  user_ids = posts.map { |p| p["user_id"] }.uniq
  users = db.fetch("SELECT * FROM users WHERE id IN (#{user_ids.join(',')})")
  user_map = users.each_with_object({}) { |u, h| h[u["id"]] = u }
```

```
  posts.each { |p| p["author"] = user_map[p["user_id"]] }
  posts
end
```

5. GraphQL Playground Returns 404

Problem: Visiting `/graphql` in the browser returns a 404.

Cause: You did not call `register_route`, or you registered a POST-only route manually.

Fix: Call `gql.register_route` which registers both POST (for queries) and GET (for the GraphiQL playground).

6. Type Mismatch Between Schema and Data

Problem: A field declared as `Int` receives a string from the database, causing unexpected results.

Cause: SQLite returns all values as strings. Your resolver passes them through without conversion.

Fix: The input validation layer (see section 13) now coerces compatible types automatically -- a string `"42"` passed as an argument to an `Int` parameter is converted before your resolver runs. However, resolver *return values* still need correct types. Cast values explicitly:

```
{
  "id" => record["id"].to_i,
  "price" => record["price"].to_f,
  "inStock" => record["in_stock"] == 1
}
```

The `to_hash` method on ORM models handles this for model properties with type declarations. Raw database hashes need manual casting.

7. Authentication Not Applied

Problem: GraphQL queries work without authentication.

Cause: `register_route` sets `auth: false` by default. No middleware checks the token.

Fix: Check `ctx[:request]` in resolvers that need authentication. Or register a custom POST route with `auth: true` and let your auth middleware run first:

```
Tina4.post "/graphql", auth: true do |request, response|
  result = gql.handle_request(request.body, context: { request: request })
  response.json(result)
end
```

8. Error Messages Expose Internal Details

Problem: Database error messages appear in GraphQL responses.

Cause: Exceptions raised in resolvers become error messages in the response.

Fix: Wrap resolver logic in `begin/rescue` and return clean error messages:

```
schema.add_query("product", type: "Product",
                 args: { "id" => { type: "Int!" } }) do |root, args, ctx|
  begin
```

```
    db = Tina4.database
    db.fetch_one("SELECT * FROM products WHERE id = ?", [args["id"].to_i])
  rescue => e
    raise "Product not found"
  end
end
```

---

14. SOAP / WSDL Services

What is SOAP?

SOAP (Simple Object Access Protocol) is an XML-based messaging protocol for exchanging structured data between systems. It predates REST and GraphQL, but remains common in enterprise integrations -- banking, healthcare, government, and ERP systems all rely on SOAP services.

A WSDL (Web Services Description Language) file describes a SOAP service: what operations are available, what parameters they accept, and what they return. Clients use the WSDL to auto-generate code for calling the service.

Tina4 includes a zero-dependency SOAP 1.1 server that generates WSDL definitions from Ruby classes and type annotations. No XML authoring required. It uses REXML (part of Ruby's standard library) for XML parsing.

Defining a SOAP Service

Create a class that extends `Tina4::WSDL`. Mark each method with `wsdl_operation` before the `def`. The `output` hash describes the response fields and their types.

```
class Calculator < Tina4::WSDL
  wsdl_operation output: { Result: :int }
  def add(a, b)
    { Result: a.to_i + b.to_i }
  end

  wsdl_operation output: { Result: :int }
  def multiply(a, b)
    { Result: a.to_i * b.to_i }
  end
end

Tina4::Router.get("/calculator") do |request, response|
  service = Calculator.new(request)
  response.call(service.handle)
end

Tina4::Router.post("/calculator") do |request, response|
  service = Calculator.new(request)
  response.call(service.handle)
end
```

That is the entire service. The `handle` method inspects the request:

- **GET** (or `?wsdl` query parameter) -- returns the auto-generated WSDL definition
- **POST** with SOAP XML body -- parses the XML, finds the operation, converts parameters, calls the method, and returns a SOAP XML response

Type Annotations Map to XSD

The `output` hash maps response element names to Ruby type symbols. Input parameter types default to `:string` and are converted based on the output type declarations.

Ruby type	XSD type
`:string`, `String`	`xsd:string`
`:int`, `:integer`, `Integer`	`xsd:int`
`:float`, `:double`, `Float`	`xsd:double`
`:boolean`, `:bool`	`xsd:boolean`
`:date`	`xsd:date`
`:datetime`	`xsd:dateTime`
`:base64`	`xsd:base64Binary`

A More Complete Example

```
class UserService < Tina4::WSDL
  wsdl_operation output: { Name: :string, Email: :string, Active: :boolean }
  def get_user(user_id)
    user = User.find(user_id.to_i)
    if user
      {
        Name: user.name,
        Email: user.email,
        Active: user.active == 1
      }
    else
      { Name: "", Email: "", Active: false }
    end
  end

  wsdl_operation output: { Total: :int, Average: :float }
  def sum_list(numbers)
    nums = numbers.split(",").map(&:to_i)
    {
      Total: nums.sum,
      Average: nums.sum.to_f / nums.length
    }
  end
end

Tina4::Router.get("/api/users/soap") do |request, response|
  service = UserService.new(request)
  response.call(service.handle)
end

Tina4::Router.post("/api/users/soap") do |request, response|
  service = UserService.new(request)
  response.call(service.handle)
end
```

Lifecycle Hooks

Override `on_request` and `on_result` to add validation, logging, or transformation around every operation call.

```
class AuditedService < Tina4::WSDL
  def on_request(request)
    puts "SOAP request from #{request.headers['x-forwarded-for'] || 'unknown'}"
  end

  def on_result(result)
    result[:Timestamp] = Time.now.iso8601
    result
  end

  wsdl_operation output: { Status: :string, Timestamp: :string }
  def ping
    { Status: "ok" }
  end
end
```

Testing with curl

Fetch the WSDL definition:

```
curl http://localhost:7147/calculator?wsdl
```

Call the add operation with a SOAP request:

```
curl -X POST http://localhost:7147/calculator \
  -H "Content-Type: text/xml" \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <add>
      <a>5</a>
      <b>3</b>
    </add>
  </soap:Body>
</soap:Envelope>'
```

Response:

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
<soap:Body>
<addResponse>
<Result>8</Result>
</addResponse>
</soap:Body>
</soap:Envelope>
```

If the operation name is wrong or the XML is malformed, the service returns a SOAP fault:

```
<soap:Fault>
  <faultcode>Client</faultcode>
  <faultstring>Unknown operation: subtract</faultstring>
</soap:Fault>
```

When to Use SOAP vs REST vs GraphQL

Scenario	Use
New API for web/mobile clients	REST or GraphQL
Integrating with legacy enterprise systems (SAP, banks, government)	SOAP
Clients require a machine-readable service contract	SOAP (WSDL)
Simple internal microservices	REST
Clients with diverse data needs	GraphQL

SOAP is rarely the right choice for new APIs. When you need to expose a service that legacy systems can consume, Tina4 makes it straightforward -- define a class, annotate your types, and the framework handles the XML.

Real-time with WebSocket

1. The Refresh Button Problem

Your project management app needs live updates. Someone moves a card from "In Progress" to "Done." Everyone else should see it -- no page refresh, no polling, no waiting. But HTTP is request-response. The client asks. The server answers. The server cannot speak first.

WebSocket tears down that wall. It establishes a persistent, bi-directional connection between browser and server. Either side can send messages at any time. The connection holds until one side closes it.

Tina4 treats WebSocket as routing. Define a WebSocket handler the same way you define an HTTP route. The path determines which handler owns the connection.

---

2. What WebSocket Is

HTTP works like this:

```
Client: "Give me /api/products"
Server: "Here are the products" (connection closes)
```

WebSocket works like this:

```
Client: "I want to upgrade to WebSocket on /ws/chat"
Server: "Upgrade accepted. Connection open."
Client: "Hello everyone!"
Server: "Alice says: Hello everyone!"  (pushed to all clients)
Server: "Bob joined the chat"  (pushed to all clients, at any time)
```

Key differences:

- **Persistent**: The connection stays open. No repeated handshakes.
- **Bi-directional**: The server can push data without the client asking.
- **Low overhead**: After the initial handshake, messages are tiny.
- **Real-time**: Messages arrive within milliseconds.

---

3. Router.websocket -- WebSocket as a Route

In Tina4, you define WebSocket handlers using `Tina4::Router.websocket`:

```
Tina4::Router.websocket "/ws/echo" do |connection, event, data|
  if event == :message
    connection.send("Echo: #{data}")
  end
end
```

This is the simplest WebSocket handler: it receives a message and sends it back with "Echo: " prepended. The handler receives three arguments:

- **connection**: The WebSocket connection object. Use it to send messages, broadcast, or close the connection.
- **event**: The event type as a symbol: `:open`, `:message`, or `:close`.
- **data**: The message data (only present for `:message` events).

Shorthand Syntax

The same handler can be written with the shorthand `Tina4.websocket`:

```
Tina4.websocket "/ws/echo" do |connection, event, data|
  if event == :message
    connection.send("Echo: #{data}")
  end
end
```

Both forms are identical. Use whichever reads better in your project.

Starting the Server

WebSocket runs alongside your HTTP server. It works with both Puma and WEBrick:

```
tina4 serve

  Tina4 Ruby v3.0.0
  HTTP server running at http://0.0.0.0:7147
  WebSocket server running at ws://0.0.0.0:7147
  Press Ctrl+C to stop
```

---

4. Connection Events

There are three events, passed as symbols:

Event	When it fires	argument
`:open`	A client connects	`nil`
`:message`	A client sends a message	The message string
`:close`	A client disconnects	`nil`

A Complete Handler

```
Tina4::Router.websocket "/ws/chat" do |connection, event, data|
  case event
  when :open
    $stderr.puts "[Chat] New connection: #{connection.id}"
    connection.send(JSON.generate({
      type: "system",
      message: "Welcome to the chat!",
      your_id: connection.id
    }))

  when :message
    message = JSON.parse(data)
    $stderr.puts "[Chat] #{connection.id}: #{message['text'] || data}"

    connection.send(JSON.generate({
      type: "message",
      from: connection.id,
      text: message["text"] || data,
      timestamp: Time.now.iso8601
    }))

  when :close
    $stderr.puts "[Chat] Disconnected: #{connection.id}"
  end
end
```

---

5. Connection Methods

The `connection` object provides three methods for communication:

Method	What it does
`connection.send(message)`	Send a message to this client only
`connection.broadcast(message)`	Send a message to all clients on the same path
`connection.close()`	Close this client's connection

Sending to a Single Client

`connection.send` sends a message to the specific client that triggered the event:

```
Tina4::Router.websocket "/ws/private" do |connection, event, data|
  if event == :message
    message = JSON.parse(data)
    action = message["action"] || ""

    if action == "get-time"
      connection.send(JSON.generate({
        type: "time",
        server_time: Time.now.iso8601
      }))
```

```
    end

    if action == "get-status"
      connection.send(JSON.generate({
        type: "status",
        uptime: 3600,
        connections: 42,
        memory_mb: (ObjectSpace.memsize_of_all / 1024.0 / 1024.0).round(2)
      }))
    end
  end
end
```

Closing a Connection

`connection.close` terminates the client's connection from the server side. This triggers the `:close` event:

```
Tina4::Router.websocket "/ws/secure" do |connection, event, data|
  case event
  when :open
    token = connection.params["token"]
    unless valid_token?(token)
      connection.send(JSON.generate({ error: "Invalid token" }))
      connection.close
    end
  when :message
    connection.broadcast(data)
  when :close
    $stderr.puts "Client disconnected"
  end
end
```

---

6. Broadcasting to All Clients

`connection.broadcast` sends a message to every client connected to the same WebSocket path. Broadcast is **path-scoped** -- clients on `/ws/chat/room-1` never receive broadcasts from `/ws/chat/room-2`:

```
Tina4::Router.websocket "/ws/announcements" do |connection, event, data|
  case event
  when :open
    connection.broadcast(JSON.generate({
      type: "system",
      message: "A new user joined",
      online_count: connection.connection_count
    }))

  when :message
    message = JSON.parse(data)

    connection.broadcast(JSON.generate({
      type: "announcement",
      from: connection.id,
      text: message["text"] || "",
      timestamp: Time.now.iso8601
```

```
    }))

  when :close
    connection.broadcast(JSON.generate({
      type: "system",
      message: "A user left",
      online_count: connection.connection_count
    }))
  end
end
```

Broadcast Excluding Sender

```
when :message
  message = JSON.parse(data)

  # Send to sender (confirmation)
  connection.send(JSON.generate({ type: "sent", text: message["text"] }))

  # Send to everyone else
  connection.broadcast(JSON.generate({
    type: "message",
    from: message["username"] || "Anonymous",
    text: message["text"],
    timestamp: Time.now.iso8601
  }), true)  # true = exclude sender
```

---

7. Path Parameters and Scoped Isolation

WebSocket paths support the same `{param}` syntax as HTTP routes. Access them with `connection.params["param_name"]`. Different resolved paths are completely isolated:

```
Tina4::Router.websocket "/ws/chat/{room}" do |connection, event, data|
  room = connection.params["room"]

  case event
  when :open
    $stderr.puts "[Room #{room}] New connection: #{connection.id}"
    connection.broadcast(JSON.generate({
      type: "system",
      message: "Someone joined room #{room}",
      room: room,
      online: connection.connection_count
    }))

  when :message
    message = JSON.parse(data)
    connection.broadcast(JSON.generate({
      type: "message",
      room: room,
      from: message["username"] || "Anonymous",
      text: message["text"] || "",
      timestamp: Time.now.iso8601
    }))

  when :close
    connection.broadcast(JSON.generate({
```

```
      type: "system",
      message: "Someone left room #{room}",
      room: room,
      online: connection.connection_count
    }))
  end
end
```

A client connecting to `/ws/chat/ruby` only sees messages broadcast on `/ws/chat/ruby`. A client on `/ws/chat/python` is in a completely separate space.

---

8. Building a Live Chat

WebSocket Handler

Create `src/routes/chat_ws.rb`:

```
$chat_users = {}

Tina4::Router.websocket "/ws/livechat/{room}" do |connection, event, data|
  room = connection.params["room"]

  case event
  when :open
    $chat_users[connection.id] = {
      id: connection.id,
      username: "Anonymous",
      room: room,
      joined_at: Time.now.iso8601
    }

    connection.send(JSON.generate({
      type: "welcome",
      message: "Connected to room: #{room}",
      your_id: connection.id,
      online: connection.connection_count
    }))

  when :message
    message = JSON.parse(data)
    type = message["type"] || "message"

    if type == "set-username"
      old_name = $chat_users[connection.id][:username]
      $chat_users[connection.id][:username] = message["username"]

      connection.broadcast(JSON.generate({
        type: "system",
        message: "#{old_name} is now known as #{message['username']}"
      }))
    end

    if type == "message"
      username = $chat_users[connection.id][:username]

      connection.broadcast(JSON.generate({
```

```
        type: "message",
        from: username,
        from_id: connection.id,
        text: message["text"] || "",
        timestamp: Time.now.iso8601
      }))
    end

    if type == "typing"
      username = $chat_users[connection.id][:username]

      connection.broadcast(JSON.generate({
        type: "typing",
        username: username
      }), true)  # Exclude sender
    end

  when :close
    username = $chat_users.dig(connection.id, :username) || "Unknown"
    $chat_users.delete(connection.id)

    connection.broadcast(JSON.generate({
      type: "system",
      message: "#{username} left the chat",
      online: connection.connection_count
    }))
  end
end
```

---

9. Live Notifications

WebSocket is great for pushing notifications to users in real time:

```
Tina4::Router.websocket "/ws/notifications/{user_id}" do |connection, event, data|
  user_id = connection.params["user_id"]

  case event
  when :open
    $stderr.puts "[Notifications] User #{user_id} connected"
    connection.send(JSON.generate({
      type: "connected",
      message: "Listening for notifications"
    }))
  when :message
    # Handle client acknowledgments if needed
  when :close
    $stderr.puts "[Notifications] User #{user_id} disconnected"
  end
end

# HTTP endpoint that triggers a notification
Tina4::Router.post("/api/orders/{order_id:int}/ship") do |request, response|
  order_id = request.params["order_id"]
  user_id = request.body["user_id"] || 0

  Tina4::Router.push_to_websocket("/ws/notifications/#{user_id}", JSON.generate({
    type: "notification",
```

```
    title: "Order Shipped",
    message: "Your order ##{order_id} has been shipped!",
    action_url: "/orders/#{order_id}",
    timestamp: Time.now.iso8601
  }))

  response.json({ message: "Order shipped, user notified" })
end
```

---

10. Connecting from JavaScript

Using Plain WebSocket

The browser's built-in `WebSocket` API is all you need:

```
<script>
    const ws = new WebSocket("ws://" + location.host + "/ws/chat/general");

    ws.onopen = function () {
        console.log("Connected to chat");
    };

    ws.onmessage = function (event) {
        const message = JSON.parse(event.data);
        console.log("Received:", message);
    };

    ws.onclose = function () {
        console.log("Disconnected from chat");
    };

    function sendMessage(text) {
        ws.send(JSON.stringify({ type: "message", text: text }));
    }
</script>
```

Using Frond.js

Tina4's Frond helper wraps WebSocket with convenience features:

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected to chat");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);
    });

    ws.on("close", function () {
        console.log("Disconnected from chat");
    });
```

```
    function sendMessage(text) {
        ws.send(JSON.stringify({ type: "message", text: text }));
    }
</script>
```

Auto-Reconnect

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,
    reconnectInterval: 3000,
    maxReconnectAttempts: 10
});
```

---

11. A Complete Chat Page

Create `src/templates/chat.html`:

```
{% extends "base.html" %}

{% block title %}Chat - {{ room }}{% endblock %}

{% block content %}
    <h1>Chat Room: {{ room }}</h1>
    <p id="status">Connecting...</p>
    <p id="online">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: scroll; padding

    <form id="chat-form" style="display: flex; gap: 8px;">
        <input type="text" id="message-input" placeholder="Type a message..."
               style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        <button type="submit" style="padding: 8px 16px; background: #333; color: white; border:
    </form>

    <script>
        const room = "{{ room }}";
        const ws = new WebSocket("ws://" + location.host + "/ws/livechat/" + room);
        const messagesDiv = document.getElementById("messages");
        let username = prompt("Enter your username:") || "Anonymous";

        ws.onopen = function () {
            document.getElementById("status").textContent = "Connected";
            ws.send(JSON.stringify({ type: "set-username", username: username }));
        };

        ws.onmessage = function (event) {
            const msg = JSON.parse(event.data);
            const div = document.createElement("div");
            div.style.marginBottom = "8px";

            if (msg.type === "message") {
                div.innerHTML = "<strong>" + msg.from + ":</strong> " + msg.text;
            } else if (msg.type === "system") {
                div.style.cssText = "color: #888; font-style: italic;";
                div.textContent = msg.message;
            }
```

```
            messagesDiv.appendChild(div);
            messagesDiv.scrollTop = messagesDiv.scrollHeight;

            if (msg.online !== undefined) {
                document.getElementById("online").textContent = "Online: " + msg.online;
            }
        };

        ws.onclose = function () {
            document.getElementById("status").textContent = "Disconnected";
        };

        document.getElementById("chat-form").addEventListener("submit", function (e) {
            e.preventDefault();
            const input = document.getElementById("message-input");
            if (input.value.trim()) {
                ws.send(JSON.stringify({ type: "message", text: input.value }));
                input.value = "";
            }
        });
    </script>
{% endblock %}
```

Create the route:

```
Tina4::Router.get("/chat/{room}") do |request, response|
  room = request.params["room"]
  response.render("chat.html", { room: room })
end
```

---

12. Server Compatibility

WebSocket works with both Puma and WEBrick. Tina4 detects the available server and sets up WebSocket handling automatically:

Server	Notes
Puma	Recommended for production. Handles concurrent connections efficiently.
WEBrick	Ships with Ruby. Good for development and testing.

For production deployments behind Nginx, configure WebSocket proxying:

```
location /ws/ {
    proxy_pass http://127.0.0.1:7147;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

---

13. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat room with username support, message broadcasting, and join/leave notifications.

Requirements

- WebSocket endpoint at `/ws/room/{room_name}` that handles `:open`, `:message`, and `:close` events
- HTTP endpoint at `GET /room/{room_name}` that serves an HTML chat page
- The chat page should prompt for a username, display messages in real time, and show online count

Test by:

- Open `http://localhost:7147/room/test` in two browser tabs
- Set different usernames
- Send messages from both tabs and verify they appear in both
- Close one tab and verify the "user left" message appears

---

14. Solution

Create `src/routes/chat_room.rb`:

```
$room_users = {}

Tina4::Router.websocket "/ws/room/{room_name}" do |connection, event, data|
  room = connection.params["room_name"]
  key = "#{room}:#{connection.id}"

  case event
  when :open
    $room_users[key] = "Anonymous"

    connection.send(JSON.generate({
      type: "system",
      message: "Welcome to room: #{room}",
      online: connection.connection_count
    }))

    connection.broadcast(JSON.generate({
      type: "system",
      message: "A new user joined",
      online: connection.connection_count
    }), true)

  when :message
    msg = JSON.parse(data)
    type = msg["type"] || "chat"

    if type == "set-name"
      old_name = $room_users[key] || "Anonymous"
```

```
      new_name = msg["name"] || "Anonymous"
      $room_users[key] = new_name

      connection.broadcast(JSON.generate({
        type: "system",
        message: "#{old_name} changed their name to #{new_name}"
      }))
    end

    if type == "chat"
      username = $room_users[key] || "Anonymous"

      connection.broadcast(JSON.generate({
        type: "chat",
        from: username,
        text: msg["text"] || "",
        timestamp: Time.now.strftime("%H:%M:%S")
      }))
    end

  when :close
    username = $room_users[key] || "Anonymous"
    $room_users.delete(key)

    connection.broadcast(JSON.generate({
      type: "system",
      message: "#{username} left the room",
      online: connection.connection_count
    }))
  end
end

Tina4::Router.get("/room/{room_name}") do |request, response|
  room = request.params["room_name"]
  response.render("room.html", { room: room })
end
```

---

15. Scaling with a Backplane

When you run a single server instance, `broadcast` reaches every connected client. But in production you often run multiple instances behind a load balancer. Each instance only knows about its own connections. A message broadcast on instance A never reaches clients connected to instance B.

A backplane solves this. It relays WebSocket messages across all instances using the shared `tina4:ws` pub/sub channel. Tina4 supports Redis and NATS adapters. The file-free local path remains the default; a network backplane needs its broker and the matching client gem.

Configuration

Set two environment variables in your `.env`:

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
```

For NATS:

```
TINA4_WS_BACKPLANE=nats
TINA4_WS_BACKPLANE_URL=nats://localhost:4222
```

Every `broadcast` call delivers to local connections first, then publishes an envelope to the backplane. Sibling instances discard the sender's echo and relay the message to their own local connections. Your WebSocket routes do not change.

Requirements

Install only the client for the adapter your application selects:

```
bundle add redis
bundle add nats-pure
```

These are application dependencies, not Tina4 core dependencies. Tina4 requires them only when it starts the selected adapter. An explicit selection is a deployment contract: an unknown adapter, missing gem, or unreachable broker fails startup with `Configured WebSocket backplane failed`. Tina4 never claims distributed delivery while running local-only.

When to Choose a Backplane

Deployment	Choice
One Tina4 process or container	Leave `TINA4_WS_BACKPLANE` unset
Multiple instances and Redis is already operated	`redis`
Multiple instances and NATS is already operated	`nats`
Unsure which broker to introduce	Redis is usually the simpler starting point

The signal is the process count, not traffic volume: once clients can land on different Tina4 instances, broadcasts need a shared backplane. Do not install NATS solely because Tina4 supports it; choose it when the deployment already uses NATS or needs NATS as its wider event bus.

Verify the setup with two Tina4 processes. Connect one WebSocket client to each process, broadcast from the first, and confirm the second client receives the frame. A single-process test proves only local delivery.

If `TINA4_WS_BACKPLANE` is not set, Tina4 broadcasts only to local connections. This is the correct default for a single instance.

---

16. Gotchas

1. WebSocket Needs a Persistent Server

Problem: WebSocket connections drop immediately.

Fix: Use `tina4 serve` which runs a persistent server. For production with Puma, configure WebSocket proxying with Nginx.

2. Events Are Symbols, Not Strings

Problem: Your handler never matches any events.

Fix: Use symbols (`:open`, `:message`, `:close`), not strings (`"open"`, `"message"`, `"close"`).

3. Messages Are Strings, Not Objects

Problem: `data` in the message handler is a string, not a Ruby hash.

Fix: Always `JSON.parse(data)` when you expect JSON messages. Always `JSON.generate(...)` when you send structured data.

4. Connection Count Is Per-Path

Problem: `connection.connection_count` returns a lower number than expected.

Cause: Connection count is scoped to the WebSocket path. Clients on `/ws/chat/room-1` and `/ws/chat/room-2` are counted separately.

5. Broadcasting Does Not Scale Across Servers

Problem: Users connected to different server instances do not see each other's messages.

Fix: Use a pub/sub backend like Redis to relay messages across server instances.

6. Large Messages Cause Disconnects

Problem: The connection drops when sending a large message.

Fix: Keep messages small (under 64KB). Use HTTP endpoints for bulk data transfer.

7. Memory Leak from Tracking Connected Users

Problem: The server's memory usage grows over time.

Fix: Always clean up in the `:close` handler: `$chat_users.delete(connection.id)`.

8. No Authentication on WebSocket Connect

Problem: Anyone can connect to your WebSocket endpoint.

Fix: Pass the token as a query parameter: `ws://localhost:7147/ws/chat?token=eyJ...`. Validate in the `:open` handler and call `connection.close` if invalid.

Server-Sent Events (SSE)

1. The Polling Problem

Your monitoring dashboard needs live metrics. CPU usage. Memory. Active connections. The numbers change every few seconds.

The obvious solution: poll. A timer fires every three seconds. The browser sends a GET request. The server queries the database. The server responds. The browser updates the page. Repeat.

That works. It also wastes resources. Nineteen out of twenty polls return the same data. Each one opens a connection, sends headers, waits for a response, and closes the connection. The server does the same work whether the data changed or not.

WebSocket solves this. A persistent, bi-directional connection. The server pushes when data changes. But WebSocket is designed for two-way communication. Chat rooms. Collaborative editing. Live gaming. Your dashboard only needs one direction: server to client.

Server-Sent Events (SSE) is the middle ground. One-way streaming over plain HTTP. The server pushes. The client listens. The browser reconnects automatically if the connection drops. No upgrade handshake. No special protocol. Just HTTP with a twist.

---

2. What SSE Is

HTTP works like this:

```
Client: "Give me /api/metrics"
Server: "Here are the metrics" (connection closes)
Client: "Give me /api/metrics" (new connection, 3 seconds later)
Server: "Here are the metrics" (connection closes)
```

WebSocket works like this:

```
Client: "Upgrade to WebSocket on /ws/metrics"
Server: "Upgrade accepted. Connection open."
Client: "Subscribe to CPU metrics"
Server: "CPU: 42%"
Server: "CPU: 38%"  (pushed at any time)
Client: "Unsubscribe"
```

SSE works like this:

```
Client: "Give me /events (keep the connection open)"
Server: "data: CPU 42%"
Server: "data: CPU 38%"  (pushed, no client request)
Server: "data: CPU 45%"  (keeps flowing)
Client: (just listens)
```

The key differences:

Feature	HTTP Polling	WebSocket	SSE
Direction	Client to server	Both ways	Server to client

Connection	New each time	Persistent	Persistent
Protocol	HTTP	WebSocket (upgrade)	HTTP
Auto-reconnect	No	No (manual)	Yes (built-in)
Binary data	Yes	Yes	No (text only)
Browser support	Universal	Universal	Universal (except IE)
Complexity	Simple	Moderate	Simple

SSE uses plain HTTP. No protocol upgrade. No special server support. The server sets `Content-Type: text/event-stream` and keeps the connection open. The browser's `EventSource` API handles reconnection, event parsing, and last-event-ID tracking.

---

3. Your First Stream

Create `src/routes/events.py`:

```
import asyncio
from tina4_python import get

@get("/events")
async def stream_events(request, response):
    async def generate():
        for i in range(10):
            yield f"data: {{\"count\": {i}, \"message\": \"tick\"}}\n\n"
            await asyncio.sleep(1)
        yield "data: {\"done\": true}\n\n"

    return response.stream(generate())
```

Three things happen here:

- `generate()` is an async generator. Each `yield` produces one SSE message.
- `response.stream()` tells Tina4 to keep the connection open and send each chunk as it arrives.
- The framework sets `Content-Type: text/event-stream`, `Cache-Control: no-cache`, and `Connection: keep-alive` for you.

Start the server and test with curl:

```
curl -N http://localhost:7147/events
```

You see one message per second:

```
data: {"count": 0, "message": "tick"}

data: {"count": 1, "message": "tick"}

data: {"count": 2, "message": "tick"}
```

Each message arrives the moment the server yields it. No buffering. No waiting for the full response.

---

4. The SSE Protocol

SSE messages are plain text with a simple format. Each message is one or more field lines followed by a blank line.

The `data:` Field

The most common field. Contains the message payload.

```
data: Hello, world
```

Multiple `data:` lines in one message get joined with newlines:

```
data: line one
data: line two
```

The client receives this as `"line one\nline two"`.

The `event:` Field

Names the event type. Without it, the browser fires `onmessage`. With it, the browser fires a named event listener.

```
yield "event: metrics\ndata: {\"cpu\": 42}\n\n"
yield "event: alert\ndata: {\"level\": \"high\", \"message\": \"CPU spike\"}\n\n"
```

On the client:

```
source.addEventListener("metrics", (e) => {
    const data = JSON.parse(e.data);
    updateDashboard(data);
});

source.addEventListener("alert", (e) => {
    const data = JSON.parse(e.data);
    showAlert(data.message);
});
```

Named events let you multiplex different data types on a single connection.

The `id:` Field

Sets the last event ID. If the connection drops, the browser sends `Last-Event-ID` in the reconnection request. Your server can resume from where it left off.

```
yield f"id: {i}\ndata: {{\"count\": {i}}}\n\n"
```

The `retry:` Field

Tells the browser how many milliseconds to wait before reconnecting after a disconnect.

```
yield "retry: 5000\n\n"  # reconnect after 5 seconds
```

The Delimiter

Every SSE message ends with two newlines: `\n\n`. This tells the browser that the message is complete. A single `\n` separates fields within one message.

```
# One complete SSE message:
yield "event: update\nid: 42\ndata: {\"value\": 100}\n\n"

# Broken down:
# event: update\n     ← event type
# id: 42\n            ← event ID
# data: {"value": 100}\n  ← payload
# \n                   ← end of message (blank line)
```

---

5. Frontend: EventSource

The browser provides `EventSource` for consuming SSE streams. It handles connection management, reconnection, and message parsing.

Basic Usage

```
const source = new EventSource("/events");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    console.log("Received:", data);
};

source.onerror = function () {
    console.log("Connection lost. Reconnecting...");
};

source.onopen = function () {
    console.log("Connected");
};
```

`EventSource` reconnects automatically when the connection drops. The `onerror` handler fires, the browser waits (default 3 seconds), then reconnects. Your server receives a new request with the `Last-Event-ID` header if you sent event IDs.

Named Events

```
const source = new EventSource("/events");

source.addEventListener("metrics", function (event) {
    updateChart(JSON.parse(event.data));
});

source.addEventListener("notification", function (event) {
    showToast(JSON.parse(event.data));
});

source.addEventListener("heartbeat", function (event) {
    // Connection is alive
});
```

Closing the Connection

```
source.close();
```

The browser stops reconnecting. The server receives a client disconnect.

With Authentication

`EventSource` does not support custom headers. Pass tokens as query parameters:

```
const token = getAuthToken();
const source = new EventSource(`/events?token=${token}`);
```

On the server:

```
@get("/events")
async def secure_events(request, response):
    token = request.query.get("token")
    payload = Auth.valid_token(token)
    if not payload:
        return response({"error": "Unauthorized"}, 401)

    async def generate():
        while True:
            yield f"data: {{\"user\": \"{payload['email']}\"}}\n\n"
            await asyncio.sleep(5)

    return response.stream(generate())
```

---

6. Real Examples

Live Dashboard

Stream database metrics every five seconds:

```
import asyncio
from tina4_python import get
from tina4_python.database import Database

@get("/events/dashboard")
async def dashboard_stream(request, response):
    async def generate():
        db = Database()
        while True:
            orders = db.fetch_one("SELECT count(*) as total FROM orders")
            revenue = db.fetch_one("SELECT sum(total) as revenue FROM orders WHERE date(created_

            yield f"data: {{\"orders\": {orders['total']}, \"revenue\": {revenue['revenue'] or 0
            await asyncio.sleep(5)

    return response.stream(generate())
```

The client updates the dashboard numbers without polling. One connection. One query every five seconds. No wasted requests.

Build Progress

Stream deployment status as it happens:

```
import asyncio
import json
from tina4_python import get

@get("/events/deploy/{build_id}")
async def deploy_stream(request, response):
    build_id = request.params["build_id"]

    async def generate():
        steps = [
            ("pull", "Pulling latest code..."),
            ("install", "Installing dependencies..."),
            ("test", "Running tests..."),
            ("build", "Building application..."),
            ("deploy", "Deploying to production..."),
            ("done", "Deployment complete"),
        ]

        for step, message in steps:
            yield f"event: progress\ndata: {json.dumps({'step': step, 'message': message, 'build
            await asyncio.sleep(2)  # simulate work

    return response.stream(generate())
```

The browser shows a progress bar that updates in real time. Each step arrives as it completes. The user watches the deployment unfold instead of staring at a spinner.

LLM Chat Streaming

Stream AI responses token by token. This is how ChatGPT works:

```
import asyncio
import json
import urllib.request
from tina4_python import get

@get("/events/chat")
async def chat_stream(request, response):
    prompt = request.query.get("q", "Hello")

    async def generate():
        # Call the LLM API with streaming enabled
        req_data = json.dumps({
            "model": "claude-sonnet-4-20250514",
            "max_tokens": 512,
            "stream": True,
            "messages": [{"role": "user", "content": prompt}],
        }).encode()

        req = urllib.request.Request(
            "https://api.anthropic.com/v1/messages",
            data=req_data,
            headers={
                "Content-Type": "application/json",
                "x-api-key": os.environ["ANTHROPIC_API_KEY"],
```

```
                "anthropic-version": "2023-06-01",
            },
        )

        with urllib.request.urlopen(req) as resp:
            for line in resp:
                line = line.decode().strip()
                if line.startswith("data: "):
                    chunk = json.loads(line[6:])
                    if chunk.get("type") == "content_block_delta":
                        text = chunk["delta"].get("text", "")
                        yield f"data: {json.dumps({'token': text})}\n\n"
                        await asyncio.sleep(0)

        yield "data: {\"done\": true}\n\n"

    return response.stream(generate())
```

The frontend appends each token as it arrives:

```
const source = new EventSource(`/events/chat?q=${encodeURIComponent(question)}`);
const output = document.getElementById("response");

source.onmessage = function (event) {
    const data = JSON.parse(event.data);
    if (data.done) {
        source.close();
        return;
    }
    output.textContent += data.token;
};
```

The response builds character by character. The user reads as the AI writes. No waiting for the full answer.

File Processing Progress

Upload a CSV, stream the processing status:

```
import asyncio
import json
from tina4_python import post

@post("/api/import")
async def import_csv(request, response):
    file = request.files.get("csv")
    if not file:
        return response({"error": "No file"}, 400)

    lines = file["content"].decode().strip().split("\n")
    total = len(lines) - 1  # minus header

    async def generate():
        for i, line in enumerate(lines[1:], 1):
            # Process each row
            cols = line.split(",")
            # ... insert into database ...
            yield f"data: {json.dumps({'processed': i, 'total': total, 'percent': round(i/total*
            await asyncio.sleep(0)  # yield control
```

```
        yield f"data: {json.dumps({'done': True, 'total': total})}\n\n"

    return response.stream(generate(), content_type="text/event-stream")
```

The browser shows a progress bar that fills as each row is processed. The user sees exactly where the import stands.

---

7. Custom Content Types

SSE defaults to `text/event-stream`, but `response.stream()` accepts any content type. Use this for newline-delimited JSON (NDJSON), chunked binary, or custom protocols.

NDJSON Streaming

```
import asyncio
import json
from tina4_python import get

@get("/api/export")
async def export_stream(request, response):
    async def generate():
        db = Database()
        result = db.fetch("SELECT * FROM products", limit=1000)
        for row in result.records:
            yield json.dumps(row) + "\n"
            await asyncio.sleep(0)

    return response.stream(generate(), content_type="application/x-ndjson")
```

Each line is a complete JSON object. The client reads line by line. No need to parse a giant array. Memory stays flat regardless of how many rows you export.

Consuming NDJSON on the Client

```
async function streamProducts() {
    const response = await fetch("/api/export");
    const reader = response.body.getReader();
    const decoder = new TextDecoder();
    let buffer = "";

    while (true) {
        const { done, value } = await reader.read();
        if (done) break;

        buffer += decoder.decode(value, { stream: true });
        const lines = buffer.split("\n");
        buffer = lines.pop(); // keep incomplete line

        for (const line of lines) {
            if (line.trim()) {
                const product = JSON.parse(line);
                addToTable(product);
            }
        }
    }
}
```

---

8. SSE vs WebSocket

Both keep a persistent connection. Both push data in real time. The choice depends on the direction of communication.

Use Case	Choose	Why
Live dashboard	SSE	Server pushes metrics. Client only listens.
Chat application	WebSocket	Both sides send messages.
Notification feed	SSE	Server pushes. Client marks as read via separate HTTP POST.
Collaborative editor	WebSocket	Both sides send cursor positions and edits.
Build/deploy progress	SSE	Server streams status. Client watches.
LLM token streaming	SSE	Server streams tokens. Client displays them.
Online gaming	WebSocket	Low-latency, bi-directional input and state.
Stock ticker	SSE	Server pushes prices. Client displays.
File upload progress	HTTP (native)	Browser tracks upload. Server tracks processing via SSE.

Rule of thumb: If the client only listens, use SSE. If the client sends data back on the same connection, use WebSocket.

SSE has one advantage WebSocket does not: automatic reconnection. The browser's `EventSource` reconnects without any code. WebSocket requires manual reconnection logic (or `frond.js` which handles it for you).

---

9. Production Considerations

Nginx Proxy Buffering

Nginx buffers responses by default. SSE messages accumulate in the buffer and arrive in batches instead of one at a time. Tina4 sets `X-Accel-Buffering: no` on streaming responses, which tells Nginx to disable buffering for that response.

If you configure Nginx manually:

```
location /events/ {
    proxy_pass http://localhost:7147;
    proxy_buffering off;
    proxy_cache off;
    proxy_set_header Connection '';
    proxy_http_version 1.1;
    chunked_transfer_encoding off;
}
```

Connection Limits

Each SSE connection is a persistent HTTP connection. Most servers limit concurrent connections. The default asyncio server handles thousands. In production:

- Monitor open connections
- Set timeouts on long-running streams
- Close streams when the client no longer needs them

```
@get("/events/metrics")
async def metrics_with_timeout(request, response):
    async def generate():
        for _ in range(3600):  # max 1 hour (one message per second)
            yield f"data: {{\"cpu\": {get_cpu()}}}\n\n"
            await asyncio.sleep(1)
        yield "event: timeout\ndata: {\"message\": \"Stream expired. Reconnect.\"}\n\n"

    return response.stream(generate())
```

Heartbeat Messages

Some proxies and load balancers close idle connections after a timeout (typically 30-60 seconds). Send a heartbeat comment to keep the connection alive:

```
async def generate():
    counter = 0
    while True:
        if has_new_data():
            yield f"data: {json.dumps(get_data())}\n\n"
        else:
            yield ": heartbeat\n\n"  # SSE comment (colon prefix) - ignored by EventSource
        counter += 1
        await asyncio.sleep(5)
```

Lines starting with `:` are SSE comments. The browser ignores them, but they keep the connection alive through proxies.

---

10. Exercise: Build a Live Metrics Dashboard

Build a dashboard that streams server metrics to the browser.

Requirements

- SSE endpoint at `GET /events/server-metrics` that streams every 3 seconds:
- Current timestamp
- A random CPU percentage (simulate with `random.randint(10, 90)`)
- A random memory percentage
- A random request count
- HTML page at `GET /dashboard` that:
- Connects to the SSE endpoint
- Displays the four metrics in cards
- Updates the values in real time (no page refresh)
- Shows a "Connected" / "Reconnecting..." status indicator

Test by:

- Open `http://localhost:7147/dashboard`
- Watch the numbers update every 3 seconds
- Stop the server and verify "Reconnecting..." appears
- Restart the server and verify it reconnects and resumes updating

---

11. Solution

Create `src/routes/server_metrics.py`:

```
import asyncio
import json
import random
from datetime import datetime, timezone
from tina4_python import get

@get("/events/server-metrics")
async def server_metrics_stream(request, response):
    async def generate():
        yield "retry: 3000\n\n"
        counter = 0
        while True:
            yield f"id: {counter}\ndata: {json.dumps({
                'timestamp': datetime.now(timezone.utc).isoformat(),
                'cpu': random.randint(10, 90),
                'memory': random.randint(30, 85),
                'requests': random.randint(100, 5000),
            })}\n\n"
            counter += 1
```

```
            await asyncio.sleep(3)

    return response.stream(generate())


@get("/dashboard")
async def dashboard_page(request, response):
    html = """
    <!DOCTYPE html>
    <html>
    <head><title>Live Dashboard</title></head>
    <body style="font-family: system-ui; max-width: 600px; margin: 40px auto; padding: 0 20px;">
        <h1>Server Metrics</h1>
        <p id="status" style="color: #888;">Connecting...</p>
        <div style="display: grid; grid-template-columns: 1fr 1fr; gap: 16px; margin-top: 20px;"
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">CPU</div>
                <div id="cpu" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Memory</div>
                <div id="memory" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Requests/min</div>
                <div id="requests" style="font-size: 32px; font-weight: 700;">--</div>
            </div>
            <div style="background: #f5f5f5; padding: 20px; border-radius: 8px;">
                <div style="color: #888; font-size: 14px;">Last Update</div>
                <div id="time" style="font-size: 16px; font-weight: 500;">--</div>
            </div>
        </div>
        <script>
            const source = new EventSource("/events/server-metrics");
            const status = document.getElementById("status");

            source.onopen = function () {
                status.textContent = "Connected";
                status.style.color = "#22c55e";
            };

            source.onmessage = function (event) {
                const data = JSON.parse(event.data);
                document.getElementById("cpu").textContent = data.cpu + "%";
                document.getElementById("memory").textContent = data.memory + "%";
                document.getElementById("requests").textContent = data.requests.toLocaleString()
                document.getElementById("time").textContent = new Date(data.timestamp).toLocaleT
            };

            source.onerror = function () {
                status.textContent = "Reconnecting...";
                status.style.color = "#ef4444";
            };
        </script>
    </body>
    </html>
    """
    return response(html)
```

Open `http://localhost:7147/dashboard`. The numbers update every three seconds. Stop the server. The status turns red: "Reconnecting..." Start it again. The status turns green. The numbers resume.

No polling. No WebSocket. No JavaScript timers. The browser handles everything.

---

12. What You Learned

- **SSE streams data from server to client** over a single HTTP connection. No upgrade. No special protocol.
- `response.stream(generator)` keeps the connection open and flushes each chunk as the generator yields it.
- **The SSE protocol** uses `data:`, `event:`, `id:`, and `retry:` fields. Messages end with `\n\n`.
- `EventSource` on the client handles connection, parsing, and automatic reconnection.
- **Named events** let you multiplex different data types on one stream.
- **Custom content types** let you stream NDJSON, binary, or any format.
- **SSE is for one-way server pushes.** Use WebSocket when the client needs to send data back on the same connection.
- **Heartbeat comments** keep connections alive through proxies. Tina4 sets `X-Accel-Buffering: no` to disable nginx buffering.

One direction. One connection. Real-time data. The server speaks. The client listens. The rest is infrastructure.

WSDL / SOAP

1. When the World Still Uses SOAP

REST gets all the attention. But banks, government portals, insurance systems, and ERP platforms built in the 2000s still speak SOAP. You need to call a mortgage calculation service. Your payment processor publishes a WSDL. Your government tax API requires SOAP envelopes.

Tina4's WSDL module lets you expose your own operations as a SOAP service and call remote WSDL services from Ruby.

---

2. What SOAP and WSDL Are

SOAP (Simple Object Access Protocol) sends XML envelopes over HTTP. A request wraps parameters in XML. The response wraps results in XML.

WSDL (Web Services Description Language) is the machine-readable contract that describes what operations a SOAP service exposes, what parameters each operation accepts, and what it returns. Clients parse the WSDL to know how to call the service.

A WSDL service call looks like this at the wire level:

```
POST /soap/calculator HTTP/1.1
Content-Type: text/xml

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <Add>
      <a>14</a>
      <b>28</b>
    </Add>
  </soap:Body>
</soap:Envelope>
```

Response:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <AddResponse>
      <result>42</result>
    </AddResponse>
  </soap:Body>
</soap:Envelope>
```

Tina4 handles all of this XML generation and parsing for you.

---

3. Defining a WSDL Service

Create a WSDL service class that inherits from `Tina4::WSDL` and defines operations.

```
# src/services/calculator_service.rb
```

```
class CalculatorService < Tina4::WSDL
  service_name "CalculatorService"
  namespace    "http://example.com/calculator"
  description  "Basic arithmetic operations"

  wsdl_operation :add,
    params: { a: :integer, b: :integer },
    returns: :integer do |params|
    params[:a] + params[:b]
  end

  wsdl_operation :subtract,
    params: { a: :integer, b: :integer },
    returns: :integer do |params|
    params[:a] - params[:b]
  end

  wsdl_operation :multiply,
    params: { a: :float, b: :float },
    returns: :float do |params|
    params[:a] * params[:b]
  end

  wsdl_operation :divide,
    params: { a: :float, b: :float },
    returns: :float do |params|
    raise "Division by zero" if params[:b] == 0
    params[:a] / params[:b]
  end
end
```

---

4. Mounting the Service

Mount the WSDL service at a path using the router.

```
# src/routes/soap.rb
Tina4::Router.mount_wsdl("/soap/calculator", CalculatorService)
```

This creates two endpoints:

- `GET /soap/calculator?wsdl` -- serves the generated WSDL XML document
- `POST /soap/calculator` -- handles incoming SOAP requests

---

5. Auto-Generated WSDL

Visit `/soap/calculator?wsdl` in a browser or curl:

```
curl "http://localhost:7147/soap/calculator?wsdl"
```

Tina4 generates the full WSDL document from your operation definitions:

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="CalculatorService"
  targetNamespace="http://example.com/calculator"
```

```
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://example.com/calculator"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <message name="addRequest">
    <part name="a" type="xsd:integer"/>
    <part name="b" type="xsd:integer"/>
  </message>
  <message name="addResponse">
    <part name="result" type="xsd:integer"/>
  </message>

  <portType name="CalculatorServicePortType">
    <operation name="add">
      <input message="tns:addRequest"/>
      <output message="tns:addResponse"/>
    </operation>
    <!-- ... -->
  </portType>
  <!-- binding and service elements follow -->
</definitions>
```

The WSDL is generated dynamically. It always reflects the current operation definitions.

---

6. Calling the SOAP Service

```
curl -X POST http://localhost:7147/soap/calculator \
  -H "Content-Type: text/xml" \
  -d '
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <add>
      <a>14</a>
      <b>28</b>
    </add>
  </soap:Body>
</soap:Envelope>'
```

Response:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <addResponse>
      <result>42</result>
    </addResponse>
  </soap:Body>
</soap:Envelope>
```

---

7. Calling a Remote WSDL Service

Tina4 provides the SOAP **server** side (`Tina4::WSDL` and the `Tina4::WSDL::Service` builder). It does not ship a SOAP client. To call an external SOAP service, send the SOAP envelope yourself with the standard library or a dedicated gem.

```
require "net/http"
require "uri"

uri  = URI("https://www.w3schools.com/xml/tempconvert.asmx")
body = <<~XML
  <?xml version="1.0" encoding="utf-8"?>
  <soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
      <CelsiusToFahrenheit xmlns="https://www.w3schools.com/xml/">
        <Celsius>100</Celsius>
      </CelsiusToFahrenheit>
    </soap:Body>
  </soap:Envelope>
XML

response = Net::HTTP.post(
  uri, body,
  "Content-Type" => "text/xml; charset=utf-8",
  "SOAPAction"   => "https://www.w3schools.com/xml/CelsiusToFahrenheit"
)

puts response.body # parse the SOAP envelope for <CelsiusToFahrenheitResult>
```

For richer client features (WSDL parsing, type coercion), use a gem such as `savon`.

---

8. Complex Types

Operations can use nested types for richer request and response shapes.

```
class OrderService < Tina4::WSDL
  service_name "OrderService"
  namespace    "http://example.com/orders"

  wsdl_type :Address do
    field :street, :string
    field :city,   :string
    field :zip,    :string
    field :country, :string
  end

  wsdl_type :OrderResult do
    field :order_id,    :string
    field :status,      :string
    field :total,       :float
  end

  wsdl_operation :place_order,
    params: { customer_email: :string, shipping_address: :Address, total: :float },
    returns: :OrderResult do |params|
    {
      order_id: SecureRandom.uuid,
      status:   "pending",
      total:    params[:total]
    }
  end
end
```

---

9. Error Handling in Operations

Raise a standard Ruby exception to return a SOAP fault.

```
wsdl_operation :divide,
  params: { a: :float, b: :float },
  returns: :float do |params|
  raise ArgumentError, "Divisor cannot be zero" if params[:b] == 0
  params[:a] / params[:b]
end
```

The caller receives:

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Server</faultcode>
      <faultstring>Divisor cannot be zero</faultstring>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

---

10. Gotchas

1. WSDL caching in production

The remote WSDL client caches the WSDL after the first fetch. If the remote service updates its WSDL, restart your app or call `client.reload_wsdl` to force a refresh.

2. Character encoding

SOAP envelopes must be UTF-8. If your operation returns strings with special characters, ensure they are UTF-8 encoded before returning.

3. Namespace conflicts

Each WSDL service must have a unique namespace. Duplicate namespaces across mounted services cause client-side parsing errors.

4. SOAP is not REST

SOAP uses `POST` for every operation, including reads. Do not expect `GET` requests to trigger SOAP operations -- only the `?wsdl` query param uses `GET`.

Dependency Injection Container

1. The Problem With Hard Dependencies

Your route handler creates a `Tina4::Database` instance directly. Your mailer instantiates its SMTP connection inline. Your PDF generator calls `Tina4::Api.new(...)` inside the method body.

Hard dependencies make code hard to test (you cannot swap the real database for a fake one), hard to reconfigure (the connection string is buried in 12 files), and hard to share (every caller creates its own instance instead of sharing one).

A DI container is a registry. You register services by name once. Everything that needs a service asks the container for it by name. Tests register fake implementations. Production registers real ones.

---

2. Registering a Service

```
Tina4::Container.register(:database) do
  Tina4::Database.new(ENV["TINA4_DATABASE_URL"])
end
```

The block is a factory. It runs the first time the service is resolved (lazy initialization). The result is cached and reused for every subsequent resolve.

Registering Multiple Services

```
Tina4::Container.register(:database) do
  Tina4::Database.new(ENV["TINA4_DATABASE_URL"])
end

Tina4::Container.register(:cache) do
  Tina4::QueryCache.new(default_ttl: 60, max_size: 1000)
end

Tina4::Container.register(:stripe) do
  Tina4::Api.new("https://api.stripe.com", {
    "Authorization"  => "Bearer #{ENV['STRIPE_SECRET_KEY']}",
    "Content-Type"   => "application/json",
    "Stripe-Version" => "2024-11-20"
  })
end

Tina4::Container.register(:mailer) do
  Tina4::Messenger.new(
    host:     ENV["TINA4_MAIL_HOST"],
    port:     ENV["TINA4_MAIL_PORT"].to_i,
    username: ENV["SMTP_USER"],
    password: ENV["SMTP_PASS"]
  )
end
```

---

3. Resolving a Service

```
db = Tina4::Container.resolve(:database)
users = db.query("SELECT * FROM users")
```

The first `resolve` call runs the factory block and caches the result. Every subsequent call returns the same instance.

---

4. Checking Registration

```
if Tina4::Container.has?(:cache)
  cache = Tina4::Container.resolve(:cache)
  cached = cache.get("products:list")
end
```

---

5. Using Services in Route Handlers

```
# @noauth
Tina4::Router.get("/api/users") do |request, response|
  db = Tina4::Container.resolve(:database)

  users = db.query("SELECT id, name, email FROM users")

  response.json({ users: users, count: users.length })
end

Tina4::Router.post("/api/orders") do |request, response|
  body   = request.body
  db     = Tina4::Container.resolve(:database)
  mailer = Tina4::Container.resolve(:mailer)
  stripe = Tina4::Container.resolve(:stripe)

  result = stripe.post("/v1/payment_intents", body: {
    amount:   (body["total"].to_f * 100).to_i,
    currency: "usd"
  })

  unless result.success?
    next response.json({ error: "Payment failed" }, 402)
  end

  order_id = db.execute(
    "INSERT INTO orders (email, total, payment_intent) VALUES (?, ?, ?)",
    body["email"], body["total"], result.body["id"]
  )

  mailer.send(
    to:      body["email"],
    subject: "Order Confirmation",
    body:    "Your order ##{order_id} has been placed."
  )

  response.json({ order_id: order_id }, 201)
end
```

---

6. reset: Reset the Container

Remove all registered services and their cached instances.

```
Tina4::Container.reset
```

Use `reset` in tests to start fresh between test cases and register test doubles.

---

7. Testing With the Container

Swap real services for fakes in tests without changing production code.

```
# test/test_orders.rb
require "minitest/autorun"

class FakeMailer
  attr_reader :sent

  def initialize
    @sent = []
  end

  def send(opts)
    @sent << opts
  end
end

class FakeStripe
  def post(path, body:)
    OpenStruct.new(
      success?: true,
      body: { "id" => "pi_test_123" }
    )
  end
end

class FakeDatabase
  def execute(sql, *args)
    42  # Simulated insert returning an ID
  end
end

class OrderTest < Minitest::Test
  def setup
    Tina4::Container.reset
    @mailer = FakeMailer.new

    Tina4::Container.register(:database) { FakeDatabase.new }
    Tina4::Container.register(:mailer)   { @mailer }
    Tina4::Container.register(:stripe)   { FakeStripe.new }
  end

  def teardown
    Tina4::Container.reset
```

```
  end

  def test_order_sends_confirmation_email
    # Simulate a POST /api/orders request
    # ...route invocation logic...

    assert_equal 1, @mailer.sent.length
    assert_equal "Order Confirmation", @mailer.sent.first[:subject]
  end
end
```

No real database, no real Stripe, no real email -- the test runs in milliseconds and never touches external services.

---

8. Service Dependencies

A factory block can resolve other services from the container.

```
Tina4::Container.register(:order_service) do
  db     = Tina4::Container.resolve(:database)
  mailer = Tina4::Container.resolve(:mailer)
  stripe = Tina4::Container.resolve(:stripe)

  OrderService.new(db: db, mailer: mailer, stripe: stripe)
end
```

```
class OrderService
  def initialize(db:, mailer:, stripe:)
    @db     = db
    @mailer = mailer
    @stripe = stripe
  end

  def place(email:, total:)
    result = @stripe.post("/v1/payment_intents", body: {
      amount: (total * 100).to_i, currency: "usd"
    })

    raise "Payment failed" unless result.success?

    order_id = @db.execute(
      "INSERT INTO orders (email, total) VALUES (?, ?)", email, total
    )

    @mailer.send(to: email, subject: "Order ##{order_id} confirmed")

    order_id
  end
end
```

```
Tina4::Router.post("/api/orders") do |request, response|
  body    = request.body
  service = Tina4::Container.resolve(:order_service)

  order_id = service.place(email: body["email"], total: body["total"].to_f)
  response.json({ order_id: order_id }, 201)
rescue => e
```

```
  response.json({ error: e.message }, 500)
end
```

The route handler has one job: parse the request, delegate to the service, return the response.

---

9. Gotchas

1. reset drops cached instances

`reset` removes the factory registration and the cached instance. Any subsequent `resolve` after `reset` raises unless you re-register.

2. Singletons by default

The container caches the first resolved instance. If your service is not thread-safe, wrapping it in a mutex or using a new instance per request is safer.

3. Circular dependencies cause infinite loops

If service A resolves B, and B resolves A during factory execution, the container loops forever. Restructure to break the cycle.

4. Registration order matters

A factory block that resolves `:database` runs when it is first resolved, not when it is registered. Registration order does not matter. Resolution order does.

Service Runner

1. Work That Runs Forever

Some work does not fit inside an HTTP request. A heartbeat that pings a health check endpoint every 30 seconds. A queue consumer that processes jobs continuously.

These are background services: long-running processes that start with the app and run until it stops.

Tina4's Service Runner manages background services. Register a service with a name and a block. The runner starts it in a background thread. The service runs for the lifetime of the app. If it crashes, the runner restarts it.

---

2. Defining a Service

```
Tina4::ServiceRunner.register("heartbeat") do
  loop do
    Tina4::Log.info("Heartbeat", status: "alive", time: Time.now.utc.iso8601)
    sleep 30
  end
end
```

The block runs in a background thread. The `loop` keeps it alive. `sleep 30` yields control between iterations.

---

3. Starting All Services

```
Tina4::ServiceRunner.start
```

Call this once, typically in your app startup file after route definitions. Called with no argument, `start` launches every registered service in a background thread.

```
# config/app.rb

require_relative "../src/routes/api"
require_relative "../src/services/background"

Tina4::ServiceRunner.start
```

---

4. Starting a Single Service

```
Tina4::ServiceRunner.start("heartbeat")
```

Useful for starting services conditionally based on environment or configuration.

---

5. Stopping Services

```
# Stop a specific service
Tina4::ServiceRunner.stop("heartbeat")

# Stop all running services (no argument)
Tina4::ServiceRunner.stop
```

`stop` signals the thread to terminate. The runner waits for the thread to finish its current iteration before stopping.

---

6. Checking Service Status

```
Tina4::ServiceRunner.is_running("heartbeat")
# => true or false

Tina4::ServiceRunner.list
# => [
#      { name: "heartbeat", running: true,  last_run: <Time>, error_count: 0, options: {...} },
#      { name: "metrics",   running: false, last_run: nil,    error_count: 0, options: {...} }
#    ]
```

---

7. Automatic Restart on Failure

If a service crashes, the runner restarts it with an exponential backoff delay.

```
Tina4::ServiceRunner.register("unreliable_service") do
  loop do
    begin
      do_something_that_might_fail
      sleep 10
    rescue => e
      Tina4::Log.error("Service error", service: "unreliable_service", error: e.message)
      # re-raise to trigger restart, or rescue and continue
      raise
    end
  end
end
```

If the block raises, the runner logs the error, waits (1s, 2s, 4s... up to 60s), then restarts the service.

Set `max_restarts` to cap automatic restarts:

```
Tina4::ServiceRunner.register("payment_sync", max_restarts: 5) do
  # ...
end
```

After 5 restarts, the service is marked as `:failed` and not restarted again.

---

8. Queue Consumer Service

The most common background service: processing a queue continuously.

```
Tina4::ServiceRunner.register("email_consumer") do
  queue = Tina4::Queue.new(topic: "emails")

  loop do
    job = queue.pop

    if job.nil?
      sleep 2  # No jobs -- wait before polling again
      next
    end

    begin
      payload = job.payload
      send_email(payload[:to], payload[:subject], payload[:body])
      job.complete

      Tina4::Log.info("Email sent", to: payload[:to], subject: payload[:subject])
    rescue => e
      job.fail(e.message)
      Tina4::Log.error("Email failed", to: payload[:to], error: e.message)
    end
  end
end
```

---

9. Scheduled Task Service

Run a task on a schedule using a service with sleep-based timing.

```
Tina4::ServiceRunner.register("daily_report") do
  loop do
    now = Time.now

    # Run at 08:00 every day
    if now.hour == 8 && now.min == 0
      Tina4::Log.info("Generating daily report")

      db = Tina4::Container.resolve(:database)
      rows = db.query("SELECT COUNT(*) as orders FROM orders WHERE DATE(created_at) = DATE('now'

      Tina4::Log.info("Daily report complete", orders_today: rows.first["orders"])

      sleep 61  # Skip past the current minute to avoid re-running
    else
      sleep 30  # Check every 30 seconds
    end
  end
end
```

---

10. Metrics Collector Service

```
Tina4::ServiceRunner.register("metrics_collector") do
  loop do
    mem_mb = `ps -o rss= -p #{Process.pid}`.strip.to_i / 1024

    Tina4::Log.info("Metrics",
      memory_mb:  mem_mb,
      pid:        Process.pid,
      threads:    Thread.list.count
    )

    sleep 60
  end
end
```

---

11. Listing Registered Services

```
Tina4::ServiceRunner.list.each do |info|
  puts "#{info[:name]}: #{info[:running] ? 'running' : 'stopped'}"
end
```

Output:

```
heartbeat: running
email_consumer: running
daily_report: running
metrics_collector: running
```

---

12. Shutdown Hooks

Register cleanup logic that runs when a service is stopped.

```
Tina4::ServiceRunner.register("database_sync") do |service|
  db = Tina4::Container.resolve(:database)

  service.on_stop do
    Tina4::Log.info("Database sync shutting down -- flushing pending writes")
    db.flush
  end

  loop do
    sync_records(db)
    sleep 5
  end
end
```

---

13. Gotchas

1. Services share the main process

Background services run in threads, not separate processes. A runaway service that consumes 100% CPU affects the HTTP server running in the same process. Keep service work lightweight or move heavy computation to separate processes via the queue system.

2. sleep is required in loops

A `loop` with no `sleep` spins the CPU at 100%. Always `sleep` between iterations.

3. Thread safety

Services share memory with the main thread. Access shared mutable state (global variables, class variables, shared caches) through a mutex.

```
COUNTER_MUTEX = Mutex.new
COUNTER = { value: 0 }

Tina4::ServiceRunner.register("counter_service") do
  loop do
    COUNTER_MUTEX.synchronize { COUNTER[:value] += 1 }
    sleep 1
  end
end
```

4. stop on shutdown

In production, call `Tina4::ServiceRunner.stop` (no argument stops every service) in a signal trap to cleanly shut down services before the process exits.

```
trap("SIGTERM") do
  Tina4::ServiceRunner.stop
  exit 0
end
```

MCP Dev Tools

1. What is MCP?

The Model Context Protocol connects AI coding tools to your running application. Claude Code, Cursor, and other assistants speak this protocol. When they connect, they see your database, routes, templates, and files -- not as static text, but as live, queryable tools.

Tina4 ships a built-in MCP server. It starts automatically in dev mode. No configuration needed.

---

2. How It Works

Set `TINA4_DEBUG=true` in your `.env` file and start the server:

```
tina4 serve
```

The console prints:

```
MCP server available at http://localhost:7147/__dev/mcp
```

Tina4 writes the connection details to `.claude/settings.json` automatically. Claude Code discovers the server on its next restart. Cursor reads the same file.

Localhost-Only Security

The built-in MCP server activates only when two conditions hold:

- `TINA4_DEBUG=true`
- The server runs on `localhost`, `127.0.0.1`, or `0.0.0.0`

Deploy to a remote server and the MCP endpoint vanishes -- even with debug enabled. This prevents accidental exposure of database queries and file editing in production.

To enable on a staging server (rare, intentional), set:

```
TINA4_MCP_REMOTE=true
```

---

3. Connecting AI Tools

Claude Code

Tina4 auto-generates `.claude/settings.json` with the current Streamable HTTP transport:

```
{
  "mcpServers": {
    "tina4-dev": {
      "type": "http",
      "url": "http://localhost:7147/__dev/mcp"
    }
  }
}
```

Restart Claude Code. It connects and lists the tools.

Prefer the command line? Register the same server in one step:

```
claude mcp add --transport http tina4-dev http://localhost:7147/__dev/mcp
```

Cursor

Copy the same MCP config into Cursor's settings. The `type` and `url` are identical.

Manual Connection

Any modern MCP client talks to a single endpoint:

- **Streamable HTTP**: `POST http://localhost:7147/__dev/mcp` -- send JSON-RPC 2.0 and read the response inline. `initialize` returns an `Mcp-Session-Id` header; send it back on later calls. `DELETE` on the same URL ends the session.

Older clients that speak the 2024-11-05 HTTP+SSE transport keep working:

- **SSE handshake** (legacy): `GET http://localhost:7147/__dev/mcp/sse` -- returns the message endpoint URL
- **Message endpoint** (legacy): `POST http://localhost:7147/__dev/mcp/message` -- accepts JSON-RPC 2.0 messages

---

4. Built-in Tools

The MCP server exposes 48 tools organized by category. The exact list lives in the framework's `Tina4::Mcp::Tools` registration block -- that is authoritative.

Database

Tool	Description
`database_query`	Execute a read-only SQL query (SELECT)
`database_execute`	Execute arbitrary SQL (INSERT, UPDATE, DELETE, DDL)
`database_tables`	List all tables in the database
`database_columns`	Get column definitions for a specific table

Example -- an AI assistant queries your database directly:

```
Tool: database_query
Arguments: {"sql": "SELECT id, name, email FROM users WHERE active = 1"}
```

The `database_execute` tool handles write operations. It commits automatically after each statement.

Routes

Tool	Description
`route_list`	List all registered routes with methods and auth status
`route_test`	Call a route and return status, body, and headers
`swagger_spec`	Return the full OpenAPI 3.0.3 specification

The `route_test` tool lets an AI assistant verify endpoints without leaving the conversation:

```
Tool: route_test
Arguments: {"method": "GET", "path": "/api/users"}
```

Templates

Tool	Description
`template_render`	Render a Frond template string with provided data

Useful for testing template snippets:

```
Tool: template_render
Arguments: {"template": "Hello {{ name }}!", "data": "{\"name\": \"Alice\"}"}
```

Files

Tool	Description
`file_read`	Read a project file (relative to project root)
`file_write`	Write or update a project file (full overwrite)
`file_patch`	Targeted edit -- replace `old_string` with `new_string` in a file
`file_list`	List files in a directory
`asset_upload`	Upload a file to `src/public/`

File operations are sandboxed. Paths that escape the project directory are rejected. An AI assistant cannot read `/etc/passwd` or write outside your project.

Migrations

Tool	Description
`migration_status`	List pending and completed migrations
`migration_create`	Create a new migration file
`migration_run`	Run all pending migrations

Data and ORM

Tool	Description
`orm_describe`	List all ORM models with fields, types, and relationships
`seed_table`	Seed a table with fake data

Infrastructure

Tool	Description
`queue_status`	Queue size by status (pending, completed, failed)
`session_list`	Active sessions with data
`cache_stats`	Response cache hit/miss statistics

Debugging

Tool	Description
`log_tail`	Read recent log entries
`error_log`	Recent errors and exceptions with stack traces
`env_list`	Environment variables (sensitive values redacted)
`system_info`	Framework version, Ruby version, project info

The `env_list` tool redacts any variable containing "secret", "password", "token", "key", or "credential" in its name.

Project introspection

Tool	Description
`git_status`	Show current branch, modified/untracked files, and recent commits
`deps_list`	List the project's declared dependencies (from `Gemfile` / `*.gemspec`)
`project_overview`	One-shot snapshot: dependencies, routes, models, tables, errors, git

Persistent code index

Tool	Description
`index_rebuild`	Refresh the persistent project index (lazy, mtime-based)
`index_search`	Find files by path, symbol, route, or summary -- use FIRST for "where is X"
`index_file`	Full index entry for one file: symbols, routes, imports
`index_overview`	Project shape: files by language, routes, models, recent edits

Framework documentation (prose)

Tool	Description
`docs_list`	List framework documentation files
`docs_search`	Search Tina4 framework docs for a query string -- use before guessing
`docs_section`	Return a full markdown section from a framework doc file

Live API RAG (reflection)

`api_*` tools query the live framework reflection -- actual class/method signatures, file/line locations, with a `framework` vs `user` source tag. Use these for "what's the signature of X?" questions; `docs_*` is for "explain queues conceptually".

Tool	Description
`api_search`	Live API search -- finds framework + user classes/methods by name/summary

`api_class`	Live class reflection -- returns full method list and signatures for an FQN
`api_method`	Live method reflection -- returns signature, params, return type, file, line

Plan management

The `plan_*` family lets an AI assistant cooperate with a markdown plan file in `plan/`. Tina4 uses these to keep a steady record of in-progress refactors and multi-session work.

Tool	Description
`plan_current`	The active plan: title, steps (done/not), next step, progress
`plan_list`	All plans in `plan/` with progress and which one is active
`plan_create`	Create a new markdown plan and make it active
`plan_switch_to`	Make a different plan the active one
`plan_complete_step`	Tick a step as done (call the moment the step finishes)
`plan_add_step`	Append a new unchecked step to the current plan
`plan_note`	Append a timestamped note/breadcrumb to the current plan
`plan_archive`	Move a finished plan to `plan/done/` and clear the current pointer
`plan_read`	Full structured view of any plan by filename
`plan_flesh`	Auto-generate concrete build steps via qwen and append them to a plan

---

5. Protocol Details

The MCP server speaks JSON-RPC 2.0 over the current Streamable HTTP transport (protocol versions `2025-06-18` and `2025-03-26`). It still answers the legacy `2024-11-05` HTTP+SSE transport so older clients keep working. The server negotiates: it echoes the version a client asks for when it can speak it, and otherwise falls back to the newest one.

Streamable HTTP (current)

One endpoint carries the whole conversation:

```
POST /__dev/mcp
Content-Type: application/json
```

`initialize` mints a session and returns it in a header:

```
Mcp-Session-Id: 0f9a...c31
```

Send that header on every later request. A request bearing an unknown session gets a `404`, the client's cue to initialize again. A notification (a message with no `id`) returns `202` with an empty body. Every other request answers inline with the JSON-RPC response as `application/json`. `GET /__dev/mcp` returns `405` with an `Allow: POST, DELETE` header; `DELETE /__dev/mcp` ends the session.

Legacy HTTP+SSE (still supported)

```
GET /__dev/mcp/sse
Content-Type: text/event-stream
```

Returns the message endpoint URL as a server-sent event:

```
event: endpoint
data: http://localhost:7147/__dev/mcp/message
```

Then POST JSON-RPC messages to that endpoint:

```
POST /__dev/mcp/message
Content-Type: application/json
```

Messages

Both transports carry the same JSON-RPC 2.0 requests:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/list",
  "params": {}
}
```

Returns JSON-RPC 2.0 responses:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "tools": [
      {"name": "database_query", "description": "Execute a read-only SQL query", "inputSchema":
    ]
  }
}
```

Lifecycle

- Client sends `initialize` -- server responds with capabilities
- Client sends `notifications/initialized` -- server acknowledges

- Client calls `tools/list` -- server returns all registered tools with schemas
- Client calls `tools/call` with tool name and arguments -- server executes and returns results
- Client calls `resources/list` and `resources/read` for data resources

---

6. Troubleshooting

MCP server not starting:

- Check `TINA4_DEBUG=true` in `.env`
- Verify the server is running on localhost (not a remote host)

Claude Code not detecting the server:

- Restart Claude Code after starting the Tina4 server
- Check `.claude/settings.json` exists with the correct URL
- Verify the port matches your server's port

Tools returning errors:

- Database tools require a bound database: call `Tina4.bind_database(db)` (or set `TINA4_DATABASE_URL`) so `Tina4.database` resolves
- File tools are sandboxed to the project directory
- `database_execute` only works on localhost

Remote server, MCP disabled:

- This is intentional. Set `TINA4_MCP_REMOTE=true` only on staging environments you trust
- Never enable MCP on production servers

Building Custom MCP Servers

1. Beyond Dev Tools

Chapter 28 covered the built-in MCP server that ships with Tina4. It exposes framework internals for AI-assisted development. This chapter goes further: you build your own MCP servers that expose your application's business logic.

A CRM system exposes customer lookup. An accounting system exposes invoice queries. A warehouse system exposes inventory checks. Any domain logic that an AI assistant should access becomes an MCP tool.

---

2. Creating an MCP Server

Require `McpServer` and create an instance on any path:

```
require "tina4"

mcp = Tina4::McpServer.new("/api/my-tools", name: "My App Tools", version: "1.0.0")
```

The server registers HTTP endpoints at:

- `POST /api/my-tools` -- Streamable HTTP (the current transport): send JSON-RPC and read the response inline. `initialize` returns an `Mcp-Session-Id` header, `DELETE` ends the session.
- `POST /api/my-tools/message` -- legacy JSON-RPC message handler
- `GET /api/my-tools/sse` -- legacy SSE discovery handshake

Register it with the router in `app.rb` before `run`:

```
mcp.register_routes
```

---

3. Registering Tools with mcp_tool

The `Tina4.mcp_tool` method turns a block into an MCP tool. Parameter metadata becomes the input schema:

```
require "tina4"

mcp = Tina4::McpServer.new("/crm/mcp", name: "CRM Tools")

Tina4.mcp_tool("lookup_customer", description: "Find a customer by email", server: mcp,
  params: [{ name: "email", type: "string" }]) do |args|
  db.fetch_one("SELECT * FROM customers WHERE email = ?", [args["email"]])
end

Tina4.mcp_tool("recent_orders", description: "Get recent orders for a customer", server: mcp,
  params: [
    { name: "customer_id", type: "integer" },
```

```
    { name: "limit", type: "integer", default: 10 }
  ]) do |args|
  db.fetch(
    "SELECT * FROM orders WHERE customer_id = ? ORDER BY created_at DESC",
    [args["customer_id"]], limit: args["limit"] || 10
  ).to_a
end
```

The registration extracts:

- **Parameter names** from the params list
- **Types** from the type field (`"string"`, `"integer"`, `"boolean"`, etc.)
- **Required vs optional** -- parameters with defaults are optional
- **Description** from the `description` argument

An AI assistant sees these tools and their schemas:

```
{
  "name": "lookup_customer",
  "description": "Find a customer by email",
  "inputSchema": {
    "type": "object",
    "properties": {
      "email": {"type": "string"}
    },
    "required": ["email"]
  }
}
```

---

4. Registering Resources with mcp_resource

Resources are read-only data endpoints. They expose reference data that AI assistants can browse:

```
Tina4.mcp_resource("crm://product-catalog", description: "All active products", server: mcp) do
  db.fetch("SELECT id, name, price, category FROM products WHERE active = 1").to_a
end

Tina4.mcp_resource("crm://tax-rates", description: "Current tax rates by region", server: mcp) d
  db.fetch("SELECT region, rate FROM tax_rates").to_a
end
```

Resources are accessed via `resources/list` and `resources/read` in the MCP protocol.

---

5. Class-Based MCP Services

Group related tools into a service class. Register each method as a tool:

```
require "tina4"

mcp = Tina4::McpServer.new("/accounting/mcp", name: "Accounting Tools")

class AccountingService
```

```
  def initialize(db)
    @db = db
  end

  def lookup(invoice_no)
    @db.fetch_one("SELECT * FROM invoices WHERE invoice_no = ?", [invoice_no])
  end

  def balances(min_amount = 0.0)
    @db.fetch("SELECT * FROM invoices WHERE paid = 0 AND total >= ?", [min_amount]).to_a
  end

  def summary(year, month)
    @db.fetch_one(
      "SELECT SUM(total) as revenue, COUNT(*) as invoice_count " \
      "FROM invoices WHERE strftime('%Y', created_at) = ? " \
      "AND strftime('%m', created_at) = ?",
      [year.to_s, format("%02d", month)]
    )
  end
end

svc = AccountingService.new(db)

Tina4.mcp_tool("invoice_lookup", description: "Find an invoice by number", server: mcp,
  params: [{ name: "invoice_no", type: "string" }]) do |args|
  svc.lookup(args["invoice_no"])
end

Tina4.mcp_tool("outstanding_balances", description: "List all unpaid invoices", server: mcp,
  params: [{ name: "min_amount", type: "number", default: 0.0 }]) do |args|
  svc.balances(args["min_amount"] || 0.0)
end

Tina4.mcp_tool("monthly_summary", description: "Revenue summary for a month", server: mcp,
  params: [
    { name: "year", type: "integer" },
    { name: "month", type: "integer" }
  ]) do |args|
  svc.summary(args["year"], args["month"])
end
```

---

6. Securing MCP Endpoints

By default, developer MCP servers are public. Add authentication with a path-scoped `before` middleware that runs ahead of the MCP routes:

```
mcp.register_routes

# Require a valid bearer token on every request under the MCP path
Tina4::Middleware.before("#{mcp.path}/*") do |request, response|
  token = request.headers["Authorization"].to_s.sub(/^Bearer /, "")
  unless Tina4::Auth.valid_token(token)
    response.json({ error: "Unauthorized" }, 401)
  end
end
```

Or check the bearer token inside individual tools:

```
Tina4.mcp_tool("sensitive_data", description: "Access restricted data", server: mcp,
  params: [{ name: "token", type: "string" }]) do |args|
  payload = Tina4::Auth.valid_token(args["token"])
  unless payload
    next { error: "Unauthorized" }
  end
  db.fetch("SELECT * FROM sensitive_table").to_a
end
```

---

7. Testing MCP Tools

Test tool functions directly, or test via the MCP protocol:

```
# Test the tool block directly
def test_lookup_customer
  result = db.fetch_one("SELECT * FROM customers WHERE email = ?", ["alice@example.com"])
  assert result != nil
  assert_equal "alice@example.com", result["email"]
end

# Test via MCP protocol
def test_mcp_tool_call
  resp = JSON.parse(mcp.handle_message({
    "jsonrpc" => "2.0",
    "id" => 1,
    "method" => "tools/call",
    "params" => {
      "name" => "lookup_customer",
      "arguments" => { "email" => "alice@example.com" }
    }
  }))
  assert resp.key?("result")
  content = resp["result"]["content"][0]["text"]
  assert content.downcase.include?("alice")
end
```

---

8. Complete Example: CRM MCP Server

Here is a full working example -- a CRM system with customer, order, and product tools:

```
# app.rb
require "tina4"

db = Tina4::Database.new("sqlite:///crm.db")
Tina4.bind_database(db)   # models resolve Tina4.database; or set per-model with Model.db = db

# Create MCP server
crm_mcp = Tina4::McpServer.new("/crm/mcp", name: "CRM Assistant", version: "1.0.0")

# Tools
Tina4.mcp_tool("find_customer", description: "Search customers by name or email", server: crm_mc
  params: [{ name: "query", type: "string" }]) do |args|
```

```
  q = args["query"]
  db.fetch("SELECT * FROM customers WHERE name LIKE ? OR email LIKE ?",
           ["%#{q}%", "%#{q}%"]).to_a
end

Tina4.mcp_tool("customer_orders", description: "Get all orders for a customer", server: crm_mcp,
  params: [{ name: "customer_id", type: "integer" }]) do |args|
  db.fetch(
    "SELECT o.*, GROUP_CONCAT(oi.product_name) as items " \
    "FROM orders o LEFT JOIN order_items oi ON o.id = oi.order_id " \
    "WHERE o.customer_id = ? GROUP BY o.id ORDER BY o.created_at DESC",
    [args["customer_id"]]
  ).to_a
end

Tina4.mcp_tool("create_note", description: "Add a note to a customer record", server: crm_mcp,
  params: [
    { name: "customer_id", type: "integer" },
    { name: "note", type: "string" }
  ]) do |args|
  db.insert("customer_notes", { customer_id: args["customer_id"], note: args["note"] })
  { success: true }
end

# Resources
Tina4.mcp_resource("crm://products", description: "Product catalog", server: crm_mcp) do
  db.fetch("SELECT * FROM products WHERE active = 1").to_a
end

Tina4.mcp_resource("crm://stats", description: "CRM statistics", server: crm_mcp) do
  customers = db.fetch_one("SELECT COUNT(*) as count FROM customers")
  orders = db.fetch_one("SELECT COUNT(*) as count, SUM(total) as revenue FROM orders")
  {
    customers: customers["count"],
    orders: orders["count"],
    revenue: orders["revenue"]
  }
end

# Register routes
crm_mcp.register_routes

Tina4.run
```

Connect Claude Code to `http://localhost:7147/crm/mcp` (Streamable HTTP) and ask:

> *"Find all customers named Smith and show their recent orders"*

The AI calls `find_customer` with `query: "Smith"`, then `customer_orders` for each result. No custom API needed. The MCP protocol handles it.

---

9. Best Practices

- **One server per domain** -- CRM tools on `/crm/mcp`, accounting on `/accounting/mcp`
- **Keep tools focused** -- one query per tool, not a Swiss-army-knife tool

• **Use param metadata** -- types and defaults become the schema. An AI assistant cannot call a tool correctly without knowing the parameter types

• **Return structured data** -- hashes and arrays, not formatted strings. Let the AI format for the user

• **Secure production endpoints** -- add a path-scoped `Tina4::Middleware.before` token check (Section 6) for any MCP server that runs outside localhost

• **Test tools directly** -- call the Ruby method in your test suite, not just through the MCP protocol

Dev Tools

1. Debugging at 2am

2am. Production monitoring pings you. A 500 error on the checkout endpoint. You open the dev dashboard. Find the failing request in the request inspector. Full stack trace with source code context. Line 47 of `src/routes/checkout.rb` -- a nil reference on the shipping address because the user skipped the form. Add a nil check. Push the fix. Go back to sleep. 30 seconds.

Tina4's dev tools are not an afterthought. They ship with the framework from day one. Set `TINA4_DEBUG=true` and you get a full development dashboard, an error overlay with source code, live reload, a request inspector with replay, a SQL query runner, a queue monitor, system info, and a gallery of ready-to-use code examples -- all without installing a single extra gem.

---

2. Enabling the Dev Dashboard

Set `TINA4_DEBUG=true` in your `.env`:

```
TINA4_DEBUG=true
```

Restart your server and navigate to:

```
http://localhost:7147/__dev
```

No token or additional environment variables needed. The dashboard runs only when debug mode is on. Set `TINA4_DEBUG=false` in production and the entire dashboard disappears.

---

3. Dashboard Overview

The dev dashboard has several sections. Navigation tabs run along the top.

System Overview

The landing page shows at a glance:

- **Framework version** -- The installed Tina4 Ruby version
- **Ruby version** -- The running Ruby version and loaded gems
- **Uptime** -- How long the server has been running
- **Memory usage** -- Current and peak memory consumption
- **Database status** -- Connection status, database engine, file size (for SQLite)
- **Environment** -- Current `.env` variables (sensitive values masked)
- **Project structure** -- Directory listing of your project with file counts

Check here first when something feels off. Is the database connected? Is the right Ruby version running? Are the environment variables loaded?

---

4. The Dev Toolbar

Visit any HTML page in your application. A debug toolbar appears at the bottom of the page. Thin bar. Click it to expand.

The toolbar shows:

Field	What It Means
Request	HTTP method and URL of the current request
Status	HTTP response status code
Time	Total request processing time in milliseconds
DB	Number of database queries executed and total query time
Memory	Ruby memory used for this request
Template	The template rendered and how long rendering took
Session	Session ID and session data summary
Route	Which route handler matched this request

Click any section to expand it. Clicking "DB" shows every SQL query that ran during the request, with the query text, parameters, execution time, and row count.

Disabling the Toolbar

The toolbar hides when `TINA4_DEBUG=false`. You can also hide it for specific routes by returning a response with the `X-Debug-Toolbar: off` header:

```
Tina4::Router.get("/api/data") do |request, response|
  response.json({ data: "no toolbar" }, 200, { "X-Debug-Toolbar" => "off" })
end
```

This is useful for API endpoints that return JSON -- the toolbar only matters for HTML pages.

---

5. Error Overlay

An unhandled exception occurs. Tina4 does not show a generic "500 Internal Server Error" page. It shows a detailed error overlay:

- **Exception class and message** -- What went wrong, in plain language
- **Stack trace** -- Every method call that led to the error, from entry point to crash
- **Source code** -- The Ruby code around the line that threw the exception, with the failing line highlighted

- **Request data** -- HTTP method, URL, headers, body, and query parameters of the request that triggered the error
- **Environment** -- Relevant `.env` variables at the time of the error

Example Error

Write this handler:

```
Tina4::Router.get("/api/users/{user_id}") do |request, response|
  user_id = request.params["user_id"]
  user = get_user_from_db(user_id)
  response.json({ name: user.name, email: user.email })
end
```

If `get_user_from_db` returns `nil` for a missing user, the error overlay shows:

```
NoMethodError: undefined method 'name' for nil:NilClass

  File: src/routes/users.rb, line 4

  2 |   user_id = request.params["user_id"]
  3 |   user = get_user_from_db(user_id)
> 4 |   response.json({ name: user.name, email: user.email })
  5 | end

  Request: GET /api/users/999
  Headers: {"Accept": "application/json"}
```

The highlighted line makes the cause obvious. `user` is `nil` and the code calls `.name` on it.

Error Overlay in Production

When `TINA4_DEBUG=false`, the error overlay is disabled. Users see a clean error page. Full error details go to `logs/error.log` for later investigation. Create custom error pages:

- `src/templates/errors/404.html` -- Page not found
- `src/templates/errors/500.html` -- Internal server error

---

6. The Gallery

The gallery is a collection of ready-to-use code examples built into the dev dashboard. Each gallery item includes:

- A description of what it does
- The complete source code (routes, templates, models)
- A "Try It" button that installs the example into your project

Available gallery items:

- **JWT Authentication** -- Complete login/register flow with token management
- **CRUD API** -- Full REST API with ORM model
- **File Upload** -- Multipart form handling with file storage

- **WebSocket Chat** -- Real-time chat with rooms
- **Email Contact Form** -- Form with Messenger integration
- **Dashboard Template** -- Admin dashboard with tina4css

Click "Try It" on any gallery item. Tina4 creates the necessary files in your project. Modify them to fit your needs.

---

7. Live Reload

When `TINA4_DEBUG=true`, Tina4 watches your project files for changes and refreshes the browser automatically. Edit a route file. Save it. The browser refreshes with the new code. No manual restart required.

```
tina4 serve

  Tina4 Ruby v3.11.12
  HTTP server running at http://0.0.0.0:7147
  File watcher active - press Ctrl+C to stop
```

Live reload watches:

- `src/**` -- Routes, ORM, middleware, templates, SCSS
- `migrations/` -- SQL migrations
- `.env` -- Environment variables

How It Works

The `tina4` Rust CLI is the sole file watcher for the whole Tina4 stack. There is no framework-side watcher (the `listen`-gem based `dev_reload.rb` was removed in 3.11.x).



- The CLI watches `src/`, `migrations/`, `.env` with the `notify` crate. Events are filtered to real source changes: metadata/access events, `__pycache__`, `.git`, `node_modules`, `vendor`, `logs`, `.log`/`.db*`/`.swp` files are ignored. A real mtime check defeats overlayfs / polling-mode spurious events (Podman, distrobox).
- On a real change, the CLI POSTs `/__dev/api/reload` to the running Ruby server. The server keeps running.
- `Tina4::DevAdmin` bumps its `@reload_mtime` counter and broadcasts `{type: "reload"}` over WebSocket at `/__dev_reload`. `GET /__dev/api/mtime` returns the counter for browsers using the polling fallback.
- The dev-toolbar script reloads the browser. SCSS/CSS changes are signalled as `type: "css"` and swap the stylesheet without a full reload.

The reload happens in under a second. Edit code. Switch to the browser. The changes are already there.

---

8. Request Inspector

The request inspector records every HTTP request to your application. For each request:

- **Timestamp** -- When the request arrived
- **Method** -- GET, POST, PUT, DELETE
- **URL** -- The full URL including query parameters
- **Status** -- The HTTP status code returned (color-coded: green for 2xx, yellow for 4xx, red for 5xx)
- **Time** -- How long the request took to process
- **Request ID** -- A unique identifier for correlating logs

Click on any request to see its full details.

Request Details Panel

- **Headers** -- All request headers (Accept, Content-Type, Authorization)
- **Body** -- The request body (for POST/PUT/PATCH), formatted as JSON if applicable
- **Query parameters** -- Parsed URL query parameters
- **Route match** -- Which route definition matched this request
- **Middleware** -- Which middleware ran and how long each took
- **Database queries** -- Every SQL query executed during this request, with timing
- **Template renders** -- Which templates rendered and how long each took
- **Response headers** -- The response headers sent back
- **Response body** -- The first 1000 characters of the response body

Filtering Requests

The inspector supports filtering:

- **By status**: Click the status code badges at the top (e.g., show only 5xx errors)
- **By method**: Filter by GET, POST, PUT, DELETE
- **By path**: Search for a URL pattern (e.g., `/api/` for API requests only)
- **By time range**: Show requests from the last 5 minutes, 1 hour, or all time

Request Replay

Click "Replay" on any request to re-send it. The inspector fires the same method, URL, headers, and body. You reproduce an error without constructing the curl command by hand.

This is the fastest path from "what happened?" to "I can see it happen again." Find a failing request. Hit Replay. Watch the error overlay show the stack trace. Fix the code. Replay again. Green status code. Done.

No more `puts` statements scattered through your code. The inspector shows what happened. Every request. Every detail. With one-click replay.

---

9. SQL Query Runner

The dev dashboard includes a SQL query runner. Execute queries against your database:

- **Execute queries** -- Run any SQL statement
- **See results** -- Formatted table with column headers and row numbers
- **View query timing** -- How long each query took
- **Browse tables** -- A sidebar lists all tables with their column definitions
- **Export results** -- Download as CSV

```
SELECT * FROM products WHERE category = 'Electronics' ORDER BY price DESC;
```

The results display in a table with column headers, row numbers, and data type indicators. Copy results, export as CSV, or run another query.

Safety

The query runner is read-write in development. You can run INSERT, UPDATE, and DELETE statements. Be careful -- there is no undo. In shared environments, consider a read-only database connection for the dev dashboard.

The query runner only works when `TINA4_DEBUG=true`. It is disabled in production.

---

10. Queue Monitor

If your application uses background job queues (Chapter 12 -- Queues), the dev dashboard includes a queue monitor. The monitor shows five categories:

- **Pending jobs** -- Jobs waiting to be processed
- **Active jobs** -- Jobs currently being processed
- **Completed jobs** -- Recently completed jobs with timing
- **Failed jobs** -- Jobs that threw exceptions, with error details
- **Dead-letter queue** -- Jobs that failed too many times

For each job, you see:

- The job method name
- The payload (serialized arguments)
- When it was enqueued
- When it started processing (if active)
- The error message and stack trace (if failed)

- How many times it has been retried

You can also take action:

- **Retry a failed job** -- Click "Retry" to move it back to the pending queue
- **Delete a job** -- Remove it from any queue
- **Pause/resume the queue** -- Stop processing without losing jobs

The queue monitor gives you a window into your background work. A job fails. You see the error. You fix the code. You hit Retry. The job processes. No log file hunting. No guesswork about what payload caused the failure.

---

11. Log Viewer

View application logs in real time from the dashboard:

```
http://localhost:7147/__dev#logs
```

Filter by level (DEBUG, INFO, WARNING, ERROR), search by keyword, and view timestamps. In development, all log levels are shown. In production with `TINA4_LOG_LEVEL=WARNING`, only warnings and errors appear.

Logging from Code

```
Tina4::Log.debug("Processing order #{order_id}")
Tina4::Log.info("User #{user_id} logged in")
Tina4::Log.warning("Rate limit approaching for IP #{ip}")
Tina4::Log.error("Failed to send email: #{error.message}")
```

Logs are written to `logs/app.log` and displayed in the dev dashboard.

---

12. System Info

The System Info tab shows detailed information about your environment:

- **Ruby Configuration** -- Version, installed gems, GEM_HOME path, memory limit
- **Database Info** -- Engine, version, connection details, table sizes, index information
- **Server Info** -- OS, hostname, server software, document root
- **Tina4 Config** -- All loaded `.env` variables (sensitive values masked), auto-discovered routes, registered middleware, ORM models
- **Disk Usage** -- Size of your project directory, data directory, logs directory

This panel solves the "it works on my machine" problem. Compare the System Info output between two environments. Spot the difference. The wrong Ruby version. A missing gem. A misconfigured environment variable. The answer is in the panel.

---

13. Health Check Endpoint

The `/health` endpoint reports application status:

```
curl http://localhost:7147/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3600,
  "version": "3.0.0",
  "framework": "tina4-ruby",
  "ruby_version": "3.3.0",
  "memory_mb": 42.5,
  "pid": 12345
}
```

In production with `TINA4_DEBUG=false`, create a `.broken` file in the project root to make the health check return a failure status. This is useful for graceful shutdown: set the file, wait for the load balancer to stop sending traffic, then restart the server.

---

14. Exercise: Debug a Broken Endpoint

Create a route with an intentional bug and use the dev tools to find and fix it.

Setup

Create `src/routes/buggy.rb`:

```
Tina4::Router.get("/api/buggy/users") do |request, response|
  users = Tina4.cache_get("all_users")
  if users
    return response.json({ users: users, source: "cache" })
  end

  # Bug 1: This query has an error
  db = Tina4.database
  users = db.fetch("SELCT * FROM users ORDER BY name")

  Tina4.cache_set("all_users", users, 300)
  response.json({ users: users, source: "database" })
end

Tina4::Router.post("/api/buggy/users") do |request, response|
  body = request.body

  # Bug 2: Missing validation
  user = User.new
  user.name = body["name"]
  user.email = body["email"]
  user.save

  # Bug 3: Wrong status code
  response.json({ message: "User created", user: user.to_hash })
end
```

Requirements

- Open the dev dashboard at `http://localhost:7147/__dev`
- Hit the `GET /api/buggy/users` endpoint and find Bug 1 using the error overlay
- Hit the `POST /api/buggy/users` endpoint without a body and find Bug 2 using the request inspector
- Fix all three bugs:
- Bug 1: Fix the SQL syntax error
- Bug 2: Add validation for required fields
- Bug 3: Return 201 instead of 200
- Use Request Replay to verify each fix

---

15. Solution

```
Tina4::Router.get("/api/buggy/users") do |request, response|
  users = Tina4.cache_get("all_users")
  if users
    return response.json({ users: users, source: "cache" })
  end

  db = Tina4.database
  users = db.fetch("SELECT * FROM users ORDER BY name")  # Fixed: SELCT -> SELECT

  Tina4.cache_set("all_users", users, 300)
  response.json({ users: users, source: "database" })
end

Tina4::Router.post("/api/buggy/users") do |request, response|
  body = request.body

  # Fixed: Added validation
  if body.nil? || body["name"].nil? || body["email"].nil?
    return response.json({ error: "Name and email are required" }, 400)
  end

  user = User.new
  user.name = body["name"]
  user.email = body["email"]
  user.save

  response.json({ message: "User created", user: user.to_hash }, 201)  # Fixed: 201
end
```

The dev tools made each bug visible. The error overlay showed the SQL syntax error with the exact line. The request inspector showed the nil body causing a `NoMethodError`. Request Replay let you re-send the fixed requests without leaving the dashboard.

---

16. Gotchas

1. Dev Dashboard Accessible on Network

Problem: Anyone on your network can access the dev dashboard.

Cause: `TINA4_DEBUG=true` makes the dashboard available at `/__dev`.

Fix: In production, set `TINA4_DEBUG=false` to disable the dashboard. In shared development environments, restrict network access.

2. Live Reload Causes Connection Drops

Problem: WebSocket connections drop every time you save a file.

Cause: The server restarts on file change, which closes all active connections.

Fix: Save files less often during WebSocket development, or use a separate terminal to run the WebSocket test client that auto-reconnects. The frond.js WebSocket client has built-in auto-reconnect.

3. Error Overlay Shows in Production

Problem: Users see Ruby stack traces when errors occur.

Cause: `TINA4_DEBUG=true` is set in the production environment.

Fix: Set `TINA4_DEBUG=false` in production. The error overlay is replaced by a clean error page. Full details go to `logs/error.log`.

4. Request Inspector Slows Down the Server

Problem: The server becomes slower as it runs longer.

Cause: The request inspector stores every request in memory. After thousands of requests, memory usage grows.

Fix: The inspector limits storage to the last 1000 requests. If you notice slowness, clear the inspector from the dashboard. In production, the inspector is disabled.

5. SQL Runner Executes Destructive Queries

Problem: Someone ran `DROP TABLE users` in the SQL query runner.

Cause: The SQL runner executes any valid SQL query. There is no confirmation step.

Fix: The SQL runner only works when `TINA4_DEBUG=true`. Never leave debug mode on in production. For sensitive development databases, use a read-only database connection for the SQL runner.

6. Template Changes Not Reflected

Problem: You edited a template but the page still shows the old version.

Cause: Template caching is enabled (`TINA4_TEMPLATE_CACHE_TTL=true`). The compiled template serves from cache.

Fix: In development, set `TINA4_TEMPLATE_CACHE_TTL=false` (this is the default when `TINA4_DEBUG=true`). If you enabled template caching manually, disable it for development.

7. Log Files Growing Without Bound

Problem: `logs/app.log` grows to gigabytes over time.

Cause: No log rotation is configured.

Fix: Set `TINA4_LOG_MAX_SIZE=10mb` and `TINA4_LOG_KEEP=5` in `.env` to rotate logs automatically.

8. Queue Monitor Shows No Jobs

Problem: The Queue Monitor tab is empty even though your application uses queues.

Cause: The queue system is not running. Jobs are enqueued but no worker processes them.

Fix: Start a queue worker: `tina4 queue work`. The queue monitor reflects the state of the queue storage (database or Redis). If no worker is running, jobs sit in "Pending" and never move to "Active" or "Completed."

9. Memory Usage Grows During Development

Problem: The server's memory usage increases over time during development.

Cause: Each live reload may not fully release old objects from the Ruby garbage collector.

Fix: This is normal during active development. Restart the server periodically during long development sessions. Ruby's GC will clean up on restart.

Tina4 CLI

1. Getting a New Developer Up to Speed

Monday morning. New developer joins the team. You hand them the repo URL. By 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line. No documentation hunts. No boilerplate from old projects. The CLI handles the scaffolding.

The Tina4 CLI is a single Rust binary. It manages all four Tina4 frameworks -- PHP, Python, Ruby, Node.js. Commands are identical across languages. Learn the CLI for Ruby, and you know it for PHP.

---

2. tina4 init -- Project Scaffolding

You saw this in Chapter 1. Now the details.

```
tina4 init my-project
```

```
Creating Tina4 project in ./my-project ...
  Detected language: Ruby (Gemfile)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-project
  bundle install
  tina4 serve
```

Language Detection

The CLI detects the language from existing files:

File Present	Language

`Gemfile`	Ruby
`composer.json`	PHP
`pyproject.toml` or `requirements.txt`	Python
`package.json`	Node.js

If no language-specific file exists, the CLI asks:

```
tina4 init my-project

No language detected. Which language?
  1. PHP
  2. Python
  3. Ruby
  4. Node.js
> 3

Creating Tina4 Ruby project in ./my-project ...
```

Explicit Language Selection

Skip the prompt by specifying the language:

```
tina4 init my-project --lang ruby
```

This creates a Ruby project with `Gemfile`, `app.rb`, and the full directory structure.

Init into an Existing Directory

Already have a project? Add Tina4 structure:

```
cd existing-project
tina4 init .
```

The CLI creates only files and directories that do not exist. It never overwrites.

---

3. tina4 serve -- Dev Server

```
tina4 serve

  Tina4 Ruby v3.0.0
  HTTP server running at http://0.0.0.0:7147
  WebSocket server running at ws://0.0.0.0:7147
  Live reload enabled
  Press Ctrl+C to stop
```

`tina4 serve` detects the language and starts the appropriate server. For Ruby, it spawns the Tina4 Ruby runtime with SCSS compilation, file watching, and live reload all wired in.

Options

```
tina4 serve --port 8080        # Custom port
tina4 serve -p 8080            # Same thing, short form
tina4 serve --host 127.0.0.1   # Bind to localhost only
tina4 serve --no-browser       # Don't open a browser tab on startup
tina4 serve --no-reload        # Don't reload the browser when files change
tina4 serve --dev              # Force the dev server even if a production server is installed
tina4 serve --production       # Production server, debug off, no live reload
```

Serving a Project by Name

You don't have to `cd` into a project first. Pass its folder name and the CLI will find it, change into it, and serve it:

```
tina4 serve my-app             # ./my-app, or my-app in your projects folder
tina4 serve my-app -p 7150     # Options still apply
```

The CLI looks for `./my-app` in the current folder first, then in the projects folder `tina4 setup` recorded. If neither has it, it stops with `Project 'my-app' not found` and names the folder it searched. Run a bare `tina4 serve` outside any project and it will look in the projects folder too: one project there gets served, and several get listed so you can pick one by name.

Bypassing the CLI

The framework refuses to start without the Rust CLI. For sandboxed environments (Docker images, CI runners, APM agents) set `TINA4_OVERRIDE_CLIENT=true` in `.env` and only then run:

```
TINA4_OVERRIDE_CLIENT=true bundle exec ruby app.rb
```

This bypasses SCSS compilation and live reload. For production deployments use `tina4 serve --production` (or set the override and run Puma directly):

```
TINA4_OVERRIDE_CLIENT=true bundle exec puma -C config/puma.rb
```

---

4. tina4 generate model -- ORM Scaffolding

The `generate model` command creates an ORM model file and a matching migration. One command produces both.

```
tina4 generate model Product
```

```
Created src/orm/product.rb
Created migrations/20260322120000_create_products_table.sql
```

The generated model:

```
class Product < Tina4::ORM
  integer_field :id, primary_key: true
  created_at: true

  table_name "products"
end
```

The generated migration:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

The model is ready to use. Add your fields and run migrations.

Adding Fields

Specify fields on the command line:

```
tina4 generate model Product --fields "name:string,price:float,category:string,in_stock:bool"
```

The generated model includes all the fields:

```
class Product < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :name
  float_field :price
  string_field :category
  boolean_field :in_stock

  table_name "products"
end
```

And the migration:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL DEFAULT '',
    price REAL NOT NULL DEFAULT 0,
    category TEXT NOT NULL DEFAULT '',
    in_stock INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

Field Types

CLI Type	Ruby Field	SQLite Column
`string`	`string_field`	`TEXT`
`int`	`integer_field`	`INTEGER`
`float`	`float_field`	`REAL`
`bool`	`boolean_field`	`INTEGER`
`text`	`text_field`	`TEXT`
`date`	`string_field`	`TEXT`

Options

Flag	Description	Example
`--fields`	Comma-separated field definitions	`--fields "name:string,price:float"`
`--auto-crud`	Enable auto-CRUD on the model	`--auto-crud`
`--soft-delete`	Add soft delete support	`--soft-delete`
`--no-migration`	Skip migration generation	`--no-migration`
`--with-route`	Also generate a CRUD route file	`--with-route`

---

5. tina4 generate route -- CRUD Route Scaffolding

The `generate route` command creates a complete CRUD route file with all five REST endpoints. It reads the model's properties and builds routes with proper type casting and nil checks.

```
tina4 generate route products
```

```
Created src/routes/products.rb
```

The generated route file:

```
Tina4::Router.get("/api/products") do |request, response|
  page = (request.params["page"] || 1).to_i
  per_page = (request.params["per_page"] || 20).to_i
  offset = (page - 1) * per_page

  products, total = Product.where("1=1", [], limit: per_page, offset: offset)
  results = products.map(&:to_hash)

  response.json({
    data: results,
    page: page,
    per_page: per_page,
    count: results.length
  })
end

Tina4::Router.get("/api/products/{product_id:int}") do |request, response|
  product = Product.find(request.params["product_id"])

  if product.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  response.json(product.to_hash)
end
```

```
Tina4::Router.post("/api/products") do |request, response|
  body = request.body

  product = Product.new
  product.name = body["name"] || ""
  product.price = (body["price"] || 0).to_f
  product.category = body["category"] || ""
  product.in_stock = body["in_stock"] ? true : false
  product.save

  response.json(product.to_hash, 201)
end

Tina4::Router.put("/api/products/{product_id:int}") do |request, response|
  product = Product.find(request.params["product_id"])

  if product.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  body = request.body
  product.name = body["name"] if body.key?("name")
  product.price = body["price"].to_f if body.key?("price")
  product.category = body["category"] if body.key?("category")
  product.in_stock = body["in_stock"] if body.key?("in_stock")
  product.save

  response.json(product.to_hash)
end

Tina4::Router.delete("/api/products/{product_id:int}") do |request, response|
  product = Product.find(request.params["product_id"])

  if product.nil?
    return response.json({ error: "Product not found" }, 404)
  end

  product.delete
  response.json(nil, 204)
end
```

The generator reads the model's properties and creates routes with type casting and nil checks. Customize the generated code. It is regular Ruby, not magic.

Options

Flag	Description	Example
`--prefix`	Custom route prefix (default: `/api`)	`--prefix /api/v2`
`--middleware`	Add middleware to all routes	`--middleware auth_middleware`

---

6. tina4 generate migration -- Migration Scaffolding

The `generate migration` command creates a timestamped migration file with `UP` and `DOWN` sections. The timestamp ensures migrations run in order.

```
tina4 generate migration add_category_to_products

Created migrations/20260322120500_add_category_to_products.sql
```

The generated file:

```
-- UP
-- Add your forward migration SQL here


-- DOWN
-- Add your rollback migration SQL here
```

Fill in the SQL:

```
-- UP
ALTER TABLE products ADD COLUMN category TEXT DEFAULT '';

-- DOWN
ALTER TABLE products DROP COLUMN category;
```

The timestamp prefix (`20260322120500`) ensures migrations run in order. Each migration runs once. The framework tracks which ones have been applied.

---

7. tina4 generate middleware -- Middleware Scaffolding

The `generate middleware` command creates a middleware file with the correct signature.

```
tina4 generate:middleware rate_limit

Created src/middleware/rate_limit.rb
```

The generated file:

```
# Rate limit middleware
Tina4.middleware do |request, response, next_handler|
  # Add your middleware logic here

  # Continue to the next middleware or route handler
  next_handler.call(request, response)

  # Or return early to block the request:
  # response.json({ error: "Rate limit exceeded" }, 429)
end
```

Reference the middleware in route definitions:

```
Tina4::Router.get("/api/data", middleware: ["rate_limit"]) do |request, response|
  response.json({ data: "protected" })
end
```

---

8. Generate All at Once

Combine flags to generate multiple files in a single command:

```
tina4 generate:crud Product
```

```
Created src/orm/product.rb
Created src/routes/products.rb
Created migrations/20260322120000_create_products_table.sql
```

Model. CRUD routes. Migration. All wired together. Ready to use.

---

9. tina4 doctor -- Health Check

The `doctor` command checks your project for common issues:

```
tina4 doctor
```

```
Tina4 Doctor -- Checking your project...

  [OK] Ruby 3.3.0 detected
  [OK] Bundler found
  [OK] tina4 gem installed (v3.0.0)
  [OK] .env file exists
  [OK] Database connection: sqlite:///data/app.db
  [OK] Database is accessible
  [OK] src/routes/ directory exists (3 route files)
  [OK] src/orm/ directory exists (2 model files)
  [OK] src/templates/ directory exists (5 templates)
  [OK] src/public/ directory exists (static files served)
  [OK] tests/ directory exists (4 test files)
  [WARN] No migrations found in migrations/
  [OK] AI context: Claude Code detected, CLAUDE.md present
  [OK] .gitignore includes .env, data/, logs/

  12 checks passed, 1 warning, 0 errors
```

Doctor checks:

- Ruby version and package manager
- Tina4 gem installation and version
- `.env` file existence and critical variables
- Database connectivity
- Directory structure
- Missing files or configurations
- AI tool context files
- Git configuration

The warnings give actionable advice. If your database is not configured, it tells you exactly what to add to `.env`.

---

10. tina4 test -- Running Tests

```
tina4 test

Running tests...

  ProductSpec
    [PASS] creates a product
    [PASS] loads a product

  2 tests, 2 passed, 0 failed (0.12s)
```

This runs all tests in the `tests/` directory. See Chapter 18 for full testing documentation.

Test Options

```
tina4 test tests/product_spec.rb            # Specific file
tina4 test --verbose                         # Verbose output
tina4 test --coverage                        # Generate coverage report
```

---

11. tina4 routes -- Route Listing

See all registered routes in your project:

```
tina4 routes

Registered Routes:

  Method  Path                        Handler              Middleware        Auth
  ------  --------------------------  -------------------  ----------------  --------
  GET     /health                     health_check         -                 public
  GET     /api/products               list_products        ResponseCache     public
  GET     /api/products/{id:int}      get_product          -                 public
  POST    /api/products               create_product       auth_middleware   secured
  PUT     /api/products/{id:int}      update_product       auth_middleware   secured
  DELETE  /api/products/{id:int}      delete_product       auth_middleware   secured
  GET     /admin                      admin_dashboard      -                 public

  7 routes registered
```

This is useful for verifying that your routes are registered and for finding the handler for a specific URL.

Filtering

```
tina4 routes --method POST          # Filter by HTTP method
tina4 routes --filter products      # Filter by path pattern
tina4 routes --middleware auth      # Filter by middleware
```

When debugging routing issues, check here first. If a route does not match, `tina4 routes` shows whether it was registered and what middleware is attached.

---

12. tina4 migrate -- Database Migrations

Run pending migrations:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322000100_create_users_table.sql
  [UP] 20260322000200_create_products_table.sql
  [UP] 20260322000300_add_category_to_products.sql

  3 migrations applied
```

Rollback

```
tina4 migrate --rollback
```

```
Rolling back last migration...
  [DOWN] 20260322000300_add_category_to_products.sql

  1 migration rolled back
```

Migration Table Auto-Upgrade

If your project was created with an early v3 release, the `tina4_migration` tracking table may use the older two-column schema. Running `tina4 migrate` detects the old layout and adds the missing `migration_id`, `batch`, and `executed_at` columns, backfilling existing data. No manual intervention needed.

Status

```
tina4 migrate:status
```

```
Migration Status:

  Status    Migration
  --------  ----------------------------------------
  Applied   20260322000100_create_users_table.sql
  Applied   20260322000200_create_products_table.sql
  Applied   20260322000300_add_category_to_products.sql
  Pending   20260322000400_create_orders_table.sql

  3 applied, 1 pending
```

Other Migration Commands

```
tina4 migrate:reset        # Roll back all migrations
tina4 migrate:fresh        # Roll back and re-run all migrations
```

---

13. tina4 queue -- Queue Management

```
tina4 queue work                    # Start processing all queues
tina4 queue work --queue emails     # Process specific queue
tina4 queue:dead                    # View dead letter queue
tina4 queue:retry 42                # Retry a dead letter job
tina4 queue:retry --all             # Retry all dead letter jobs
tina4 queue:clear --older-than 7d   # Clear old dead letter jobs
tina4 queue:stats                   # Show queue statistics
```

---

14. tina4 build -- Build Commands

```
tina4 scss                      # Compile SCSS to CSS
tina4 build                       # Bundle and minify JavaScript
tina4 build                          # Run all build steps
```

---

15. tina4 deploy -- Deployment

```
tina4 deploy docker                 # Generate Dockerfile and docker-compose.yml
tina4 deploy:systemd                # Generate systemd service file
tina4 deploy:nginx                  # Generate Nginx config
```

---

16. Environment-Specific Commands

The CLI has no `--env` flag. Tina4 Ruby picks the environment file itself when it loads, so you choose it with an environment variable in front of the command:

```
ENVIRONMENT=production tina4 serve      # Loads .env.production (falls back to .env if it's mis
ENVIRONMENT=test tina4 migrate          # Runs migrations against the settings in .env.test
TINA4_ENV_FILE=.env.staging tina4 serve  # Loads exactly this file
```

`TINA4_ENV_FILE` wins over `ENVIRONMENT`. A variable already set in the real process environment is never overwritten by any of these files.

---

17. Exercise: Scaffold a Feature in 5 Commands

Scaffold a complete "Customer" feature from scratch using only CLI commands.

Requirements

Starting from an existing Tina4 Ruby project, run 5 commands to create:

- A Customer ORM model with name, email, phone, and company fields
- A CRUD route file with all five REST endpoints
- A migration that creates the customers table
- Run the migration to create the table

- Run the doctor to verify everything

Expected Commands

```
# 1. Generate the model with route and migration
tina4 generate:crud Customer

# 2. Edit the model to add fields (manual step)
# Add fields to src/orm/customer.rb

# 3. Edit the migration to add columns (manual step)
# Add columns to the migration file

# 4. Run the migration
tina4 migrate

# 5. Run the doctor to verify everything is set up
tina4 doctor
```

---

18. Solution

Command 1: Generate everything:

```
tina4 generate:crud Customer
```

Command 2: Edit `src/orm/customer.rb`:

```
class Customer < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :name
  string_field :email
  string_field :phone
  string_field :company

  table_name "customers"
end
```

Command 3: Edit the migration file:

```
-- UP
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    phone TEXT,
    company TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_customers_email ON customers(email);

-- DOWN
DROP TABLE IF EXISTS customers;
```

Command 4: Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322120000_create_customers_table.sql

  1 migration applied
```

Command 5: Verify with doctor:

```
tina4 doctor
```

```
  [OK] src/orm/ directory exists (3 model files)
  [OK] src/routes/ directory exists (4 route files)
  [OK] Database connection: sqlite:///data/app.db
  ...
```

Now test the API:

```
curl -X POST http://localhost:7147/api/customers \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Corp", "email": "alice@corp.com", "phone": "+1-555-0100", "company": "Alic
```

```
{
  "id": 1,
  "name": "Alice Corp",
  "email": "alice@corp.com",
  "phone": "+1-555-0100",
  "company": "Alice Corp",
  "created_at": "2026-03-22 12:00:00"
}
```

From zero to a working CRUD API. Five commands. Under two minutes.

---

19. Gotchas

1. tina4 Command Not Found

Problem: Running `tina4` gives "command not found".

Cause: The Tina4 CLI is not installed or not in your PATH.

Fix: Install the CLI: `curl -fsSL https://tina4.com/install.sh | sh`. Verify with `tina4 --version`. If installed but not found, add the installation directory to your PATH:

```
echo 'export PATH="$HOME/.tina4/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

2. Wrong Language Detected

Problem: `tina4 init` creates a PHP project instead of Ruby.

Cause: A `composer.json` file exists in the directory from a previous project.

Fix: Use explicit language selection: `tina4 init my-project --lang ruby`. Or delete the conflicting language file before running `init`.

3. Generated Files Overwrite Existing Code

Problem: Running `tina4 generate route products` overwrites your custom route file.

Cause: The generate command creates files at fixed paths.

Fix: The CLI warns you before overwriting. Check if the file exists first. If you need to regenerate, rename the existing file: `mv src/routes/products.rb src/routes/products_backup.rb`.

4. Migration Order Issues

Problem: A migration fails because it references a table that does not exist yet.

Cause: Migration files run in alphabetical (timestamp) order. If migration B depends on a table created by migration A, but A has a later timestamp, B runs first and fails.

Fix: Use consistent timestamps. The `tina4 generate migration` command uses the current timestamp. Generate migrations in order to ensure correct execution. If you need to fix ordering, rename the migration files to adjust their timestamps.

5. tina4 serve Uses Wrong Port

Problem: `tina4 serve` starts on port 7147 but you need port 8080.

Cause: The default port for Ruby is 7147 unless overridden.

Fix: Set it in `.env`: `TINA4_PORT=8080`. Or pass it as a flag: `tina4 serve --port 8080`. The `.env` value takes precedence over the default. The command-line flag overrides everything.

6. Doctor Shows False Warnings

Problem: `tina4 doctor` warns about missing migrations, but your project does not use migrations.

Cause: Doctor checks for common conventions. It warns about missing migrations regardless of your approach.

Fix: These are warnings, not errors. Ignore warnings that do not apply to your project. Doctor is a guide, not a gatekeeper.

7. Model Name Must Be PascalCase

Problem: `tina4 generate model order_item` creates a model class named `order_item` which is not valid Ruby convention.

Cause: The CLI uses the argument as-is for the class name.

Fix: Use PascalCase for model names: `tina4 generate model OrderItem`. The CLI converts it to snake_case for the table name (`order_items`).

8. build:css Fails

Problem: `tina4 scss` fails with "sass not found".

Cause: The Sass compiler is not installed.

Fix: Install it: `gem install sass` or `npm install -g sass`.

---

9. deploy:docker Generates Wrong Dockerfile

Problem: The generated Dockerfile uses PHP instead of Ruby.

Cause: The CLI cannot detect the language without a `Gemfile`.

Fix: Make sure your project has a `Gemfile` so the CLI detects Ruby. Or use `tina4 deploy docker --lang ruby`.

---

20. Documentation

```
tina4 docs      # Download framework-specific book chapters to .tina4-docs/
tina4 books     # Download the complete Tina4 book (all languages) to tina4-book/
```

`tina4 docs` detects your project language and downloads only the relevant chapters. The documentation is available in Markdown format, optimised for AI tools and local reference.

---

21. Test Port (Dual-Port Development)

When `TINA4_DEBUG=true`, Tina4 automatically starts a second HTTP server on `port + 1000`:

- **Main port** (e.g. 7147) - hot-reload enabled, for AI dev tools
- **Test port** (e.g. 8147) - stable, no hot-reload, for user testing

This prevents the browser from refreshing mid-test when AI tools edit files.

Setting	Effect
`TINA4_NO_AI_PORT=true`	Disables the test port entirely
`TINA4_NO_RELOAD=true`	Disables hot-reload on the main port too
`--no-reload`	CLI flag equivalent of TINA4*NO*RELOAD

Vibe Coding with AI

Why Tina4 is Built for AI-Assisted Development

Tell your AI assistant: "Add a product catalog with search, pagination, and category filtering." In most frameworks, the AI needs to know which packages to install, which config files to create, which naming conventions to follow, how to wire everything together. It hallucinated half of it.

With Tina4, the AI reads one file -- `CLAUDE.md` -- and knows everything. Every method signature. Every import path. Every convention. It generates correct, runnable code on the first attempt because there is one way to do things in Tina4.

This is not accidental. Tina4 was designed from the ground up for AI-assisted development. Here is why.

This chapter explains coding assistants and the `tina4 ai` skills installer. To call a model from your application, see Chapter 39: AI Client.

The Zero-Dependency Advantage

When an AI writes Rails code, it might suggest `ActiveRecord::Base` or `ApplicationRecord`. Both exist. Both behave differently. It might suggest `render json:` or `respond_to do |format|`. The AI guesses.

Tina4 gives it nothing to guess about. One ORM. One queue. One template engine. One cache. No ambiguity. No alternatives. No "it depends on which gem you installed." The AI knows.

```
# There is only one way to cache in Tina4
cached = Tina4.cache_get("products") || Tina4.cache_set("products", db.fetch("SELECT * FROM prod

# There is only one way to queue in Tina4
Tina4::Queue.produce("emails", { to: "user@test.com", subject: "Welcome" })

# There is only one way to send email in Tina4
mail = Tina4::Messenger.new
mail.to = "user@test.com"
mail.subject = "Welcome"
mail.send
```

Zero dependencies means zero confusion for AI.

CLAUDE.md -- The AI's Instruction Manual

Every Tina4 project includes a `CLAUDE.md` file that tells AI assistants exactly how the framework works. It contains:

- Every method signature with parameters and return types
- The project structure convention (`src/routes/`, `src/orm/`, `src/templates/`)

- The `.env` variable reference
- Code style rules (routes use `response.json`, ORM uses `to_h`)
- Database connection string formats
- Common gotchas and how to avoid them

When you open a Tina4 project in Claude Code, Cursor, or GitHub Copilot -- the AI reads this file first and immediately understands how to write correct Tina4 code.

Skills -- Deep Framework Knowledge

Beyond CLAUDE.md, Tina4 ships with three AI skill files:

tina4-maintainer

For framework maintenance -- porting features between languages, reviewing PRs, running benchmarks, checking parity.

tina4-developer

For application development -- creating routes, defining models, writing templates, setting up auth, configuring queues.

tina4-js

For frontend development -- tina4-js signals, Tina4Element components, reactive templates, WebSocket client.

Supported AI Tools

Tina4 auto-detects and installs context for eight tools. The exact list lives in the framework's `Tina4::AI` module:

Tool	Detection (context file)	Context installed
Claude Code	`CLAUDE.md`	CLAUDE.md + `.claude/` skills
Cursor	`.cursorules`	`.cursorules` (+ `.cursor/`)
GitHub Copilot	`.github/copilot-instructions.md`	`.github/copilot-instructions.md`
Windsurf	`.windsurfrules`	`.windsurfrules`
Aider	`CONVENTIONS.md`	`CONVENTIONS.md`
Cline	`.clinerules`	`.clinerules`
OpenAI Codex	`AGENTS.md`	`AGENTS.md`

Google Antigravity	`.antigravity/context.md`	`.antigravity/context.md`

Run `tina4 ai` to see which tools are detected and install context files via the menu.

The AI Chat in Dev Dashboard

The dev dashboard at `/__dev` includes an AI chat tab. By default it talks to a local **qwen2.5-coder** model served via Ollama, grounded by Tina4's built-in RAG index of the framework source. Nothing leaves your machine.

Configure it in `.env`:

```
TINA4_AI_URL=http://localhost:11434      # Ollama HTTP endpoint
TINA4_AI_MODEL=qwen2.5-coder             # Default coding model
TINA4_RAG_URL=http://localhost:11434     # RAG embedding endpoint (defaults to TINA4_AI_URL)
```

If you prefer a remote provider, point `TINA4_AI_URL` at any OpenAI-compatible endpoint and set `TINA4_AI_MODEL` accordingly.

The Convention Advantage

AI thrives on convention. When every Tina4 project follows the same structure, the AI never has to ask "where should I put this?"

```
src/
  routes/hello.rb     -> AI knows: this is a route file
  orm/product.rb      -> AI knows: this is an ORM model
  templates/page.html -> AI knows: this is a Frond template
  migrations/         -> AI knows: SQL migration files go here
```

The AI generates code that drops into the right directory with the right naming convention, every time.

Prompt Engineering for Tina4

Here are effective prompts for AI-assisted Tina4 Ruby development:

Creating a New Feature

```
Add a product catalog to my Tina4 Ruby project:
1. Create a Product model with fields: name, category, price, in_stock
2. Create a migration for the products table
3. Create CRUD routes at /api/products
4. Create a product listing template with category filters
5. Add response caching for the product list (5 minutes)
```

The AI generates all five files with correct Tina4 syntax because it read CLAUDE.md.

Adding Authentication

```
Add JWT authentication to my Tina4 Ruby project:
1. Create a users table migration with name, email, password_hash, role
2. Create register and login routes at /api/auth/register and /api/auth/login
3. Create auth middleware that validates Bearer tokens
4. Protect the /api/tasks group with auth middleware
5. Use Tina4::Auth.get_token and Tina4::Auth.valid_token
```

Building a Dashboard

```
Create an admin dashboard for my Tina4 Ruby project:
1. Create a GET /admin route that renders a dashboard template
2. The dashboard shows: total users, total products, recent orders
3. Use tina4css for styling (cards, tables, grid)
4. Use frond.js for AJAX data loading
5. Cache the dashboard stats for 60 seconds
```

Adding WebSocket

```
Add a real-time notification system to my Tina4 Ruby project:
1. Create a WebSocket endpoint at /ws/notifications/{user_id}
2. When a task is assigned, push a notification via WebSocket
3. Include a JavaScript snippet that connects and shows browser notifications
4. Auto-reconnect on disconnect
```

The Ruby Advantage for AI

Ruby's expressiveness makes AI-generated code particularly clean:

Block Syntax

```ruby
Tina4::Router.get("/api/products") do |request, response|
  products = Tina4.cache_get("products:all")
  if products.nil?
    db = Tina4.database
    products = db.fetch("SELECT * FROM products ORDER BY name")
    Tina4.cache_set("products:all", products, 300)
  end

  response.json({ products: products, count: products.length })
end
```

The block syntax is unambiguous. The AI knows exactly where the route handler starts and ends. No curly brace confusion, no indentation ambiguity.

Hash Syntax

```ruby
response.json({
  user: { id: user.id, name: user.name, email: user.email },
  tasks: tasks.map(&:to_h),
  stats: { total: total, completed: completed }
})
```

Ruby's symbol-key hash syntax (`name:` instead of `"name" =>`) is concise and consistent. The AI generates clean, idiomatic Ruby.

Enumerable Methods

```
# Filter products
electronics = products.select { |p| p[:category] == "Electronics" }

# Transform data
names = products.map { |p| p[:name] }

# Aggregate
total = items.sum { |item| item[:price] * item[:quantity] }

# Count with condition
featured_count = products.count { |p| p[:featured] }
```

These are the Ruby idioms the AI knows and uses. No external libraries, no complex method chains -- just Ruby.

Vibe Coding Workflow

The "vibe coding" workflow with AI and Tina4 looks like this:

1. Describe What You Want

Tell the AI what you need in plain English. Be specific about the data model, the endpoints, and the behavior.

2. Review the Generated Code

The AI generates the routes, models, migrations, and templates. Review them for correctness and completeness.

3. Run and Test

```
tina4 migrate
tina4 serve
```

Test the endpoints with curl or the Swagger UI. If something is wrong, tell the AI what needs to change.

4. Iterate

"The product list should be paginated. Add page and per_page query parameters." The AI updates the code.

5. Deploy

```
docker compose build
docker compose up -d
```

From idea to production in minutes, not days.

Real-World Example: Building an API in 5 Minutes

Here is a real conversation with an AI assistant:

You: "Add a blog to my Tina4 Ruby app. Posts with title, body, published flag, and user_id. CRUD API at /api/posts. Only published posts should be visible to anonymous users. Authors can see their own drafts."

AI generates:

- `migrations/20260322_create_posts_table.sql` -- the migration
- `src/orm/post.rb` -- the model with fields and relationships
- `src/routes/posts.rb` -- CRUD routes with authentication and filtering
- `tests/posts_spec.rb` -- RSpec tests for all scenarios

All using correct Tina4 conventions:

```
Tina4::Router.get("/api/posts") do |request, response|
  db = Tina4.database

  # Check if user is authenticated (optional)
  user_id = nil
  auth_header = request.headers["Authorization"] || ""
  if auth_header.start_with?("Bearer ")
    token = auth_header.sub("Bearer ", "")
    if Tina4::Auth.valid_token(token)
      payload = Tina4::Auth.get_payload(token)
      user_id = payload["user_id"]
    end
  end

  if user_id
    # Authenticated: show published + own drafts
    posts = db.fetch(
      "SELECT * FROM posts WHERE published = 1 OR user_id = :user_id ORDER BY created_at DESC",
      { user_id: user_id }
    )
  else
    # Anonymous: show published only
    posts = db.fetch("SELECT * FROM posts WHERE published = 1 ORDER BY created_at DESC")
  end

  response.json({ posts: posts, count: posts.length })
end
```

The AI knows:

- `Tina4::Router.get` for routes
- `request.headers["Authorization"]` for the token
- `Tina4::Auth.valid_token` and `Tina4::Auth.get_payload` for JWT
- `db.fetch` with named parameters for queries
- `response.json` for JSON responses

It did not hallucinate. It did not guess. It read CLAUDE.md and wrote correct code.

Why This Matters

Traditional development: write every line, look up every API, debug every typo. A simple CRUD feature takes an hour.

Vibe coding with Tina4: describe what you want. The AI generates correct code. You review and deploy. The same feature takes 5 minutes.

The insight: AI can only be as good as the framework it targets. A framework with 200 gems, 15 configuration files, and 3 ways to do everything gives the AI too many choices. A framework with zero dependencies, one way to do everything, and a complete reference in CLAUDE.md gives the AI exactly what it needs.

Every Tina4 design decision -- zero deps, convention over configuration, identical API across 4 languages, CLAUDE.md -- exists to make AI-assisted development work.

You bring the ideas. The AI brings the implementation. Tina4 is the bridge.

Exercise: Vibe-Code a Feature

Open your Tina4 Ruby project with an AI assistant (Claude Code, Cursor, or Copilot) and try this prompt:

```
Add a comment system to my blog:
1. Create a comments table with post_id, author_name, body, created_at
2. Create a Comment model with belongs_to :post relationship
3. Add POST /api/posts/{id}/comments to add a comment
4. Add GET /api/posts/{id}/comments to list comments
5. Modify GET /api/posts/{id} to include comments
6. Add RSpec tests for the comment endpoints
```

Verify the AI generates:

- A migration file in `migrations/`
- A model file in `src/orm/`
- Route handlers in `src/routes/`
- Test file in `tests/`

Run `tina4 migrate && tina4 test` to verify everything works.

Gotchas

1. AI Generates Rails Syntax Instead of Tina4

Problem: The AI writes `render json:` instead of `response.json`.

Cause: The AI defaulted to Rails conventions.

Fix: Make sure CLAUDE.md is in your project root. Mention "Tina4 Ruby" explicitly in your prompt.

2. AI Invents Non-Existent Methods

Problem: The AI calls `Tina4::Router.resource` which does not exist.

Cause: The AI hallucinated a method from another framework.

Fix: Review generated code against the CLAUDE.md reference. Ask the AI to only use methods documented in CLAUDE.md.

3. AI Generates Tests That Do Not Match the Implementation

Problem: Tests reference endpoints or response formats that differ from the actual routes.

Fix: Generate routes first, then ask the AI to generate tests that match the existing routes.

4. AI Forgets to Run Migrations

Problem: The app crashes because the table does not exist.

Fix: Always run `tina4 migrate` after generating migrations. Include it in your prompts: "...and tell me the commands to run."

5. AI Uses require Instead of Auto-Loading

Problem: The AI adds `require_relative "../orm/product"` at the top of route files.

Cause: Ruby convention is to require files explicitly, but Tina4 auto-loads everything in `src/routes/` and `src/orm/`.

Fix: Remove the require statements. Tina4 handles loading automatically.

6. AI Generates Over-Engineered Solutions

Problem: The AI creates service objects, repository patterns, and dependency injection for a simple CRUD feature.

Fix: Tell the AI: "Keep it simple. Use Tina4's built-in features only. No extra abstractions." Tina4's philosophy is simplicity -- the AI should follow it.

7. AI Does Not Know About to_h

Problem: The AI serializes model objects with `to_json` instead of `to_h`.

Fix: Tina4 ORM objects use `to_h` (Ruby idiom for hash conversion) followed by `response.json`. The CLAUDE.md file documents this pattern.

Environment Variables

> **■■ *BREAKING CHANGE: Tina4 v3.12.0***
>
> *Every framework env var now requires the `TINA4_` prefix. The legacy un-prefixed names (`DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc.) no longer work. Setting them at startup makes the framework refuse to boot with a list of renames.*
>
> *Run `tina4 env --migrate` to rewrite your existing `.env` automatically, or rename manually using the table below. The runtime guard prints the same mapping if it detects legacy names.*
>
> ***Conventional names stay un-prefixed:*** `PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT`. *These are runtime/PaaS conventions, not framework config.*

Tina4 Ruby is configured through environment variables, read from `.env` at the project root. Every variable has a sensible default, so most projects set three or four values and leave the rest alone.

This chapter lists every variable the Ruby framework reads, grouped by subsystem. Start with the minimum-config examples at the end, then come back here when you need to tune something specific.

Core Server

Variable	Default	Description
`HOST`	`0.0.0.0`	Bind address. `0.0.0.0` listens on every interface. `127.0.0.1` restricts to localhost.
`TINA4_HOST`	*(inherits `HOST`)*	Tina4-specific alias for `HOST`.
`PORT`	`7147`	HTTP server port. The Rust CLI prefers `TINA4_PORT` but falls back to `PORT`.
`TINA4_PORT`	*(inherits `PORT`)*	Explicit Tina4-specific port override. Takes precedence over `PORT` when both are set.
`TINA4_HOST_NAME`	`localhost:7147`	Fully-qualified host used in generated absolute URLs (Swagger, OAuth redirects, emails).
`TINA4_DEBUG`	`false`	Master debug toggle. Enables Swagger UI, dev dashboard, live reload, template dump filter, error overlay. Never set to `true` in production.
`TINA4_ENV`	`development`	Runtime environment label. Values like `development`, `staging`, `production` control dev-only features. Falls back to `RACK_ENV` then `RUBY_ENV`.
`RACK_ENV`	*(none)*	Rack-standard environment label. Used if `TINA4_ENV` is unset.
`RUBY_ENV`	*(none)*	Ruby-ecosystem environment label. Used if `TINA4_ENV` and `RACK_ENV` are unset.

`TINA4_NO_BROWSER`	`false`	Stops `tina4 serve` from opening your browser on every restart. Recommended during active development.
`TINA4_NO_RELOAD`	`false`	Disables the dev hot-reload signal from the Rust CLI. Use when you want a stable server for debugging.
`ENVIRONMENT`	*(none)*	PaaS-style environment label. Read by tooling alongside `RACK_ENV`/`RUBY_ENV`.
`TINA4_SHUTDOWN_TIMEOUT`	`10`	Seconds the supervisor waits for graceful shutdown before terminating workers.
`TINA4_SUPERVISOR_URL`	*(derived from `TINA4_PORT` + 2000)*	Override URL for the local Rust CLI supervisor used by the dev dashboard.
`TINA4_TEMPLATE_ROUTING`	`on`	Controls automatic template-to-route binding. Set to `off`, `false`, `0`, `no`, or `disabled` to require explicit route declarations.
`TINA4_ALLOW_LEGACY_ENV`	`false`	Bypass the v3.12 legacy-env startup guard. Intended for CI/migration scripts only.
`TINA4_SUPPRESS`	`false`	Suppresses the startup banner. Useful for headless processes and CI runs.
`TINA4_ENV_FILE`	`.env`	Path or filename of the dotenv file loaded at boot. Resolved against the project root when relative.
`TINA4_HEALTH_PATH`	`/__health`	Health-check route mounted by `Tina4::Health`. The legacy `/health` path stays registered as an alias unless it's the configured value.

Routing

Variable	Default	Description
`TINA4_TRAILING_SLASH_REDIRECT`	`false`	When truthy, the Rack app issues a `301` redirect that strips trailing slashes from request paths.

Secrets and Authentication

Variable	Default	Description
`TINA4_SECRET`	`tina4-default-secret`	JWT signing secret. Must be long, random, and unique per environment. **Never commit.**
`TINA4_TOKEN_LIMIT`	`60`	JWT token lifetime in minutes.
`TINA4_JWT_ALGORITHM`	`HS256`	JWT signing algorithm.
`TINA4_API_KEY`	*(empty)*	Static API key used by `Tina4::Auth.validate_api_key` as a fallback to JWT.
`TINA4_API_KEY`	*(empty)*	Legacy alias for `TINA4_API_KEY`.

Database

Variable	Default	Description
`TINA4_DATABASE_URL`	*(required for non-SQLite)*	Connection URL. Scheme selects the driver: `sqlite`, `postgres`, `mysql`, `firebird`.
`TINA4_DATABASE_USERNAME`	*(empty)*	Overrides the username embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_PASSWORD`	*(empty)*	Overrides the password embedded in `TINA4_DATABASE_URL`.
`TINA4_DATABASE_FIREBIRD_PATH`	*(empty)*	Overrides the database path/alias parsed from `TINA4_DATABASE_URL` for Firebird. Useful for Windows backslash paths and split-config setups.
`TINA4_DATABASE_URL`	*(empty)*	Legacy alias for `TINA4_DATABASE_URL`.
`TINA4_AUTOCOMMIT`	`true`	Standalone writes auto-commit on their own connection (durable + visible across a pool); explicit transactions stay atomic. Set `false` for strict manual-commit mode.
`TINA4_DB_CACHE`	`false`	Enables in-memory query-result caching for read queries.
`TINA4_DB_CACHE_TTL`	`30`	Query cache TTL in seconds when `TINA4_DB_CACHE=true`.
`TINA4_ORM_PLURAL_TABLE_NAMES`	`true`	When `true`, the ORM pluralises class names into table names (`User` → `users`). Set `false` to keep them singular.

`TINA4_DB_POOL`	`0`	Connection pool size used by `Tina4::Database.new(url, pool:)` when the caller doesn't pass `pool:` explicitly. `0` disables pooling.

CORS

Variable	Default	Description
`TINA4_CORS_ORIGINS`	*(empty)*	Comma-separated allowed origins. Empty denies every cross-origin request; set the origins your front end is served from. `*` allows any origin and has to be set explicitly.
`TINA4_CORS_METHODS`	`GET, POST, PUT, PATCH, DELETE, OPTIONS`	Allowed request methods.
`TINA4_CORS_HEADERS`	`Content-Type,Authorization,X-Request-ID`	Allowed request headers.
`TINA4_CORS_CREDENTIALS`	`false`	Send `Access-Control-Allow-Credentials: true`.
`TINA4_CORS_MAX_AGE`	`86400`	Preflight cache lifetime in seconds.

Security Headers

Variable	Default	Description
`TINA4_CSP`	`default-src 'self'`	`Content-Security-Policy` header.
`TINA4_CSRF`	`false`	CSRF token validation on POST/PUT/PATCH/DELETE. Off by default in Ruby; enable with `true`.
`TINA4_HSTS`	*(empty/off)*	`Strict-Transport-Security` max-age in seconds. Set `31536000` in production with HTTPS.
`TINA4_FRAME_OPTIONS`	`SAMEORIGIN`	`X-Frame-Options` header.
`TINA4_REFERRER_POLICY`	`strict-origin-when-cross-origin`	`Referrer-Policy` header.
`TINA4_PERMISSIONS_POLICY`	`camera=(), microphone=(), geolocation=()`	`Permissions-Policy` header.

Rate Limiting

Variable	Default	Description
`TINA4_RATE_LIMIT`	`100`	Maximum requests per window per IP. Set `0` to disable.
`TINA4_RATE_WINDOW`	`60`	Rate-limit window in seconds.

Sessions

Variable	Default	Description
`TINA4_SESSION_BACKEND`	`file`	Storage backend. Options: `file`, `redis`, `valkey`, `mongo`, `database`.
`TINA4_SESSION_TTL`	`3600`	Session expiry in seconds.
`TINA4_SESSION_SAMESITE`	`Lax`	SameSite cookie attribute. Options: `Strict`, `Lax`, `None`.
`TINA4_SESSION_PATH`	`data/sessions`	Filesystem path for the file backend.
`TINA4_SESSION_NAME`	`tina4_session`	Session cookie name. Used unless the caller passes an explicit `cookie_name`.
`TINA4_SESSION_HTTPONLY`	`true`	Sets the `HttpOnly` attribute on the session cookie. Set `false` to allow JavaScript access.
`TINA4_SESSION_SECURE`	`false`	Sets the `Secure` attribute on the session cookie. Enable in production behind HTTPS.

Valkey/Redis session backend

Variable	Default	Description
`TINA4_SESSION_VALKEY_HOST`	`localhost`	Valkey/Redis host for the session handler.
`TINA4_SESSION_VALKEY_PORT`	`6379`	Valkey/Redis port.
`TINA4_SESSION_VALKEY_PASSWORD`	*(none)*	Valkey/Redis auth password.
`TINA4_SESSION_VALKEY_DB`	`0`	Valkey/Redis database number.
`TINA4_SESSION_VALKEY_PREFIX`	`tina4:session:`	Key prefix used for session entries.
`TINA4_SESSION_VALKEY_TTL`	`86400`	Per-key expiry in seconds.

Cache

Variable	Default	Description
`TINA4_CACHE_BACKEND`	`memory`	Response cache backend. Options: `memory`, `file`, `redis`, `valkey`, `memcached`, `mongodb`, `database`. Falls back to `file` if the configured backend is unreachable.
`TINA4_CACHE_DIR`	`data/cache`	Cache directory for the file backend.
`TINA4_CACHE_TTL`	`0`	Default cache TTL in seconds (`0` disables caching for the global instance; the singleton helper uses `60` when unset).
`TINA4_CACHE_MAX_ENTRIES`	`1000`	Maximum cache entries before eviction.
`TINA4_CACHE_URL`	`redis://localhost:6379`	Connection URL for remote cache backends. For `database`, falls back to `TINA4_DATABASE_URL` when unset.
`TINA4_CACHE_USERNAME`	*(none)*	Username for the cache backend. May also be embedded in `TINA4_CACHE_URL`.
`TINA4_CACHE_PASSWORD`	*(none)*	Password for the cache backend. May also be embedded in `TINA4_CACHE_URL` (e.g. `redis://:pass@host`). Memcached is unauthenticated.

Templates

Variable	Default	Description
`TINA4_TEMPLATE_CACHE_TTL`	`0`	Frond template cache TTL in seconds. `0` keeps compiled templates cached permanently; positive values expire entries after `N` seconds.

GraphQL

Variable	Default	Description
`TINA4_GRAPHQL_AUTO_SCHEMA`	`true`	When `true`, the GraphQL module auto-generates a schema from registered ORM models. Set `false` to require an explicit schema.
`TINA4_GRAPHQL_ENDPOINT`	`/graphql`	Path the GraphQL handler is mounted at when the caller doesn't pass an explicit path.

Queues

Variable	Default	Description
`TINA4_QUEUE_BACKEND`	`file`	Queue backend. Options: `file`, `kafka`, `rabbitmq`, `mongo`, `database`.
`TINA4_QUEUE_URL`	*(none)*	Connection URL for remote backends. Parsed for `host`/`port` when set.

Kafka queue backend

Variable	Default	Description
`TINA4_KAFKA_BROKERS`	*(none)*	Comma-separated broker list (e.g. `localhost:9092`).
`TINA4_KAFKA_GROUP_ID`	`tina4_consumer_group`	Kafka consumer group ID.

RabbitMQ queue backend

Variable	Default	Description
`TINA4_RABBITMQ_HOST`	`localhost`	RabbitMQ host.
`TINA4_RABBITMQ_PORT`	`5672`	RabbitMQ port.
`TINA4_RABBITMQ_USERNAME`	`guest`	RabbitMQ username.
`TINA4_RABBITMQ_PASSWORD`	`guest`	RabbitMQ password.
`TINA4_RABBITMQ_VHOST`	`/`	RabbitMQ virtual host.

MongoDB queue backend

Variable	Default	Description
`TINA4_MONGO_URI`	*(none)*	Full MongoDB connection string. Overrides host/port when set.
`TINA4_MONGO_HOST`	`localhost`	MongoDB host.
`TINA4_MONGO_PORT`	`27017`	MongoDB port.
`TINA4_MONGO_USERNAME`	*(none)*	MongoDB username.
`TINA4_MONGO_PASSWORD`	*(none)*	MongoDB password.
`TINA4_MONGO_DB`	`tina4`	MongoDB database name.
`TINA4_MONGO_COLLECTION`	`tina4_queue`	MongoDB collection name for jobs.

WebSocket

Variable	Default	Description
`TINA4_WS_BACKPLANE`	*(none)*	Backplane type. Options: `redis`, `nats`. Required for multi-instance broadcasts.
`TINA4_WS_BACKPLANE_URL`	`redis://localhost:6379` (Redis) / `nats://localhost:4222` (NATS)	Connection URL for the chosen backplane.

Email

Variable	Default	Description
`TINA4_MAIL_HOST`	`localhost`	SMTP server hostname.
`TINA4_MAIL_PORT`	`587`	SMTP server port.
`TINA4_MAIL_USERNAME`	*(none)*	SMTP authentication username.
`TINA4_MAIL_PASSWORD`	*(none)*	SMTP authentication password.
`TINA4_MAIL_FROM`	`dev@localhost`	Default sender email address.
`TINA4_MAIL_FROM_NAME`	*(empty)*	Default sender display name.
`TINA4_MAIL_ENCRYPTION`	`tls`	Connection encryption. Options: `tls`, `ssl`, `none`.
`TINA4_MAIL_IMAP_HOST`	*(inherits mail host)*	IMAP server for inbound mail.
`TINA4_MAIL_IMAP_PORT`	`993`	IMAP server port.
`TINA4_MAIL_IMAP_USERNAME`	*(inherits SMTP username)*	IMAP authentication username. Mapped from legacy `IMAP_USER`.
`TINA4_MAIL_IMAP_PASSWORD`	*(inherits SMTP password)*	IMAP authentication password. Mapped from legacy `IMAP_PASS`.
`TINA4_MAIL_IMAP_ENCRYPTION`	`tls`	IMAP connection encryption. Accepts `tls`, `starttls`, or `none`.
`TINA4_MAIL_CAPTURE`	`false`	Force local DevMailbox capture and skip SMTP, even when an SMTP host exists. A missing host also captures.
`TINA4_MAIL_REDIRECT_TO`	*(none)*	Comma-separated safety recipients for real SMTP delivery. Replaces To, Cc, and Bcc.
`TINA4_MAILBOX_DIR`	`data/mailbox`	Storage directory for captured mail. This setting does not enable or disable capture.

> `TINA4_DEBUG` *does not suppress email. On staging, set* `TINA4_MAIL_CAPTURE=true` *to keep every message local, or set* `TINA4_MAIL_REDIRECT_TO=qa@example.com,product@example.com` *to exercise SMTP without reaching original recipients. Capture takes precedence when both are set.*
>
> `TINA4_MAIL_HOST`, `TINA4_MAIL_PORT`, `TINA4_MAIL_USERNAME`, `TINA4_MAIL_PASSWORD`, `TINA4_MAIL_FROM`, `TINA4_MAIL_FROM_NAME`, `TINA4_MAIL_IMAP_HOST`, `TINA4_MAIL_IMAP_PORT` *are accepted as legacy aliases. New projects should use the* `TINA4_MAIL_*` *names.*

Logging

Variable	Default	Description
`TINA4_LOG_LEVEL`	`INFO`	Console severity threshold. Options: `ALL`, `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`, `NONE`, or the matching bracketed framework constant.
`TINA4_LOG_FILE_LEVEL`	`ALL`	Independent severity threshold for the file sink.
`TINA4_LOG_STRICT`	`false`	Raise when a configured log sink fails instead of continuing with the remaining sinks. Accepts only `true` or `false`.
`TINA4_LOG_MAX_SIZE`	`10`	Per-file log size limit in megabytes. Rotated when exceeded.
`TINA4_LOG_KEEP`	`5`	Number of rotated log files to retain.

Logs default to stdout. Set `TINA4_LOG_OUTPUT=file` plus `TINA4_LOG_FILE=app.log` to write to disk; Ruby's stdlib `Logger` rotates at `TINA4_LOG_ROTATE_SIZE` bytes and keeps `TINA4_LOG_ROTATE_KEEP` backups natively.

Variable	Default	Description
`TINA4_LOG_FILE`	*(empty = stdout only)*	Explicit log file path. Absolute, or resolved relative to `TINA4_LOG_DIR`.

`TINA4_LOG_DIR`	`logs`	Directory used when `TINA4_LOG_FILE` is set without an absolute path.
`TINA4_LOG_FORMAT`	`text`	Log line format. Accepts `text` or `json`.
`TINA4_LOG_OUTPUT`	`stdout`	Output sink. Accepts `stdout`, `file`, or `both`.
`TINA4_LOG_CRITICAL`	`false`	When `true`, gates the new `critical` level and raises on log write failures instead of swallowing them.
`TINA4_LOG_ROTATE_SIZE`	`10485760`	Per-file rotation threshold in bytes (10 MB). `0` disables rotation. Handled natively by stdlib `Logger.new(path, shift_age, shift_size)`.
`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated backups to retain.

Uploads

Variable	Default	Description
`TINA4_MAX_UPLOAD_SIZE`	`10485760`	Maximum multipart upload size in bytes (10 MB). Requests larger than this are rejected before parsing.

Services (background tasks)

Variable	Default	Description
`TINA4_SERVICE_DIR`	`src/services`	Directory scanned for service classes.
`TINA4_SERVICE_SLEEP`	`5`	Seconds the service runner sleeps between iterations.

Localisation

Variable	Default	Description
`TINA4_LOCALE`	`en`	Default locale for `Tina4::Localization`.
`TINA4_LOCALE_DIR`	`src/locale`	Directory containing locale JSON files.

Application AI Client and Developer AI Tools

The app-facing `Ai` client and the developer dashboard share the `TINA4_AI_*` namespace. The client supports local OpenAI-compatible servers, OpenAI, and Anthropic. See Chapter 39: AI Client for code examples.

Variable	Default	Description
`TINA4_AI_PROVIDER`	`local`	Provider used by the app-facing client: `local`, `openai`, or `anthropic`.
`TINA4_AI_URL`	Provider-specific	Base URL or full endpoint. Local defaults to `http://localhost:11437`; OpenAI and Anthropic use their public API bases.
`TINA4_AI_MODEL`	Provider-specific	Default model. Local uses `llama3.2`, OpenAI uses `gpt-4o-mini`, and Anthropic uses `claude-3-5-haiku-latest`.
`TINA4_AI_KEY`	*(none)*	Required before an OpenAI or Anthropic request. Local requests need no key.
`TINA4_AI_TIMEOUT`	`60`	Total request deadline in seconds, including retries and response reads.
`TINA4_AI_CONNECT_TIMEOUT`	`10`	Connection-establishment deadline in seconds.
`TINA4_AI_MAX_RETRIES`	`2`	Retries before output for connection failures, HTTP 429, and HTTP 5xx.

`TINA4_EMBED_URL`	*(inherits `TINA4_AI_URL`)*	Optional embedding base URL or full endpoint.
`TINA4_RAG_URL`	`http://localhost:11438`	RAG service used by the developer dashboard.
`TINA4_RAG_TOPK`	`4`	Number of RAG matches returned per query.
`TINA4_VISION_URL`	`http://localhost:11437/api/chat`	Vision endpoint used by developer tools, not the app-facing AI client.
`TINA4_IMAGE_URL`	`http://localhost:11437/api/generate`	Image endpoint used by developer tools, not the app-facing AI client.
`TINA4_SUPERVISOR_URL`	`http://localhost:9999`	Rust supervisor URL used by developer tools.
`TINA4_MCP_REMOTE`	`false`	Allows MCP to bind beyond localhost. Do not enable it on a public production interface.
`TINA4_NO_AI_PORT`	`false`	Disables the developer AI port listener.
`TINA4_OVERRIDE_CLIENT`	`false`	Lets the framework start without the Rust client in containers and CI.

MCP

Variable	Default	Description
`TINA4_MCP`	*(inherits `TINA4_DEBUG`)*	Enables the MCP server. Defaults to whatever `TINA4_DEBUG` is set to; pass `true`/`false` to override explicitly.
`TINA4_MCP_PORT`	*(derived from `TINA4_PORT` + 2000)*	Port the MCP server binds to. Defaults to the HTTP port plus `2000`.

Swagger / OpenAPI

Variable	Default	Description
`TINA4_SWAGGER_TITLE`	`Tina4 API`	OpenAPI spec title. Falls back to `PROJECT_NAME`.
`TINA4_SWAGGER_DESCRIPTION`	`Auto-generated API documentation`	OpenAPI spec description.
`TINA4_SWAGGER_VERSION`	*(Gem version)*	Overrides the spec `version`.
`TINA4_SWAGGER_ENABLED`	*(inherits `TINA4_DEBUG`)*	Enables the Swagger UI and `/swagger.json` route. Defaults to `TINA4_DEBUG`; set explicitly to override.
`TINA4_SWAGGER_CONTACT_EMAIL`	*(empty)*	Contact email rendered into the OpenAPI `info.contact` block.
`TINA4_SWAGGER_LICENSE`	*(empty)*	SPDX license name (e.g. `MIT`) rendered into the OpenAPI `info.license` block.
`PROJECT_NAME`	*(none)*	Alternative OpenAPI title source when `TINA4_SWAGGER_TITLE` is unset.

Configuration Recipes

The tables above list every knob. These are the setups most apps actually reach for, ready to paste into `.env`. Each block sets only what the feature needs. Everything else keeps its default.

PostgreSQL in production

One URL points the ORM, the migrations, and the query builder at Postgres. Credentials can ride in the URL or sit in their own variables, which keeps the password out of your shell history.

```
TINA4_DATABASE_URL=postgresql://localhost:5432/myapp
TINA4_DATABASE_USERNAME=myapp
TINA4_DATABASE_PASSWORD=changeme
TINA4_DB_POOL=4
```

`TINA4_DB_POOL=4` opens four connections and rotates across them. Leave it at `0` for a single connection on a small app.

A shared cache on Redis

The response cache and the cross-request query cache both speak to the same Redis. Point them at it and every instance shares one cache, invalidated globally on every write.

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379
TINA4_DB_CACHE=true
TINA4_DB_CACHE_BACKEND=redis
TINA4_DB_CACHE_URL=redis://localhost:6379
```

If Redis is down or the driver is missing, the cache logs a warning and falls back to the file backend. It never silently stops caching.

Sessions that survive more than one box

The file backend is fine for a single server. Move sessions to Redis the moment you run more than one instance, so a user stays logged in whichever instance answers the next request.

```
TINA4_SESSION_BACKEND=redis
TINA4_SESSION_REDIS_HOST=localhost
TINA4_SESSION_REDIS_PORT=6379
TINA4_SESSION_SECURE=true
TINA4_SESSION_SAMESITE=Strict
```

`TINA4_SESSION_SECURE=true` keeps the cookie off plain HTTP. Turn it on once you have TLS.

A queue on RabbitMQ or Kafka

One URL is enough for RabbitMQ; the per-field variables only exist for split configs. The queue API stays identical whichever backend you pick.

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_URL=amqp://guest:guest@localhost:5672/
```

Kafka reads a broker list instead:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_KAFKA_BROKERS=localhost:9092
TINA4_KAFKA_GROUP_ID=myapp_workers
```

WebSocket broadcasts across instances

A single server broadcasts in memory. Add a backplane and a message sent on one instance reaches clients connected to every other instance.

```
TINA4_WS_BACKPLANE=redis
TINA4_WS_BACKPLANE_URL=redis://localhost:6379
TINA4_WS_ALLOWED_ORIGINS=https://myapp.com
```

Set the origin allow-list in production. Empty allows every origin, which is fine in dev and risky on the public internet.

Locked-down production headers

The defaults are already safe. These four tighten the screws for a public site on HTTPS.

```
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
```

```
TINA4_HSTS=31536000
TINA4_FRAME_OPTIONS=DENY
TINA4_SESSION_SECURE=true
```

Never pair `TINA4_CORS_CREDENTIALS=true` with `TINA4_CORS_ORIGINS=*`. Name your real origins instead.

The dev dashboard AI, kept local

The dashboard AI talks to a local model through Ollama by default, so nothing leaves your machine. Point the URLs elsewhere only when you run the hosted Tina4 AI services.

```
TINA4_AI_URL=http://localhost:11437/api/chat
TINA4_AI_MODEL=qwen2.5-coder:14b
TINA4_RAG_URL=http://localhost:11438
```

Minimal `.env` for Development

```
TINA4_DEBUG=true
TINA4_LOG_LEVEL=ALL
TINA4_NO_BROWSER=true
```

Debug mode lights up the Swagger UI, the dev dashboard, detailed error pages, and live reload. Keeping the browser flag on stops a new tab opening every time you save a file.

Minimal `.env` for Production

```
TINA4_SECRET=your-long-random-secret-here
TINA4_DATABASE_URL=postgresql://user:password@db-host:5432/myapp
TINA4_CORS_ORIGINS=https://myapp.com,https://www.myapp.com
TINA4_HSTS=31536000
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USERNAME=noreply@myapp.com
TINA4_MAIL_PASSWORD=your-smtp-password
TINA4_MAIL_FROM=noreply@myapp.com
```

No `TINA4_DEBUG`. It defaults to `false`, which is what you want in production. Set a real secret, a real database, locked-down CORS origins, HSTS, and SMTP credentials if you send email. Everything else has a production-appropriate default.

Feature Module Settings

These settings are read by focused framework modules rather than the server bootstrap. They remain part of the public configuration surface.

Variable	Default	Description

`TINA4_FEEDBACK_DEV_USER`	*(empty)*	Local-development identity for testing the feedback widget without authentication. Never use this as production authentication.
`TINA4_KAFKA_ASSIGN_TIMEOUT`	`15`	Seconds the first Kafka dequeue waits for partition assignment. Later polls use the normal short interval.
`TINA4_MCP_URL`	`https://mcp.tina4.com`	Grounding service URL shown and used by the developer admin.
`TINA4_QUEUE_PATH`	`data/queue`	Base directory for the file-backed queue. Relative paths resolve from the working directory.
`TINA4_SESSION_MEMCACHED_HOST`	`localhost`	Memcached host for the session backend.
`TINA4_SESSION_MEMCACHED_PORT`	`11211`	Memcached port for the session backend.
`TINA4_SESSION_MEMCACHED_PREFIX`	`tina4:session:`	Key prefix for Memcached sessions. Give each application a distinct prefix when sharing a server.
`TINA4_SESSION_REDIS_DB`	`0`	Redis database number used by the session backend.
`TINA4_SWAGGER_CONTACT_TEAM`	*(empty)*	Contact name placed in the generated OpenAPI `info.contact` object.
`TINA4_SWAGGER_CONTACT_URL`	*(empty)*	Contact URL placed in the generated OpenAPI `info.contact` object.

Deployment

1. From Development to Production

The app works on `localhost:7147`. Now it needs to run around the clock on a real server. Handle thousands of concurrent users. Survive restarts. Hold steady on memory. The gap between "works on my machine" and "works in production" is where projects stumble.

This chapter covers everything for a production deployment: environment configuration, Puma server setup, Docker packaging, health checks, graceful shutdown, SSL/TLS, scaling, and monitoring.

When you run `tina4 init`, the framework generates a production-ready `Dockerfile` and `.dockerignore` in your project root. The Dockerfile uses a multi-stage build: the first stage installs gem dependencies and the second stage copies only the runtime artifacts into a slim image. You do not need to write a Dockerfile from scratch -- the generated one is a solid starting point.

---

2. Production .env Configuration

Development defaults optimize for debugging. Production defaults optimize for performance and security. The first deployment step: configure `.env` for production.

Create a production `.env`:

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7147

# Database
TINA4_DATABASE_URL=sqlite:///data/app.db

# Security
TINA4_CORS_ORIGINS=https://yourdomain.com
JWT_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_TEMPLATE_CACHE_TTL=true
```

Key Differences from Development

Setting	Development	Production	Why
`TINA4_DEBUG`	`true`	`false`	Hides stack traces, disables toolbar
`TINA4_LOG_LEVEL`	`ALL`	`WARNING`	Reduces log noise
`TINA4_CORS_ORIGINS`	Your dev front end (for example `http://localhost:3000`), or empty	Your domain	Only listed origins can call the API from a browser

Sensitive Values

Production secrets never go into version control. The `.env` file is gitignored by default. For deployment, use environment variables from your hosting platform, CI/CD secrets, or a secrets manager.

```
# Docker: pass env vars at runtime
docker run -e JWT_SECRET=your-secret -e TINA4_DATABASE_URL=sqlite:///data/app.db my-app

# Fly.io: set secrets
fly secrets set JWT_SECRET=your-secret

# Railway: use the dashboard or CLI
railway variables set JWT_SECRET=your-secret
```

---

3. Puma Configuration

Puma is the production server for Tina4 Ruby. It runs multiple worker processes, handles concurrent requests, and supports graceful shutdown.

Create `config/puma.rb`:

```
# Port
port ENV.fetch("TINA4_PORT", 7147)

# Workers (processes) -- set to number of CPU cores
workers ENV.fetch("WEB_CONCURRENCY", 2)

# Threads per worker - Puma's own config var, not a Tina4 setting.
threads_count = ENV.fetch("PUMA_MAX_THREADS", 5)
threads threads_count, threads_count

# Environment
environment ENV.fetch("RACK_ENV", "production")

# Preload app for faster worker boot
preload_app!

# PID file
pidfile ENV.fetch("PIDFILE", "tmp/pids/server.pid")
```

```
# Logging
stdout_redirect "logs/puma.stdout.log", "logs/puma.stderr.log", true

# Worker timeout
worker_timeout 60

# Graceful shutdown
on_worker_boot do
  # Re-establish the database connection after fork: close the inherited
  # connection and rebind a fresh one from TINA4_DATABASE_URL.
  Tina4.database&.close
  Tina4.bind_database(Tina4::Database.from_env)
end
```

Start with Puma:

```
bundle exec puma -C config/puma.rb
```

How Many Workers?

Start with `(2 * CPU cores) + 1`:

CPU Cores	Workers	Use Case
1	3	Small VPS, hobbyist
2	5	Small production app
4	9	Medium production app
8	17	High-traffic app

The built-in WEBrick server is for development only. It handles one request at a time. Always use Puma in production.

---

4. Docker Deployment

Docker is the most portable deployment path. Your app runs the same way on your laptop, in CI, and on the production server.

Dockerfile

```
FROM ruby:3.3-slim

WORKDIR /app

# Install build dependencies
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    libsqlite3-dev \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Install gems (copy lock file for better layer caching)
COPY Gemfile Gemfile.lock ./
RUN bundle install --without development test

# Copy application code
COPY . .

# Create necessary directories
RUN mkdir -p data logs secrets tmp/pids

# Expose port
EXPOSE 7147

# Health check
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:7147/health || exit 1

# Start with Puma
CMD ["bundle", "exec", "puma", "-C", "config/puma.rb"]
```

.dockerignore

```
.git
.env
*.gem
.bundle
tmp
log
data/*.db
logs/*.log
.claude
node_modules
spec
```

Docker Compose

For a complete setup with supporting services:

```
services:
  app:
    build: .
    ports:
      - "7147:7147"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - JWT_SECRET=${JWT_SECRET}
```

```
      - TINA4_DATABASE_URL=sqlite:///data/app.db
      - TINA4_CACHE_BACKEND=redis
      - TINA4_CACHE_URL=redis://redis:6379
    volumes:
      - app-data:/app/data
      - app-logs:/app/logs
    depends_on:
      - redis
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:7147/health"]
      interval: 30s
      timeout: 5s
      retries: 3

  worker:
    build: .
    command: tina4 queue work
    environment:
      - TINA4_DATABASE_URL=sqlite:///data/app.db
    volumes:
      - app-data:/app/data
    depends_on:
      - app
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    restart: unless-stopped

volumes:
  app-data:
  app-logs:
  redis-data:
```

Building and Running

```
# Build the image
docker compose build

# Start everything
docker compose up -d

# View logs
docker compose logs -f app

# Stop everything
docker compose down
```

---

5. Health Checks

Production deployments need a health check endpoint. Load balancers, container orchestrators, and monitoring tools all rely on it to verify the app is running.

Create a health check route:

```
Tina4::Router.get("/health") do |request, response|
  db = Tina4.database
  db_ok = false

  begin
    db.fetch_one("SELECT 1")
    db_ok = true
  rescue => e
    # Database is down
  end

  status = db_ok ? "ok" : "degraded"
  status_code = db_ok ? 200 : 503

  response.json({
    status: status,
    version: "1.0.0",
    framework: "tina4-ruby",
    database: db_ok ? "connected" : "disconnected",
    ruby_version: RUBY_VERSION,
    uptime_seconds: (Time.now - $start_time).to_i,
    memory_mb: ((`ps -o rss= -p #{Process.pid}`.to_i) / 1024.0).round(1),
    pid: Process.pid,
    timestamp: Time.now.utc.iso8601
  }, status_code)
end
```

This endpoint:

- Returns `200` when everything is healthy
- Returns `503` when the database is down (so the load balancer stops routing traffic)
- Includes version information for deployment tracking
- Runs fast (no heavy queries, no authentication)

For graceful shutdown, create a `.broken` file in the project root to make the health check return a failure status. Wait for the load balancer to drain traffic, then restart the server.

---

6. Graceful Shutdown

Deploy a new version. The old process must stop. Graceful shutdown finishes active requests before terminating.

Tina4 handles this when it receives a `SIGTERM` signal (the standard shutdown signal from Docker, Kubernetes, and systemd):

- Stop accepting new connections

- Wait for active requests to complete (up to 30 seconds)
- Close database connections
- Flush logs
- Exit

Configuring Shutdown Timeout

```
TINA4_SHUTDOWN_TIMEOUT=30
```

Docker Stop Grace Period

Docker sends `SIGTERM`, waits for the grace period, then sends `SIGKILL`. Match the Docker grace period to your shutdown timeout:

```
services:
  app:
    stop_grace_period: 30s
```

Graceful Restart Pattern

```
# Signal the load balancer to stop routing traffic
touch .broken

# Wait for health checks to fail and traffic to drain
sleep 30

# Restart the application
sudo systemctl restart tina4-app

# Remove the signal file
rm .broken
```

---

7. Log Rotation

In production, logs grow without limit unless rotated. Tina4 writes logs to `logs/app.log` and `logs/error.log`.

Using logrotate (Linux)

Create `/etc/logrotate.d/tina4`:

```
/app/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 deploy deploy
    postrotate
        kill -USR1 $(cat /app/tmp/pids/server.pid) 2>/dev/null || true
    endscript
}
```

This rotates logs daily, keeps 14 days of history, and compresses old logs. The `USR1` signal tells Puma to reopen its log files after rotation.

Docker Logging

Docker captures stdout/stderr automatically. Configure log rotation in the compose file:

```
services:
  app:
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "5"
```

This keeps at most 50MB of logs (5 files of 10MB each).

---

8. Reverse Proxy with Nginx

In production, Tina4 runs behind Nginx. Nginx handles:

- SSL/TLS termination (HTTPS)
- Static file serving (faster than Ruby)
- Request buffering
- Rate limiting
- WebSocket proxying

Create `/etc/nginx/sites-available/my-app`:

```
upstream tina4_app {
    server 127.0.0.1:7147;
}

server {
    listen 80;
    server_name yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # Static files (served by Nginx, faster than Ruby)
    location /css/ {
        alias /app/src/public/css/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /js/ {
```

```
        alias /app/src/public/js/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    location /images/ {
        alias /app/src/public/images/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # WebSocket upgrade
    location /ws/ {
        proxy_pass http://tina4_app;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 86400;
    }

    # Application
    location / {
        proxy_pass http://tina4_app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

---

9. SSL/TLS with Let's Encrypt

HTTPS is non-negotiable in production. Let's Encrypt provides free SSL certificates with automatic renewal.

With Nginx and Certbot

Install Certbot:

```
sudo apt install certbot python3-certbot-nginx
```

Obtain a certificate:

```
sudo certbot --nginx -d yourdomain.com
```

Certbot modifies your Nginx configuration to include SSL settings and sets up automatic renewal. Verify auto-renewal works:

```
sudo certbot renew --dry-run
```

Certbot renews certificates 30 days before expiry. The renewal runs via a systemd timer or cron job that Certbot creates during installation.

With Docker (Using Traefik as Reverse Proxy)

Traefik handles SSL termination and automatic certificate provisioning. Add it to your Docker Compose setup:

```
services:
  reverse-proxy:
    image: traefik:v3.0
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      - "--certificatesresolvers.letsencrypt.acme.tlschallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.email=you@example.com"
      - "--certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - letsencrypt:/letsencrypt

  app:
    build: .
    labels:
      - "traefik.http.routers.app.rule=Host(`yourdomain.com`)"
      - "traefik.http.routers.app.tls=true"
      - "traefik.http.routers.app.tls.certresolver=letsencrypt"
    environment:
      - TINA4_DEBUG=false
      - JWT_SECRET=${JWT_SECRET}

volumes:
  letsencrypt:
```

Traefik detects your app container through Docker labels, provisions a certificate from Let's Encrypt, and handles all HTTPS traffic. No Nginx configuration needed.

Certificate Monitoring

Certificates expire. Even with auto-renewal, things go wrong. Set up monitoring:

```
# Check certificate expiry manually
echo | openssl s_client -connect yourdomain.com:443 2>/dev/null | openssl x509 -noout -dates

# Verify Certbot timer is active
sudo systemctl list-timers | grep certbot
```

Use an external monitoring service (Uptime Robot, Better Uptime) that checks certificate expiry and alerts you 14 days before it expires.

---

10. Process Management with systemd

Create `/etc/systemd/system/tina4-app.service`:

```
[Unit]
```

```
Description=Tina4 Ruby Application
After=network.target

[Service]
Type=simple
User=deploy
WorkingDirectory=/app
ExecStart=/usr/local/bin/bundle exec puma -C config/puma.rb
Restart=always
RestartSec=5
Environment=RACK_ENV=production

[Install]
WantedBy=multi-user.target
```

Create `/etc/systemd/system/tina4-worker.service`:

```
[Unit]
Description=Tina4 Queue Worker
After=network.target tina4-app.service

[Service]
Type=simple
User=deploy
WorkingDirectory=/app
ExecStart=/usr/local/bin/tina4 queue work
Restart=always
RestartSec=5
Environment=RACK_ENV=production

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable tina4-app tina4-worker
sudo systemctl start tina4-app tina4-worker
sudo systemctl status tina4-app
```

---

11. Scaling

A single server handles many applications. When traffic outgrows one server, you scale.

Multiple Workers

Puma runs multiple worker processes. Configure the count in `config/puma.rb`:

```
workers ENV.fetch("WEB_CONCURRENCY", 4)
threads 5, 5
```

Start with the number of CPU cores on your server. For I/O-heavy applications (database queries, external API calls), double the core count. CPU-bound work benefits less from extra workers.

Load Balancing with Nginx

When you run multiple Tina4 instances, Nginx distributes traffic across them:

```
upstream tina4_backend {
    server 127.0.0.1:7147;
    server 127.0.0.1:7247;
```

```
    server 127.0.0.1:7347;
    server 127.0.0.1:7447;
}

server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://tina4_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Start four instances on different ports. Ruby's framework default is `7147`; use any free ports for the additional instances. Set `TINA4_OVERRIDE_CLIENT=true` so Puma can run without going through `tina4 serve`:

```
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7147 bundle exec puma -C config/puma.rb &
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7247 bundle exec puma -C config/puma.rb &
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7347 bundle exec puma -C config/puma.rb &
TINA4_OVERRIDE_CLIENT=true TINA4_PORT=7447 bundle exec puma -C config/puma.rb &
```

Docker Scaling

With Docker Compose, scale horizontally:

```
docker compose up -d --scale app=4
```

Scaling Considerations

Scaling introduces shared-state problems. When four instances serve requests, each must agree on the state of the world.

Sessions: Store sessions in Redis, not in-memory. Otherwise, a user who logs in on instance 1 appears logged out on instance 2.

Database: SQLite handles one writer at a time. Under high load with multiple instances, switch to PostgreSQL or MySQL. If you must use SQLite, enable WAL mode.

File uploads: Store uploaded files in shared storage (S3, a mounted volume) -- not the local filesystem of a single container.

Cache: Use Redis as the cache backend so all instances share the same cache.

---

12. Monitoring

Your app runs in production. You need to know when it breaks, slows down, or runs out of resources.

Log Aggregation

Switch to JSON-formatted logs for production. Structured logs feed into aggregation services:

```
TINA4_LOG_FORMAT=json
```

Example JSON log entry:

```
{
  "timestamp": "2026-03-22T14:30:00.123Z",
  "level": "WARNING",
  "message": "Rate limit exceeded",
  "request_id": "req-abc123",
  "ip": "203.0.113.42",
  "path": "/api/products",
  "method": "GET"
}
```

Services that ingest JSON logs:

Service	Strengths
Grafana Loki	Open source, pairs with Grafana dashboards
Elastic Stack (ELK)	Full-text search across logs
Datadog	Managed service, correlates logs with metrics
AWS CloudWatch	Native for AWS deployments

Prometheus Metrics

Enable metrics in `.env`:

Metrics are available at `/metrics` in Prometheus format:

```
curl http://localhost:7147/metrics
```

Uptime Monitoring

Point an external monitoring service at your health endpoint:

```
https://yourdomain.com/health
```

Services like Uptime Robot, Pingdom, or Better Uptime ping this endpoint every 30-60 seconds. When it stops responding or returns a non-200 status, you receive an alert.

Application Performance Monitoring (APM)

Uptime monitoring tells you the app is running. APM tells you how well it performs. APM agents track:

- Request latency (average, p95, p99)
- Database query performance (slow queries, connection pool usage)
- Error rates (which endpoints fail, how often)
- Memory and CPU usage over time

A basic monitoring stack for a small team: Uptime Robot for availability alerts (free tier covers it), JSON logs shipped to Grafana Loki for debugging, and `docker stats` for resource usage. Add APM when your application serves enough traffic to warrant the cost.

---

13. Exercise: Docker Deploy

Deploy a Tina4 Ruby application using Docker.

Requirements

- Create a `Dockerfile` that:
- Uses `ruby:3.3-slim` as the base image
- Installs dependencies with `bundle`
- Copies the application code
- Exposes port 7147
- Includes a health check
- Runs the app with Puma
- Create a `docker-compose.yml` that:
- Builds and runs the app
- Starts a Redis container for caching
- Starts a queue worker
- Mounts volumes for data persistence
- Sets environment variables for production
- Create a `/health` endpoint that checks database connectivity
- Build, run, and verify:

```
# Build
docker compose build

# Start
docker compose up -d

# Test health
curl http://localhost:7147/health

# Test the app
curl http://localhost:7147/api/products

# View logs
docker compose logs -f app

# Stop
docker compose down
```

---

14. Solution

The Dockerfile, docker-compose.yml, and Puma config are shown in sections 3 and 4. The health check route is shown in section 5. Combine them in your project, then:

```
docker compose up -d --build
```

```
[+] Building 18.4s
[+] Running 3/3
  Container redis     Started
  Container my-app    Started
  Container worker    Started
```

```
curl http://localhost:7147/health
```

```
{
  "status": "ok",
  "version": "1.0.0",
  "framework": "tina4-ruby",
  "database": "connected",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

The app runs in production mode with Redis caching, persistent data volumes, a queue worker, automatic restarts, and health monitoring.

HTTP Caching, Compression, and ETags

Tina4 applies gzip to eligible dynamic responses and then computes a strong ETag from the final post-compression bytes. The identity and gzip representations therefore have different ETags, and `Vary: Accept-Encoding` keeps intermediary caches from mixing them. A matching `If-None-Match` returns `304 Not Modified` with an empty body while preserving the validators.

Static files use a weak validator derived from file size and modification time. Streaming responses bypass dynamic compression and ETag generation because their complete representation is not buffered. Test both identity and gzip requests when validating cache behavior.

15. Gotchas

1. .env Not Loaded in Docker

Problem: Environment variables from `.env` are not available in the container.

Cause: Docker does not read `.env` files automatically. The `.env` file belongs in `.dockerignore` (never ship secrets in the image).

Fix: Pass environment variables via `docker run -e`, the `environment` section in `docker-compose.yml`, or an `env_file` directive. For secrets, use Docker secrets or your platform's secret management.

2. SQLite Database Lost on Container Restart

Problem: All data disappears when the container restarts.

Cause: The SQLite database file sits inside the container. When the container is recreated, the file is gone.

Fix: Mount a volume for the data directory: `-v $(pwd)/data:/app/data`. In Docker Compose, use a named volume: `volumes: [app-data:/app/data]`.

3. WebSocket Connections Drop Behind Nginx

Problem: WebSocket connections fail or drop behind Nginx.

Cause: Nginx does not proxy WebSocket by default. It treats the upgrade request as a regular HTTP request.

Fix: Add WebSocket proxy headers in your Nginx config:

```
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_read_timeout 86400;
```

4. Puma Workers Not Reconnecting to Database

Problem: Database errors after Puma forks worker processes.

Cause: Puma forks the master process to create workers. The forked processes inherit the parent's database connection, which is invalid after fork.

Fix: Use `on_worker_boot` in Puma config to re-establish the connection: close the inherited one and rebind a fresh `Tina4::Database` from the environment:

```
on_worker_boot do
  Tina4.database&.close
  Tina4.bind_database(Tina4::Database.from_env)
end
```

5. Built-in Server Used in Production

Problem: The server handles one request at a time and performance is terrible.

Cause: WEBrick (the built-in server) is single-threaded. It is for development only.

Fix: Use Puma in production: `bundle exec puma -C config/puma.rb`.

6. Static Files Slow

Problem: CSS, JS, and images load slowly in production.

Cause: Ruby serves static files. Every static file request goes through the Ruby process.

Fix: Serve static files from Nginx (see the Nginx config in section 8). Nginx serves static files from memory-mapped files, orders of magnitude faster than Ruby.

7. Memory Usage Grows Over Time

Problem: The container's memory usage climbs until it crashes with OOM (out of memory).

Cause: Memory leaks -- unclosed database connections, growing caches without TTL, accumulating data in global variables.

Fix: Set TTLs on all cache entries. Close database connections. Avoid storing request data in module-level variables. Use `docker stats` to monitor memory. Set memory limits in Docker Compose: `deploy: {resources: {limits: {memory: 512M}}}`. Set Puma's `worker_timeout` and consider periodic worker restarts.

8. SSL Certificate Not Renewing

Problem: Your HTTPS certificate expires and the site goes down.

Cause: The auto-renewal process failed. Common reasons: DNS changes, firewall blocking port 80 for ACME challenges, or the renewal service crashed.

Fix: Monitor certificate expiry with an external service. Check renewal logs:

```
sudo certbot renew --dry-run
sudo systemctl list-timers | grep certbot
```

9. Scaled Instances Have Different State

Problem: Users see inconsistent data across requests when running multiple app instances.

Cause: In-memory sessions and cache are not shared between instances. A user who logs in on instance 1 appears logged out when the load balancer routes the next request to instance 2.

Fix: Store sessions in Redis. Use Redis as the cache backend. Store uploaded files in shared storage (S3 or a mounted volume). All instances must read from and write to the same data stores.

10. Queue Worker Stops Processing

Problem: The queue worker stops processing jobs without error messages.

Cause: The worker process crashed or was killed by OOM.

Fix: Use `Restart=always` in systemd. Monitor the worker with health checks. Set up alerting on dead letter queue growth.

Building a Complete App

1. Putting It All Together

Twenty chapters of building blocks. Routing. Templates. Databases. ORM. Authentication. Middleware. Queues. WebSocket. Caching. Frontend. GraphQL. Testing. Dev tools. CLI scaffolding. Deployment. Now all of it works together in one application.

TaskFlow -- a task management system with:

- User registration and JWT authentication
- Task creation, assignment, and tracking
- A dashboard with real-time updates via WebSocket
- Email notifications when tasks are assigned
- Caching for dashboard performance
- A full test suite
- Docker deployment

Not a toy project. A production-ready application. Every major Tina4 feature in one codebase.

---

2. Planning the App

Models

Model	Table	Fields
User	users	id, name, email, password*hash, role, created*at
Task	tasks	id, title, description, status, priority, created*by, assigned*to, due*date, completed*at, created*at, updated*at

Relationships

- A User has many Tasks (created by them)
- A User has many Tasks (assigned to them)
- A Task belongs to a User (creator)
- A Task belongs to a User (assignee)

Routes

Method	Path	Description	Auth
POST	/api/auth/register	Register a new user	public
POST	/api/auth/login	Login, get JWT	public
GET	/api/profile	Get current user profile	secured
GET	/api/tasks	List tasks (with filters)	secured
GET	/api/tasks/{id}	Get a single task	secured
POST	/api/tasks	Create a task	secured
PUT	/api/tasks/{id}	Update a task	secured
DELETE	/api/tasks/{id}	Delete a task	secured
GET	/api/dashboard/stats	Dashboard statistics	secured
GET	/admin	Dashboard HTML page	public

Templates

- `base.html` -- Base layout with sidebar and topbar
- `dashboard.html` -- Dashboard with stats, task list, quick actions
- `login.html` -- Login page
- `register.html` -- Registration page

---

3. Step 1: Init Project and Set Up Database

```
tina4 init taskflow --lang ruby
cd taskflow
bundle install
```

Update `.env`:

```
TINA4_DEBUG=true
JWT_SECRET=taskflow-dev-secret-change-in-production
JWT_EXPIRY=86400
```

Create Migrations

Create `migrations/20260322150000_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
```

```
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `migrations/20260322150100_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT DEFAULT '',
    status TEXT NOT NULL DEFAULT 'todo',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to);
CREATE INDEX idx_tasks_created_by ON tasks(created_by);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

Run migrations:

```
tina4 migrate
```

```
Running migrations...
  [UP] 20260322150000_create_users_table.sql
  [UP] 20260322150100_create_tasks_table.sql

  2 migrations applied
```

Verify the database:

```
curl http://localhost:7147/health
```

```
{"status": "ok", "database": "connected"}
```

---

4. Step 2: ORM Models

Create `src/orm/user.rb`:

```
class User < Tina4::ORM
  integer_field :id, primary_key: true
```

```
  string_field :name
  string_field :email
  string_field :password_hash
  string_field :role, default: "user"
  string_field :created_at

  table_name "users"

  has_many :created_tasks, class_name: "Task", foreign_key: "created_by"
  has_many :assigned_tasks, class_name: "Task", foreign_key: "assigned_to"
end
```

Create `src/orm/task.rb`:

```
class Task < Tina4::ORM
  integer_field :id, primary_key: true
  string_field :title
  string_field :description, default: ""
  string_field :status, default: "todo"
  string_field :priority, default: "medium"
  integer_field :created_by
  integer_field :assigned_to
  string_field :due_date
  string_field :completed_at
  string_field :created_at
  string_field :updated_at

  table_name "tasks"

  belongs_to :creator, class_name: "User", foreign_key: "created_by"
  belongs_to :assignee, class_name: "User", foreign_key: "assigned_to"
end
```

---

5. Step 3: Authentication

Create `src/routes/middleware.rb`:

```
def auth_middleware(request, response, next_handler)
  auth_header = request.headers["Authorization"] || ""

  if auth_header.empty? || !auth_header.start_with?("Bearer ")
    return response.json({ error: "Authorization required" }, 401)
  end

  token = auth_header.sub("Bearer ", "")

  unless Tina4::Auth.valid_token(token)
    return response.json({ error: "Invalid or expired token" }, 401)
  end

  request.user = Tina4::Auth.get_payload(token)
  next_handler.call(request, response)
end
```

Create `src/routes/auth.rb`:

```
# @noauth
```

```
Tina4::Router.post("/api/auth/register") do |request, response|
  body = request.body

  errors = []
  errors << "Name is required" if body["name"].nil? || body["name"].empty?
  errors << "Email is required" if body["email"].nil? || body["email"].empty?
  errors << "Password must be at least 8 characters" if body["password"].nil? || body["password"

  return response.json({ errors: errors }, 400) unless errors.empty?

  db = Tina4.database
  existing = db.fetch_one("SELECT id FROM users WHERE email = ?", [body["email"]])
  return response.json({ error: "Email already registered" }, 409) unless existing.nil?

  hash = Tina4::Auth.hash_password(body["password"])
  db.execute("INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)",
    [body["name"], body["email"], hash])

  user = db.fetch_one("SELECT id, name, email, role FROM users WHERE id = last_insert_rowid()")
  response.json({ message: "Registration successful", user: user }, 201)
end

# @noauth
Tina4::Router.post("/api/auth/login") do |request, response|
  body = request.body

  db = Tina4.database
  user = db.fetch_one("SELECT * FROM users WHERE email = ?", [body["email"]])

  if user.nil? || !Tina4::Auth.check_password(body["password"] || "", user["password_hash"])
    return response.json({ error: "Invalid email or password" }, 401)
  end

  token = Tina4::Auth.get_token({
    user_id: user["id"], email: user["email"], name: user["name"], role: user["role"]
  })

  response.json({
    message: "Login successful",
    token: token,
    user: { id: user["id"], name: user["name"], email: user["email"], role: user["role"] }
  })
end

Tina4::Router.get("/api/profile", middleware: "auth_middleware") do |request, response|
  db = Tina4.database
  user = db.fetch_one("SELECT id, name, email, role, created_at FROM users WHERE id = ?",
    [request.user["user_id"]])
  response.json(user)
end
```

Test Registration and Login

```
# Start the server
tina4 serve

# Register a user
curl -X POST http://localhost:7147/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Johnson", "email": "alice@example.com", "password": "securepass123"}'
```

```
{
  "message": "Registration successful",
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

```
# Login
curl -X POST http://localhost:7147/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securepass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

Authentication works. Save the token for the next steps.

---

6. Step 4: Task CRUD

Create `src/routes/tasks.rb`:

```
Tina4::Router.group("/api/tasks", middleware: "auth_middleware") do

  # List tasks with filters
  Tina4::Router.get("") do |request, response|
    db = Tina4.database
    user_id = request.user["user_id"]

    status = request.params["status"]
    priority = request.params["priority"]
    assigned = request.params["assigned_to_me"]

    sql = "SELECT t.*, u1.name AS creator_name, u2.name AS assignee_name FROM tasks t LEFT JOIN
    conditions = []
    params = {}

    if assigned == "true"
      conditions << "t.assigned_to = :user_id"
      params[:user_id] = user_id
    end

    if status && !status.empty?
      conditions << "t.status = :status"
      params[:status] = status
    end

    if priority && !priority.empty?
```

```
    conditions << "t.priority = :priority"
    params[:priority] = priority
  end

  sql += " WHERE #{conditions.join(' AND ')}" unless conditions.empty?
  sql += " ORDER BY t.created_at DESC"

  tasks = db.fetch(sql, params)
  response.json({ tasks: tasks, count: tasks.length })
end

# Get single task
Tina4::Router.get("/{id:int}") do |request, response|
  db = Tina4.database
  id = request.params["id"]

  task = db.fetch_one("SELECT t.*, u1.name AS creator_name, u2.name AS assignee_name FROM task

  if task.nil?
    return response.json({ error: "Task not found" }, 404)
  end

  response.json(task)
end

# Create task
Tina4::Router.post("") do |request, response|
  body = request.body
  user_id = request.user["user_id"]

  if body["title"].nil? || body["title"].empty?
    return response.json({ error: "Title is required" }, 400)
  end

  db = Tina4.database
  db.execute(
    "INSERT INTO tasks (title, description, status, priority, created_by, assigned_to, due_dat
    [body["title"], body["description"] || "", body["status"] || "todo", body["priority"] || "
  )

  task = db.fetch_one("SELECT * FROM tasks WHERE id = last_insert_rowid()")

  # Queue notification if assigned to someone else
  if body["assigned_to"] && body["assigned_to"].to_i != user_id
    assignee = db.fetch_one("SELECT name, email FROM users WHERE id = ?", [body["assigned_to"]
    if assignee
      Tina4::Queue.produce("send-email", {
        to: assignee["email"],
        subject: "New task assigned: #{body['title']}",
        template: "emails/task-assigned.html",
        data: { task_title: body["title"], assignee_name: assignee["name"], assigner_name: req
      })
    end
  end

  # Notify dashboards: the client sends this change over its /ws/tasks
  # socket, and the WebSocket handler (Step 6) relays it to every other
  # connected client via connection.broadcast.
  notification = {
```

```
    type: "task_update",
    action: "created",
    task: task
  }

  # Invalidate cache
  Tina4.cache_delete("dashboard:stats:#{user_id}")

  response.json(task.merge(notify: notification), 201)
end

# Update task
Tina4::Router.put("/{id:int}") do |request, response|
  db = Tina4.database
  id = request.params["id"]
  body = request.body

  existing = db.fetch_one("SELECT * FROM tasks WHERE id = ?", [id])
  return response.json({ error: "Task not found" }, 404) if existing.nil?

  completed_at = existing["completed_at"]
  if body["status"] == "done" && existing["status"] != "done"
    completed_at = Time.now.strftime("%Y-%m-%d %H:%M:%S")
  elsif body["status"] && body["status"] != "done"
    completed_at = nil
  end

  db.execute(
    "UPDATE tasks SET title = ?, description = ?, status = ?, priority = ?, assigned_to = ?, d
    [
      body["title"] || existing["title"],
      body["description"] || existing["description"],
      body["status"] || existing["status"],
      body["priority"] || existing["priority"],
      body.key?("assigned_to") ? body["assigned_to"] : existing["assigned_to"],
      body["due_date"] || existing["due_date"],
      completed_at,
      id
    ]
  )

  # Notify dashboards over the /ws/tasks socket (relayed by the
  # WebSocket handler in Step 6 via connection.broadcast).
  task = db.fetch_one("SELECT * FROM tasks WHERE id = ?", [id])
  notification = {
    type: "task_update",
    action: "updated",
    task: task
  }

  # Invalidate cache
  Tina4.cache_delete("dashboard:stats:#{request.user['user_id']}")

  response.json(task.merge(notify: notification))
end

# Delete task
Tina4::Router.delete("/{id:int}") do |request, response|
  db = Tina4.database
```

```
    id = request.params["id"]

    existing = db.fetch_one("SELECT * FROM tasks WHERE id = ?", [id])
    return response.json({ error: "Task not found" }, 404) if existing.nil?

    db.execute("DELETE FROM tasks WHERE id = ?", [id])

    # Notify dashboards over the /ws/tasks socket (relayed by the
    # WebSocket handler in Step 6 via connection.broadcast).
    notification = {
      type: "task_update",
      action: "deleted",
      task: { id: id }
    }

    Tina4.cache_delete("dashboard:stats:#{request.user['user_id']}")

    response.json({ deleted: id, notify: notification }, 200)
  end

end
```

Test the Task API

```
# Set your token from the login step
TOKEN="eyJhbGciOiJIUzI1NiIs..."

# Create a task
curl -X POST http://localhost:7147/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Design database schema", "priority": "high"}'
```

```
{
  "id": 1,
  "title": "Design database schema",
  "priority": "high",
  "status": "todo",
  "created_by": 1,
  "created_at": "2026-03-22 10:00:00"
}
```

```
# Create more tasks
curl -X POST http://localhost:7147/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Build API endpoints", "priority": "high"}'

curl -X POST http://localhost:7147/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Write documentation", "priority": "medium", "due_date": "2026-04-01"}'

# List all tasks
curl http://localhost:7147/api/tasks \
  -H "Authorization: Bearer $TOKEN"

# Update a task status
curl -X PUT http://localhost:7147/api/tasks/1 \
  -H "Content-Type: application/json" \
```

```
  -H "Authorization: Bearer $TOKEN" \
  -d '{"status": "done"}'

# Delete a task
curl -X DELETE http://localhost:7147/api/tasks/3 \
  -H "Authorization: Bearer $TOKEN"
```

Every endpoint responds. Create. Read. Update. Delete. The CRUD cycle works.

---

7. Step 5: Dashboard Stats with Caching

Create `src/routes/dashboard.rb`:

```
Tina4::Router.get("/api/dashboard/stats", middleware: "auth_middleware") do |request, response|
  user_id = request.user["user_id"]

  cache_key = "dashboard:stats:#{user_id}"
  stats = Tina4.cache_get(cache_key)

  if stats.nil?
    db = Tina4.database

    total = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE assigned_to = ? OR created_b
    todo = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = ? OR created_b
    in_progress = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = ? OR cr
    done = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = ? OR created_b
    overdue = db.fetch_one("SELECT COUNT(*) AS count FROM tasks WHERE (assigned_to = ? OR create

    recent = db.fetch(
      "SELECT t.*, u.name AS assignee_name FROM tasks t LEFT JOIN users u ON t.assigned_to = u.i
    )

    stats = {
      total_tasks: total["count"],
      todo: todo["count"],
      in_progress: in_progress["count"],
      done: done["count"],
      overdue: overdue["count"],
      recent_tasks: recent
    }

    Tina4.cache_set(cache_key, stats, 30)
  end

  response.json(stats)
end

Tina4::Router.get("/admin") do |request, response|
  response.render("dashboard.html", {
    title: "TaskFlow Dashboard"
  })
end
```

Verify the stats endpoint:

```
curl http://localhost:7147/api/dashboard/stats \
  -H "Authorization: Bearer $TOKEN"
```

```
{
  "total_tasks": 2,
  "todo": 1,
  "in_progress": 0,
  "done": 1,
  "overdue": 0,
  "recent_tasks": [...]
}
```

Hit it again within 30 seconds. The second response comes from cache -- no database queries.

---

8. Step 6: WebSocket Real-Time Updates

Create `src/routes/task_ws.rb`. WebSocket handlers are registered as routes with `Tina4::Router.websocket`. The handler receives `(connection, event, data)` where `event` is a symbol (`:open`, `:message`, `:close`):

```
Tina4::Router.websocket("/ws/tasks") do |connection, event, data|
  case event
  when :open
    connection.send({ type: "connected", message: "Listening for task updates" }.to_json)

  when :message
    message = JSON.parse(data)
    if message["type"] == "task_update"
      # Relay a task change to every other client on /ws/tasks.
      # connection.broadcast is path-scoped: only clients on /ws/tasks receive it.
      connection.broadcast(data)
    elsif message["type"] == "ping"
      connection.send({ type: "pong" }.to_json)
    end

  when :close
    # Connection closed -- cleanup if needed
  end
end
```

Any user creates, updates, or deletes a task. The task routes (Step 4) notify connected clients over the same `/ws/tasks` socket, and `connection.broadcast` fans the change out to every other client on that path.

---

9. Step 7: Email Notifications

Create the queue consumer for task assignment notifications:

```
Tina4::Queue.consume("send-email") do |job|
  mail = Tina4::Messenger.new
  mail.to = job.payload["to"]
  mail.subject = job.payload["subject"]

  if job.payload["template"]
    mail.html_template = job.payload["template"]
    mail.template_data = job.payload["data"] || {}
```

```
  else
    mail.body = job.payload["body"] || ""
  end

  mail.send
  true
end
```

Create `src/templates/emails/task-assigned.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body>
    <div class="container">
        <h2>New Task Assigned</h2>
        <p>Hi {{ assignee_name }},</p>
        <p><strong>{{ assigner_name }}</strong> assigned you a new task:</p>

        <div class="card">
            <div class="card-body">
                <h3>{{ task_title }}</h3>
            </div>
        </div>

        <p><a href="http://localhost:7147/admin">View Dashboard</a></p>
    </div>
</body>
</html>
```

Start the queue worker in a separate terminal:

```
tina4 queue work
```

---

10. Step 8: Dashboard Template

Create `src/templates/dashboard.html`:

```
{% extends "base.html" %}

{% block title %}TaskFlow Dashboard{% endblock %}

{% block content %}
<h1>Dashboard</h1>

<div id="stats" class="row mb-4">
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Total</h6><h2 id="stat-total">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Todo</h6><h2 id="stat-todo">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">In Progress</h6><h2 id="stat-progress">--</h2>
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Done</h6><h2 id="stat-done">--</h2>
```

```
    </div></div></div>
    <div class="col-md-2"><div class="card"><div class="card-body">
        <h6 class="text-muted">Overdue</h6><h2 id="stat-overdue" class="text-danger">--</h2>
    </div></div></div>
</div>

<div class="row">
    <div class="col-md-8">
        <div class="card">
            <div class="card-header d-flex justify-content-between">
                <span>Recent Tasks</span>
                <button class="btn btn-sm btn-primary" onclick="showNewTaskForm()">
                    New Task
                </button>
            </div>
            <div class="card-body">
                <table class="table table-striped" id="task-table">
                    <thead>
                        <tr><th>Title</th><th>Assignee</th><th>Priority</th><th>Status</th></tr>
                    </thead>
                    <tbody id="task-list"></tbody>
                </table>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card">
            <div class="card-header">Live Updates</div>
            <div class="card-body" id="live-updates">
                <p class="text-muted">Connecting...</p>
            </div>
        </div>
    </div>
</div>

<script src="/js/frond.min.js"></script>
<script>
    var token = localStorage.getItem("token");
    if (token) frond.setToken(token);

    // Load dashboard stats
    frond.get("/api/dashboard/stats", function (data) {
        document.getElementById("stat-total").textContent = data.total_tasks;
        document.getElementById("stat-todo").textContent = data.todo;
        document.getElementById("stat-progress").textContent = data.in_progress;
        document.getElementById("stat-done").textContent = data.done;
        document.getElementById("stat-overdue").textContent = data.overdue;

        var tbody = document.getElementById("task-list");
        tbody.innerHTML = "";
        (data.recent_tasks || []).forEach(function (task) {
            var tr = document.createElement("tr");
            var badgeColor = {todo: "secondary", in_progress: "warning",
                              done: "success"};
            tr.innerHTML = "<td>" + task.title + "</td>" +
                "<td>" + (task.assignee_name || "Unassigned") + "</td>" +
                "<td>" + task.priority + "</td>" +
                "<td><span class='badge bg-" + (badgeColor[task.status] || "secondary") +
                "'>" + task.status + "</span></td>";
```

```
            tbody.appendChild(tr);
        });
    });

    // WebSocket for live updates
    var ws = frond.ws("/ws/tasks");
    var updates = document.getElementById("live-updates");

    ws.on("open", function () {
        updates.innerHTML = "<p class='text-success'>Connected - listening for updates</p>";
    });

    ws.on("message", function (raw) {
        var msg = JSON.parse(raw);
        if (msg.type === "task_update") {
            var div = document.createElement("div");
            div.innerHTML = "<strong>" + msg.action + ":</strong> " + msg.task.title;
            updates.prepend(div);

            // Refresh stats
            frond.get("/api/dashboard/stats", function () {});
        }
    });
</script>
{% endblock %}
```

Open `http://localhost:7147/admin` in your browser to see the full dashboard with stats, task list, and live update panel.

---

11. Step 9: Tests

Create `tests/taskflow_spec.rb`:

```
require "tina4"

RSpec.describe "TaskFlow" do
  let(:client) { Tina4::TestClient.new }
  let(:token) do
    email = "test#{rand(100000)}@example.com"
    client.post("/api/auth/register", {
      name: "Test User", email: email, password: "password123"
    })
    result = client.post("/api/auth/login", {
      email: email, password: "password123"
    })
    result.json["token"]
  end

  it "registers a user" do
    result = client.post("/api/auth/register", {
      name: "Alice", email: "alice#{rand(100000)}@example.com", password: "securePass123"
    })
    expect(result.status).to eq(201)
  end

  it "rejects duplicate email" do
```

```
  email = "dup#{rand(100000)}@example.com"
  client.post("/api/auth/register", {
    name: "First", email: email, password: "password123"
  })
  result = client.post("/api/auth/register", {
    name: "Second", email: email, password: "password123"
  })
  expect(result.status).to eq(409)
end

it "creates a task" do
  result = client.post("/api/tasks", { title: "Test Task" },
    headers: { "Authorization" => "Bearer #{token}" })
  expect(result.status).to eq(201)
  expect(result.json["title"]).to eq("Test Task")
  expect(result.json["status"]).to eq("todo")
end

it "lists tasks" do
  client.post("/api/tasks", { title: "Task 1" }, headers: { "Authorization" => "Bearer #{token
  client.post("/api/tasks", { title: "Task 2" }, headers: { "Authorization" => "Bearer #{token

  result = client.get("/api/tasks", headers: { "Authorization" => "Bearer #{token}" })
  expect(result.status).to eq(200)
  expect(result.json["tasks"]).to be_an(Array)
end

it "updates a task status" do
  created = client.post("/api/tasks", { title: "Update Me" },
    headers: { "Authorization" => "Bearer #{token}" })
  id = created.json["id"]

  result = client.put("/api/tasks/#{id}", { status: "done" },
    headers: { "Authorization" => "Bearer #{token}" })
  expect(result.status).to eq(200)
  expect(result.json["status"]).to eq("done")
  expect(result.json["completed_at"]).not_to be_nil
end

it "deletes a task" do
  created = client.post("/api/tasks", { title: "Delete Me" },
    headers: { "Authorization" => "Bearer #{token}" })
  id = created.json["id"]

  result = client.delete("/api/tasks/#{id}",
    headers: { "Authorization" => "Bearer #{token}" })
  expect(result.status).to eq(204)
end

it "returns dashboard stats" do
  client.post("/api/tasks", { title: "Stats Task" },
    headers: { "Authorization" => "Bearer #{token}" })

  result = client.get("/api/dashboard/stats",
    headers: { "Authorization" => "Bearer #{token}" })
  expect(result.status).to eq(200)
  expect(result.json["total_tasks"]).to be >= 1
end
```

```
  it "requires authentication for tasks" do
    result = client.get("/api/tasks")
    expect(result.status).to eq(401)
  end
end
```

Run the tests:

```
tina4 test
```

```
Running tests...

  TaskFlow
    registers a user
    rejects duplicate email
    creates a task
    lists tasks
    updates a task status
    deletes a task
    returns dashboard stats
    requires authentication for tasks

  8 examples, 0 failures (0.84s)
```

All green. The application works.

---

12. Step 10: Docker Deployment

Use the Dockerfile and docker-compose.yml from Chapter 34. Create `.env.production`:

```
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
JWT_SECRET=your-production-secret-at-least-32-characters
TINA4_DATABASE_URL=sqlite:///data/app.db
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://redis:6379
```

Deploy:

```
docker compose up -d --build
```

Verify:

```
curl http://localhost:7147/health
```

```
{"status": "ok", "version": "1.0.0", "database": "connected"}
```

TaskFlow runs in production. Authenticated APIs. Real-time WebSocket updates. Email notifications. Cached dashboard stats. Tested. Dockerized.

---

13. The Complete Project Structure

```
taskflow/
├── .env
├── .env.example
├── .gitignore
├── Gemfile
├── Gemfile.lock
├── app.rb
├── Dockerfile
├── docker-compose.yml
├── config/
│   └── puma.rb
├── src/
│   ├── routes/
│   │   ├── auth.rb               # Registration, login
│   │   ├── tasks.rb              # Task CRUD
│   │   ├── dashboard.rb          # Dashboard stats + page
│   │   ├── middleware.rb         # Auth middleware
│   │   └── task_ws.rb            # WebSocket event handlers
│   ├── orm/
│   │   ├── user.rb               # User model with relationships
│   │   └── task.rb               # Task model with relationships
│   ├── migrations/
│   │   ├── 20260322150000_create_users_table.sql
│   │   └── 20260322150100_create_tasks_table.sql
│   ├── templates/
│   │   ├── base.html             # Base layout
│   │   ├── dashboard.html        # Dashboard page
│   │   ├── emails/
│   │   │   └── task-assigned.html # Assignment notification
│   │   └── errors/
│   │       ├── 404.html
│   │       └── 500.html
│   └── public/
│       ├── css/
│       │   └── tina4.css
│       └── js/
│           ├── tina4.min.js
│           └── frond.min.js
│   └── locales/
│       └── en.json
├── data/
│   └── app.db
├── logs/
├── secrets/
└── tests/
    └── taskflow_spec.rb
```

Every file has a purpose. Every directory follows the convention. A new developer looks at this structure and knows where to find things.

---

14. What You Built

This chapter used every major concept from the book:

Feature	Chapter
Route definitions	Chapter 2
Request/response handling	Chapter 3
Frond templates	Chapter 4
Database queries	Chapter 5
ORM models (User, Task)	Chapter 6
JWT authentication	Chapter 8
Auth middleware	Chapter 10
Queue-based email	Chapter 12
Email notifications (Messenger)	Chapter 16
Cache (dashboard stats)	Chapter 11
Frontend (tina4css dashboard)	Chapter 17
WebSocket (live updates)	Chapter 23
Testing (full test suite)	Chapter 18
Docker deployment	Chapter 34

---

15. What to Build Next

TaskFlow is a solid foundation. Here are ideas for extending it.

Features:

- **Task comments** -- Add a Comment model with a `task_id` foreign key. Display comments on the task detail page.
- **File attachments** -- Let users upload files to tasks. Store them in `data/uploads/` and serve them via a route.
- **Team management** -- Add a Team model. Users belong to teams. Tasks are scoped to teams.
- **Task labels/tags** -- Many-to-many relationship between tasks and labels for categorization.
- **Due date reminders** -- Use the queue system to schedule reminder emails 24 hours before a task's due date.
- **Activity log** -- Record every change to a task (who changed what, when) for audit trails.
- **Search** -- Full-text search across task titles and descriptions.
- **Calendar view** -- Render tasks on a calendar based on their due dates.
- **Mobile API** -- The API already works for mobile apps. Add push notification support.

Technical improvements:

- **Rate limiting per user** -- Replace the global rate limiter with per-user limits.
- **Database upgrade** -- Switch from SQLite to PostgreSQL for better concurrency.
- **CI/CD pipeline** -- Add GitHub Actions to run tests on every push.
- **API documentation** -- Generate OpenAPI/Swagger docs from your route definitions.
- **Internationalization** -- Add `src/locales/` files for multiple languages.

---

16. Gotchas

1. Circular Dependencies Between Models

Problem: User loads tasks, tasks load users, infinite loop.

Fix: Use lazy loading. Do not eager-load relationships that create cycles.

2. Cache Not Invalidated on Related Model Changes

Problem: Dashboard stats are stale after task updates.

Fix: Invalidate cache in every write operation: `Tina4.cache_delete("dashboard:stats:#{user_id}")`.

3. Queue Worker Not Processing Emails

Problem: Task assignment emails are queued but never sent.

Fix: Run `tina4 queue work` in a separate terminal or as a systemd service.

4. Token Expired During Long Sessions

Problem: Users get logged out while actively using the dashboard.

Fix: Set a reasonable JWT expiry (`86400` for 24 hours) and implement token refresh.

5. WebSocket Notifications Not Received

Problem: Real-time dashboard updates do not work.

Fix: Ensure the WebSocket endpoint is configured, the server is running, and the client JavaScript connects to the correct URL. Check the browser console for connection errors.

6. Database Locked Under Load

Problem: SQLite returns "database is locked" errors with concurrent requests.

Fix: For production with concurrent users, switch to PostgreSQL. SQLite handles one writer at a time.

7. Missing Migration Before Deployment

Problem: The app crashes in production because a table does not exist.

Fix: Always run `tina4 migrate` as part of your deployment pipeline. Add it to your Dockerfile or CI/CD script.

---

17. Closing Thoughts -- The Tina4 Philosophy

You built a complete application. User auth. CRUD. Real-time updates. Email. Caching. Tests. Deployment. Your project has one dependency: `tina4`.

No separate ORM gem. No template engine gem. No authentication library. No WebSocket server. No caching library. No testing framework. No CLI tool. No CSS framework. No JavaScript helpers. All built in.

One framework. Zero extra dependencies. Everything you need.

The same patterns work in PHP, Python, and Node.js. Same project structure. Same CLI commands. Same `.env` variables. Same template syntax. Learn Tina4 once. Use it everywhere.

Build things. Ship them. Keep it simple.

Release Notes

v3.13.138 (2026-09-24) - Safer requests, templates, and database operations

Licensing from this release. Tina4 is available under MPL-2.0, with separate commercial terms available from Code Infinity. Copyright Code Infinity. Previously published releases retain their original licences.

This release brings together the framework hardening work, database correctness fixes, measured-time guidance for the AI skills, and verifiable release artifacts.

Request and template boundaries. Frond escapes output by default and preserves only explicitly trusted output. Review templates that intentionally render HTML when upgrading; never mark untrusted input as safe. Header validation, request-body limits, security headers, CORS handling, model-field validation, and diagnostic redaction follow the shared contracts. Explicit Content-Type headers survive response conversion and binary bodies preserve bytes.

Credentials and development access. API clients retain credentials only for their configured origin, including uploads, downloads and streams. Use a separate client when authenticating to another service. Forwarded host and protocol require a trusted proxy. Development reads and writes enforce the raw-peer, Host and browser-origin boundary; remote non-MCP development access requires the dedicated token. The documented MCP transport API-key fallback is unchanged. File access checks resolved paths so symlinks cannot expose secrets or escape the project.

Database and service correctness. Statements that return rows retain those rows, filtered counts and write results agree across adapters, SQL placeholders preserve literal text, and ORM fields mapped to database columns read back correctly. Database operations and explicit transactions reserve their connection; a fully occupied pool fails clearly instead of sharing an in-use connection. Session transports reuse MongoDB connections and handle replica-set replies. Mail transports verify certificates and require STARTTLS when configured. Optional driver errors identify the needed install, and unconfirmed broker writes fail clearly.

Skills and release evidence. The maintainer skill estimates work from measured agent durations. Release packages carry checksums, an SPDX software inventory, and build provenance; release Actions are pinned to commits. The updated PowerShell skills installer is digitally signed. The ISO programme remains in progress; this release makes no certification or conformance claim.

The accompanying CLI 3.8.91 verifies update checksums before replacing an installed binary, refuses missing checksum manifests, and corrects generated Node.js package imports. It also includes the browser-opening and CI behavior fixes. Tina4 JS 1.7.2 ships the merged client hardening and verified browser bundle. Refresh the AI skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` or the documented PowerShell installer.

v3.13.105 (2026-08-19) - Cross-API invariants, honest logs, portable tests

A bug release with teeth. Five audit bugs closed in the queue and ORM cache seams — the places where two spellings of the same intent quietly disagreed. Route inspection stops booting the app. The startup log stops lying about `@noauth`. Hot reload reaches the modules that captured the changed one. Firebird's migration ledger tolerates any case the driver hands back. And every framework developer skill gained a spine: it announces every step before taking it, and it warns with ■ when it is out of date.

Queue and ORM audit bugs — five fixed

- `Model.clear_cache()` **invalidates the DB layer too.** When both

`TINA4_AUTO_CACHING` and `TINA4_DB_CACHE` are enabled, a manual `clear_cache()` used to leave the DB-layer cache holding stale rows. It now cascades to `db.cache_clear()` on this model's bound connection.

- `Queue.retry()` **revives every dead letter.** The no-arg form used a

generator inside `any(...)`, which short-circuits after the first success. Now it materialises the loop so all N dead letters revive, not just one.

- `Job.retry()` **removes the dead-letter file.** Iterating `dead_letters()`

and calling `.retry()` on each used to leave the failed directory carrying every revived job. The next `dead_letters()` call re-reported them and consumers processed each job twice.

- **MongoDB** `retry_job(id)` **finds the dead letter.** The old filter looked

for `{_id: job_id, topic: self._topic, status: "failed"}` — three reasons that combination could never match a real dead-letter doc (fresh `_id`, `.dead_letter` topic namespace, `dead` status). The new filter reads the dead-letter namespace by `data.id`, deletes the record, and upserts the original back to pending.

- **MongoDB** `purge(status)` **returns a count.** Used to return `None` and

honour only the `pending` status. Now returns `deleted_count`, honours every status, and scopes the delete so `purge("pending")` no longer nukes completed or reserved docs under the same topic.

Route inspection scans, never boots

`tina4 routes` walks canonical route files without executing `app.py` or starting the server. The prior implementation booted the app on the same port and terminated whichever process was holding it — a nasty surprise for anyone running `tina4 routes` in a live shell. Closes tina4-python #104.

Startup log tells the truth about @noauth / @secured

Python decorators apply bottom-up, so `@post()` logged the route as `auth=required` a microsecond before the outer `@noauth()` flipped the flag. `@noauth()` and `@secured()` now emit a corrective `Route auth updated:` line whenever they change the auth default on a route another decorator already logged. Closes tina4-python #103, where a public login endpoint appeared to log its own security bug. Python-only — PHP, Ruby and Node do not log an auth field at registration time.

Hot reload reaches every module that imports the changed one

A route file that did `from src.orm.Todo import Todo` kept its stale `Todo` reference after the model changed — the API then lied about the schema on the next request. `_auto_discover` now cascades a re-import to every module that captured a binding from the changed one, recursive with a visited set, bounded to the discovery scope, fail-loud. Closes tina4-python #102.

Firebird migration ledger is case-agnostic

`tina4_migration` reads and writes work regardless of the case the Firebird driver returns for the `migration_name` column. Uses the atomic sequence pattern the other engines already have.

Test-harness fixes

- **SIGTERM port probe checks both interfaces.** The graceful-shutdown test

probed only `0.0.0.0` with `SO_REUSEADDR`, which succeeds on macOS while the framework's default `127.0.0.1` listener holds the port — a silent false negative on every Mac dev machine. Now probes both interfaces and refuses to conclude the port is free until both agree.

- **MySQL connect-timeout assertion tolerates driver cleanup time.** The

assertion compared the outer clock to the wrapper's rendered message. mysql-connector adds around 0.7 seconds of cleanup after aborting, so the two stopwatches legitimately differed. The assertion now parses the reported elapsed via regex and checks it against the configured bound, not against the outer clock.

Skills grow a spine

- **Announce before you act.** Every framework developer skill now instructs

itself to say what it is about to do — one line before each file write, migration, or dependency install — so the developer can stop the work before it spends their afternoon. Same rhythm across Python, PHP, Ruby and Node.

- ■ **stale-skill detection.** Every framework developer skill also checks

whether a newer skill is available (compared against the latest published skill version at `https://tina4.com/skills/<name>/version`, never against the project's framework version) and prepends ■ to every reply if it is behind. Your framework version stays your call; the skill just refuses to pretend its advice is current.

Upgrade notes

- No API changes. All fixes preserve existing signatures.
- Consumers iterating `dead_letters()` and calling `job.retry()` on each — now safe. Previously it produced duplicates on the next `dead_letters()` call.
- `Model.clear_cache()` under `TINA4_DB_CACHE=true` now also flushes the DB-layer cache. This is the expected behaviour.
- Lab full-suite verified: 25,263 tests across all four frameworks, zero failures, zero skips.

v3.13.104 (2026-08-17) - OIDC SSO and PostGIS land

Two big features, uniform across all four frameworks: a provider-neutral OIDC handoff for SSO logins, and a shared `Point` contract for PostGIS geospatial queries. Both wire through the same small API in Python, PHP, Ruby and Node.

OIDC / OpenID Connect — provider-neutral, session handoff

`Sso.from_issuer(...)` discovers a real OIDC provider (Keycloak, Auth0, Okta, anything speaking the standard) via its `.well-known/openid-configuration` endpoint and hands a validated identity into an existing Tina4 `Session`. The `login` / `callback` / `refresh` / `logout` cycle runs against the real provider — no mock. PKCE with S256 by default. Reserved SSO session keys (`Sso.PENDING_KEY`, `Sso.SESSION_KEY`) never leak through `session.all()`.

Opt in with `TINA4_SSO_ISSUER` plus the client id, secret and redirect URI.

PostGIS spatial support

`PointField` on an ORM model creates the right column for whichever engine you are on — PostGIS `geometry(Point, SRID)` on PostgreSQL, JSON on the others. `select_distance`, `intersects` and `bbox` predicates read the same way across engines but use each engine's native functions where available. GeoJSON in, GeoJSON out. Refuses hostile input (garbage geometry, injected column names, NaN or Inf ordinates) before any SQL runs.

Opt in by declaring a `PointField` on a model. Existing models unchanged.

Swagger is SSO-aware

The Swagger generator recognises OIDC-secured routes and emits the right `security` scheme in the OpenAPI 3.0.3 spec.

v3.13.102 (2026-08-15) - Unified client and CI hardening

Small release. Framework developer skills teach the unified `tina4 init` / `tina4 serve` workflow consistently across the four languages, and CI now verifies the downloaded native CLI asset before running the metrics handoff.

v3.13.103 (2026-08-16) - Metrics you can trust, releases that cannot lie

This release restores one public version across Python, PHP, Ruby, and Node.js. Version 3.13.102 reached PyPI and Packagist, but its RubyGems and npm packages still contained 3.13.101. Skip 3.13.102 and upgrade to 3.13.103.

What changed

• Dev-admin hands metrics work to the signed Tina4 client 3.8.76. Frameworks do

not keep a second metrics analyser or silently fall back to one.

• `has_referencing_test` means that test source refers to the file. It does not

claim that the test ran, passed, or covered the file.

• Frond expression parsing and evaluation are split into smaller internal steps,

while public APIs and the shared 84-case expression corpus remain unchanged.

• Tina4 skills lead with `tina4 init` and `tina4 serve`, and keep scaffolding

prominent in every language.

• Release workflows now reject a tag that differs from the source package

version before publishing.

No runtime packages were added. Language extensions remain extensions, not framework dependencies.

+## v3.13.101 (2026-08-14) - One AI client, four languages

One class. Three methods. No provider SDK. Tina4 applications can now call local models, OpenAI, and Anthropic through the same small API: `Tina4::Ai.chat`, `Tina4::Ai.complete`, and `Tina4::Ai.embed`.

What ships

- The native `tina4 metrics` CLI now owns every code-health calculation. Framework-level metrics commands and quick census implementations are gone.
- Dev-admin keeps its metrics tab through two thin native-JSON adapters: full analysis and file detail.
- `Tina4::Ai.chat` returns one `ChatResponse` with text, model, token usage, finish reason, and the raw provider response.
- `Tina4::Ai.complete` sends one user message and returns its text.
- `Tina4::Ai.embed` accepts one string or a list and preserves that input shape in its output.
- `stream: true` yields ordered text deltas from OpenAI-compatible and Anthropic streams.

Failure is data

Hosted providers refuse to send a request without `TINA4_AI_KEY`. Connection and total-request deadlines stay separate. Retries cover connection failures, HTTP 429, and HTTP 5xx responses, but stop after the first streamed delta. Errors never include the key, prompt, or provider response body.

Upgrade notes

This release adds a new API. Vision, image generation, tools, agents, and conversation state remain outside this release. The client uses Ruby's standard library; your dependency tree does not grow.

v3.13.100 (2026-08-14) - Frond keeps the whole page

This fast-follow release fixes template inheritance and puts a ceiling on Frond's caches. The template engine now rejects a second `{% extends %}` tag, preserves nested root blocks, and evicts stale compiled state instead of letting a long-running process grow without limit.

Template inheritance

- A template may declare one parent. A second `{% extends %}` now raises a clear error instead of changing the parent without warning (`6bed111`)
- Multi-level inheritance now resolves each parent without recursing through the same child again. Nested root blocks survive the final substitution pass (`914bc16`, `1a85fe4`)

Bounded caches

- Template, fragment, and expression caches now have size bounds and TTL sweeps. Long-running workers release stale entries while rendered output stays unchanged (`5f49e32`)

Frond extension scope

- Class calls to `add_filter`, `add_global`, and `add_test` remain process-global. Instance calls now affect that renderer only, so one request or test cannot seed every Frond instance created later (`ad4e696`)

AI skill installer

- Skill downloads retry transient network and HTTP failures. A short GitHub raw-content outage no longer drops the whole install (`386dfa8`)

Release consistency

- `Tina4::VERSION` and both current-version markers in the AI-facing guide now agree on `3.13.100`. The version guard caught the stale guide before release (`f4b0383`)

Upgrade notes

No valid template syntax changes. Code that relied on an instance extension leaking globally must register on `Frond` itself. A template with two `{% extends %}` tags was already contradictory; it now fails at the line that needs fixing. Cache eviction may trigger a reparse, but it does not change the rendered bytes.

The Frond AOT compiler is not part of this release. Parser and compiler parity for Ruby and Node.js remains planned for `3.13.101`.

v3.13.99 (2026-08-13) - Stability, parity, and secure by default

The biggest step yet on the road to **3.14 stable**. This release closes out Phases 1 through 5 of the v3 parity audit: roughly thirty breaking changes, and every one of them is a security fix, a parity fix, or a correctness fix, never a change made for its own sake. Two conformance grids, logging and database adapters, are now fully proven against real services in all four frameworks, and a small batch of genuinely new bugs gets fixed alongside them. Read "Possible breaking" before you upgrade: Ruby carries more of these than any other framework this time, including a renamed route-param accessor and a corrected request-body merge.

Security

Security now defaults on instead of requiring opt-in.

• Security headers emit by default, including `Content-Security-Policy: default-src 'self'`, which blocks inline scripts and third-party CDNs. Relax the policy with `TINA4_CSP`; HSTS emits only on HTTPS, when `TINA4_HSTS` is set (`6e44205`)

• The CSRF `403` body is unified to `{error, code, message, status}`, where Ruby used to send `{error: "CSRF_INVALID"}`. `TINA4_CSRF=true` now actually attaches the CSRF middleware, where it used to be inert, and a blank `TINA4_SECRET` fails closed now instead of minting a forgeable public-default token (`dbcd6a2`)

• The dev server binds `127.0.0.1` by default; set `TINA4_HOST=0.0.0.0` to expose it. A cross-origin `/__dev` mutation is refused, and `.env` is never served through the file endpoints (`ab06e5c`)

• A symlink whose real path escapes the public directory is refused, dotfiles (`.env`, `.git`) 404 instead of serving, and the `src/assets` / `assets` search directories are dropped. `TINA4_PUBLIC_DIR` is honoured now (`e24a3fa`)

• `{% include %}`, `{% extends %}`, and `{% import %}` in a Frond template are confined to the templates directory. A path that escapes it, with `..`, an absolute path, or a symlink, now raises (`a1ff8af`)

• `tina4 serve` on a busy port no longer kills whatever holds it. It reclaims only a port held by an identifiable Tina4 dev server, refuses a foreign holder, and honours `TINA4_NO_TAKEOVER` / `--no-kill` (`75f78ae`)

• A hostile inbound `X-Request-ID` (CRLF, illegal characters, an over-long value) is sanitized to a fresh id instead of echoed back raw, closing a response-header and log-injection path (`1bfc060`)

• The dead `render_production_error` function is removed; nothing called it. The dev error overlay now redacts `Authorization`, `Cookie`, and `Set-Cookie` headers plus secret-looking body fields, and caps the rendered stack at 50 frames (`93c1e06`)

• A repeated multipart file field now yields a list instead of silently dropping every upload but the last. The new safe-save helper rejects `..` and absolute filenames, and an over-limit upload now answers `413` mid-stream, per chunk, instead of after buffering the whole body (`dab136f`)

- A reflected XSS in the `403` error page is closed: the raw request path is now `CGI.escapeHTML`-escaped before it renders (`f2b8d84`)

Data integrity

Footguns that used to fail silently now fail loud, or stop failing at all.

- An unparseable or unsupported MongoDB WHERE clause now raises instead of silently matching every document. A DELETE or UPDATE with no WHERE is rejected outright. Write an explicit WHERE, or call `truncate()` for the whole collection (`d45c119`)
- `truncate()` on a Mongo collection now actually empties it. It used to report success while leaving every document in place (`abc5aa5`)
- The MSSQL adapter now raises on a genuinely unbindable parameter type instead of emitting it as a bareword. Code relying on the old silent stringify now errors, which is the point: it was preventing an injection/corruption risk. MSSQL pagination converges on `OFFSET`/`FETCH` in all four frameworks, and binary parameters travel as `0x` literals (`937a35c`)
- Firebird write results are correct now: `db.insert.last_id` and `db.update` / `db.delete.affected_rows` return real values instead of `nil` or `0` (`286cd03`)

ORM and validation

Ten fixes bring the ORM's write path, validation, and generated schema into agreement across all four frameworks. Ruby has the longest list this release.

- `decimal_field` now emits a real `DECIMAL(p,s)` column, where it used to emit `REAL` and silently drop precision and scale. Generated DDL changes; an existing `REAL` column is unaffected until you re-create it (`df93891`)
- A foreign-key auto `related_name` is now smart-pluralized (`Category` becomes `categories`, not `categorys`). Code using the misspelled accessor must update (`df93891`)
- `validate()` on save now enforces length, type, and format, where it used to check only for `null`. A model that previously saved an over-length or wrong-format value now returns `false` from `save()` and writes nothing (`b427f8b`)
- The two validators now speak one canonical message vocabulary; `in_list` renders a compact JSON list (`must be one of ["a","b","c"]`) (`b427f8b`)
- `create_table()` now injects the configured `soft_delete_field` for a soft-delete model that does not declare it itself (`a6ce6ad`)
- A soft-deleted child is no longer returned through relationship traversal, lazy or eager (`df9bbe6`)
- A REST list `?page` below 1 now clamps to page 1, instead of handing a negative offset to the driver (`c8f8a93`)
- `load()` now JSON-coerces json columns, where it used to return a raw string through that path, and its signature changes to `load(filter, params, include)`, from `load(arg, params)` with no `include` (`1c6d0c5`)
- AutoCrud returns `422` with field errors for an invalid create or update, where it used to return `500`. The write body is now allow-listed: `is_deleted` is never client-writable, and a

client-supplied primary key is stripped on both create and update, closing a mass-assignment hole and an IDOR-shaped redirect (`d5cdd95`)

• `seed_table`'s `seed` keyword is dropped; `FakeData#boolean` now returns a native `true`/`false` instead of `0`/`1`; and `seed_orm`'s idempotency skip, which used to silently return `seeded: 0` whenever the table already held enough rows, is opt-in now via `idempotent:` (`3dc8ee8`)

Request model - the big one

`request.params` is route-params-only now, in all four frameworks. Ruby had the worst version of this bug: it used to merge the query string, the parsed body, AND the route params into `params`, so `request.params["id"]` could silently return a client-supplied `?id=` or body value that shadowed the real route parameter. Client input lives only in `request.query` and `request.body` now.

The route-param accessor itself is renamed: `path_params` becomes `params`, with no alias. A malformed JSON body used to parse to `{}`; it now returns the raw string it failed to parse. An empty body used to parse to `{}`; it now returns Ruby's native `nil`. `header()` lookup is case-fold only now (it no longer also converts `_` to `-`) (`776b3c0`).

Migrations

• `rollback` is fail-safe now: a missing `.down.sql` file or a failed down script raises and leaves the `tina4_migration` ledger row in place, instead of deleting the tracking row and leaving the schema applied but untracked (`8f8640b`)

HTTP

• Your responses now gzip-compress when eligible (body over 1024 bytes, `Accept-Encoding: gzip`, a compressible content type); a cacheable 200 gets a strong ETag and a matching `If-None-Match` gets a 304. The static-file ETag format is unified to `W/"<size>-<mtime>"` on all four frameworks, so a cache revalidates once instead of on every deploy (`aa2f2d4`)

• Error pages can emit JSON now, not only HTML: `403`, `404`, and `500` all negotiate `Accept` the way the other three frameworks do, and `404` carries a `request_id` too (`f2b8d84`)

• The OpenAPI spec converges on one shape across languages: an undecorated route emits only `200`, `summary`/`tags` always populate, and `description` is omitted when unset instead of emitting `description:""` (`d7d1514`)

• Your route-group prefix join is normalized now, matching PHP, so a route no longer mis-registers on a bare concatenation or an uncollapsed double slash (`6eb11bd`)

Dev tooling

• `tina4ruby serve` now honours `TINA4_PORT`, where it used to read only the bare `PORT` variable. A Ruby app that already sets `TINA4_PORT` binds its main, AI, and supervisor ports off that base now (`838a0ac`)

• `serverInfo.version` over MCP now reports the real framework version instead of `1.0.0`, a bug Ruby shared with Python (`80a8d6a`)

- The inline `@tests` descriptor builders are renamed: `Tina4::Testing.assert_equal`/`assert_raises`/`assert_true`/`assert_false` become `expect_equal`, `expect_raises`, `expect_true`, `expect_false`. `tina4ruby test` now discovers and runs the inline surface with a real exit code (`e477caf`)

Proven in all four

Two conformance grids close out fully proven, tested against real services in every framework, no mocks:

- **Logging.** A real per-language runner now exercises the shared logger contract end to end. Ruby now rejects a legacy bracket-wrapped log-level spelling that duplicated the variable name inside the brackets; use the plain level name instead, for example `TINA4_LOG_LEVEL=ALL` (`f108b7c`)
- **Database adapters** (ADR-0044). A real runner against SQLite, PostgreSQL, MySQL, MSSQL, and Firebird proved every adapter implements the same interface. Ruby's `DatabaseAdapter::CONTRACT` turned out to be fictional: it named methods (`open`, `fetch`, `autocommit`) that no driver actually implemented, and omitted `execute_many`/`fetch_one` entirely. `get_database_type` existed on none of the seven drivers and is added to all of them now, closing a gap that would have broken every non-SQLite, non-Postgres `Database.new` the moment the contract was enforced (`2f56a8e`)

Bug fixes

- A route path containing a literal parenthesis, like `/products/(sale)`, now matches correctly everywhere. Ruby and Python were already correct; PHP and Node compiled the literal characters as regex syntax (`4c69cff`)

Possible breaking

Read these before you upgrade. Every entry here is a security, parity, or correctness fix, none is a change made for its own sake.

- `request.params` **is route-params-only.** Ruby used to merge the query string, the body, AND the route params into it. Read query values from `request.query` and body values from `request.body` now. This is the single largest behaviour change in the release.
- `path_params` **is renamed** `params`**, with no alias.** Update every call site.
- **Security headers, including CSP, emit by default.** An app depending on inline scripts or a third-party CDN needs `TINA4_CSP` to relax the policy.
- **The CSRF** `403` **body shape changed** from `{error: "CSRF_INVALID"}` to `{error, code, message, status}`.
- `TINA4_CSRF=true` **now actually attaches the CSRF middleware.** If you set it and never noticed CSRF enforcement, it enforces now.
- **A blank** `TINA4_SECRET` **fails closed.** Set a real secret; the framework no longer falls back to a guessable public default.
- **The dev server binds** `127.0.0.1` **by default.** Set `TINA4_HOST=0.0.0.0` to expose it on your network.

• **The static-file search path drops `src/assets`/`assets`.** Move assets under the configured public directory.

• **An unparseable Mongo WHERE now raises instead of matching everything.** Add an explicit WHERE, or call `truncate()`.

• **The MSSQL adapter raises on an unbindable parameter type** instead of silently stringifying it.

• `decimal_field` **emits real** `DECIMAL(p,s)` **DDL.** Existing `REAL` columns are unaffected until re-created.

• **A misspelled** `related_name` **accessor (**`categorys`**) is now correctly** `categories`**.** Update the accessor name.

• `validate()` **now enforces length/type/format on save**, where it used to check only for `null`. A previously-saving value may now fail to save.

• `load()`**'s signature changes** to `load(filter, params, include)`, and it JSON-coerces json columns.

• **AutoCrud returns** `422`**, not** `500`**, on invalid input**, and never accepts `is_deleted` or a client-supplied primary key in the write body.

• `seed_table`**'s** `seed:` **keyword is dropped**, and `seed_orm`'s idempotency skip is opt-in now via `idempotent:`.

• `FakeData#boolean` **returns** `true`/`false`, not `0`/`1`.

• **A malformed JSON body returns the raw string, and an empty body returns** `nil`, where both used to return `{}`.

• **The static-file** `ETag` **format changed**, so every cache revalidates once on upgrade.

• `tina4ruby serve` **now honours** `TINA4_PORT`**.** An app that sets it but expected `PORT` to win must reconcile the two.

• **A legacy bracket-wrapped log-level spelling is rejected.** Use the plain level name, for example `TINA4_LOG_LEVEL=ALL`.

• **The inline testing descriptors are renamed** `assert_*` to `expect_*`, and `tina4ruby test` now actually runs them.

Coming in 3.13.100

Frond's compiler, extensibility, auto-escaping, sandboxing, and caching work (features 48 through 60) is deferred to the 3.13.100 fast-follow, alongside a refreshed Carbonah benchmark harness and a configurable database column-name casing option.

v3.13.98 (2026-08-11) - Skills discipline

A maintenance release. No API changed, and no behaviour changed in your app.

The bundled AI coding skills - the ones that install with `curl -fsSL https://tina4.com/install-skills.sh | sh` - moved onto one shared plan discipline. A plan file has one format (Scope, Tests, Bugs, Commits), and a checkbox is ticked only when the work is verified green at HEAD, ~~never on a claim. That keeps~~ an agent's plan file honest, so it

always reflects what actually shipped.

The framework package also shed internal dead weight: duplicate design-system source files that were never compiled or served at runtime. Nothing your code imports or renders changed.

Nothing to upgrade. If you use the Tina4 AI skills, re-run the installer.

v3.13.97 (2026-08-07) - Behaviour corrections

A small bug-fix release on the road to **3.14 stable**. No new surface, no renames. Two behaviours come back into line with the Python reference, and two correct ones get locked so they stay that way. A destroyed session stays destroyed. A RabbitMQ queue refuses an operation it cannot honour instead of draining itself. Read "Possible breaking" before you upgrade.

Sessions

- `destroy` no longer resurrects. A `set` then `save` after `destroy` re-created the session under the id you just destroyed. `destroy` clears the session id now, so a later `save` writes nothing (`100007f`)

Queues

- `clear` and `purge` on a RabbitMQ backend raise a named refusal. A broker delivers messages to consumers and cannot address them by status, so neither call can be honoured. Both drained the live queue and destroyed its messages before; they raise now, rather than throw away data you meant to keep (`2894bcd`)

Proven

- `force_delete` is locked by a regression test. Ruby was already correct; the test keeps it that way (`f96ad8a`)
- `distinct` dedups values by equality, so two equal dates collapse to one row. A parity test now locks it across all four frameworks (`9459107`)

Possible breaking

- `clear` **and** `purge` **raise on a RabbitMQ queue.** They drained the live queue and destroyed every message before. Catch the refusal, or guard the call. A database-backed queue still clears and purges as before.
- **A destroyed session no longer accepts a write.** A `set` plus `save` after `destroy` used to re-create the session under the old id; it writes nothing now. Save before you destroy, or destroy last.

v3.13.96 (2026-08-07) - One paginate envelope, one message shape

Another step toward **3.14 stable**, and the theme is still parity: the same call means the same thing in Python, PHP, Ruby and Node. This release settles two subsystems that had drifted into four shapes. Pagination returns one envelope of seven keys with a true total. The Messenger IMAP path returns one message shape, addresses mail by a real IMAP UID, and hands back attachments you can write straight to disk. Some of these are behaviour changes, so read "Possible breaking" at the end before you upgrade.

Pagination

- `to_paginate` takes no arguments and reads every field from the query that ran; pass one and it raises (`52b44a8`)
- The envelope is exactly seven snake_case keys: `records`, `total`, `page`, `per_page`, `total_pages`, `limit`, `offset` (`52b44a8`)
- `total` is a true `COUNT(*)` for the filter, never the last page's row count (`52b44a8`)
- The AutoCrud REST list endpoint returns the same seven keys (`d3edeeb`)

```
# Fetch the page you want, then describe it. No arguments.
result = db.fetch("SELECT * FROM orders", [], limit: 20, offset: 40)
response.json(result.to_paginate)
# { records: [...], total: 250, page: 3, per_page: 20,
#   total_pages: 13, limit: 20, offset: 40 }
```

Messenger

- `send` returns `{success, message, id}` on both paths, with `id` from a real `Message-ID` header (`f90d10a`)
- `inbox` items are exactly `{uid, subject, from, to, date, snippet, seen}`. `from` and `to` are header strings now, not arrays of `{name, email}`; the `read` key is renamed `seen`; `date` is ISO-8601; `flags` and `size` are dropped; `snippet` is decoded plain text (`f90d10a`)
- `read` uses `body_text` / `body_html` where it used `body` / `html`, returns `from` / `to` / `cc` as strings, and puts the Message-ID in a `headers` hash (`f90d10a`)
- `inbox` and `read` are callable positionally, `inbox("INBOX", 10, 0)` and `read(uid, "INBOX")`, as well as by keyword (`f90d10a`)
- IMAP credentials are separate from SMTP: `TINA4_MAIL_IMAP_USERNAME` / `TINA4_MAIL_IMAP_PASSWORD` (falling back to the SMTP pair), plus `imap_username:` / `imap_password:` constructor args (`f90d10a`)
- `uid` is a String now, and `inbox` pages newest first. `read` / `mark_read` / `delete` still accept a String or an Integer, and the newest-first order matches the other three (`5568071`)
- `read` folds each attachment's decoded bytes into its attachment item, so you write it straight to disk (`68ac021`)

Swagger

- `info.version` defaults to `1.0.0`, where it defaulted to the framework version so an undocumented app claimed API `v3.13.x`, and `info.description` defaults to an empty string (`2114f0b`)
- An undecorated route emits only `200`; `401` appears only on a secured route (`2114f0b`)
- `operationId` keeps a path's underscores, so `/__health` and `/health` stay distinct (`2114f0b`)
- AutoCrud emits `components.schemas` keyed by the model class name, with a `required` array from column nullability (`2114f0b`)

Migrations

- `code` is the canonical kind for a code migration, the same word in all four frameworks. An unknown kind raises and names the valid ones (`5f27c0f`)

Server

- An oversized request body answers `413`, not `500` (`022c0a7`)
- `TINA4_PORT` is the port variable and wins; bare `PORT` still works but is deprecated and goes in 3.14 (`0601e7c`)

Build and skills

- The framework SCSS compilers are gone; the `tina4` Rust CLI owns SCSS now (`5e0ca63`)
- tina4-css is pinned to one artefact and one URL (`580c475`)
- The skills point at the ADRs and make `tina4 metrics`, Carbonah and mcp.tina4.com the lean, green, grounded workflow (`8f13a26`)

Proven in all four

The Messenger, Swagger and pagination behaviours are locked by machine-checked contract suites that run against real services in every framework: a live GreenMail server for the mailbox, a real 250-row SQLite table for pagination. No mocks.

Possible breaking

Read these before you upgrade. Each affects an app that relied on the old shape:

- `to_paginate` **takes no arguments.** The old `to_paginate(page, per_page)` form raises. Fetch the page, then call `to_paginate`.
- `total` **is the true count.** It was the last page's row count; it is now a `COUNT(*)` for the filter. Use `records.length` for the page size.
- **The paginate envelope is seven keys.** A reader of the dropped alias keys must move to the seven canonical keys.
- **A Messenger** `uid` **is a String** now, not an Integer. Code doing `uid.is_a?(Integer)` breaks; `uid == "3"` and `uid.to_i` keep working.

• `inbox` **pages newest first.** `inbox(limit: n).first` returned the oldest message in that page and now returns the newest, matching the other three.

• `inbox` **and** `read` **changed keys.** `from` / `to` are strings, not arrays; `read` renamed `body` / `html` to `body_text` / `body_html` and moved the Message-ID into `headers`; dates are ISO-8601.

• **Swagger defaults moved.** `info.version` is `1.0.0`, `info.description` is empty. `TINA4_SWAGGER_VERSION` / `TINA4_SWAGGER_DESCRIPTION` still override.

• **An unknown migration kind raises** instead of being ignored.

• **The framework no longer compiles SCSS.** Use the `tina4` Rust CLI.

v3.13.95 (2026-08-06) - Preparing for the 3.14 stable release

One of several releases still to come before **3.14 stable**. The theme is parity: the same call now means the same thing in Python, PHP, Ruby and Node. Method shapes and inputs are unchanged - this is bug fixes, optimizations, and a much harder test suite behind them. See "Possible breaking" at the end for the handful of behaviour corrections that could affect an app relying on the old, wrong result.

Queues

• Ruby's queue store now uses the same layout as Python, PHP and Node. Existing jobs are migrated, not dropped (`758b176`)

• The dev dashboard queue panel lists the store it counts (`758b176`)

• The AMQP vhost is read as the path segment (`7a6ae9a`)

• `Job#fail` reaches the broker, so a failed job is retried or dead-lettered

Logging

• The leaked logger sink is impossible by construction rather than patched per victim (`6d4c505`)

• An explicit argument beats the environment, which beats the default (`e840485`). `TINA4_LOG_DIR` was ignored in a booted app; `tina4.log` now goes to `logs/`

Databases

• Every connect is bounded, and the cases Ruby cannot bound are stated plainly (`bcb64d6`)

• `count()` returns the true total, not the last page's row count (`ed5f74e`)

• `toPaginate` reports the page it is on (`13d6210`)

• `update()` matches the primary key in data case-insensitively (`6ce79d3`)

• Firebird: a column name is folded back only when Firebird folded it (`e153fdf`)

Sessions

• Per-engine DDL: the database backend works on MSSQL (`7826656`)

• A backend failure is loud, then degrades, on the request path too (`b7c2665`)

- No network I/O in a handler constructor (`73110bc`)
- Zero-dependency MongoDB transport, and `TINA4_SESSION_PATH` parity (`2524a8a`)
- `TINA4_SESSION_REDIS_PREFIX` is honoured (`71719d6`)

Document store

- `find()` is deferred, and there is one sort contract (`3af815f`, `97f1711`)
- `find_one` and `to_list` work on both providers (`badf8b1`)

Local by default, production by environment variable

The document store runs on a local SQLite file with no configuration and no services, and becomes a real MongoDB collection when you set one environment variable. The call sites are identical either way. The same shape holds across the framework: SQLite is the default database, the queue is file-backed until you point it at a broker, and the cache is in-memory until you give it a URL.

Bulk insert

Pass a list of rows to `insert()` and the framework builds one prepared statement and runs it inside a single transaction on a single connection, instead of a round trip per row. Same call in all four languages.

```
$db->insert("orders", [
    ["customer_id" => 1, "total" => 9.99],
    ["customer_id" => 2, "total" => 14.50],
    ["customer_id" => 3, "total" => 22.00],
]);
```

Tooling

The `tina4` CLI reached 3.8.67 over this cycle:

- `tina4 init` now wires the project up rather than printing instructions for it
- `tina4 deploy docker` gained `--runtime cli|fpm|swoole` for PHP
- The skills installer worked on macOS but installed nothing on Debian/Ubuntu. Fixed
- Skills install for Codex as well as Claude
- `tina4 metrics` gained cross-file duplication detection, Rust support, and now refuses a file it cannot parse instead of scoring it
- `tina4 serve` no longer opens a duplicate browser tab

The documentation site you are reading runs on **tina4press**, our own zero-Vue static site generator built on tina4-js. 271 pages, no framework runtime on the page.

Possible breaking

Method shapes and inputs did not change. These are behaviour corrections, so they only affect an application that depended on the previous, incorrect result:

- `count()` **in Ruby and Node** returned the number of rows the last page

produced. It now returns the true total. Code that treated it as a page size will see a different number.

- **The AMQP vhost** was read as `"/"` plus the path segment, so

`amqp://host/production` looked for `//production`. If you worked around this by naming your vhost with a leading slash, remove it.

- **Configuration precedence.** An explicit argument now beats the environment.

If you both set `TINA4_LOG_DIR` and passed a directory to `configure()`, the code now wins. Setting only one is unaffected. Ruby deployments that read `tina4.log` from the project root should read `logs/tina4.log`.

- **The Mongo session database default in Node** is now `tina4`, matching the

other three. Set it explicitly if you relied on the old default.

- **An unknown connection scheme, or an unknown** `TINA4_SESSION_BACKEND`**,**

raises instead of silently falling back to SQLite or to file sessions.

- **A missing MongoDB driver** raises rather than falling through.

v3.13.94 (2026-07-29) - A write with no filter is an error

`update()` and `delete()` with no WHERE clause used to be accepted. Two of the four frameworks then wrote every row in the table. The other two changed nothing and said they had succeeded. Neither answer is a write, and neither told you which one you got.

An unfiltered write now refuses to run and names what is missing. `truncate()` is the explicit spelling for the case where you really did mean every row.

Breaking. Read the migration note below before upgrading.

The write path (BREAKING)

A write needs a target. `update(table, data)` with no filter now takes the primary key out of `data` and uses it as the WHERE clause. With neither a filter nor the complete primary key, it raises and names the columns it wanted. `delete(table)` with no filter raises too. `truncate(table)` is the whole-table spelling, and it always was.

A failed write raises rather than returning a falsy result the caller never inspected.

Primary keys are read as a LIST, because a key can span several columns and every one of them belongs in the WHERE. Match a composite key on its first column alone and you hit every row that shares that value: the same data-loss shape the fix exists to prevent. Two of the four SQLite drivers had a second bug underneath that one. `PRAGMA table_info` returns `pk` as the 1-BASED POSITION within the key, not a boolean, so a composite key reports `pk=1, pk=2`. Code that tested `pk == 1` saw a key one column wide and truncated the rest.

Migration. Two changes to make, both mechanical:

- An `update()` that relied on the primary key travelling inside `data` still

works, and now works on composite keys. An `update()` with neither a filter nor a full primary key was already broken. Add the filter.

- Replace a deliberate delete-everything with `truncate(table)`.

Nothing else in the write path moved. A filtered write behaves as it did before.

The Frond sandbox revokes capability, it does not skip a step

`sandbox()` filtered the tag and filter lists but let a denied name fall through to the default path, so a template could still reach an escape filter it was never granted. A denied filter cannot confer safety now, and the tag gate collapsed to one check at dispatch instead of several that could disagree.

The shared render corpus grew from 72 cases to 84, covering `|safe` and `|escape`. It is one committed fixture with one answer key, byte-identical across all four frameworks, so a divergence is a failing test rather than a discovery six months later.

Messenger: one name, one signature

The dev branch and the send branch had drifted into two shapes of the same method, and capture keyed off debug mode rather than whether a transport was available. One signature behind one name now, and capture follows availability.

Base images that boot

No CI had ever built or run a base image. Two of the four had therefore never served a request at all, and the two that did boot were each wrong in their own way. `docker run` is a supported entry point, so all four are now gated in CI on Docker Hub, on `/health`, and on the version they actually serve.

The `FROM` to `COPY` recipe for adding your own database driver is documented in all four image headers, and in one case that documented line did not work until this release. An install instruction nobody runs is a claim, not a feature.

Frond, measured against the engines it replaces

The benchmark page carried seven categories and none of them timed a template. That absence flattered us, because template rendering is the one axis where Frond competes head-on with the engine it replaced. It is a headline category now, and the numbers are not kind.

Every engine renders the same page: a 20-row product list with a loop, an index, an even/odd class, an uppercase filter, two-decimal money, and a conditional footer. Output is compared byte for byte and proven identical before any timing counts, so nothing here is a strawman.

Frond loses in all four languages, on its own compiled path rather than the interpreter fallback. The competition compiles a template to code the host runtime optimises. Frond walks a tree and calls back into engine primitives per hole. What Frond buys is the zero in the dependency column, and one template language that renders the same across four runtimes.

The per-language multiples are in each framework's own note below. Publishing them costs us the comparison and buys the reader a real number.

Also in this release

MQTT test infrastructure locks its password file to `0600` and the broker's own user, so a run cannot leave a world-readable credential behind.

Ruby specifics

Two bugs, one of them older than the other. `sqlite_driver#columns` tested `r[:pk] == 1` and reported only the first column of a composite key, so anything reading `columns()` saw a truncated key. `primary_key()` then used that single column as the whole WHERE clause. The write fix alone would have shipped on top of a driver that could not describe the key it was matching.

A failed connect raises `Tina4::DatabaseConnectionError` with the real reason and a password-redacted target, instead of surfacing as a `NoMethodError` on nil from inside the driver.

Kafka needs the `rdkafka` gem and says so. Ruby does not hand-roll the wire protocol, which is why none of the Kafka format work in this release applies here.

The base image set the legacy environment variables the framework refuses to boot on. `Tina4.run!` now registers the built-in routes, so an `app.rb` no longer serves 404 on `/health`, and `serve` accepts `--managed`, which OptionParser had never declared.

Frond: roughly 55x slower than ERB, which ships in Ruby's own standard library. One caveat the data demands: ERB and Erubi land within noise of each other and near a clock-quantisation boundary, so treat the exact multiple as approximate. The order of magnitude is not in doubt.

v3.13.92 (2026-07-27) - The algorithm you configure is the algorithm that signs

Ruby read `TINA4_JWT_ALGORITHM` nowhere. Set HS512, restart, and every token came out HS256 with a header that agreed with the code rather than with you.

Ruby also stamped an `nbf` claim on every token it issued, which none of the other three did. Three reported issues sit behind this release, one shared contract, and Ruby is the framework whose issuing behaviour changes.

The header names the algorithm that signed

`hmac_encode` and `hmac_decode` both hardcoded SHA-256. `HMAC_ALGORITHMS` now maps HS256, HS384 and HS512 to their `OpenSSL::Digest` classes and both ends take the digest from the map, so the `alg` in the header is always the one that produced the signature. The family ships with Ruby, so no gem was added to get it.

The assertion that catches this measures the signature's byte length: 32, 48, 64. Comparing one algorithm's token against another's proves nothing, because the header is part of the signing input, so two tokens differ either way.

RS256 is not in the Ruby set, and it is not coming. PHP has OpenSSL bindings for it and Node has `node:crypto`, so RSA signing costs those two nothing and stays available there. Ruby and Python would need a third-party package, and the core takes no dependencies. RS256 is a

documented PHP and Node extra, permanently.

TINA4*JWT*ALGORITHM is read for real

`resolve_algorithm` reads it now: an explicit argument wins, then the environment, then HS256. A blank value counts as unset. An algorithm Tina4 cannot sign raises and names the supported set, because the silent downgrade was the whole defect: a weaker token than the one you asked for, and nothing said a word.

get_token no longer stamps nbf (Breaking, Ruby only)

Every token Ruby issued carried `"nbf" => now`. It duplicated `iat`, bought no security at all, and manufactured clock-skew rejections between hosts whose clocks disagreed by a second. RFC 7519 keeps `nbf` for a token you deliberately post-date, and that still works: pass your own and validation enforces it.

Python, PHP and Node never stamped it. Ruby was the outlier, so Ruby moved.

Migration: if you relied on a Ruby-issued token carrying an `nbf` claim, pass `"nbf" => <epoch>` yourself in the payload. Nothing else changes, and every token already in circulation keeps validating.

nbf validation gains 60 seconds of leeway

Ruby was the one framework that already honoured `nbf`, and it honoured it to the second. A token minted on a host one second ahead was refused for nothing at all.

`JWT_LEEWAY_SECONDS` is 60 now, so a post-dated token is accepted up to a minute early. RFC 7519 allows a small leeway, which is exactly what this is. A token carrying no `nbf` is unconstrained, as before.

`refresh_token` used to strip `nbf` along with `iat` and `exp`. With the automatic claim gone, that would have erased a caller's deliberate post-dating on every refresh. It now drops only the two timing claims it owns and leaves your `nbf` where you put it.

The header cannot choose the verifier

Ruby already refused a token whose header did not say HS256. That check now names the configured algorithm instead, so a forged `none`, HS384, HS512 or RS256 header is refused before any signature work, whichever algorithm you signed with.

This closed no live hole in any of the four frameworks. Each already verified with its own configured algorithm and never read the header to pick one, so a forged header changed the signing input and broke the signature anyway. What generalising the check buys is one behaviour across four languages, an exit before the wasted work, and a test that fails the day someone refactors `hmac_decode` into trusting the header. That mistake has its own CVE list. Now it has a test in all four.

Overrides on authenticate_request are honoured

`authenticate_request(headers, algorithm: ...)` accepted the keyword with a hardcoded `"HS256"` default and then dropped it, so an explicit argument lost to the environment. It is a real override now: explicit, ~~then environment, then HS256.~~

A misconfigured algorithm raises (Breaking)

An algorithm Tina4 cannot sign raises an `ArgumentError` naming the variable and the values it accepts. Resolution happens outside `hmac_decode`'s rescue on purpose, so a configuration error surfaces instead of decaying into a `nil` invalid-token result and a silent 401 on every request.

A malformed or forged token still returns `nil`, exactly as before. Only the configuration error escapes.

Migration: set `TINA4_JWT_ALGORITHM` to `HS256`, `HS384` or `HS512`, or leave it unset for the HS256 default. A value that used to be ignored now stops the first token operation and tells you why.

The live Mongo queue specs stop dropping each other's database

Fixed, tests only, no runtime change. Every example in the live MongoDB queue group ended by dropping the database it had used, and the name was one fixed constant shared by the whole group. So example B, or a second `rspec` process on the same `mongod`, could drop the database example A was still working in.

That produced a real failure that never reproduced in isolation: "expected 1, got 0" on the retry example, because the seeded document had been dropped out from under the update. A failure that vanishes when you run the file alone is a cross-process failure, not a logic bug.

Each example now builds its own database name from the prefix, the process id and a random suffix. The pid separates concurrent processes, the suffix separates examples and repeat runs inside one process, and the existing drop can now only destroy data the example created. The specs still hit a real MongoDB. Isolation comes from namespacing the data, never from faking the service.

The tests that hold it

Thirty examples for the algorithm and `nbf` contract, real `Auth`, real OpenSSL HMAC, real process environment, no doubles anywhere. Each ran against the old code first and was watched to fail: re-hardcoding SHA-256 fails the digest examples, restoring the automatic `nbf` fails eight, and removing the leeway fails the in-leeway example. Full suite on macOS with Ruby 4.0.0, against a live SQLite and a live MongoDB: 4,271 examples, zero failures, 74 pending.

v3.13.91 (2026-07-27) - The offenders list finally points at code worth fixing

`tina4 metrics` ranked `public/js/frond.js` as the worst code in the framework, at cyclomatic complexity 191. Second place went to `register_dev_tools` at 139. Neither number was real. The first belongs to a file wrapped in one anonymous function. The second belongs to a registrar that declares twenty small handlers inside itself.

Both scores were the same arithmetic mistake, and it had been in every framework since the metrics module shipped.

A function is no longer charged for the functions inside it

Complexity was measured across a function's whole span. A branch inside a nested function landed on that function and on every function wrapping it. Count the same branch twice, three times, four, once per level of nesting.

The deeper the nesting, the worse the lie:

```
def outer(a):
    def inner1(x):
        if x: return 1
        if x > 2: return 2
        return 3
    def inner2(y):
        if y: return 1
        if y > 2: return 2
        return 3
    return inner1(a) + inner2(a)
```

`outer` branches on nothing. It scored 5. It now scores 1, and each inner keeps its own 3. The branches moved to where they belong. None went missing.

Python and the Rust engine walk a real syntax tree, so they stop descending at a nested function or class. PHP, Ruby and Node scan text, where that surgery would be three risky rewrites. They apply an identity instead:

```
own(F) = raw(F) - sum over direct children C of (raw(C) - 1)
```

A raw score is 1 plus every decision in the span, so `raw - 1` is the total decision count of a whole subtree. Subtract that for each direct child and what remains is the function's own work. It needs only the line and LOC each extractor already reports.

Closures, blocks, lambdas and arrow functions stay exactly where they are. None of them appear in the function list, so nothing subtracts them, and their decisions remain with the function that contains them.

Breaking: every complexity number drops, and so does every file total. A `tina4 metrics --fail-on` gate that failed on an inflated score may now pass. Run the gate once before you trust the old threshold, and lower it if the inflation was holding your ceiling up.

Ruby complexity was double what it should have been

In the tree-sitter Ruby grammar, `if`, `unless`, `while`, `when` and `rescue` name two things: the construct, and the keyword token inside it. The engine matched on the name alone, so it counted both.

`return 1 if y` scored 3. It has one branch.

Every Ruby number the engine produced came out at roughly double. The fix ignores anonymous nodes, which is what a bare keyword token is. The engine and `metrics.rb` now agree on the same file, method for method.

LOC means one thing again

Every framework counted file LOC as code lines, skipping blanks and comments. Every framework counted function LOC as a raw line span. One word, two units, sitting side by side in the same payload.

The dashboard sized its bubbles in one unit and printed its function table in the other. A function documented with care looked bigger than a function with no comments at all.

Each language now has one definition of a code line, shared by both levels, so they cannot drift apart again. `Swagger.generate` reports 167 lines in Python and 167 in the Rust engine, which is the first time two implementations have been asked the same question and given the same answer.

Fixing it in PHP surfaced a third bug. A bare `}` is a token with no line number, so the scan for a function's last line landed on the whitespace before it. Every PHP function was one line short. That line is back.

Nothing but the dashboard reads function LOC. Violations and the maintainability index both use file LOC, which was right all along, so no threshold moves.

The dev dashboard reads the coupling it is given

The bubble chart now uses the numbers the engine resolves. Files that everything depends on drift to the centre. A ring around each bubble runs blue for stable and amber for unstable. Hovering prints the real figures behind them.

The chart draws none of this unless the coupling actually resolved. The older metrics modules never mapped an import to a file, so they reported zero dependents for everything and an instability of exactly 1.0. Drawing that would paint every ring at maximum instability, which is a confident lie, so the chart leaves those channels dark instead.

Thirty-three regression tests hold all of it in place across the four frameworks, and seven more in the engine. Each was run against the old code first and watched to fail. A test that has never failed has never proved anything.

v3.13.90 (2026-07-27) - A scaffolded model and its migration finally agree

`tina4 generate crud Todo` wrote a model that declared a `name` column and a migration that never created it. The first write died:

```
table todo has no column named name
```

This sits on the path https://tina4.com/llms.txt hands to AI assistants, so the documented way to stand up a REST API returned a 500 on its first POST.

One default, defined once (php#186, ruby#33, nodejs#38)

The default field set lived inside the model template. Other generators each carried their own copy, and the migration builder had none, so it built the table from an empty field list: `id` and `created_at`, nothing else. The default now lives in a single constant that flows into the model, the migration, the form, the view and the co-emitted test alike.

Ruby hid a second trap. An empty array is truthy in Ruby, so `fields_override || parse_fields(...)` short-circuited to the empty array and the fallback never fired. That check now tests for content, not truthiness.

Python shipped this fix in 3.13.89 and missed one generator: the form kept its own inline copy of the default. The output matches while the default happens to be `name`, and diverges the moment it changes, at which point the form renders an input for a column the table does not have. Fixed, with a source invariant that fails if any generator restates the literal again.

A failed write no longer reports success

The generated create and update routes serialised their result without checking it. `create()` and `save()` report failure by return value; they do not raise. In Node a failed insert therefore returned HTTP 201 carrying unsaved data. In Ruby it surfaced as a NoMethodError on `false` and buried the real cause. Both now check the result and return 400 with a clear message. PHP already checked, and a test pins that behaviour in place.

Every generator test in all four frameworks passed `--fields` explicitly, which is why one bug survived in four languages. Each framework now exercises the no-fields path and asserts that every field the model declares reaches the migration, ending with a real write against a real database.

v3.13.89 (2026-07-27) - A typo'd tag no longer renders what it was hiding

Two Frond bugs, both present identically in all four frameworks since v3, both compatibility gaps against Twig and Jinja2. One of them was quietly showing people content that was supposed to be gated.

An unknown tag raises instead of leaking its body (Breaking, Security)

Look at this template and decide what a visitor sees:

```
{% iff user.is_admin %}
    <a href="/admin">Admin console</a>
{% endiff %}
```

Everyone. `iff` is a typo, and Frond did not recognise it, so the tag rendered nothing and its body rendered as ordinary content. The admin link went to every visitor. Worse than a broken page: the template reads as guarded, so a reviewer scrolling past sees a check that is not there.

The same shape hid behind any misspelling. `{% ifff %}`, `{% unles %}`, `{% i %}`, a tag copied from another engine. Each one silently removed its own condition.

An unrecognised tag now raises, naming it and listing what Frond does know:

```
Frond: unknown tag "iff" -- known tags are: autoescape, block, cache, extends,
for, from, if, import, include, live, macro, raw, set, spaceless
```

Frond has no plugin system for tags, so an unrecognised name is always a typo, never an extension. Twig and Jinja2 both raise on one. Frond now does too.

Two things deliberately still render nothing. A stray terminator, an `{% endif %}` with no `{% if %}`, and an empty `{% %}`. Neither has a body, so neither can expose anything, and both have always been silent.

What to do: run your templates once. A typo'd tag now fails on the page that contains it, which is where you want to find it.

set works as a block (Fixed)

```
{% set badge %}
    <span class="badge">{{ order.status|upper }}</span>
{% endset %}

<td>{{ badge }}</td>
<td class="mobile-only">{{ badge }}</td>
```

That is core syntax in both Twig and Jinja2, and until now it did the wrong thing in all four frameworks: the body printed inline where the block stood, and the variable was never bound. `{% set g %}Hello{% endset %}[{{ g }}]` rendered `Hello[]` instead of `[Hello]`.

It captures now. The captured value is marked safe, so markup you wrote in the body renders as markup rather than as escaped angle brackets, matching both reference engines. A value interpolated into the body is still escaped on the way in, which is the right place for it: the escaping happens once, where the untrusted value enters.

Blocks nest, and a loop inside the body renders into the capture rather than to the page:

```
{% set rows %}{% for i in items %}<li>{{ i }}</li>{% endfor %}{% endset %}
<ul>{{ rows }}</ul>
```

One rule decides which form you get: an `=` anywhere in the tag means assignment. So `{% set label = "a = b" %}` stays an assignment and is never read as a block. The inline form is untouched.

The expression corpus grew to 82

Three entries cover the block form: a plain capture, a capture that must not be double-escaped, and an inline `set` whose value contains an `=`. The unknown-tag rule is locked by named tests in each framework rather than the corpus, because the corpus compares rendered output and this one raises.

v3.13.88 (2026-07-27) - json_encode gives you JSON again

3.13.87 HTML-escaped the `json_encode` filter in all four frameworks. That was wrong, and this release reverts it.

Entity-encoded JSON is not JSON. `{"a":1}` inside a `<script>` block is a SyntaxError, and a script block is most of what the filter is for. One change broke all four

languages at once.

The same pass fixed a bug that had been there far longer: a value JSON cannot represent used to escape as itself, or as nothing at all.

json_encode emits JSON, not entities (Breaking, reverts 3.13.87)

`{{ product | json_encode }}` renders `{"id":1,"name":"Widget"}` again. Put it straight into a script block:

```
<script>
    const product = {{ product | json_encode }};
</script>
```

The output is still safe on the page. Frond escapes the four characters that can break out of HTML, as JSON unicode escapes rather than entities. A `</script>` inside a string arrives as `\u003c/script\u003e` and cannot close the block early. A single quote becomes `\u0027`, so the value also drops into a single-quoted attribute untouched:

```
<div data-product='{{ product | json_encode }}'>
```

Use single quotes there. JSON carries its own double quotes, so a double-quoted attribute needs `| e` on top. This is the model Jinja2 uses for `tojson`, and it is why the result is marked safe.

`| raw` after `json_encode` is now a no-op. If you added one for 3.13.87 you can delete it, and nothing breaks if you leave it.

A value JSON cannot represent becomes null (Fixed)

Reported as tina4-php#184 by justin-k-bruce, who hit it in production: two infinite sort keys in a 657-row grid blanked the whole screen.

JSON has no Infinity and no NaN. All four frameworks handled that differently and all four were wrong. Python wrote a bare `Infinity`, which no parser reads. PHP's `json_encode` returned false, which arrived as an empty string, so the payload vanished with no error anywhere. Ruby fell back to Ruby inspect output. Only Node.js already did the right thing.

Every framework now writes `null`, which is what `JSON.stringify` has always produced and the only answer the grammar allows:

```
{{ readings | json_encode }}
```

```
{"low":1.5,"high":null}
```

The rule behind it is wider than one value type. This filter never returns an empty string and never returns a token that would fail to parse. A payload always arrives, in the worst case as `null`. An empty script assignment is at least a loud error; a silently wrong one is not.

Two smaller differences went with it. Forward slashes stay unescaped, so `a/b` no longer comes back as `a\/b` from PHP. Non-ASCII text stays raw, so `cafe` with an accent is itself rather than a `\u00e9` escape from Python.

to_json and tojson are the same filter

All three names now share one serializer. They cannot drift apart again.

The `to_json` indent argument is gone. PHP's pretty printer has a fixed four-space indent and cannot honour an arbitrary width, so supporting the argument in three frameworks and not the fourth broke byte parity on the one filter whose entire job is a wire format.

The expression corpus grew to 79

The cross-framework expression fixture added seven entries: the json_encode escape shape, a non-finite value, a NaN nested in an object, the `to_json` alias, a filter pipe feeding a `~` concatenation, that same expression inside a ternary, and `number_format` with locale separators.

The last three close tina4-php#170 and tina4-php#171. Both were reported against 3.13.68 and both were already fixed by the 3.13.87 expression work; they are locked in now so they cannot come back.

What to change in your templates

Grep for `json_encode`. Three cases:

- Inside a `<script>` block: nothing to do. It works again.
- Inside a single-quoted attribute: nothing to do.
- Inside a double-quoted attribute: add `| e`.

v3.13.87 (2026-07-27) - Frond expressions agree in all four frameworks

Frond has always been described as one template language with four implementations. That was an assumption. Nobody had ever measured it.

So we measured it. Seventy-two expressions covering filters, operators, ternaries, null coalescing, concatenation, comparisons, math precedence, missing values, dotted paths, arrays, hashes, escaping and filter chains, rendered through Python, PHP, Ruby and Node.js against one identical dataset. Eleven of the seventy-two disagreed.

All eleven are fixed. Every framework now renders all seventy-two identically.

The corpus is no longer a one-off script. It ships as a test fixture in all four repositories, byte for byte identical, with a single shared answer key. If one framework drifts, its own suite turns red while the other three stay green, and the failure names the expression. Changing the contract on purpose now means changing the answer key in four repositories in the same change, which is the point.

Booleans print as true or false (Breaking)

Ruby disagreed with itself. `{{ n > 3 }}` printed `false`, but `{{ flag }}` with a stored `false` printed nothing at all.

Both now print `false`.

The cause was a falsy guard. Lookup used `value[part] || value[part.to_sym]`, and because only `nil` and `false` are falsy in Ruby, a stored `false` fell through to a missing symbol

key, became `nil`, and rendered empty. The string concatenation path carried the same bug. Both now probe with `key?` instead of truthiness, so "absent" and "false" stay different things.

If a template leaned on the blank output of a bare false variable, wrap it in a conditional.

{% import "file" as alias %} now works (New)

The alias import form rendered as nothing at all. The tag parsed, the macros were never registered, and `{{ forms.button("Save") }}` produced an empty string with no error and no warning. Only `{% from "file" import name %}` worked.

Both forms now work and behave identically. The function-call pattern was widened to accept a dot in the callee name, because a dot is not a word character and `forms.button(...)` was never recognised as a call in the first place.

A second macro bug went with it: a parameter declared with a default value, as in `{% macro greet(name, greeting = "Hello") %}`, was mis-parsed. Defaulted parameters now bind correctly.

The not operator works in output (Fixed)

`{{ not user.active }}` rendered as nothing.

Every logical operator was matched with spaces on both sides, so a leading `not` with nothing to its left matched none of them, fell through to variable lookup, and was resolved as a variable literally named "not user.active". Finding no such variable, it rendered empty.

`{% if not x %}` and `{{ x and not y }}` always worked, so the operator itself was never broken. Only the standalone output expression was lost, and before booleans printed lowercase a dropped expression and a false value looked identical.

A leading `not` now routes to the same evaluator `{% if %}` uses. It also binds looser than the filter pipe, so `{{ not x|upper }}` means `not (x|upper)`.

json_encode escaping

Ruby has always escaped `{{ data | json_encode }}`, and still does. Use `{{ data | json_encode | raw }}` inside a `<script>` block.

The note records that PHP returned raw JSON from this filter and was brought in line.

The gate that keeps this true

Every one of these bugs survived because each implementation looked correct on its own. Reading the code four times would not have found them. Rendering the same expression through all four and diffing the bytes found eleven in an afternoon.

That diff is now a test. `frond_expression_corpus.txt` and `frond_expression_expected.txt` live in every framework's test directory as identical bytes, and every suite renders the corpus and compares against the shared answer key. Alongside it sit named regression tests for each bug above, each carrying the negative case that was failing before the fix.

One more thing worth naming, because it is a pattern rather than an incident. Every boolean bug was a falsy guard: `|| ""`, `a[k] || a[k.to_sym]`, `true ? '1' : ''`. In a template engine, "absent" and "false" are different things, and code that conflates them prints nothing where it should print something.

v3.13.86 (2026-07-25) - Writes return a DatabaseResult

`db.insert`, `db.update`, and `db.delete` now return a `Tina4::DatabaseResult` instead of a plain Hash. The object is truthy and answers `success?`, so an `if db.insert(...)` check still reads the same. What is new is the metadata it carries: `.affected_rows` on every write, and `.last_id` after an insert.

Before, a single insert returned `{ success: true, last_id: ... }`, update and delete returned `{ success: true }`, and only a batch insert returned a DatabaseResult. Now every write returns the same object, matching the Python master and the PHP and Node frameworks.

Breaking: code that read the old Hash keys must change. `result[:last_id]` becomes `result.last_id`, and `result[:success]` becomes `result.success?`. A failed write raises rather than returning a falsy value, so rescue the error instead of testing the return.

The ORM is unaffected: `save` reads `db.get_last_id`.

v3.13.85 (2026-07-24) - The dev-admin bundle ships once

Every install carried the dev-admin dashboard twice. `tina4-dev-admin.js` and `tina4-dev-admin.min.js` sat side by side in the framework's public assets, byte-for-byte identical, and nothing referenced the first one. The route that serves the dashboard, the SPA shell that loads it, and the asset resolver that finds it all name the `.min.js`.

The unminified copy is gone. That is 0.92MB removed from every install, in Python, Ruby and Node.js. PHP had already dropped its copy and is unchanged.

Nothing about the dashboard changes. The file that ships is the same file that was always being served, and the two were identical anyway, so the only difference is what you download.

Why the surviving file is still called `.min.js`

It is not minified, and it never was. The two files had the same SHA-256, so the `.min` suffix described an intention rather than a fact.

Renaming it would break every project that references the asset by path, for no benefit, so the name stays. If the bundle is ever genuinely minified the name will finally be honest; until then it is simply the name of the bundle.

A gate, so the duplicate cannot come back

Nothing compared the two files, which is how 0.92MB per package went unnoticed. All four frameworks now test the shipped assets directly: the `.min.js` is present, the unminified duplicate is absent, exactly one `tina4-dev-admin*.js` exists (so a differently-named copy cannot slip through), and the surviving file is the real bundle rather than a stub.

The Ruby half of this was quietly broken in a way worth naming. Two specs read the unminified file behind a `skip ... unless File.exist?` guard. Deleting the file would have turned both into silent skips: a green suite with two dead assertions, and no signal that the asset had vanished. They now read the shipped bundle with no skip guard, so a missing asset fails loudly. A skip that hides a missing file is not a passing test.

v3.13.84 (2026-07-24) - Every generated Dockerfile actually starts

`tina4 deploy docker` wrote Dockerfiles that could not run. Building each one for real found that of the eight Dockerfile generators in the stack (four templates in the `tina4` CLI, plus one inside each framework's own CLI), exactly one was correct.

The Python image is the clearest case. Its `CMD` was:

```
CMD ["python", "-m", "tina4_python.cli", "serve", "--production"]
```

`tina4_python/cli.py` used to be a module, and `python -m` is valid for a module. The v3 restructure replaced it with the package `tina4_python/cli/`, which has no `__main__.py`, so `-m` cannot execute it. Containers died on startup with "'tina4_python.cli' is a package and cannot be directly executed". The code moved; the hard-coded string in a different repo did not, and nothing tested the generated file.

Every generator now names an entry point that exists, and asks for production mode:

language	CMD
Python	`tina4python serve --production`
PHP	`php vendor/bin/tina4php serve --host 0.0.0.0 --port 7145 --production`
Ruby	`bundle exec tina4ruby serve --production`
Node.js	`npx tina4nodejs serve --production`

All four were verified by scaffolding a project, running `tina4 deploy docker`, building the image, and starting a container.

`serve` no longer kills PID 1 (all four frameworks)

Before starting, the CLI reclaims the port from a stale dev server by reading `lsof -ti` and signalling what it finds. It never validated that output. Where `lsof` exists but prints a different shape, a non-numeric field coerced to 0 or 1, and signalling PID 0 sends the signal to every process in the caller's own process group.

In a container the server is PID 1, so it killed itself. The Node image logged `Killed existing process on port 7148 (PID: 1 ...)` and exited 143. The PHP image logged the same attempt and survived by luck.

Port reclaiming is now skipped entirely inside a container, where there is no stale sibling to reclaim from. Outside one, only all-digit PIDs are accepted, and PID 0, PID 1 and the current process are never signalled. The message reports only what was really killed.

Ruby: the image builds

The builder stage was `ruby:3.3-slim`, which carries no compiler, so `bundle install` failed with `Gem::Ext::BuildError` as soon as it reached a gem with a native extension (`date`, `json`, `nio4r`, `sqlite3` are all in the default set). The builder is now the full `ruby:3.3` image, and the slim runtime installs just `libsqlite3-0`, which the compiled extensions link against.

Bundler's `deployment true` is also gone. It demands a lockfile matching the image's bundler exactly, so a `Gemfile.lock` generated by a newer bundler on a developer machine hard-failed the build.

A gate, so this cannot rot again

Nothing tested a generated artifact. The CLI's deploy tests covered argument parsing and nothing else, and the docs truth-check inspects prose, not template payloads.

The CLI now carries five contract tests over the Dockerfile templates that fail on exactly the mistakes above: no `python -m` on a package, no dev-only runner such as tsx in a production CMD, every CMD names a published entry point, every CMD requests production, and every template sets `TINA4_OVERRIDE_CLIENT`.

The port-reclaim rule is pinned down too, in all four frameworks. Deciding which PIDs to signal now lives in one pure function per language (`selectable_pids`, `tina4SelectablePids`, `selectablePids`), so the safety rule is testable without touching a real process: 43 tests assert that a junk token, PID 0, PID 1, the current process and the current process group are each refused, and that a genuine stale sibling is still reclaimed. One of them feeds in the exact line from the container log that started this.

Also in the CLI (tina4 v3.8.58)

The four bundled Dockerfile templates carry the corrected commands, and the Ruby template gains its build toolchain. Each framework's own `docker` generator was corrected to match, so the two paths agree. The PHP generator now detects whether a project has `bin/tina4php` or `vendor/bin/tina4php` and writes whichever exists, because composer does not link a root package's own bin into `vendor/bin`.

v3.13.83 (2026-07-24) - Swagger UI stops serving itself in production

Security. With swagger disabled, `/swagger/openapi.json` returned 404 and `/swagger` returned 200. The gate was real. The static files walked around it.

The framework ships the Swagger UI as files inside its own public directory. Static serving runs before route matching, and it resolves a directory to its index, so `/swagger` became `swagger/index.html` and never reached the gated route. A production server kept serving the whole UI on four paths:

```
/swagger                        200
/swagger/                       200
/swagger/index.html             200
/swagger/oauth2-redirect.html   200
```

The tell was the mismatch. The spec route obeyed `TINA4_SWAGGER_ENABLED`; the UI ignored it. Static serving now checks the swagger gate before it resolves an index, so all four paths return 404 when swagger is off and serve as before when it is on. Fixed in all four frameworks, each with a lock-in test that fails against the old code. (python#97)

The banner advertises only what answers (python#99)

Every boot printed both dev links, whatever the environment:

```
Swagger:   http://localhost:7147/swagger
Dashboard: http://localhost:7147/__dev
```

In production both URLs 404. That misled two readers at once. An operator scanning a production log believed a dev surface was exposed. A developer clicking the link landed on a dead page.

The banner now prints a row only when that surface answers. Each framework builds those rows through one pure function of the port and two booleans, so the contract is unit tested instead of inferred from stdout: `banner_surface_lines` in Python, `App::bannerSurfaceLines` in PHP, `Tina4.banner_surface_lines` in Ruby, `bannerSurfaceLines` in Node.

`logger` and `base64` are declared dependencies

Ruby 4 removed both from the default gems. `tina4ruby` requires `logger`, and nothing in its dependency tree provided it, so a fresh install on Ruby 4 could fail at require time. The gemspec now declares `logger` (~> 1.6) and `base64` (~> 0.2). `base64` arrives through `jwt` today; declaring it directly means a change there cannot break us.

The CLI runs no watcher in production (tina4 v3.8.57)

`tina4 serve --production` started the file watcher and the SCSS watcher anyway. Both burn CPU, both contend with the single server process, and neither has anything to do: production compiles SCSS once at boot and never re-imports code. `--production` now starts neither. The CLI stays to supervise the server process and shut its tree down cleanly on Ctrl-C.

A stale CA no longer turns a missing certificate into six failures (python#98)

The MQTT TLS tests trusted whatever CA file happened to sit in the shared temp directory. Once that file went stale, the tests did not skip. They failed, six of them, in all four frameworks, pointing at TLS code that was correct. Each suite now verifies that the CA actually validates the broker certificate before it treats the TLS environment as present, and skips when it does not.

v3.13.82 (2026-07-23) - MQTT 3.1.1: talk to any broker, no dependency

Tina4 now speaks MQTT. The new `Tina4::Mqtt` client publishes and subscribes to any MQTT 3.1.1 broker - Mosquitto, EMQX, HiveMQ, AWS IoT - and adds nothing to your dependency tree. It is built on `socket` and a lazily-required stdlib `openssl` alone, and it is the same client in all four frameworks.

```
mqtt = Tina4::Mqtt.new   # reads TINA4_MQTT_URL, default mqtt://127.0.0.1:1883
mqtt.publish("fleet/meter-42/telemetry", '{"kwh":12.5}', qos: 1)

mqtt.consume("fleet/+/telemetry", qos: 1) do |message|
  next if message.duplicate?
  store(message.topic, message.payload)
end
```

- **QoS 0 and QoS 1, retained messages, and a Last Will.** `publish`, `subscribe`, and `consume` mirror the Queue you already know. `consume` acknowledges a message only after your block has processed it, so a block that fails leaves the message for redelivery - at-least-once, which is what QoS 1 is for.
- **QoS 2 is refused, loudly.** Asking for exactly-once raises an error that names the limit and the fix - a QoS 1 consumer keyed on device id and timestamp - rather than silently downgrading to at-least-once and double-processing forever.
- **TLS with a per-client trust store.** An `mqtts://` connection verifies the broker against the CA you supply and no other. A self-signed certificate is rejected unless you provide its CA, and a CA loaded for one client never leaks into the next.
- **Username and password auth**, over plain or TLS connections, from the URL or from explicit arguments.
- **No mocks.** Every test runs against a real Mosquitto broker - the anonymous, authenticated, and TLS listeners, plus an EMQX SUBACK-refusal path - so the wire protocol is verified for real, not simulated.

This release also:

- **Counts 98 built-in features.** Ruby ships all 97 shared features plus its native ERB engine, for 98.
- **Breaking:** `SqlTranslation` **is renamed to** `SQLTranslator` (file `sql_translator.rb`) so the class name matches across all four frameworks. Update your references.
- **Fixes four parity defects** from the feature-recount audit (D5, D6, D11, D12), so Ruby matches the other three frameworks exactly.
- **Fails the test run on a** `before(:all)` **service skip**, closing a hole where a class-wide "service unavailable" skip passed CI green.
- **Stops and deregisters a single background task.** `Tina4::Background.stop_task(task)` ends just that task and removes it from the registry.
- **Ships the compiled Tina4 CSS assets** and honours SCSS `!default` instead of leaking it into the output.
- **Reaches** `doctor`**,** `setup`**, and** `deploy` from `tina4ruby` by delegating to the shared Rust CLI.

v3.13.81 (2026-07-21) - `tina4ruby test` exit code, locked in

Ruby needed no fix here. `tina4ruby test` already exited non-zero when a test failed. This release adds a real lock-in spec so the contract can never drift: it spawns the actual CLI against a failing project and asserts a non-zero exit. Parity with the python#96 fix.

This release re-aligns all four frameworks on one version. Python, Ruby, and Node skip 3.13.80, a PHP-only patch that shipped the `\Tina4\Test` base class; PHP moves from 3.13.80 to 3.13.81. Everyone is back on 3.13.81.

v3.13.79 (2026-07-19) - Session cookies get Secure behind a proxy, honour SameSite, and a renamed cookie is read back

If you run Ruby behind a TLS-terminating proxy, the session cookie shipped without `Secure` and always used `SameSite=Lax`, whatever you configured. This release fixes both and reads a renamed cookie back.

- **Security: the session cookie now goes through the builder.** The `Set-Cookie` was hand-written in `rack_app.rb` and never went through the cookie builder, so `TINA4_SESSION_SECURE` was a silent no-op and `SameSite` was hardcoded to `Lax`, ignoring `TINA4_SESSION_SAMESITE`. The emit path now routes through the builder: `Secure` is proxy-aware through `Request.secure_scheme?` (the `x-forwarded-proto` first hop, else the native scheme), honours `TINA4_SESSION_SECURE`, and `SameSite=None` forces `Secure`. `TINA4_SESSION_SAMESITE` is honoured.
- **Plain HTTP is unchanged.** Without a proxy header and without TLS, the cookie stays non-Secure.
- `TINA4_SESSION_NAME` **is now read back.** The name resolves through one method (`Session.cookie_name`) on both sides, the same as the other three frameworks; the default is byte-identical.
- `TINA4_SESSION_BACKEND` **now selects the handler.** Backend selection was unreachable before, so the setting did nothing. The Redis/Valkey and Mongo session handlers now read their connection from the environment instead of a hardcoded `localhost`.
- **A background no-overlap spec** was added, matching the other three frameworks.

Reported by justin-k-bruce (ruby#31). Real specs, no mocks.

v3.13.77 (2026-07-16) - Background task scheduling confirmed, no code change

Ruby needed no fix this release. A cross-framework check of a reported Python bug (`background()` running a slow task concurrently with itself) confirmed Ruby was already correct: each task owns a thread that sleeps, calls, and sleeps again, so a run can never overlap the next one. Python and Node.js were brought in line with that behaviour.

v3.13.76 (2026-07-16) - Migrations apply again on a database created before 3.13.55

If your database was created by Tina4 v3 3.13.54 or earlier, every new migration failed and none could ever be applied. This release fixes that. Ruby never created that column, so this only reached apps pointed at a database whose tracking table came from tina4-python; the same hardening now applies.

- **The bookkeeping insert now writes the columns your table actually has.** The 3.13.55 rename added `migration_name` and left the old `migration_id` column in place, calling it harmless. It was harmless on reads and anything but harmless on writes: that column is `NOT NULL`, the insert never filled it, so recording a migration raised a not-null violation, the migration rolled back, and the database was stuck. The runner now builds its insert from the table's real columns and fills any legacy one it finds. No schema change, no `ALTER`, every engine.
- **Fresh databases are untouched.** A table with no legacy column still gets exactly the six canonical columns. That is the case CI and every new project exercised, which is why this only ever bit long-lived staging and production databases.

Thanks to justin-k-bruce, who reported it against 3.13.75 with a full compatibility matrix and a working patch. Real-database regression tests now cover a pending migration on a legacy table in all four frameworks.

v3.13.75 (2026-07-14) - Static assets revalidate, so a deploy reaches users without a hard refresh

The built-in static file handler (everything under public/) now lets a browser cache an asset but forces it to revalidate on every use. A redeployed CSS or JS file reaches the browser on the next page load, with no manual hard refresh - and an unchanged file costs a cheap 304 Not Modified, not a full re-download.

- **Cache-Control and validators on every static response.** Each asset carries `Cache-Control: no-cache, must-revalidate`, an `ETag`, and a `Last-Modified`. Before, a static asset carried only its Content-Type.
- **Conditional requests get a 304.** The handler answers `If-None-Match` and `If-Modified-Since` with a `304 Not Modified` and no body, so a revalidation is a small round trip rather than a re-download. Real-file, real-request tests lock the behaviour in.

This lands identically across Python, PHP, Ruby, and Node.js. It closes the class of "I already reported this" where a browser kept serving a fixed-but-cached front-end asset.

v3.13.74 (2026-07-13) - A lock-in test for the connection tester

The dev dashboard "Test connection" panel was already correct on Ruby - it lists the tables through `db.tables` and reads the version through `db.fetch_one`. The same panel was broken in three different ways in Python, PHP, and Node.js, so this release adds the regression test Ruby was missing.

• **A real-SQLite spec drives the endpoint end to end.** It opens a live database with two tables and asserts the handler returns success, the real table count, and the version. No mocks, so the "zero tables" class of bug cannot slip in here either.

v3.13.73 (2026-07-13) - A failed migration re-applies cleanly

This release makes a previously-failed migration run again on the next migrate, at full parity across the four frameworks.

• **A leftover `passed = 0` row no longer wedges the next run.** When a migration succeeds, the runner deletes any existing bookkeeping row for that migration name before it writes the fresh `passed = 1` row. A migration that failed earlier - whether you recorded it with `record_migration(name, batch, passed: 0)` or carried it over from a v2 table - re-applies cleanly instead of colliding on the unique `migration_name`. The `tina4_migration` table holds at most one row per migration, and the latest run wins. The delete-then-insert path is identical on every engine, so there is no dialect-specific behaviour to reason about.

• **The v2 upgrade tells you what happens next.** When the v2 to v3 upgrade finds `passed = 0` rows, it logs that those migrations re-apply on the next migrate, instead of asking you to clear them by hand.

v3.13.72 (2026-07-12) - Frond number_format, filter precedence, and a sandbox fix

This release sharpens the Frond template engine, locks in a database error contract, and brings the dev dashboard to parity across the four frameworks.

• `number_format` **reads all three arguments.** The filter now honours the full Twig signature, `number_format(decimals, decimalPoint, thousandsSep)`:

``twig {{ 1234.5 | number_format(2, ',', '.') }} {# renders 1.234,50 #} ``

The one-argument form is unchanged, so every existing template behaves as before. (php#170)

• **The filter pipe binds tighter than concat.** `|` now groups before `~`:

``twig {{ amount|number_format(2) ~ ' EUR' }} {# (amount|number_format(2)) ~ ' EUR' -> 1,234.50 EUR #} ``

The rule holds at any nesting depth, including both branches of a ternary. On Ruby it also clears a latent case where a pipe inside parentheses rendered empty. (php#171)

• **The sandbox allow-list covers every filter path (Security).** A filter applied inside a `~` concatenation or a ternary condition now respects the `{% sandbox %}` filter allow-list. A filter you did not allow-list no longer runs its code in sandbox mode.

• **A malformed request path was already safe here.** The Node worker gained a guard this release for a path like `//` (or `///`, `/\`); Ruby never crashed on it and returns a normal 404.

• **Database errors still fail loud (python#57).** `execute()` and `fetch()` raise on failure and record the message on `get_error()` rather than returning `false` or an empty result. This shipped in 3.13.38; this release adds a real-PostgreSQL regression test across all four

frameworks so it can never slip back to a silent failure.

• **Dev dashboard parity (`TINA4_DEBUG`).** The dev-admin dependency installer (`deps/install`), the grounding-token proxy, and the Migrate, Test, and Seed run-chips now match across all four frameworks. This is development-only; nothing changes in production.

v3.13.71 (2026-07-11) - AI skills: sharper tina4_code guidance

A skills-and-docs release; no change to the Ruby gem. The bundled Tina4 AI skills now state WHY `tina4_code` is deprecated: in a boot-and-verify gate (scaffold the output, boot it, run it) `tina4_code` failed where a strong model grounded with `tina4_context` passed, so the tools point to grounding plus a strong model over the self-hosted coder. The recommendation is unchanged - ground with `tina4_context` and write the code yourself; only the rationale is sharper. Running `curl -fsSL https://tina4.com/install-skills.sh | sh` now installs these updated skills by default.

v3.13.70 (2026-07-11) - Column defaults on INSERT, a Firebird charset override, and stacked Swagger metadata

ORM honours column defaults on INSERT (#165)

An INSERT now omits any column you never set on the model, so the database applies that column's own DEFAULT. Previously `save` serialised every declared column, sending an explicit NULL for the ones you left alone; a `NOT NULL DEFAULT <x>` column then failed the insert, because a DB default applies only when the column is omitted, not when NULL is passed. The rule is now precise: a column you never touch is omitted and the database fills it in; a column you explicitly set to `nil` is written as NULL; a column with a non-nil ORM default is still written. When every insertable column is unset, the row inserts with the engine's all-defaults form. UPDATE is unchanged. Shipped across all four frameworks, verified with real-SQLite positive and negative tests.

Firebird connection charset override (#160)

The Firebird adapter no longer hardcodes the connection charset to UTF8. You can override it, in precedence order, with a `?charset=` query on the connection URL (`firebird://host:port/path?charset=NONE`), an explicit `charset:` keyword on `connect`, or the `TINA4_DATABASE_CHARSET` environment variable. The default stays UTF8, so nothing changes unless you ask for it. This fixes double-encoded UTF-8 bytes read from a legacy NONE database. Shipped across all four frameworks.

Stacked Swagger metadata all survives (#59)

Stacking summary, description, and tags on one route keeps every value in the generated OpenAPI spec, whatever the order. Ruby was already correct; the fix landed in PHP and Node, and all four now carry a lock-in test so the behaviour cannot drift again.

v3.13.69 (2026-07-10) - The Api client grows up: uploads, downloads, redirects, and a breaking upload signature

The built-in HTTP `Api` client gained four zero-dependency capabilities, shipped across all four frameworks.

- **Multipart upload.** `Api#upload` POSTs a `multipart/form-data` body from a file on disk (`file_path:`) OR from in-memory bytes (`file_bytes:` plus `filename:`), with optional extra text fields, so you never need a temp file. The part Content-Type is guessed from the filename, and the client's default headers (including any Authorization) are sent too. A missing file or no source returns a clean error response, never raises, and sends nothing over the wire.
- **Streaming download.** `Api#download` streams a GET body straight to disk in 64KB chunks, so a multi-megabyte file never lands in memory whole. The returned `ApiResponse` carries `path` (and no body), and writes nothing on an error status.
- **Transport seam.** An injectable `transport:` (any object responding to `#call`) lets application developers unit-test code that calls an `Api` without a live server. Tina4's own suite never injects a fake (the no-mock rule stands); every framework test hits a real local server.
- **Opt-in cookie jar.** Pass `cookies: true` for a per-client in-memory jar: the client parses `Set-Cookie` (leading `name=value`, last write wins) and replays the accumulated `Cookie` header on later requests.

`Net::HTTP` does not auto-follow redirects, so the client now follows them itself in a bounded loop (up to 10 hops) and strips the `Authorization` and `Cookie` headers on a cross-origin hop (a different scheme, host, or port), so neither a bearer token nor a session cookie can leak to a host you did not authenticate to. Same-origin redirects keep them. This also corrects a documentation claim that the redirect auth-strip was Python-only; it now applies in Ruby too.

Breaking

- `Api#upload` `file_path` **is now a keyword argument.** It was a required positional argument. Switch `api.upload(path, "/tmp/x.png")` to `api.upload(path, file_path: "/tmp/x.png")`. This reconciles Ruby to the canonical cross-framework signature.

Also shipping

- **AI coder rule-path skill.** The AI coding-assistant scaffolder writes each tool's rule and context files to the correct path, across all four frameworks.

v3.13.68 (2026-07-10) - Steadier test suite

The documentation-search performance spec no longer fails at random. It timed a single request against a 50ms budget, and one slow sample on a busy machine turned the whole suite red. It now takes the best of several samples, so the check measures real speed instead of scheduler noise. Test-only; nothing in the framework changed.

v3.13.67 (2026-07-10) - The MCP table browser, locked down

The `database_tables` dev-tool now has a real behavioural test. A sibling bug in the PHP framework (#164) fataled the same tool: it called a method that does not exist instead of listing tables, and the test never caught it because it only checked the tool was registered, never that it ran. This framework already listed tables correctly, but carried the same blind spot in its own tests. The new test invokes the real handler against a real SQLite database and asserts a table list comes back, so this class of drift is caught here too. Shipped across all four frameworks.

v3.13.66 (2026-07-10) - A self-describing CLI, and generators that ship their tests

The command line grew up. The `tina4` client no longer keeps its own copy of what each framework can do. It asks the framework, then forwards the request. A command you add to the framework shows up in the client automatically; one you remove simply stops being offered. The whole class of client-versus-framework drift is gone.

The self-describing CLI

- `commands` / `commands --json`**.** Every framework CLI now prints its own command

table from a single source. The `tina4` client reads that manifest to render an accurate `tina4 --help`, caches it by a cheap fingerprint of the resolved CLI, and refreshes with `--refresh`. Dispatch never depends on the manifest, so a discovery miss shortens the help listing and never breaks a command.

- **Pass-through dispatch.** The client keeps its own conductor commands (serve, scss,

setup, init, deploy, agent, doctor, install, update, build) and forwards everything else to the framework verbatim. It carries no per-command flag knowledge, so it can never fall out of parity.

- `queue` **is now a top-level command** in all four frameworks: `queue work`,

`queue stats`, `queue retry`, `queue clear`, wired to the real queue. Run a worker straight from the CLI instead of only scaffolding one.

- `build` **builds the deployable Docker image** (`docker build`), replacing the old

library-packaging behaviour. `build` produces the image; `deploy` ships it.

- **Ruby gains** `migrate:create`, matching the other three.

Generators now ship a test with the code

Every code-producing `generate` subcommand writes a real, passing test next to the code it scaffolds. `generate model Product` also gives you a `Product` test that talks to a real database. The tests use real collaborators (real SQLite, a real test client, a real queue), never mocks, and pass the moment they are generated.

Writing those tests exposed real bugs in the scaffolds that no string-matching test would have caught: the generated `auth` current-user endpoint rejected valid tokens in Python and PHP, and the generated migration created then immediately dropped its table in Ruby and PHP. All four are fixed and locked in by the new tests.

Databases

- **PHP silent PDO fallback.** SQLite and PostgreSQL adapters prefer the native

extension and fall back to the matching PDO driver when it is missing. The developer gets a working database either way, with identical behaviour (native types, raw-byte BLOBs, last insert id, transactions, fail-loud errors).

- **ORM `where()` takes an order.** `Model.where(...)` now accepts `order_by` /

`orderBy` (and Ruby also gains `limit`/`offset`), matching `find()`, `all()`, and the query builder.

Fixes

- The Frond browser helper now applies a 30 second request timeout by default.
- SQLite datetime adapters no longer emit a deprecation warning on Python 3.12+.
- Node route handlers accept a `void` return in TypeScript without a type error.

Breaking

- **Frond `request()` now times out after 30 seconds by default.** A request that

used to hang forever now fails after 30 seconds and calls `onError`. Pass `timeout: 0` to restore the old unbounded behaviour, or a millisecond value to set your own.

- `build` **changed target.** It now builds a Docker image rather than packaging the

framework as a library. Projects that relied on the old behaviour should call their packaging tool directly.

- `generate` **writes an extra test file per scaffold.** If you script generation and

assert on the exact set of created files, expect one more file (the co-emitted test).

v3.13.56 (2026-07-08) - Skills that own up when they drift

Every AI skill now tells the assistant how to report itself when it is wrong. A skill is documentation, and documentation drifts. When a skill still describes a method, default, or column the framework no longer has, an assistant writes confident code against an API that is gone. This release closes that loop.

Every skill, and every project context file the AI installer writes (CLAUDE.md, .cursorules, .github/copilot-instructions.md, .windsurfrules, CONVENTIONS.md, .clinerules, AGENTS.md), now carries one instruction: if Tina4 behaves differently from what the skill describes, that is a bug in the skill. Tell the developer, then report it at https://tina4.com/report-a-skill. The report lands as an issue on the documentation repository, gets fixed at the source, and ships to everyone.

The skills themselves are corrected too. The ORM soft-delete guidance now names the real `is_deleted` column (it wrongly said `deleted_at`), the tina4-js signal-persistence reference ships alongside the skill, and the per-framework skill copies are back in sync with the canonical set.

The web framework runtime does not change in this release. Update your installed skills with `curl -fsSL https://tina4.com/install-skills.sh | sh` (re-run to refresh in place).

v3.13.55 (2026-07-07) - One migration tracking schema on every engine

The `tina4_migration` bookkeeping table now has the same shape on every framework and every engine. Before this release the four frameworks each named and typed the tracking table a little differently. A project that moved between them, or a tool that read the table directly, met a different schema each time.

The canonical table is six columns: an auto-increment `id`, a `migration_name` (unique, the migration file stem), a `description`, a `batch`, an `executed_at` timestamp, and a `passed` flag. The auto-increment and the column types follow the engine: `AUTOINCREMENT` on SQLite, `SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY(1,1)` on SQL Server, and a generator on Firebird.

Existing installs upgrade in place, and no applied migration re-runs. The runner detects the old name column (`migration_id` in Python, `migration` in PHP, `name` in Node; Ruby already used `migration_name`), adds `migration_name`, copies the values across, and backfills the new columns. The old column stays where it is, ignored. A migration already marked applied stays applied.

No new third-party dependencies. Shipped across all four frameworks.

v3.13.54 (2026-07-07) - Migrations honour the SET TERM directive

A Firebird trigger or stored procedure now survives the migration splitter. Those bodies end their inner statements with a semicolon, the same character the runner uses to separate one statement from the next. Run under the default terminator, a trigger body split apart on its own punctuation and the migration failed.

A `SET TERM` line fixes it. Wrap the block in the universal isql idiom and the runner switches its active terminator, so the whole body travels as one statement:

```
SET TERM ^ ;
CREATE OR ALTER TRIGGER t_bi FOR t ACTIVE BEFORE INSERT AS
BEGIN
  IF (NEW.id IS NULL) THEN NEW.id = GEN_ID(GEN_T, 1);
END^
SET TERM ; ^
```

The runner consumes each `SET TERM` line instead of sending it to the engine, restores the previous terminator when the block ends, and handles a multi-character terminator such as `!!`. A migration with no `SET TERM` splits on the semicolon exactly as before. Shipped across all four frameworks.

PHP and Ruby also repair the Firebird v2 to v3 upgrade. Firebird returns column names in upper case, so the migration tracker read a null migration name and treated every applied

migration as pending, re-running the lot. The tracking-table reads now normalise to one key shape. PHP additionally records a row on an upgraded table with its original 14-character id and detects the Firebird dialect through the Database facade.

Thanks to justin-k-bruce for the contribution.

v3.13.53 (2026-07-06) - JSONField: JSON document columns

Store a JSON document in a column. A model field can now hold a whole object or array. The framework encodes it to JSON when it writes and decodes it back to a native Hash when it reads, so the attribute is always live data, never a raw string.

```
class Event < Tina4::ORM
  json_field :payload            # Hash or Array
```

The column type follows the engine. PostgreSQL gets native `JSONB`, MySQL native `JSON`, SQL Server `NVARCHAR(MAX)`, Firebird a text `BLOB`, and SQLite `TEXT` (queryable through JSON1). One field declaration, the right column on every database.

A value that cannot be encoded to JSON does not slip through. `save()` fails loud: it rolls back, returns `false`, and records the cause, so a half-written row never reaches the table. A mutable default is copied per instance, so two records never share and mutate the same object.

No new third-party dependencies.

v3.13.52 (2026-07-04) - Frond live blocks, pgsql:// URL scheme, SCSS colour functions

Frond live blocks. A page can now carry a region that keeps itself current. Wrap the region in `{% live %}` and Frond paints it on the server with the first request, then refreshes it over the transport you name.

```
{% live "prices" poll 5 %}
  <ul>{% for row in rows %}<li>{{ row.name }}: {{ row.price }}</li>{% endfor %}</ul>
{% endlive %}
```

The block renders server-side on first paint, so a crawler and a client with no JavaScript both see real content. After that it refreshes on its own. `poll N` re-fetches every N seconds. `sse` streams updates over Server-Sent Events. `ws "/path"` rides a WebSocket route you already own. A data provider feeds every refresh: `@live_source` in Python, `Frond::liveSource` in PHP, `Frond.live_source` in Ruby, `Frond.liveSource` in Node. The provider re-runs with the live request, so a block that reads the signed-in user reads it again on every refresh, and an authenticated block cannot serve one user another user's data. For poll and SSE, Tina4 mounts one always-on endpoint, `GET /__frond/live/{name}`. For a WebSocket block, `push_live(name, data)` re-renders the block and broadcasts the fresh HTML to every client on that path. Nested live blocks are rejected, and a block's optional `src` attribute is same-origin only.

One client script drives it, `frond.js`, byte-identical across Python, PHP, Ruby, and Node. No build step. No framework on the page.

pgsql:// is a Postgres URL scheme again (#58). A connection string like `pgsql://user:pass@host/db` was rejected. v3 registered only `postgresql://` and `postgres://` and dropped the older spelling, but `pgsql` is the scheme PDO, Laravel, and Doctrine all use, so real config files carried it and Tina4 refused to start. `pgsql://` now resolves to the PostgreSQL driver in all four frameworks, next to the two existing spellings. Same driver, three accepted names.

SCSS colour functions evaluate at compile time. `rgba(#3498db, 0.5)` used to pass through to the stylesheet as literal text, and the browser dropped the whole rule because `rgba()` cannot take a hex string. The built-in SCSS compiler now evaluates the colour functions: `rgba(#hex, a)` and `rgb(#hex)` expand to real channel values, `mix(c1, c2, weight)` blends two colours, and `lighten()` and `darken()` shift a colour through HSL. The output is byte-identical across all four compilers, down to the same integer rounding, so a shared stylesheet renders the same colour whichever framework served it.

No new third-party dependencies.

v3.13.51 (2026-07-03) - MCP Streamable HTTP transport, Firebird fixes

The built-in dev MCP server now speaks the current MCP Streamable HTTP transport, the one Claude Code and today's MCP clients expect. It still answers the older 2024-11-05 HTTP+SSE transport, so nothing that already worked stops working.

One endpoint carries the whole session. A client POSTs JSON-RPC to `/__dev/mcp` and reads the response inline. `initialize` mints a session and returns it in an `Mcp-Session-Id` header; the client sends that header back on every later call. A request with an unknown session gets a `404`, its cue to initialize again. A notification returns `202`. `GET` on the endpoint returns `405` with `Allow: POST, DELETE`, and `DELETE` ends the session. The server negotiates the protocol version: it echoes the version a client asks for when it can speak it, otherwise it picks the newest one it knows.

Tina4 writes the connection details to `.claude/settings.json` for you, now with `"type": "http"` and the bare `/__dev/mcp` URL. Prefer the command line? Run `claude mcp add --transport http tina4-dev http://localhost:7147/__dev/mcp`. The change lands uniformly across Python, PHP, Ruby, and Node. Python and Node keep a full persistent legacy SSE stream; PHP and Ruby serve the current transport plus a one-shot legacy handshake, with no long-lived connection required.

Why the transport slipped past us. Our MCP tests spoke our own JSON-RPC shape over the endpoints we built, never the wire a real client speaks. They stayed green while a real Claude Code client could not connect. Every framework now ships a no-mock transport test that drives the real session lifecycle, and each was verified end to end against a live server booted through the tina4 CLI.

Firebird (PHP). Three reported issues are fixed. The migration runner and the ORM now call `execute()` rather than the old `exec()` (#120). Parameterized DML no longer throws a type error when it fetches the last insert id (#121). NULL parameters bind correctly again, rewritten to a literal `NULL` because the ibase driver cannot bind a PHP null (#123). The `exec()` method stays

as a deprecated alias for `execute()`.

Migration recording on upgraded schemas (PHP). The fail-loud `execute()` surfaced a quiet bug. On a database whose `tina4_migration` table had been upgraded in place from the old v2 layout, a new migration never recorded itself, so it re-ran on every boot. The old `exec()` had swallowed the constraint error. The runner now supplies the legacy `migration_id` column when it is present, so a migration records once and stays recorded.

No new third-party dependencies.

v3.13.50 (2026-07-02) - Path route params match INTEGER primary keys on SQLite (Ruby fix)

A route path parameter like `{id}`, matched against a real HTTP request, must find an INTEGER primary-key row. On Ruby it did not. Rack delivers the request path as ASCII-8BIT, so an untyped `{id}` capture reached the SQL bind as a binary string, and the sqlite3 gem bound it as a BLOB. SQLite gives a BLOB no numeric affinity, so `WHERE id = ?` never matched an INTEGER column - `GET /api/users/{id}` returned 404 for a row that plainly existed (and `GET /api/users` listed it). The router now relabels path captures as UTF-8 so they bind as TEXT, which SQLite coerces to the column's integer affinity, and the row matches. Typed `{id:int}` params were never affected - they cast to an Integer. The SQLite driver is left alone on purpose: coercing every binary string there would corrupt genuine BLOB writes, so the encoding is fixed at the source (the router).

Python, PHP, and Node were confirmed unaffected - their string path params already bind as TEXT - and each gains a real regression test: real router extraction feeding a real SQLite integer-primary-key lookup, no mocks, so the contract cannot silently drift. No new third-party dependencies.

v3.13.47 (2026-06-25) - Open-issue batch: migration comment splitting, global middleware, SCSS interpolation

Three reported issues, fixed and locked in with tests against the real thing.

Migration statement splitter (#54). A migration whose SQL carried a `;` inside a `-- ...` line comment fragmented into broken pieces, because the runner split on the `;` delimiter before it stripped comments. A `CREATE TABLE` with a trailing `-- drop then re-add; old way` comment raised "incomplete input" on SQLite. The splitter is now a single-pass, quote- and comment-aware scanner: it strips `--` line and `/* */` block comments, copies single- and double-quoted string literals verbatim (honouring the `''`/`""` escape), keeps `$$`/`//` stored-procedure blocks intact, and splits on the delimiter only outside all of that. A `;` or `--` inside a comment or a string literal can no longer split or corrupt a statement. Named regression specs plus an end-to-end migrate against a real temp SQLite database lock it in, with no mocks.

Global middleware lock-in (#55). Middleware registered globally with `Tina4::Router.use(...)` already ran on every route in Ruby. A lock-in spec now guards the contract so the regression fixed in the Python master this release cannot creep in here.

SCSS `#{}` interpolation (#116). The SCSS compiler did not support interpolation, so `calc(100% - #{$gap})` left the `#{...}` in the output and corrupted the CSS around it. The compiler now resolves `#{ ... }` before variable substitution and nesting: a `$variable` inside the braces resolves to its value and anything else inlines verbatim, so `calc(100% - #{$gap})` becomes `calc(100% - 20px)` and `.icon-#{$name}` becomes `.icon-home`. Shipped across all four frameworks for parity.

No new third-party dependencies. Full suite: 3,593 passing.

v3.13.46 (2026-06-24) - MSSQL fetch_one fix + real MySQL/MSSQL batch tests

The MySQL and MSSQL batch-insert specs were placeholders that skipped even when the engines were provisioned. They now run for real against both engines and assert the full batch contract, including a bad row rolling the whole batch back. Making the MSSQL one run exposed a real adapter bug: apply*limit emitted an OFFSET/FETCH page with no ORDER BY, which SQL Server rejects, so any fetch*one of an unordered query (a COUNT or MAX aggregate) raised "Incorrect syntax near '0'". It now adds a no-op ORDER BY when the query has none, matching the Python master. No new third-party dependencies.

v3.13.45 (2026-06-24) - Real-service test hardening

The Valkey and Redis session handlers talk through the `redis` client gem. The test bundle now installs it alongside the other optional service drivers, so the live Valkey session round-trip - write a value, read it back, destroy it against a real Valkey - runs on every pass with the infrastructure up instead of skipping. Under `TINA4_REQUIRE_SERVICES` a provisioned service that goes unexercised is a hard failure, so this closes a gap where the session path slipped through untested. The gem stays in the optional group, so a plain install on a machine without it is unchanged. No framework runtime change. No new third-party dependencies.

v3.13.44 (2026-06-24) - Real-service bug-fix sweep (no mocks)

Standing up live infrastructure - PostgreSQL, MongoDB, Redis, Valkey, Memcached, RabbitMQ, Kafka - and running the suites against the real services surfaced a batch of bugs that mock-based and skipped tests had hidden. This release fixes them across the family and makes the no-mock rule absolute: a test that touches a dependency exercises the real service, never a stand-in. **Migrations on PostgreSQL/MySQL/MSSQL:** the migration runner built its bookkeeping table (`tina4_migration`) with SQLite-only DDL (`id INTEGER PRIMARY KEY AUTOINCREMENT`) for every non-Firebird engine, so `migrate` failed on PostgreSQL with a syntax error and never applied a single file. The tracking-table id is now engine-aware - `SERIAL` on PostgreSQL, `AUTO_INCREMENT` on MySQL, `IDENTITY` on MSSQL, `AUTOINCREMENT` on SQLite. The existing migration tests only covered SQLite, which is why this shipped; a gated live-PostgreSQL migration spec now guards the engine-aware path. **Kafka:** dequeue waits for the partition assignment on first subscribe, so a single short poll right after produce no longer returns nothing. **MongoDB queue:** the backend honours an explicit db when a connection URI is given - it was defaulting to the admin database and writing every job there. **ORM:** a UUID primary-key insert returns its id. **Tests:** rdkafka is now an optional development dependency, so the live Kafka path has a real spec. No new third-party runtime dependencies. All four suites pass against live services.

A parity sweep shipped in the same release. The RabbitMQ backend now acknowledges each job: it dequeues with manual ack and acks on a new `complete` step, so an at-least-once consumer neither drops nor redelivers work. The Redis and Valkey session handlers gained a zero-dependency raw RESP fallback, so sessions work without the optional `redis` gem (matching Python and Node), and the MongoDB session handler drops and recreates its TTL index instead of failing when the expiry changes. The live Redis, Valkey, and MongoDB session round-trips run by default whenever the service is reachable, no longer pending on an optional gem. The smoke tests were rewritten to assert real behaviour.

MySQL and MSSQL join the provisioned test services (#262). Both engines now run live round-trip tests by default, gated on reachability the same way the other services are. The non-skippable real-service gate fails on a MySQL or MSSQL skip under `TINA4_REQUIRE_SERVICES`, so a missing engine in CI breaks the build instead of passing quiet. CI gained a MySQL 8 container and a SQL Server 2022 container. Running the suites against these two engines for the first time surfaced adapter bugs that no prior test could reach, and Ruby carried several.

The MySQL driver now connects over TCP. When a port is named it rewrites a `localhost` host to `127.0.0.1`, so the connection no longer falls back to a UNIX socket and reaches the provisioned server. Boolean binding is fixed in the SQLite and MSSQL drivers: a raw boolean now binds as `1`/`0` at the parameter boundary instead of crashing or stringifying. The insert id now survives. On MySQL the driver captures the id at write time, so a later `COMMIT` no longer resets it to zero; on MSSQL the driver runs the `INSERT` and `SELECT SCOPE_IDENTITY` in one batch, so `last_id` returns the new `IDENTITY` value and holds through a later `SELECT`. The MSSQL row-count probe was already correct here and earned a regression test to keep it that way.

The no-mock rule reached further this release (#250). The messenger SMTP and IMAP tests now talk to a real GreenMail mail server, the WebSocket backplane tests run against a real Redis

backplane, and the HTTP-client tests hit a real loopback HTTP server. Every in-test double in those paths is gone. The dev mailbox carried a real crash: it wrote message JSON as raw UTF-8 bytes, so reading a message with an accented name under a US-ASCII locale - a common CI default - raised a decode error and broke the inbox. Ruby now reads and writes those files as UTF-8, and a non-ASCII round-trip regression test guards the path. Python, PHP, and Node were never vulnerable, and each gained the same regression test for parity.

v3.13.43 (2026-06-22) - Queue: MongoDB fail() now dead-letters correctly

Part of a cross-framework queue-lifecycle unification (the active backend owns the whole lifecycle). Ruby already routed the lifecycle to the active backend, and `complete` acknowledged MongoDB correctly. But the MongoDB `fail` requeue path reset the job to pending without persisting the incremented attempt count - so every poll saw a stale count, the `attempts >= max_retries` check never tripped, and a repeatedly-failing job was requeued forever instead of dead-lettering. The requeue now persists the incremented attempts, so `fail` dead-letters at `max_retries`. `consume(topic)` already honoured its argument. Verified end-to-end against a live MongoDB: complete yields no redelivery; topic isolation holds; fail dead-letters at the limit. A lock-in spec asserts the requeue carries the incremented attempt count. Full suite: 3,593 examples, zero failures.

v3.13.42 (2026-06-22) - Swagger configurability for external and public APIs

Closes four gaps that pushed teams to hand-roll their own OpenAPI spec instead of using the built-in generator. **Configurable security schemes:** the built-in `bearerAuth` scheme honours `TINA4_SWAGGER_BEARER_FORMAT` (default `JWT`; set `opaque` for `sk_live_`-style keys), and setting `TINA4_SWAGGER_API_KEY_NAME` emits an `apiKeyAuth` scheme (`TINA4_SWAGGER_API_KEY_IN` is `header`/`query`/`cookie`). Register any scheme - including an `oauth2` flow with scopes - programmatically with `Tina4::Swagger.add_security_scheme(name, definition)` (`Tina4::Swagger.reset_registry` clears it). **Per-route security:** a route declares its own requirement through `swagger_meta:` - `security:` as a scheme name plus a `scopes:` array, a `{name => [scopes]}` map, a list of maps for an OR requirement, or `"public"` to mark a write route open (emits `security: []`). A secured route with no explicit declaration falls back to `TINA4_SWAGGER_DEFAULT_SCHEME` (default `bearerAuth`). Scopes stay spec-valid: only `oauth2`/`openIdConnect` schemes carry them, every other type gets `[]`, so the output validates against 3.0 and 3.1. **Path filtering:** `TINA4_SWAGGER_INCLUDE` documents only routes whose path starts with one of its comma-separated prefixes; `TINA4_SWAGGER_EXCLUDE` drops matching prefixes; framework internals (`/swagger`, `/__dev`) are always excluded. **OpenAPI 3.1 opt-in:** `TINA4_SWAGGER_OPENAPI` (default `3.0.3`) emits `3.1.0` when set to `3.1`/`3.1.0`. **Reusable component schemas:** register a shared shape with `Tina4::Swagger.add_schema(name, schema)` and reference it from a route's `request_schema:` / `response_schemas:` metadata, extending the ORM-model `$ref` mechanism to arbitrary schemas. Identical behaviour and tests across all four frameworks. Zero new third-party dependencies. Full suite: 3,576 examples, zero failures.

v3.13.41 (2026-06-22) - Queue reservation/visibility timeout (at-least-once delivery)

A targeted fix for silent job loss in multi-replica / rolling-deploy setups. When a consumer reserved a queue message and then died before acknowledging - a crash, an OOM kill, a Kubernetes pod eviction - the message was stranded forever: never re-delivered, never retried, never dead-lettered. The file backend deleted the job on pop (lost outright); the MongoDB backend flipped the document to `processing` without advancing `available_at` and never re-evaluated it. Now a popped job is held as a reservation with `available_at = now + visibility_timeout` (plus a `reserved_at` stamp). If the consumer does not acknowledge in time, the next dequeue reclaims the abandoned reservation: it increments `attempts` and re-enqueues the job, or dead-letters it once it has hit `max_retries`. A dead consumer can no longer strand a job - standard at-least-once delivery, the contract SQS and RabbitMQ already provide. The window is configurable via `TINA4_QUEUE_VISIBILITY_TIMEOUT` (default 300 seconds; `<= 0` disables the reclaim) or the per-queue `visibility_timeout:` option; `complete`/`fail`/`retry` clear the reservation. RabbitMQ and Kafka are unchanged - the broker already owns redelivery there. Regression tests lock the behaviour in across all four frameworks (file backend: reclaim after the timeout, no reclaim before it, dead-letter past max_retries, complete/fail clear the reservation, env override, disable-at-zero; MongoDB: dequeue advances `available_at` and the reclaim requeues or dead-letters). Zero new third-party dependencies. Full suite: 3,560 examples, zero failures.

v3.13.40 (2026-06-22) - MCP security hardening + Swagger/OpenAPI overhaul

A coordinated cross-framework release with two themes: MCP transport security and a full Swagger/OpenAPI sweep. **MCP security:** the built-in dev MCP server now authorises every request on the raw socket peer rather than a configured host name, closing a remote-reach surface where a debug box bound to `0.0.0.0` exposed the file and database tools to unauthenticated callers. The gate is two layers - a host-independent capability check (`TINA4_MCP`, else `TINA4_DEBUG`) and a per-request authorisation: loopback always passes, while a remote caller needs `TINA4_MCP_REMOTE=true` AND a token matching `TINA4_MCP_TOKEN` (fallback `TINA4_API_KEY`), sent as `Authorization: Bearer`, `X-MCP-Token`, or `X-Api-Key`. Every MCP surface returns 404 to a disallowed caller, the SQL tool is read-only (SELECT/WITH, no stacked statements), and the file tools are sandboxed to the project root. **Swagger:** the production on/off switch is wired for real - set `TINA4_SWAGGER_ENABLED=false` to disable `/swagger` and `/swagger/openapi.json` in any environment, or `true` to expose them in production (it falls back to `TINA4_DEBUG` when unset). Secured routes now carry a `bearerAuth` security requirement, so the documentation no longer presents protected endpoints as public. ORM models become reusable `components.schemas` referenced by `$ref` across all four frameworks. The spec is valid where it was not: wildcard and splat routes emit proper `{name}` path parameters, `operationId` values are de-duplicated, and WebSocket routes no longer leak an invalid method. **Swagger configuration:** `TINA4_SWAGGER_SERVERS` (comma-separated) drives a multi-server block, `TINA4_SWAGGER_UI_CDN` points the UI assets at a self-hosted mirror for air-gapped use, and the generator adds typed-parameter formats, enums, top-level tags, and multipart request bodies. **SqliteDocStore (new):** a pymongo-style document store with a zero-config SQLite fallback. `get_collection(name)` returns a real Mongo collection when a Mongo URI is set (`TINA4_MONGO_URI`, then `TINA4_SESSION_MONGO_URI`, then the legacy `TINA4_SESSION_MONGO_URL`), otherwise a SQLite-backed collection over a local file (`TINA4_DOC_STORE_PATH`, default `data/tina4_docstore.db`) - the call sites are identical, only the backend differs. It pushes filters down to JSON1 `json_extract` (equality, `$in`, `$nin`, `$gt`/`$gte`/`$lt`/`$lte`, `$ne`, `$exists`, `$regex`, `$or`, `$and`, dotted nested keys), supports `$set`/`$unset`/`$inc`/replace/upsert with lazy `sort`/`limit`/`skip`/projection cursors, ships a zero-dependency 12-byte ObjectId, and round-trips times and ObjectIds so values stay queryable. Develop against the local store and switch to MongoDB in production by setting one env var. **Developer experience:** the blank-`TINA4_SECRET` warning now explains why it fired - the run was not detected as development - and gives both fixes (set `TINA4_SECRET`, or set `TINA4_DEBUG=true` to auto-generate one into `.env.local`); the legacy-env strict check now hints the names may come from a `.env` baked into a Docker image and points at `tina4 env --migrate`. **Session Mongo env parity:** the session and DocStore Mongo URI is canonical as `TINA4_SESSION_MONGO_URI` across all four frameworks, with `TINA4_SESSION_MONGO_URL` kept as a back-compat legacy alias (Python and PHP historically read `_URL`). **Tests:** new DB contract tests (execute raises on a bad statement, read-after-write, generator monotonicity, transaction bracketing) and a queue isolation contract (a job in one topic never leaks into another, and a queue on a fresh storage path starts empty). **Breaking:** the Swagger and MCP production gates are now enforced - if you relied on `/swagger` being reachable in production, set `TINA4_SWAGGER_ENABLED=true`, and a remote MCP caller now needs

`TINA4_MCP_REMOTE=true` and a token. Zero new third-party dependencies. Full suite: 3,545 examples, zero failures.

v3.13.39 (2026-06-21) - Auto-migration, unified critical log level, fail-loud ORM, per-route WebSocket auth

A cross-framework parity sweep that hardens the data layer and tightens a few safe-by-default behaviours. **Migrations:** pending migrations can now run on startup, gated by `TINA4_AUTO_MIGRATE` and off by default so existing apps are untouched. A footgun and clear-bug pass adds per-file transactions, row-existence tracking, numeric-aware ordering (so `10_` sorts after `2_`, not after `1_`), `CREATE TABLE` idempotency, a URL-safe `//` delimiter, and smart/curly-quote normalization in migration SQL before it runs. **ORM:** `save()`, `create()`, the query builder, and the Mongo path now fail loud instead of swallowing errors - `save()` returns `self` on success and `false` with the reason on `get_error` when a write fails, and the silent fallbacks are gone. **Logging:** `critical` is now a first-class top-level severity, a new `Tina4::Log.enabled?(level)` lets callers skip building an expensive log payload, and logs default to stdout-only in production and container environments to avoid file bloat. **WebSockets:** a route can require authentication on the upgrade itself, before the socket is established. **Security:** the `/__dev/mcp` endpoint enforces its localhost guard and honors `TINA4_MCP_REMOTE`, and the built-in `Api` client strips the `Authorization` header on a cross-origin redirect, fixes a dead `verify_ssl` setting, and gains opt-in retry with backoff. **Env:** defaults align to the canonical `TINA4_` manifest with a uniform `.env.example`, plus an autocommit durability fix and the ORM-plural default turned off. **Tooling:** the `tina4 metrics` coverage detection now counts full-package-path requires and short (3-character) constants, and the complexity counter no longer over-counts string literals. Breaking: `critical` is now a top-level severity and the previous strict flag was renamed - update any logging configuration that relied on it. Full suite: 3,462 examples, zero failures.

v3.13.38 (2026-06-19) - Coordinated security & robustness release

A large bundled release closing a cross-framework hardening sweep. **WebSockets:** the Redis/NATS backplane is now wired for real - local-first delivery, then a published envelope on the shared `tina4:ws` channel, relayed with an origin guard (no own-echo, no cluster loop) - and rack WS upgrades are now owned by a shared manager, so a broadcast reaches **every** connection (previously each socket spun an isolated engine). Plus an origin allow-list (`TINA4_WS_ALLOWED_ORIGINS`), an idle reaper (`TINA4_WS_IDLE_TIMEOUT`), and SSE hardening (mid-stream error / client disconnect). **Sessions:** the API-key fast-path is now timing-safe (`validate_api_key`), the guessable default session secret was removed, and a log-loud-and-degrade backend-failure policy applies (`TINA4_SESSION_STRICT` to re-raise). **GraphQL/WSDL:** a SOAP `<!DOCTYPE>` is rejected before parsing (SOAP 1.1 forbids DTDs - closes the REXML entity-expansion / external-entity surface), a recursion-depth guard (`TINA4_GRAPHQL_MAX_DEPTH`, default 50) catches deep queries **and** circular fragments, resolver/SOAP faults are masked in production (full detail only under `TINA4_DEBUG`), and GraphQL directives (`@skip`/`@include`/`@auth`/`@role`) are now actually parsed and honored - the parser never populated them before. **Tooling:** a new `tina4 metrics` command reports the top-N code-health offenders with `--top/--json/--fail-on/--path`, and the coverage test-detection is now precise (a real `require`/defined-constant reference, not a name-substring scan). Minimal dependencies. Full suite: 3,323 passing.

v3.13.37 (2026-06-18) - Dev-admin editor: syntax highlighting now works

The dev-admin file-read endpoint returned `{path, content, bytes}` with **no** `language` **field**, so the dashboard editor couldn't pick a CodeMirror grammar - nothing highlighted. It now returns a `language` (canonical extension map matching the Python master, plus no-extension `Dockerfile`), and the rebuilt editor bundle adds the Ruby/Rust/Go/Java/SCSS grammars - so `.rb`, `.ts`, and friends highlight correctly. Dev-mode tooling only. Full suite: 3,154 passing.

v3.13.36 (2026-06-18) - Instant WebSocket dev-reload + dev-admin file browser fix

Dev-reload is now a WebSocket push, matching Python. `tina4 serve` POSTs `/__dev/api/reload`; the server re-loads changed route files in-process (`rescan_routes!`, mtime-tracked, no respawn) and broadcasts `{type, file, mtime}` over `/__dev_reload` (held open by a process-wide manager; the upgrade needs a hijack-capable server such as Puma). The injected client is WebSocket-primary and only polls `/__dev/api/mtime` when the socket is down. **Also fixed:** the dev-admin file browser returned `type` instead of `is_dir`, so folders never rendered in the dashboard tree - `/__dev/api/files` now returns `is_dir`, `has_children`, real per-entry `git_status` and the repo `branch`, full parity with Python/PHP. Full suite: 3,149 passing.

v3.13.35 (2026-06-17) - Live MCP endpoint for AI agents

The built-in MCP server is now actually reachable, and its tools actually work. Two bugs: the dev tools were never registered on the default server (so it had zero tools), and `route_list` called `route[:method]` (subscript) on `Tina4::Route` objects that only expose attr-readers (every call raised). Both fixed; `DevAdmin.handle_request` now mounts `/__dev/mcp` (JSON-RPC) + `/__dev/mcp/sse` in debug mode, giving an AI agent (Claude Desktop/Code) live access (DB queries, file I/O, routes, docs) scoped to the running project. 17 new specs; full suite 3,136 examples.

v3.13.34 (2026-06-17) - Cross-framework parity release (no functional change in Ruby)

Version alignment with Python/PHP/Node. Ruby's example app, env handling, and AI dual-port dev mode (main port hot-reloads; port+1000 stable) were already correct - verified live this release (main injects the reload script; port+1000 shows the AI-port badge with no reload). No code change. Full suite: 3,119 passing.

v3.13.33 (2026-06-17) - Queues: priority pop + automatic dead-lettering (Warning: behavioural change)

Behavioural change. `job.fail(reason)` now re-enqueues (incrementing `attempts`) until `attempts >= max_retries`, then dead-letters - a `consume` loop retries `max_retries` times automatically. `pop`/`consume` are now priority-ordered (was FIFO); new additive `retry_backoff:`. Bug fixes: `consume(id:)` no longer raises and `pop_by_id` now works (implements `find_by_id`). Only the file backend changed. Queue chapter rewritten to match. Full suite: 3,119 passing.

v3.13.32 (2026-06-17) - Caching: per-query bypass + X-Cache headers (chapter rewritten to match code)

Added a per-query bypass - `db.fetch(..., no_cache: true)` (also `fetch_one`/`fetch_all`) skips lookup + store. `Tina4::ResponseCache` now sets `X-Cache: HIT|MISS` + `X-Cache-TTL` (this fixed a bug where the MISS header silently no-op'd on a real Request). The caching chapter was rewritten to match code (real `cache_stats` shapes, all seven backends + file fallback, the three cache layers, accurate env/defaults), dropping earlier aspirational claims. Full suite: 3,099 passing.

v3.13.31 (2026-06-17) - Request/response parity with Python/PHP/Node (Warning: breaking: request.body)

Breaking. `request.body` now returns the **parsed** body (JSON/form -> Hash) like the other three frameworks; the raw string moves to `request.body_raw`. Apps that read the raw body - webhook signature/HMAC verification, `JSON.parse(request.body)`, SOAP/XML - must switch to `request.body_raw`. Reading fields via `request.body["key"]` now works directly. (Framework internals graphql/wsdl were repointed.)

Additive: uploaded files gain a raw-bytes `content` field with indifferent string/symbol keys (`file["content"]` / `file[:content]`), parity with the others - materialised **lazily** on first access so `:tempfile`-only handlers never buffer large uploads in memory. `response.stream` now accepts a positional generator (`stream(gen)`) in addition to the block form. Full suite: 3,090 passing.

v3.13.30 (2026-06-16) - Cross-framework parity release (no functional change in Ruby)

Version alignment with Python/PHP/Node, which gained typed-route-param coercion this release. Ruby already coerced `{id:int}` -> `Integer` and `{price:float}` -> `Float` (it was the reference for the cross-framework fix), so there is no behavioural change here. Verified green against the routing and auth suites. Full suite: 3,079 passing.

v3.13.29 (2026-06-16) - Live API search ranks qualified queries + resolves natural names

Parity with the Python master fix for the `api_*` live-reflection tools. (Ruby's `Frond.add_filter`/`add_global`/`add_test` are plain methods, already indexed - the metaprogramming gap that hit Python/PHP doesn't apply here.)

- **Class-qualified ranking.** `api_search("Frond.add_test")` now ranks `Tina4::Frond#add_test` first - the owning class, fqn segments, and an exact `Class.method` match are scored, instead of the qualifier being dead weight.
- **Natural-name lookups.** `api_class`/`api_method` resolve a bare class name (`Database`) and longer nested paths via a new resolver, not just the exact fqn.

The bundled AI skills now tell assistants to query `api_*` before guessing. Full suite: 3,079 passing.

v3.13.27 (2026-06-16) - Frond template-engine parity fixes

A 50-case cross-engine audit (every Frond tag, filter, and test rendered through all four frameworks with identical templates) surfaced five places where Ruby's output diverged from the Twig/Jinja standard. All are now fixed to match:

- **Expression-parser literal bug** - `{{ 'a' ~ 'b' }}` and `{{ 'Y' if x else 'N' }}` came out mangled (the literal detector stripped the outer quotes off the *whole* expression before concatenation / inline-if ran). The detector now only treats an expression as a single string literal when the opening quote's match is the final character, so concatenation and inline-if of two quoted literals work.
- `{{ x | e }}` **/** `nl2br` return a `SafeString`, so the auto-escaper no longer double-escapes them; `nl2br` escapes its input and emits `<br />`.
- `url_encode` percent-encodes spaces as `%20` (was form-encoding them as `+`).

Behavioural note: these change rendered output for the affected cases - correctness fixes toward the documented Twig/Jinja behaviour. `TINA4_AUTOCOMMIT=false` strict mode remains

unsupported in Ruby (unchanged). Full suite: 3,075 examples.

v3.13.26 (2026-06-16) - pooling parity confirmed; standalone writes auto-commit

Ruby already had the correct behaviour, so this release is a parity + test pass rather than a code change. Standalone writes auto-commit on their own connection (the `pg`, `sqlite3`, and `mysql2` drivers run in autocommit mode by default), pooled connections are independent (`PG.connect` opens a fresh backend per pool slot), and explicit transactions (`start_transaction`/`commit`/`rollback`) stay atomic. A new regression test asserts a standalone write is visible across pooled connections, bringing Ruby in line with the pooling fix shipped to Python, PHP, and Node.

The cross-framework default is now **autocommit on for standalone writes** (durable + pool-visible); explicit transactions stay atomic. Note: `TINA4_AUTOCOMMIT=false` strict manual-commit mode is **not** supported in Ruby - its drivers don't expose an autocommit toggle - but the autocommit-on default matches the other frameworks.

Verified live on PostgreSQL: standalone write visible from a separate connection, explicit rollback discards, explicit commit persists, pooled standalone writes visible across every round-robin connection. Full suite: 3,071 passing.

v3.13.24 (2026-06-15) - unified cache backends across response, KV, and persistent DB cache

The response/KV cache now supports **seven backends**, selected by `TINA4_CACHE_BACKEND`: `memory` (default), `file`, `redis`, `valkey`, `memcached`, `mongodb`, and `database`. `TINA4_CACHE_URL` carries the connection string for `redis`/`valkey`/`memcached`/`mongodb`, or a SQL URL for the `database` backend (which falls back to `TINA4_DATABASE_URL`). Credentials can be embedded in the URL (`redis://user:pass@host`, `redis://:pass@host`, `mongodb://user:pass@host`) or supplied via `TINA4_CACHE_USERNAME` / `TINA4_CACHE_PASSWORD` (mirroring `TINA4_DATABASE_USERNAME`/`_PASSWORD`); memcached is unauthenticated. The usual `TINA4_CACHE_TTL` (60), `TINA4_CACHE_MAX_ENTRIES` (1000), and `TINA4_CACHE_DIR` (`data/cache`) still apply.

Graceful fallback: if a configured backend's driver is missing or the service/credentials are unreachable or wrong, the cache logs a warning and falls back to the **file** backend - a real persistent cache, never a silent no-op.

The **persistent DB query cache** (`TINA4_DB_CACHE=true`) now routes through the same backend set via `TINA4_DB_CACHE_BACKEND` + `TINA4_DB_CACHE_URL`, so multiple instances share one cache with global write-invalidation. `cache_stats` now reports a `backend` field alongside `mode`.

Full suite: 3,070 examples passing.

v3.13.23 (2026-06-15) - request-scoped DB query cache, on by default

A new **request-scoped query cache** protects your database from rapid repeat reads. Within a single request, identical `SELECT`s and ORM reads are deduped automatically - the DB is hit once and subsequent identical reads are served from memory. The cache is **cleared at the start of every request** (so it never serves stale rows across requests) and **flushed on any write** (insert/update/delete/execute). For non-request contexts (scripts, workers) a short safety TTL applies.

It is **on by default** via `TINA4_AUTO_CACHING=true` (off-switch `TINA4_AUTO_CACHING=false`); the in-request TTL is `TINA4_AUTO_CACHING_TTL` (default 5 seconds). The existing `TINA4_DB_CACHE` (default `false`) remains the separate *persistent* cross-request cache (TTL `TINA4_DB_CACHE_TTL`, default 30s) and is not cleared per request. `cache_stats` now reports a `mode` field: `"request"` (default), `"persistent"`, or `"off"`.

Also fixed: the response-cache default TTL changed `0` -> `60` seconds, matching Python, PHP, and Node.

Full suite: 3,049 examples passing.

v3.13.22 (2026-06-15) - session default TTL standardised to 1 hour

The default session lifetime now matches across all four frameworks: **3600 seconds (1 hour)**. Ruby previously defaulted to 86400s (24 hours). The session cookie `Max-Age` and the file-handler gc window now use 3600 by default - override via `Session.new(env, max_age: ...)`. PHP and Node already used 3600 and are unchanged.

v3.13.21 (2026-06-15) - docs: `render()` corrections + version re-sync

Documentation consistency pass - no behavior change. The `response.template(...)` reference in `llms.txt` is corrected to `response.render(...)` - the real method; `template` is only the route-level binding, not a response method. Version re-synced to 3.13.21 with the other frameworks (this release also carries a Python-side JWT-secret security hardening).

Full suite: 3,040 examples passing.

v3.13.19 (2026-06-15) - return domain objects, construct from JSON, and one database binder

Three ergonomic improvements surfaced by the live side-by-side review of the book's own examples across all four frameworks.

`response` serializes domain objects

Return an ORM model, an array of models, or a query result straight from a route - Tina4 serializes it to JSON. No more hand-rolled `to_h` / `to_json`:

```
Tina4::Router.get("/api/users") do |request, response|
  response.json(User.all)        # array of models -> JSON array
end
```

A single model becomes a JSON object; an array of models or a `DatabaseResult` becomes a JSON array. Plain Hashes, Arrays and Strings behave exactly as before - purely additive.

Construct a model from a JSON object string

```
Widget.new('{"name": "Alice"}')      # JSON object string -> one record
Widget.new(name: "Alice")            # still works
Widget.new("name" => "Alice")        # still works
```

Passing an **Array** to a single-record constructor now raises a clear `ArgumentError`. To build many records, map over the list.

Warning: Breaking - one database binder: `bind_database`

The ORM-to-database binder is now `Tina4.bind_database` (the `Tina4.database = db` writer is gone; `Tina4.database` remains as a reader). The default is unchanged - models still auto-bind to `TINA4_DATABASE_URL`, so apps relying on the `.env` default need **no change**.

```
# Most apps: nothing to do - the .env default is auto-bound.

Tina4.bind_database(Tina4::Database.new("sqlite:///app.db"))   # override the default

# Register a NAMED connection and point a model at it:
Tina4.bind_database(
  Tina4::Database.new("postgres://.../analytics", username: "u", password: "p"),
  name: :analytics
)

class Visit < Tina4::ORM
  self.db = :analytics      # uses the analytics connection (symbol = named connection)
end
```

`Tina4.bind_database(db, name: :...)` registers a named connection; a model selects it with `self.db = :...`. A missing named connection raises a clear error.

Migration: replace `Tina4.database = db` -> `Tina4.bind_database(db)`. Reading `Tina4.database` is unchanged.

Full suite: 3,040 examples passing. Shipped with parity across all four frameworks.

v3.13.18 (2026-06-15) - ORM `Model.query` + foreign-key wiring fixes

Found by the live side-by-side validation against PostgreSQL.

- `Model.query` **raised** `NoMethodError` - it called `QueryBuilder.from`, but the factory is `from_table`. Fixed: `MyModel.query.where(...).get` now works.

- `foreign_key_field` **with a string / forward reference never wired the has_many side** - the deferred registry (`apply_fk_registry!`) was never invoked (no `inherited` hook). An `inherited` hook on `Tina4::ORM` now applies it as model classes load, so `references: "Author"` (string) wires both `belongs_to` and `has_many` regardless of definition order. (A bare constant used before its class is defined is still plain-Ruby ordering - use the string form to defer.)
- Doc: the has_many accessor is the declaring class name (lowercased) + `"s"` - the cross-framework convention - overridable with `related_name:` (the CLAUDE.md "tableName" wording was wrong).

Full suite: 3,018 examples, 0 failures.

v3.13.17 (2026-06-15) - PostgreSQL reads return native Ruby types

Found by the live side-by-side validation against PostgreSQL. The `pg` gem returns every column as a String by default - `id` as `"1"`, a boolean as `"t"`, timestamps as strings - so a Tina4 app written on SQLite (native types) silently changed behaviour on PostgreSQL, diverging from Python and Node. The PostgreSQL driver now installs `PG::BasicTypeMapForResults` on the connection, so reads decode by type: integer -> `Integer`, boolean -> `true`/`false`, float -> `Float`, numeric -> `BigDecimal`, timestamp -> `Time`, date -> `Date`. `uuid`/`json`/`jsonb` stay strings; `bytea` stays binary.

So `db.fetch(...)[0]` is now `{id: 1, active: true, created: <Time>}` instead of all-strings - matching SQLite, Python, and Node.

Full suite: 3,011 examples, 0 failures.

v3.13.16 (2026-06-15) - `create_table` works on PostgreSQL + `DatabaseResult` index access

Found by the live documentation-verification pass - running the book's own samples against a real PostgreSQL database. The documented code-first schema path, `create_table`, was silently broken on PostgreSQL: it emitted SQLite-only DDL (`AUTOINCREMENT`/`DATETIME`), PG rejected it, the error was swallowed, and it returned `true` while creating **no table**.

Root cause: `get_database_type` didn't exist

`create_table` called `db.get_database_type` to pick engine-appropriate types - but that method was never defined on `Database`, so the engine was always blank and every column fell back to SQLite DDL. (This also means the v3.13.11 engine-aware `BooleanField` work had **never actually fired on Ruby** until now.) `get_database_type` is now implemented, and `create_table` is engine-aware:

- **datetime ->** `TIMESTAMP` on PostgreSQL/Firebird; `DATETIME` on SQLite/MySQL/MSSQL.
- **boolean -> native** `BOOLEAN` (PostgreSQL/MySQL), `BIT` (MSSQL), `INTEGER` (SQLite/Firebird); boolean `DEFAULT`s engine-aware (`TRUE`/`FALSE` vs `1`/`0`).
- Auto-increment translated per ~~engine (`SERIAL` on PostgreSQL~~) via `SQLTranslator`.

- **A failed `CREATE` now returns `false`** (and logs) instead of reporting success.

`DatabaseResult` index + slice access

`result[0]` already worked; widened `[]` to ranges/slices (`result[1, 2]`, `result[1..3]`) and added `to_ary` for destructuring - full parity with the documented behaviour.

Verified against PostgreSQL 16: a model with `id` (auto-increment) + string + boolean + datetime creates, inserts, and round-trips (`SERIAL`, `boolean DEFAULT true`, `timestamp`; `WHERE active = TRUE` matches). New `postgres_create_table_spec` (PG-gated). Full suite: 3,010 examples, 0 failures. Shipped with parity across all four frameworks.

v3.13.14 (2026-06-13) - Logs reach stdout in containers + per-request logging + schema-qualified tables (#48)

Cross-framework release (all four). Deployed Docker containers were getting no application logs. Ruby actually *did* write to stdout by default (`TINA4_LOG_OUTPUT=both`), but it **never set** `$stdout.sync` - and a container's stdout is a non-TTY pipe, which Ruby block-buffers. Logs sat in the buffer until it filled or the process exited, so `docker logs` looked empty (and a crash lost the tail). A follow-on report - the dev server going silent after startup - surfaced a second gap: requests were never logged to stdout.

Per-request logging - on by default in dev

Every request now logs one line through `Tina4::Log` (-> stdout), on by default in dev and opt-in for production via `TINA4_LOG_REQUESTS`:

```
2026-06-12T10:15:03.221Z [INFO   ] GET /api/users -> 200 (12.3ms)
```

`rack_app` emits it after every request (the dev inspector previously only fed the `/__dev` UI). Format is identical across all four frameworks: `METHOD /path -> STATUS (Nms)`. Default: on under `TINA4_DEBUG`, off in production; `TINA4_LOG_REQUESTS=true`/`false` overrides. `RequestLoggerMiddleware` dropped its `[RequestLogger]` prefix for parity.

What changed (stdout)

- `$stdout.sync = true` is set in `Log.configure` (unless output is file-only). Logs now flush to the container's stdout immediately.
- **Default log level is `INFO`** (was `[TINA4_LOG_ALL]`). Surfaces request/startup/warn/error without debug noise.
- `TINA4_LOG_LEVEL` **now accepts plain names** (`ERROR`, `info`) in addition to the legacy bracket form (`[TINA4_LOG_ERROR]`) - so the env value is portable with Python/PHP/Node. Unknown values fall back to INFO.

```
# In a container, default config:
Tina4::Log.info("worker started")
# pre-v3.13.14: buffered on the non-TTY pipe -> docker logs lagged / lost on crash
# v3.13.14:    flushed immediately to stdout
```

Why it spanned all four

The same logging-in-containers gap showed up in every framework:

Framework	Pre-v3.13.14 cause	Fix
Python	`not _is_production` gate suppressed stdout; default ERROR	stdout always on (flushed); default INFO
PHP	`$stdout = $development` (file-only in prod); no `TINA4_LOG_LEVEL` read	stdout default on + `fflush`; reads `TINA4_LOG_LEVEL`; default INFO
Ruby	stdout written but never flushed (block-buffered on non-TTY); default ALL	`$stdout.sync = true`; default INFO; accepts plain + bracket names
Node	`!isProduction()` gate suppressed console; default DEBUG	console always on; production emits JSON; default INFO

The Rust `tina4` CLI was already correct (inherits child stdio).

Schema-qualified tables (#48) + a PostgreSQL `fetch()` regression

Issue #48 - *"Database Table Does Not Exist"* on PostgreSQL. A model whose table lives in a non-default schema (`gift_cards.gift_card`, MSSQL `dbo.widget`, MySQL `otherdb.table`, SQLite ATTACH `extra.widget`) was invisible to the framework's introspection. `table_exists?`, `tables`, and `columns` hardcoded the default namespace (`public`) and matched the whole dotted string as one flat name - so plain reads worked, but `create_table`, migrations, and auto-CRUD were blind to the table and reported it missing.

Each driver now resolves a qualified name through a shared `SchemaSplit` helper, and `Database#table_exists?` delegates to the driver when it can answer:

- **PostgreSQL** - `table_exists?` uses `to_regclass()` (honours schema + `search_path`); `columns` filters by `table_schema`; `tables` lists every non-system schema and returns non-`public` tables schema-qualified.
- **MySQL** - schema = database; a qualified name checks that catalog, a bare name defaults to `DATABASE()`.
- **MSSQL** - honours `dbo.table`; a bare name matches in any schema.
- **SQLite** - honours an ATTACH alias (`extra.widget`) for both `table_exists?` and `columns`.
- **Firebird** - N/A (no schemas).

Verified against a live PostgreSQL 16 container: `table_exists?('gift_cards.gift_card') -> true`, `tables -> ['gift_cards.gift_card', 'gift_cards.transaction']`, `columns -> 12 columns` - identical results across all four frameworks.

> ***PHP also fixed a v3.13.12 regression found while cross-checking #48.*** *Its* `PostgresAdapter` *referenced* ~~`stripTrailingSemicolons()`~~ *(added in v3.13.12)*

> *and the new `splitSchema()` but never mixed in `SqlNormalizerTrait` - so **every PostgreSQL `fetch` / `fetchOne` / `getColumns` fatalled**. It shipped silently because the PostgreSQL test suite skips without a live server. Fixed and pinned by server-free reflection guards.*

Tests

- Ruby: 2,999 passed (+23 new - level resolution + `$stdout.sync`; request-log gate + dispatch; #48 schema split + SQLite ATTACH introspection)
- Family: Python 2,829 | PHP 2,394 | Ruby 2,999 | Node 3,628 - **11,850 total, zero regressions.** (PHP also fixed #119, a `cli-server` boot crash, and the PG `fetch` regression above.)

---

v3.13.12 (2026-06-11) - SQL safety + implicit ORM binding + `fetch_all` correctness

Three high-impact fixes that close out long-standing footguns. All three ship with full parity across all four frameworks - Ruby gets the auto-discover wiring as the headline change.

`fetch_all` actually fetches ALL rows now (no silent 100-row truncation)

Pre-v3.13.12 the convenience method defaulted to `limit: 100` and silently truncated. The name says `fetch_all` - it should fetch them all:

```
# 150 rows in the table
db.fetch_all("SELECT * FROM rows")
# pre-v3.13.12: returns 100 rows, silently drops the other 50
# v3.13.12:    returns all 150 rows
```

The new default is `limit: nil`, which the driver's `apply_limit` already treats as "no LIMIT injection" - your SQL runs verbatim. To opt back into a cap, pass `limit:` explicitly:

```
db.fetch_all("SELECT * FROM events", limit: 500)   # capped
db.fetch_all("SELECT * FROM users")                # all rows
```

`db.fetch` (the paginated sibling that returns a `DatabaseResult` with count metadata) keeps its 100-row default - pagination is its job. Only the `fetch_all` convenience changed.

Breaking change: callers who relied on the silent 100-row cap now get every row. For very large tables, switch to `fetch` (which paginates with metadata) or pass an explicit `limit:`.

Trailing `;` is now stripped from user SQL in `fetch` / `fetch_one`

The framework appends `LIMIT n OFFSET m` to the user-supplied query (and wraps it in `SELECT COUNT(*) FROM (...) AS subq` for the count probe). When the user's query already ended with a `;`, both rewrites broke:

```
db.fetch("SELECT * FROM users;")
# pre-v3.13.12: syntax error near "LIMIT" - the appended LIMIT followed a ;
# v3.13.12:    works - trailing ; is stripped before LIMIT is appended
```

The strip is conservative: only trailing whitespace + semicolons are removed (any number of them, including `;;`), nothing inside the statement is touched. Parameters and quoting are unchanged - the existing parameter-binding defense against injection still does all the heavy lifting.

Lives as `Tina4::Database.strip_trailing_semicolons(sql)` and is called from `fetch` and `fetch_one`.

Ruby ORM now auto-discovers `TINA4_DATABASE_URL` (the binding fix)

This was the Ruby outlier. When `TINA4_DATABASE_URL` was set in `.env` but `Tina4.bind!` had never been called, the model's `db` accessor returned `nil` - every `save` / `find` / `where` silently no-op'd. Python, PHP, and Node already discovered the env var on first use; Ruby had the helper (`auto_discover_db`) defined but never called.

```
# .env has TINA4_DATABASE_URL=sqlite://./app.db, no explicit Tina4.bind! anywhere
User.find(1)
# pre-v3.13.12: nil  (db accessor returned nil, query never ran)
# v3.13.12:     #<User id: 1, ...>  (auto-discovered on first model access)
```

The `db` accessor on `Tina4::ORM` now resolves in this order:

```
def db
  @db || Tina4.database || auto_discover_db
end
```

Explicit `Tina4.bind!(db)` still takes precedence - use it to bind a second database or override the env-driven default. The behaviour now matches Python's `database_url_auto_discover()`, PHP's adapter auto-init, and Node's `initDatabase()` env fallback.

Cross-framework parity

Fix	Python	PHP	Ruby	Node
`fetch_all`/`fetchAll` returns ALL rows by default	[x] `limit=0` default	[x] `$limit = 0` default	[x] `limit: nil` default	[x] already correct (`limit?` undefined)
Strip trailing `;` from fetch SQL	[x] shared helper on `DatabaseAdapter`	[x] `SqlNormalizerTrait` on 5 adapters	[x] `Tina4::Database.strip_trailing_semicolons`	[x] exported `stripTrailingSemicolons`
Implicit ORM binding from env	[x] already worked	[x] already worked	[x] **fixed** (wired `auto_discover_db`)	[x] already worked

Tests

- Python: 2,811 passed (+24 new)
- PHP: 2,316 passed (+13 new)

- Ruby: 2,980 passed (+18 new)
- Node: 3,612 passed across 95 files (+16 new)

11,719 tests across the family, +71 new for v3.13.12, zero regressions.

---

v3.13.11 (2026-06-11) - ORM correctness pass

Mirrors Python's ORM correctness pass. Two Ruby-side changes plus regression-pinning tests.

#50.1 - Callable Proc defaults are now resolved per-instance

```
class GiftCard < Tina4::ORM
  integer_field :id, primary_key: true, auto_increment: true
  datetime_field :created_at, default: -> { Time.now }
end
```

Pre-v3.13.11 the Proc was stored verbatim; on save it reached the driver as the Proc object. Now `initialize` invokes the Proc per instance, so each row gets a fresh value. Classes are excluded - `default: Integer` survives verbatim.

BooleanField - engine-aware DDL on PG / MySQL / MSSQL

`Tina4::ORM.create_table` now picks each engine's native bool type. SQLite and Firebird stay on INTEGER (SQLite has no native bool; Firebird's driver round-trip is uneven). PostgreSQL gets `BOOLEAN`, MySQL gets `BOOLEAN` (alias for `TINYINT(1)`), MSSQL gets `BIT`.

#50.2 - natural-key INSERT (already correct, now pinned)

Ruby's `save()` already routes through the `@persisted` flag - set to `false` by `initialize`, `true` by `from_hash` and after a successful save - so natural-key INSERT was working correctly all along. Pinned with a regression spec so a future refactor can't silently break it.

PG error-visibility fixes (Python only)

The `tina4.request.error` event hook, the explicit-txn log gap, the COUNT-probe swallow, and the BooleanField PG cascade are all psycopg2-specific. Ruby's `pg` gem uses libpq in autocommit mode - the cascade never happens. No Ruby changes needed.

Tests

2,962 examples passing, 7 pending (+10 new - `spec/orm_v3_13_11_spec.rb`). No regressions.

---

v3.13.9 (2026-06-10)

Non-destructive AI installer - `Tina4::AI.install_selected` / `install_all` no longer clobber the user's `CLAUDE.md`. They write (or refresh) a marker-bracketed Tina4 skill block and leave the rest of the file alone.

The bug

Pre-v3.13.9 the installer wrote a full developer guide to `CLAUDE.md` (and to `.cursorules` / `.github/copilot-instructions.md` / `.windsurfrules` / `CONVENTIONS.md` / `.clinerules` / `AGENTS.md` / `.antigravity/context.md`) on every run, clobbering whatever the user had put there. Comment in the old code: *"Always overwrite -- user chose to install"* - well, sort of, but they didn't choose to lose their notes.

The fix

A marker-bracketed skill block - HTML comments for `.md` files, `#`-prefixed line comments for rule files:

```
<!-- tina4-skills:start -->
## Tina4 Skills

- **tina4-maintainer** -- Read `.claude/skills/tina4-maintainer/SKILL.md` for framework-level ch
- **tina4-developer** -- Read `.claude/skills/tina4-developer/SKILL.md` before building features
- **tina4-js** -- Read `.claude/skills/tina4-js/SKILL.md` for frontend work.
<!-- tina4-skills:end -->
```

Four behaviours:

- **Fresh install** -> write the framework guide plus the skill block.
- **Marker refresh** (idempotent) -> file exists with our markers -> replace only the bracketed block.
- **One-time migration** -> file starts with the pre-v3.13.9 framework header -> replace the old dump with the new framework guide + skill block.
- **Preserve user content** -> file exists with the user's own content (no markers, no old header) -> append the skill block to the end, leave everything else verbatim.

The Ruby implementation also force-encodes UTF-8 on both read and write, so `File.read` returning `ASCII-8BIT` no longer trips up the string concatenation with non-ASCII content (em-dashes, [x] characters in the skill block).

Same algorithm in Python / PHP / Node

Identical four-branch logic, identical marker syntax, identical canonical action verbs in the log output. Skill content stays consistent across the family.

Tests

18 new specs in `spec/ai_installer_spec.rb`. All four branches plus marker detection, block replacement, idempotency, old-header detection, and rule-file vs markdown-file behaviour.

2,952 examples, 0 failures, 7 pending - no regressions.

What you'll see when you re-install

```
[OK] Migrated (replaced old framework dump in) CLAUDE.md   <- first run after upgrade
[OK] Refreshed skill block in CLAUDE.md                     <- every subsequent run
[OK] Appended skill block to CLAUDE.md                      <- user-curated file
```

---

v3.13.7 (2026-06-10)

Two changes from the 24rent app-platform team (PLATFORM-2159) - one observability hook, one production-safety fix. Both ship across **all four frameworks** with identical event payload shape.

NEW: `tina4.request.error` event

When `handle_500` catches a route exception, it now emits `tina4.request.error` **before** rendering the 500 page. Listeners receive a hash `{ exception:, request: }` and can ship the failure to CloudWatch / Sentry / Slack - even though the framework caught it.

```
Tina4::Events.on("tina4.request.error") do |payload|
  exc = payload[:exception]
  req = payload[:request]

  Tina4::Log.error("Route error: #{exc.class}: #{exc.message}",
                   method: req&.method, path: req&.path)
  # ...or POST to your centralised logging pipeline
end
```

- **Fires for caught route exceptions.** Does NOT fire for 404s - those aren't server errors.
- **Listener errors are swallowed + warning-logged** so a broken listener can't break the 500 render.
- **Listeners fire in priority order** (higher priority first, matching the existing `Tina4::Events.on(event, priority: N)`).
- **Identical event name + payload across Python / PHP / Node** - only the per-language syntax differs.

The framework already logged via `Tina4::Log.error` before - that line is unchanged.

FIX: Stack trace removed from production 500 body (CWE-209)

Before v3.13.7, an unhandled route exception in Ruby would render `"#{error.message}\n#{error.backtrace.first(10).join("\n")}"` into the 500 response body - absolute file paths, the top 10 frames, the exception message - **regardless of** `TINA4_DEBUG`. That's CWE-209 / OWASP A05: information disclosure.

The framework's own `lib/tina4/templates/errors/500.twig` now guards the trace block with `{% if error_message %}`. When `TINA4_DEBUG=false`, `handle_500` passes an empty `error_message` and the trace block doesn't render. The trace stays in `Tina4::Log.error` (server-side) and reaches observability via the new event.

When `TINA4_DEBUG=true`, the rich `Tina4::ErrorOverlay` page is unchanged.

Tests

Six new specs in `spec/router_error_event_spec.rb`: event payload shape, dev/prod symmetry, listener priority order, no traceback markers in prod body, request_id still surfaces, listener-error safety.

- 2,934 examples passing, 7 pending (PG container), no regressions.

Background

Reported by DevProx on the 24rent platform - they centralise observability by scraping structured JSON lines from stderr -> CloudWatch -> a Slack notifier. Route-level exceptions weren't surfacing because the framework caught them silently. The event hook fixes that without forcing any team's logging convention; the trace-leak fix is independently a security concern.

---

v3.13.6 (2026-06-09)

Two fixes: one Ruby-specific (spec contamination from v3.13.5), one cross-framework polish (driver install hints).

Spec contamination from Frond static-facade - fixed

v3.13.5 introduced `Tina4::Frond.add_filter / add_global / add_test` as a class-level registry (matching Python / PHP / Node). One side effect: globals set in one spec leaked into specs that expected the missing-variable fallback.

`spec/spec_helper.rb` now resets the registry between examples:

```
config.after(:each) do
  # ...existing cleanup...
  Tina4::Frond.clear_registry if defined?(Tina4::Frond) && Tina4::Frond.respond_to?(:clear_regis
end
```

No production code change - only the test harness. Matches the autouse fixture in Python and the `clearRegistry()` call in Node's `i18n-leaf-alias.test.ts`.

Better driver install hints (#47)

Driver gems (`pg`, `mysql2`, `tiny_tds`, `ruby-odbc`, `mongo`, `fb`) now raise a multi-line `LoadError` suggesting both Bundler and bare-gem install:

```
The 'pg' gem is required for PostgreSQL connections. Install one of:
    bundle add pg     # if your project uses Bundler
    gem install pg    # bare driver
```

Replaces the previous bare `LoadError` (or single-line `Install: gem install pg`).

#46 - PostgreSQL transaction cascade (no fix needed)

The cascade behaviour that prompted Python's #46 fix is psycopg2-specific. Ruby's `pg` gem uses libpq in autocommit mode by default - each statement is its own transaction, so a failed query does not poison subsequent ones. Verified.

Tests

2,928 passing, 7 pending (Postgres container).

---

v3.13.5 (2026-06-05)

Frond static-facade parity across PHP, Ruby, Node.js. Closes the last documented v3 parity gap (tina4-python task #32). Python's `Frond.add_filter` / `add_global` / `add_test` have worked as classmethods since v3.13.0 - now PHP / Ruby / Node match.

What changes

Filters, globals, and tests registered at app-startup persist across `new Frond()` instances. Every framework now supports the same pattern:

```
// PHP
\Tina4\Frond::addFilter("money", fn($v) => number_format((float)$v, 2));
\Tina4\Frond::addGlobal("APP_NAME", "My App");
\Tina4\Frond::addTest("positive", fn($v) => $v > 0);
```

```
# Ruby
Tina4::Frond.add_filter("money") { |v| "%.2f" % v.to_f }
Tina4::Frond.add_global("APP_NAME", "My App")
Tina4::Frond.add_test("positive") { |v| v > 0 }
```

```
// Node.js
Frond.addFilter("money", (v) => Number(v).toFixed(2));
Frond.addGlobal("APP_NAME", "My App");
Frond.addTest("positive", (v) => Number(v) > 0);
```

```
# Python - already shipped in v3.13.0
Frond.add_filter("money", lambda v: f"{float(v):.2f}")
Frond.add_global("APP_NAME", "My App")
Frond.add_test("positive", lambda v: v > 0)
```

In every framework, registering at the class level updates a static registry. The next `new Frond()` drains that registry into its own filter/global/test maps automatically. No need to thread a single `Frond` instance through the application - register at startup, render everywhere.

Instance form still works

Existing per-instance registration continues to work, and now propagates to the class registry too - so the lifecycle is symmetric:

```
$frond = new \Tina4\Frond();
$frond->addFilter("currency", $fn);
// Future `new Frond()` instances also see "currency"
```

`clearRegistry()` for test fixtures

Every framework exposes a class-level method to wipe user-registered filters/globals/tests without touching the built-ins (upper, lower, length, defined, even, ...). Useful in test setup/teardown to prevent state leaks between specs.

```
\Tina4\Frond::clearRegistry();
```

```
Tina4::Frond.clear_registry
```

```
Frond.clearRegistry();
```

```
Frond.clear_registry()
```

Implementation notes per framework

Framework	Mechanism
Python	`_ClassOrInstanceMethod` descriptor - one method, dual-callable via `__get__`
PHP	`__call` + `__callStatic` magic-method pair - PHP can't have same-name static and instance methods
Ruby	Same-name class method and instance method - Ruby naturally allows this
Node.js	TypeScript class supports same-name `static foo()` and `foo()` instance methods - distinct lookup spaces

Test count

Framework	Before	After	New
Python	2,741	2,741	0 (already covered)
PHP	2,858	2,871	+13
Ruby	2,907	2,928	+21
Node.js	3,508	3,526	+18
Total	**12,014**	**12,066**	**+52**

Upgrade

Drop-in patch. No breaking changes. Existing instance-form code (`$frond->addFilter(...)` / `frond.add_filter` / `frond.addFilter`) keeps working unchanged. The new static form is purely additive.

v3.13.4 (2026-06-04)

Three middleware/header bug fixes across all four frameworks, plus Python chapter 10 + 18 docs rewrites. Reported in tina4-book#140 and tina4-book#141 by MichaelC8E.

PY-10-02 - `@middleware()` no longer silently disables auth (SECURITY)

Before: Applying `@middleware(...)` to a POST/PUT/PATCH/DELETE route silently flipped `auth_required = false`, removing the framework's built-in Bearer-token gate. A developer adding custom logging or rate-limiting middleware to an admin endpoint would, with no warning, open it to unauthenticated callers.

After: Middleware is purely additive. Write routes stay Bearer-token-gated by default. Use `@noauth()` to open a write route, `@secured()` to lock a read route. Same rule across all four

frameworks.

This is a **behaviour change** - if your code relied on the old auto-disable to handle auth in custom middleware, add `@noauth()` (and have your middleware enforce auth on its own).

PY-10-03 - `request.headers` is now case-insensitive

Before: `request.headers["Content-Type"]` returned `None`/`undefined`/`nil`. The dict was lowercase-only; mixed-case lookups silently failed. Six chapter 10 examples (`Content-Type`, `X-API-Key`, `Authorization`, `User-Agent`) were broken.

After: HTTP headers are case-insensitive per RFC 7230 Section 3.2. Same is true in every framework:

Framework	Implementation
Python	`CaseInsensitiveDict` (dict subclass, normalises string keys to lowercase on read/write)
PHP	`Tina4\CaseInsensitiveArray` (ArrayAccess + IteratorAggregate + Countable)
Ruby	`Tina4::CaseInsensitiveHash < Hash` (overrides `[]`, `[]=`, `key?`, `delete`, etc.)
Node	Proxy wrapper around `http.IncomingHttpHeaders`

`request.headers.get("Content-Type")`, `request.headers.get("content-type")`, and `request.headers.get("CONTENT-TYPE")` all return the same value. Existing lowercase code keeps working unchanged.

PY-10-01 - Function-based middleware now runs

Before: Chapter 10 taught Express-style `async def mw(req, resp, next_handler)` in 8+ examples, but the Python framework's dispatcher only looked for class-based `before_*`/`after_*` methods. Function-style middleware was silently inert - body never executed. PHP and Ruby had similar gaps (closures ran but no `next` continuation).

After: Express-style continuation chain is implemented across the family. Python adds `_is_function_middleware()` + `_invoke_handler_with_middleware()`. PHP wraps closures with `array_reverse` continuation. Ruby uses lambdas + `reverse_each`. Node already had `next()` continuation - added a regression test to keep it green.

```
@middleware(my_mw)
@post("/api/orders")
async def create_order(req, resp):
    ...

async def my_mw(req, resp, next_handler):
    if not authorised(req):
```

```
        return resp.json({"error": "forbidden"}, 403)
    result = await next_handler(req, resp)   # continue the chain
    return result
```

First-declared middleware is the outermost layer; calling `next_handler` descends to the next layer (or the route handler if last). Omitting the `next_handler` call short-circuits the chain.

Python chapter rewrites - book + docs

- **Chapter 18 (Testing)** - Fixed PY-18-04 (test runner output now shows real pytest output, not the fictional `[PASS] test_addition` format), PY-18-07a (added missing `from src.orm.Product import Product` import), PY-18-08 (`resp.status_code` -> `resp.status` across 14+ call sites, positional body `self.post(path, dict)` -> keyword `self.post(path, json=dict)`).
- **Chapter 10 (Middleware)** - Added two callouts: headers are case-insensitive in v3.13.4+; `@middleware()` is purely additive (does not change auth_required). Existing mixed-case header examples now work against v3.13.4.

Test count

Framework	Before	After	New
Python	2,725	2,741	+16
PHP	2,844	2,858	+14
Ruby	2,887	2,906	+19
Node.js	3,477	3,508	+31
Total	**11,933**	**12,013**	**+80**

Upgrade

PY-10-02 is a behaviour change with a security implication. Audit routes that use `@middleware()` on POST/PUT/PATCH/DELETE: if you rely on custom middleware to handle auth, add `@noauth()` above `@middleware()` (and make sure your middleware enforces auth). Otherwise, no action - your write routes were always supposed to require Bearer tokens.

PY-10-01 and PY-10-03 are purely additive - no breaking changes.

v3.13.3 (2026-06-03)

Two reporter-driven ergonomic additions, shipped across all four frameworks with full parity per `feedback_parity`.

`Env` typed env-var helpers (tina4-python#43)

Reading env vars by hand gets old fast: every boolean flag becomes a `os.getenv("TINA4_DEBUG", "false").lower() in ("1", "true", "on", "yes")` incantation. Every numeric tuning knob needs a try/except around `int()`. `Env` centralises it:

```
from tina4_python import Env
```

```
debug   = Env.bool("TINA4_DEBUG", default=False)
workers = Env.int("WORKERS", default=4)
rate    = Env.float("RATE_LIMIT", default=10.0)
region  = Env.str("AWS_REGION", default="us-east-1")
```

Same API across all four frameworks:

- **Python** - `from tina4_python import Env`
- **PHP** - `Tina4\Env::bool / int / float / str`
- **Ruby** - `Tina4::Env.bool / int / float / str`
- **Node.js** - `import { Env } from "@tina4/core"`

Truthy tokens (case-insensitive after `strip`/`trim`): `1`, `true`, `on`, `yes`, `y`, `t`. Falsy: `0`, `false`, `off`, `no`, `n`, `f`, empty string. Anything else returns the `default` - never raises. `int`/`float` parse failures log a warning via `Log` and fall back to default.

Function-name in log lines (tina4-python#41)

Opt-in via `TINA4_LOG_FUNC=true`. When enabled, the calling function name is injected into every log line so a `tail -f` gives you free context:

```
2026-06-03T14:22:18.341Z [INFO   ] [super_trooper] Hello from inside the function
```

Or in JSON mode:

```
{"timestamp":"...","level":"INFO","function":"super_trooper","message":"Hello..."}
```

Default off - zero overhead unless opted in. When on, ~5% per-call cost from the stack walk.

Per-framework implementation:

- **Python** - `inspect.currentframe()` walk past Log's own frames
- **PHP** - `debug_backtrace(DEBUG_BACKTRACE_IGNORE_ARGS)` + `{closure}` filter
- **Ruby** - `caller_locations(2, 16)` + block-noise regex
- **Node.js** - `new Error().stack` regex parse + anonymous filter

Anonymous frames (`<lambda>`, `<module>`, `{closure}`, anonymous IIFEs) are filtered as noise - showing `[<lambda>]` would be uglier than nothing.

Test count

Framework	Before	After	New
Python	2,675	2,725	+50
PHP	2,780	2,844	+64
Ruby	2,839	2,887	+49 (+1 pre-existing rack_app fail unchanged)
Node.js	3,420	3,477	+57
Total	**11,714**	**11,933**	**+220**

Upgrade

Drop-in patch. No breaking changes. Two new exports (`Env`, plus one new env var `TINA4_LOG_FUNC`). Existing logs and existing code keep working unchanged.

v3.13.2 (2026-06-03)

Bug-fix patch - three field reports, fixed with full cross-framework parity audit per `feedback_crosscheck_bugs`.

SCSS calc() with mixed units (tina4-python#42, tina4-php#116, tina4-nodejs#1)

The SCSS math evaluator silently folded mixed-unit arithmetic by keeping operand 1's unit and dropping operand 2's, producing wrong CSS:

- `max-height: calc(100vh - 170px)` -> `calc(-70vh)` (negative, layout-breaking)
- `width: 100% - 20px` -> `80%` (pixel term silently lost)
- `padding: 1rem + 4px` -> `5rem`

Fixed in Python, PHP, and Node - the evaluator now captures both operand units, only folds when units match (or one side is unitless for `*`/`/`), and masks `calc(...)` ranges so the browser computes them as intended. Ruby unaffected (delegates to libsass).

Router.group docs taught a crashing pattern (tina4-python#44)

The Python book and docs site showed `Router.group("/api/v1", lambda: [...])` with a zero-arg lambda. Source intentionally passes a `RouteGroup` instance to the callback, so users hit `TypeError: <lambda>() takes 0 positional arguments but 1 was given`. Docs rewritten to `lambda group: [group.get(...), group.post(...)]` matching the real contract (Node has always taught this correctly; PHP and Ruby use ambient state, no group arg needed).

DATABASE*URL -> TINA4*DATABASE_URL drift (tina4-python#45)

Three real bugs:

- **Python ORM error message** told users to "set DATABASE_URL in .env" - but the v3.12 boot guard rejects that bare name. Users following the error message hit a hard stop.
- **Python dev-admin `.env` writer** stripped the `TINA4_` prefix when updating existing rows, actively corrupting the config every time the user saved a new connection through the dashboard.
- **Node** `Database.fromEnv()` defaulted to `"DATABASE_URL"` as the env-var key, missing the project's actual connection. The ORM error message had the same drift.

All fixed. PHP and Ruby audited - already correct in both.

Test count

Framework	Before	After	New
Python	2,665	2,675	+10
PHP	2,774	2,780	+6
Ruby	2,839	2,839	0 (parity bump only)
Node.js	3,406	3,420	+14
Total	**11,684**	**11,714**	**+30**

Upgrade

Drop-in patch. No breaking changes. No new public API.

v3.13.1 (2026-06-02)

Cross-framework parity patch. Closes the remaining audit-flagged docs-vs-code gaps that didn't make 3.13.0 - the documentation claimed APIs across PHP / Ruby / Node that only Python had. This release ships those APIs everywhere and rewrites the PHP chapters that referenced fictional symbols.

Convenience parity additions (Groups A / B)

Three highest-impact cross-framework methods every documentation set already claimed existed. PHP / Ruby / Node now match Python:

- `db.fetchAll(sql, params)` / `db.fetch_all` / `$db->fetchAll` - returns the records list directly. Symmetric with `fetch_one`. For the 80% case where you don't need the `DatabaseResult` metadata.
- `Database.getConnection(url)` / `.get_connection` / `::getConnection` - classmethod factory matching SQLAlchemy's `engine.connect()`. Falls back to in-memory SQLite when no URL or env resolves.
- `Api(bearerToken=, username=, password=, headers=, verifySsl=)` **ergonomic kwargs** - three setter calls collapse to one constructor. Bearer wins over basic-auth when both are passed. `verifySsl=False` is the positive form of `ignoreSsl=true`.

Decorator-style GraphQL resolvers across the family

Python `@GraphQL.resolve` shipped in 3.13.0. This release adds:

- **PHP** - `GraphQL::resolve("Type", "field", $callable)` static method + class-level resolver registry that `new GraphQL()` drains into its schema.
- **Ruby** - `Tina4::GraphQL.resolve("Type", "field") { |root, args, ctx| ... }` with block-based registration.
- **Node.js** - `GraphQL.resolve(typeName, fieldName, resolver)` matching the cross-framework shape.

All four frameworks now support the FastAPI / Strawberry / Ariadne pattern where resolvers register at module-import time before any `GraphQL` instance is constructed, and where post-startup registrations land in the active default singleton via `setDefault(gql)` / `Tina4::GraphQL.default_instance = gql`.

Class-based service pattern across the family

`class FooWorker extends Service { run() { ... } }` - chapter 27 / equivalent docs have long taught this pattern. Until 3.13.1, only the runner was real:

- **PHP** - new `Tina4\Service` abstract base class + `ServiceRunner::registerService($name, $service)` static helper.
- **Ruby** - new `Tina4::Service` class + `Tina4::ServiceRunner.register_service(name, service)`.
- **Node.js** - new `Tina4Service` abstract class + `ServiceRunner.registerService(name, service)`.
- **Python** - new `tina4_python.service.Service` base + `ServiceRunner.register_service(name, service)` (this release closes the gap; Python had only the function-style runner before).

All four ship `run()` (abstract), `stop()`, and `should_stop()` / `shouldStop()` helpers backed by an internal flag. Function-style services using bare callables continue to work alongside the new class-based pattern.

PHP chapter rewrites (`docs/php/` and `book-2-php/`)

The 3.13.0 audit found that the PHP testing-chapter disaster was the tip of a larger pattern - multiple PHP chapters taught APIs that didn't exist. 3.13.1 rewrites all seven of them:

- **Chapter 15 - Logging** - primary surface now `Tina4\Log::info()/warning()/error()` instead of the legacy `Tina4\Debug::message()` shim (still works).
- **Chapter 18 - Testing** - `$response->statusCode` -> `$response->status` across 23 occurrences; CLI section updated (`tina4 test` runs the suite; `vendor/bin/phpunit` for targeted runs).
- **Chapter 19 - Scaffolding** - v2 `Tina4\Get::add()` / `Post::add()` / `Put::add()` / `Delete::add()` syntax replaced with `Tina4\Router::get/post/put/delete`; fictional `->description()` chain replaced with real `->swagger([...])`.
- **Chapter 22 - GraphQL** - chapter's decorator pattern (`GraphQL::resolve("Type", "field", $fn)`) now matches real source (built this release).
- **Chapter 25 - WSDL** - `@wsdl_operation` docblock replaced with `#[WSDLOperation([...])]` PHP attribute; methods now return associative arrays matching the response-shape spec; `Router::soap()` -> `Router::any()` + manual `(new Service($request))->handle()`.
- **Chapter 27 - ServiceRunner** - `new ServiceRunner()` + `->add()` instance API replaced with `ServiceRunner::registerService()` + `ServiceRunner::start()` static API. The `Tina4\Service` base class the chapter teaches now exists.

- **Chapter 34 - Deployment** - un-prefixed env vars (`SECRET`, `CORS_ORIGINS`, `SMTP_USER`, `JWT_SECRET`, `API_KEY`, `SWAGGER_TITLE`) replaced with `TINA4_`-prefixed forms. The v3.12 boot guard rejects the legacy names with `exit(2)`.

Test count

Net new across the family this release:

Framework	Before	After	New
Python	2,654	2,665	+11
PHP	2,749	2,774	+25
Ruby	2,827	2,839	+12
Node.js	3,384	3,406	+22
Total	**11,614**	**11,684**	**+70**

Upgrade

Drop-in patch - no breaking changes. Existing source-code patterns from 3.13.0 continue to work; the new methods are additive. Documentation rewrites in chapters 15 / 18 / 19 / 22 / 25 / 27 / 34 redirect copy-paste examples to the real APIs the framework actually ships.

v3.13.0 (2026-06-01)

The docs-vs-code parity release. A cross-framework audit of 381 markdown files surfaced 146 hallucinations, signature drifts, and stale references across Python, PHP, Ruby, Node, and tina4-js. 3.13.0 closes the chapter-18 disaster pattern - where documentation taught a class-based API that didn't exist - by shipping the missing pieces, renaming the misnamed pieces, and rewriting the aspirational chapters.

The headline fire: `Test` class with HTTP helpers

Every framework's chapter 18 has long shown integration tests like:

```
class UserApiTest(Test):           # tina4_python.test.Test
    def test_health(self):
        resp = self.get("/health")
        assert_equal(resp.status, 200)
```

Until 3.13.0, only `Test` (the bare assertion base) existed - calling `self.get(...)` crashed with `AttributeError`. The HTTP test client lived in a separate `TestClient` class that the docs never mentioned.

This release mixes `TestClient.get / post / put / patch / delete` into the `Test` base across every framework:

- **Python** - `tina4_python.test.Test` (extends `unittest.TestCase`, pytest auto-discovers)
- **PHP** - `Tina4\Test` (extends `PHPUnit\Framework\TestCase`)

- **Ruby** - `Tina4::Test` (zero-dep; built-in `run_all` runner)
- **Node.js** - `Tina4Test` (zero-dep; built-in `Tina4Test.runAll()` runner)

Plus positional assertions on every framework - `assertEqual(actual, expected, message)`, `assertNotEqual`, `assertTrue`, `assertFalse`, `assertNull`/`assertNullValue`, `assertNotNull`/`assertNotNullValue`, `assertRaises` - matching the documented `(actual, expected, message)` shape.

`Auth.valid_token` now returns the payload, not a bare bool - BREAKING

The most common silent-fail pattern caught by the audit. Every framework's docs claimed `valid_token` returned the decoded JWT payload; every framework's source returned `bool` and forced a second `get_payload` call.

Framework	Before	After	
Python	`Auth.valid_token(token) -> bool`	`Auth.valid_token(token) -> dict \	None`
PHP	`Auth::validToken(token) -> bool`	`Auth::validToken(token) -> array \	null`
Ruby	`Auth.valid_token(token) -> Boolean`	`Auth.valid_token(token) -> Hash \	nil`
Node	`validToken(token) -> boolean`	`validToken(token) -> Record<string, unknown> \	null`

Matches PyJWT / firebase-jwt-ruby / firebase/php-jwt / jsonwebtoken conventions. Truthy/falsy contract preserved - existing `if (validToken(t))` callers keep working because a non-null object is truthy and null is falsy.

Python-specific groups (mirrored to PHP/Ruby/Node in follow-up patches)

The Python framework is the reference per `feedback_python_master`. Six groups landed in tina4-python:

- **Group A - ergonomic additions**: `Database.get_connection()`, `db.fetch_all()`, `db.pool`, `DatabaseResult.columns`, `Job.error`, `Queue.produce(delay_until=datetime)`, module-level `migrate(db)`/`rollback(db)`/`status(db)`, module-level `i18n.t()`, dict-style `session[key]`, `WebSocketConnection.connection_count`. All zero-risk additions - no signatures changed.
- **Group B - signature expansions**: `Api(bearer_token=, username=, password=, headers=, verify_ssl=)` kwargs, `Model.find(pk)` int overload (Active Record convention), `@description(summary, detail=, params=, query=)`, `@tags(str | list)`, `@example_response(status_code, data)`, `response.render(template, data, status_code)`, `response.cookie(name, value, options_dict)`, `response(data, headers={})`, `@get(path, description=, middleware=["ResponseCache:300"])` with string-form middleware parser.

• **Group C - mixins + decorators**: the Test HTTP mixin (covered above), `Frond.add_filter / add_global / add_test` callable as classmethod OR instance method via a `_ClassOrInstanceMethod` descriptor, `@GraphQL.resolve("Type", "field")` decorator with class-level registry - chapter 22's pattern now works as documented.

• **Group D - return-type changes (BREAKING)**: `Container.reset()` now clears singleton cache only (factories survive); new `Container.reset_all()` for the old wipe-everything behaviour. `queue.dead_letters()` returns `list[Job]` with `.error` populated, not `list[dict]`. `Model.where(..., with_count=True)` returns `(list, int)` tuple for pagination UIs.

• **Group E - renames (BREAKING)**: `ai.install_all()` -> `ai.install_context()`; new `ai.detect_ai()`, `ai.detect_ai_names()`, `ai.status_report()`. `queue.consume(id=)` -> `queue.consume(job_id=)`. `Api.send_request()` -> `Api.send()`. `I18n(locale=, path=)` preferred over `I18n(locale_dir=, default_locale=)` (legacy kept). `TINA4_TOKEN_EXPIRES_IN` preferred over `TINA4_TOKEN_LIMIT` for JWT expiry (both honoured; new wins; constructor arg overrides both).

• **Group F - top-level re-exports + scaffolder**: `from tina4_python import Api, WSDL, wsdl_operation, GraphQL, AutoCrud, Messenger, on, emit, once, off, tests` now resolve. `Model.select()` with no args defaults to `SELECT * FROM <table>` so the CRUD-list scaffolder template's emitted code actually runs.

PHP-specific: `Tina4\Debug` shim

Chapter 15 of the PHP logging docs taught `Tina4\Debug::message($msg, TINA4_LOG_INFO, [...])`. Neither the class nor the constants existed. Real logger is `Tina4\Log`.

This release ships a `Tina4\Debug` compatibility shim that forwards to `Tina4\Log`, plus defines the `TINA4_LOG_*` level constants - so the chapter's code samples run as-written. For new code, prefer `\Tina4\Log::info()` etc.

Documentation sweep

Aside from the source-side changes, the audit caught hundreds of stale references in docs site + book + AI skills + CLAUDE.md files. All fixed in this release:

• ~80 occurrences of `from tina4 import` -> `from tina4_python import` (the Python package is `tina4_python`, not `tina4`)

• `from tina4_python.router` -> `from tina4_python.core.router`

• `TINA4_SESSION_HANDLER` -> `TINA4_SESSION_BACKEND` (matches the env var the framework actually reads)

• `DATABASE_NAME=` -> `TINA4_DATABASE_URL=` (legacy un-prefixed names get rejected by the v3.12 boot guard)

• `@cached(True, max_age=N)` -> `@cached(max_age=N)` (bogus first arg)

• `Template.render()` -> `response.render()` (Template class doesn't exist; renamed to Frond)

- `Debug.error()` -> `Log.error()` in Python (Debug class doesn't exist)
- `Producer` / `Consumer` (removed in v3.2.0) -> `Queue.push / consume`
- `Email` -> `Messenger`, `event.fire / @listener` -> `emit / @on`, `gql` singleton -> `GraphQL()` + `@GraphQL.resolve`
- **Security fix**: `Auth.check_password(hash, password)` -> `(password, hash)` in skill ref - the bcrypt comparison was returning False every time due to reversed args (silent-failure auth)
- `request.files['content']` is **raw bytes** - drop `base64.b64decode()` from upload examples
- Deployment chapter env vars all `TINA4_`-prefixed (un-prefixed names brick boot under v3.12 guard)

Aspirational chapters rewritten

Two Python chapters were built on APIs that didn't exist:

- **Chapter 22 (GraphQL)**: rewritten around the new `@GraphQL.resolve("Type", "field")` decorator (the FastAPI/Strawberry pattern). The previous `gql.schema.add_query("name", {dict})` form still works but is no longer the primary documented path.
- **Chapter 25 (WSDL)**: rewritten around the real subclass pattern (`class Calculator(WSDL): @wsdl_operation({"Result": int}) def Add(self, ...): ...; Calculator(request).handle()`). The previous `WSDL(service_name=, namespace=, endpoint=)` constructor + `handle_wsdl` / `handle_request` API was entirely fictional.

tina4-js doc drift caught

The cross-framework audit's synthesis pass dropped tina4-js findings; the raw agent transcripts had 23 real findings:

- Every import in CLAUDE.md and `docs/js/09-graphql.md` used `"tina4-js"` (with hyphen). The npm package is named `tina4js`. Fixed.
- `pwa({...})` was treated as callable; real API is `pwa.register({...})`. `PWAConfig.icon` is a single string, not an `icons: [...]` array.
- `static props = { label: { type: String, default: "..." } }` - the `{ type, default }` wrapper is fictional. Real shape is `static props = { label: String }`.
- `router.navigate('/users/42')` - `navigate` is a top-level export, not a method on `router`.
- Chapter 14's `<slot>` inside a `static shadow = false` component - slots are a Shadow DOM feature. Chapter 14 contradicted chapter 4. Switched to `shadow = true`.

Test counts

Net new across the family:

Framework	Before	After	New
Python	2,537	2,654	+117
PHP	2,742	2,749	+20
Ruby	2,800	2,816	+16 (1 pre-existing unrelated rack_app failure)
Node.js	3,366	3,384	+18
Total	**11,445**	**11,603**	**+171**

Upgrade

`Auth.validToken` is the breakage to know about - your `if Auth::validToken($t)` style code keeps working unchanged because non-null arrays are truthy and null is falsy. If you do `=== true` / `=== false` strict comparisons, switch to `!== null` / `=== null`.

Python: `ai.install_all()` -> `ai.install_context()`, `queue.consume(id=)` -> `consume(job_id=)`, `Api.send_request()` -> `Api.send()`, `Container.reset()` semantic change (use `reset_all()` for old behaviour).

Everything else is additive - new properties, new kwargs, new convenience methods that match what the docs have promised for years.

v3.12.14 (2026-06-01)

Two independent fixes ship together as 3.12.14. **Python** - the `tina4_python.test` class-based xUnit testing surface that the chapter 18 documentation has always promised but never actually existed. Reports came in of developers copy-pasting `from tina4_python.test import Test, assert_equal, assert_true` straight out of the book and getting `ModuleNotFoundError`. The fix was to build the module to match the documentation, not the other way around. **PHP** - `:named` placeholder translation for the four non-PDO adapters where `ORM::save()` was silently failing.

Python - `tina4_python.test` xUnit testing surface

The testing chapter taught a `Test` base class with positional assertions:

```
from tina4_python.test import Test, assert_equal, assert_true

class BasicTest(Test):
    def test_addition(self):
        assert_equal(2 + 2, 4, "Basic addition should work")

    def test_string_contains(self):
        greeting = "Hello, World!"
        assert_true("World" in greeting, "Greeting should contain 'World'")
```

The module did not exist. Every developer who followed the chapter hit an immediate import error. The other surface, `tina4_python.Testing` with the inline `@tests` decorator, has always

existed - but the two are for different purposes and the docs only documented one of them.

The fix ships the missing module - `tina4_python/test/__init__.py` - with the `Test` base class (inherits `unittest.TestCase`, so pytest discovers any subclass regardless of class-name convention) and 13 positional assertions. The signatures are uniform: `(actual, expected, message)`. The 2-arg legacy `(value, message)` form keeps working - a type-based dispatch detects which shape the caller used. `assert_raises` accepts three forms: docs form (`callable, exception, message`), context-manager form (`with assert_raises(X):`), and unittest order (`exception, callable`). Lifecycle hooks come in both flavours - snake_case `set_up`/`tear_down` (the Tina4 idiom) and camelCase `setUp`/`tearDown` (for users coming from unittest) - without double-calling when a subclass uses either one.

```
# 13 assertions, all uniform (actual, expected, message)
assert_equal(actual, expected, message="")
assert_not_equal(actual, expected, message="")
assert_true(actual, expected=True, message="")
assert_false(actual, expected=False, message="")
assert_none(actual, expected=None, message="")
assert_not_none(actual, expected="not None", message="")
assert_in(item, container, message="")
assert_not_in(item, container, message="")
assert_is_instance(value, expected_type, message="")
assert_greater(actual, expected, message="")
assert_less(actual, expected, message="")
assert_almost_equal(actual, expected, places=7, message="")
assert_raises(callable, exception_class, message="")
```

51 new tests in `tests/test_test_module.py` pin the contract: BasicTest from the chapter runs verbatim, every assertion fails when it should and passes when it should, both 2-arg and 3-arg shapes work, snake_case and camelCase lifecycle hooks fire once each (never both). Full Python suite: 2,537 passing.

`tina4 test` continues to run pytest (`subprocess.run([sys.executable, "-m", "pytest", "tests/"] + args)`).

Cross-framework parity check (testing)

PHP / Ruby / Node testing chapters already teach native conventions correctly - PHPUnit, RSpec, and `node:test` respectively. No fake API to fix. The Python-specific gap was that Tina4 had two testing surfaces (`tina4_python.Testing` for inline `@tests` decorator, `tina4_python.test` for class-based suites) and only one of the two existed. The other three frameworks defer to a single native runner each, so the same trap doesn't apply.

PHP - :named placeholder translation across non-PDO adapters

The ORM's `save()` emits `:named` placeholders because PDO would accept them. Four of the five PHP database adapters do not use PDO. `MySQLAdapter` (mysqli), `MSSQLAdapter` (sqlsrv), `FirebirdAdapter` (ibase/fbird), and `PostgresAdapter` (pgsql) all bind positionally. Every INSERT/UPDATE through `save()` against those four engines failed silently. Reads worked because read paths typically use `?` or no params.

A single helper, `SqlTranslation::namedToPositional($sql, $params)`, translates `:name` to `?` and reorders `$params` to match the SQL order. Wired into the four affected adapters

at the top of their prepare/execute paths. The helper skips string literals and SQL comments, so a literal `:colon` inside a value stays as a value. Duplicate names bind once per occurrence, so `WHERE id = :id AND parent_id = :id` works as expected.

`SQLite3Adapter` is untouched. ext-sqlite3 natively accepts `:name` via `SQLite3Stmt::bindValue`. The other four did not, and now do.

15 unit tests pin the helper in `tests/SqlTranslationNamedToPositionalTest.php`: order preservation, duplicate names, quoted strings, line and block comments, unknown placeholders, null values, and the `0`-as-value case. Full PHP suite: 2,290 passing.

Cross-framework parity check (`:named` placeholders)

Python (`mysql-connector-python` uses `%s`), Ruby (`mysql2` uses `?`), and Node (`mysql2` uses `?`) build their INSERT/UPDATE SQL with positional placeholders from the ORM down. No `:named` ever emitted. Audited the MySQL adapter and `save()` path in each before shipping; confirmed clean. PHP-only fix.

Upgrade

Drop in for both Python and PHP. No `.env` changes, no API changes.

Python users who followed chapter 18 and hit `ModuleNotFoundError` - bump to `3.12.14`, the `from tina4_python.test import Test, assert_equal, ...` import now resolves. Existing tests written against `tina4_python.Testing` (the inline `@tests` decorator) continue to work - that surface was not touched.

PHP users - `:named` and `?` both work, and the framework picks the right form for whichever driver is underneath. Existing ORM `save()` calls start succeeding on MariaDB/MySQL, PostgreSQL, MSSQL, and Firebird.

Ruby and Node users - no framework change shipped in 3.12.14. Stay on `3.12.13` or bump to `3.12.14` for version alignment. Both are functionally identical.

v3.12.13 (2026-05-29)

Consolidated parity release. PHP ran ahead through two independent patch releases (3.12.11-3.12.12) while Python / Ruby / Node stayed at 3.12.10. This release realigns all four frameworks on **3.12.13** and ships the cross-framework dev-admin parity sweep - five tiers of work that bring PHP, Ruby and Node up to Python's AI-assisted development surface.

Cross-framework dev-admin parity sweep (Tier 1-5)

The Python framework had pulled ahead on a series of dev-admin features driven by real frustration with the AI coder loop ("Applying a small patch went and messed up my whole file", "Says it is creating files but then doesn't", repeated import-error spirals). This release ports the full set to PHP, Ruby, and Node - same intent, language-idiomatic implementations.

Tier 1 - MCP defensive write layer. `file_write` and `file_patch` now refuse prose-as-filenames (the LLM occasionally emits `## FILE: I'll implement Step 1 by creating the database migration` and the parser used to write a zero-byte file with that sentence as its filename), normalise bare top-level `routes/` / `orm/` / `templates/` / `seeds/` /

`controllers/` / `middleware/` paths to their canonical `src/<dir>/` form (auto-discovery only scans `src/`, so a file at `templates/foo.twig` was dead weight), back up existing files to `.tina4/backups/<flat-path>.<ISO-ts>.bak` before overwrite, and refuse suspicious truncations (>200B file -> <30% size = almost always a truncated LLM response). Every attempt logs to `.tina4/agent.log` with a structured category (`write.ok` / `write.refused` / `write.path_normalized` / `write.import_failed`) - the supervisor reads that file on every turn so it sees what broke last time and can self-correct without asking the developer "what's the error?".

Tier 2 - Post-write syntax verification. PHP shells out to `php -l`, Ruby to `ruby -c`, Node to `node --check` (and single-file `tsc --noEmit --allowJs --skipLibCheck` for `.ts`). On parse error the tool result gets an `import_error` field AND a `write.import_failed` log entry surfaces in the next supervisor turn's failure context. Catches hallucinated framework APIs (`CharField` doesn't exist in `tina4_python.orm.fields` - should be `StrField`; `auto_now_add` keyword on `Field.__init__()`) at write time instead of letting them propagate to a runtime 500 the user only discovers by hitting the URL.

Tier 3 - `/__dev/api/threads` + `/__dev/api/chat` proxy. The SPA now talks to the Rust supervisor agent the same way regardless of framework. `_supervisor_base_url()` matches Python's 4-step ladder (`TINA4_SUPERVISOR_URL` -> `TINA4_AGENT_PORT` -> `PORT+2000` -> `9145`). `active_file` rides through `/chat` POST verbatim so deictic phrases ("fix this", "explain this") bind to the editor's open file without the supervisor asking. The Node port forwards SSE chunks as they arrive; PHP and Ruby buffer (functional - EventSource parses fine - but feels less snappy until a future round of Rack/PHP-FPM streaming work).

Tier 4 - Customer feedback widget. A floating bubble for end-users of a shipped Tina4 app, gated by `TINA4_ENABLE_FEEDBACK=true` AND a non-empty `TINA4_FEEDBACK_WHITELIST`. The framework's response middleware injects `<script src="/__feedback/widget.js" data-tina4-feedback></script>` immediately before the LAST `</body>` tag on text/html responses, ONLY for whitelisted users, NEVER on `/__dev` or `/__feedback` paths (no double-bubble UX on the developer dashboard). One conversational turn at a time POSTs to `/__feedback/api/turn` -> server-side identity stamp from the verified JWT (clients cannot fake `sender`) -> forward to the Rust agent's intake-only agent (zero tools, JSON-only output). Finalised tickets land in the dev admin sidebar with `kind:"feedback"`. Rate-limited at 5 turns/hour per user.

Tier 5 - Stale-source overlay badge + `list_plans()` merge. The error overlay now stamps `captured_at` on render and tags each stack frame whose source file has been modified since: "FILE MODIFIED @ HH:MM:SS UTC - source may not match what failed". Stops the user from chasing ghosts when the AI coder rewrote the file between the error and the page reload. `list_plans()` reads from BOTH `plan/` (user-curated canonical) AND `.tina4/plans/` (AI-planner output), dedupes by filename with `plan/` winning on collision, sorts newest-first, and returns a `path` field so the SPA can open the right file regardless of source dir.

Test counts. Per-framework deltas across the sweep:

Framework	Before -> After (full suite)

Python	2453 -> 2453 (canonical - no new tests, just released)
PHP	2235 -> 2714 (+479)
Ruby	2747 -> 2800 (+53)
Node	3263 -> 3368 (+105)

PHP's larger delta reflects new tests + the 3.12.11 + 3.12.12 lineage rolling forward.

Why all four frameworks at once. Per the cross-framework parity rule: a feature that exists in only one framework is technical debt. The Python-only Tier 1-5 surface had been accumulating for two weeks while the UX was settling. With it settled, this release closes the gap in one coordinated sweep.

Folded-in from PHP 3.12.11 - file upload regression (`tina4-book#139`)

`WebSocket::parseHttpHeaders()` previously split the entire raw HTTP request on `\r\n` and iterated every line for a `:` to fill the headers map. Multipart body parts have their own `Content-Type`, `Content-Disposition`, and `Content-Transfer-Encoding` headers - those lines matched the parser and overwrote the real request `Content-Type: multipart/form-data; boundary=...` with whatever the last body part's content type was (typically `application/pdf`, `image/png`). Downstream `str_contains($contentType, 'multipart/form-data')` then failed, the multipart branch was skipped, `$parsedFiles` was never set, and `$request->files` came out empty. Every file upload through the stream-socket server was silently lost - the body landed in `$request->body` as a raw multipart string with no way to parse it.

Fix. Stop the parser at the first `\r\n\r\n` (RFC 9112 Section 2.2 boundary between headers and body) before splitting into lines. One logical change in `Tina4/WebSocket.php`. 9 regression tests in `tests/BookIssue139Test.php` cover single-part, multi-part, and mixed-header cases.

Cross-framework parity check. Python (`http.server`), Ruby (`webrick`/`puma`), and Node (built-in `http` module) all delegate header parsing to upstream stdlib HTTP parsers that already split headers from body correctly. PHP was the only framework with a hand-rolled HTTP parser in this code path. No port needed.

Folded-in from PHP 3.12.12 + Python 3.12.13 - v2 `tina4_migration` auto-upgrade (#115)

Projects upgrading from tina4 ^2.x to ^3.x carried a v2-shaped `tina4_migration` table that v3's `ensureMigrationsTable()` left untouched (the `CREATE TABLE IF NOT EXISTS` short-circuited). The v3 reader then selected columns that didn't exist, fell into the "never seen this migration, run it" branch, and re-applied already-applied migrations - typically failing on duplicate-column / table-already-exists errors when the SQL was non-idempotent. The AirOffices ~190-migration codebase tripped on this in March 2026 and needed a manual SQL backfill at the time.

Framework	v2 schema	v3 schema

PHP	`migration_id VARCHAR(14)`, `description`, `content BLOB`, `passed`	`id INT PK`, `migration`, `batch`, `applied_at`
Python	`description` as identifier, `content`, `passed`	`migration_id`, `migration_name`, `executed_at`

Fix. `ensureMigrationsTable()` (PHP) and `_ensure_tracking_table()` (Python) now detect a v2-shaped table (v2 columns present, v3 columns absent) and call an in-place upgrade that ALTERs in the v3 columns alongside the v2 ones, then backfills v3 fields from the v2 data. v2 columns are kept in place so a manual rollback path stays open - they're simply ignored by v3 readers. The match is by file stem: a v2 row's identifier is matched against `migrations/` files by basename (Python uses `000001_create_users.sql` -> stem `000001_create_users` -> v2 description `create_users`).

Cross-framework parity check. Ruby and Node never shipped a v2 migration table with the trapping shape - their v2 lineages used a different column layout that v3's tracker tolerated. Nothing to port.

Folded-in from PHP 3.12.11 - request URL parity

`$request->url` now returns the full absolute URL (`https://host:port/path?query`) instead of just the path. `$request->queryString` (raw query bytes) added for parity with `request.query_string` on the other frameworks. Drop-in - old code that read `$request->path` (untouched) keeps working.

Upgrade

Drop in. No `.env` changes, no API changes.

For projects upgrading from v2.x: the v2 `tina4_migration` auto-upgrade runs once on first boot against v3 - back up your migrations table beforehand if you're paranoid. The upgrade is non-destructive (v2 columns are kept alongside the new v3 ones).

For projects using the dev admin AI coder loop: the new MCP defensive layer will silently rewrite `## FILE: routes/foo.py` to `src/routes/foo.py` and log a `write.path_normalized` entry. If you were relying on the old behaviour (writes landing wherever the LLM emitted them), this will move some files. Run `tail -n 50 .tina4/agent.log | grep path_normalized` after upgrading to see what got rewritten.

For shipping apps that want the customer feedback widget: set `TINA4_ENABLE_FEEDBACK=true` AND `TINA4_FEEDBACK_WHITELIST=alice@example.com,bob@example.com` in `.env`. The widget appears only for those users on non-`/__dev` pages.

v3.12.10 (2026-05-14)

Version-alignment release. PHP ran ahead through three independent patch releases (3.12.7-3.12.9) while Python / Ruby / Node stayed at 3.12.6. This release realigns all four frameworks on **3.12.10** and ships the ORM `save()` fix.

PHP - `ORM->save()` no longer swallows write failures (#114)

`ORM->save()` called `update()`/`insert()` but ignored their `bool` return - it only caught exceptions. The PHP adapter's `exec()` returns `false` on a bad statement instead of throwing, so a failed `UPDATE` (commonly: one referencing a public model property with no matching DB column, since `getDbData()` includes every public property) slipped through. The empty transaction got committed and `save()` returned `$this` - the documented success signal. Callers relying on the `save(): static|false` contract believed the row persisted when nothing changed. **Silent data loss** - no exception, no log.

Fix. `save()` now captures the `bool` return of `update()`/`insert()`, rolls back, and returns `false` on a falsy result.

```
$ok = $this->_exists || ... ? $this->update() : $this->insert();
if ($ok === false) { $this->_db->rollback(); return false; }
$this->_db->commit();
```

Cross-framework parity check. Python, Ruby and Node don't have this exact failure mode - they build the write payload from declared fields only (not all public properties), and their DB adapters raise on bad SQL, which the existing `try/except` already catches. PHP was the outlier on both counts. 3 regression tests in `tests/Issue114Test.php`; PHP suite 2235 -> 2238 passing.

Also in the PHP 3.12.7-3.12.9 patch line

These shipped to PHP between 3.12.6 and this release; folded into the consolidated 3.12.10 line:

- **3.12.7** - `Request` now normalises caller-provided header keys to lowercase. Some upstream entry points (Apache+PHP-FPM custom mappings, certain proxies, hand-written test fixtures) hand headers in with original case. The constructor only looks them up by lowercase key, so without normalisation `multipart/form-data` content-type detection silently missed and the body fell through as raw bytes - a follow-up to the #135 fix.
- **3.12.8 / 3.12.9** - Router gained RFC 9110 HTTP method conformance: proper `HEAD` and `OPTIONS` handling, `405 Method Not Allowed` with an `Allow` header listing the methods a route does support.

Python / Ruby / Node

Version-only bump 3.12.6 -> 3.12.10 to realign with PHP. No behavioural changes in these three since 3.12.6.

Upgrade

Drop in. No `.env` changes, no API changes. PHP users on 3.12.9 get the `save()` fix; everyone else gets a version-number realignment.

---

v3.12.6 (2026-05-06)

Python-only fix release. PHP / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

Python - psycopg2 `%` substitution no longer trips PL/pgSQL function bodies (#40)

A migration containing a PL/pgSQL function with literal `%` characters in a `RAISE EXCEPTION` (or `format()`) call used to fail with the misleading:

> *RuntimeError: Migration failed: list index out of range*

The error message gave no hint that the `%` chars were the problem. The user-facing failure looked like a tina4 internal bug - actually psycopg2's argument-substitution system tripping on the literal percent signs.

Root cause. `PostgreSQLAdapter.execute(sql, params)` always called `cursor.execute(sql, params or [])`. psycopg2 interprets `%` as parameter placeholders WHENEVER the `params` arg is supplied - even an empty list `[]`. So a function body containing `RAISE EXCEPTION 'thing % conflicts with %', a, b` (perfectly valid PL/pgSQL) blew up because psycopg2 thought `%` was a placeholder and there were no values to substitute.

Fix. New `PostgreSQLAdapter._safe_execute(cursor, sql, params)` helper routes empty/None params through `cursor.execute(sql)` (no second arg), which makes psycopg2 skip the substitution pass entirely. Literal `%` chars flow through untouched. Applied at every `cursor.execute(...)` call site in the adapter (5 spots across `execute`, `fetch`, `fetch_one`).

Tests. 5 new unit tests in `tests/test_postgres_percent_substitution.py` pin the helper's branching. 3 live-Postgres regression tests in `tests/test_postgres_plpgsql_percent.py` exercise a real CREATE FUNCTION + trigger flow with literal `%` in the body - skipped automatically when no Postgres is reachable. Full suite: 2453 passing (was 2448).

Cross-framework parity check. PHP (`pg_query` vs `pg_query_params`) and Ruby (`exec` vs `exec_params`) already branch on params presence so they don't have this bug. Node uses `$1` placeholders not `%`, so the same class of bug doesn't apply.

Long-standing tina4-js #37 confirmed fixed

`frond.form.submit` not following 3xx redirects - fixed in frond v2.1.2 back on April 11, 2026 (`xhr.responseURL` comparison + `window.location.href` navigation). All four framework `public/js/frond.min.js` copies carry the fix. The original issue stayed open because the reporter never confirmed against the patched build.

Upgrade

Drop in. No `.env` changes, no API changes.

v3.12.5 (2026-05-06)

PHP-only bug fix release. Python / Ruby / Node ship the same version stamp for parity but carry no behavioural changes.

PHP - multipart bodies with file uploads now parse correctly (#135)

Two stacked bugs in `Tina4\Request::__construct` made `$request->body` come through as the raw multipart bytes (~11 KB blobs starting with `------WebKitFormBoundary...`) whenever the request included a file upload:

- The constructor called `$this->parseBody()` BEFORE initialising `$this->files`. Inside parseBody's multipart branch, the line `$this->files = array_merge($this->files, $parsed['files'])` read an uninitialised typed property - fatal `Error`.
- After fixing the init order, that same line tried to mutate the `readonly` `$files` property - another fatal `Error`.

Both errors got swallowed by the upstream error handler and the route handler received the raw multipart payload instead of the parsed associative array. Routes that worked fine for ordinary form posts broke the moment a file field came along.

Fix. Move `$this->files` initialisation AFTER `parseBody()` runs. parseBody stashes extracted multipart files on a new private mutable `$multipartFiles`; the constructor merges them into the readonly `$files` in a single assignment that respects the readonly contract.

4 new regression tests in `tests/Issue135Test.php` pin the constructor's contract. Full PHP suite: 2235 passing (was 2231).

Upgrade

Drop in. No `.env` changes, no API changes, no other framework changes.

v3.12.4 (2026-05-06)

Documentation-truth release. The `audit-truth.py` CI gate (introduced post-3.12.3) flagged 39 env vars referenced in docs that no framework actually read. This release closes that gap: 25 of them now exist in code, the other 14 are deleted from docs (11 hallucinations + 6 clustering vars deferred to tina4#2). Both audit gates (CLI drift + env-var drift) are now strict in CI.

25 new env vars across all 4 frameworks

Server: `TINA4_HOST`, `TINA4_SUPPRESS`, `TINA4_ENV_FILE`. Health: `TINA4_HEALTH_PATH` (default `/__health`, with `/health` kept as a legacy alias), `TINA4_TRAILING_SLASH_REDIRECT`. Sessions: `TINA4_SESSION_HTTPONLY`, `TINA4_SESSION_NAME`, `TINA4_SESSION_SECURE`. Templates: `TINA4_TEMPLATE_CACHE_TTL` (`0` = permanent). GraphQL: `TINA4_GRAPHQL_AUTO_SCHEMA`, `TINA4_GRAPHQL_ENDPOINT`. Mail: `TINA4_MAIL_IMAP_ENCRYPTION` (`tls`/`starttls`/`none`). MCP: `TINA4_MCP`, `TINA4_MCP_PORT`. Swagger: `TINA4_SWAGGER_ENABLED`, `TINA4_SWAGGER_CONTACT_EMAIL`, `TINA4_SWAGGER_LICENSE`. Database: `TINA4_DB_POOL` (env override on the existing `Database(url, pool=N)` constructor argument).

Logging - env-driven file output + rotation

Six new vars give you full control over logging without touching code:

Var	Default	What it does
`TINA4_LOG_FILE`	*(empty - stdout only)*	Path to a log file. Empty leaves you on stdout.
`TINA4_LOG_DIR`	`logs`	Directory for log files (joined with `_LOG_FILE` if relative).
`TINA4_LOG_FORMAT`	`text`	`text` or `json`. JSON mode emits one structured record per line.
`TINA4_LOG_OUTPUT`	`stdout`	`stdout`, `file`, or `both`. Strict - `stdout` means stdout only.
`TINA4_LOG_CRITICAL`	`false`	Enables a `Log.critical()` level above `error`. Off = no-op.
`TINA4_LOG_ROTATE_SIZE`	`10485760` (10 MB)	Rotate when the file exceeds this many bytes. `0` disables rotation.
`TINA4_LOG_ROTATE_KEEP`	`5`	Number of rotated files to retain (`app.log.1` ... `app.log.N`). Older ones are deleted.

Implementation uses each language's stdlib - Python's `logging.handlers.RotatingFileHandler`, Ruby's `Logger.new(path, shift_age, shift_size)`, and a roll-your-own atomic-rename pattern in PHP and Node. Zero new dependencies in any framework.

Documentation-truth CI gate now strict on both axes

The `audit-truth.py` script now blocks merges to `main` of `tina4-documentation` whenever a doc references a `tina4 <command>` or `TINA4_*` env var that doesn't exist in source. Previously CLI drift was strict; env drift was warn-only. Today both are strict.

Tests added

- Python: +53 tests in `tests/test_env_vars.py` (2395 -> 2448)
- PHP: +59 tests in `tests/EnvVarTest.php` (2172 -> 2231)
- Ruby: +51 examples in `spec/env_vars_spec.rb` (2696 -> 2747)
- Node: +59 tests in `test/envVars.test.ts` (3204 -> 3263)

Cross-framework total: 10,689 tests passing, +222 from 3.12.3.

Upgrade path

Drop in. No breaking changes - every new env var is opt-in with a sensible default. If you were setting any of the 17 deleted vars in your `.env`, the boot guard will warn (then ignore) - clean them out at your leisure.

v3.12.3 (2026-05-05)

Cross-framework parity sweep. Two minor breaking changes in the Ruby and PHP public API that bring all four frameworks onto the same shape.

Breaking changes (Ruby + PHP only)

Ruby Container - predicate now uses `?` suffix.

```
# before (3.12.2 and earlier)
Tina4::Container.has(:mailer)        # outdated

# after (3.12.3)
Tina4::Container.has?(:mailer)       # idiomatic Ruby predicate
```

This brings Ruby in line with Python (`has()`), PHP (`has()`), and Node (`has()`) while still respecting Ruby's `?`-suffix idiom for predicates returning bool. The pre-existing `resolve` -> `get` rename happened earlier; only the predicate was lagging.

ResponseCache public surface - middleware-only across all four frameworks.

The cache has always been middleware. Two of the four frameworks (PHP, Ruby) historically exposed lookup/store as public methods, which let users couple to internals. The public API is now consistent across all four: use the middleware on a route, and read stats with module-level helpers.

```
# Ruby - module-level helpers (parity with Python)
Tina4.cache_stats   # -> { hits:, misses:, size:, backend:, keys: }
Tina4.clear_cache   # flush all entries

# PHP - static methods on the class
\Tina4\Middleware\ResponseCache::cacheStats();
\Tina4\Middleware\ResponseCache::clearCache();
```

Internal methods that used to be public (`get`, `lookup`, `store`, `cache_response`) are now private. Tests that needed them retain access via `_internal*` test seams marked `@internal`.

Doc parity - CLAUDE.md and book chapter 33

- **CLAUDE.md**: every framework's "Key Method Stubs" section now covers the same surface area Python documents - Queue, QueryBuilder, Frond, Api, Background Tasks, ResponseCache, etc. PHP added 4 sections; Ruby added 5; Node added 13.
- **Book chapter 33**: env var tables are now grounded in source. Each framework's chapter 33 lists every `TINA4_*` var its source actually reads. Found and fixed several gaps - Ruby was missing `TINA4_CACHE_*`, `TINA4_QUEUE_*`, `TINA4_KAFKA_*`, `TINA4_RABBITMQ_*`, `TINA4_MONGO_*`, `TINA4_WS_BACKPLANE`, and the entire `TINA4_SESSION_VALKEY_*`

block.

Other fixes

- **Ruby** `lib/tina4/ai.rb` - subprocess output is now force-encoded to UTF-8 before `String#strip`, fixing `Encoding::CompatibilityError` that crashed 4 ai specs on systems with non-ASCII pip output.
- **Node** `test/serverParity.test.ts` - sets `TINA4_OVERRIDE_CLIENT=true` so `start()` actually runs, plus emits the `N passed, M failed` summary line the runner expects. The test was effectively a no-op before; now it's recorded properly.

Genuine gaps surfaced by the parity audit (follow-up, not blocking 3.12.3)

The chapter 33 audit flagged env vars Python documents that no other framework actually reads - Ruby/PHP/Node lack `TINA4_OPEN_BROWSER`, `TINA4_DEV_POLL_INTERVAL`, `TINA4_PUBLIC_DIR`, `TINA4_TOKEN_EXPIRES_IN` alias, plus a few framework-specific gaps (Ruby has no Mongo session backend; Node `TINA4_CSRF` defaults to `false` vs Python's `true`). Tracked for a future patch.

Upgrade path

Symptom	Fix
Ruby: `NoMethodError: undefined method 'has' for Tina4::Container`	Replace `has(:key)` with `has?(:key)`
PHP: `BadMethodCallException` calling `$cache->lookup(...)`	Use the middleware: `[ResponseCache::class, 'beforeCache']` / `[..., 'afterCache']`. Or call `_internalLookup` if you really need direct access (test code only - `@internal`).
Ruby: `NoMethodError: undefined method 'get' for ResponseCache instance`	Use `Tina4.cache_stats` / `Tina4.clear_cache` for stats. Lookup goes through the middleware.

No `.env` changes from 3.12.2.

v3.12.2 (2026-05-05)

Quality-of-life patch. Two related portability fixes - no breaking changes from 3.12.1.

Firebird URL auto-detect

Firebird is the awkward one in the stack. Every other engine has a server-side database name (`postgres://host:port/dbname`), but Firebird wants either an absolute file path on the server, a Windows drive-letter path, or an alias. The classic URI form needs a double slash to keep the leading `/` of an absolute path through the URL parser - unintuitive to anyone used to the way postgres / mysql / mssql encode the database name.

The framework now accepts five equivalent forms and normalises all of them transparently:

URL path you write	Resolved Firebird identifier
`//abs/path/db.fdb` (classic double-slash)	`/abs/path/db.fdb`
`/abs/path/db.fdb` (single-slash, intuitive)	`/abs/path/db.fdb`
`/C:/Data/db.fdb` (Windows drive letter)	`C:/Data/db.fdb`
`/C%3A/Data/db.fdb` (URL-encoded colon)	`C:/Data/db.fdb`
`/employee` (Firebird alias)	`employee`

For ops setups that keep server URL and DB location in separate config layers - or for Windows backslash paths that fight URL encoding - set `TINA4_DATABASE_FIREBIRD_PATH`. The env override wins over whatever path is in the URL.

```
TINA4_DATABASE_FIREBIRD_PATH=C:\firebird\data\app.fdb
TINA4_DATABASE_URL=firebird://SYSDBA:masterkey@localhost:3050/ignored
```

Shipped to all 4 frameworks. 11 regression tests per framework (8 unit + 3 live).

Bug fix specific to PHP - `mysqli` localhost+port quirk

PHP's `mysqli` has a long-standing quirk where `host == "localhost"` triggers a Unix socket lookup and IGNORES the port argument entirely. Connecting to `mysql://...:53306` against a Docker container fails with "No such file or directory" - `mysqli` is hunting for `/tmp/mysql.sock` instead of opening a TCP connection. `MySQLAdapter::rewriteHostForTcp()` now rewrites `localhost` to `127.0.0.1` when a non-zero port is specified, forcing the TCP code path. Bare `mysql:///db` (no port) is preserved so existing socket-based setups keep working.

Other fixes

- **chore(python):** `pyproject.toml` had drifted to `3.10.41` while `__init__.py` read `3.12.1`. Synced both to 3.12.2 so `uv build` and runtime introspection now agree.
- **chore(claude.md, all 4):** stale framework version banners in `CLAUDE.md` headers updated.

No `.env` changes from 3.12.1, no migration needed. Existing 3.12.1 installs upgrade by changing one version number.

v3.12.1 (2026-05-04)

CI-only patch - no framework code changes from 3.12.0.

- **fix(ci, all 4):** every `publish.yml` workflow now declares `permissions: contents: write` on the publish job. Without this, `softprops/action-gh-release` 403'd against the default `GITHUB_TOKEN` on repos whose default Workflow permissions setting was read-only (Ruby and Node hit this every release; PHP and Python worked by luck of repo settings). The explicit declaration makes the workflow self-sufficient.
- **chore(ci):** bumped `softprops/action-gh-release` from `@v1` (unmaintained) to `@v2`.

No `.env` changes, no API changes, no migration needed. Existing 3.12.0 installs can upgrade without touching anything else.

The version-bump itself is the test: a successful 3.12.1 release proves the workflow fix works on Ruby and Node where 3.12.0 needed manual `gh release create`.

v3.12.0 (2026-05-04)

> ***Warning: Breaking change - read before upgrading.*** *Every framework env var now uses the `TINA4_` prefix. Existing `.env` files set with `DATABASE_URL`, `SECRET`, `SMTP_HOST`, `HOST_NAME`, etc. will cause the framework to refuse to boot. Run `tina4 env --migrate` to rewrite, or follow the rename table below.*

Why this release

Tina4's env vars had grown inconsistent. Some had the `TINA4_` prefix (`TINA4_DEBUG`, `TINA4_LOCALE`, `TINA4_CACHE_BACKEND`), others didn't (`DATABASE_URL`, `SECRET`, `SMTP_HOST`). Newcomers had to guess which convention applied to which feature. Existing tools and PaaS dashboards collided with un-prefixed names like `SECRET` and `API_KEY` that other libraries also read. Documentation drifted - 91 env-var names appeared in the docs that didn't exist in any framework, and 22 framework-specific env vars in the code didn't match the names users were told to set.

This release closes all three gaps with a single hard rename. No deprecation period, no fallback chain. The framework refuses to boot if it detects a legacy name in the environment, prints a list of every var to rename, and tells you which command to run.

What changed

- **22 env vars renamed** to `TINA4_*` form. See the migration table below.
- `tina4 env --migrate` **CLI** added to all four frameworks. Reads your `.env`, rewrites it in place, leaves a `.env.bak` backup, prints a diff. Idempotent.
- **Boot-time guard** scans `os.environ` (or the language equivalent) for the 22 legacy names. If any are present, prints the rename map and exits with code 2. Bypass with `TINA4_ALLOW_LEGACY_ENV=true` for migration scripts that need both names set during transition.
- **All 4 framework books rewritten.** Chapter 33 (Environment Variables) is now a clean canonical list - every var prefixed, descriptions current, legacy names removed.
- **Doc-vs-code drift closed.** Of the 91 stale env vars previously documented, 61 were renames (corrected), 32 were never implemented (removed). The `audit-links.py` CI gate stays at 0 broken links / 0 broken anchors.
- **Frond bundle** rebuilt at v2.1.3 - `frond.min.js` footer now shows the version explicitly so users can verify what they have.

Bug fixes shipped alongside the rename

• **#38 PostgreSQL UUID-PK transaction abort** - the post-INSERT `lastval()` probe is now wrapped in a SAVEPOINT, so UUID-PK INSERTs no longer poison the outer transaction with `InFailedSqlTransaction`. Live regression test against PostgreSQL 16. (Affects all 4 frameworks where the PG adapter does this probe.)

• **#39 Landing page + template auto-routing** -

• Auto-routing now scans `src/templates/pages/` only. Partials, layouts, base.twig, errors/, components/, and `_*` files never auto-serve from a URL.

• `TINA4_TEMPLATE_ROUTING=off` kills the feature entirely.

• `src/public/index.html` auto-serves at `/` (and `/foo/` serves `src/public/foo/index.html`) - SPA hosting Just Works.

• The framework landing page only renders when `TINA4_DEBUG=true`. Production never shows it; framework version, dev-admin link, and gallery don't leak to real users.

• The malformed `HTTP/1.1 404 OK` status line is fixed - every status code now uses its canonical RFC 7231/9110 reason phrase.

• **#37 frond.form.submit redirect handling** - verified shipped at v2.1.x; `xhr.responseURL` change triggers `window.location` navigation correctly.

• **#36 Session file handler** - re-verified safeguards (lazy save, WebSocket skip, probabilistic GC, new-and-empty skip) all still in place.

Migration - every renamed var

Legacy name	New name
`DATABASE_URL`	`TINA4_DATABASE_URL`
`DATABASE_USERNAME`	`TINA4_DATABASE_USERNAME`
`DATABASE_PASSWORD`	`TINA4_DATABASE_PASSWORD`
`DB_URL`	`TINA4_DATABASE_URL` (alias dropped)
`SECRET`	`TINA4_SECRET`
`API_KEY`	`TINA4_API_KEY`
`JWT_ALGORITHM`	`TINA4_JWT_ALGORITHM`
`SMTP_HOST`	`TINA4_MAIL_HOST`
`SMTP_PORT`	`TINA4_MAIL_PORT`
`SMTP_USERNAME`	`TINA4_MAIL_USERNAME`
`SMTP_PASSWORD`	`TINA4_MAIL_PASSWORD`
`SMTP_FROM`	`TINA4_MAIL_FROM`
`SMTP_FROM_NAME`	`TINA4_MAIL_FROM_NAME`
`IMAP_HOST`	`TINA4_MAIL_IMAP_HOST`
`IMAP_PORT`	`TINA4_MAIL_IMAP_PORT`
`IMAP_USER`	`TINA4_MAIL_IMAP_USERNAME`
`IMAP_PASS`	`TINA4_MAIL_IMAP_PASSWORD`
`HOST_NAME`	`TINA4_HOST_NAME`
`SWAGGER_TITLE`	`TINA4_SWAGGER_TITLE`
`SWAGGER_DESCRIPTION`	`TINA4_SWAGGER_DESCRIPTION`
`SWAGGER_VERSION`	`TINA4_SWAGGER_VERSION`
`ORM_PLURAL_TABLE_NAMES`	`TINA4_ORM_PLURAL_TABLE_NAMES`

Names that stay un-prefixed (not framework config)

`PORT`, `HOST`, `NODE_ENV`, `RACK_ENV`, `RUBY_ENV`, `ENVIRONMENT` - these are runtime / PaaS conventions, not framework config. Heroku, Railway, Vercel, and friends set them; we keep reading them.

How to upgrade

- **Backup your `.env`:** `cp .env .env.bak.pre-v3.12`

- **Run the migration:** `tina4 env --migrate` - rewrites your `.env` in place.
- **Update PaaS dashboards:** Heroku, Railway, Vercel, Render, Fly.io etc - rename the same vars in your provider's env-var UI.
- **Restart your app.** The boot guard verifies nothing legacy remains.

If your app uses `SECRET`, `DATABASE_URL`, or any other listed name in places besides `.env` (e.g. your CI pipeline's `env:` blocks), update those too - the boot guard checks `os.environ`, not just `.env`.

Parity

All 4 frameworks aligned at **3.12.0**:

- tina4-python 3.11.32 -> 3.12.0
- tina4-php 3.11.32 -> 3.12.0
- tina4-ruby 3.11.32 -> 3.12.0
- tina4-nodejs 3.11.32 -> 3.12.0

Coordinated release across PyPI, Packagist, RubyGems, npm.

v3.11.32 (2026-04-25)

Critical fix - pool + transactions are now actually atomic. Plus a coordinated parity release that aligns all four frameworks at the same version after months of drift.

Before this release, creating a `Database` with `pool > 0` silently broke transactions. The pool's round-robin checkout rotated to a different adapter on every call - so `start_transaction()` pinned its flag on adapter A, the executes autocommitted on adapters B and C, and the final `commit()` / `rollback()` landed on adapter D, which had nothing to commit. Result: `rollback()` was a no-op, writes leaked through, and no error or log surfaced the problem.

The fix pins one adapter to the calling context for the lifetime of a transaction. Each language uses its own primitive:

- **Python** - `threading.local()` on the `Database` instance
- **Ruby** - `Thread.current[:tina4_pinned_adapter_<obj_id>]`
- **Node.js** - `AsyncLocalStorage` from `node:async_hooks` (async-safe across overlapping awaits)
- **PHP** - per-instance property (PHP-FPM is one process per request; threading.local is unnecessary)

While pinned, every database call routes to the same adapter. `commit()` and `rollback()` release the pin so subsequent calls round-robin again.

- **fix (database / all 4):** adapter pinning across transaction scope in `Database._get_adapter()` (and language equivalents). Every backend is affected - SQLite, PostgreSQL, MySQL, MSSQL, Firebird. Firebird exposed it loudest because of its honest "commit-empty-txn is a real no-op" semantics; the others mostly hid the bug behind eager autocommits but still lost rollback atomicity.

• **tests (all 4):** new regression suite - three INSERTs followed by `rollback()` under `pool=4` now leaves zero rows (was leaking three). Three INSERTs followed by `commit()` persists exactly three. Pin-release after commit/rollback verified. `pool=0` regression test added so single-connection mode stays unaffected.

• **parity / version alignment:** all 4 frameworks bumped to 3.11.32 - closes the cross-framework version drift that had built up (PHP at 3.11.31, Python at 3.11.24, Ruby and Node at 3.11.19). A single coordinated release across all four registries: PyPI, Packagist, RubyGems, npm.

No migration needed. Code using `pool=0` (the default for every adapter except where explicitly raised) is unaffected. Code using `pool>0` will now actually honour transactions instead of silently dropping them.

If you've been seeing intermittent "writes vanished" or "rollback didn't help" reports on a pooled `Database`, this release is the cause and the cure.

v3.11.13 (2026-04-16)

Issue-driven release. Everything reported in the open tina4-book issues either was fixed in this version or is already fixed in 3.11.12; this release consolidates the remaining bits and corrects documentation drift.

• **feat (router / all 4):** Explicit typed-parameter system shared across Python, PHP, Ruby, Node. Adds `alpha`, `alnum`, `slug`, `uuid`, and explicit `string` types in addition to the existing `int`/`integer`, `float`/`number`, `path`/`.*`. **Unknown type names now throw at registration** - `{name:str}`, `{id:inetger}`, etc. raise with a clear message listing the valid types instead of silently falling through to the default matcher. Fixes tina4-book#125. +45 new tests across the four suites.

• **fix (gallery / python+php+ruby):** Gallery Try-It / View buttons now open the deployed example in a new tab (`window.open(url, '_blank')`) instead of navigating away from the gallery home. Fixes tina4-book#115.

• **fix (ruby gemspec):** `sqlite3` promoted from `add_development_dependency` to `add_dependency`. Matches the "zero-config SQLite on first run" promise. Fixes tina4-book#100.

• **docs (tina4-book):** PHP Chapter 2 updated - correct port (7145), `->noAuth()` on write-method examples, and an explicit callout explaining the secure-by-default policy for POST/PUT/PATCH/DELETE. Addresses tina4-book#87, #94, #123.

• **docs (tina4-book):** Python `@template` decorator ordering corrected (must sit BELOW the route decorator) in book chapters 04 and 10; Python `request->query` vs `request->params` distinction in PHP chapter 1.

• **tests (python):** Session-handler tests updated to reflect the real default TTL of 3600s (were stale at 1800s).

• **verified already fixed in earlier 3.11.x releases** - closed comments posted on all of these:

• #79 dotted numeric index (`{{ items.0.name }}`)

• #80 `truncate` filter

- #82 `{{ parent() }}` / `{{ super() }}` across all 4 frameworks
- #83 Ruby dashboard - WEBrick is runtime dep
- #89 `load_dotenv` rename, `DatabaseResult` methods, SQLite WAL locking
- #91 Ruby `request.params` symbol + string keys via `IndifferentHash`
- #93 Ruby `/docs/*` and bare `/*` wildcard routes
- #97 Frond ternary operator
- **parity:** All 4 frameworks bumped to 3.11.13.

v3.11.12 (2026-04-16)

Breaking: `sqlite:///X` URLs are now relative to the project root (cwd), matching the documented convention. For absolute paths use four slashes (`sqlite:////abs/path.db`) or a Windows drive letter (`sqlite:///C:/Users/app.db`).

Before this release, `TINA4_DATABASE_URL=sqlite:///data/app.db` was interpreted differently by every framework. Python/Node/Ruby tried to open `/data/app.db` (absolute) which crashed on macOS with `OSError: [Errno 30] Read-only file system: '/data'`. PHP did the same under the hood. All four frameworks now agree: three slashes = relative, four slashes = absolute.

- **fix (all 4):** `sqlite:///X` resolves under cwd; parent directory auto-created only when inside cwd. Absolute paths are trusted and never mkdir'd at root.
- **fix (python):** `_ensure_folders` no longer creates a bogus `src/migrations/` directory. The migration runner always looks at `migrations/` at the project root - there is only one correct location.
- **parity (php, ruby, node):** Same `sqlite:///X` parsing as Python. Dedicated `resolve_path` / `resolveSqlitePath` helpers in each framework so adapters consistently handle `:memory:`, `./` forms, Windows drive letters.
- **tests:** 9 new Python tests in `TestSQLiteConnectionPath` + `TestProjectFolders`. 4 new PHP tests in `DatabaseUrlTest` covering relative/absolute/Windows/bruce-regression. 6 new Ruby specs in `database_drivers_spec.rb :: SqliteDriver.resolve_path`. Node URL tests expanded in `database.test.ts` with the full relative/absolute/Windows/:memory: matrix.
- **parity:** All 4 frameworks bumped to 3.11.12.

Migration note: If your `.env` has `TINA4_DATABASE_URL=sqlite:///data/app.db`, it will now create `./data/app.db` in the project root (which is what most users actually want). If you genuinely want an absolute path, change to `sqlite:////data/app.db` (four slashes).

v3.11.11 (2026-04-16)

• **fix (python ORM):** `Field.validate` no longer re-coerces values that are already the correct type. Previously, any PostgreSQL/MSSQL read of a row containing a `DateTimeField` crashed because `datetime(datetime_instance)` raises `TypeError`. The fix accepts native driver types (`datetime`, `bytes`, `int`, `bool`, `float`, `str`) without re-wrapping, and parses ISO-8601 strings into `datetime` for SQLite. See `tina4-python/plan/orm-field-validate-native-types.md`.

• **fix (python ORM):** `BooleanField` vs `IntegerField` ordering handled explicitly. `BooleanField(1)` still coerces to `True`, `IntegerField(True)` still coerces to `1`; no regression for either direction (bool is a subclass of int in Python).

• **tests (python):** 10 new `TestFieldsNativeTypes` cases covering datetime/int/bool/float/bytes/string/ForeignKey round-trips.

• **tests (parity):** Regression-guard "datetime round-trip on read path" tests added to PHP (`ORMV3Test`), Ruby (`orm_spec`) and Node.js (`orm.test.ts`) so an equivalent bug can't creep in there later.

• **parity:** All 4 frameworks bumped to 3.11.11.

v3.11.10 (2026-04-15)

• **fix (php):** Hot-reload loop - DevAdmin's polling fallback used `mt=0` as the baseline, so the first poll after every page load triggered `location.reload()`, which reset `mt=0` again. Loop now initialises the baseline on the first poll.

• **fix (php):** Reload sentinel removed - PHP was the only framework recursively walking `src/` and touching `src/.reload_sentinel` on every reload POST. The sentinel lived inside the Rust CLI's watched tree and fed back into the watcher, triggering a second loop. Replaced with the same in-memory counter used by Python/Ruby/Node.

• **fix (php):** Polling no longer starts more than once when the WebSocket reconnect retry budget is exhausted (added a `pollStarted` guard).

• **feat (parity):** `GET /__dev/api/queue/topics` and `GET /__dev/api/queue/dead-letters` added to PHP, Ruby and Node (previously only in Python). PHP queue endpoints now read from the real `Tina4\Queue` backend instead of returning stubs.

• **feat (devadmin):** Refreshed `tina4-dev-admin.js` bundle (87.8 KB) across all 4 frameworks - adds the topic selector dropdown, inline payload expand/copy, and corrected version display.

• **tests:** 4-way parity tests for hot-reload: mtime starts at 0, POST /__dev/api/reload bumps the counter, no sentinel file is written to disk, mtime is monotonic across successive reloads. Mirrored in `tina4-php/tests/DevAdminTest.php`, `tina4-python/tests/test_dev_admin.py`, `tina4-ruby/spec/dev_admin_spec.rb`, `tina4-nodejs/test/devAdmin.test.ts`.

• **parity:** All 4 frameworks bumped to 3.11.10.

---

v3.11.9 (2026-04-15)

Catch-up release covering v3.11.0 -> v3.11.9 across all 4 frameworks.

- **feat (websocket):** Full WebSocket parity across Python/PHP/Node/Ruby - `get_client_rooms()` / `getClientRooms()`, `route()` usable as decorator or direct handler registration, matching room/broadcast semantics, plus new parity tests on all 4.
- **feat (graphql):** Input validation and field-level `@auth` directives with context threading.
- **feat (graphql):** Auto-discovery of schemas; removed legacy DevAdmin HTML/JS in favour of the new UI.
- **feat (devadmin - Python):** Queue tab with topic selector, dead-letter listing and replay endpoints, inline payload expand/copy, version display.
- **feat (cli):** Rust CLI now owns file watching - frameworks receive `POST /__dev/api/reload` and internal watchers are disabled when launched by the Rust CLI (`--managed`).
- **fix (cli):** `parseFlags` / `parse_flags` / `parseCliArgs` no longer swallow `host:port` or positional args after boolean flags.
- **fix (scss):** SCSS recompilation loop fixed; output path corrected to `src/public/css/` to match CLI and static serving.
- **fix (frond - Python):** Numeric dotted index for lists (`items.0.name`) now resolves correctly.
- **fix (router - Ruby):** Bare `/*` wildcard capture exposed under `"*"` key for parity.
- **fix (orm - PHP):** Three data-sync bugs fixed: `load()` double-fill, `getPrimaryKeyValue`, `save()` ID sync.
- **fix (graphql):** `from_orm` / `fromOrm` list resolver used `select(skip=)` instead of `all(offset=)`.
- **fix (metrics):** Windows backslash paths normalised to forward slashes.
- **fix (app - PHP):** No longer crashes on notices/deprecations in loaded files; `run()` now prints the banner when starting the server directly.
- **chore:** Example demo store ships with the repo; Windows-friendly setup; `.env.example` and setup scripts added.
- **parity:** All 4 frameworks bumped to 3.11.9. PHP aligned to the 3.x tag scheme on `v3`.

v3.10.99 (2026-04-12)

- **breaking:** `auto_map` now defaults to `true` - ORM models automatically map between camelCase properties and snake_case DB columns. Set `self.auto_map = false` on your model class to restore the old behaviour.
- **feat:** `to_h(case:)` parameter - pass `case: 'camel'` to get camelCase keys (for JSON APIs) or `case: 'snake'` (default) for snake_case keys matching DB columns. All aliases (`to_dict`, `to_hash`, `to_assoc`, `to_object`) support the parameter.
- **feat:** Frond `replace` filter now accepts Hash args - `{{ v|replace({"T": " ", "-": "/"}) }}` for multiple substitutions in one call.

• **tests:** 6 new parity tests covering `to_h(case:)`, `auto_map` default, `replace` filter (Hash + positional), and `ServiceRunner` registration. 2,519 tests passing.

• **parity:** All features shipped identically across Python, PHP, Ruby, Node.js.

v3.10.97 (2026-04-11)

• **fix:** frond.form.submit redirect handling - XHR follows 3xx redirects transparently; fixed by detecting `xhr.responseURL` mismatch and navigating instead.

• **dep:** Updated frond.min.js to v2.1.2.

• **parity:** All 4 frameworks bumped to 3.10.97.

v3.10.93 (2026-04-11)

• **fix:** Frond bracket depth tracking in `find_outside_quotes` - expressions like `arr[i % 2]` no longer treated as top-level arithmetic.

• **fix:** Frond subscript expression evaluation - bracket content uses `eval_expr()` instead of simple variable lookup, enabling `arr[loop.index0 % 2]`.

• **fix:** Frond slice with variable bounds - `items[start:end]` evaluates bounds through `eval_expr()`.

• **docs:** Developer skills updated - Metrics Dashboard guidance, Frond Template Parity rules, `@noauth` security warnings.

• **parity:** All Frond fixes applied identically across Python, PHP, Ruby, Node.js. 2,513 tests passing.

v3.10.92 (2026-04-10)

• **breaking:** Rename `ErrorOverlay` methods - `render` -> `render_error_overlay`, `render_production` -> `render_production_error`, `debug_mode?` -> `is_debug_mode`.

• **feat:** Add `Server.handle(env)` for cross-framework parity.

• **breaking:** Rename `WebSocketBackplane.create` -> `WebSocketBackplane.create_backplane`.

• **feat:** Add `ScssCompiler.compile`, `add_import_path`, `set_variable` methods.

• **feat:** Add `DevAdmin.register` method.

• **parity:** 44/44 cross-framework features green. 2,487 tests passing.

v3.10.91 (2026-04-10)

• **feat:** Add parity methods - `Response.send` params, `Middleware.check`/`is_preflight`, AI/Log aliases, MCP optional router.

• **breaking:** Rename `from()` -> `from_table()`, `error_envelope` -> `error_response`, remove aliases.

v3.10.90 (2026-04-09)

• **docs:** Chapter 4 (Templates) - new "Dumping Values for Debugging" section covering both `{{ x|dump }}` and `{{ dump(x) }}` forms, their shared `<pre>value.inspect</pre>` output, and the `TINA4_DEBUG=true` production gate. Filter table entry updated to reference the new section.

• **docs:** `plan/parity/parity-template.md` updated with a cross-framework dump helper comparison table and marks dump parity as confirmed across all 4 frameworks at v3.10.89.

• **chore:** Version sync release - brings all 4 frameworks to the same patch version (3.10.90) so downstream users can upgrade PHP/Python/Ruby/Node.js in lockstep without hunting version mismatches.

v3.10.89 (2026-04-09)

• **feat:** `{{ dump(value) }}` global function form added to Frond alongside the existing `{{ value|dump }}` filter. Both call a single `Tina4::Frond.render_dump` helper and produce identical output (`<pre>value.inspect</pre>` HTML-escaped).

• **security:** Dump is now **gated on** `TINA4_DEBUG=true`. In production (env var unset or `false`) both the filter and function silently return an empty `SafeString`. This prevents accidental leaks of internal state, object shapes, and sensitive values into rendered HTML when a developer leaves a `{{ dump(x) }}` call in a template.

• **test:** 3 new `spec/frond_spec.rb` examples covering debug-mode output, production silencing, function/filter parity, and function-form production silencing.

v3.10.86 (2026-04-09)

• **feat:** `foreign_key_field` DSL auto-wires both sides of a foreign key relationship. Declaring `foreign_key_field :user_id, references: User` registers the integer column, calls `belongs_to :user` on the declaring class, and calls `has_many :posts` on the referenced class. Supports `related_name:` for custom has-many names and deferred wiring via a module-level registry so the referenced class can be defined either before or after the declaring one.

• **feat:** Cross-framework parity - same FK auto-wiring semantics now available in Python (`ForeignKeyField`), PHP (`$foreignKeys`), and Node.js (`type: "foreignKey"`)

• **docs:** Chapter 6 (ORM) updated with a new "foreign*key*field - Auto-Wired Relationships" section

v3.10.85 (2026-04-09)

• Version bump for parity with Python and PHP releases

v3.10.84 (2026-04-09)

• **fix:** Router/middleware was setting `request.user` / `request.auth` / auth payload to `true` (boolean) instead of the actual JWT payload after `valid_token?` was changed to return bool - any code reading `request.user["sub"]` etc. would have failed silently or crashed

• **fix:** CSRF middleware was not correctly rejecting invalid tokens (nil check on bool result always passed)

• **add:** Headless routing auth payload integration tests to prevent regression

v3.10.83 (2026-04-08)

• **feat:** WebSocket rooms - `join_room`, `leave_room`, `broadcast_to_room`, `room_count`, `get_room_connections`

• **feat:** Queue signature parity - instance-scoped `push`/`pop`/`retry`, no topic params on public methods

• **feat:** Auth cleanup - canonical `getToken`/`validToken` methods

• Full parity across Python, PHP, Ruby, Node.js

---

v3.10.70 (2026-04-06)

• **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framework handles chunked transfer encoding, keep-alive, and `text/event-stream` content type

• **New:** Chapter 24 added to documentation: Server-Sent Events

• Feature count: 45 (was 44)

• Full parity across Python, PHP, Ruby, Node.js

Tina4 Ruby follows semantic versioning. The major version (3) marks the initial Ruby launch - Tina4 Ruby is new in the v3 line, alongside Tina4 for Node.js. Minor versions (3.1, 3.2, etc.) introduce features and non-breaking API additions. Patch versions carry bug fixes and small improvements.

This chapter covers every v3 release from the initial launch through the current stable line. Each section groups releases by minor version, highlights the changes that affect your code, and shows migration steps for anything that breaks.

---

v3.10.68 (2026-04-03) - Full Parity Release

• **100% API parity** across Python, PHP, Ruby, Node.js - 30+ issues fixed

• **ORM:** save() returns self/false, arrays not tuples, toDict/toAssoc, scope registers method, where()/all() on Node, count() on PHP

• **Auth:** expires*in minutes, PBKDF2 260k, env TINA4*SECRET fallback, API key fallback

• **Session:** dual-mode flash(), get_flash, cookieHeader, getSessionId

• **Database:** execute() bool/DatabaseResult, get*last*id/get_error, getColumns, cacheStats

• **Request/Response:** files dict, query, cookies, contentType, xml(), callable

- **Queue:** consume() poll_interval
- **WebSocket:** event naming, connection properties
- **GraphQL:** schema_sdl() + introspect() on all 4
- **Events:** emitAsync() on all 4
- **i18n:** zero-dep YAML support

v3.10.67 (2026-04-03)

- **load() is now an instance method** - `model.load(sql, params)` calls select_one internally, populates the instance, returns `true`/`false`. Use `find(id)` for PK lookups
- **api.upload()** added to tina4-js - sends FormData with Bearer token auth for multipart file uploads
- **ORM CLAUDE.md rewrite** - all method stubs now match actual API signatures
- **File upload docs** - `request.files` format documented in CLAUDE.md

v3.10.66 (2026-04-03)

- **Metrics file detail fix** - clicking bubbles in framework scanning mode now resolves paths correctly via scan root tracking

v3.10.65 (2026-04-03)

- **Metrics 3-stage test detection** - filename, path, and content matching
- **Metrics framework mode** - scans framework source with correct relative paths
- **tina4 console** - interactive REPL with framework loaded
- **tina4 env** - interactive environment configuration
- **Brand** - "TINA4 - The Intelligent Native Application 4ramework"
- **Quick references** - 36 sections, DotEnv API documented
- **37 chapters** - 7 new (Events, Localization, Logging, API Client, WSDL/SOAP, DI Container, Service Runner)
- **MongoDB + ODBC adapters** across all 4 frameworks
- **Pagination standardized** - limit/offset primary, merged dual-key response
- **Port kill-and-take-over** on startup

---

v3.10.60 (2026-04-03)

- **tina4 console** - already existed, now matches Python/PHP/Node API
- **tina4 env** - interactive environment configuration
- **Brand update** - "TINA4 - The Intelligent Native Application 4ramework"

- **Imperative relationships** - query*has*one/many/belongs_to for ad-hoc queries
- **Port kill-and-take-over** - default port always reclaimed
- **MongoDB adapter** (mongo gem), **ODBC adapter** (ruby-odbc gem)
- **Pagination standardized** - limit/offset primary, merged dual-key response
- **CORS fix** - returns empty string when origin not allowed

---

v3.10.57 (2026-04-02)

- **MongoDB adapter** - `Database.new("mongodb://host:port/db")`, requires `gem install mongo`
- **ODBC adapter** - `Database.new("odbc:///DSN=MyDSN")`, requires `gem install ruby-odbc`
- **Imperative relationships** - `query_has_one`/`query_has_many`/`query_belongs_to`
- **Pagination standardized** - limit/offset primary, merged dual-key to_paginate response
- **Test port at +1000** - user testing port (e.g. 8147) stable, no hot-reload
- **CORS fix** - returns empty string when origin not allowed
- **ORM TINA4*DATABASE*URL discovery** - auto-connect from env
- **108 features at 100% parity**, 2,333 tests

---

v3.10.54 (2026-04-02)

- **Auto AI dev port** - second WEBrick on port+1 with no-reload when TINA4_DEBUG=true
- **TINA4*NO*RELOAD** env var + --no-reload CLI flag
- **CORS fix** - returns empty string when origin not allowed (not *)
- **ORM TINA4*DATABASE*URL discovery** - auto-connect from env
- **QueryBuilder docs** - added to ORM chapter

---

v3.10.48 - April 2, 2026

Bug Fixes

Puma requires `--production` flag - Puma no longer auto-selected when `TINA4_DEBUG=false`. Use `tina4ruby serve --production` to enable Puma. Added FakeData (46), Gallery (16), and DevReload (37) tests.

---

v3.10.46 - April 1, 2026

Test Coverage

344 new tests added across cache (56), ORM (19), Frond (28), database drivers (85), auth (21), SCSS (10), dotenv (30), queue backends (10), migration (10), session handlers (11), router (14), log (13), CSRF middleware (17). Fixed session handler DB key bug (symbol vs string). Ruby now at 2,274 tests with full parity across all 49 core areas.

---

v3.10.45 - April 1, 2026

Notes

Version bump for parity with PHP CLI serve fix. No Ruby-specific changes.

---

v3.10.44 - April 1, 2026

New Features

Database tab redesign - Split-screen layout with tables navigation on the left and query editor + results on the right. Click-to-select table highlighting.

Copy CSV / Copy JSON - Copy query results to clipboard in CSV or JSON format.

Paste data - Modal for pasting JSON arrays or CSV/tab-separated data. Auto-generates INSERT statements targeting the selected table, or prompts for a new table name with CREATE TABLE generation. SQL input passes through unchanged.

Multi-statement execution - Query runner handles batched SQL statements in a transaction.

Database badge on load - Table count shows immediately without clicking the Database tab.

Star wiggle animation - Empty star (*) on the landing page with delayed wiggle animation at random intervals.

Bug Fixes

Default port - Ruby default port set to 7147 (PHP=7145, Python=7146, Ruby=7147, Node=7148).

SQLite LIMIT fix - Prevents double-LIMIT errors in the database browser.

browseTable quote escaping - Fixed table name click handlers.

ORM table name pluralization - Fixed default table name resolution. Table names are now pluralized by default (adding "s" suffix), only skipping when `TINA4_ORM_PLURAL_TABLE_NAMES` is explicitly set to false.

QueryBuilder closed-connection detection - `ensure_db!` now checks if the resolved database connection is still open, raising a proper error instead of crashing with `ArgumentError: prepare called on a closed database`.

Metrics directory validation - `quick_metrics` and `full_analysis` now check directory existence before `_resolve_root` fallback, so missing-directory errors are raised correctly.

Test Coverage

88 new tests added (DevMailbox 40, Static files 18, CLI scaffolding 30), plus 13 v3.10.44 feature specs and 60 pre-existing ORM/metrics bug fixes. 1,913 tests passing, 0 failures.

---

v3.10.40 - April 1, 2026

Bug Fixes

Dev overlay version check - Fixed misleading "You are up to date" message when running a version ahead of what's published on RubyGems. The overlay now shows a purple "ahead of RubyGems" message. Also added a breaking changes warning (red banner with changelog link) when a major or minor version update is available.

---

v3.10.39 - April 1, 2026

New Features

`Container.singleton(name, &block)` - Register a memoized factory. The block is called once on first `resolve()` and the same instance is returned on all subsequent calls. `register()` with a block is now always transient (new instance per call), matching Python's behavior.

```
Tina4::Container.singleton(:db) { Tina4::Database.new(ENV["TINA4_DATABASE_URL"]) }
db1 = Tina4::Container.resolve(:db)  # creates instance
db2 = Tina4::Container.resolve(:db)  # same instance
```

`Router.match(method, path)` - primary route lookup (replaces `find_route`; consistent with Python, PHP, Node.js). `Router.add(method, path, handler)` - primary imperative registration (replaces `add_route`; all convenience methods delegate to this).

`Router.get_routes` **and** `Router.list_routes` - explicit listing methods (remove ambiguous `routes` alias).

AI installer - `ai_spec.rb` and smoke tests updated to reflect the menu-based API (`installed?`, `install_selected`, `install_all`, `generate_context`).

---

v3.10.38 -- April 1, 2026

Code Metrics & Bubble Chart

The dev dashboard (`/__dev`) now includes a **Code Metrics** tab with a PHPMetrics-style bubble chart visualization. Files appear as animated bubbles sized by LOC and colored by maintainability index. Click any bubble to drill down into per-function cyclomatic complexity.

The metrics engine uses `Ripper` (Ruby stdlib) for zero-dependency static analysis covering cyclomatic complexity, Halstead volume, maintainability index, coupling, and violation detection. File analysis is sorted worst-first. Results are cached for 60 seconds.

AI Context Installer

`tina4ruby ai` now presents a simple numbered menu instead of auto-detection. Select tools by number, comma-separated or `all`. Already-installed tools show green. Generated context includes the full skills table.

Dashboard Improvements

Full-width layout, sticky header/tabs, full-screen overlay.

Cleanup

Removed `demo/` directory. Removed old `plan/` spec documents, replaced with `PARITY.md` and `TESTS.md`. Central parity matrix added to tina4-book.

---

v3.10.x -- Previous Releases

Released: March 28 -- 30, 2026

The v3.10 line is the most active release series. It delivered Auto-CRUD, ORM transaction safety, Frond template engine hardening, and full cross-language parity with the Python, PHP, and Node.js implementations.

v3.10.29 -- Version Parity (March 30)

Version parity release. All four Tina4 frameworks now share the same version number and feature set.

v3.10.27 -- Frond Macro HTML Escaping Fix (March 30)

Bug fix: Macro output was HTML-escaped when used inside `{{ }}` expressions. Characters like `<`, `>`, and `"` rendered as `<`, `>`, `"` instead of raw HTML. Nested macro calls double-escaped.

```
# BEFORE (broken): macro output escaped

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# Template: {{ my_macro() }}

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
```

```
- Full parity across Python, PHP, Ruby, Node.js

# Rendered: &lt;div class=&quot;card&quot;&gt;...&lt;/div&gt;

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js


# AFTER (fixed): macro output treated as safe HTML

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# Template: {{ my_macro() }}

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# Rendered: <div class="card">...</div>

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

v3.10.25 -- ORM Transaction Fix (March 30)

Bug fix: ORM `save` and `delete` called `commit` without an active transaction on SQLite. This raised `cannot commit -- no transaction is active` errors.

```
# BEFORE (broken):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

user = User.new(name: "Alice")
user.save  # => RuntimeError: cannot commit

# AFTER (fixed): save/delete wrap operations in a transaction block

## v3.10.70 (2026-04-06)
```

```
- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

user = User.new(name: "Alice")
user.save  # => works on all database engines
```

v3.10.22 -- Unique Form Tokens (March 30)

Form tokens now include a nonce in the JWT payload. Each token is unique per form render, which prevents replay attacks.

```
# In your Frond template:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

<input type="hidden" name="formToken" value="{{ formTokenValue() }}">
```

v3.10.18 -- Frond Ternary Parser Fix (March 29)

Bug fix: The Frond template ternary/inline-if parser failed on quoted strings containing special characters.

```
# BEFORE (broken):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# {{ status == "active" ? "Yes" : "No" }}  =>  parse error

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js


# AFTER (fixed):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# {{ status == "active" ? "Yes" : "No" }}  =>  "Yes"

## v3.10.70 (2026-04-06)
```

```
- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

v3.10.16 -- Template Filters: to*json*, *j*sescape (March 28)

Three new Frond template filters for working with data in JavaScript contexts.

```
# Convert a Ruby hash to JSON inside a template:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

<script>
  const data = {{ user|to_json }};
  const name = "{{ user.name|js_escape }}";
</script>
```

v3.10.15 -- Replace Filter Backslash Fix (March 28)

Bug fix: The `|replace` filter mishandled backslash characters in replacement strings.

```
{# Before (broken) - backslash produced corrupted output #}
{{ "hello\\world"|replace("\\\\", "/") }}
{# rendered: helo/world (ate a character) #}

{# After (fixed) - backslash escaping works correctly #}
{{ "hello\\world"|replace("\\\\", "/") }}
{# renders: hello/world #}
```

v3.10.14 -- get*next*id() (March 28)

Pre-generate the next primary key before inserting a record. The method detects your database engine and uses the correct sequence or auto-increment mechanism.

```
next_id = User.get_next_id
user = User.new(id: next_id, name: "Alice")
user.save
```

v3.10.13 -- ORM Auto-Commit on Write (March 28)

Write operations (`save`, `delete`) now auto-commit by default. No more forgotten `commit` calls leaving data uncommitted.

v3.10.12 -- Session GC and NATS Backplane (March 28)

- Session garbage collection runs on a configurable interval
- NATS added as a WebSocket backplane option alongside Redis

v3.10.11 -- Frond Variable Key Access (March 28)

Bug fix: Accessing a hash value with a variable key (`dict[variable_key]`) failed in Frond templates.

```
# BEFORE (broken):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# {% set key = "name" %}

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# {{ user[key] }}  =>  empty

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js


# AFTER (fixed):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# {% set key = "name" %}

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

# {{ user[key] }}  =>  "Alice"

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

v3.10.10 -- Firebird Migration Runner Fixes (March 28)

Firebird migrations now use generators and `VARCHAR` instead of `AUTOINCREMENT` and `TEXT`. The migration tracking table uses a proper Firebird sequence (~~`GEN_TINA4_MIGRATION_ID`~~).

v3.10.6 -- WSDL/SOAP Rewrite (March 28)

Complete rewrite of the WSDL/SOAP module. Frond templates now support dotted function names in expressions.

v3.10.5 -- Frond Quote-Aware Operator Matching (March 28)

Bug fix: Operators inside quoted strings were incorrectly parsed as expression operators. The Frond engine now respects quote boundaries.

v3.10.4 -- Auto-CRUD REST Endpoint Generator (March 28)

Generate a complete CRUD interface from a single method call. The generator creates searchable, sortable, paginated HTML tables with create/edit/delete modals, plus REST API routes for POST, PUT, and DELETE.

```
Tina4::Router.get("/admin/users") do |request, response|
  Tina4::CRUD.to_crud(request, model: User, fields: [:name, :email, :role])
end
```

v3.10.2 -- Frond Hash Method Calls (March 28)

Frond templates can now call methods on Hash and object values inside expressions.

v3.10.1 -- autoMap and Case Conversion (March 28)

- `auto_map` class attribute added to ORM for cross-language API parity (no-op in Ruby since `snake_case` is native)
- `Tina4.snake_to_camel("my_field")` returns `"myField"`
- `Tina4.camel_to_snake("myField")` returns `"my_field"`

v3.10.0 -- Optimized For-Loops (March 28)

The Frond template engine rewrote its for-loop renderer. Templates with large iteration counts render faster.

---

v3.9.x

Released: March 26 -- 27, 2026

v3.9.0 -- QueryBuilder, Sessions, Path Injection (March 26)

Three features arrived together.

QueryBuilder. A fluent SQL builder that integrates with the ORM.

```
# Through the ORM:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

```
admins = User.query
  .where("role = ?", ["admin"])
  .order_by("name")
  .limit(10)
  .get

# Standalone:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

rows = Tina4::QueryBuilder.from("users")
  .where("active = ?", [true])
  .select("name", "email")
  .get
```

The builder supports `where`, `or_where`, `join`, `left_join`, `group_by`, `having`, `order_by`, `limit`, `first`, `count`, `exists`, and `to_sql`.

Path parameter injection. Route handlers receive path parameters as named arguments.

```
Tina4::Router.get("/users/{id:int}") do |request, response, id|
  user = User.find(id)
  response.json(user.to_hash)
end
```

Auto-start sessions. Every route handler has access to `request.session` with zero configuration. The session API includes `get`, `set`, `delete`, `has`, `clear`, `destroy`, `save`, `regenerate`, `flash`, `get_flash`, and `all`.

v3.9.1 -- Security Defaults (March 27)

Breaking change: POST, PUT, PATCH, and DELETE routes now require authentication by default.

```
# BEFORE (v3.8.x): all routes open

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Tina4::Router.post("/api/users") do |request, response|
  # anyone could call this
end

# AFTER (v3.9.1): unauthenticated requests get 401

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
```

```
- Full parity across Python, PHP, Ruby, Node.js

# To allow public access, add .public:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Tina4::Router.post("/api/users").public do |request, response|
  # open to all
end
```

This release also added:

- CSRF middleware with session-bound form tokens
- Standardized environment variables for CORS headers, session TTL, token limits
- Queue parity: `push` with priority/delay, `size(status)`, `message.retry`

v3.9.2 -- NoSQL QueryBuilder, WebSocket Backplane (March 27)

- QueryBuilder works with MongoDB
- WebSocket backplane support for multi-process deployments
- `SameSite=Lax` set as the default cookie policy

---

v3.8.x

Released: March 25 -- 26, 2026

v3.8.0 -- Base64 Filters, Template Cache (March 25)

- `base64encode` and `base64decode` filters in Frond templates
- Production template cache: single filesystem scan at startup, O(1) lookups after

v3.8.1 -- Security Headers Middleware (March 25)

A built-in middleware that sets `X-Frame-Options`, `Strict-Transport-Security`, `Content-Security-Policy`, and `X-Content-Type-Options` on every response.


```
# In your .env:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```


```
TINA4_MAX_UPLOAD_SIZE=10485760  # 10 MB (default)
```

Upload size limits and input validation also landed in this release.

v3.8.2 -- Connection Pooling (March 26)

Database connections now pool. Pass `pool: N` to the constructor for round-robin, mutex-protected pooling.

```
db = Tina4::Database.new("sqlite://data.db", pool: 5)
```

v3.8.3 -- Claude Code Commands (March 26)

Seventeen `.claude/commands/` slash commands shipped for AI-assisted development.

v3.8.7 -- Benchmark and Stability (March 26)

- Keyword argument fix for `run!()`: `port:`, `host:`, and `debug:` no longer crash the environment loader
- Updated benchmarks against Roda, Sinatra, and Rails

---

v3.7.x

Released: March 25, 2026

v3.7.0 -- Template Auto-Serve, Firebird Migrations (March 25)

The framework serves `index.html` or `index.twig` from `src/templates/` at `/` without a route definition. User-registered `GET /` routes take priority.

Firebird migrations now check `RDB$RELATION_FIELDS` before executing `ALTER TABLE ADD`. Columns that exist are skipped.

v3.7.1 -- Full Template Auto-Serve (March 25)

Any `.twig` or `.html` file in `src/templates/` is now browsable by URL path. `/hello` serves `src/templates/hello.twig`. Production mode caches the lookup table at startup.

---

v3.6.x

Released: March 25, 2026

v3.6.0 -- Architectural Parity (March 25)

Breaking change: `fetch(skip:)` is replaced by `fetch(offset:)`. No alias.

```
# BEFORE (v3.5.x):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

users = User.fetch(limit: 10, skip: 20)
```

```
# AFTER (v3.6.0):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

users = User.fetch(limit: 10, offset: 20)
```

Other changes:

- Source directories follow the `src/` prefix convention across all languages
- `TINA4_LOCALE` is the only supported locale environment variable (other names removed)
- Migration file paths standardized to `src/migrations/`

---

v3.5.x

Released: March 24, 2026

v3.5.0 -- Bundled Frontend, Swagger CRUD, Middleware (March 24)

- `tina4js.min.js` (13.6 KB) ships inside the gem. The reactive frontend library loads without a CDN or npm install
- Auto-CRUD routes now include Swagger metadata
- Middleware standardized to `before_*` and `after_*` naming with three built-in middlewares

---

v3.4.x

Released: March 24, 2026

v3.4.0 -- Auth, WebSocket, DatabaseResult (March 24)

Breaking change: Auth method names changed. The old names remain as aliases.

```
# BEFORE (v3.3.x):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

token = auth.create_token(payload)
valid = auth.validate_token(token)

# AFTER (v3.4.0 -- preferred):

## v3.10.70 (2026-04-06)
```

```
- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

token = auth.get_token(payload)
valid = auth.valid_token(token)

# Old names still work but are deprecated.

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

HS256 authentication. Set `TINA4_SECRET` in your `.env` and auth uses HS256. Provide RSA key files and it uses RS256. The framework picks the right algorithm.

```
# .env for HS256:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

TINA4_SECRET=my-secret-key

# .env for RS256:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js
```

Bug fix: Base64url padding in HS256 tokens caused validation failures. Fixed.

WebSocket improvements:

- `Router.websocket("/ws/chat")` for route-based WebSocket handlers
- Path-scoped broadcast: messages sent to `/ws/chat` reach only clients connected to that path
- `send_text` renamed to `send` on `WebSocketConnection` (`send_text` kept as alias)

DatabaseResult enhancements:

- `columns` returns column names
- `column_info` provides schema metadata (type, nullable, default) on demand
- `to_paginate` formats results for paginated responses

Frond template additions:

- Ternary-with-filter: `{{ value ? value|upper : "default" }}`
- `data_uri` filter for inline file display in templates

---

v3.3.x

Released: March 24, 2026

v3.3.0 -- Queue API, Route Chaining (March 24)

Breaking change: `Producer` and `Consumer` classes removed. Use `queue.produce()` and `queue.consume()` directly.

```
# BEFORE (v3.2.x):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

producer = Tina4::Producer.new(queue)
producer.send(message)
consumer = Tina4::Consumer.new(queue)
consumer.listen { |msg| handle(msg) }

# AFTER (v3.3.0):

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

queue.produce("channel", { data: "payload" })
queue.consume("channel") do |job|
  handle(job)
  job.complete
end
```

Route chaining. Mark routes as authenticated or cached with chainable modifiers.

```
Tina4::Router.get("/dashboard").secure do |request, response|
  response.html("<h1>Dashboard</h1>")
end

Tina4::Router.get("/static-page").cache(ttl: 3600) do |request, response|
  response.html("<h1>Cached for one hour</h1>")
end
```

Other additions:

- MongoDB queue backend
- Database session handler for full backend parity

- Valkey added to session handler options
- Migration parity: advanced SQL splitting, status tracking, rollback via CLI
- Auto-increment port if the default is in use; browser opens on startup

---

v3.2.x

Released: March 23, 2026

v3.2.0 -- Flexible Route Handlers (March 23)

Route handlers now accept zero, one, or two parameters. The framework detects what your block expects and provides the right objects.

```
# Zero params -- just return a response:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Tina4::Router.get("/health") { "OK" }

# One param -- response only:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Tina4::Router.get("/hello") { |response| response.html("Hello") }

# Two params -- request and response:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Tina4::Router.get("/echo") do |request, response|
  response.json({ body: request.body })
end

# Named :request or :req -- single param receives the request:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
```

```
- Full parity across Python, PHP, Ruby, Node.js

Tina4::Router.post("/submit") { |request| process(request.body) }
```

Bug fix: The 500 error overlay crashed because it did not receive the Rack environment. Fixed.

---

v3.1.x

Released: March 21 -- 22, 2026

v3.1.0 -- ORM Relationships, Caching, Queues (March 22)

The largest feature release after the initial launch. Fourteen capabilities landed in one version.

ORM relationships. Define `has_many`, `has_one`, and `belongs_to` with eager loading.

```
class User < Tina4::ORM
  has_many :posts
  has_one :profile
end

class Post < Tina4::ORM
  belongs_to :user
end

user = User.find(1)
user.posts  # => eager-loaded array of Post objects
```

Caching. Switch between memory, Redis, and file cache by setting one environment variable.

```
# .env:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

TINA4_CACHE_BACKEND=redis
TINA4_CACHE_URL=redis://localhost:6379

# Code stays the same:

## v3.10.70 (2026-04-06)

- **New:** SSE (Server-Sent Events) support via `response.stream()` - pass a generator, framewor
- **New:** Chapter 24 added to documentation: Server-Sent Events
- Feature count: 45 (was 44)
- Full parity across Python, PHP, Ruby, Node.js

Tina4::Cache.set("key", "value", ttl: 300)
Tina4::Cache.get("key")
```

Database query caching. Set `TINA4_DB_CACHE=true` for transparent query result caching.

Queue system. Switch between SQLite, RabbitMQ, and Kafka via `.env` without changing code.

Messenger. Unified messaging driven by environment configuration.

Scaffolding. `tina4 generate model User`, `tina4 generate route api/users`, `tina4 generate migration create_users`, `tina4 generate middleware auth`.

Frond template engine. `raw`/`endraw` blocks and `from` imports.

Performance. Frond pre-compilation caches parsed tokens. File rendering runs faster.

Other additions:

- Production server auto-detection (Puma, cluster mode)
- GitHub Actions CI/CD
- Error pages: clean 404/500/403 without branding
- `numeric_field` type in ORM
- `truthy?()` helper method
- Log rotation

v3.1.1 -- DevMailbox Fix (March 22)

Bug fix: DevMailbox timestamp precision was insufficient for reliable sort ordering.

v3.1.2 -- Documentation Fixes (March 22)

README code examples updated to match the actual v3 API. Quick start guide added.

---

v3.0.x

Released: March 21, 2026

v3.0.0 -- Initial Release (March 21)

The initial Ruby release. Zero gem dependencies. Everything the framework needs -- HTTP server, template engine, ORM, migrations, auth, queue, GraphQL, WebSocket, WSDL -- ships inside a single gem.

Core features:

- Rack-based HTTP server (compatible with Puma, Thin, WEBrick)
- Frond template engine (Twig-compatible syntax)
- ORM with support for SQLite, PostgreSQL, MySQL, MSSQL, and Firebird
- JWT authentication (RS256)
- Queue system
- GraphQL endpoint
- WebSocket server
- WSDL/SOAP service generation

- DevAdmin dashboard with developer tooling
- AI coding tool integration (auto-detect and install context for seven tools)
- Full test suite passing

Quick start:

```
require "tina4"

Tina4::Router.get("/") { |request, response| response.html("<h1>Hello Tina4!</h1>") }

Tina4.run!
```

```
gem install tina4ruby
```

The server starts on port 7147 by default. Set `host: "0.0.0.0"` for Docker deployments.

Pre-Release (v0.x)

Released: March 18, 2026

Versions v0.4.0 through v0.5.2 were development previews. They established the gem structure and basic routing but lacked the ORM, template engine, and queue system. If you used a v0.x release, upgrade directly to v3.0.0 -- there is no migration path from v0.x.

Feature Catalog

Tina4 3.14 has **140 numbered catalog entries**. The number describes the framework family's implementation and audit inventory. It does not mean that 140 features have reached four-language parity, and it does not mean that every entry lives inside each backend package.

This chapter is the map. The earlier chapters explain the public APIs available to Ruby applications. The numbered audit packets define the parity work and the clean-room formula for another language.

Read the status correctly

A feature moves through four distinct states:

- **Catalogued** means the feature has a stable number, name, owner, and audit packet.
- **Shipped** means code exists in one or more released components.
- **Audited** means the team has measured all relevant implementations and recorded contradictions and decisions.
- **Contract-proven** means a shared fixture passes in every applicable implementation with real dependencies.

Do not collapse those states into one green tick. At version 3.13.101, the contract ledger reports **55 fixtures and 282 proven invariants, with 0 owed and 0 broken inside those fixture-covered contracts**. That result proves the named contracts. It does not certify the remaining catalog entries.

Where the features live

Range	Owner	Meaning
1-109, 126-132, 135-140	Backend frameworks	Runtime, developer, testing, and application-facing capabilities. Availability and maturity still require per-feature evidence.
110-125 and 134	Shared Rust CLI	One language-neutral client used with all four backends. These are not four separate backend implementations.
133	Verification contract	Carbonah benchmark and report shape, not an application runtime API.
Provider entries	Selected integration	Each selectable provider has its own number because another language must implement and test it separately. Providers can require an external service, driver, language extension, or extra package.

Feature 63 includes browser helpers and the separate tina4-js frontend package. The backend books explain their integration, but tina4-js is not embedded four times.

Current parity boundary

Evidence	Python	PHP	Ruby	Node.js
Catalog membership	Inventory source	Audit in progress	Audit in progress	Audit in progress
Fixture-covered contracts	Proven	Proven	Proven	Proven
Entire 140-entry catalog	Not yet proven	Not yet proven	Not yet proven	Not yet proven

The initial catalog followed the Python module inventory, but Python is not a permanent master. Tina4 promotes the best implementation after audit and captures that decision in an ADR and shared fixture.

Known missing feature surfaces remain tracked in PHP issue 185 and Node.js issue 37. An empty issue list in another repository is not parity evidence. The fixture ledger is the evidence.

Dependency boundary

The Ruby gem declares runtime gems for Rack/Puma serving, SQLite, mail, XML, logging, and encoding. Tina4 Ruby is lean, but it is not a zero-dependency gem.

"Core dependency" and "provider dependency" are different facts. A local SQLite application can stay small while PostgreSQL, RabbitMQ, Kafka, Redis, MongoDB, S3, or hosted AI still requires the driver or service that speaks that protocol.

Numbered catalog

Foundation

#	Catalog entry
1	DotEnv and typed environment
2	Structured logger

Database and providers

#	Catalog entry
3	Database adapter interface
4	Database URL parser
5	Database facade and safe writes
6	Query builder
7	SQL translator
8	SQLite provider
9	PostgreSQL provider
10	MySQL provider
11	MSSQL provider
12	Firebird provider
13	ODBC provider
14	MongoDB SQL-translation provider
15	Migrations
16	Race-safe database sequences

ORM and data layer

#	Catalog entry
17	ORM base class
18	ORM fields and column mapping
19	Input and request validation
20	Soft delete
21	Declarative ORM relationships
22	Imperative ORM relationships
23	ORM scopes
24	Paginated database and ORM results
25	ORM result caching
26	ORM instance loading
27	Automatic CRUD from models
28	Seeder and fake data

HTTP core

#	Catalog entry
29	HTTP request model
30	HTTP response model and representation types
31	Router and dispatch
32	Route groups
33	Middleware pipeline

HTTP policies

#	Catalog entry
34	CORS middleware
35	Rate limiting
36	Security headers middleware
37	CSRF protection

HTTP runtime

#	Catalog entry
38	Health and readiness endpoints
39	Graceful shutdown
40	HTTP compression and ETag
41	Static assets and cache revalidation
42	Configurable error pages
43	Request ID tracking
44	File upload contract
45	Swagger and OpenAPI
46	Default landing page
47	In-process background tasks

Frond template engine

#	Catalog entry
48	Frond lexer
49	Frond parser
50	Frond compiler
51	Frond runtime
52	Frond filters
53	Frond tags
54	Frond expression tests
55	Frond functions
56	Frond extensibility API
57	Frond auto-escaping
58	Frond sandboxing
59	Frond template caching
60	Frond fragment caching

Frontend assets

#	Catalog entry
61	SCSS compiler
62	Tina4 CSS
63	Frond and Tina4 browser helpers

Authentication

#	Catalog entry
64	JWT and request authentication

Sessions and providers

#	Catalog entry
65	Session lifecycle
66	File session provider
67	Redis session provider
68	Valkey session provider
69	MongoDB session provider
70	Database session provider
71	Memcached session provider

Cache and providers

#	Catalog entry
72	Cache interface and provider selection
73	Memory cache provider
74	File cache provider
75	Redis cache provider
76	Valkey cache provider
77	Memcached cache provider
78	MongoDB cache provider
79	Database cache provider
80	HTTP response cache

Integrations and storage

#	Catalog entry
81	Standard-library HTTP API client
82	GraphQL
83	WebSocket protocol and server
84	Redis WebSocket backplane
85	NATS WebSocket backplane
86	WSDL and SOAP
87	Localization and i18n
88	Email and messenger
89	Queue lifecycle
90	Lite queue provider
91	RabbitMQ queue provider
92	Kafka queue provider
93	MongoDB queue provider
94	MQTT client
95	Document store interface
96	SQLite document store provider
97	MongoDB document store provider
98	Realtime collaboration
99	Local realtime attachment storage
100	S3 realtime attachment storage
101	Model Context Protocol server
102	Local source and documentation context index
103	Live framework and application API index
135	App-facing LLM client
137	GIS spatial points and queries
138	RBAC (role/permission guards)

139	Graph databases (Ultipa, Neo4j, Memgraph, ArangoDB)
140	Provider-neutral Web Push notifications

Enterprise authentication

#	Catalog entry
136	Configuration-first OpenID Connect SSO

Application runtime

#	Catalog entry
104	Event and listener system
105	Dependency injection container
106	Service runner
107	HTML element builder
108	AI coding-tool integration
109	Lazy feature loading and preload manifest

CLI

#	Catalog entry
110	CLI project initialization
111	CLI development server
112	CLI migrations
113	CLI seeding
114	CLI test runner
115	CLI route inspection
116	CLI interactive console
117	CLI environment management
118	CLI queue management
119	CLI container build
120	CLI code generation
121	CLI code metrics
122	CLI command discovery and help
123	CLI doctor
124	CLI guided setup
125	CLI deployment
134	CLI AI-skills installation and refresh

Developer runtime

#	Catalog entry
126	Development error overlay
127	Development admin dashboard
128	Dual development and test ports
129	Development port takeover
130	Dynamic framework version

Testing and verification tools

#	Catalog entry
131	In-process HTTP test client
132	Inline testing API
133	Carbonah benchmark contract

What parity means

Parity applies to observable contracts: response shapes, status codes, environment keys, error messages, file formats, and security rules. Host-language code remains idiomatic. A Python method can use snake_case while PHP and Node.js use camelCase, but all three must return the same public data.

The catalog gives another language a checklist. The audit packet supplies the decisions. The fixture supplies the proof. A name alone proves nothing; a green contract earns the claim.

Real-time Collaboration (WebRTC)

1. The Media Server You Do Not Have to Run

You want a call button. Two people click it and they are talking: audio, video, a shared screen. Next to the call sits a chat panel and a place to drop a file. This is the Slack shape, the Teams shape, and the usual advice is heavy. Stand up a media server. Wire in TURN. Operate all of it.

You do not need any of that to start. Modern browsers already speak WebRTC to each other directly, peer to peer. What they cannot do is *find* each other. One browser has to hand its connection offer to the other and get an answer back. That hand-off is called **signalling**, and it is a tiny amount of text relayed through a server. The media itself, the actual audio and video, never touches your server. It flows browser to browser.

`Tina4::Realtime` is that relay, plus the two things every collaboration tool needs around a call: persistent **chat** and permissioned **file** transfer. It shipped in tina4-ruby 3.13.57, carries zero extra dependencies, and mounts with one line. Media is peer to peer, a **mesh**. Tina4 carries no media and never parses your SDP. It forwards the handshake and nothing more.

Ruby is mesh-only. There is one media backend, and `mount` has no `media:` parameter. An SFU backend is a future drop-in, not a Phase-1 option here. The rest of this chapter is the backend surface. The browser side is the tina4-js `rtc` module, which fetches `/api/rtc/config` and discovers every path from there, so the client never hardcodes a URL.

---

2. What You Get, and How It Differs from Raw WebSocket

The WebSocket chapter gave you the raw `Tina4::Router.websocket` primitive: a path, a handler, a `(connection, event, data)` block. That is the tool you reach for when you build your *own* protocol. `Tina4::Realtime` is the opposite end. It is a *pre-built* protocol for calls, chat, and files, assembled from that same primitive underneath.

The module gives you three surfaces, and you enable only the ones you want:

- **calls** - a WebRTC signalling relay (mesh, peer to peer) plus a self-describing ICE-config endpoint. The relay forwards offer, answer, and ICE frames between peers in a room. It is public: no login to join a room.
- **chat** - persistent channels and messages backed by framework-owned ORM models, a secured chat WebSocket with live presence, typing, and read receipts, and a history endpoint for catch-up on reconnect.
- **files** - permissioned upload and download through a pluggable storage backend (local filesystem by default, S3-compatible optional).

The distinction in one line: with `Tina4::Router.websocket` you write the handler and invent the message protocol; with `Tina4::Realtime.mount` the protocol is written for you and the browser discovers every path from one config endpoint. Under the hood the realtime WebSockets *are* Tina4 WebSockets. Same `(connection, event, data)` convention, same room

manager. You are handed the handlers pre-written.

This is a cross-language feature. The same paths, JSON shapes, env vars, and `tina4_rt_*` tables exist in tina4-python, tina4-php, and tina4-nodejs. Python is the reference implementation, and the Ruby-specific differences are called out in the footguns at the end.

---

3. Mounting the Module: `Tina4::Realtime.mount`

Call `mount` once at boot, in `app.rb`, after `Tina4.initialize!` and `Tina4.bind_database`, before the server starts. It registers the routes and WebSockets and returns the resolved path map, a Hash with String keys. That is the same map the config endpoint later serves back to clients.

```
Tina4::Realtime.mount(prefix: "", authorize: nil, storage: nil, features: nil)
```

```
Tina4::Realtime.mount                                          # calls only (default)
Tina4::Realtime.mount(features: %w[calls chat])                # add persistent chat
Tina4::Realtime.mount(prefix: "/api/collab",
                      features: %w[calls chat files])          # relocate the whole surface
```

Keyword	Meaning
`prefix:`	Mounts the whole surface under this path. Leading and trailing slashes are stripped, so `"/api/collab"` and `"api/collab/"` both resolve to `/api/collab`. Default `""` (root).
`authorize:`	Membership guard `->(identity, channel_id) { true_or_false }` used by **chat** and **files**. `identity` is the String user id from the JWT. Defaults to a `ChannelMember` membership check.
`storage:`	A `StorageBackend` instance for the **files** feature. Defaults to the env-selected store (`local`).
`features:`	Array of any of `"calls"`, `"chat"`, `"files"`. Default `["calls"]`.

The return value is the resolved path map, so you log it or assert against it. Base paths are String-keyed; the config endpoint later appends the `{room}`, `{channel}`, and `{id}` template tokens for the client:

```
Tina4::Realtime.mount
# => {"backend"=>"mesh", "config"=>"/api/rtc/config", "signalling"=>"/ws/rtc"}

Tina4::Realtime.mount(features: %w[calls chat])
# => {"backend"=>"mesh", "config"=>"/api/rtc/config", "signalling"=>"/ws/rtc",
#     "chat"=>"/ws/chat", "messages"=>"/api/channels"}

Tina4::Realtime.mount(features: %w[files])
```

```
# => {"backend"=>"mesh", "config"=>"/api/rtc/config", "files"=>"/api/files"}
```

`config` is added by any enabled feature. `calls` sets it outright; `chat` and `files` set it with `||=`. So even a chat-only or files-only mount still exposes `/api/rtc/config`.

What each feature wires

Feature	Route registered	Auth
any (has `config`)	`GET {p}/api/rtc/config`	public (no `.secure`)
`calls`	`WS {p}/ws/rtc/{room}`	public (unauthenticated)
`chat`	`WS {p}/ws/chat/{channel}`	secured - `.secure`, valid JWT on upgrade
`chat`	`GET {p}/api/channels/{id}/messages`	secured - `.secure`
`files`	`POST {p}/api/files` (upload)	write route - default bearer-token gate
`files`	`GET {p}/api/files/{key}` (download)	secured - `.secure`

If `chat` or `files` is enabled, `ensure_chat_tables` runs at mount and creates the `tina4_rt_*` tables. A missing database logs an error and does not crash boot. That footgun waits for you at the end of the chapter.

---

4. The Config Bootstrap: `GET {p}/api/rtc/config`

This is the one URL the frontend has to know. It fetches the config, reads back the paths and ICE servers, and drives everything else from there. The client and server never drift, because the server is the single source of truth for its own routes. Move `prefix:` on the server and the client follows.

The body is feature-gated. Only keys for enabled features appear.

```
{
  "backend": "mesh",
  "iceServers": [ /* Tina4::Realtime.ice_servers */ ],   // calls
  "signalling": "/ws/rtc/{room}",                        // calls
  "chat": "/ws/chat/{channel}",                          // chat
  "messages": "/api/channels/{id}/messages",             // chat
  "files": "/api/files"                                  // files
}
```

`{room}`, `{channel}`, and `{id}` are literal template tokens the client substitutes with a real room name, channel id, or message id before connecting.

This endpoint is public. No token required, because the client needs the ICE config before it has authenticated anyone into a call. It returns your ICE and TURN configuration, including freshly-minted ephemeral TURN credentials. That is by design, and it is exactly why those credentials are short-lived.

```
// Browser: discover everything from one fetch, then wire the sockets.
const cfg = await fetch("/api/rtc/config").then(r => r.json());

const signallingUrl = cfg.signalling.replace("{room}", roomId);   // /ws/rtc/room-42
const pc = new RTCPeerConnection({ iceServers: cfg.iceServers });
const ws = new WebSocket(`wss://${location.host}${signallingUrl}`);
```

---

5. ICE and TURN Servers: `Tina4::Realtime.ice_servers`

Before two browsers connect, each gathers candidate addresses using **ICE**. A **STUN** server tells a browser its public address. Peers on the same network find each other with STUN alone. A **TURN** server relays media when a direct path is impossible: strict NAT, symmetric firewalls. `ice_servers` builds the list the client hands to `RTCPeerConnection`, straight from the environment.

It always includes a STUN entry. It adds a TURN entry only when both `TINA4_RTC_TURN_URL` and `TINA4_RTC_TURN_SECRET` are set. TURN credentials use the coturn `use-auth-secret` scheme: a time-limited username and credential pair minted on each request, so you never ship static TURN passwords to the browser.

```
username   = (Time.now.to_i + ttl).to_s                                    # expiry epoch
credential = Base64.strict_encode64(OpenSSL::HMAC.digest("SHA1", secret, username))
```

```
# No TURN env set - STUN only:
Tina4::Realtime.ice_servers
# => [{"urls"=>["stun:stun.l.google.com:19302"]}]

# TINA4_RTC_TURN_URL + TINA4_RTC_TURN_SECRET set:
# => [{"urls"=>["stun:stun.l.google.com:19302"]},
#     {"urls"=>["turn:turn.example.com:3478"], "username"=>"1783546725", "credential"=>"ie7Mm...
```

STUN and TURN URLs are comma-separated in the env and split into arrays.

Environment variables

Var	Default	Effect
`TINA4_RTC_STUN_URLS`	`stun:stun.l.google.com:19302`	Comma-separated STUN URLs.
`TINA4_RTC_TURN_URL`	-	Comma-separated TURN URLs; enables TURN when set together with the secret.
`TINA4_RTC_TURN_SECRET`	-	coturn `use-auth-secret` shared secret (drives the ephemeral credentials).
`TINA4_RTC_TURN_TTL`	`3600`	Ephemeral TURN credential lifetime, in seconds.
`TINA4_RTC_BACKEND`	-	Read only for cross-language config parity; Ruby ignores it. The backend is always `mesh`.

```
# Enable a coturn relay for peers behind symmetric NAT.
export TINA4_RTC_TURN_URL="turn:turn.example.com:3478"
export TINA4_RTC_TURN_SECRET="a-long-random-shared-secret"
export TINA4_RTC_TURN_TTL="3600"
```

---

6. Signalling: The Mesh Relay

This is the relay that lets two browsers negotiate a peer connection. It registers at `WS {p}/ws/rtc/{room}` with `Tina4::Router.websocket`, and it is not `.secure`, so it is public. Anyone can join any room. The Ruby WebSocket convention is `(connection, event, data)` where `event` is a **Symbol**:

```
# connection : the WebSocket connection
# event      : :open | :message | :close
# data       : the payload String on :message; nil on :open / :close
```

The whole handler is a mesh relay. It moves opaque frames between peers and gets out of the way:

```
Tina4::Router.websocket "/ws/rtc/{room}" do |connection, event, data|
  room = connection.params[:room].to_s
  next if room.empty?                                  # empty room -> no-op

  key = "rtc:#{room}"
  case event
  when :open
    connection.join_room(key)
  when :message
    connection.broadcast_to_room(key, data, exclude_self: true)  # relay raw, verbatim
  end
end
```

The behaviour is deliberately minimal:

- `room = connection.params[:room].to_s`. An empty room is a no-op, and the handler returns.
- `:open` calls `connection.join_room("rtc:#{room}")`.
- `:message` calls `connection.broadcast_to_room("rtc:#{room}", data, exclude_self: true)`, relaying the raw payload to the other peers untouched.

Tina4 never parses the SDP. Peers address each other by putting a `to` field in their own frames and filtering on it. Rooms are namespaced `rtc:<room>` so signalling rooms never collide with chat channels, which use `chat:<channel>`, on the shared WebSocket manager.

The socket is public and the room is a URL segment, so treat the room id as a capability. Gate call access at your app layer if a call must be private: issue unguessable room ids, or check membership before you hand the client a room name.

For a private call, do not mount the built-in `calls` feature and assume its `authorize:` Proc protects signalling: that Proc applies only to chat and files. Register an application-owned signalling route with `Tina4::Router.websocket(path) { ... }.secure`, then verify the authenticated identity in `connection.auth` belongs to the requested room on open and on every relayed frame. Browser clients carry the JWT with `new WebSocket(url, ["bearer", token])`.

The current `tina4-js` `rtc.call()` helper opens the built-in public signalling socket and has no signalling-token option. Use a small application WebSocket client for the protected route (or add token support to that client API in a separately versioned change). Unguessable room names reduce discovery; they are not authorization.

---

7. Chat: Secured Channels, Presence, History

Chat is the opposite of signalling: secured. The chat socket registers with `.secure`, so a valid JWT is required on the upgrade. The router rejects an unauthenticated upgrade before the handler ever runs. The handler is `chat_handler(connection, event, data)`.

- Channels are addressed by **integer id**. The handler requires `connection.params[:channel]` to match `\A\d+\z`. A non-integer channel makes the handler return silently: the socket opens and does nothing, no error frame.
- `identity = Tina4::Realtime.identity(connection.auth)`, the String id from the verified token.
- The room key is `chat:<channel_id>`.

Every inbound frame is a JSON object. Every broadcast is a `.to_json` String.

Event / message	Server behaviour

`:open`	Authorize. Fail: send `{"type":"error","error":"not a member of this channel"}` then `close`. OK: `join_room`, send the caller the roster `{"type":"presence","event":"roster","users":[...]}`, then broadcast `{"type":"presence","event":"join","user_id":<id>}` (exclude self).
`:close`	Broadcast `{"type":"presence","event":"leave","user_id":<id>}` (exclude self).
`"typing"`	Broadcast `{"type":"typing","user_id":<id>}` (exclude self).
`"read"`	Advance the member's read cursor (`last_read_at = now`), broadcast `{"type":"read","user_id":<id>,"at":<iso8601>}` (exclude self).
`"message"`	Strip `body`; empty or whitespace is silently dropped. Otherwise persist a `Message` row and broadcast `{"type":"message","message":<saved>}` to everyone including the sender, so the sender's optimistic message reconciles with its server `id` and `created_at`.

`"type"` defaults to `"message"` when absent. Unknown types are ignored, and non-Hash payloads are ignored.

Authorization is re-checked on every inbound `:message` frame, not just on join. Membership can be revoked mid-session, and the server never trusts an identity carried in the payload. The roster is the sorted, de-duplicated set of authenticated identities currently in the room, collected from each live connection's `auth`.

A saved message, the shape broadcast and returned by history, looks like this:

```
{ "id": 42, "channel_id": 7, "user_id": "12", "body": "hi team",
  "thread_id": null, "created_at": "2026-07-08T10:15:00Z" }
```

`thread_id` is `null` for a top-level message, or the parent message id for a threaded reply.

Chat history: `GET {p}/api/channels/{id}/messages`

The catch-up-on-reconnect endpoint. A client opens the socket for live traffic and calls this to backfill what it missed.

- Identity comes from `request.user`, the router-attached, verified JWT payload.

- `channel_id <= 0` returns 400. Not authorized returns 403. Otherwise the message list.
- Query params: `before` (return messages with `id < before`) and `limit` (default 50, floored at 1, capped at 200).
- Messages come back newest-first, the standard infinite-scroll-backwards shape. Each item uses the saved-message JSON above.

```
# Page backwards from message id 900, 50 at a time.
curl -H "Authorization: Bearer $JWT" \
  "http://localhost:7147/api/channels/42/messages?before=900&limit=50"
```

---

8. Files: Upload and Download

Enable with `features: %w[files]`. Files store through a `StorageBackend`: the `storage:` argument, or the env-selected store, default `LocalStorage`. The backend resolves once at mount via `Tina4::Realtime::Storage.select(storage)`.

`POST {p}/api/files` - upload

Multipart: a file field named `file`, plus a form field `channel_id` (required, integer). Invalid or missing `channel_id` returns 400; not a channel member returns 403; no file returns 400. The blob stores under an opaque, collision-free `storage_key` (`SecureRandom.hex(16)` plus a sanitized, length-capped extension, never a user-controlled path). An `Attachment` row records the metadata only, never the blob. It responds 201:

```
{ "id": 9, "key": "3f8c...e1.png", "filename": "diagram.png",
  "mime": "image/png", "size": 20481,
  "url": "/api/files/3f8c...e1.png" }
```

`url` is `store.url(key)` when the backend exposes a direct URL (an S3 presigned URL, say), otherwise the app download route `{files}/{key}`.

This route registers with `Tina4::Router.post` without `.no_auth`, so the default write-route bearer gate applies: a tokenless upload 401s.

```
curl -X POST http://localhost:7147/api/files \
  -H "Authorization: Bearer $JWT" \
  -F "channel_id=42" \
  -F "file=@./diagram.png"
```

`GET {p}/api/files/{key}` - download (secured)

- Looks up the `Attachment` by `storage_key`; missing returns 404. Authorizes against the attachment's `channel_id`; a non-member returns 403.
- If the backend has a direct URL, it 302-redirects to it. Otherwise it streams the bytes (200) with `Content-Disposition: inline; filename="..."` and `Content-Type` from `attachment.mime` (default `application/octet-stream`). A missing blob on disk returns 404.

The download route needs `.secure` because a `GET` is public by default. The upload does not, because writes already require a token.

Storage backends

`Tina4::Realtime::Storage.select(storage = nil)` resolves from the `storage:` argument or `TINA4_STORAGE_BACKEND` (`local` default, `s3`). An `s3` backend that cannot be built (the `aws-sdk-s3` gem is missing, or the config is incomplete) falls back to `LocalStorage` with a warning. A real persistent store, never a silent no-op. Ruby rescues both `StandardError` and `LoadError` here, because a missing gem raises `LoadError`, which is not a `StandardError`.

Var	Default	Effect
`TINA4_STORAGE_BACKEND`	`local`	`local` or `s3`.
`TINA4_STORAGE_DIR`	`data/rt_storage`	Local filesystem directory.
`TINA4_STORAGE_URL`	-	S3 endpoint URL (S3-compatible / MinIO); `force_path_style` is on.
`TINA4_STORAGE_KEY` / `TINA4_STORAGE_SECRET`	-	S3 credentials.
`TINA4_STORAGE_BUCKET`	-	S3 bucket (required for S3; raises `ArgumentError` if absent).
`TINA4_STORAGE_REGION`	`us-east-1`	S3 region.

`LocalStorage` resolves every key inside its root and rejects path traversal (it raises `ArgumentError` on an unsafe key); its `url` returns `nil`, so blobs serve through the permissioned download route. `S3Storage` returns a presigned GET URL from `url`, so clients fetch large blobs straight from object storage and skip your app server.

You can also pass a custom backend instance directly:

```
Tina4::Realtime.mount(features: %w[chat files], storage: MyStorageBackend.new)
```

---

9. Auth, Identity, and the Data Model

Identity is always taken from the verified token, never from the message body. Two pieces make membership work: how an identity is extracted, and how membership is checked.

Extracting identity, `Tina4::Realtime.identity(auth)`. It pulls a stable String user id from a verified JWT payload, trying the claims `user_id`, then `sub`, then `id` (String or Symbol keys). It returns `nil` when `auth` is not a Hash or none of those claims are present. Identities round-trip as Strings, so an integer id, a UUID, or an email all fit. WebSocket identity comes from `connection.auth`, the verified payload the router attached on the secured upgrade. HTTP identity comes from `request.user`, also router-attached. Ruby matches the PHP and Node ports here: it does not re-parse the `Authorization` header the way Python's HTTP handlers do.

Checking membership, `Tina4::Realtime.authorized?(identity, channel_id)`**.** This is the shared guard for chat channels and file access. A `nil` identity is always denied. If a custom `authorize:` Proc was passed to `mount`, it wins (`!!proc.call(identity, channel_id)`); otherwise the secure default requires channel membership:

```
ChannelMember.count("channel_id = ? AND user_id = ?", [channel_id, identity]).positive?
```

Any exception logs and returns `false` (deny). Because the guard runs on every inbound chat frame, a custom `authorize:` must be cheap.

```
# Open every channel to any logged-in user (skip the per-channel membership check).
Tina4::Realtime.mount(
  features: %w[calls chat],
  authorize: ->(identity, _channel_id) { !identity.nil? }   # keep it cheap - runs per frame
)
```

Data model

The chat and files features own a small set of `Tina4::ORM` models, all with the `tina4_rt_` table prefix so they never collide with your app's own tables. Ruby is snake_case end to end: columns, attributes, and JSON keys match with no mapping layer. Tables are created on demand at mount (`ensure_chat_tables` iterates them in dependency order).

Model	Table	Key fields
`Workspace`	`tina4_rt_workspaces`	`id`, `name`, `created_at`
`Channel`	`tina4_rt_channels`	`id`, `workspace_id`, `name`, `kind` (`public`/`private`/`dm`, default `public`), `created_at`
`ChannelMember`	`tina4_rt_channel_members`	`id`, `channel_id`, `user_id` (String, <=128), `role` (default `member`), `last_read_at` (read cursor)
`Message`	`tina4_rt_messages`	`id`, `channel_id`, `user_id` (String), `body` (Text), `thread_id` (nullable parent id), `created_at`, `edited_at` (nullable)
`Attachment`	`tina4_rt_attachments`	`id`, `channel_id`, `message_id` (nullable), `storage_key`, `filename`, `mime`, `size`, `thumb_key` (nullable)

`workspace_id` and `channel_id` are plain integer FK columns queried directly, no ORM relationship wiring, because the control plane does not need it. `user_id` is a String everywhere, so any JWT identity shape fits.

Because these are ordinary Tina4 ORM models, you seed a workspace, a channel, and its members exactly as in the ORM chapter. `save` returns the model on success, so you chain straight through to the fresh `id`:

```
ws = Tina4::Realtime::Workspace.new(name: "Acme").save
channel = Tina4::Realtime::Channel.new(
  workspace_id: ws.id, name: "general", kind: "public"
).save
Tina4::Realtime::ChannelMember.new(
  channel_id: channel.id, user_id: "12", role: "owner"
).save
```

Now user `12` can open `WS /ws/chat/#{channel.id}` and pass the history and file checks.

---

10. A Complete Example

A mesh video call with a chat side-panel, using only the browser's built-in `RTCPeerConnection` and `WebSocket`. No client library required.

Backend - `app.rb`

Mount the surface after the database is bound, and seed one channel so chat has somewhere to land.

```
# app.rb - after Tina4.initialize! and a bound database.
require "tina4"

# Chat/files need a bound DB BEFORE mount(features: [...]) - see the footguns.
Tina4.bind_database(Tina4::Database.new("sqlite://data/app.db"))

paths = Tina4::Realtime.mount(features: %w[calls chat files])
Tina4::Log.info("realtime mounted: #{paths.inspect}")

# One-time seed so a channel + member exist.
if Tina4::Realtime::Channel.where("name = ?", ["general"], limit: 1).empty?
  ws = Tina4::Realtime::Workspace.new(name: "Acme").save
  channel = Tina4::Realtime::Channel.new(
    workspace_id: ws.id, name: "general", kind: "public"
  ).save
  Tina4::Realtime::ChannelMember.new(
    channel_id: channel.id, user_id: "12", role: "owner"
  ).save
end
```

Browser - bootstrap from the config endpoint

The client fetches `/api/rtc/config` first and never hardcodes a path.

```
// 1. Discover paths + ICE servers (public endpoint).
const cfg = await (await fetch("/api/rtc/config")).json();

// 2. Open the signalling socket for a room (public, no token).
const room = "standup";
const wsUrl = location.origin.replace(/^http/, "ws")
            + cfg.signalling.replace("{room}", room);
const signal = new WebSocket(wsUrl);
```

```
// 3. One RTCPeerConnection per call, using the server's ICE list.
const pc = new RTCPeerConnection({ iceServers: cfg.iceServers });

pc.onicecandidate = (e) => {
  if (e.candidate) signal.send(JSON.stringify({ kind: "ice", candidate: e.candidate }));
};
pc.ontrack = (e) => { document.getElementById("remote").srcObject = e.streams[0]; };

// 4. Handle relayed signalling frames. Tina4 forwards raw payloads verbatim.
signal.onmessage = async (evt) => {
  const msg = JSON.parse(evt.data);
  if (msg.kind === "offer") {
    await pc.setRemoteDescription(msg.sdp);
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
    signal.send(JSON.stringify({ kind: "answer", sdp: answer }));
  } else if (msg.kind === "answer") {
    await pc.setRemoteDescription(msg.sdp);
  } else if (msg.kind === "ice") {
    await pc.addIceCandidate(msg.candidate);
  }
};

// 5. The caller adds their camera/mic and sends an offer.
async function startCall() {
  const media = await navigator.mediaDevices.getUserMedia({ video: true, audio: true });
  document.getElementById("local").srcObject = media;
  media.getTracks().forEach((t) => pc.addTrack(t, media));
  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);
  signal.send(JSON.stringify({ kind: "offer", sdp: offer }));
}
```

Browser - the chat side-panel (secured, needs a JWT)

The chat upgrade carries the bearer token in the WebSocket subprotocol, the browser-friendly transport Tina4 accepts.

```
const channelId = 42;                       // integer id, not a name
const chatUrl = location.origin.replace(/^http/, "ws")
              + cfg.chat.replace("{channel}", channelId);
const chat = new WebSocket(chatUrl, ["bearer", jwt]);   // WS carries the JWT

chat.onmessage = (evt) => {
  const m = JSON.parse(evt.data);
  if (m.type === "message")  addLine(m.message.user_id, m.message.body);
  if (m.type === "presence") renderRoster(m);   // roster / join / leave
  if (m.type === "typing")   showTyping(m.user_id);
  if (m.type === "error")    console.warn(m.error);   // e.g. not a member
};

function send(text) { chat.send(JSON.stringify({ type: "message", body: text })); }
```

Start it the usual way. The WebSockets run alongside the HTTP server:

```
tina4 serve
```

The signalling socket is public: anyone with the room name joins. The chat socket and the history and file endpoints require a valid JWT and channel membership. Two browser tabs, both

members of channel `42`, joined to room `standup`, see each other's video peer to peer and each other's messages through the relay.

---

11. Footguns and Hard Rules

Ruby is mesh-only; `TINA4_RTC_BACKEND` is ignored. `mount` has no `media:` param, and the path map and config body hardcode `"backend" => "mesh"`. Setting `TINA4_RTC_BACKEND` does nothing in Ruby; it exists for cross-language config parity. An SFU/LiveKit backend is a future drop-in, not a Phase-1 option here.

Chat needs a bound database, but a missing one does not crash boot. With `features:` that include `chat` or `files`, `ensure_chat_tables` runs at mount. If no DB is bound, it logs an error and continues. `mount` still returns the full path map and registers every route, so the failure surfaces only at query time. Bind a DB (`Tina4.bind_database(db)` or `TINA4_DATABASE_URL`) before calling `mount(features: [...])`, or chat, history, and files will error per request while the app looks healthy.

The signalling WebSocket is public. `/ws/rtc/{room}` is not `.secure`, so anyone can join any room and receive relayed signalling frames. Only the chat WebSocket is JWT-secured. Gate call access at your app layer if you need it.

The config endpoint is public. `/api/rtc/config` returns your ICE and TURN config, including freshly-minted ephemeral TURN credentials. That is by design, which is exactly why the TURN credentials are time-limited (`TINA4_RTC_TURN_TTL`).

The WS handler signature is `(connection, event, data)` and `event` is a Symbol (`:open`, `:message`, `:close`); `data` is the payload String on `:message`, `nil` otherwise. This is the Ruby framework convention, not `(connection, data, event)`. The PHP port fires `($connection, $data, $event)`, so argument order differs across languages.

Channels are addressed by integer id. A non-integer `{channel}` makes the chat handler return silently, no error frame. The client sees a socket that opens and does nothing. Pass the numeric channel id, never its name.

Chat authorization is re-checked on every frame, and identity is always taken from the verified token (`connection.auth` / `request.user`), never from the message payload. Keep a custom `authorize:` cheap.

A message with an empty or whitespace `body` is silently dropped: no persist, no broadcast. `read`, `typing`, and unknown types never persist anything.

Upload is protected by the default write gate, not `.secure`; download is `.secure`. Do not add `.no_auth` to the upload route thinking it needs auth added; writes already require a token. The download route needs `.secure` only because a `GET` is public by default.

Where to Go Next

- **The WebSocket chapter** - the raw `Tina4::Router.websocket` primitive these handlers are built on, and the connection API (`join_room`, `broadcast_to_room`, `send_json`).
- **The ORM chapter and the Authentication chapter** - you seed workspaces, channels, and members with the ORM, and the JWT that secures chat is the same one from the auth chapter.
- **The tina4-js `rtc` module** - the browser side, which fetches `GET /api/rtc/config` so the client and server never drift on paths.

You started this chapter wanting a call button. You end it with calls, chat, presence, read receipts, history, and file transfer, mounted in one line, carrying no media, running no server you did not already have. The hard part of real-time was never the video. It was finding the other browser, and Tina4 relays that in a few lines of text.

AI Client

One class. Three methods. No provider SDK. The Tina4 AI client sends chat, completion, embedding, and streaming requests through one provider-neutral API.

This chapter covers the application-facing `Tina4::Ai` class. The `tina4 ai` command serves a different purpose: it installs Tina4 skills and context for coding assistants.

Configure a provider

The client supports `local`, `openai`, and `anthropic`. Local mode targets an OpenAI-compatible endpoint and needs no API key.

```
TINA4_AI_PROVIDER=local
TINA4_AI_URL=http://localhost:11437
TINA4_AI_MODEL=llama3.2
TINA4_AI_TIMEOUT=60
TINA4_AI_CONNECT_TIMEOUT=10
TINA4_AI_MAX_RETRIES=2
```

Use the hosted OpenAI service by changing three values:

```
TINA4_AI_PROVIDER=openai
TINA4_AI_URL=https://api.openai.com/v1
TINA4_AI_MODEL=gpt-4o-mini
TINA4_AI_KEY=your-api-key
```

Anthropic uses the same keys:

```
TINA4_AI_PROVIDER=anthropic
TINA4_AI_URL=https://api.anthropic.com/v1
TINA4_AI_MODEL=claude-3-5-haiku-latest
TINA4_AI_KEY=your-api-key
```

An explicit method argument wins over the environment. The environment wins over the built-in default. This lets one request use another model without changing the rest of the process.

Complete one prompt

`Tina4::Ai.complete` sends one user message and returns the response text.

```
require "tina4"

answer = Tina4::Ai.complete("Explain why this query needs an index")
puts answer
```

Pass keyword arguments when one call needs a different model or provider:

```
answer = Tina4::Ai.complete(
  "Summarize this incident report",
  provider: "openai",
  model: "gpt-4o-mini",
  temperature: 0.2,
  max_tokens: 300,
  timeout: 20
)
```

Hold a chat

`Tina4::Ai.chat` accepts a non-empty list of messages. Each message needs a `system`, `user`, or `assistant` role and string content.

```
require "tina4"

response = Tina4::Ai.chat([
  { role: "system", content: "Answer as a concise database engineer." },
  { role: "user", content: "When should I use a composite index?" }
])

puts response.text
puts response.model
puts response.usage["total_tokens"]
puts response.finish_reason
```

The `ChatResponse` object carries five fields:

Field	Meaning
`text`	Normalized response text
`model`	Model reported by the provider
`usage`	`prompt_tokens`, `completion_tokens`, and `total_tokens`
`finish_reason`	Provider finish reason, or `nil`
`raw`	Original provider response

Use `raw` only when you need provider-specific metadata. Keep application logic on the normalized fields so a provider change does not spread through your code.

Stream text

Set `stream: true` to receive ordered text deltas. The enumerator yields text only and ignores provider metadata events.

```
require "tina4"

chunks = Tina4::Ai.chat(
  [{ role: "user", content: "Write a short release announcement." }],
  stream: true
)

chunks.each do |chunk|
  print chunk
  $stdout.flush
end
```

Tina4 may retry before the first delta arrives. It never retries after yielding text because that could duplicate content the caller has already displayed.

Create embeddings

`Tina4::Ai.embed` preserves the input shape. One string returns one vector. A list returns one vector per input, in the same order.

```
require "tina4"

vector = Tina4::Ai.embed("Tina4 keeps application code small")

vectors = Tina4::Ai.embed([
  "The router finds a matching handler",
  "The ORM maps a row to a model"
])
```

Set `TINA4_EMBED_URL` when the embedding service uses a different base URL. Anthropic does not expose embeddings through this contract, so `provider: "anthropic"` raises `Tina4::AiConfigError`.

Handle failures

All client failures inherit from `Tina4::AiError`:

Error	Meaning
`Tina4::AiConfigError`	Missing key, invalid provider, bad option, or unsupported capability
`Tina4::AiHTTPError`	Provider returned a failing HTTP status or the transport failed
`Tina4::AiTimeoutError`	Connection or total request deadline expired
`Tina4::AiParseError`	A successful response did not match the provider contract

```
require "tina4"

begin
  puts Tina4::Ai.complete("Create a migration plan")
rescue Tina4::AiError => error
  warn "AI request failed: #{error.message}"
end
```

Hosted providers fail before sending when `TINA4_AI_KEY` is missing. Error text never includes the key, prompt, or provider response body. A failure stays a failure; Tina4 never turns it into an empty answer.

Timeouts and retries

`TINA4_AI_CONNECT_TIMEOUT` bounds connection setup. `TINA4_AI_TIMEOUT` bounds the whole request, including retries and response reads. `TINA4_AI_MAX_RETRIES` applies only to connection failures, HTTP 429, and HTTP 5xx responses.

Other HTTP 4xx responses and malformed successful responses run once and fail. The client carries transient trouble for a bounded distance, then hands the error back to your code.

OpenID Connect SSO

Tina4 completes one provider-neutral OpenID Connect flow. Configure an issuer and Tina4 discovers the provider, runs Authorization Code with PKCE, regenerates the Session, and sends the normalized identity through the existing secured route gate. The browser keeps the normal Tina4 Session cookie.

Configure SSO

```
TINA4_SSO_ISSUER=https://identity.example.com/realms/my-app
TINA4_SSO_CLIENT_ID=my-app
TINA4_SSO_CLIENT_SECRET=replace-me
TINA4_SSO_REDIRECT_URI=https://app.example.com/auth/callback
TINA4_SSO_SCOPES=["openid", "profile", "email"]
TINA4_SSO_VERIFY=introspection
TINA4_SSO_POST_LOGOUT_REDIRECT_URI=https://app.example.com/
```

SSO stays off until the issuer, client id, and redirect URI exist. Production issuer and callback URLs require HTTPS; loopback HTTP remains available for development. When configured, Tina4 mounts `GET /auth/login`, `GET /auth/callback`, and `POST /auth/logout`. A route collision stops startup with an error.

Use the identity

Secured routes use the same gate as local Tina4 authentication. The normalized identity is available as `request.user`. Provider tokens never enter it, logs, error pages, Swagger, or `session.all`.

```
if Tina4::Sso.configured?
  sso = Tina4::Sso.from_issuer
  metadata = sso.discover
  identity = sso.identity(request)
end
```

Method	Result
`Tina4::Sso.configured?`	Reports whether required environment values exist.
`Tina4::Sso.from_issuer`	Creates the client and discovers the issuer.
`discover(force = false)`	Returns validated metadata. Force refreshes it.
`login(request, return_to = "/")`	Stores one-use state and returns the authorization URL.
`callback(request, query = nil)`	Validates the callback and regenerates Session.
`identity(request)`	Returns the normalized identity or `nil`.
`refresh(request)`	Replaces credentials and identity atomically.

`logout(request, return_to = "/")`	Destroys Session and returns the safe logout target.

The return path must be local and absolute. Tina4 rejects schemes, hosts, protocol-relative URLs, backslashes, and control characters.

Claims and provider recipes

```
TINA4_SSO_CLAIM_MAP={"username":"preferred_username","roles":"realm_access.roles","groups":"grou
```

The runtime does not name providers. Keycloak uses a realm issuer such as `https://sso.example.com/realms/my-realm`. Microsoft Entra ID uses its tenant-specific v2 issuer. Register the exact callback URI and map claims only when roles or groups use different names.

Version 3.13.104 supports introspection without another gem and requires a client secret. Selecting `jwks` fails during configuration until the application installs a supported cryptography capability. State, nonce, code, and verifier are one-use. Logout always destroys the local Session first.

When configured, Swagger publishes the issuer discovery URL as an `openIdConnect` scheme.

GIS and PostGIS

Tina4 stores geographic points in PostGIS, measures distance in metres, and returns map-ready GeoJSON. Coordinates always use longitude first. GPS may supply coordinates, but GIS is the feature.

Define a point field

Enable PostGIS with `CREATE EXTENSION IF NOT EXISTS postgis;`.

```
class ChargePoint < Tina4::ORM
  point_field :location
end

site.location = Tina4::Point.new(18.4241, -33.9249)
site.save
```

Tina4 creates `geography(Point,4326)` and an idempotent GiST index. Unsupported database engines and PostgreSQL without PostGIS fail clearly. `Tina4::Point.parse` accepts a Point, number pair, WKT or EWKT, GeoJSON Point or Feature, and WKB or EWKB. It rejects invalid ranges, non-finite numbers, non-point geometry, and conflicting SRIDs before SQL runs.

Query and return GeoJSON

```
nearby = ChargePoint.query
  .within_distance("location", [18.42, -33.92], 5_000)
  .select_distance("location", [18.42, -33.92], alias_name: "metres")
  .order_by_distance("location", [18.42, -33.92])
  .get

feature = site.to_feature
collection = Tina4::ORM.feature_collection(nearby)
```

The query builder also provides `intersects` for bound WKT, EWKT, or GeoJSON geometry and `bbox` for a validated bounding box. Spatial values use bound parameters. Column and alias names are validated as identifiers.

`to_feature` moves the chosen Point into `geometry` and keeps other selected fields in `properties`. A null point creates `"geometry": null`. `feature_collection` preserves query order. Supply a geometry field for models with several Point fields and an include list to limit properties.

Version 3.13.104 persists Point fields only. It does not persist lines, polygons, rasters, tiles, geocoding results, or routes. `intersects` may accept a polygon as query geometry. PostGIS is the only storage provider.

IoT and MQTT

Tina4 includes a zero-dependency MQTT 3.1.1 client for telemetry, device state, asset tracking, and EV charging. It connects to an external broker such as Mosquitto, EMQX, HiveMQ, or AWS IoT. Tina4 is not a broker.

Configure and publish

```
TINA4_MQTT_URL=mqtt://127.0.0.1:1883
TINA4_MQTT_CLIENT_ID=warehouse-api
TINA4_MQTT_KEEPALIVE=60
TINA4_MQTT_TLS_VERIFY=true
TINA4_MQTT_CA_FILE=/run/secrets/mqtt-ca.pem
```

Use `mqtt://` or `tcp://` for plain TCP and `mqtts://` for TLS. Put credentials in URL user information or constructor arguments. There are no username or password environment variables.

```
mqtt = Tina4::Mqtt.new
packet_id = mqtt.publish(
  "fleet/meter-42/telemetry",
  '{"kwh":12.5}',
  qos: 1
)
mqtt.disconnect
```

Construction connects by default. QoS 0 waits for no acknowledgement. QoS 1 returns the packet id after PUBACK. Tina4 refuses QoS 2.

```
mqtt = Tina4::Mqtt.new(clean_session: false)
mqtt.consume("fleet/+/telemetry", qos: 1) do |message|
  process(message.topic, message.to_s)
end
```

`consume` acknowledges after successful block handling. Use `receive(ack: false)` followed by `message.acknowledge` for manual control. Messages expose topic, byte payload, QoS, packet id, retained and duplicate flags, acknowledgement, text through `to_s`, and `to_h`.

Retained publishes store current state; an empty retained payload clears it. Configure `will_topic`, `will_payload`, `will_qos`, and `will_retain` for an unclean disconnect. `disconnect` suppresses the will; `kill` lets the broker publish it. TLS verification defaults to true. `tls?`, `cipher`, and `tls_version` report the connection. One client has one socket reader.